

Table of Contents

Chapter 27: Introduction to Spring 4.3	1111
Overview of Spring 4.3	1112
Core and Beans	1112
Context	1113
Spring Expression Language (SpEL)	1113
Data Access using Java Database Connectivity (JDBC)	1113
Data Access using Object Relational Mapping	1113
Aspect Oriented Programming (AOP)	1113
Transactions	1113
Java Management Extensions	1113
Remoting	1114
Java Messaging Service (JMS)	1114
Object XML Mapping (OXM)	1114
Web Frameworks	1114
Web MVC	1114
Test	1114
New and Enhanced features of Spring 4.3	1114
Dependency Injection	1115
Spring Libraries	1116
The Spring Tool Suite	1117
Developing a Spring 4.3 Application	1117
Creating the Spring Project	1118
Configuring the Build Path	1120
Creating the Alarm Interface	1120
Creating the SoundAlarm Class	1120
Creating the VisualAlarm Class	1121
Creating the Door Class	1121
Creating the Configuration File	1122
Creating the Client Class	1122
Running the Application	1123
Summary	1123
Chapter 28: Spring Configuration	1125
Spring Containers	1126
BeanFactory	1127
ApplicationContext	1128
Configuring Beans	1129
XML-Based Configuration	1129

Annotations-Based Configuration	1136
Summary	1138
Chapter 29: Spring Web MVC	1159
Spring MVC Architecture	1140
Components of Spring MVC.....	1140
DispatcherServlet.....	1141
Controller.....	1141
ViewResolvers.....	1142
Model	1143
Request Mapping	1143
Developing a Simple Spring MVC Application	1145
Setting Up the SpringMVCEXample Application.....	1146
Configuring the DispatcherServlet Class	1146
Creating the login.jsp Page.....	1147
Creating the LoginBean Class	1147
Creating the Controller Class	1148
Creating the results.jsp Page	1149
Creating the Spring Configuration Files.....	1149
Executing the Application	1150
Custom Binding.....	1151
Using Session Attributes	1153
Other MVC Annotations	1154
Connecting an MVC Application to the Business Layer	1154
Setting up the SpringMVCWithJdbc Application	1155
Configuring the DispatcherServlet Class	1156
Setting up the Database	1156
Creating the PersonsDAO Class	1156
Creating the CreatePerson.jsp File	1157
Creating the PersonBean Class	1157
Creating the CreatePersonController Class.....	1158
Creating the results.jsp Page	1159
Creating the Spring Configuration File	1159
Executing the Application	1160
Exception Handling	1161
RESTful Applications.....	1161
Summary	1164



1

Introduction to Java

<i>If you need information on:</i>	<i>See page:</i>
Features of Java	2
New Features of Java 8	4
Introducing Java Environment	6
Installing Java	9
Java's Reserve Words	11
Starting a Java program	13
Line 1—public class App	13
Line 2—public static void main(String[] args)	14
Line 3—System.out.println("Hello from Java!");	14
Compiling Code	15
Commenting Your Code	20
Importing Java Packages and Classes	21
Summary	23

Java is a programming language, which inherits its object-oriented features from C++. It was created by James Gosling, a software developer at Sun Microsystems, in 1991. It was first called Oak, which was later renamed as Java in 1995, when it was first released for public use. The driving inspiration behind Java was to create something that could be used on all computers. Java has been popularized through the Internet where a huge and increasing number of corporations are adopting it internally because it can be used on all computers irrespective of the operating system. In other words, when you compile a Java program, the bytecode is generated, which can be executed across multiple operating systems. In fact, some versions of C++, such as Microsoft's C#, are now modeled after Java.

Another advantage of a Java program is that it consumes less memory. For comparison with other programming languages, let's take an example of a Microsoft Visual C++ program whose memory consumption is high, typically starting off at a minimum of 5 MB for a full Microsoft Foundation Class (MFC) program. The memory consumption of 5 MB of the Microsoft Visual C++ program does not even count the Dynamic Link Libraries (DLLs) that the Windows platform needs to run the program. Due to large size, it becomes difficult to download Microsoft Visual C++ program as part of a Web page whenever required.

Java programs, on the other hand, are constructed differently. Java itself is implemented as the Java Virtual Machine (JVM), which is an application that actually runs your Java program. When JVM is installed on a computer, it can run Java programs. Java programs, therefore, do not need to be self-sufficient, and they do not have to include all the machine-level code that actually run on the computer. Instead, Java programs are compiled into compact bytecodes, and it is these bytecodes that the JVM reads and interprets to run your program. When you download a Java applet on the Internet, you are actually downloading a bytecode file. In this way, your Java program can be very small because all the machine-level code to run your program are already on the target computer and do not have to be downloaded. To host Java on all computers, Sun only had to rewrite JVM to work on those computers. As your program is stored in a bytecode file, it will run on any computer on which JVM is installed.

Java programs were originally interpreted by the JVM, which performed bytecode-by-bytecode execution, thereby making interpretation a slow process. To overcome the slow process, Java 2 introduced the just-in-time (JIT) compiler, which is built in the JVM. The JIT compiler actually reads the sections of bytecodes and then compiles these sections into machine language that can be executed on the machine in less time (the whole Java program is not compiled at once because Java performs runtime checks on various sections of the code). The latest version of Java introduced by Oracle Corporation (acquired Sun Microsystems) is Java Standard Edition (SE) 8.

In this chapter, you learn about the features of Java programming language along with the introduction of new features in Java SE 8. Further, the chapter introduces you to Java platform and explains JVM, Java Development Kit (JDK), Java Runtime Environment (JRE), and Java Application Programming Interfaces (APIs). It also discusses the downloading and installing the Java Software Development Kit. Finally, you learn how to create, compile, and run a simple Java program.

Features of Java

Java is a simple, robust, platform-independent, and portable programming language. It is designed to meet the real-world requirements with its following features:

- ☐ Simple
- ☐ Secure
- ☐ Portable
- ☐ Robust
- ☐ Multithreading
- ☐ Platform independent
- ☐ Distributed
- ☐ Dynamic

Simple

Java is simple to learn and use. In Java, it is easy to write and debug programs because Java does not use pointers. Moreover, it provides a bug-free system because of strong memory management and also has an automatic memory allocation and de-allocation system. Unlike other programming languages, which provide dozens of complicated ways to perform a simple task, Java provides a simple and clear approach to achieve a given task. If a programmer understands the basic concepts of OOP, it becomes easier for him/her to learn Java.

Secure

In today's world, everybody needs safety and security. Because of the threat of hackers, people feel unsafe while performing commerce over the Internet. To overcome such fears, Java provides safety and security features.

While using a Java-compatible browser, everyone can safely download Java applets by relaxing their minds from the fear of virtual infection or malicious intent. The Java programs are confined to the Java runtime environment; they can access only that part of your computer that is required for program execution. The data outside the Java runtime environment cannot be accessed by them and therefore, downloading a Java application through the Internet will not harm your computer. That's why Java is a secure language.

When Java executes a program, the JVM can strictly monitor what goes on, which makes it great for the Internet applications. You are already aware that security has become an extremely important issue on the Internet, and Java rises to the task of taking care of the same. The JVM can watch all that a program does; and if it does something questionable, such as trying to write a file, it can prevent that operation. That alone makes Java more attractive than C++, which has no such restrictions, for the Internet.

You can also tailor Java security the way you like it, which offers a very flexible solution. As you'll see in this book, you can now specify, on a program-by-program basis, just what privileges you want to give to the downloaded code. You can now also "sign" your Java code in a way that shows it comes from you without any malicious modifications. We'll take a look at all this and more in this book.

In a nutshell, Java has a winning combination for the Internet—Java programs are small, secure, platform-independent, object-oriented, and powerful. They also implement other features that programmers like. Java programs are often simple to write compared to C++. The end result is that you can write your Java program once, and it can be easily downloaded and run on all kinds of machines—the perfect recipe for the Internet! That's the reason Java has soared so high.

Portable

In the distributed world of the Internet, an application developed with the help of a programming language might be accessed on various computers having different kinds of operating systems. But, it's not guaranteed that the application is portable, that is, it runs successfully on all operating systems. To overcome this portability issue, Java introduces a concept of what is known as bytecode.

Bytecode is a set of instructions generated by the Java compiler while compiling a Java program. In other programming languages, a program is compiled into an executable code; but in Java, a program gets compiled into an intermediate code called bytecode. This bytecode then gets executed by the Java runtime system called the JVM. So, in order to run your Java program, you should have only the JVM installed. It is also considered to be the interpreter of bytecode. In this way, Java has solved the problem of portability.

Robust

Generally, programs fail because of the following two reasons:

- ❑ Memory Management—Lack of memory management for allocating and de-allocating memory for objects
- ❑ Exceptional Conditions—Occurrence of unexpected exceptions

Considering these reasons, Java was designed to provide the ability to create robust programs. Therefore, Java compels the programmers to resolve mistakes during compilation. Moreover, it also frees the programmer from the worries of common causes of programming errors as the program code is verified during compilation.

Java also eradicates the problem of memory management by performing automatic garbage collection. It manages the memory allocation and de-allocation for unused objects.

Multithreading

Java is a programming language that is designed for the distributed environment of the Internet; and for that, the concept of multithreading is implemented. This feature helps you write interactive programs, wherein multiple tasks can be performed simultaneously, thus making Java a robust programming language.

Java is inherently multi threaded as there can be multiple executing threads in a single Java program. For example, a Java programmer can run three Java applets on the same page, provided that each applet gets equal time from the CPU.

The multi threaded feature of Java makes it very responsive and robust. It is very important to note that as Java is highly multi threaded, it is difficult to port it to architectures, such as Windows 3.1 or PowerMac, which do not natively support multithreading.

Platform-Independent

It is one of the most important features of Java as it is the first programming language that is not bound to any specific operating system. In other words, Java programs can be executed anywhere on any system. Before the evolution of Java, no other programming language was platform-independent, and it can therefore be said that Java is a revolutionary technology. Apart from cross-platform in source form, Java is also platform-independent in compiled binary form.

A Java program does not execute natively on the host machine; instead, a Java interpreter reads the bytecode and executes the corresponding native machine instructions. The Java interpreter is a special naive program that reads the bytecode, an intermediate form after compilation. Therefore, in order to port a Java program to a new platform, you need the interpreter and some of the library routines. Moreover, the Java compiler is also written in Java, and the bytecodes are precisely defined, which remain the same on all platforms.

Distributed

Java is a distributed language as it can be used to create applications for communicating over the network. Java can communicate over the network because it supports TCP/IP (Transmission Control Protocol/Internet Protocol). The TCP/IP protocol is a set of network communication protocols.

Dynamic

During the runtime of a Java program, the relevant information that is used to verify and resolve access to objects is required. This concept of providing runtime information is referred to as dynamically linking the code. This feature of Java adds strength to the applet environment, in which small fragments of bytecode are dynamically updated on a running system.

New Features of Java 8

Java SE 8 is a major release that brought many new features and enhancements in Java programming language. Let's have a brief introduction of some noteworthy features of Java SE 8.

- ❑ Java programming language— Following are new changes in Java programming language:
 - Lambda expressions are introduced as a new language feature. They allow you to pass functions as a method argument or treat code as data.
 - Following is the syntax of lambda expression:

```
view sourceprint?1 (argtype arg...) -> { return some expression.. probably using these arguments }
```
 - Method references help in reading lambda expressions for methods that already have a name.
 - Default methods enable new functionality to be added to the interfaces of libraries and ensure binary compatibility with code written in older versions of those interfaces.
 - Repeated use of annotations enables you to apply the same annotation type more than once to the same type of declaration.

- A new method, `String.join()`, is added to join the strings. For example:

```
String abc= String.join("String1 ", "Java", "8");
```

- The preceding code will be evaluated as:

```
String1 Java 8
```

- **Java compiler**—There is a new feature of using the `-parameters` option with `javac` command – to store formal parameter names and enable the Reflection API to retrieve them. Let's create the following class:

```
import java.lang.reflect.Method;
import java.lang.reflect.Parameter;

public class ParameterNames {
    public static void main(String[] args) throws Exception {
        Method m = ParameterNames.class.getMethod( "main", String[].class );
        for( final Parameter parameter: m.getParameters() ) {

            System.out.println( "Parameter: " + parameter.getName() );

        }
    }
}
```

If you compile the previous class without using `-parameters` argument and then run this program, you will see the output as:

```
Parameter: arg0
```

However, if you compile the previous class with the `-parameters` argument and then run this program, you will see the output with the actual name of parameter that you have passed.

```
Parameter: args
```

- **Java libraries**— Following changes have taken place in Java libraries features:
 - **Streams**— A new package, `java.util.stream`, provides a Stream API to support functional-style operations on streams of elements. Stream API can be integrated with Collections API to perform bulk operations on collections, such as sequential or parallel map-reduce transformations.
 - **Date/Time API (JSR 310)**— Java 8 provides a new Date-Time API (JSR 310), which provides a new package, called `java.time`. This package contains all the classes for date, time, date/time, time zones, instants, duration, and clocks manipulation. Some examples of classes contained in this package are `Clock`, `LocaleDate`, `LocalTime`, and `Duration`. The `Clock` class provides the access to the current instant, date and time using a time zone. It can be used in place of using the `System.currentTimeMillis()` and `TimeZone.getDefault()` methods. The `LocaleDate` class holds only the date part and the `LocalTime` class holds only the time part without a time-zone in the ISO-8601 calendar system. The `Duration` class, finally, provides an amount of time in terms of seconds and nanoseconds.
 - **Nashorn JavaScript engine**— Java 8 provides a new Nashorn JavaScript engine that allows you to develop and execute certain kinds of JavaScript application on JVM. The engine works like `javax.script.ScriptEngine` and permits Java and JavaScript interoperability.
 - **Parallel Arrays**— Java 8 provides various new methods allowing parallel arrays processing. For example, the `parallelSort()` method significantly speeds up the sorting on multicore machines.
 - **Concurrency**— Java 8 adds new methods to the `java.util.concurrent.ConcurrentHashMap` class that support aggregate operations on the basis of the newly added streams facility and lambda expressions. A new class, `java.util.concurrent.locks.StampedLock`, has been added to provide a capability-based lock with three modes for controlling read/write access. In addition, some classes are added to the `java.util.concurrent.atomic` package, which includes `DoubleAccumulator`, `DoubleAdder`, `LongAccumulator`, and `LongAdder`.
- **Java tools**— Java 8 provides a new set of command line tools, which include Nashorn engine (`js` command) and class dependency analyzer (`jdeps` command). The role of these tools is described as follows:

- **Nashorn engine**— The `jjs` command accepts a list of JavaScript source code files as arguments and runs them. For example, if a file, `myfunc.js`, is created with the following content:

```
function demofunc() {  
  
    return 1;  
  
};  
print( demofunc() + 1 );
```

- Now, the `jjs` command is used to execute the `myfunc.js` file:

```
jjs myfunc.js
```

- The output of the preceding code will be:

```
2
```

- **Class dependency analyzer**— The `jdeps` command is used to show the package-level or class-level dependencies of Java class files. It accepts .class file, a directory, or JAR file as an input, and displays the dependencies to the console.
- ❑ **Java runtime (JVM)**— The PermGen space has been replaced with Metaspace (JEP 122). The JVM options, `XX:PermSize` and `-XX:MaxPermSize`, are replaced with `-XX:MetaSpaceSize` and `XX:MaxMetaspaceSize`, respectively.
- ❑ **JDBC**— The JDBC-ODBC Bridge has been removed. In addition, JDBC 4.2 has been introduced with various new features. Some of these are as follows:
 - Addition of `REF_CURSOR` support
 - Addition of `java.sql.DriverAction` interface
 - Addition of security check on `deregisterDriver` method in the `DriverManager` class
 - Addition of the `java.sql.SQLType` interface
 - Addition of the `java.sql.JDBCType` enum

Introducing Java Environment

The Java environment comprises numerous development tools, classes, and methods. The development tools for Java are provided as a part of the system known as Java Development Kit (JDK). Moreover, the Application Programming Interface (API) comprises the classes and interfaces used in Java programming. In this section, we discuss the following:

- ❑ Java Development Kit
- ❑ Java Platforms
- ❑ Java Virtual Machine
- ❑ Java API
- ❑ Java Programs

Java Development Kit

Keeping in mind the developers, Sun introduced JDK originally, which is now an Oracle product. JDK provides a collection of the Java tools, which are used while developing and running Java programs. The kit comprises the following development tools:

- ❑ `java`—Serves as a Java interpreter used to run Java applets and applications by reading and interpreting bytecode files
- ❑ `javac`—Serves as a Java compiler used to translate Java source code into bytecode files
- ❑ `javacdoc`—Creates HyperText Markup Language (HTML) documentation for Java source code files
- ❑ `javap`—Serves as a Java disassemble used to convert bytecode files into a Java program description
- ❑ `jdb`—Serves as a Java debugger used to find errors in Java programs
- ❑ `appletviewer`—Facilitates to run Java applets

- ❑ `jar`—Serves as an archive used to package related class libraries into a single executable JAR file and also helps to manage the JAR files
- ❑ `javah`—Serves as the C header and stub generator used to write native methods

Java Platforms

Oracle provides various Java editions to develop and run Java programs. The Java platform is not bound to any one processor or operating system, but the execution engine (virtual machine) and a compiler can run identically on any operating system. The following subsections are the different editions of the Java platform.

What is Java Standard Edition (Java SE)?

The Java SE Edition of Java is the most commonly used among the three Java editions. It is used for developing desktop and console-based applications. It consists of:

- ❑ A runtime environment
- ❑ Sets of APIs to build a wide variety of applications, comprising standalone applications that can run on various operating systems, such as Microsoft Windows, Linux, and Solaris

What is Java Enterprise Edition (Java EE)?

Java EE is a platform primarily used for building server-side applications by using its convenient component-based approach. It is a platform-independent, Java-centric environment from Oracle, which is meant for the development and online deployment of Web-based Enterprise applications. The Java EE platform consists of a set of service—APIs and protocols. These services provide the functionality for developing multi tiered, Web-based applications.

Java EE is a Java-based, runtime platform created by Sun. It is now owned by Oracle after the acquisition of Sun by Oracle on January 27, 2010. Java EE is used for developing, deploying, and managing multi tier server-centric applications on an enterprise-wide scale. It builds on the features of Java SE and adds distributed communication, threading control, scalable architecture, and transaction management. A strong competitor to the Microsoft's .NET Framework, Java EE is a Java platform designed for the mainframe-scale computing, typical of large enterprises. For the simplification of application development in a thin client-tiered environment, Sun designed Java EE in collaboration with partners such as IBM.

Some of the key features and services of Java EE are as follows:

- ❑ At the client tier, it supports pure HTML as well as Java applets or applications. It relies on Java Server Pages and Servlet code to create HTML or other formatted data for the client.
- ❑ Enterprise JavaBeans (EJB) provides a layer for storing the platform's logic. The EJB server provides functions such as threading, concurrency, security, and memory management. These services are transparent to the author.
- ❑ Java Database Connectivity (JDBC), which is the Java equivalent to Open Database Connectivity (ODBC), is the standard interface for Java databases.

The Java Servlet API increases consistency for developers without the need of a Graphical User Interface (GUI).

What is Java Micro Edition (Java ME)?

It enables you to build Java applications for micro-devices, which include handheld devices, such as cell phones and Personal Digital Assistants (PDAs), and other similar devices with limited display and memory support. It is a competitor to Microsoft's .NET Compact Framework.

The devices that you use nowadays, such as cell phones, PDAs, televisions, and set-top boxes, are often powered by Java ME. This is because Java ME provides a robust and flexible environment to enable the applications meant to run on devices that are low on resources, such as processor and memory. The Java Community Process, which comprises the experts from the leading device manufacturers, defined the standard set of Java APIs that are part of the Java ME environment along with the JVM. The same sets of APIs along with the JVM are also included in the other Java platforms that include Java SE, Java EE, and Java Card environments.

The strength and advantages of the Java technology can be provided to the end user of the embedded consumer devices by using Java ME in such devices. The inclusion of a flexible user interface, a robust security model, a variety of built-in network protocols, and an extensive support for networked and offline applications that are automatically downloadable makes Java ME a versatile platform for a wide range of consumer devices. The applications written for the Java ME specifications are also inherently platform-independent and can therefore be used on a variety of devices even though written for one.

The Java ME architecture includes various configurations, profiles, and optional packages for the developers to choose from. The chosen packages and configurations can be combined to build a complete Java runtime environment to go in rhyme with the target device and the target market. The selection is optimized for the processing power, memory, and I/O capabilities of any related category of devices. This eventually results in a common platform that benefits from each category of devices by providing an engaging user experience.

The Java ME configuration includes a JVM and a set of class libraries. The number of libraries is kept to a bare minimum because it is meant to be executed on a device with a modest hardware configuration. They then provide the basic functionality for a particular category of devices having common characteristics. There are two Java ME configurations: the Connected Limited Device Configuration (CLDC) and the Connected Device Configuration (CDC).

Java Virtual Machine

In all programming languages, a compiler translates source code into machine code for a specific computer. In Java, a compiler translates the Java source code into an intermediate code known as bytecode for a virtual machine, known as JVM. Bytecode is the machine language for JVM and is not machine-specific. Instead, the machine-specific code is generated by the Java interpreter, which serves as a mediator between virtual machine and real machine. Figure 1.1 shows the process of compilation and conversion of bytecode into machine code:

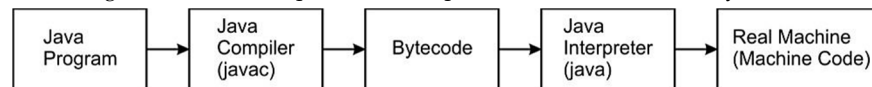


Figure 1.1: Displaying the Process of Compilation and Execution in Java

In order to run a Java program on a different computer, you need a Java interpreter for Java bytecode. Although Java bytecode interpreter is required depending upon the type of operating system in use, once a computer has an interpreter, it can run a Java bytecode. In other words, the most important feature of Java is that the same compiled program can run on any operating system, as shown in Figure 1.2:

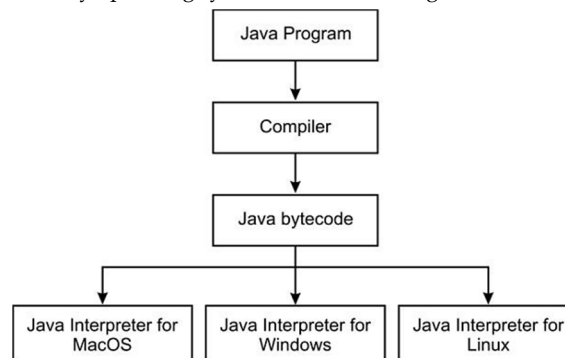


Figure 1.2: Displaying the Process of Running a Program on Any Operating System

Java API

It is a collection of classes, interfaces, and methods provided in the form of Java packages. In other words, it is a large collection of already created classes and interfaces, which provide many useful capabilities, such as Graphical User Interface (GUI), date, time, and calendar capabilities. The categorization of Java API in the form of Java packages is as follows:

- ❑ Application Programming Packages
 - **java.lang**—Provides classes that are fundamental to the design of the Java programming language
 - **java.util**—Provides legacy collection classes, event model, collection framework, date and time capabilities, internationalization, and other utility classes, such as string tokenizer
 - **java.io**—Provides classes for system input and output through data streams, serialization, and the file system
- ❑ Applet and Network Programming Packages
 - **java.awt**—Provides classes for creating user interface and painting graphics and images
 - **java.applet**—Provides classes that are necessary for creating an applet and those that are used to communicate with its applet context
 - **java.net**—Provides classes that are used for implementing networking in Java programs

Java Programs

A Java programmer can create either a Java application or a Java applet depending upon the requirement. In Java, you create the following types of program:

- ❑ Applications
- ❑ Applets

We'll use the term program in this book to refer to both applets and applications. Now, let's discuss each of them in detail.

Applications

A Java application is a program that is created by using Java programming language. Java applications are console-based, Character User Interface (CUI)-based, and GUI-based standalone applications. Once you become used to creating applications, you will find that creating an application by using Java is also as simple as creating an application with any other programming language. In order to execute a Java application, you need to perform the following steps:

- ❑ Compile the Java source code by using `javac` compiler to translate the source code into bytecode
- ❑ Execute the bytecode by using the Java interpreter

Apart from applications, you can also create applets in Java.

Applets

An applet is a program written in the Java programming language that can be included in an HTML page in the same way as an image is included. An applet can be used in both `static` and `dynamic` web pages to either display content or share information through the pages. You need to import the `java.applet` package that provides classes to either create an applet or allow an applet to communicate with its applet context.

The applet context is an application that is responsible for loading and running applets. For example, the applet context could be a web browser or an applet development environment. In other words, an applet is a Java application designed to be transmitted over the Internet and is executed by a Java-compatible web browser. After understanding the types of Java program, let's create a simple Java program. But to do this, you need to create a source code file by using a text editor first. After creating the Java program, you need to compile it by using the Java compiler `javac` and execute by using the Java interpreter `java`. The compiler generates a `.class` file by translating the source code into bytecode, which is then executed by the Java interpreter.

Installing Java

In this section, you will learn to download and install Java SE 8 (Java 1.8) on your system.

Currently, you can find it at <http://www.oracle.com/technetwork/java/javase/downloads/jdk8-downloads-2133151.html>. The kit gets downloaded in the form of an executable application file. Next, you can follow the instructions on the Oracle's website in order to install JDK 1.8. We'd love to be able to provide the actual installation instructions here, but that's one of the biggest pitfalls a Java book can fall into, even the one

that's designed to be as complete as possible. We have been writing about Java ever since it first came out, and it turns out that the actual installation instructions are often very volatile. As these instructions are changed quite frequently, the ones we have provided in the previous books instantly became obsolete. For that reason, the absolutely best thing you can do is to see how Sun wants you to install the SDK; therefore, you should refer to the installation instructions as posted on the Java site. The installation process has been getting easier with every version and beta of Java, and now, it typically involves only running the file you've downloaded.

One thing you should be sure to do, as indicated in the Sun installation instructions, is to ensure your machine can find the Java tools, including the Java compiler itself. To do that, verify that the Java bin directory is in your computer's path. For example, in Windows, usually the bin directory path is `C:\Program Files\Java\jdk1.8\bin` for SDK 1.8.

TIP

The Java Software Development Kit was called the Java Development Kit, JDK, in earlier Java versions. In fact, Oracle often seems to be of two minds here—sometimes, you'll see references to the SDK and sometimes, you'll see references to JDK. This is one of those times—although the kit is now called the SDK, it's installed by default into a directory named `\jdk1.8`.

In Windows 7, you can follow these steps to configure Java 8 on your system:

- ❑ Click the Windows icon on the taskbar, right click the Computer, and select the Properties option. You can also reach the same location by opening the Control Panel | System and Security | System.
- ❑ Click the Advanced system settings option in the System Properties window. This opens the System Properties dialog box.
- ❑ Click the Environment Variables button to open the Environment Variables dialog box.
- ❑ Edit the `PATH` variable, setting the way you want. You may append the new path to the already existing `PATH` variable or create a new `PATH` variable. Finally, click OK. Note that if you are appending the `PATH` variable, the different paths must be separated by semicolons (;).

You can also set the `PATH` variable temporarily with the help of the command line. The path shall remain set only till the command line is open; once you exit from the command line, the `PATH` variable set also gets erased. The following command is used to temporarily set the `PATH` variable:

```
SET PATH= C:\Program Files\Java\jdk1.8\bin;
```

When the bin directory is in the path, you'll be able to use the Java tools directly from the command line, instead of having to preface them with a pathname each time you want to use them on the command line.

What about CLASSPATH?

Java veterans will wonder about the environment variable named `CLASSPATH` when they install Java SDK. The `CLASSPATH` variable, as you'll soon see in this chapter, tells Java where to find compiled bytecode files—both the ones you create and the ones required by the system that come with the SDK itself. `CLASSPATH` has been the focus of a great deal of confusion when working with Java, and we are glad to say that Sun has made things easier.

When you install the SDK, you don't have to worry about setting the `CLASSPATH` now because the SDK will know where to find its own installed libraries. However, if you want to search other custom bytecode files when compiling a file, you'll have to set the `CLASSPATH` yourself. You'll see how to do this when we will discuss compiling programs in this chapter. (There are two ways to indicate to the Java compiler where to find bytecode files you particularly want to search—by setting the `CLASSPATH` environment variable and by using the `-CLASSPATH` compiler switch.)

The current version of the SDK does not require `CLASSPATH` to be defined; and if it has been defined by some other Java version or system, it is likely to create problems. If you want to keep the `CLASSPATH` environment variable for any reason, you need to use the command line option to define the `CLASSPATH` temporarily.

Subfolders Created by JDK

After installing JDK, several folders get created on your hard drive. The location of these folders formed depends on the particular system; however, in general, the JDK folder is formed under `Program Files\Java` on your boot

drive. The name of the JDK root folder also includes the version you have installed. The different subfolders created under the JDK root folder and additional files are described in Tables 1.1 and 1.2, respectively:

Table 1.1: The different subfolders created in the JDK root folder

Folder	Description
bin	The compiler and other Java development tools
demo	Demo programs from which you can learn various Java features
include	Files required to integrate Java with other languages
jre	Runtime Environment files
lib	Library files, including the Java API class library
sample	Samples, with source code, of programming for certain Java API's
db	Oracle's distribution of the Apache Derby relational database

Table 1.2: Additional files in the JDK root folder

File	Description
README.html	The Java readme file in the HTML format
THIRDPARTYLICENSEREADME.txt	A license file for third parties, regarding usage and redistribution of JDK
LICENSE	The Java license that you agreed to when you downloaded JDK
COPYRIGHT	The copyright notice about all the copyright and export laws applied by Sun
register.html	For free register of the JDK installed with Oracle
release	It contains details such as Java version, OS name, and OS version
register_ja.html and register_zh_CN	These HTML files are, respectively, the Japanese and Chinese versions of <code>register.html</code>
Src	A <code>.zip</code> file that contains the code for Java API classes

Creating Source Files

Java programs are just plain text files made up of Java statements and declarations, and we'll start investigating them in the next solution. To create a Java program, you should have a text editor or word processor that can save files in plain text format.

Saving text in plain text format is a simple achievement that's beyond many fancy word processors. You might have trouble with word processors, such as Microsoft Word, although you can save plain text files with Word using the File | Save As dialog box. The general rule is that if you can type the file out at the command line (note that that's DOS on DOS- and Windows-based computers) and don't see any odd, non-alphanumeric characters, it's a plain text file. The real test, of course, is whether the Java compiler, which translates your program into a bytecode file, can read and interpret your program.

In addition, your programs should be stored in files that have the extension `.java`. For example, if you are writing an application named `App`, you should store the actual Java program in a file named `App.java`. You pass this file to the Java compiler to create the actual bytecode file, as you'll see in the next few pages.

Java's Reserve Words

When you are writing a Java code, you should know that Java reserves certain words for itself as a part of the Java language. Some of the reserve words are as follows:

- `abstract`—It specifies that a class or method will be implemented later in a subclass
- `assert`—It signifies an assertion statement
- `boolean`—It is a data type that can hold True and False values only
- `break`—It is a control statement for breaking out of loops

- ❑ `byte`—It is a data type that can hold 8-bit data values
- ❑ `byvalue`—It is reserved for future use
- ❑ `case`—It is used in the `switch` statements to mark blocks of text
- ❑ `cast`—It is reserved for future use
- ❑ `catch`—It catches exceptions generated by the `try` statements
- ❑ `char`—It is a data type that can hold unsigned 16-bit Unicode characters
- ❑ `class`—It declares a new class
- ❑ `const`—It is reserved for future use
- ❑ `continue`—It sends control back outside a loop
- ❑ `default`—It specifies the default block of code in a `switch` statement
- ❑ `do`—It starts a `do-while` loop
- ❑ `double`—It is a data type that can hold 64-bit floating-point numbers
- ❑ `else`—It indicates alternative branches in an `if` statement
- ❑ `enum`—It declares enumerated type variables
- ❑ `extends`—It indicates that a class is derived from another class or an interface is derived from another interface
- ❑ `false`—It indicates one of the two permissible values for a `boolean` variable
- ❑ `final`—It indicates that a variable holds a constant value or that a method will not be overridden
- ❑ `finally`—It indicates a block of code in a `try-catch` structure that will always be executed
- ❑ `float`—It is a data type that holds a 32-bit floating-point number
- ❑ `for`—It is used to start a `for` loop
- ❑ `future`—It is reserved for future use
- ❑ `generic`—It is reserved for future use
- ❑ `goto`—It is reserved for future use
- ❑ `if`—It tests a `true/false` expression and branches accordingly
- ❑ `implements`—It specifies that a class implements an interface
- ❑ `import`—It references other classes
- ❑ `inner`—It is reserved for future use
- ❑ `instanceof`—It indicates whether an object is an instance of a specific class or implements a specific interface
- ❑ `int`—It is a data type that can hold a 32-bit signed integer
- ❑ `interface`—It declares an interface
- ❑ `long`—It is a data type that holds a 64-bit integer
- ❑ `native`—It specifies that a method is implemented with native (platform-specific) code
- ❑ `new`—It creates new objects
- ❑ `null`—It indicates that a reference does not refer to anything
- ❑ `operator`—It is reserved for future use
- ❑ `outer`—It is reserved for future use
- ❑ `package`—It declares a Java package
- ❑ `private`—It is an access specifier indicating that a method or variable may be accessed only in the class it's declared in
- ❑ `protected`—It is an access specifier indicating that a method or variable may only be accessed in the class it has declared (or a subclass of the class it has declared in or other classes in the same package)

- ❑ `public`—It is an access specifier used for classes, interfaces, methods, and variables indicating that an item is accessible throughout the application (or where the class that defines it is accessible)
- ❑ `rest`—It is reserved for future use
- ❑ `return`—It sends control and possibly a return value back from a called method
- ❑ `short`—It is a data type that can hold a 16-bit integer
- ❑ `static`—It indicates that a variable or method is a class method (rather than being limited to one particular object)
- ❑ `strictfp`—It restricts the precision and rounding of floating-point numbers
- ❑ `super`—It refers to a class's base class (used in a method or class constructor)
- ❑ `switch`—It is a statement that executes code on the basis of a test value
- ❑ `synchronized`—It specifies critical sections or methods in a multithreaded code
- ❑ `this`—It refers to the current object in a method or constructor
- ❑ `throw`—It creates an exception
- ❑ `throws`—It indicates what exceptions may be thrown by a method
- ❑ `transient`—It specifies that a variable is not a part of an object's persistent state
- ❑ `try`—It starts a block of code that will be tested for exceptions
- ❑ `true`—It indicates one of the two permissible values for a `boolean` variable
- ❑ `var`—It is reserved for future use
- ❑ `void`—It specifies that a method does not have a return value
- ❑ `volatile`—It indicates that a variable may change asynchronously
- ❑ `while`—It starts a `while` loop

Starting a Java program

Here's a sample Java application that we'll develop over the next few sections, all the way through the compiling and running stages. Place this code in a file named `App.java`:

```
public class App {
    public static void main(String[] args) {
        System.out.println("Hello from Java!");
    }
}
```

If you are new to Java, this might look strange to you. The idea here is that this application will print the text "Hello from Java!" when you compile and run it. For example, here's how it looks in a DOS window under Windows:

```
C:\>java App
Hello from Java!
```

Not the most significant of programs but a good one to get us started. Let's take this program apart line by line.

Line 1—`public class App`

Here's the first line in `App.java`:

```
public class App { . . . }
```

This line indicates that we are creating a new Java class named `App`. After we translate this class into bytecodes, the JVM will be able to create objects of this class and run them. This code is just to get us started with Java programming.

Note the keyword `public` in the preceding code. This keyword is an *access specifier*. The `public` access specifier indicates that this class is available anywhere in a program that makes use of it.

Also note that if you make a class `public`, Java insists that you name the file after it. That is, you can only have one `public` class in a `.java` file. The reason for this is that the Java compiler will translate the `.java` file into a bytecode file with the extension `.class`, which means that `App.java` will be translated into `App.class`; and if the JVM needs the `App` class, it'll know to look in the `App.class` file. Because JVM uses the name of the

file to determine what public classes are in the file, you can only have one public class in a file. For that reason, the code for the `App` class must be in a file named `App.java` (note that Java is pretty particular about this, and capitalization counts here, even in Windows that normally doesn't pay any attention to capitalization). The actual implementation of the class we are defining here will go between the curly braces:

```
public class App { . . . }
```

Java always encloses blocks of code within curly braces, that is, "{" and "}".

Line 2—`public static void main(String[] args)`

Here's the next line of code in our application:

```
public class App { public static void main(String[] args) { . . . } }
```

What's happening here is that we are creating a *method* in the `App` class. A method in OOP is like a function or subroutine in standard programming—it's a block of code that you can pass control to and that can return a value. Methods provide handy ways of wrapping code into a single functional unit; when you call a method, JVM executes the code in the method.

Here, the idea is that we are creating a method named `main`, which is the method that the JVM will look for when it starts an application (applets do not have a `main` method). When it finds the `main` method, the JVM passes control to it, and we'll place the code we want to execute in this method's code block.

There are a few things to note before continuing. The `main` method must be declared with the `public` access specifier, which means it may be called outside its class. It must be declared `static` as well, which means that `main` is a class method, not an object method. It must not return a value when it's finished executing, which is why we use the keyword `void` in this code (in other words, a return value of type `void` means that there actually is no return value). Finally, note the argument in the parentheses following the word `main`: `String[] args`. You place an argument list in the parentheses of a method declaration like this to indicate what values are passed to the method and can be used by the code in the method. In this case, we are indicating that `main` has passed an array of string values called `args`. These string values hold the values passed from the command line when you start the application; for example, if you type `java app Hello there`, "Hello" and "there" would be the two strings in the `args` array. Because we won't use any command-line arguments in this application, we won't use `args` in the code for the `main` method.

This line of code then starts the `main` method. The whole job of this method is to print out the text "Hello from Java!" which is done in the next line of code.

Line 3—`System.out.println("Hello from Java!");`

The `main` method has one line of code in it:

```
    System.out.println("Hello from Java!"); {
    }
```

This is the actual line of code that does the work of printing the statement. In this case, we are using some of the code that the programmers at Sun have already created to display the text "Hello from Java!" In particular, the `java.lang` package's `System` class is used here. Libraries of classes are called *packages* in Java, and the `java.lang` package is built into every Java program, which means you don't have to take special steps to make use of it, as you do with other Java packages. The `java.lang` package's `System` class includes a *field* (that is, a data member of the class) called `out`, and this field in turn has a method named `println`, which does the actual displaying of text.

To refer to the `System` class's `out` field, we use the terminology `System.out`. To use the `out` field's `println` method (which stands for *print line*), we use the terminology `System.out.println`. To print the text "Hello from Java!" we pass that text to `System.out.println` by enclosing it in quotes.

Also note that this line of code ends with a semicolon. This end-of-statement convention is something that Java has inherited from C and C++ (in fact, Java has inherited a lot from C and C++), and you end nearly all statements in Java with a semicolon. If this isn't something you are used to, you'll pick it up pretty quickly because the Java compiler refuses to translate your code into bytecodes until the semicolons are in place.

That's it, then—you've created your new application and stored it in a file named `App.java`. What's the next step? To get it actually run, take a look at the next solution.

Compiling Code

Now we have to compile the program and then we can run it. To translate a Java program into a bytecode file that the JVM can use, you use the Java compiler `javac` (for example, on Windows machines, this program will be called `javac.exe`, which is in the Java bin directory). Here's how you use `javac` in general:

```
javac [options] [sourcefiles] [@files]
```

The arguments to `javac` are:

- `options`—Command-line options
- `sourcefiles`—One or more source files to be compiled (such as `App.java`)
- `@files`—One or more files that list source files

To compile `App.java`, use the following command:

```
C:\>javac App.java
```

Note that to make sure `javac` can find `App.java`, you should first make sure the current directory is the one that has `App.java` in it. For example, in Windows, if you've stored `App.java` in `C:\Nancy`, to compile `App.java`, first make sure `C:\Nancy` is the current directory (you can use the change directory command, `cd`, like this: `cd C:\Nancy`). Then, run `javac` like this: `C:\Nancy>javac App.java`. Alternatively, you can specify the path of `App.java` like this: `C:\>javac C:\Nancy\App.java`.

In addition, note that in this book, we'll use the generic prompt `C:\>` for the command line, but this actually stands for the current working directory, whatever that may be in your case—for example, `C:\Bertie\Development>`, `C:\Programs>`, Unix prompts, such as `%`, `/home/Nancy/programs:`, and so on. That is to say `C:\>` is just going to be a placeholder for the actual prompt on your machine for the current directory you are in.

The Java compiler `javac` takes the file `App.java` (assuming there are no errors), compiles it, translates it, and then creates a new file called `App.class`. If errors occur, the Java compiler will tell you what they are, including what line of code is wrong—in this case, we have forgotten the `println` method and tried to use the one called `printline`:

```
C:\>javac App.java
App.java:3: error: cannot find symbol
    System.out.printline("Hello from Java!"); }
                   ^
    symbol:   method printline(String)
    location: variable out of type PrintStream
1 error
```

When `App.java` is successfully compiled to bytecode, the new file, `App.class`, contains all that JVM will need to create objects from the `App` class. So, we have created `App.class`. Now, how do you actually run it in JVM?

Compiling Code: Using Command-Line Options

There are quite a number of options—that is, command-line directives—you can use with `javac`. For example, here's how you can use the `-d` option to have `javac` place the file `App.class` in an existing directory called `temp`, which, in this case, is a sub-directory of the current directory:

```
javac -d temp App.java
```

The following is the list of `javac` options; note that the options that start with `-X` (called *non-standard options*) are marked that way by Sun because they may change in the future:

- `-classpath classpath`—It sets the user class path, overriding the user class path in the `CLASSPATH` environment variable. If neither `CLASSPATH` nor `-classpath` is specified, the user class path will be the current directory. Note that if the `-sourcepath` option is not used, the user class path is searched for source files as well as class files.
- `-d directory`—It sets the destination directory for `.class` files. For readers who know what Java packages are, if a class is a part of a package, `javac` puts the `.class` file in a sub-directory

that reflects the package name, creating directories as needed. For example, if you specify `-d c:\classes` and the class is called `com.package1.Class1`, the class file is called `c:\classes\com\package1\Class1.class`. If `-d` is not specified, `javac` puts the class file in the same directory as the source file. Note that the directory specified by `-d` is not automatically added to your user class path.

- ❑ `-deprecation`—It shows a description of each use or override of a deprecated member or class. (Without `-deprecation`, `javac` only shows the names of source files that use or override deprecated members or classes.)
- ❑ `-encoding`—It sets the source file encoding name. If `-encoding` is not specified, the platform default converter is used.
- ❑ `-g`—It generates all debugging information, including local variables. By default, only line number and source file information are generated.
- ❑ `-g:none`—It makes the compiler unable to generate any debugging information.
- ❑ `-g:{keyword list}`— It generates only some kinds of debugging information, specified by a comma-separated list of keywords. The valid keywords are `source` (source file debugging information), `lines` (line number debugging information), and `vars` (local variable debugging information).
- ❑ `-nowarn`— It disables all warning messages.
- ❑ `-O`—It optimizes code for performance in terms of the quickest execution time. Note that using the `-O` option may slow down compilation, produce larger class files, and make the program difficult to debug. Note that before version 1.2, the `-g` and `-O` options of `javac` could not be used together. As of version 1.2, you could combine the `-g` and `-O` options, but you might get odd results, such as missing variables and relocated or missing code. The `-O` option no longer automatically turns on `-depend` or turns off `-g`.
- ❑ `-sourcepath`—It specifies the source code path to search for class or interface definitions. As with the user class path, source path entries are separated by semicolons (;) and can be directories, `.jar` (Java Archive) files or `zip` files. If you use packages, the local pathname within the directory or archive must reflect the package name, as you'll see later. Note that classes found through the class path are subject to automatic recompilation if their source code files are found.
- ❑ `-verbose`—It creates “verbose” output. This includes information about each class loaded and each source file compiled.
- ❑ `-X`—It displays information about non-standard options and quits.
- ❑ `-Xdepend`—It searches all reachable classes for more recent source files to recompile. This option will more reliably discover classes that need to be recompiled, but it can slow down the compilation process dramatically.
- ❑ `-Xstdout`—It sends compiler messages to `System.out`. By default, the compiler messages go to `System.err`, which you'll learn more about later.
- ❑ `-Xverbosepath`—It describes how paths and standard extensions were searched to find source and class files.
- ❑ `-Joption`—You use this option to pass an option to the Java launcher called by `javac`. For example, `-J-Xms64m` sets the start-up memory to 64 MB. Although this option does not begin with `-X`, it's not a standard option of `javac`. It's a common convention for `-J` to pass options to the underlying JVM executing applications written in Java.

Cross-Compilation Options

Cross-compilation options are considered an advanced topic; `javac` supports cross-compiling, where classes are compiled with the bootstrap (default) and extension classes of a different Java platform implementation. You must use `-bootclasspath` and `-extdirs` when cross-compiling. Here are the cross-compilation options:

- ❑ `-target version`—It generates class files that will work on JVMs with the specified version.

- ❑ `-bootclasspath bootclasspath`—It cross-compiles against the specified set of boot classes. As with the user class path, boot class path entries are separated by semicolons (;) and can be directories, `.jar` files or `.zip` files.
- ❑ `-extdirs directories`—It cross-compiles against the specified extension directories; `directories` is a semicolon-separated list of directories. Each `.jar` file in the specified directories is automatically searched for class files.

Compiling Code: Checking for Deprecated Methods

The Java compiler `javac` will now tell you if you are using a method that's been deprecated, which it does not use unless you specifically asked it to. Even better, you can use the `-deprecation` option to make sure you get all the details.

The `-deprecation` option is a good one, and it's the standard option we use to make sure we avoid deprecated methods. Suppose you have a program that uses `addItem` (a deprecated method) of the `List` class, and when you try to compile it, `javac` gives you the following result:

```
C:\>javac App.java
Note: App.java uses or overrides a deprecated API.
Note: Recompile with -Xlint:deprecation for details.
```

That's not much help. However, using the `-deprecation` option, you can pinpoint the exact problem:

```
C:\>javac -deprecation App.java

App.java:7: warning: [deprecation] addItem(String) in List has been deprecated
    list.addItem("Dreamtechpress");
      ^
1 warning
```

NOTE

You can also use `-Xlint compiler option` to pinpoint the warning point in the program.

Running Code

You run Java applications with the program named `Java` (in Windows, for example, this is the `java.exe` file in the Java bin file). The Java program, called the `java` tool, is what actually runs JVM. Here's how you can use the `java` tool:

```
java [options] class [argument ...]
java [options] -jar file.jar [argument ...]
```

The parameters used in the preceding lines are explained as follows:

- ❑ `options`—Command-line options, which we'll cover in a topic coming right up.
- ❑ `class`—The name of the class to be invoked.
- ❑ `file.jar`—The name of the Java Archive (JAR) file to be invoked. This is used only with `-jar`.
- ❑ `argument`—A command-line argument passed to the `main` method.

For example, to run the application named `app`, which is in the file `App.class`, you could execute the following command at the command line (note that you omit the `".class"` part of `App.class` here):

```
C:\>java App
```

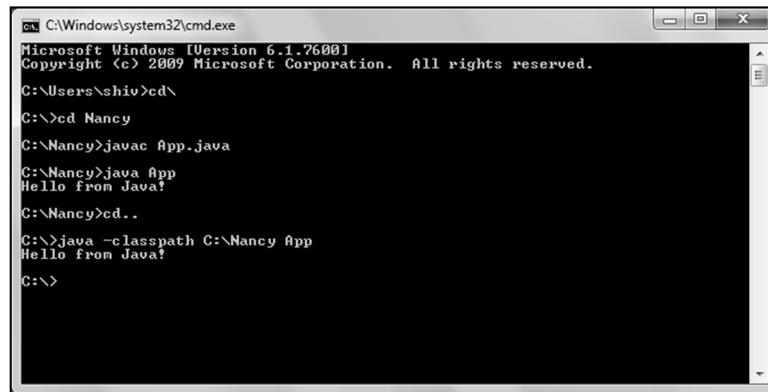
The result appears immediately:

```
C:\>java App
Hello from Java!
```

TIP

To make sure the `java` tool can find `App.class` in this example, you should first make sure the current directory is the one that has `App.class` in it. For example, in Windows, if you've stored `App.class` in `C:\Nancy`, to run it, first make sure `C:\Nancy` is the current directory (you can use the change directory command, `cd`, like this: `cd C:\Nancy`). Then, run Java like this: `C:\Nancy>java App`. Alternatively, you can specify the path of `App.class` like this: `C:\>java -classpath C:\Nancy App`.

You can see how this works in a DOS window under Windows in Figure 1.3:



```
C:\Windows\system32\cmd.exe
Microsoft Windows [Version 6.1.7600]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.

C:\Users\shiv>cd\
C:\>cd Nancy
C:\Nancy>javac App.java
C:\Nancy>java App
Hello from Java!
C:\Nancy>cd..
C:\>java -classpath C:\Nancy App
Hello from Java!
C:\>
```

Figure 1.3: Running an Application in a DOS Window

That's all it takes—now you've written, compiled, and run your first application.

Note that if your application isn't responding or if you want to stop it for some reason, you can type `Ctrl+C`. If that doesn't work, try the `Esc` key.

You'll also see how to create windowed applications in this book, and when you run one of these applications with the `java` tool, you get the results as shown in Figure 1.4 below.

There's one thing to note about Figure 1.4—the console window (a DOS window here) hangs around in the background and waits for the application to finish before continuing (that is, before the DOS prompt reappears in this case). If you don't want a console window associated with your windowed application, you can use the `javaw` tool like this:

```
C:\>javaw App
```

Here's how you use `javaw` in general—just like the `java` tool:

```
javaw [options] class [argument ...]
javaw [options] -jar file.jar [argument ...]
```

The parameters used by `javaw` are as follows:

- ❑ `options`—Command-line options, which we'll cover later in the chapter.
- ❑ `class`—The name of the class to be invoked.
- ❑ `file.jar`—The name of the Java Archive (JAR) files to be invoked. This is used only with `-jar`.
- ❑ `argument`—A command-line argument passed to the `main` method.

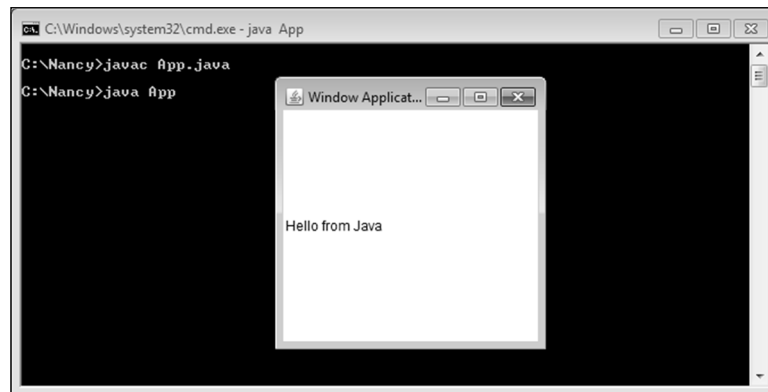


Figure 1.4: Running a Windowed Application

When you launch a Java windowed application like this, the console window does not wait for the application to finish; if you are running in DOS, the windowed application appears and the DOS prompt reappears in the DOS window. This gives a more professional feel to those applications that you release for general use.

While we are discussing compiling and running code, there's another detail we should cover—the command-line options you can use with the `javac` and `java` commands. We'll take a look at them in the next two topics.

Running Code: Using Command-Line Options

If all `.class` files are stored in one directory and you don't want to keep switching to that directory to run them, then you can use the `java` tool's `-classpath` option or set the `CLASSPATH` environment variable so that the Java compiler will search for your classes correctly.

TIP

For more on `CLASSPATH`, an important topic in Java programming, see the upcoming solution "Finding Java Classes with `CLASSPATH`".

We'll take a look at using the command-line options in this solution; you use these options with the `java` and `javaw` tools as follows:

```
java [options] class [argument ...]
java [options] -jar file.jar [argument ...]
javaw [options] class [argument ...]
javaw [options] -jar file.jar [argument ...]
```

The command-line options you can use with these tools are as follows (note that non-standard options, which means they might not be supported in the future, begin with an `X`):

- ❑ `-classpath classpath` or `-cp classpath`—It specifies a list of directories, `.jar` files or `.zip` files to search for class files. You separate class path entries with semicolons (;). Note that specifying `-classpath` or `-cp` overrides any setting of the `CLASSPATH` environment variable. Used with `java` or `javaw`, `-classpath` or `-cp` only specifies the class path for user classes. If `-classpath` and `-cp` are not used and `CLASSPATH` is not set, the user class path is limited to the current directory, which is referred to with a dot (.). See the topic "Finding Java Classes with `CLASSPATH`" later in this chapter for more information.
- ❑ `-Dproperty=value`—It sets a system property value.
- ❑ `-jar`—It executes a program encapsulated in a JAR file. The first argument is the name of a JAR file instead of a start-up class name. When you use this option, the JAR file is the source of all user classes, and the other user class path settings are ignored.
- ❑ `-verbose` or `-verbose:class`—It displays information about each class loaded.
- ❑ `-verbose:gc`—It reports on each garbage collection event. Garbage collection involves automatic memory management in Java.
- ❑ `-verbose:jni`—It reports information about the use of native (that is, platform-specific) methods and other Java Native Interface activity.
- ❑ `-version`—It displays version information and exits.
- ❑ `-?` or `-help`—It displays usage information and exits.
- ❑ `-X`—It displays information about non-standard options and exits.
- ❑ `-Xbootclasspath:bootclasspath`—It specifies a semicolon-separated list of directories, `.jar` files or `.zip` files to search for boot class files. Note that these will be used in place of the boot class files included with Java itself.
- ❑ `-Xdebug`—It starts with the debugger enabled.
- ❑ `-Xnoclassgc`—It disables class garbage collection.
- ❑ `-Xmsn`—It indicates the initial size of the memory pool you want to use (this value must be greater than 1000). To multiply the value by 1000, append the letter `k`. To multiply the value by 1 million, append the letter `m`. The default value is `1m`.

- ❑ `-Xmxn`—It specifies the maximum size of the memory pool (this value must be greater than 1000). To multiply the value by 1000, append the letter k. To multiply the value by 1 million, append the letter m. The default value is 64m.
- ❑ `-Xrunhprof[:help][:<suboption>=<value>, ...]`—It enables CPU, heap, or monitor profiling. This option is usually followed by a list of comma-separated pairs of the form `<suboption> = <value>`.
- ❑ `-Xrs`—It reduces the use of operating system signals.
- ❑ `-Xcheck:jni`—It performs additional checks for Java Native Interface (JNI) functions.
- ❑ `-Xfuture`—It performs strict class-file format checks.

Commenting Your Code

Sometimes, a code can be very cryptic and hard to decipher. For that reason, Java lets you place descriptive comments in your code to let you explain to anyone who reads that code how the program works and what it does. As an example, let's add comments to the application we have already developed in the previous topics:

```
public class App { public static void main(String[] args) {
    System.out.println("Hello from Java!"); }
}
```

Java supports three types of comment, two of which are taken from C++. You can surround a comment of any length with the characters `/*` and `*/` as follows:

```
/* This application prints out "Hello from Java!" */
public class App { public static void main(String[] args) {
    System.out.println("Hello from Java!"); }
}
```

The Java compiler will ignore all the text between the `/*` and `*/` markers. You can split comments between these markers across multiple lines as follows:

```
/* This application prints out "Hello from Java!"
Created by: G. Whiz, 1/1/00 */
public class App {

    public static void main(String[] args) {
        System.out.println("Hello from Java!"); }
}
```

In fact, in many corporate environments, you are expected to use a standard comment header, created with the `/*` and `*/` forms of comment, for all new code. It might look something as follows:

```
/* *****
 * This application prints out "Hello from Java!" *
 * *
 * Author: G. Whiz *
 * Imports: None *
 * Parameters: Command-line arguments *
 * Returns: None *
 * Assumptions: None *
 * Creation date: 1/1/00 *
 * Last Update: 1/1/01 *
 * ***** */
public class App {
    public static void main(String[] args) {

        System.out.println("Hello from Java!"); }
}
```

Java also supports a one-line comment using a double slash (`//`). The Java compiler will ignore everything on a line after the `//` marker, so you can create whole lines that are comments or just add a comment to an individual line as follows:

```
/* This application prints out "Hello from Java!" */
public class App { //Create the App class
    //Create main(), the entry point for the application.
```

```

    public static void main(String[] args) {
        //Print out the message with
        System.out.println("Hello from Java!"); }
}

```

Finally, Java also supports a documentation comment, which starts with `/**` and ends with `*/`. This comment is designed to be used with the `javadoc` tool, which can create documentation for you nearly automatically. An example of using `/**` and `*/` is as follows:

```

/** This application prints out "Hello from Java!" */
public class App {
    public static void main(String[] args) {
        System.out.println("Hello from Java!"); }
}

```

Commenting your code can be invaluable in team environments where you share your code source files with others. It's also handy if someone else is going to take over a project that you have been working on.

Importing Java Packages and Classes

The classes that Sun has created for you to use are stored in class libraries called packages. To make a class in a package available to your code, you have to import the package, which means the compiler will search that package for classes. You can also import individual classes that are not part of a package. By default, only the basic Java statements are available to you in an application, that is, the ones in the core `java.lang` package. The compiler automatically imports the `java.lang` package for you, but to use the rest of the classes that come with Java, you'll have to do your own importing with the `import` statement. Here's how you use that statement:

```
import [package1[.package2...]](classname[*]);
```

Note that you put a dot (.) between the package and the class names to keep them separate. The standard Java packages themselves are stored in a large package called `java`, so the `util` package is really called the `java.util` package (there are other large packages like the `java` package available; for example, the extensive `swing` package is stored in the `javax` package). You can refer to the `Date` class in `java.util` as `java.util.Date`. Here's how to import that class into a program:

```

import java.util.Date;
public class App { . . .
}

```

Note that if you are going to use `import` statements for importing classes into a program, the `import` statements should be at the top of the code. Now, we are free to use the `Date` class as follows (note that we are creating an object from the `Date` class by using the Java `new` operator):

```

import java.util.Date;
public class App {
    public static void main(String[] args) {
        System.out.println("Today = " + new Date()); }
}

```

When you run this application, you'll see the current date displayed as follows:

```

C:\>java App
Today = Tue July 17 14:05:15 IST 2012

```

You can see that by studying the general form of the preceding `import` statement, there's also a shorthand technique that loads in all the classes in a package—you can use an asterisk (*) as a wildcard to stand for all the classes in a particular package. Here's how that would look if you wanted to import all the classes in the `java.util` package at once:

```

import java.util.*;
public class App {

    public static void main(String[] args) {
        System.out.println("Today = " + new Date()); }
}

```

TIP

Importing packages and classes only indicates to the compiler where to look for the code it needs—it does not increase the size of your code. For that reason, the bytecode file `App.class` will be of the same size regardless of whether you use the `import java.util.Date;` statement or the `import java.util.*;` statement.

This is fine if you stick with importing the Sun-provided classes because Java knows where to look for the classes it was installed with. But what if you want to import your own classes or ones provided by a third party? Here's an example. Suppose you have a user-defined package named `printer`, having a class named `Printer` in a file named `Printer.java`, and that class has one method named `print`:

```
package printer;
public class Printer {
    public void print() { System.out.println("Hello from Java!"); }
}
```

You might want to make use of the `print` method in other classes. In this case, we are creating a new object of the `Printer` class using the `new` operator and using that object's `print` method in an application named `App`:

```
public class App {
    public static void main(String[] args) { (new Printer()).print(); }
}
```

To do this, you can import the `Printer` class as follows (note that you can also place the code for the `Printer` class in the same file as the `App` class, in which case you wouldn't have to import the `Printer` class):

```
import printer.*;
public class App {
    public static void main(String[] args) { (new Printer()).print(); }
}
```

Here, we have learned how to import a Java built-in and a user-created class in our Java program.

However, suppose you want to store all your classes in a directory named, say, `c:\classes`. How will the Java compiler find `Printer.class` there? To answer that question, take a look at the next solution on `CLASSPATH`

Finding Java Class with `CLASSPATH`

By default, Java will be able to find its bootstrap classes (the ones it comes with), extension classes (those that use the Java Extension Framework), and classes in the current directory (that is, where you are compiling your program). Classes can be stored in the `.class` files, `.jar` files, and `.zip` files. Java can search all these types of files.

NOTE

Bootstrap classes are the classes that come by default with JDK, and Extension classes are the classes that use the Java Extension Framework. But, what if you want to have Java search for classes in another directory or in a `.jar` file supplied by a third party? You can do that with the `CLASSPATH` environment variable because Java uses this variable to determine where you want to search for classes.

Here's an example that was first introduced in the previous solution. Say that you have a class named `Printer` in a file named `Printer.java` and that class has one method named `print`:

```
public class Printer {
    public void print() { System.out.println("Hello from Java!"); }
}
```

In the previous solution, you want to use the `print` method in another class—as in this case where we are creating a new object of the `Printer` class by using the `new` operator and using that object's `print` method in an application named `App`:

```
import printer.*;
public class App {
    public static void main(String[] args) { (new Printer()).print(); }
}
```

This works if `Printer.class` is in the same directory in which you are compiling this application, because the Java compiler will search the current directory by default. But, suppose you want to store all your classes in a directory named `c:\classes`. How will the Java compiler find `Printer.class` there?

To make the Java compiler search `c:\classes`, you can set the `CLASSPATH` environment variable to include that directory. By default, there are no paths or directories in `CLASSPATH`, but you can add a semicolon-separated list to `CLASSPATH`, like this (note that it's important here not to have any spaces around the equals sign):

```
SET CLASSPATH=c:\classes;c:\newclasses
```

You can also determine the current `CLASSPATH` setting by typing the `SET CLASSPATH` at the command line and pressing Enter. To permanently set the classpath in Windows 7, follow these steps:

1. Click the Windows icon on the taskbar, right-click the Computer, and select the Properties option. You can also reach the same location by opening the Control Panel | System and Security | System.
2. Click the Advanced system settings option in the System Properties window. This opens up the System Properties dialog box.
3. Click the Environment Variables button to open the Environment Variables dialog box.
4. Edit the `CLASSPATH` setting the way you want.
5. You may append the new classpath to the already existing `CLASSPATH` variable or create a new `CLASSPATH` variable.
6. Finally, click OK.

Note that if you are appending the `CLASSPATH` variable, the different paths must be separated by semicolons (;).

Now, the Java compiler (and other Java tools, such as the `java` tool) will know enough to search `c:\classes` and `c:\newclasses` automatically. This means that the following code will now work if `Printer.class` is in `c:\classes` because that directory is in `CLASSPATH`:

```
import printer.*;
public class App {
    public static void main(String[] args) { (new Printer()).print(); }
}
```

You can append the current settings in `CLASSPATH` to a new setting, like this:

```
SET CLASSPATH=c:\classes;c:\newclasses;%CLASSPATH%
```

Note that you can also search `.jar` and `.zip` files for classes, as shown here:

```
SET CLASSPATH=server.jar;classes.zip;%CLASSPATH%
```

Originally, `CLASSPATH` was a big headache for beginners in Java programming because no classes were considered bootstrap classes, which meant that you had to set up and understand `CLASSPATH` before you could use Java at all. That's been fixed with the concept of bootstrap classes, which are the classes that come with Java (and are searched automatically). However, if you want to use non-standard packages or store your own classes in other directories, it's important to know how to set `CLASSPATH`.

Summary

In this chapter, you got familiar with Java 8, its environment, and designing of programs. The chapter introduced you to Swings and Utilities. In addition, it discussed the enhancements in JDBC and provided a brief description of garbage collector. You also learned how to install Java. Toward the end, you developed simple Java programs with the introduction of how to import the packages.

In the next chapter, you will learn about the variables, arrays, and strings.



2

Variables, Arrays, and Strings

<i>If you need information on:</i>	<i>See page:</i>
Variables	27
Data Typing	28
Arrays	28
Strings	30
What Data Types are Available?	30
Creating Integer Literals	31
Creating Floating-Point Literals	32
Creating Boolean Literals	32
Creating Character Literals	32
Creating String Literals	33
Creating Binary Literals	34
Using Underscores in Numeric Literals	34
Declaring Integer Variables	35
Declaring Floating-Point Variables	35
Declaring Character Variables	36
Declaring Boolean Variables	36
Initializing Variables Dynamically	37
Conversion between Data Types	38
Declaring One-Dimensional Arrays	40
Creating One-Dimensional Arrays	40
Initializing One-Dimensional Arrays	40
Declaring Multi-Dimensional Arrays	41
Creating Multi-Dimensional Arrays	41

<i>If you need information on:</i>	<i>See page:</i>
Initializing Multi-Dimensional Arrays	42
Creating Irregular Multi-Dimensional Arrays	42
Getting an the Length of an Array	43
Understanding General Form of Static Import	44
Importing Static Members	44
The String Class	44
Creating Strings	48
Getting String Length	50
Concatenating Strings	50
Getting Characters and Substrings	50
Searching For and Replacing Strings	51
Changing Case in Strings	52
Checking for Empty String	52
Formatting Numbers in Strings	52
The StringBuffer Class	53
Creating StringBuffers	55
Getting and Setting StringBuffer Lengths and Capacities	56
Setting Characters in String Buffers	56
Appending and Inserting Using StringBuffers	57
Deleting Text in StringBuffers	57
Replacing Text in StringBuffer	57
Using the Wrapper Class	58
Autoboxing and Unboxing of Primitive Types	59
Learning the Fundamentals of Varargs Methods	61
Summary	65

This chapter begins our discussion of *Java syntax*, and you'll see a great deal of syntax usage in this chapter. We are going to cover how Java stores and retrieves data in variables, arrays, and strings. Working with data is a fundamental part of any significant program, and the information in this chapter is essential for any Java programmer. Even if you are familiar with Java and have already been programming in it, take a look at the material in this chapter because there's a lot coming up.

Variables

There are various kinds of variable that serve as containers for storing data in the memory. Different kinds of variable store data in different formats, and the selection of variable should be made depending upon the data that they will be storing and the memory that will be required to be kept aside for storing it. For example, an integer variable type, the `int` type, can store up to 4 bytes (or 32 bits) and contain only integer values between 2,147,483,648 and 2,147,483,647. There are some more variables that Java provides such as floating-point numbers, short, long, byte, and char, and all of them will be explained in this chapter.

Before you use a variable in Java, you must *declare* it, specifying its data type. Here's how you declare variables in Java:

```
type name [= value][, name [= value]...];
```

Here's an example showing how to declare a variable of the `int` type, which means a numeric value will be stored in it (the variable is named `days`):

```
public class App {
    public static void main(String[] args) {
        int days;
        . . .
    }
}
```

This code allocates 32 bits of storage in memory and labels the location of that storage, as far as the Java compiler is concerned, as `days`, which means you can now refer to that name in code. Here's how to store a numeric value 365 in the `days` variable using the Java assignment operator (`=`):

```
public class App {
    public static void main(String[] args) {
        int days;
        days = 365;
        . . .
    }
}
```

Here, the value 365 is an integer *literal*, which means a literal value that you place directly in your code. We'll take a look at what kinds of literal Java allows throughout this chapter. To verify that `days` now hold 365, you can print it out on the console:

```
public class App {
    public static void main(String[] args) {
        int days;
        days = 365;
        System.out.println("Number of days = " + days);
    }
}
```

Here's the result of this code:

```
C:\>java App
Number of days = 365
```

As you can see, we have created a variable, stored data in it, and fetched the same data to print it on the screen. That's how it works.

There's also a convenient shortcut that lets you initialize a variable when you declare it. Here, `days` are declared and initialized to 365 in one step:

```
public class App {
    public static void main(String[] args) {
        int days = 365;
        System.out.println("Number of days = " + days);
    }
}
```

The `int` type is only one kind of simple variable you can use. Mentioned below are the various possibilities:

- ❑ **Integers**—Refer to the types `byte`, `short`, `int`, and `long` that hold signed, whole-value numbers.
- ❑ **Floating-point numbers**—Refer to the types `float` and `double` that hold signed floating-point numbers.
- ❑ **Character**—Refers to the `char` type that holds characters such as letters and numbers.
- ❑ **Boolean**—Refers to the type that holds only two types of values: *true* and *false*.

Data Typing

Java puts considerable emphasis on its data types. It's a *strongly-typed* language, which means it insists that the simple variables you declare and use must fit in the listed types.

Every simple variable must have a type (and in fact, every expression—every combination of terms that Java can evaluate to get a value—has a type as well). Also, Java is very particular about maintaining the integrity of those types, especially if you try to assign a value of one type to a variable of another type. In fact, Java is more strongly typed than a language such as C++. In C++, for example, you can assign a floating-point number to an integer, and C++ will handle the type conversion for you, but you cannot do that in Java. You can, however, convert between certain data types in Java such as between the integer types. We'll take a look at that later in this chapter.

TIP

When working with variables, you might find the Java compiler issuing a lot of errors and warnings about data types, which can take some time getting used to; bear in mind that the inspiration for making Java very particular about adhering to data types and not mixing them easily is to prevent errors in your code.

That's an overview of what's going on in Java with simple data types and variables; it's now time to take a look at compound data storage in depth, which as far as this chapter is concerned means *arrays*.

Arrays

Simple types are good only for storing single data items. However, sometimes the data that needs to be stored is a lot more complex. For instance, you want to start a new bank, say, JP Bank (short for Java Programming Bank) and keep a track of the amount in each account indexed by account number. This situation presents the need to work with compound data. The usage of arrays is appropriate in such a scenario.

An array allows grouping of simple data to form a *compound data structure* that can be referred to by using a single name. Every data item stored in that compound data structure is stored at a particular position in the array and can be referred by the numeric index (or position) of the data in the data structure. It is vital to store the data in indexed form with the index sorted numerically because computers perform millions of operations in a second and data stored in indexed form can be referenced using the numerical value. This way the entire data structure can be traversed by simply incrementing the index's value, and the values can be accessed with ease.

Here's an example. In this case, we'll start the Java Programming Bank out with 100 new accounts, and each one will have its own entry in an array named `accounts[]`. The square braces at the end of the `accounts[]` represent that it is an array and the value of the index can be placed enclosed in those braces to refer to the value stored at a particular position. Here's how we create the `accounts[]` array, making each entry in it of the floating-point type `double` for extra precision. First, we declare the array; then, we create it with the `new` operator, which is what Java uses to actually allocate memory:

```
public class App {
    public static void main(String[] args) {
        double accounts[];
        accounts = new double[100];
        . . .
    }
}
```

Now that we have created an array with 100 items, we can refer to those items numerically, like this (note that we are storing \$43.95 in account 3 and printing that amount out):

```
public class App {
    public static void main(String[] args) {
```

```

double accounts[];
accounts = new double[100];
accounts[2] = 43.95;
System.out.println("Account 3 has $" + accounts[2]); }
}

```

Here's the result of this program:

```

C:\>java App
Account 3 has $43.95

```

As you can see, you can now refer to the items in the array by using a numeric index, which organizes them in an easy way. In Java, the lower bound of an array you declare this way is 0, so the statement `accounts = new double[100]` creates an array whose first item is `accounts[0]` and last item is `accounts[99]`.

You can combine the declaration and creation steps into one step, as shown below:

```

public class App {
    public static void main(String[] args) {
        double accounts[] = new double[100];
        accounts[2] = 43.95;
        System.out.println("Account 3 has $" + accounts[2]); }
}

```

You can also initialize an array with values when you declare it if you enclose the list of values you want to use in curly braces, as you'll see in this chapter. For example, this code creates four accounts and stores 43.95 in `accounts[3]`:

```

public class App {
    public static void main(String[] args) {
        double accounts[] = { 0, 0, 0, 43.95 };
        System.out.println("Account 4 has $" + accounts[3]); }
}

```

It turns out that some of the customers in the Java Programming Bank are unhappy, though; they want a checking account in addition to a savings account. How will you handle this and still keep things indexed by account number?

The `accounts[]` array is a one-dimensional array, also called a *vector*, which means you can think of it as a single list of numbers that you can index with one number. However, arrays can have multiple dimensions in Java, which means you can have multiple array indexes. In this next example, we'll extend `accounts[]` into a two-dimensional array—`accounts[][]`—to handle both a savings account and a checking account. The first index of `accounts[][]` will be 0 for savings accounts and 1 for checking accounts, and the second index will be the account number as before. Here's how this works in code:

```

public class App {
    public static void main(String[] args) {
        double accounts[][] = new double[2][100];
        accounts[0][2] = 43.95;
        accounts[1][2] = 2385489382.06;
        System.out.println("Savings account 3 has $" + accounts[0][2]);
        System.out.println("Checking account 3 has $" + accounts[1][2]); }
}

```

After creating a two-dimensional array, values in it can be referenced using two index values. For example, here we are assuming the value 0 for savings account and 1 for checking account. The savings account details in the account number 3 can now be referenced as `accounts[0][2]` and that in the checking account can be referenced as `accounts[1][2]`. Shown here are the results when you run this application:

```

C:\>java App
Savings account 3 has $43.95
Checking account 3 has $2.38548938206E9

```

Note that we have given account 3 a checking balance of \$2,385,489,382.06 (wishful thinking) and that Java has printed that out as 2.38548938206E9. This is Java's shorthand for 2.38548938206X10⁹—not an inconsiderable bank balance by any means.

TIP

You'll see a lot of arrays in this chapter, but you should know that Java now supports much more complex data structures than arrays. These data structures are built into the language. Java now supports hashes and maps as well as other types of data structure, and as you'll see all that when we take a look at the collection classes in the forthcoming chapter.

Strings

You may have noticed that we have been using the + operator to create the text to print in the previous examples like this:

```
public class App {
    public static void main(String[] args) {
        double accounts[][] = new double[2][100];
        accounts[0][2] = 43.95;
        accounts[1][2] = 2385489382.06;
        System.out.println("Savings account 3 has $" + accounts[0][2]);
        System.out.println("Checking account 3 has $" + accounts[1][2]); }
}
```

That's because their own class in Java—the `String` class—supports text strings and you can think of the `String` class as defining a new data type.

For example, here's how we create a string named `greeting` that holds the text "Hello from Java!":

```
public class App {
    public static void main(String[] args) {
        String greeting = "Hello from Java!";
        . . .
    }
}
```

Now we can treat this string as we would other types of variables, including printing it out:

```
public class App {
    public static void main(String[] args) {
        String greeting = "Hello from Java!";
        System.out.println(greeting); }
}
```

Here's the result of this application:

```
C:\>java App
Hello from Java!
```

Although strings are not one of the simple data types in Java, yet they deserve a place in this chapter because most programmers treat them as they would any other data type. In fact, many programmers would argue that strings should be a simple data type in Java as they are in other languages. The reason they are not has to do with Java lineage, which stretches back to C. C has no string simple data type; in C, you handle strings as one-dimensional arrays of characters, which is pretty awkward. One of the things that made programmers happy about C++ was that most implementations included a `String` class that you could use much as you would any other data type. Java follows this usage, implementing strings as a class, not as an intrinsic data type, but string handling is so fundamental to programming that it makes sense to start looking at string variables in this chapter.

There are two string classes in Java—`String` and `StringBuffer`. You use the `String` class to create text strings that cannot change, and you can use `StringBuffer` to create strings you can modify. As you can see in the preceding code, you can use strings much as you would any simple data type in Java. We'll take a look at using strings in this chapter as well as in the next chapter (which is on using operators such as + and -). We'll also take a look at using operators on strings.

That's enough overview for now—it's now time to start creating and using variables, arrays, and strings.

What Data Types are Available?

What simple data types can you use to create variables in Java? You'll find them in Table 2.1 below. The simple data types can be broken up by category, as we did at the beginning of this chapter, like this:

- **Integers**—Refer to the `byte`, `short`, `int`, and `long` types. They contain signed, whole-value numbers.

- ❑ **Floating point numbers**—Refer to the `float` and `double` types. They contain signed, floating-point numbers.
- ❑ **Character**—Refers to the `char` type that holds characters such as letters and numbers.
- ❑ **Boolean**—Refers to the type that holds only two types of values: `true` and `false`.

Table 2.1: Variable types

Variable type	Bytes of storage	Range
<code>boolean</code>	1 bit	<code>True</code> , <code>False</code>
<code>byte</code>	1	-128 to 127
<code>char</code>	2	'\u0000' to '\uffff'
<code>double</code>	8	4.9E-324 to 1.7976931348623157E308
<code>float</code>	4	1.4E-45 to 3.4028235E+38
<code>int</code>	4	-2,147,483,648 to 2,147,483,647
<code>long</code>	8	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
<code>short</code>	2	-32,768 to 32,767

That's an overview of what simple data types are available; to put each of them to work, see the following solutions.

TIP

It's worth realizing that you can't completely count on the numeric precision listed in Table 2.1 because Java implementations do differ by machine. However, you can use the `StrictMath` class (you can find more about these classes at <http://docs.oracle.com/javase/8/docs/api/>) to ensure that the same mathematical precision is used no matter what platform your program is running on.

Creating Integer Literals

A literal is a constant value that you can use directly in your Java code, and there are a number of rules that govern them. We have already used integer literals in this chapter, which is the most common type programmers use. Here's the example:

```
public class App {
    public static void main(String[] args) {
        int days = 365;
        System.out.println("Number of days = " + days);
    }
}
```

Here, we are assigning an integer literal with a value of 365 to the variable `days`. By default, integer literals are of the `int` type. However, if you assign them to other integer types, such as `short`, Java converts the literal type automatically. On the other hand, `long` values can have more digits than `int` values, so Java provides an explicit way of creating `long` literals: You append an `L` to the end of the literal. Here's an example:

```
public class App {
    public static void main(String[] args) {
        long value;
        value = 1234567890123456789L;
        System.out.println("The value = " + value);
    }
}
```

Here's the result of this code:

```
C:\>java App
The value = 1234567890123456789
```

You can also create literals in octal format by starting them with a leading zero and in hexadecimal format by starting with `0x` or `0X`. Here are some examples:

```
public class App { public static void main(String[] args) {
    int value; value = 16;
    System.out.println("16 decimal = " + value);
    value = 020;
}
```

```

System.out.println("20 octal = " + value + " in decimal");
value = 0x10;
System.out.println("10 hexadecimal = " + value + " in decimal"); }

```

Here's what this program displays:

```

C:\>java App
16 decimal = 16
20 octal = 16 in decimal
10 hexadecimal = 16 in decimal

```

Creating Floating-Point Literals

Floating-point literals are of type `double` by default in the Java code; examples include 3.1415926535, 1.5, and 0.1111111. The standard notation for floating-point literals is to have a whole number followed by a fractional part. You can also indicate a power of 10 with *e* or *E*, like this:

```
1.345E10
```

This is the same as 1.345×10^{10} or -9.999×10^{-23} , which is the same as -9.999×10^{-23} . Here's an example in which we are trying to assign a floating-point literal to a variable of type `float`:

```

public class App { public static void main(String[] args) {
    float value;
    value = 1.5;
    System.out.println("The value = " + value); }
}

```

Unfortunately, the default type for floating-point literals is `double`, so Java gives me the following error:

```

C:\>javac App.java
App.java:2: error: incompatible types: possible lossy conversion from double to float
    value = 1.5;
           ^
    1 error

```

We can fix this by explicitly making my literal into a `float`, like this:

```

public class App {
    public static void main(String[] args) {
        float value;
        value = 1.5f;
        System.out.println("The value = " + value); }
}

```

Now the code runs as you'd expect:

```

C:\>java App
The value = 1.5

```

Creating Boolean Literals

Boolean values can only be true or false in Java (not 0 or 1 or other numeric values as in other languages—this is the part of Java's strong data typing). This means that *only* two Boolean literals you can use are true and false. Here's an example using true as a Boolean literal:

```

public class App {
    public static void main(String[] args) {
        boolean value;
        value = true;
        System.out.println("The value = " + value); }
}

```

Here's the result of this program:

```

C:\>java App
The value = true

```

Creating Character Literals

The basic form of a Java character literal is just a value that corresponds to a character in the Unicode character set (for more on Unicode, see <http://www.unicode.org>). Character literals are actually numbers that act as

indexes into the Unicode character set, not actual characters. For example, the Unicode code for the letter C is 67. Therefore, the following application prints out a C:

```
public class App {
    public static void main(String[] args) {
        char char3;
        char3 = 67;
        System.out.println("The third letter in alphabet = " + char3); }
}
```

Here's the result of the above code:

```
C:\>java App
The third letter in alphabet = C
```

However, you can also refer to the Unicode code for the letter C with a character literal, which you would enclose in single quotes, like this:

```
public class App {
    public static void main(String[] args) {
        char char3; char3 = 'C';
        System.out.println("The third letter in alphabet = " + char3); }
}
```

Here's is the result of this code, which, obviously, is the same as the result of the previous code:

```
C:\>java App
The third letter in alphabet = C
```

Besides enclosing characters in single quotes to make character literals, you can also enclose special *character escape sequences* in single quotes to make character literals that you couldn't make by typing a single character. Here are the escape sequences:

- ☐ \ ' (single quote)
- ☐ \ " (double quote)
- ☐ \ \ (backslash)
- ☐ \ b (backspace)
- ☐ \ ddd (octal character)
- ☐ \ f (form feed)
- ☐ \ n (newline; called a *line feed* in DOS and Windows)
- ☐ \ r (carriage return)
- ☐ \ t (tab)
- ☐ \ uxxxx (hexadecimal Unicode character)

For example, if you want to show a double quotation mark in the displayed text, you can use the \ " escape sequence, like this:

```
public class App {
    public static void main(String[] args) {
        System.out.println("He said, \"Hello!\""); }
}
```

Here's the result of this code:

```
C:\>java App
He said, "Hello!"
```

Creating String Literals

Here we'll print out some multiline text using the \ n character literal to start a new line:

```
public class App {
    public static void main(String[] args) {

        System.out.println("Here is \nsome multiline\ntext"); }
}
```


Here's the output of this application:

```
C:\>java App
Here is
some multiline
text
```

As with most other programming languages, you can enclose text string literals in double quotes (unlike single character literals, which you enclose in single quotes). You can also use the character escape sequences introduced in the previous topic. Note that string literals in Java code are actually converted by the compiler into `String` objects, not inherent simple data types (which means that odd code such as `"Hello".length()` is perfectly legal and will return the length of the string "Hello").

Creating Binary Literals

One of the features that Java has implemented is to assign the values in binary format. If you are assigning a value in binary format, you need to use the letters `0b` before keying in the actual value. The binary literals express the integral type in the binary number system.

The literals are useful in a number of programming tasks to be performed. They are frequently used to create numeric arrays. The relation between the data members of the array, which is represented using the binary literals, is more apparent than the relation represented using the members in the hexadecimal or octal notations. The reason behind the apparent relation representation is that the numbers represented using binary literals are more detailed than the hexadecimal or octal literals. A good example of the relation between the array members is rotation of 2 bits among the successive array members, as shown in the following example:

```
public class App {
    public static void main(String[] args) {
        int val = 0b00010000;
        int myPhases[] = {
            0b00110011,
            0b11001100,
            0b00110011,
            0b11001100 };
        System.out.println("The value in val is: " + val);
        System.out.println("And following are the values stored in the array myPhases:");
        System.out.println(myPhases[0]);
        System.out.println(myPhases[1]);
        System.out.println(myPhases[2]);
        System.out.println(myPhases[3]); }
    }
```

The output of the program is as follows:

```
C:\>java App
The value in val is: 16
And following are the values stored in the array myPhases:
51
204
51
204
```

Using Underscores in Numeric Literals

The use of underscores in numeric literals is a newly introduced feature of Java 8. This allows the programmer the freedom to use underscores in numeric literals, making it more readable and minimizing the typographical errors that are very likely otherwise. You can use any number of underscore characters (`_`) to break the groups of digits in the long numeric literals. The literals still remain very much ready for performing mathematical operations. This feature was added to Java just for the sake of adding more readability to the numeric values.

An example that demonstrates the usage of underscores in numeric literals is as follows:

```
public class App {
    public static void main(String[] args) {
        long card = 1345_4489_4545_7252L;
    }
```

```

    System.out.println("Your Credit Card number is: " + card);
  }
}

```

The output of the preceding code is as follows:

```

C:\>java App
Your Credit Card number is: 1345448945457252

```

The underscores will not be displayed during the execution, as they are only for the convenience of the programmer. For example, in the above program, we have created a variable of the type `long` and saved a hypothetical 16-digit credit card number that contains underscores to separate every four digits. The `L` at the end of the credit card number in the program signifies that the value is of the `long` data type.

It is important to note that you can put underscores in a numeric literal only in between the digits of the numeric literal. Or the underscore character should be accompanied by at least one digit on each side. You cannot place an underscore in the numeric literals in the following places:

- ☐ At the start or the end of the numeric literal
- ☐ Next to the decimal point in the floating-point literal
- ☐ In front of an `F` or `L` suffix in the floating-point or long literal
- ☐ Places where a string of digits is expected in the numeric literal

Declaring Integer Variables

Java uses four types of integer, each with its own number of bytes put aside for it in memory—`byte` (1 byte), `short` (2 bytes), `int` (4 bytes), and `long` (8 bytes). For the range of possible values each type can handle, see the topic “What Data Types Are Available?” in the Immediate Solutions section of this chapter. Which one you use depends on the range of data you want to use as well as other considerations such as how much memory is available (in case you want to set up a lot of integers).

Here’s an example that puts all the integer types to use, declares an integer of each type, assigns each type some data, and then displays that data:

```

public class App {
    public static void main(String[] args) {
        byte byte1;
        short short1;
        int int1;
        long long1;
        byte1 = 1;
        short1 = 100;
        int1 = 10000;
        long1 = 100000000;
        System.out.println("byte1 = " + byte1);
        System.out.println("short1 = " + short1);
        System.out.println("int1 = " + int1);
        System.out.println("long1 = " + long1); }
}

```

Here’s the result of this application:

```

byte1 = 1
short1 = 100
int1 = 10000
long1 = 100000000

```

Declaring Floating-Point Variables

Java has two built-in types of floating-point variables, each with its own number of bytes set aside for it in memory: `float` (4 bytes) and `double` (8 bytes). For the range of possible values each type can handle, see the topic “What Data Types Are Available?” in this chapter. The one you use depends on the range of data you want to use as well as other considerations such as how much memory is available (in case you want to set up a lot of floating-point values).

Here's an example that declares and uses both a `float` and a `double` (note that we explicitly make each literal value either a `float` or a `double` literal so that there'll be no problem with type conversions):

```
public class App {
    public static void main(String[] args) {
        float float1;
        double double1;
        float1 = 1.111111111F;
        double1 = 1.111111111111E+9D;
        System.out.println("float1 = " + float1);
        System.out.println("double1 = " + double1); }
}
```

Here's the output of the code (note that we have exceeded the precision allowed for a `float`, so its value is rounded):

```
C:\>java App
float1 = 1.111112
double1 = 1.111111111111E9
```

Declaring Character Variables

Here's an example that declares two `char` variables: `char1` and `char2`. This example demonstrates that you can assign either a Unicode code or a character literal to a `char` (in fact, the compiler translates character literals into Unicode codes):

```
public class App {
    public static void main(String[] args) {
        char char1, char2;
        char1 = 65;
        char2 = 'B';
        System.out.println("char1 = " + char1);
        System.out.println("char2 = " + char2); }
}
```

Here's the result of this code:

```
C:\>java App
char1 = A
char2 = B
```

Here's a sneak peak at a future topic, in which we add some text to the end of `char1`, converting it to a string, and increment the value in `char2`, changing it from 'B' to 'C':

```
public class App {
    public static void main(String[] args) {
        char char1, char2;
        char1 = 65;
        char2 = 'B';
        System.out.println("char1 = " + char1);
        System.out.println("char2 = " + char2);
        System.out.println("char1 + 1 = " + char1 + 1);
        System.out.println("char2 = " + ++char2); }
}
```

Here's the output of the new version of this program:

```
C:\>java App
char1 = A
char2 = B
char1 + 1 = A1
char2 = C
```

Declaring Boolean Variables

You declare Boolean variables with the `boolean` keyword. Boolean variables can take only two values in Java — `true` and `false` (not numerical values such as 0 and 1 as in other programming languages). Here's an example in which we declare and use two Boolean variables:

```
public class App {
    public static void main(String[] args) {
```

```

boolean boolean1, boolean2;
boolean1 = true;
boolean2 = false;
System.out.println("boolean1 = " + boolean1);
System.out.println("boolean2 = " + boolean2);
}
}

```

Here's the result of this code:

```

C:\>java App
boolean1 = true
boolean2 = false

```

Boolean values are usually used in tests to determine program flow. We are going to jump the gun here and give you a sneak peek into the next chapter. Here, we are using these two Boolean variables with the Java `if` statement. We test the value in `boolean1` with the `if` statement, which makes the code display the message "boolean1 is true" (if it's true) and "boolean1 is false" (otherwise):

```

public class App {
    public static void main(String[] args) {
        boolean boolean1, boolean2;
        boolean1 = true;
        boolean2 = false;
        System.out.println("boolean1 = " + boolean1);
        System.out.println("boolean2 = " + boolean2);
        if(boolean1) {System.out.println("boolean1 is true"); }
        else { System.out.println("boolean1 is false"); }
    }
}

```

Here's the new result from this code:

```

C:\>java App
boolean1 = true
boolean2 = false
boolean1 is true

```

Initializing Variables Dynamically

So far, we have been declaring variables and then assigning values to them, like this:

```

public class App {
    public static void main(String[] args) {
        int int1;
        int1 = 1;
        System.out.println("int1 = " + int1); }
}

```

However, we can combine these two steps into one by initializing a variable when we declare it, like this:

```

public class App {
    public static void main(String[] args) {
        int int1 = 1;
        System.out.println("int1 = " + int1); }
}

```

Here's how to declare and initialize multiple variables:

```

public class App {
    public static void main(String[] args) {
        int int1 = 1, int2 = 2, int3 = 3;
        System.out.println("int1 = " + int1 + ", int2 = " + int2 + ", int3 = " + int3); }
}

```

Here's the result of this program:

```

C:\>java App
int1 = 1, int2 = 2, int3 = 3

```

Up to this point, we have just assigned constant values to variables, but you can assign any expression (an *expression* is any combination of Java terms that yields a value) to a variable when that variable is declared, as long as the expression is valid at that time. For example, here we are assigning a value of 2 to `int1`, a value of 3 to `int2`, and the value of `int1` times `int2` to `int3` using the Java multiplication operator (`*`):

```
public class App {
    public static void main(String[] args) {
        int int1 = 2, int2 = 3;
        int int3 = int1 * int2;
        System.out.println("int1 = " + int1 + ", int2 = " + int2 +
            ", int3 = " + int3); }
    }
}
```

Here's what this code gives you when you run it:

```
C:\>java App
int1 = 2, int2 = 3, int3 = 6
```

Note that the Java compiler has no idea what `int1` times `int2` will be when it creates the *bytecodes* for this application. This means the actual value with which `int3` is initialized will be determined at runtime, which is why this process is called *dynamic initialization*.

As in C++, in Java, you can also intersperse your variable declarations throughout your code, as we are doing here:

```
public class App {
    public static void main(String[] args) {
        int int1 = 2, int2 = 3;
        System.out.println("int1 = " + int1 + ", int2 = " + int2);
        int int3 = int1 * int2;
        System.out.println("int3 = " + int3); }
    }
}
```

Here's the result of this code:

```
C:\>java App
int1 = 2, int2 = 3
int3 = 6
```

Conversion between Data Types

Java is a strongly typed language; and as a result, you are often faced with the situation of assigning a variable of one type to a variable of another type. The assigning of a variable of one type to a variable of another type can be done by either relying on automatic type conversion or making an explicit type cast. We'll take a look at both here.

Automatic Conversions

When you are assigning one type of data to a variable of another type, Java will convert the data to the new variable type automatically if both the following conditions are true:

- ❑ The data type and the variable types are compatible.
- ❑ The target type has a larger range than the source type.

For example, you can assign a `byte` value to an `int` variable because `byte` and `int` are compatible types, and `int` variables have a larger range than `byte` values. Therefore, no data will be lost in the type conversion. Here's an example:

```
public class App {
    public static void main(String[] args) {
        byte byte1 = 1;
        int int1;
        int1 = byte1;
        System.out.println("int1 = " + int1); }
    }
}
```

The Java compiler has no problem with this code, and it makes the type conversion automatically. Here's the result of this program:

```
C:\>java App
int1 = 1
```

Converting a data type to another with a larger range is called *widening conversion*. In widening conversions, the numeric types, such as the integer and floating-point types, are compatible with each other. On the other hand, `char` and `boolean` types are not compatible with each other or with the numeric types.

Casting to New Data Types

Performing a conversion from a data type of larger range to one that has a smaller range is called *narrowing conversion*. The Java compiler does not perform automatic narrowing conversions because there are chances of losing accuracy. To perform a narrowing conversion, it must be done with an explicit cast, which looks like this:

```
(target-data-type) value
```

For example, in this code, we are converting an integer type into a byte type:

```
public class App {
    public static void main(String[] args) {
        byte byte1; int int1 = 1;
        byte1 = (byte) int1;
        System.out.println("byte1 = " + byte1);
    }
}
```

Trying to perform a narrowing conversion would throw an exception, but performing the same with the explicit type cast would not. This is due to the fact that Java assumes that you are aware of the possibility of losing data by converting a potentially larger data with a larger data type into a smaller type. Simply put, the programmer is responsible for any loss of data while performing any narrowing conversions. For example, when you put a floating-point number into a `long`, the fractional part of the number will be truncated, and you may lose more data if the floating-point value is outside the range that a `long` can hold.

Here's the output of this code:

```
C:\>java App
byte1 = 1
```

One thing to note is that the Java compiler also automatically promotes types as needed when it evaluates expressions. For example, consider the following code, in which everything looks like it involves only bytes:

```
public class App {
    public static void main(String[] args) {
        byte byte1 = 100; byte byte2 = 100; byte byte3;
        byte3 = byte1 * byte2 / 100;
        System.out.println("byte3 = " + byte3);
    }
}
```

Here Java gives the following error:

```
C:\>javac App.java
App.java:4: error: incompatible types: possible lossy conversion from int to byte
        byte3 = byte1 * byte2 / 100;
                        ^
1 error
```

However, because Java knows that multiplying bytes can result in integer-sized values, it automatically promotes the result of the `byte1 * byte2` operation to an integer, which means you actually have to use an explicit cast here to get back to the byte type:

```
public class App {
    public static void main(String[] args) {
        byte byte1 = 100;
        byte byte2 = 100;
        byte byte3;
        byte3 = (byte) (byte1 * byte2 / 100);
        System.out.println("byte3 = " + byte3);
    }
}
```

This code compiles and runs as you'd expect—but it wouldn't without the `(byte)` cast:

```
C:\>java App
byte3 = 100
```

TIP

In general, the Java compiler promotes byte and short types to int types in expressions. If one operand is a long, the entire expression is made a long. Similarly, if one operand is a float, the whole expression is made a float; if one operand is a double, the whole expression is made a double.

Declaring One-Dimensional Arrays

As explained earlier in this chapter, arrays provide an easy way of handling a set of data by index, which is great for computers because then you can manipulate the index in your code. Java supports one-dimensional and multi-dimensional arrays, and we'll take a look at both of them here. Getting an array ready for use is a two-step process. First, you must declare the array. Here's how you declare a one-dimensional array in general:

```
type name[];
```

For example, here's how to declare an array of double values, which we'll name `accounts[]`:

```
public class App {
    public static void main(String[] args) {
        double accounts[];
        . . . }
}
```

TIP

In fact, there's another way of doing this that follows the pointer-declaration syntax in C++. You can also declare arrays with the brackets ([]) after the type, not the name of the variable, like this: `double[] accounts`.

Unlike declaring simple variables, declaring an array does not set aside memory for the array because Java isn't sure how big you want it to be yet. This means there's another step to the process—actually creating the array. See the next topic for the details.

Creating One-Dimensional Arrays

After you've declared a one-dimensional array, the next step is to actually create that array by allocating memory for it. As you'll see in the forthcoming chapter, the Java memory allocation operator is the `new` operator. Therefore, we can create and use the `new` array like this:

```
public class App {
    public static void main(String[] args) {
        double accounts[];
        accounts = new double[100];
        accounts[2] = 1335.67;
        System.out.println("Account 3 is overdue by $" + accounts[2]); }
}
```

Here, we have created an array of exactly 100 `double` values, all of which Java initializes to 0. The lower bound of all Java arrays is 0, so the first element in the array is `accounts[0]` and the last element is `accounts[99]`. If the array index is outside the range 0 to 99, Java will create a fatal error, and the program will halt. Here's the result of this program:

```
C:\>java App
Account 3 is overdue by $1335.67
```

In fact, you can combine the declaration and creation processes into one step for arrays, like this:

```
public class App {
    public static void main(String[] args) {
        double accounts[] = new double[100];
        accounts[2] = 1335.67;
        System.out.println("Account 3 is overdue by $" + accounts[2]); }
}
```

Initializing One-Dimensional Arrays

To initialize the data in one-dimensional arrays, you just place the values between curly braces, one value after the other, separated by commas, beginning with the first value in the array. Here's an example that initializes the first four elements of the `accounts[]` array with data:

```
public class App {
    public static void main(String[] args) {
        double accounts[] = {238.45, 999.33, 0, 1335.67};
        System.out.println("Account 4 is overdue by $" + accounts[3]); }
    }
}
```

Declaring Multi-Dimensional Arrays

You can declare multi-dimensional arrays in much the same way you declare one-dimensional arrays; just include a pair of square brackets for every dimension in the array:

```
type name[][][]...
```

We looked at declaring multi-dimensional arrays previously in this chapter; for example, here's how to declare a two-dimensional array with two rows and 100 columns:

```
public class App {
    public static void main(String[] args) {
        double accounts[][] = new double[2][100];
        . . . }
    }
}
```

TIP

In fact, there's another way of doing this that follows the pointer-declaration syntax in C++. You can also declare arrays with the brackets ([]) after the type, not the name of the variable, like this: double[][] accounts.

That's how it works with two-dimensional arrays—the left index specifies the row in the array and the right index specifies the column. Of course, you don't have to limit yourself to two dimensions.

Here's how you can declare a four-dimensional array:

```
public class App {
    public static void main(String[] args) {
        double accounts[][][] = new double[2][3][4][5];
        . . . }
    }
}
```

As you can see, it's as easy to declare multi-dimensional arrays as it is to declare one-dimensional arrays. Now what about actually creating the declared array? See the next topic for the details.

Creating Multi-Dimensional Arrays

You create a new multi-dimensional array by allocating memory for it with the new operator, giving the array the dimensions you want. Here's how this looks in an example:

```
public class App {
    public static void main(String[] args) {
        double accounts[][];
        accounts = new double[2][100];
        accounts[0][2] = 43.95;
        accounts[1][2] = 2385489382.06;
        System.out.println("Savings account 3 has $" + accounts[0][2]);
        System.out.println("Checking account 3 has $" + accounts[1][2]); }
    }
}
```

Here's the result of this code:

```
C:\>java App
Savings account 3 has $43.95
Checking account 3 has $2.38548938206E9
```

You can also condense the declaration and memory allocations into one step, like this:

```
public class App {
    public static void main(String[] args) {
        double accounts[][] = new double[2][100];
        accounts[0][2] = 43.95;
        accounts[1][2] = 2385489382.06;
        System.out.println("Savings account 3 has $" + accounts[0][2]);
        System.out.println("Checking account 3 has $" + accounts[1][2]); }
    }
}
```


Here's an example that creates and uses a four-dimensional array:

```
public class App {
    public static void main(String[] args) {
        double accounts[][][] = new double[2][3][4][5];
        accounts[0][1][2][3] = 43.95;

        System.out.println("Account [0][1][2][3] has $" +
            accounts[0][1][2][3]);
    }
}
```

Here's the result of this program:

```
C:\>java App
Account [0][1][2][3] has $43.95
```

Multi-dimensional arrays are actually arrays of arrays, which means that if you have a two-dimensional array (`array[][]`), you can actually treat it as an array of one-dimensional arrays, which you can access as `array[0]`, `array[1]`, `array[2]`, and so on. Here's a slightly advanced example of doing just that using a `for` loop (which you'll see more of in the next chapter) and using the `length` property (which you'll see in a few topics) to find the length of an array:

```
public class App {
    public static void main(String[] args) {
        double array[][] = {{1, 2, 3}, {3, 2, 1}, {1, 2, 3}};
        int sum = 0, total = 0;
        for(int outer_index = 0; outer_index < array.length; outer_index++){
            for(int inner_index = 0; inner_index < array[outer_index].length; inner_index++) {
                sum += array[outer_index][inner_index];
                total++;
            }
        }
        System.out.println("Average array value = " + (sum / total));
    }
}
```

Here's the result of this code:

```
C:\>java App
Average array value = 2
```

So far, all the arrays we have used have had the same number of elements for each dimension, but you don't need to set things up that way. To learn more, you can take a look at the topic after the next one. First let's look at initializing multi-dimensional arrays.

Initializing Multi-Dimensional Arrays

You can initialize multi-dimensional arrays with data when declaring them in much the same way you initialize one-dimensional arrays—just include a set of curly braces for each dimension and place the values with which you want to initialize the array in those curly braces. For example, here's how to initialize a two-dimensional array:

```
public class App {
    public static void main(String[] args) {
        double accounts[][] = { {10.11, 19.56, 4343.91, 43.95},
            { 11.23, 54.23, 543.62, 2385489382.06 } };
        System.out.println("Savings account 3 has $" + accounts[0][2]);
        System.out.println("Checking account 3 has $" +
            accounts[1][2]);
    }
}
```

Here's what running this code yields:

```
C:\>java App
Savings account 3 has $4343.91
Checking account 3 has $543.62
```

Creating Irregular Multi-Dimensional Arrays

As with many other programming languages, multi-dimensional arrays are actually arrays of arrays in Java. This means you can construct arrays as you like, as in the below example, in which each row of a two-dimensional array has a different number of elements:

```
public class App {
    public static void main(String[] args) {
```

```
double array[][] = new double[5][];
array[0] = new double[500]; array[1] = new double[400];
array[2] = new double[300]; array[3] = new double[200];
array[4] = new double[100]; array[3][3] = 1335.67;
System.out.println("Account [0][400] has $" + array[0][400]);
System.out.println("Account [1][300] has $" + array[1][300]);
System.out.println("Account [2][200] has $" + array[2][200]);
System.out.println("Account [3][100] has $" + array[3][100]);
System.out.println("Account [4][50] has $" + array[4][50]);
System.out.println("Account [3][3] has $" + array[3][3]); }
}
```

What's happening here is that we are treating each row of a two-dimensional array as a one-dimensional array by itself and creating each of those one-dimensional arrays separately.

Each dimensional array defined in this program fetches the element according to the value of index taken individually. Note here that the value of the array variable that is not defined is taking the default value; as in this case, the default value of double is 0.0.

Here's the result of this code:

```
C:\>java App
Account [0][400] has $0.0
Account [1][300] has $0.0
Account [2][200] has $0.0
Account [3][100] has $0.0
Account [4][50] has $0.0
Account [3][3] has $1335.67
```

Now, let's fetch the value that exceeds the array index as shown in the following program:

```
public class App {
    public static void main(String[] args) {
        double array[][] = new double[5][];
        array[0] = new double[500];
        array[1] = new double[400];
        array[2] = new double[300];
        array[3] = new double[200];
        array[4] = new double[100];
        array[3][3] = 1335.67;
        System.out.println("Account [0][500] has $" + array[0][500]);
        System.out.println("Account [1][500] has $" + array[1][500]);
        System.out.println("Account [2][400] has $" + array[2][400]);
        System.out.println("Account [3][300] has $" + array[3][300]);
        System.out.println("Account [4][200] has $" + array[4][200]);
        System.out.println("Account [3][3] has $" + array[3][3]); }
    }
```

The preceding program gets compiled but running the code yields, is the exceptional handling error as follows:

```
C:\>java App
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: 500
    at App.main(App.java:10)
```

Getting an the Length of an Array

It's often useful to know the length of an array, especially if you are iterating over all elements in the array in your code. To find the number of elements in an array named `array1`, you can use the term `array1.length`. Here's an example from the next chapter that uses a `for` loop to find the average student grade from a set of six grades (here, the term `grades.length` returns a value of 6):

```
public class App {
    public static void main(String[] args) {
        double grades[] = {88, 99, 73, 56, 87, 64};
        double sum, average;
        sum = 0;
        for (int loop_index = 0; loop_index < grades.length; loop_index++) {
            sum = sum + grades[loop_index]; }
    }
```

```

        average = sum / grades.length;
        System.out.println("Average grade = " + average); }
    }

```

Here's the result of this code:

```

C:\>java App
Average grade = 77.83333333333333

```

Understanding General Form of Static Import

To access static members, it is required to qualify references with the class they belong to. For example,

```
double calc = Math.cos(Math.PI * theta);
```

To understand this better, you sometimes add static members to an interface and inherit the same from that interface. This is not advised though, because a class of the static members of another class you are using is just a plain implementation detail. Whenever a class implements an interface, it becomes the part of the class's public API, and implementation detail should not interfere with other public APIs.

The static import allows unqualified access to static members. You do not need to *inherit* from the type containing static members. Alternatively, the program may import the members individually:

```
import static java.lang.Math.PI;
```

or collectively:

```
import static java.lang.Math.*;
```

The static members may be used without criterion, once imported:

```
double calc = cos(PI * theta);
```

The static import declaration mentioned previously and the normal import declaration are comparable.

You can use the static import instead of declaring local copies of constants, or utilizing inheritance (the Constant Interface Antipattern). This means that the static import is used when we require frequent access to static members from a couple of classes. However, excess use of this feature can make your program unreadable and difficult to maintain, creating problems with the namespace of all the static members you import.

Importing Static Members

In a program, you can import the static members of a class by simply importing the named package of that class. For example, in the `Circumference` class, you can use the constant `PI` defined in the `Math` class by using either its fully qualified name `Math.PI` or simply `PI` if you import the `PI` member of the `Math` package:

```
double circumference_circle() { return 2 * Math.PI*radius; }
```

This shows the need for the static member of the `Circumference` class with the name `PI` and would provide a much more accurate definition of π . The `Math` prefix to the name `PI` doesn't really add clarity to the code, and it would be better without it.

We can avoid the prefixing of `PI` to the `Math` class name by importing the `PI` member name from the `Math` class:

```

import static java.lang.Math.PI;
class Circumference {
    . . .
    double circumference_circle() { return 2*PI*radius; }
}

```

All the static members can also be imported by using the `*` notation. This import applies only to the classes defined in a named package for the static members of a class. If you require importing the name of static member of a class that you define, the definition of the class should be included in a named package. Names of static members with no names in the default package cannot be imported. The class name in the static import statement must always be qualified with its package name.

The **String** Class

In many languages, text strings are fundamental data types inherent to the specific language; but in Java, strings are handled with the `String` and `StringBuffer` classes. Let's take a look at the `String` class first.

`String` objects hold text strings that you can't change; if you want to change the actual text in the string, you should use the `StringBuffer` class instead. Here's an example in which we create a string and print it out (notice how much this code makes the `String` class look like any other simple data type):

```
public class App {
    public static void main(String[] args) {
        String s1 = "Hello from Java!";

        System.out.println(s1); }
}
```

Here's the result of this code:

```
C:\>java App
Hello from Java!
```

The `String` class is enormously powerful, with methods that enable you to convert the string into a character array, convert numbers into strings, search strings, create substrings, change the case of the string, get a string length, compare strings, and much more. The `String` class is obviously a class, not an intrinsic data type, which means you create objects of that class with constructors. A *constructor* is just like a normal method of a class, except you use it to create an object of that class. You'll get a sneak peek at the `String` class constructors here. The `String` class also has a data member you use when comparing strings (which we'll take a look at in the next chapter). This data member is shown in Table 2.2. The `String` class constructors, which you can use to create `String` objects, appear in Table 2.3, and the methods of the `String` class appear in Table 2.4.

We'll put the material you see in the tables below to use in the next few topics, in which we'll create and use `String` objects.

Table 2.2: String class field summary

Field	Does this
<code>static Comparator<String> CASE_INSENSITIVE_ORDER</code>	It yields a comparator (which you'll see more about later) that orders <code>String</code> objects, as in <code>compareToIgnoreCase</code>

Table 2.3: String class constructor summary

Constructor	Does this
<code>String()</code>	It initializes a new <code>String</code> object so that it holds an empty character sequence
<code>String(byte[] bytes)</code>	It constructs a new <code>String</code> object by converting the array of bytes by using the platform's default character encoding
<code>String(byte[] bytes, Charset charset)</code>	It constructs a new <code>String</code> by decoding the specified array of bytes using the specified charset
<code>String(byte[] ascii, int hibyte)</code>	It is deprecated because it did not properly convert bytes into characters
<code>String(byte[] bytes, int offset, int length)</code>	It constructs a new <code>String</code> object by converting the subarray of bytes using the default character encoding
<code>String(byte[] bytes, int offset, int length, Charset charset)</code>	It constructs a new <code>String</code> by decoding the specified subarray of bytes using the specified charset
<code>String(byte[] ascii, int hibyte, int offset, int count)</code>	It is deprecated because it did not properly convert bytes into characters
<code>String(byte[] bytes, int offset, int length, String charsetName)</code>	It constructs a new <code>String</code> object by converting the subarray of bytes using the specified charset
<code>String(byte[] bytes, String charsetName)</code>	It constructs a new <code>String</code> object by converting the array of bytes using the specified charset
<code>String(char[] value)</code>	It allocates a new <code>String</code> object so that it represents the sequence of characters contained in the character array argument
<code>String(char[] value, int offset, int count)</code>	It allocates a new <code>String</code> object that contains characters from a subarray of the character array argument

Table 2.3: String class constructor summary

Constructor	Does this
<code>String(int[] codePoints, int offset, int count)</code>	It allocates a new <code>String</code> object that contains characters from a subarray of the Unicode code point array argument
<code>String(String original)</code>	It initializes a new <code>String</code> object so that it represents the same sequence of characters as the argument string
<code>String(StringBuffer buffer)</code>	It allocates a new <code>String</code> object that contains the sequence of characters contained in the string buffer argument
<code>String(StringBuilder builder)</code>	It allocates a new <code>String</code> object that contains the sequence of characters contained in the string builder argument

Table 2.4: String class methods

Method	Does this
<code>char charAt(int index)</code>	It yields the character at the given index
<code>int codePointAt(int index)</code>	It yields the character (Unicode code point) at the specified index
<code>int codePointBefore(int index)</code>	It yields the character (Unicode code point) before the specified index
<code>int codePointCount(int beginIndex, int endIndex)</code>	It yields the number of Unicode code point in the specified range of this string
<code>int compareTo(String anotherString)</code>	It compares two strings lexicographically
<code>int compareToIgnoreCase(String str)</code>	It compares two strings lexicographically, ignoring case
<code>String concat(String str)</code>	It concatenates the given string to the end of this string
<code>boolean contains(CharSequence s)</code>	It returns <code>True</code> if this string contains the specified sequence of char values
<code>boolean contentEquals(CharSequence cs)</code>	It compares this string to the specified <code>CharSequence</code>
<code>boolean contentEquals(StringBuffer sb)</code>	It compares this string to the specified <code>StringBuffer</code>
<code>static String copyValueOf(char[] data)</code>	It yields a <code>String</code> object that's equivalent to the given character array
<code>static String copyValueOf(char[] data, int offset, int count)</code>	It yields a <code>String</code> object that's equivalent to the given character array by using offsets
<code>boolean endsWith(String suffix)</code>	It returns <code>True</code> if the string ends with the given suffix
<code>boolean equals(Object anObject)</code>	It compares this string to an object
<code>boolean equalsIgnoreCase(String anotherString)</code>	It compares this <code>String</code> object to another <code>String</code> object, ignoring case
<code>static String format(Locale l, String format, Object...args)</code>	It returns a formatted string using the given locale, format string, and arguments
<code>static String format(String format, Object...args)</code>	It returns a formatted string using the given format string and arguments
<code>byte[] getBytes()</code>	It converts this <code>String</code> object into bytes with the default charset, storing the result in a new byte array
<code>byte[] getBytes(Charset charset)</code>	It encodes this <code>String</code> into a byte sequence using the given charset, storing the result in a new byte array
<code>void getBytes(int srcBegin, int srcEnd,</code>	It is deprecated because it did not properly convert characters into bytes

Table 2.4: String class methods	
Method	Does this
<code>byte[] dst, int dstBegin)</code>	
<code>byte[] getBytes (String charsetName)</code>	It converts this <code>String</code> object into bytes with the given charset, storing the result in a new byte array
<code>void getChars (int srcBegin, int srcEnd, char[] dst, int dstBegin)</code>	It copies characters from this string into the destination array
<code>int hashCode()</code>	It yields a hash code for this string
<code>int indexOf(int ch)</code>	It yields the index within this string of the first occurrence of the given character
<code>int indexOf(int ch, int fromIndex)</code>	It yields the index within this string of the first occurrence of the given character, starting the search at the given index
<code>int indexOf(String str)</code>	It yields the index within this string of the first occurrence of the given substring
<code>int indexOf(String str, int fromIndex)</code>	It yields the index within this string of the first occurrence of the given substring, starting at the given index
<code>String intern()</code>	It yields a representation for the <code>String</code> object
<code>boolean isEmpty()</code>	It yields <code>True</code> if length of string is 0
<code>int lastIndexOf(int ch)</code>	It yields the index within this string of the last occurrence of the given character
<code>int lastIndexOf(int ch, int fromIndex)</code>	It yields the index within this string of the last occurrence of the given character, searching backward from the given index
<code>int lastIndexOf(String str)</code>	It yields the index within this string of the rightmost occurrence of the given substring
<code>int lastIndexOf (String str, int fromIndex)</code>	It yields the index within this string of the last occurrence of the given substring, searching backward from the given index
<code>int length()</code>	It yields the length of this string
<code>boolean matches(String regex)</code>	It returns <code>True</code> if this string matches the given regular expression
<code>int offsetByCodePoints(int index, int codePointOffset)</code>	It returns the index within this string which is offset from the given index by <code>codePointOffset</code> code points
<code>boolean regionMatches (boolean ignoreCase, int toffset, String other, int ooffset, int len)</code>	It tests whether two string regions are equal, allowing you to ignore case
<code>boolean regionMatches(int toffset, String other, int ooffset, int len)</code>	It tests whether two string regions are equal
<code>String replace(char oldChar, char newChar)</code>	It yields a new string by replacing all occurrences of <code>oldChar</code> in this string with <code>newChar</code>
<code>String replace(CharSequence target, CharSequence replacement)</code>	It yields a new string by replacing each substring of this string that matches the literal target sequence with the given literal replacement sequence
<code>String replaceAll(String regex, String replacement)</code>	It yields a new string by replacing each substring of this string that matches the given regular expression with the given replacement
<code>String replaceFirst(String regex, String replacement)</code>	It yields a new string by replacing the first substring of this string that matches the given regular expression with the given replacement
<code>String[] split(String regex)</code>	It splits this string around matches of the given regular expression
<code>String[] split(String regex, int limit)</code>	It splits this string around matches of the given regular expression and the limit given as parameter

Table 2.4: String class methods	
Method	Does this
<code>boolean startsWith(String prefix)</code>	It tests whether this string starts with the given prefix
<code>boolean startsWith(String prefix, int toffset)</code>	It tests whether this string starts with the given prefix, beginning at the given index
<code>CharSequence subSequence(int beginIndex, int endIndex)</code>	It returns a new character sequence, which is a subsequence of this sequence
<code>String substring(int beginIndex)</code>	It yields a new string, which is a substring of this string
<code>String substring(int beginIndex, int endIndex)</code>	It yields a new string, which is a substring of this string, allowing you to specify the end index
<code>char[] toCharArray()</code>	It converts this string to a new character array
<code>String toLowerCase()</code>	It converts all the characters in this <code>String</code> object to lowercase by using the rules of the default locale, which is returned by <code>Locale.getDefault()</code> method
<code>String toLowerCase(Locale locale)</code>	It converts all the characters in this <code>String</code> object to lowercase using the rules of the given <code>Locale</code>
<code>String toString()</code>	It returns the string object
<code>String toUpperCase()</code>	It converts all the characters in this <code>String</code> object to uppercase using the rules of the default locale, which is returned by <code>Locale.getDefault()</code> method
<code>String toUpperCase(Locale locale)</code>	It converts all the characters in this <code>String</code> object to uppercase using the rules of the given locale
<code>String trim()</code>	It removes white space from both ends of this string
<code>static String valueOf(boolean b)</code>	It yields the string representation of the <code>boolean</code> argument
<code>static String valueOf(char c)</code>	It yields the string representation of the <code>char</code> argument
<code>static String valueOf(char[] data)</code>	It yields the string representation of the <code>char</code> array argument
<code>static String valueOf(char[] data, int offset, int count)</code>	It yields the string representation of a specific subarray of the <code>char</code> array argument
<code>static String valueOf(double d)</code>	It yields the string representation of a <code>double</code> type argument
<code>static String valueOf(float f)</code>	It yields the string representation of a <code>float</code> type argument
<code>static String valueOf(int i)</code>	It yields the string representation of an <code>int</code> type argument
<code>static String valueOf(long l)</code>	It yields the string representation of a <code>long</code> type argument
<code>static String valueOf(Object obj)</code>	It yields the string representation of an <code>Object</code> type argument

Creating Strings

Let's take a look at some of the many ways of creating `String` objects. Here's a way you've already seen:

```
public class App { public static void main(String[] args) {
    String s1 = "Hello from Java!";
    . . . }
}
```

In fact, when you use a string literal such as "Hello from Java!" in your code, Java treats it as a `String` object. So what's really happening here is that one `String` object is assigned to another.

Of course, you can also declare a string first and then assign a value to it:

```
public class App { public static void main(String[] args) {
String s1 = "Hello from Java!";
String s2;
s2 = "Hello from Java!";
. . . }
}
```

Here's a case in which we use one of the `String` class constructors. In this case, we are just creating an empty string and then assigning data to it:

```
public class App {
    public static void main(String[] args) {
String s1 = "Hello from Java!";
String s2;
s2 = "Hello from Java!";
String s3 = new String();
s3 = "Hello from Java!";
. . . }
}
```

You can also pass a text string to the `String` class constructor directly to create a new string, like this:

```
public class App {
    public static void main(String[] args) {
String s1 = "Hello from Java!";
String s2;
s2 = "Hello from Java!";

String s3 = new String();
s3 = "Hello from Java!";
String s4 = new String("Hello from Java!");
. . . }
}
```

Other `String` class constructors are available that can take character arrays or subsets of character arrays (the `String` class knows which constructor you are using by the number and type of arguments you pass to it). You can even use the `String` class `valueOf` method to get a string representation of numeric values:

```
public class App {
    public static void main(String[] args) {
String s1 = "Hello from Java!";
String s2;
s2 = "Hello from Java!";
String s3 = new String();
s3 = "Hello from Java!";
String s4 = new String("Hello from Java!");
char c1[] = {'H', 'i', ' ', 't', 'h', 'e', 'r', 'e'};
String s5 = new String(c1);
String s6 = new String(c1, 0, 2);
String s7 = String.valueOf(c1);
System.out.println(s1);
System.out.println(s2);
System.out.println(s3);
System.out.println(s4);
System.out.println(s5);
System.out.println(s6);
System.out.println(s7);  }
}
```

TIP

To convert a string into a number, you can use the numeric wrapper classes, such as `Integer`, `Long`, `Float`, and so on, using methods such as `Integer.parseInt()` and `Long.parseLong()`.

At the end of this code, we print out all the strings we have created. Here's what appears when the program is run:

```
C:\>java App
Hello from Java!
```



```

Hello from Java!
Hello from Java!
Hello from Java!
Hi there
Hi
Hi there

```

Getting String Length

Here's an example that shows how to use the `String` class's `length()` method (note that it also shows how Java treats string literals as `String` objects by using `length()` method on a string literal):

```

public class App {
    public static void main(String[] args) {
        String s1 = "Hello from Java!";
        System.out.println("\"" + s1 + "\"" + " is " + s1.length() + " characters long");
        System.out.println("\"" + "Hello" + "\"" + " is " + "Hello".length() + " characters long");
    }
}

```

Here's the output of this program:

```

C:\>java App
"Hello from Java!" is 16 characters long
"Hello" is 5 characters long

```

Concatenating Strings

Concatenating strings means joining them together, and we have already used the `+` operator in this book to do just that. However, there's another way to concatenate strings—you can use the `String` class `concat()` method to join two strings and create a new one.

How does that look in code? Here's an example where we use both the `+` operator and the `concat()` method to create the same string:

```

public class App {
    public static void main(String[] args) {
        String s1 = "Hello";
        String s2 = s1 + " from";
        String s3 = s2 + " Java!";
        String s4 = s1.concat(" from");
        String s5 = s4.concat(" Java!");
        System.out.println(s3);
        System.out.println(s5);
    }
}

```

Here's the result of the preceding code:

```

C:\>java App
Hello from Java!
Hello from Java!

```

As you have already seen when printing out numbers, when you concatenate a numeric value with a string, the numeric value is concatenated as a string.

TIP

Note that concatenating numbers does indeed treat them as strings, so be careful—for example, `System.out.println("3 + 3 = " + 3 + 3)` displays `3 + 3 = 33`, not `3 + 3 = 6`.

Getting Characters and Substrings

The `String` class provides a number of methods that let you dissect strings into their component characters and substrings. For example, you can use the `charAt()` method to get the character at a specific position:

```

public class App {
    public static void main(String[] args) {
        String s1 = "Hello from Java!";
        char c1 = s1.charAt(0);
    }
}

```

```

    System.out.println("The first character of \"" + s1 + "\" is " + c1);
    . . . }
}

```

You can use the `toCharArray()` method to convert a `String` object into a `char` array, and you can use the `getChars()` method to get the number of characters:

```

public class App {
    public static void main(String[] args) {
        String s1 = "Hello from Java!";
        char c1 = s1.charAt(0);
        System.out.println("The first character of \"" + s1 + "\" is " + c1);
        char chars1[] = s1.toCharArray();
        System.out.println("The second character of \"" + s1 + "\" is " + chars1[1]);
        char chars2[] = new char[5];
        s1.getChars(0, 5, chars2, 0);
        System.out.println("The first five characters of \"" + s1 + "\" are " + new
            String(chars2));
        . . .
    }
}

```

You can also use the `substring()` method to create a new string that's a substring of the old one, like this:

```

public class App {
    public static void main(String[] args) {
        String s1 = "Hello from Java!";
        char c1 = s1.charAt(0);
        System.out.println("The first character of \"" + s1 + "\" is " + c1);
        char chars1[] = s1.toCharArray();
        System.out.println("The second character of \"" + s1 + "\" is " + chars1[1]);
        char chars2[] = new char[5];
        s1.getChars(0, 5, chars2, 0);
        System.out.println("The first five characters of \"" + s1 + "\" are " + new
            String(chars2));
        String s2 = s1.substring(0, 5);
        System.out.println("The first five characters of \"" + s1 + "\" are " + s2);
    }
}

```

Here's the result of running this program:

```

C:\>java App
The first character of "Hello from Java!" is H
The second character of "Hello from Java!" is e
The first five characters of "Hello from Java!" are Hello
The first five characters of "Hello from Java!" are Hello

```

Searching For and Replacing Strings

You can search strings for characters and substrings by using the `indexOf()` and `lastIndexOf()` methods. The `indexOf()` method returns the zero-based location of the first occurrence in a string of a character or substring, and `lastIndexOf()` returns the location of the last occurrence of a character or substring.

Here's an example that shows how to use `indexOf()` and `lastIndexOf()` methods:

```

public class App {
    public static void main(String[] args) {
        String s1 = "We have drawn a nice drawing.";
        System.out.println("The first occurrence of \"draw\" is " + "at location " +
            s1.indexOf("draw"));

        System.out.println("The last occurrence of \"draw\" is " + "at location " +
            s1.lastIndexOf("draw"));
        . . .
    }
}

```

The `String` class also has a `replace()` method, which lets you replace all occurrences of a single character with another single character. You might think this violates the idea that you can't change the text in a `String` object; however, this method creates an entirely new `String` object. Here's an example showing how this method works (note that we turn all occurrences of the letter *h* into the letter *f* in a text string):

```
public class App {
    public static void main(String[] args) {
        String s1 = "we have drawn a nice drawing";
        System.out.println("The first occurrence of \"draw\" is " + "at location " +
            s1.indexOf("draw"));
        System.out.println("The last occurrence of \"draw\" is " + "at location " +
            s1.lastIndexOf("draw"));
        String s2 = "Edna, you're hired!";
        System.out.println(s2.replace('h', 'f'));
    }
}
```

Here's the result of this code:

```
C:\>java App
The first occurrence of "draw" is at location 8
The last occurrence of "draw" is at location 21
Edna, you're fired!
```

Changing Case in Strings

You can use the `toLowerCase()` method to convert a string into lowercase, and you can use the `toUpperCase()` method to convert it into uppercase.

Here's how this looks in code:

```
public class App {
    public static void main(String[] args) {
        System.out.println("Hello from Java!".toLowerCase());
        System.out.println("Hello from Java!".toUpperCase());
    }
}
```

Here's the result of this program:

```
C:\>java App
hello from java!
HELLO FROM JAVA!
```

Checking for Empty String

Java SE 6 onwards, you can check for empty string by using a new method `isEmpty()` which has been introduced in `String` class. The `isEmpty()` method returns true if and only if the length of string is 0. Here's an example:

```
public class App {
    public static void main(String[] args) {
        String str="";
        System.out.println("The string is Empty?-"+str.isEmpty());
    }
}
```

Here's the result:

```
C:\>java App
The string is Empty?-true
```

Formatting Numbers in Strings

You can format numbers in strings by using the `NumberFormat` class of the `java.text` package. This class supports the `format()`, `setMinimumIntegerDigits()`, `setMinimumFractionDigits()`, `setMaximumIntegerDigits()`, and `setMaximumFractionDigits()` methods. Here's an example, using the `setMaximumFractionDigits()` method, in which we round off a double value as we format it:

```
import java.text.*;
public class App {
    public static void main(String[] args) {
        double value = 1.23456789;
        NumberFormat nf = NumberFormat.getNumberInstance();
        nf.setMaximumFractionDigits(6);
        String s = nf.format(value);
        System.out.println(s); }
}
```

Here's the result:

```
C:\>java App
1.234568
```

The **StringBuffer** Class

The `StringBuffer` class gives you much of what the `String` class offers—and something more: The ability to modify the actual string. Here's an example that uses the `StringBuffer` class `replace` method to change the contents of a `StringBuffer` object from "Hello from Java!" to "Hello to Java!":

```
public class App {
    public static void main(String[] args) {
        StringBuffer s1 = new StringBuffer("Hello from Java!");
        s1.replace(6, 10, "to");
        System.out.println(s1); }
}
```

Here's the result of running this code:

```
C:\>java App
Hello to Java!
```

You'll find the `StringBuffer` class constructors in Table 2.5 and its methods in Table 2.6. We'll put the `StringBuffer` class to work in the next few topics.

Table 2.5: StringBuffer class constructors

Constructors	Does this
<code>StringBuffer()</code>	It constructs a string buffer with no characters in it and a capacity of 16 characters
<code>StringBuffer(CharSequence seq)</code>	It constructs a string buffer that contains the same characters as the specified <code>CharSequence</code>
<code>StringBuffer(int capacity)</code>	It constructs a string buffer with no characters in it and the given capacity
<code>StringBuffer(String str)</code>	It constructs a string buffer so that it represents the same sequence of characters as the argument string

Table 2.6: StringBuffer class methods

Method	Does this
<code>StringBuffer append(boolean b)</code>	It appends the string representation of the boolean argument to the sequence
<code>StringBuffer append(char c)</code>	It appends the string representation of the char argument to the sequence
<code>StringBuffer append(char[] str)</code>	It appends the string representation of the char array argument to the sequence
<code>StringBuffer append(char[] str, int offset, int len)</code>	It appends the string representation of a subarray of the char array argument to the sequence
<code>StringBuffer append(CharSequence s)</code>	It appends the specified <code>CharSequence</code> to the sequence

Table 2.6: StringBuffer class methods

Method	Does this
<code>StringBuffer append(CharSequence s, int start, int end)</code>	It appends a subsequence of the specified <code>CharSequence</code> to the sequence
<code>StringBuffer append(double d)</code>	It appends the string representation of the <code>double</code> argument to the sequence
<code>StringBuffer append(float f)</code>	It appends the string representation of the <code>float</code> argument to the sequence
<code>StringBuffer append(int i)</code>	It appends the string representation of the <code>int</code> argument to the sequence
<code>StringBuffer append(long l)</code>	It appends the string representation of the <code>long</code> argument to the sequence
<code>StringBuffer append(Object obj)</code>	It appends the string representation of the <code>Object</code> argument to the sequence
<code>StringBuffer append(String str)</code>	It appends the string to this character sequence
<code>StringBuffer append(StringBuffer sb)</code>	It appends the given <code>StringBuffer</code> to the sequence
<code>String appendCodePoint(int codePoint)</code>	It appends the string representation of the <code>codePoint</code> argument to the sequence
<code>int capacity()</code>	It yields the current capacity of the string buffer
<code>char charAt(int index)</code>	It yields the given character of the sequence represented by the string buffer, as indicated by the <code>index</code> argument.
<code>int codePointAt(int index)</code>	It yields the character (Unicode code point) at the specified index.
<code>int codePointBefore(int index)</code>	It yields the character (Unicode code point) before the specified index.
<code>int codePointCount(int beginIndex, int endIndex)</code>	It yields the number of Unicode code points in the range of given index arguments.
<code>StringBuffer delete(int start, int end)</code>	It removes characters in a substring of the sequence.
<code>StringBuffer deleteCharAt(int index)</code>	It removes the character at the given position in the sequence.
<code>void ensureCapacity(int minimumCapacity)</code>	It ensures that the capacity of the buffer is at least equal to the given minimum.
<code>void getChars(int srcBegin, int srcEnd, char[] dst, int dstBegin)</code>	It copies characters from the sequence into the destination character array <code>dst</code> .
<code>int indexOf(String str)</code>	It returns the index of first occurrence of the given substring.
<code>int indexOf(String str, int fromIndex)</code>	It returns the index of first occurrence of the given substring, starting at the given <code>fromIndex</code> .
<code>StringBuffer insert(int offset, boolean b)</code>	It inserts the string representation of the <code>boolean</code> argument into the sequence.
<code>StringBuffer insert(int offset, char c)</code>	It inserts the string representation of the <code>char</code> argument into the sequence.
<code>StringBuffer insert(int offset, char[] str)</code>	It inserts the string representation of the <code>char</code> array argument into the sequence.
<code>StringBuffer insert(int index, char[] str, int offset, int len)</code>	It inserts the string representation of a subarray of the <code>str</code> array argument into the sequence.
<code>StringBuffer insert(int dstOffset, CharSequence s)</code>	It inserts the given <code>CharSequence</code> into the sequence.
<code>StringBuffer insert(int dstOffset, CharSequence s, int start, int end)</code>	It inserts a subsequence of the given <code>CharSequence</code> into the sequence.
<code>StringBuffer insert(int offset, double d)</code>	It inserts the string representation of the <code>double</code> argument into the

Table 2.6: StringBuffer class methods

Method	Does this
<code>double d)</code>	sequence.
<code>StringBuffer insert(int offset, float f)</code>	It inserts the string representation of the <code>float</code> argument into the sequence.
<code>StringBuffer insert(int offset, int i)</code>	It inserts the string representation of the second <code>int</code> argument into the sequence.
<code>StringBuffer insert(int offset, long l)</code>	It inserts the string representation of the <code>long</code> argument into the sequence.
<code>StringBuffer insert(int offset, Object obj)</code>	It inserts the string representation of the <code>Object</code> argument into this sequence.
<code>StringBuffer insert(int offset, String str)</code>	It inserts the string into the sequence.
<code>StringBuffer lastIndexOf(String str)</code>	It yields the index of the rightmost occurrence of the specified substring.
<code>StringBuffer lastIndexOf(String str, int fromIndex)</code>	It yields the index of the last occurrence of the specified substring.
<code>int length()</code>	It yields the length (in characters).
<code>int offsetByCodePoints(int index, int codePointOffset)</code>	It yields the index within this sequence that is offset from the given index by the <code>codePointOffset</code> code points.
<code>StringBuffer replace(int start, int end, String str)</code>	It replaces the characters in a substring of this sequence with the characters in the given string.
<code>StringBuffer reverse()</code>	It replaces the character sequence contained in this sequence with the reverse of the sequence.
<code>void setCharAt(int index, char ch)</code>	It sets the character at the given index of the sequence to <code>ch</code> .
<code>void setLength(int newLength)</code>	It sets the length of the character sequence.
<code>CharSequence subSequence(int start, int end)</code>	It yields a new sequence of characters, which is a subset of this sequence.
<code>String substring(int start)</code>	It yields a new string that contains a subsequence of characters currently contained in this sequence. The substring begins at the given index.
<code>String substring(int start, int end)</code>	It yields a new string that contains a subsequence of characters currently contained in this sequence.
<code>String toString()</code>	It yields a string representing the data in this sequence.
<code>Void trimToSize()</code>	It tries to reduce the storage space used for the sequence.

Creating StringBuffers

You can create `StringBuffer` objects using the `StringBuffer` class constructors. For example, here's how to create an empty `StringBuffer` object (which is set up with space for 16 characters, by default) and then insert some text into it:

```
public class App { public static void main(String[] args) {
    StringBuffer s1 = new StringBuffer();
    s1.insert(0, "Hello from Java!");
    System.out.println(s1);
    . . .
}
```

Here's how to initialize a new `StringBuffer` object with a string:

```
public class App {
    public static void main(String[] args) {
        StringBuffer s1 = new StringBuffer();
        s1.insert(0, "Hello from Java!");
        System.out.println(s1);
    }
}
```

```
StringBuffer s2 = new StringBuffer("Hello from Java!");
System.out.println(s2);
. . .
```

You can also create a `StringBuffer` object with a specific length, like this:

```
public class App {
    public static void main(String[] args) {
        StringBuffer s1 = new StringBuffer();
        s1.insert(0, "Hello from Java!");
        System.out.println(s1);
        StringBuffer s2 = new StringBuffer("Hello from Java!");
        System.out.println(s2);
        StringBuffer s3 = new StringBuffer();
        s3.insert(0, "Hello from Java!");
        System.out.println(s3);
    }
}
```

Here's the result of this code:

```
C:\>java App
Hello from Java!
Hello from Java!
Hello from Java!
```

Getting and Setting StringBuffer Lengths and Capacities

The `StringBuffer` class provides the `length()` method to find the length of the text in the `StringBuffer` objects. Apart from this, the class provides the `capacity()` method to identify the amount of memory space allocated for the text. You can also set the length of the text in a `StringBuffer` object with the `setLength()` method, which lets you truncate strings or extend them with null characters (that is, characters whose Unicode codes are 0).

Let's create a program to identify the string length and capacity. We will also change the length of the string:

```
public class App {
    public static void main(String[] args) {
        StringBuffer s1 = new StringBuffer("Hello from Java!");
        System.out.println("The length is " + s1.length());
        System.out.println("The allocated length is " + s1.capacity());
        s1.setLength(2000);
        System.out.println("The new length is " + s1.length());
    }
}
```

Here's what this program looks like when it's run:

```
C:\>java App
The length is 16
The allocated length is 32
The new length is 2000
```

Setting Characters in String Buffers

To read characters in a `StringBuffer` object, you can use the `charAt()` and `getChars()` methods, just as you can with `String` objects. However, in `StringBuffer` objects, you can also set individual characters by using the `setCharAt()` method.

Here's an example in which we change the text: "She had a wild look in her eyes" to "She had a mild look in her eyes" by using `setCharAt()`:

```
public class App {
    public static void main(String[] args) {
        StringBuffer s1 = new
```

```
StringBuffer("She had a wild look in her eyes");
s1.setCharAt(10, 'm');
System.out.println(s1); }
}
```

Here's the result:

```
C:\>java App
She had a mild look in her eyes
```

Appending and Inserting Using StringBuffer

"The `setCharAt()` method doesn't do it for me", the Novice Programmer says. "I really need some way of editing the text in `StringBuffer` objects as a string, not as individual characters". "OK", you say. "Use the `append()` and `insert()` methods". You can use the `append()` method to append strings to the text in a `StringBuffer` object, and you can use the `insert()` method to insert text at a particular location.

Here's an example that starts with the text "Hello", appends " Java!", and then inserts "from" into the middle of the text, using `append()` and `insert()` methods:

```
public class App {
    public static void main(String[] args) {
        StringBuffer s1 = new StringBuffer("Hello");
        s1.append(" Java!");
        System.out.println(s1);
        s1.insert(6, "from ");
        System.out.println(s1); }
}
```

Here's what this code produces:

```
C:\>java App
Hello Java!
Hello from Java!
```

Deleting Text in StringBuffer

You can delete text in a `StringBuffer` object by using the `delete()` and `deleteCharAt()` methods. For example, here's how to change the text "We are not having a good time." to "We are having a good time." with `delete()` method. (To use this method, you just specify the range of the characters you want to delete):

```
public class App {
    public static void main(String[] args) {
        StringBuffer s1 = new
        StringBuffer("we are not having a good time.");
        s1.delete(7, 11);
        System.out.println(s1); }
}
```

Here's the result:

```
C:\>java App
we are having a good time.
```

Replacing Text in StringBuffer

In fact, you've already seen how to use `replace()` method; you just specify a character range and the new text that should replace that range, like this:

```
public class App {
    public static void main(String[] args) {
        StringBuffer s1 = new StringBuffer("Hello from Java!");
        s1.replace(6, 10, "to");
        System.out.println(s1); }
}
```


Here's the result of the preceding code:

```
C:\>java App
Hello to Java!
```

Using the **Wrapper Class**

Java uses primitive data types such as `int`, `double`, `char` which does not come under the hierarchy of objects. These data types cannot be called by reference. There could be a situation in which we need to use the primitive data types as objects. Java provides a *wrapper class* in order to use the primitive data types as objects, which wraps the primitive data type within an object. So, every primitive data type has a corresponding class defined in the Java API class library.

Table 2.7 illustrates the primitive data type and the corresponding wrapper class:

Table 2.7: Wrapper classes for the primitive types	
Primitive type	Wrapper class
<code>int</code>	<code>Integer</code>
<code>short</code>	<code>Short</code>
<code>long</code>	<code>Long</code>
<code>byte</code>	<code>Byte</code>
<code>float</code>	<code>Float</code>
<code>double</code>	<code>Double</code>
<code>char</code>	<code>Character</code>
<code>boolean</code>	<code>Boolean</code>

There are different methods defined in order to return the value of an object. For example, `byteValue()` method returns the value of an object as `byte`; similarly, `doubleValue()` method will return the values as `double`. The following example is giving you the demonstration of using the wrapper class:

```
public class App {
    public static void main (String args[]) {
        Integer intwrap = new Integer(250);
        Character chrwrap = new Character('H');
        int int1 = intwrap.intValue();
        char chr1= chrwrap.charValue();
        System.out.println(int1 + " " + intwrap);
        System.out.println(chr1+" " + chrwrap); }
}
```

This program compiles and upon executing shows the result as:

```
C:\>java App
250 250
H H
```

The following lines from the preceding program are *encapsulating* a value within an object. This feature is known as *boxing*:

```
Integer intwrap = new Integer(250);
Character chrwrap = new Character('H');
```

The following lines of code are extracting the values from the type wrapper. This feature is known as *unboxing*:

```
int int1 = intwrap.intValue();
char chr1= chrwrap.charValue();
```

Autoboxing and Unboxing of Primitive Types

As only object references can be held by a collection, you are required to *box* primitive values into the corresponding wrapper class (for example, `Integer` for `int`). If you need to take out an object from a collection, you get the `Integer` value that you had put in earlier. As in the case of wrapper class, there is a need to define the object by using the `new` keyword (which allocates the memory for the variable). If this boxing and unboxing needs to be done manually every time, it would complicate your code, therefore, making it difficult to write and understand. Here, the autoboxing and unboxing comes in and is done automatically.

Let's consider the following example which demonstrates the feature of autoboxing and unboxing. Here's the whole program, which is `App.java`:

```
public class App {
    public static void main(String args[]) {
        Integer intbox1,intbox2,intbox3;
        intbox1 =250;
        int a= intbox1;
        System.out.println("The value of intbox1 is "+ intbox1);
        System.out.println("The value of int1 is "+ a);
        intbox2=500;
        System.out.println("value of intbox2 is "+ intbox2);
        intbox2=intbox2+1;
        System.out.println("value of intbox2 after increment is " + intbox2);
        intbox3= intbox2+(intbox2/5);
        System.out.println("value of intbox 3 after evaluation is "+ intbox3);
        Boolean bbox1= true;
        if (bbox1)
            System.out.println ("bbox1 contains the true value");
        Character chbox1='H';
        char chbox2= chbox1;
        System.out.println("value of chbox2 is "+ chbox2);
        Integer intbox4 = Autobox2.mthdbox(500);
        System.out.println("The value obtained from the method is "+ intbox4); }
}
class Autobox2 {
    static int mthdbox(Integer b) { return b; }
}
```

The output of the program will be:

```
C:\> java App
The value of intbox1 is 250
The value of int1 is 250
value of intbox2 is 500
value of intbox2 after increment is 501
value of intbox3 after evaluation is 601
bbox1 contains the true value
value of chbox2 is H
The value obtained from the method is 500
```

Let's discuss what is happening in the preceding program, from the previous code we have:

```
public class App {
    public static void main(String args[]) {
        Integer intbox1,intbox2,intbox3;
        intbox1 =250;
        int a= intbox1;
        System.out.println("The value of intbox1 is "+ intbox1);
        System.out.println("The value of a is "+ a);
        . . . }
}
```

The following code line in the preceding program is encapsulating a value within an object:

```
intbox1 =250;
```

This feature is known as *autoboxing* as we have not defined the object explicitly by using the `new` keyword. Again, the following line is shown as extracting the values. This feature is known as *unboxing*:

```
int a= intbox1;
```

If we'll print the results of the two variables defined, it will yield the same result as assigned. If you need an `int`, it is required that you *unbox* the `Integer` by using the `intValue()` method.

Let's take another section of the code from the program:

```
public class App {
    public static void main(String args[]) {
        Integer intbox1,intbox2,intbox3;
        intbox1 =250;
        int a= intbox1;
        System.out.println("The value of intbox1 is "+ intbox1);
        System.out.println("The value of a is "+ a);
        intbox2=500;
        System.out.println("value of intbox2 is "+ intbox2);
        intbox2=intbox2+1;
        System.out.println("value of intbox2 after increment is " + intbox2);
        intbox3= intbox2+(intbox2/5);
        System.out.println(" value of intbox 3 after evaluation is "+ intbox3);
        . . . }
    }
```

In these lines, the expression is being calculated and the result obtained will again be *reboxed*. In the line, `intbox2=500`, autoboxing is taking place.

Note the expression `intbox2=intbox2+1`. It will automatically unbox the variable and then result back into the variable by reboxing. You'll get the result of different variables used in the expression. It will display the values before evaluating and after evaluating the expression. You can see this later in this section. Let's take some more lines of the code:

```
public class App {
    public static void main(String args[]) {
        Integer intbox1,intbox2,intbox3;
        intbox1 =250;
        int a= intbox1;
        System.out.println("The value of intbox1 is "+ intbox1);
        System.out.println("The value of a is "+ a);
        intbox2=500;
        System.out.println("value of intbox2 is "+ intbox2);
        intbox2=intbox2+1;
        System.out.println("value of intbox2 after increment is " + intbox2);
        intbox3= intbox2+(intbox2/5);
        System.out.println("value of intbox 3 after evaluation is "+ intbox3);
        Boolean bbox1= true;
        if (bbox1)
            System.out.println ("bbox1 contains the true value");
        Character chbox1='H';
        char chbox2= chbox1;
        System.out.println("value of chbox2 is "+ chbox2);
        . . . }
    }
```

You can take the `boolean` and `char` variables also. The auto-unboxing takes place in the `if` condition when the condition expression gets evaluated. Similarly, the line `Character chbox1='H'` autoboxes the `char` and `char chbox2= chbox1` auto-unboxes the `char`.

At times, situations are such that you may require passing values of primitive type to a method that requires the argument to be referenced to an object. In such cases, the compiler will automatically convert those primitive values to the required class types where permissible. Let's take more lines of code:

```

public class App {
    public static void main(String args[]) {
        Integer intbox1,intbox2,intbox3;
        intbox1 =250;
        int a1= intbox1;
        System.out.println("The value of intbox1 is "+ intbox1);
        System.out.println("The value of a1 is "+ a1);
        intbox2=500;
        System.out.println("value of intbox2 is "+ intbox2);
        intbox2=intbox2+1;
        System.out.println("value of intbox2 after increment is " + intbox2);
        intbox3= intbox2+(intbox2/5);
        System.out.println("value of intbox 3 after evaluation is "+ intbox3);
        Boolean bbox1= true;
        if (bbox1)
            System.out.println ("bbox1 contains the true value");
        Character chbox1='H';
        char chbox2= chbox1;
        System.out.println("value of chbox2 is "+ chbox2);
        Integer intbox4 = Autobox2.mthdbox(500);
        System.out.println("The value obtained from the method is "+ intbox4);
    }
}
class Autobox2 {
    static int mthdbox(Integer b1) { return b1; }
}

```

Here we are defining a method `mthdbox()` with `Integer` type parameter value `b1` is being passed, which will return the `int` value in class `Autobox2`. So, in order to call the method `mthdbox()`, note the expression `Integer intbox4 = Autobox2.mthdbox(500)` defined in the `App` class. The value can only be called by explicitly mentioning the class of the function from which it is called. So, the value of `b1` will be displayed as a result. The `return` statement will auto-unbox the expression.

Alternately, you can perform the autoboxing in methods by using another way as shown here:

```

class App {
    static int mthdbox(Integer b1) { return b1; }
    public static void main(String args[]) {
        Integer intbox4 = mthdbox(500);
        System.out.println(intbox4);
    }
}

```

Here, there is no need to explicitly call the function by using the class name as it itself contains the `main` function.

Learning the Fundamentals of Varargs Methods

Varargs means variable-length arguments or a method that takes a variable number of arguments. This feature was introduced in Java SE 5. This makes it easier to create methods whose argument list widely varies.

To understand this better, we can compare this with `arrays`. If you needed to create a method that could take an arbitrary number of values, you could put the values into the array prior to invoking the method.

For example, shown here is the `MessageFormat` class that formats a message:

```

Object[] arguments = {
    new Integer(7), new Date(), "a disturbance in the Force" };
String result = MessageFormat.format("Nature caused {2} , on {1,date}
{1,time}" + "{0,number,integer}.". , arguments);

```

Basically, instead of passing multiple arguments in an array, `varargs` automates and hides the process. Additionally, it is upwardly well matched with the existing APIs. For example, the `MessageFormat.format()` method now has this declaration:

```

public static String format(String pattern, Object... arguments);

```

The three periods after `Object` indicate that you can pass an array or a sequence of arguments as final argument. However, it must be noted that `varargs` can be used only in the final argument position.

Varargs can be useful in core APIs including reflection, message formatting, and the new `print` facility. If you are an API designer, you should use them in moderation—only when the benefit is truly undeniable. Care must be taken so that the varargs method is not overloaded or it will be difficult for you to figure out which overloaded method gets called; but you can take advantage of them whenever the API offers them. Let's now understand the concept of varargs through a small example given here:

```
public class App {
    static void mthdvar(int ...int1) {
        System.out.print("Total number of arguments passed are: "
            +int1.length+ " Contents: ");
        for (int int2:int1) System.out.print(int2 +" ");
        System.out.println(); }
    public static void main (String args[]) {
        mthdvar();
        mthdvar(100);
        mthdvar(200,300,400); }
}
```

The result of the preceding code is as follows:

```
Total number of arguments passed are: 0 Contents:
Total number of arguments passed are: 1 Contents: 100
Total number of arguments passed are: 3 Contents: 200 300 400
```

So, in the preceding examples, we have defined a method `mthdvar()`, in which `int1` is operated as an array. The three dots used in the line will be known by the compiler as an array. As you can see in the `main()`, method, `mthdvar()` has been called by using three different arguments; in the first case, we have not passed any arguments.

The contents of this array arguments can be fetched by using the `for-each` statement. Let's briefly go through the `for-each` loop.

TIP

The statement for (type any-var : collections) is in general form. The word type is for the data type which should be the same as defined in the array or passed in the argument list. Collection is being considered as the type of objects whether array or some list value specifies one at a time from first element to the last element. Let's take a small example to understand this concept.

```
public class App {
    public static void main (String args[]) {
        int list1[]={10, 20, 30, 40, 50, 60, 70, 80, 90, 100};
        int total=0;
        for (int int1:list1) {
            System.out.println("Values of element is :"+int1);
            total=total+int1; }
        System.out.println("Total sum of all the values defined is: "+ total); }
}
```

Its output is displayed as follows:

```
Values of element is :10
Values of element is :20
Values of element is :30
Values of element is :40
Values of element is :50
Values of element is :60
Values of element is :70
Values of element is :80
Values of element is :90
Values of element is :100
Total sum of all the values defined is: 550
```

Overloading Varargs Methods

The overloading feature of varargs allows a set of functions that perform a similar operation to be collected under a common mnemonic name, such as `print()`. This removes the lexical intricacy of providing each function with a unique name. Alternatively, depending on the particular data in use, the function overloading is

used when you have a single task, which has two or more implementations or algorithms. By overloading the function, you can hide other available algorithms from the user.

Overloading methods should be dealt with very carefully. The compiler decides which version of an overloaded method will be called based on declared compile-time type, not runtime type. In the case in which overloaded methods have the same number of arguments, the rules regarding this decision can sometimes be a bit tricky. If there is ambiguity, the design can be simplified by:

- using different method names and avoiding overall overloading.
- retaining overloading, but ensuring each method has a distinct number of arguments.

In addition, it is recommended that varargs should not be used when a method is overloaded, since this makes it harder to determine which method is being called.

NOTE

Overloading requires methods with distinct signatures. The signature of a method includes its name and the ordered list of its argument types. All other items appearing in a method header, such as exceptions, return type, final, and synchronized, do not contribute to a method's signature.

Here's an example to understand the concept:

```
public class App {
    public static void Averg(Integer... List) {
        Integer total=0; int ListCount=0;
        for(Integer cnt:List) {
            total+=cnt;
            ListCount++; }
        System.out.println((double)total/ListCount);
        System.out.println((int)total/ListCount); }
    public static void main(String args[]) {
        Averg(7); Averg(1,2,3);
        Averg(); }
}
```

Here's the result:

```
C:\> java App
7.0
7
2.0
2
NaN
Exception in thread "main" java.lang.ArithmeticException: / by zero
    at App.Averg(App.java:8)
    at App.main(App.java:11)
```

In this scenario, we need to be careful that the implementation handles an empty vararg list correctly. Here we could have avoided the issue by adding a runtime check to make sure that the `List` was not empty, even better would be to make the first integer necessary so that the last call to `Averg` would not generate a type mismatch error. The last call to `Averg` also generates `NaN`, Not a Number, as it is the result of `0/0`.

Similar type of elements must be passed to the varargs list, although that type can be of any type or a user-defined type.

Trying to combine varargs with optional parameters is not a good idea—though it just doesn't work most of the time (type mismatch errors are common). The resulting functionality generally isn't very useful even when the code compiles without errors.

Also, once you are inside the function body, your varargs parameter is a list, completely impossible to tell apart from a list passed as a parameter. This means that passing a variable number of arguments through one function to another could be really tough.

```
Test( varargs List ) { Counter( List ); }
Counter ( varargs List ) { print( ListCount(List) ); }
main () { Test( 1, 2, 3 ) ; }
```

Here's an example to understand the concept:

```
public class App {
    public static void Test(Integer... List) { counter(List); }
    public static void counter(Integer... List) {
        int ListCount=0;
        for(Integer cnt:List) { ListCount++; }
        System.out.println(ListCount); }
    public static void main(String args[]) { Test(1,2,3); }
}
```

Here's the result:

```
C:\>java App
3
```

Fortunately, this issue doesn't come up too often.

Learning the Ambiguity in Varargs Methods

However, varargs has its own advantages, but this has not been very popular with the developer community. It is possible that the programmers may seem to be comfortable with the earlier solutions and are not so with this feature. Also, there may be some unknown problems with this new technology. To have a better view of this, we can proceed by putting various examples and scenarios as follows.

When the compiler creates code to call the `varargs` method, it may create an instance of a special class for a chosen small number of arguments. It is obvious that a special implementation of *Indexed for one element* will always be faster on modern HotSpot than a single-element array (although native iteration with `for-each` facility will require extra `Iterator` object, but that can be fixed, if needed, by recognizing *Indexed* object as a special case in `for-each` statement). Performance will certainly be better for the array-based varargs when the number of arguments is large, but it is a matter of debate that people who really need varargs and drive their inclusion into Java do not care about performance much, and it will be perfectly fine for them to deal with an extra object that implements *Indexed* interface and wraps underlying array for many argument cases.

The varargs syntax introduces an inconsistency between the behavior of parameters and local variables. At present, the parameter declarations behave exactly like local variables: passing values `a` and `b` to a method `Test(int x, String y)` is directly equivalent to `int x = a; String y = b`. Using varargs, you can pass values `a` and `b` to the method with a single parameter like this: `Trial(Object... x)`. This implies an equivalent syntax for initializing a local variable to multiple values like this: `Object... x = a, b;`. However, such syntax does not exist. The problem is not the use of syntax. It shows that the varargs feature doesn't feel well integrated into the language.

When you use overloaded methods, with varargs it is more difficult to tell which method will get called. If this is combined with autoboxing, it can be even more confusing. We have no idea which `Test` method gets called. However, this problem would not exist with *Concise Object Array Literals*.

To make varargs acceptable, it can be said that methods declared with a `"..."` parameter type needs to use it as the last argument type (like in C, C++, C#), and require that the method is less precise than any other overloading method declaring more specific argument types (like in C++). But it is difficult when reading the caller code if a more specific method will be called or if the vararg method will be used. This is determined at compile time by looking at the methods of the callee `class`; but if that `class` is extended later with more specific methods, the old caller will start using them instead of using the vararg method. This is a case where the type signature safety has not been taken care of. Many more examples are available that show the ambiguity in varargs, but we need to find those by getting accustomed to the technology and to popularize the use of the same.

Using Non-Reifiable Formal Parameters

The non-reifiable formal parameter is a type of parameter passed to the methods and classes whose type information is not completely available at runtime. Examples of the non-reifiable types are the parameterized types such as `ArrayList<Integer>` and `List<Double>`. These parameters lose their type information during the type erasure process, which is performed at the compile time. In the type erasure process, the

compiler removes all information of the type parameters and type arguments within a class or method that uses generics. For example, after the type erasure process, the `Cat<String>` type is converted to the raw `Cat` type. A raw type is the name of a class or interface without any type information. The process of type erasure is performed to make the Java applications using generics compatible with the application developed before the generics.

The type erasure process has consequences when parameters of a vararg method are of the non-reifiable type. You will understand this better with an example:

```
import java.util.*;
public class App {
    public static void faultyGenericMethod(List<String>... listAdd) {
        Object[] objectCollArray = listAdd; // valid
        objectCollArray[0] = Arrays.asList(new Integer(44));
        String s = listAdd[0].get(0); // ClassCastException thrown at this
                                   // line
    }
    public static void main (String args[]){
        List<String> test=new ArrayList<String>(5);
        faultyGenericMethod(test);
    }
}
```

Here, the first statement of the `faultyGenericMethod()` method assigns the formal list non-reifiable parameter `listAdd` to the object array `objectCollArray`. The list parameter can be assigned to the object array because the formal parameters of the Vararg method are converted to arrays during compilation. In our case, the `listAdd` parameter of the `List<String>` type is converted to `List[]`, which can be assigned to the object `[]` `objectCollArray`. The assignment of the `listAdd` parameter to the `objectCollArray` object array can cause potential heap pollution. The heap pollution is caused because the `Integer` object whose type does not match the parameterized `String` type of the `listAdd` vararg formal parameter can also be assigned to the object array `objectCollArray`. The `objectCollArray` object in turn refers to the `listAdd` formal parameter whose type is `List<String>`. As a result, the compiler does not raise any warning or error when the `List` object of `Integer` type is added to the first element of the `objectCollArray` array, as shown in the following code statement:

```
objectCollArray[0] = Arrays.asList(new Integer(44));
```

At the runtime, when the first element of the `listAdd` formal parameter, whose parameterized type is `String`, is assigned to a `String` variable `s`, the `ClassCastException` exception is thrown. The `ClassCastException` exception is thrown because the first element of the `listAdd` formal parameter is of `Integer` type and is assigned to a `String` variable `s`. The retrieval and assignment of the first element of the `listAdd` parameter is shown in the following code statement:

```
String s = listAdd[0].get(0);
```

Here are the exceptions that the above program throws upon compiling and executing:

```
C:\>javac App.java
Note: App.java uses unchecked or unsafe operations.
Note: Recompile with -Xlint:unchecked for details.
C:\>java App
Exception in thread "main" java.lang.ClassCastException: java.lang.Integer cannot be
    cast to java.lang.String
    at App.faultyGenericMethod<App.java:6>
    at App.main<App.java:11>
```

Summary

In this chapter, we have learned about the use of variables, literals, data types, arrays, strings, and Wrapper classes. After getting the conceptual knowledge on the concepts, the Immediate Solutions section has helped you to declare and initialize variables, create binary literals, and use underscores in numeric literals. You also came to know how to perform operations on arrays and strings. You have learned the use of `StringBuffer` classes and wrapper classes. The chapter ends with the discussion on autoboxing and unboxing of primitive types, and the fundamentals of varargs methods.



3

Operators, Conditionals, and Loops

<i>If you need information on:</i>	<i>See page:</i>
Operators	69
Conditionals	70
Loops	71
Operator Precedence	72
Incrementing and Decrementing (++ and --)	73
Unary NOT (~ And !)	73
Multiplication and Division (* and /)	74
Modulus (%)	74
Addition and Subtraction (+ and -)	74
Shift Operators (>>, >>>, and <<)	75
Relational Operators (>, >=, <, <=, ==, and !=)	75
Bitwise and Bitwise Logical AND, XOR, and OR (&, ^, and /)	76
Logical (&& and)	77
The if-then-else Operator (?:)	78
Assignment Operators (= and [operator]=)	79
Using the Math Class	80
Changes in the Math Class	81
Class StrictMath	81
Comparing Strings	82
The if Statement	82
The else Statement	83
Nested if Statement	83
The if-else Ladders	83

<i>If you need information on:</i>	<i>See page:</i>
The switch Statement	84
Using Strings in switch Statement	85
The while Loop	85
The do-while Loop	87
The for Loop	87
The for-each Loop	89
Supporting for-each in Your Own Class	91
Nested Loops	92
Using the break Statement	93
Using the continue Statement	93
Summary	94

In the previous chapter, we took a look at how Java handles data in basic ways. In this chapter, we'll start doing something with that data as we examine the Java operators, conditionals, and loops.

Storing a lot of data in your program is fine; but unless you do something with it, it's not of much use. Using operators, you can manipulate your data—add, subtract, divide, multiply, and more. With conditionals, you can alter a program's flow by testing the values of your data items. Using loops, you can iterate over all data items in a set, such as an array, working with each data item in succession in an easy way. These represent the next step up in programming power from the previous chapter, and we'll discuss all three of these programming topics here.

Operators

Java provides a very easy way to work with data in a program. Like most languages, it provides a number of built-in operators. For example, if one variable contains value 10 and another variable contains 20, the variables can be added using the + operator, as shown in this code:

```
public class App {
    public static void main(String[] args) {
        int operand1 = 20, operand2 = 10, sum;
        sum=operand1+operand2;
        System.out.println(operand1 + " + " +operand2 + " = " + sum); }
}
```

Here's the result of this code:

```
C:\>java App
20 + 10 = 30
```

So, what operators does Java offer? Here's a list of all of them:

- ☐ --(decrement)
- ☐ - (subtraction)
- ☐ ! (logical unary NOT)
- ☐ != (not equal to)
- ☐ % (modulus)
- ☐ %= (modulus assignment)
- ☐ & (logical AND)
- ☐ && (short-circuit AND)
- ☐ &= (bitwise AND assignment)
- ☐ * (multiplication)
- ☐ *= (multiplication assignment)
- ☐ / (division)
- ☐ /= (division assignment)
- ☐ ?: (ternary if-then-else)
- ☐ ^ (logical XOR)
- ☐ ^= (bitwise XOR assignment)
- ☐ | (logical OR)
- ☐ || (short-circuit OR)
- ☐ |= (bitwise OR assignment)
- ☐ ~ (bitwise unary NOT)
- ☐ + (addition)
- ☐ ++ (increment)
- ☐ += (addition assignment)
- ☐ < (less than)

- ❑ << (shift left)
- ❑ <<= (shift left assignment)
- ❑ <= (less than or equal to)
- ❑ = (assignment)
- ❑ -= (subtraction assignment)
- ❑ == (equal to)
- ❑ > (greater than)
- ❑ >= (greater than or equal to)
- ❑ >> (shift right)
- ❑ >>= (shift right assignment)
- ❑ >>> (shift right with zero fill)
- ❑ >>>= (shift right zero fill assignment)

You'll see these operators at work in this chapter. Operators that take one operand are called *unary* operators. Those that take two operands—for example, addition ($a + b$)—are called *binary* operators. There's even an operator, `?:`, that takes three operands—the *ternary* operator.

NOTE

Besides the built-in operators, we'll also cover the Java `Math` class in this chapter, which allows you to add a lot more math power to your programs, including exponentiation (unlike other languages, Java has no built-in exponentiation operator), logarithms, trigonometric functions, and more. `StrictMath` classes (you can find more about these classes at <http://java.sun.com/javase/8/docs/api/>) ensure that the same mathematical precision is used no matter what machine your program is running on.

Conditionals

The next step up from using simple operators is to use conditional statements, also called *branching statements*, in your code. You use conditional statements to make decisions based on the value of your data and make the flow of the program go in different directions accordingly.

Say you wanted to report on the weather; and if it's less than 80 degrees Fahrenheit, you want to print out a message that reads "It's not too hot". You can do this by checking the current temperature with a Java `if` statement that compares the value in the variable `temperature` to 80; and if that value is under 80, it prints out your message:

```
public class App {
    public static void main(String[] args) {
        int temperature = 73;
        if (temperature < 80) { System.out.println("It's not too hot."); }
    }
}
```

The `if` statement tests whether its condition (the part that appears in the parentheses) is true, which in this case is `temperature < 80`. The Java `<` (less than) relational operator is used to test whether the value in `temperature` is less than 80. Because we have set that value to 73, the `if` statement's condition is true, which means the code in the body of the `if` statement will be executed. Here's the result of this code:

```
C:\>java App
It's not too hot.
```

You can make `if` statements more complex by adding `else` clauses. These clauses must follow the `if` statement and are executed when the `if` statement condition turns out to be false. Here's an example:

```
public class App {
    public static void main(String[] args) {
        int temperature = 73;
        if (temperature < 80) { System.out.println("It's not too hot!"); }
        else { System.out.println("It's too hot!"); }
    }
}
```

As you'll see, there are other conditional statements as well, and we'll put them to work in this chapter, giving the Java syntax a thorough workout.

Loops

Loops enable you to execute a set of statements for a number of times. For example, you may want to print your name five times on the console or calculate the sum of the first hundred natural numbers; in such cases, loops are used. The loop execution depends upon a loop condition. The statements inside a loop are executed as long as the loop condition remains true. Generally, the condition involves a loop index that gets incremented or decremented every time the loop executes.

The general syntax of a `for` loop is as follows:

```
for (initialization_expression; test_conditon; iteration_expression) {
    statement;
}
```

Note that the statement that makes up the body of the `for` loop can be a compound statement, which means it can be made up of several single statements enclosed within curly braces.

You can initialize the loop index with initialization expression, provide a test condition for loop termination, and modify loop index value in iteration expression. Note that we are declaring and initializing the loop index at the same time. Isn't it similar to C++? Yes, C++ also allows you to simultaneously declare and initialize the loop index. A simple example of a `for` loop which prints `Dreamtech Press` five times is as follows:

```
public class App {
    public static void main(String[] args) {
        for (int loop_index = 1; loop_index < 6; loop_index++){
            System.out.println("Dreamtech Press");
        }
    }
}
```

This code prints `Dreamtech Press`, using `loop_index`, which starts at 1 and is steadily incremented each time through the loop by 1 till the loop test condition evaluates true. The result of this code is:

```
C:\>java App
Dreamtech Press
Dreamtech Press
Dreamtech Press
Dreamtech Press
Dreamtech Press
```

There are statements that cause an unconditional jump to other statement in the code. Such statements are called jump statements. These are mainly used to interrupt the control flow within the switch statements and loops. There are three jump statements supported by Java: `break`, `continue`, and `return`. Generally, the `break` statement allows exit from a loop or switch before the execution completes, and the `continue` statement takes control to the beginning of a loop or switch. For the `return` statement, please see the topic "Returning Values from Methods".

Here's a simple example of the `break` statement to make things clearer. In this case, the loop index is initialized with value 1 and is incremented by 1 for every execution of the loop. In the code, the test condition states that the loop should execute till the loop index is less than 7:

```
public class App {
    public static void main(String[] args) {
        for (int loop_index = 1; loop_index < 7; loop_index++){
            if(loop_index==5) break;
            System.out.println(loop_index);
            System.out.println("Dreamtech Press");
        }
    }
}
```

This code prints the value of `loop_index` until it becomes 5. Then the `break` statement takes control out of the loop and `Dreamtech Press` gets printed. The result of this code is:

```
C:\>java App
1
2
3
```

It's now time to start by using operators, conditional statements, and loops.

Operator Precedence

When a single Java statement contains a number of operators, you may also face problem. An important concern is which operator does Java execute first? For example, take a look at a program code in which we tries to add 6 and 24 and then divide the sum by 6:

```
public class App {
    public static void main(String[] args) {
        double value;
        value = 6 + 24/6;
        System.out.println("value is " + value); }
}
```

The actual result of this code is:

```
C:\>java App
value is 10.0
```

The Programmer was right; it's different from what he expected. Actually, it's the operator precedence that determines which operator is to be executed first, which second, and so on. Operator precedence rule states that the operators are executed in the order of highest to lowest precedence. The Programmer is getting 10 as output because the / operator has higher precedence than the + operator. Therefore, 24 is firstly divided by 6 and then the result is added to 6, which produces 10.

You can use parentheses to specify Java the way you want the operators to execute. Here's how the Programmer's code would now look; see the parentheses around "6 + 24," which have been used to make sure that this operation is performed first:

```
public class App {
    public static void main(String[] args) {
        double value;
        value = (6 + 24)/6;
        System.out.println("value is " + value); }
}
```

The result of this code is:

```
C:\>java App
value is 5.0
```

Table 3.1 spells out the Java operator precedence, from highest to lowest (operators on the same line have the same precedence). Note that at the very highest level of precedence, you'll find (), [] (the array operator, which you use to get data items at a specific index in an array), and . (the dot operator, which you use to specify methods and data members of objects). This means that, for example, you can always use parentheses to set the execution order of operations in Java statements.

We'll go over all these Java operators, in order of precedence, in this chapter, starting with the incrementing and decrementing operators: ++ and --.

Table 3.1: Operator precedence

Operators			
()	[]	.	
++	--	~	!
*	/	%	
+	-		
>>	>>>	<<	
>	>=	<	<=
==	!=		

Table 3.1: Operator precedence

Operators			
&			
^			
&&			
?:			
=	[operator]=		

Incrementing and Decrementing (++ and --)

The ++ operator increments its operand by 1, and the -- operator decrements its operand by 1. For example, if `value` holds 10 after you execute `value++`, `value` will hold 11.

The ++ and -- operators were introduced in C to make incrementing and decrementing values, which are very common and easy operations. In fact, they were so popular that the incrementing operator was used in C++'s name, indicating that C++ is an incremented version of C.

Here's an important point to note: ++ and -- can be used as either *postfix* operators (for example, `value++`) or *prefix* operators (for example, `++value`). Prefix operators are executed *before* the rest of the statement, whereas postfix operators are executed *after* the rest of the statement. For example, consider the following code:

```
value2 = value1++;
```

The preceding statement post increments `value1`. Note that `value2` will still contain its original value and `value1` gets incremented; the case would have been different if it was a pre-increment operator. Here's an example to make it clear:

```
public class App {
    public static void main(String[] args) {
        int value1 = 10, value2 = 10;
        System.out.println("value1 = " + value1);
        System.out.println("value2 = " + value2);
        value2 = value1++;
        System.out.println("After value2 = value1++");
        System.out.println("value1 = " + value1);
        System.out.println("value2 = " + value2);
        value2 = ++value1;
        System.out.println("Now, after value2 = ++value1");
        System.out.println("value1 = " + value1);
        System.out.println("value2 = " + value2); }
}
```

The result of this code is:

```
C:\>java App
value1 = 10
value2 = 10
After value2 = value1++
value1 = 11
value2 = 10
Now, after value2 = ++value1
value1 = 12
value2 = 12
```

Unary NOT (~ And !)

The ~ operator is the bitwise unary NOT operator, and ! is the logical unary NOT operator. The ~ operator flips all the bits of numeric arguments, and the ! operator flips true values to false and false values to true. Here's an

example in which we flip all the bits of the most positive short value, 32767, to find the most negative short value, and we also flip a Boolean value from true to false:

```
public class App {
    public static void main(String[] args) {
        short s1 = 32767;
        boolean boolean1 = true;
        System.out.println("Most negative short = " + ~s1);
        System.out.println("!true = " + !boolean1); }
}
```

Here's the result of this code:

```
C:\>java App
Most negative short = -32768
!true = false
```

If we had set `s1` to 0 and then flipped its bits with the `~` operator to 1111111111111111 in binary, Java would have displayed the resulting value as `-1`, because it uses two's complement notation for negative numbers. This means the leading bit is 1 for negative numbers and 0 for zero and positive numbers.

TIP

So, why are 1111111111111111 binary equal to -1 in a short variable? If you add it to 1, you end up with 1000000000000000 binary, a number too large for a 16-bit short, so the leading 1 is lost and you end up with 0 (in other words, $-1 + 1 = 0$).

Multiplication and Division (* and /)

You use `*` to multiply values and `/` to divide values in Java. Here's an example in which we use `*` and `/` on double values, and then we do the same thing on integer values. We perform multiplication and division on integer values to show that the fractional part of math results is truncated when you use integers. Therefore, if you want to perform a division operation and still retain precision, you probably shouldn't be using integers. Here's the code:

```
public class App {
    public static void main(String[] args) {
        double double1 = 4, double2 = 6, double3 = 5, doubleResult;
        doubleResult = double1 * double2 / double3;
        System.out.println("4 * 6 / 5 = " + doubleResult);
        int int1 = 4, int2 = 6, int3 = 5, intResult;
        intResult = int1 * int2 / int3;
        System.out.println("With integer math, 4*6/5 = " + intResult); }
}
```

Here's the result of this code:

```
C:\>java App
4 * 6 / 5 = 4.8
With integer math, 4 * 6 / 5 = 4
```

Modulus (%)

Modulus operator returns the remainder of a division operation. For example, `11%3` returns 2 and `7%2` returns 1 as output. Note that the modulus operator is especially useful when converting between bases because you can use it to successively strip digits off a number by using the modulus operator with the base you are converting into. To see how this works, take a look at the topic "The while Loop," later in this chapter. It contains a full example.

Addition and Subtraction (+ and -)

The old standby numeric operators are `+` and `-`, which you use for addition and subtraction respectively. Here's an example:

```
public class App {
    public static void main(String[] args) {
        int operand1 = 5, operand2 = 4, sum, diff;
        sum = operand1 + operand2;
        diff = operand1 - operand2;
```

```

        System.out.println(operand1 + " + " + operand2 + " = " + sum);
        System.out.println(operand1 + " - " + operand2 + " = " + diff); }
    }

```

Here's the result of this code:

```

C:\>java App
5 + 4 = 9
5 - 4 = 1

```

Shift Operators (>>, >>>, and <<)

You use the shift operators to shift all the bits of a number left or right a specified number of binary places. There are three shift operators—right shift (>>) operator, unsigned right shift (>>>), and left shift (<<) operator. Here's how you use these operators:

```

new_value = value << number_places;
new_value = value >> number_places;
new_value = value >>> number_places;

```

For example, `16 >> 2` shifts the number 16 right by two binary places, which is the same as dividing it by 4; therefore, `16 >> 2` equals 4. You commonly use the shift operators when packing binary values into an `int` or `long` as fields, because you can add a number to the `int` or `long` and then shift it left to make room for the next field of data.

Here's something you should know: The >> operator respects the sign of its operand; and because a negative value means that the leftmost bit is 1, shifting a negative number right introduces a new 1 at the left. Therefore, shifting 111111111111100, which is -4 as a `short`, turns it into 111111111111110, which is -2. Also, shifting -1, which is 11111111111111, gives you 11111111111111, which is still -1. If you really want to work with the actual bits in a number when you shift them right and not have a one added to the left when shifting negative numbers, use the unsigned right shift operator (>>>). This introduces a zero at the left, whether or not the number you are shifting is positive or negative.

Here's an example that puts the shift operators to work:

```

public class App {
    public static void main(String[] args) {
        int value = 16, negvalue = -1;
        System.out.println(value + " << 2 = " + (value << 2));
        System.out.println(value + " >> 2 = " + (value >> 2));
        System.out.println(negvalue + " >> 2 = " + (negvalue >> 2));
        System.out.println(negvalue + " >>> 22 = " + (negvalue >>> 22)); }
}

```

Here's the result of this code:

```

C:\>java App
16 << 2 = 64
16 >> 2 = 4
-1 >> 2 = -1
-1 >>> 22 = 1023

```

Relational Operators (>, >=, <, <=, ==, and !=)

Relational operators allow you generate logical conditions that can be used with conditional as well as looping statements. For example, here's how you could check to make sure the budget is greater than zero using a Java `if` statement:

```

public class App {
    public static void main(String[] args) { int budget = 1;
        if (budget < 0) { System.out.println("Uh oh."); }
        else { System.out.println("Still solvent."); }
    }
}

```

Here's the result of this code:

```

C:\>java App
Still solvent.

```

Here's a list of all the relational operators; these operators will return true if their operands match the given descriptions:

- ❑ > (greater than; for example, `operand1 > operand2` returns true if `operand1` is greater than `operand2`.)
- ❑ >= (greater than or equal to)
- ❑ < (less than)
- ❑ <= (less than or equal to)
- ❑ == (equal to)
- ❑ != (not equal to)

You can combine the logical conditions you create with a relational operator with the logical operators (see the next topic for the details).

TIP

Here's a Java pitfall to avoid: When you are creating a logical condition, bear in mind that you probably want to use == instead of =. For example, the expression `budget == 0` is true if the value in `budget` is 0, but the expression `budget = 0` assigns a value of 0 to `budget`. Be careful, because using = instead of == in logical conditions is a very common mistake.

Bitwise and Bitwise Logical AND, XOR, and OR (&, ^, and |)

The bitwise operators let you examine the individual bits of values. For example, when you use the & bitwise operator with two operands, each bit in one operand is logically *anded* with the corresponding bit in the other operand. If both bits are 1, a one appears in that place in the result; otherwise, a zero will appear in that place. For example, the Programmer could test whether the third bit of a value is set to 1 by *anding* the value with a number for which he knows only one bit—the third bit—is set to 1. If the result of the AND operation is not zero, the third bit of the original value was set. Here's how this would look in code:

```
public class App {
    public static void main(String[] args) {
        int value = 12;
        int bit3setting = value & 1 << 3;
        if (bit3setting != 0) { System.out.println("Bit 3 is set."); }
        else { System.out.println("Bit 3 is not set."); }
    }
}
```

Here's the result:

```
C:\>java App
Bit 3 is set.
```

You can find the bitwise operators in Table 3.2. In overview, here's how they work: The OR operator (|) returns 0 when both bits are 0 and returns 1 otherwise. The AND operator (&) returns 1 when both bits are 1 and returns 0 otherwise. Finally, the XOR operator (^, called the *exclusive OR*) returns 1 when one bit is 0 and the other is 1, and it returns 0 otherwise.

When the &, ^, and | operators operate on Boolean (true/false) values, they are considered bitwise *logical* operators. The bitwise logical operators work the same as the bitwise operators (substitute false for 0 and true for 1), as you can see in Table 3.3.

In overview, here's how the bitwise logical operators work: The OR operator (|) returns false when both operands are false, and it returns true otherwise. The AND operator (&) returns true when both operands are true, and it returns false otherwise. The XOR operator (^) returns true when one operand is false and one is true, and it returns false otherwise.

Table 3.2: The OR, AND, and XOR bitwise operators

x	y	x y (OR)	x & y (AND)	x ^ y (XOR)
0	0	0	0	0
1	0	1	0	1

Table 3.2: The OR, AND, and XOR bitwise operators

x	y	x y (OR)	x & y (AND)	x ^ y (XOR)
0	1	1	0	1
1	1	1	1	0

Table 3.3: The OR, AND, and XOR bitwise logical operators

x	y	x y (OR)	x & y (AND)	x ^ y (XOR)
False	False	False	False	False
True	False	True	False	True
False	True	True	False	True
True	True	True	True	False

Here's an example in which we tie two logical conditions together, displaying a message if either is true, using the `|` operator:

```
public class App {
    public static void main(String[] args) {
        int budget = 1; boolean fired = false;
        if (budget < 0 | fired == true) { System.out.println("uh oh."); }
        else { System.out.println("Still solvent."); }
    }
}
```

Here's the result:

```
C:\>java App
Still solvent.
```

In this next example, we insist that the temperature be between 60 and 90 degrees, using the `&` bitwise logical operator, before printing out a message:

```
public class App {
    public static void main(String[] args) {
        int temperature = 70;
        if (temperature < 90 & temperature > 60) {
            System.out.println("Time for a picnic.");
        }
    }
}
```

Here's the result:

```
C:\>java App
Time for a picnic.
```

As you can see, the bitwise logical operators can be very useful. Java also includes two logical operators: `&&` and `||`. We'll take a look at them next.

Logical (&& and ||)

The two logical operators you usually use in logical expressions are the logical AND (`&&`) operator and logical OR (`||`) operator. Table 3.4 shows how these operators work:

Table 3.4: The OR and AND logical operators

x	y	x y (OR)	x && y (AND)
False	False	False	False
True	False	True	False
False	True	True	False
True	true	true	true

The OR operator (`||`) returns true if any of its operands is true and false if all operands are false. The AND (`&&`) operator returns false if any of its operands is false and true only when all operands result in true.

The `&&` and `||` operators also have another interesting property—they are *short-circuit operators*, which means that if they can determine all they need to know by evaluating the left operand, they won't evaluate the right operand. This is very useful in cases such as the following, where we are testing both whether a value holds 0 and whether its reciprocal is less than 1000. If the value is indeed 0, the second part of the expression, where its reciprocal is calculated, is not executed. This way a divide-by-zero overflow error doesn't occur. Here's the code:

```
public class App {
    public static void main(String[] args) { double value = 0;
        if (value != 0 && 1 / value < 1000) {
            System.out.println("The value is not too small."); }
        else {
            System.out.println("The value is too small."); }
    }
}
```

Here's the result:

```
C:\>java App
The value is too small.
```

The logical operators differ from the bitwise logical operators because the logical operators are short-circuit operators. To see this at work, take a look at the following code, where the assignment in the `if` statement is performed when we use the `&` operator but not when we use the `&&` short-circuit operator:

```
public class App {
    public static void main(String[] args) {
        double int1 = 0, int2 = 1, int3 = 1;
        if (int1 != 0 & (int2 = 2) == 1) { }
        System.out.println("int2 = " + int2);
        if (int1 != 0 && (int3 = 2) == 1) { }
        System.out.println("int3 = " + int3); }
}
```

Here's the result:

```
C:\>java App
int2 = 2.0
int3 = 1.0
```

The if-then-else Operator (?:)

There's a Java operator that acts much like an `if-else` statement—the ternary operator (`?:`). This operator is called a *ternary operator* because it takes three operands—a condition and two values:

```
value = condition ? value1: value2;
```

If *condition* is true, the `?:` operator returns *value1*, and it returns *value2* otherwise.

In this way, the preceding statement works like the following `if` statement:

```
if (condition) { value = value1; }
else { value = value2; }
```

Here's an example where we convert an integer between 0 and 15 into a hexadecimal digit using the `?:` operator. This operator is perfect here, because we can use it to return a string made from the value itself if the value is less than 10 or a letter digit if the value is 10 or greater, like this:

```
public class App {
    public static void main(String[] args) {
        int value = 15;
        String digit, chars[] = {"a", "b", "c", "d", "e", "f"};
        digit = value < 10 ? String.valueOf(value) : chars[value - 10];
        System.out.println(value + " = 0x" + digit); }
}
```

Here's the result:

```
C:\>java App
15 = 0xf
```

Assignment Operators (= and [operator]=)

The most basic operators are the assignment operators, and we have been using these operators throughout the book already. You use the = operator to assign a variable a literal value or the value in another variable as follows:

```
public class App {
    public static void main(String[] args) {
        int value = 12;
        System.out.println("The value = " + value); }
}
```

Here's the result:

```
C:\>java App
The value = 12
```

As in C++, you can perform multiple assignments in the same statement (this works because the assignment operator itself returns the assigned value):

```
public class App {
    public static void main(String[] args) {
        int value1, value2, value3;
        value1 = value2 = value3 = 12;
        System.out.println("value1 = " + value1);
        System.out.println("value2 = " + value2);
        System.out.println("value3 = " + value3); }
}
```

Here's the result:

```
C:\>java App
value1 = 12
value2 = 12
value3 = 12
```

Also, as in C++, you can combine many operators with the assignment operator (=). For example, += is the addition assignment operator, which means `value += 2` is a shortcut for `value = value + 2`. Here's an example that puts the multiplication assignment operator to work:

```
public class App {
    public static void main(String[] args) {
        int value = 10;
        value *= 2;
        System.out.println("value * 2 = " + value); }
}
```

Here's the result:

```
C:\>java App
value * 2 = 20
```

There are quite a few combination assignment operators. Here's a list of them:

- ☐ %= (modulus assignment)
- ☐ &= (bitwise And assignment)
- ☐ *= (multiplication assignment)
- ☐ /= (division assignment)
- ☐ ^= (bitwise Xor assignment)
- ☐ |= (bitwise Or assignment)
- ☐ += (addition assignment)
- ☐ <<= (shift left assignment)
- ☐ <= (less than or equal to)
- ☐ -= (subtraction assignment)
- ☐ >>= (shift right assignment)
- ☐ >>>= (shift right zero fill assignment)

That completes the list of Java operators, but there's one more popular way of handling math in Java—the `Math` class. This class is part of the `java.lang` package (which the Java compiler imports by default). We'll take a look at this class in the next topic.

Using the **Math** Class

You can use the `java.lang.Math` class to perform any math operations. For example, here's how to solve the Programmer's problem by using the `Math.pow()` method:

```
public class App {
    public static void main(String[] args) {
        System.out.println("3 x 3 x 3 x 3 = " + Math.pow(3, 4)); }
}
```

The result is:

```
C:\>java App
3 x 3 x 3 x 3 = 81.0
```

The constants and some commonly used methods of the `java.lang.Math` class are as follows:

- ❑ `double E`—The constant *e* (2.7182818284590452354)
- ❑ `double PI`—The constant *pi* (3.14159265358979323846)
- ❑ `double sin(double a)`—Trigonometric sine
- ❑ `double cos(double a)`—Trigonometric cosine
- ❑ `double tan(double a)`—Trigonometric tangent
- ❑ `double asin(double a)`—Trigonometric arcsine
- ❑ `double acos(double a)`—Trigonometric arccosine
- ❑ `double atan(double a)`—Trigonometric arctangent
- ❑ `double atan2(double a, double b)`—Trigonometric arctangent, two-operand version
- ❑ `double exp(double a)`—Raise *e* to a power
- ❑ `double log(double a)`—Log of a value
- ❑ `double sqrt(double a)`—Square root of a value
- ❑ `double pow(double a, double b)`—Raise to a power
- ❑ `double IEEEremainder(double f1, double f2)`—IEEE remainder method
- ❑ `double ceil(double a)`—Ceiling method
- ❑ `double floor(double a)`—Floor method
- ❑ `double rint(double a)`—Random integer
- ❑ `int round(float a)`—Rounds a float
- ❑ `long round(double a)`—Rounds a double
- ❑ `double random()`—Random number
- ❑ `int abs(int a)`—Absolute value of an `int`
- ❑ `long abs(long a)`—Absolute value of a `long`
- ❑ `float abs(float a)`—Absolute value of a float
- ❑ `double abs(double a)`—Absolute value of a double
- ❑ `int min(int a, int b)`—Minimum of two `int` types
- ❑ `long min(long a, long b)`—Minimum of two `long` types
- ❑ `float min(float a, float b)`—Minimum of two float types
- ❑ `double min(double a, double b)`—Minimum of two double types
- ❑ `int max(int a, int b)`—Maximum of two `int` types
- ❑ `long max(long a, long b)`—Maximum of two `long` types

- ❑ `float max(float a, float b)`—Maximum of two float types
- ❑ `double max(double a, double b)`—Maximum of two double types

Changes in the **Math** Class

Various new methods are added to `java.lang.Math` class. They are as follows:

- ❑ `double copySign(double magnitude, double sign)`—Copy sign of second argument to first argument
- ❑ `double copySign(float magnitude, float sign)`—Copy sign of second argument to first argument
- ❑ `int getExponent(double d)`—It returns unbiased exponent used in representation
- ❑ `int getExponent(float f)`—It returns unbiased exponent used in representation
- ❑ `double nextAfter(double start, double direction)`—It returns floating-point value adjacent to first argument in the direction of second argument
- ❑ `float nextAfter(float start, double direction)`—It returns floating-point value adjacent to first argument in the direction of second argument
- ❑ `double nextUp(double d)`—Floating-point value adjacent to d in direction of positive infinity
- ❑ `float nextUp(float f)`—Floating-point value adjacent to f in direction of positive infinity
- ❑ `double scalb(double d, int scaleFactor)`—It returns $d \times 2^{\text{scaleFactor}}$
- ❑ `float scalb(float f, int scaleFactor)`—It returns $f \times 2^{\text{scaleFactor}}$

Class **StrictMath**

The `StrictMath` class consists of methods that let you perform basic numeric operations, such as elementary exponential, logarithm, square root, and trigonometric functions. The `java.lang.StrictMath` class is defined with respect to `fdlibm` version 5.3. For this class, use IEEE 754 core function version (residing in a file whose name begins with letter e) where `fdlibm` provides more than one definition for a function (such as `acos`). The methods that require `fdlibm` semantics are `sin`, `cos`, `tan`, `asin`, `acos`, `atan`, `exp`, `log`, `log10`, `cbrt`, `atan2`, `pow`, `sinh`, `cosh`, `tanh`, `hypot`, `expm1`, and `log1p`. This ensures the portability of Java program as these methods produce same result across different platforms. The description of the methods of the `StrictMath` class is as follows:

- ❑ `static double cbrt(double a)`—It returns the cube root of a double value.
- ❑ `static double cosh(double x)`—It returns the hyperbolic cosine of a double value.
- ❑ `static double expm1(double x)`—It returns $e^x - 1$.
- ❑ `static double hypot(double x, double y)`—It returns $\sqrt{x^2 + y^2}$ without intermediate overflow or underflow.
- ❑ `static double log(double a)`—It returns the natural logarithm (base e) of a double value.
- ❑ `static double log10(double a)`—It returns the base 10 logarithm of a double value.
- ❑ `static double log1p(double x)`—It returns the natural logarithm of the sum of the argument and 1.
- ❑ `static double rint(double a)`—It returns the double value that is closest in value to the argument and is equal to a mathematical integer
- ❑ `static double signum(double d)`—It returns the signum function of the argument; zero if the argument is zero; 1.0 if the argument is greater than zero; -1.0 if the argument is less than zero.
- ❑ `static float signum(float f)`—It returns the signum function of the argument; zero if the argument is zero, 1.0f if the argument is greater than zero, -1.0f if the argument is less than zero.
- ❑ `static double sinh(double x)`—It returns the hyperbolic sine of a double value.
- ❑ `static double tanh(double x)`—It returns the hyperbolic tangent of a double value
- ❑ `static double toDegrees(double angrad)`—It converts an angle measured in radians to an approximately equivalent angle measured in degrees

- ❑ `static double toRadians(double angdeg)`—It converts an angle measured in degrees to an approximately equivalent angle measured in radians.
- ❑ `static double ulp(double d)`—It returns the size of an ulp of the argument.
- ❑ `static float ulp(float f)`—It returns the size of an ulp of the argument

Comparing Strings

When you are working with the `String` class, there are some methods you can use much like operators. For example, you can use the `equals` method, `equalsIgnoreCase()` method, and `compareTo()` method as follows:

- ❑ `s1.equals(s2)`—It returns true if `s1` equals `s2`.
- ❑ `s1.equalsIgnoreCase(s2)`—It returns true if `s1` equals `s2` (ignoring case).
- ❑ `s1.compareTo(s2)`—It returns a value less than zero if `s1` is less than `s2` lexically, returns zero if `s1` equals `s2`, or returns a value greater than zero if `s1` is greater than `s2`.

Here's an example putting these methods to work:

```
public class App {
    public static void main(String[] args) {
        String s1 = "abc";
        String s2 = "abc";
        String s3 = "ABC";
        String s4 = "bcd";
        if (s1.equals(s2)) { System.out.println("s1 == s2"); }
        else { System.out.println("s1 != s2"); }
        if (s1.equalsIgnoreCase(s3)) {
            System.out.println("s1 == s3 when ignoring case"); }
        else { System.out.println("s1 != s3 when ignoring case"); }
        if (s1.compareTo(s4) < 0) { System.out.println("s1 < s2"); }
        else if (s1.compareTo(s4) == 0) { System.out.println("s1 == s2"); }
        else if (s1.compareTo(s4) > 0) { System.out.println("s1 > s2"); }
    }
}
```

Here's the result of this code:

```
C:\>java App
s1 == s2
s1 == s3 when ignoring case
s1 < s2
```

The if Statement

The `if` statement lets you execute a statement or a set of statements depending upon the result of test condition. However, if the test condition evaluates false, no statement is executed. In general, the `if` statement is used as follows:

```
if (condition)
    statement;
```

Note that *statement* can be a compound statement also, which means it can be made up of a number of statements enclosed in curly braces.

Start by checking whether the value is greater than zero; and if so, just print out the value itself, as done in the following code:

```
public class App {
    public static void main(String[] args) {
        int value = 10;
        if(value > 0)
            System.out.println("Abs(" + value + ") = " + value); }
}
```

Here's the result of this code:

```
C:\>java App
Abs(10) = 10
```

Another example for the Programmer where an `if` statement is used to execute multiple statements is as follows:

```
public class App {
    public static void main(String[] args) {
        int value = 10;
        if(value > 0) {
            System.out.println("The number was positive.");
            System.out.println("Abs(" + value + ") = " + value); }
    }
}
```

Here's the result of this code:

```
C:\>java App
The number was positive.
Abs(10) = 10
```

The `else` Statement

The `else` statement lets you specify statement(s) which gets executed when the `if` condition evaluates to false. Here's an example using the `else` statement, which can also display absolute value of negative numbers:

```
public class App {
    public static void main(String[] args) {
        int value = -10;
        if(value > 0) {
            System.out.println("Abs(" + value + ") = " + value); }
        else {
            System.out.println("Abs(" + value + ") = " + -value); }
    }
}
```

Here's the result of this code:

```
C:\>java App
Abs(-10) = 10
```

Nested `if` Statement

The nested `if` statement allows that one `if` statement can be written inside another `if` statement. Here's an example to show how a nested `if` statement works:

```
public class App {
    public static void main(String[] args) {
        double value = 2;
        if (value != 0) {
            if (value > 0)
                System.out.println("The result = " + (1 / value));
            else
                System.out.println("The result = " + (-1 / value));
        }
    }
}
```

Here's the result of this code:

```
C:\>java App
The result = 0.5
```

The `if-else` Ladders

An entire sequence of the `if-else` statements is called an `if-else` ladder. Here's an example to show how an `if-else` ladder works:

```
public class App {
    public static void main(String[] args) {
        String month = "March";
        if(month == "January")
            System.out.println("It's January");
        else if (month == "February")
            System.out.println("It's February");
        else if (month == "March")
```

```

        System.out.println("It\'s March");
    else if (month == "April")
        System.out.println("It\'s April");
    else if (month == "May")
        System.out.println("It\'s May"); }
}

```

In this case, we test the value in a string variable successively until we find a match to the current year of the calendar (we are testing only for the first five months). Here's the result of this code:

```

C:\>java App
It's March

```

Note that although it's possible to create `if-else` ladders in this way, Java actually includes a statement expressly for situations like this—the `switch` statement. We'll take a look at this statement in the next topic.

The **switch** Statement

The `switch` statement is Java's multipath branch statement; it provides the same kind of functionality as an `if-else` ladder does (see the previous topic) but in a form that's much easier to work with. Here's what the `switch` statement looks like in general:

```

switch (expression) {
    case value1:
        statement1;
        [break;]
    case value2:
        statement2;
        [break;]
    case value3:
        statement3;
        [break;]
    . . .
    default:
        default_statement;
}

```

Here, the value of *expression*, which must be of type `byte`, `char`, `short`, or `int`, is compared against the various test values in the `case` statements: *value1*, *value2*, and so on. If the expression matches one of the `case` statements, the code associated with that `case` statement—*statement1*, *statement2*, and so on—is executed. If execution reaches a `break` statement, the `switch` statement ends. Here's an example in which we display the day of the week based on a numeric value by using a `switch` statement:

```

public class App {
    public static void main(String[] args) {
        int day = 5;
        switch(day) {
            case 0:
                System.out.println("Today is Sunday."); break;
            case 1:
                System.out.println("Today is Monday."); break;
            case 2:
                System.out.println("Today is Tuesday."); break;
            case 3:
                System.out.println("Today is Wednesday."); break;
            case 4:
                System.out.println("Today is Thursday."); break;
            case 5:
                System.out.println("Today is Friday."); break;
            default:
                System.out.println("Today is Saturday."); }
        }
    }
}

```

Here's the result of this code:

```

C:\>java App
Today is Friday.

```

Try to execute the preceding example without writing the `break` statements and see what it returns as the day today. You'll find that it returns `Today is Friday` and `Today is Saturday`. This signifies that if you don't specify a `break` statement at the end of a case statement, the execution will continue with the code in the next case statement. This may prove useful when you want to execute the same code for multiple case test values, as shown in the following code:

```
public class App {
    public static void main(String[] args) {
        int temperature = 78;
        switch(temperature) {
            case 60:
            case 61:
            case 62:
            case 63:
            case 64:
                System.out.println("Very cold"); break;
            case 65:
            case 66:
            case 67:
            case 68:
            case 69:
                System.out.println("It's cool"); break;
            case 70:
            case 71:
            case 72:
            case 73:
            case 74:
            case 75:
                System.out.println("It's warm"); break;
            default:
                System.out.println("Uff, very hot!! "); } }
}
```

Here's the result of this code:

```
C:\>java App
Uff, very hot!!
```

Note that a default case executes if none of the above cases match or when no `break` statements are specified.

Using Strings in switch Statement

The `switch` statement has been upgraded in Java 7. Now, you can also use Strings inside the `switch` statements. The following is an example to make it clearer to you:

```
public class App {
    public static void main(String[] args) {
        String country="India";
        switch(country) {
            case "India": System.out.println("India");break;
            case "China": System.out.println("China");break;
            default: System.out.println("Neither India nor China");} }
}
```

Here's the result of this code:

```
C:\>java App
India
```

where the first case matches with the `switch` statement and therefore executed, and `India` gets printed.

The while Loop

A `while` loop executes a statement(s) inside a loop body as long as the loop test condition evaluates to true. In general, a `while` loop looks as follows:

```
while(condition)
    statement;
```

Note that the loop is executed not even once if the test condition evaluates to false. An example that shows how a `while` loop works is as follows:

```
public class App {
    public static void main(String[] args) { int value = 10;
        while (value > 0) {
            System.out.println("Current value = " + value--); }
    }
```

In this case, we display a value, successively subtract 1 from that value and then display the result. When the value becomes 0, the `while` loop stops because the condition (`value > 0`) has become false. In other words, the loop executes till `value` remains positive.

Here's what this `while` loop returns:

```
C:\>java App
Current value = 10
Current value = 9
Current value = 8
Current value = 7
Current value = 6
Current value = 5
Current value = 4
Current value = 3
Current value = 2
Current value = 1
```

Here's an example where a `while` loop has been used to calculate the factorial of 6:

```
public class App {
    public static void main(String[] args) {
        int value = 6, factorial = 1, temp;
        temp = value; //make a destructive copy.
        while (temp > 0) {factorial *= temp--; }
        System.out.println(value + "! = " + factorial); }
    }
```

Here's how the program calculates the factorial of 6:

```
C:\>java App
6! = 720
```

A more advanced example is as follows:

```
import java.util.*;
public class App { public static void main(String[] args) {
    int value = 32, temp = value;
    StringBuffer sb = new StringBuffer();
    Stack <String> st = new Stack <String>();
    while (temp > 0) {
        st.push(String.valueOf(temp % 16));
        temp >>= 4; }
    while(!st.empty()) {sb.append(new String((String) st.pop())); }
    System.out.println("Converting " + value + " to Hexadecimal yields 0x" + sb); }
}
```

In this case, we are converting a number into hexadecimal by successively stripping off hex digits with the modulus operator. Because the digits come out in reverse order, we are using a `while` loop to push them onto a Java *stack*, which you'll see when we discuss the collection classes. After pushing the digits onto the stack, we pop them in another `while` loop to reverse the order of the digits and create the `StringBuffer` object to display.

Here's what this program's output looks like:

```
C:\>java App
Converting 32 to Hexadecimal yields 0x20
```

Here's a fact that can come in handy: Because `null` statements are valid in Java, a `while` loop doesn't have to have a body at all. Here's an example showing a crude way of calculating an integer square root (note that all the work here takes place in the condition part of the loop):

```
public class App { public static void main(String[] args) {
    int target = 144, sqrt = 1;
```

```

        while (++(sqrt) * (sqrt) != target) ;
        System.out.println("sqrt(" + target + ") = " + sqrt); }
    }

```

Here's the result:

```

C:\>java App
sqrt(144) = 12

```

Another type of `while` loop—the `do-while` loop—is discussed in the next topic.

The **do-while** Loop

The `do-while` loop is similar to a `while` loop. It is also used to repeat a set of statements a number of times. However, the difference lies where the test condition is evaluated. A `do-while` loop tests the condition after the loop body is executed, whereas a `while` loop tests the condition before the loop body executes. Both loops terminate once the test condition evaluates to false. A `do-while` loop looks as follows:

```

do
    statement;
while(condition);

```

It can be seen in the preceding lines that a `do-while` loop will always execute at least once even if the condition evaluates to false. This is because the condition is tested after the loop body gets executed. Here's an example to show how a `do-while` loop works:

```

public class App {
    public static void main(String[] args) {
        int values[] = {1, 2, 3, 0, 5}, test, index = 0;
        do {
            test = 5 * values[index++];
            System.out.println(test);
        } while (test < 15); }
}

```

The result is as follows:

```

C:\>java App
5
10
15

```

Question arises when to use a `while` loop and when to use a `do-while` loop? The answer is whenever the situation demands that looping statements should execute *only* if the condition is true, you use a `while` loop. The following code makes it clearer, where a `do-while` loop evaluates the reciprocal of a value but can only test whether the value is a nonzero value at the end of the loop:

```

public class App {
    public static void main(String[] args) {
        double value = 0;
        do {
            System.out.println("The reciprocal = " + 1 / value);
        } while (value > 0);
    }
}

```

It's far better in this case to use a `while` loop to test for 0 first:

```

public class App {
    public static void main(String[] args) {
        double value = 0;
        while (value > 0) {
            System.out.println("The reciprocal = " + 1 / value); }
    }
}

```

The **for** Loop

The Java `for` loop is a good choice when you want to use a numeric index that you automatically increment or decrement each time through the loop, such as when you are working with an array. Here's what the `for` loop

looks like in general (note that *statement* can be a compound statement, including several single statements inside curly braces):

```
for (initialization_expression; test_conditon; iteration_expression) {
    statement
}
```

Initialization_expression lets you initialize the loop index, *test_condition* signifies a conditional expression, such as `loop_index<=5`, and *iteration_expression* increments or decrements the loop index. You can also use multiple loop indexes in a `for` loop.

An example to show how a `for` loop works is as follows:

```
public class App {
    public static void main(String[] args) {
        int loop_index;
        for (loop_index = 1; loop_index <= 5; loop_index++) {

            System.out.println("This is iteration number " + loop_index);
        }
    }
}
```

The loop index is initialized with value 1, the loop terminates when the index exceeds 5, which means the loop is executed only 5 times, and every time the loop index is incremented by 1:

Here's the result:

```
C:\>java App
This is iteration number 1
This is iteration number 2
This is iteration number 3
This is iteration number 4
This is iteration number 5
```

Here's an example that finds the average student score by looping over all the scores and summing them (note that we are actually declaring and initializing the loop index to 0 in the initialization expression):

```
public class App {
    public static void main(String[] args) {
        double grades[] = {88, 99, 73, 56, 87, 64};
        double sum, average;
        sum = 0;
        for (int loop_index = 0; loop_index < grades.length;
            loop_index++) {
            sum += grades[loop_index];
        }
        average = sum / grades.length;
        System.out.println("Average grade = " + average);
    }
}
```

Here's the result of this code:

```
C:\>java App
Average grade = 77.83333333333333
```

When you declare a loop variable (such as `loop_index` in this example), the *scope* of that variable is limited to the `for` loop body.

Note that you can use very general expressions in a `for` loop. Java lets you separate expressions in a `for` loop with a comma, as shown in the following example in which we are using two loop indexes:

```
public class App {
    public static void main(String[] args) {
        for (int loop_index = 0, doubled = 0; loop_index <= 10;
            loop_index++, doubled = 2 * loop_index) {
            System.out.println("Loop index " + loop_index + " doubled equals " + doubled);
        }
    }
}
```

Here's the result of this code:

```
C:\>java App
Loop index 0 doubled equals 0
Loop index 1 doubled equals 2
Loop index 2 doubled equals 4
Loop index 3 doubled equals 6
```

```

Loop index 4 doubled equals 8
Loop index 5 doubled equals 10
Loop index 6 doubled equals 12
Loop index 7 doubled equals 14
Loop index 8 doubled equals 16
Loop index 9 doubled equals 18
Loop index 10 doubled equals 20

```

You don't have to give a `for` loop to anybody at all—in fact, you can use a `null` statement. Here's an example in which we are summing all the elements of an array in a `for` loop without any code in its body:

```

public class App {
    public static void main(String[] args) {
        int array[] = {1, 2, 3, 4, 5}, sum = 0;
        for (int loop_index = 0; loop_index < array.length;
            sum += array[loop_index++]);
        System.out.println("The sum = " + sum); }
}

```

Here's the result of this code:

```

C:\>java App
The sum = 15

```

You can even turn a `for` loop into a `while` loop. Here's an example that's adapted from the previous factorial example in the topic ("The while Loop"):

```

public class App {
    public static void main(String[] args) {
        int value = 6, factorial = 1, temp;
        temp = value;
        for( ;temp > 0; ) { factorial *= temp--; }
        System.out.println(value + "! = " + factorial); }
}

```

Here's the result of this code:

```

C:\>java App
6! = 720

```

The `for-each` Loop

Further to the `for` loop, a new construct called the `for-each` loop is available. This enhanced construct allows you to automatically play through the elements of a collection or an array. The `for-loop` construct does the looping automatically, which would otherwise have to be done manually earlier. In its place, you can simply specify the array to iterate through and a variable to access each element, as shown here:

```

for (variable: collection)
    statement;

```

To display the output, the program shown here prints out each element passed in along the command line:

```

public class Args {
    public static void main(String args[]) {
        for (String element: args) { System.out.println(element); } }
}

```

Compile it with your compiler and then pass in some random set of command-line arguments:

```

C:\>javac Args.java
C:\>java Args Dreamtech Press Learning Solutions

```

You'll get the provided arguments printed out to standard output, one per line:

```

Dreamtech Press
Learning
Solutions

```

Although this is nothing great, it avoids the manual writing of the code where you had to update the index variable yourself:

```

for (int k=0; k < args.length; k++) { System.out.println(args[k]); }

```


You can also perform the preceding program by initializing the values in the program itself instead of giving values at runtime. Let's write another program using enumeration. This is created by using the new `enum` keyword:

```
public class App { enum Country {India,Japan,China}
    public static void main (String[] args) {
        for(Country country : Country.values()) {
            System.out.println("The country is " +country); } }
}
```

The identifiers with the `enum` statement, i.e. India, Japan and China, are called enumeration constants. Their type is the type of the enumeration in which they are declared, i.e., `Country`. Once you have created an enumeration, you can create a variable of the type:

```
for(Country country : Country.values())
```

Here, `country` is a variable of enumeration type `Country`. Each of the values in the enumeration is passed to the variable and thus display on the screen:

```
C:\> java App
The country is India
The country is Japan
The country is China
```

The construct “for (variable: collection)” doesn't add any new keywords to the language. Alternatively, it extends the meaning of the basic for loop. This for-each loop says, for each element in the “collection”, assigns it to the variable “element”, and executes the statement.

The construct works for both arrays and implementations of the `Collection` interface, like `ArrayList`, `HashSet`, and the keys from a `Map` like a `Properties` object. As you can infer from their names, the collection classes let you group elements in various ways. The collection classes also define various methods that make working with those items easier. Collection was finally added by J2SE 1.2. With the new versions of JDK, the power of Collection Framework has significantly increased and its use has been streamlined.

In fact, it has simplified the way you track a group of objects. A few simple methods are needed to use collection classes. Before moving onto the program, let's learn about the most basic collection class, `ArrayList`. An `ArrayList` is similar to an array but avoids many of the most common problems of arrays. For example, an array can't grow or shrink in size once created; therefore, you must know in advance the size of an array, whereas an `ArrayList` supports dynamic arrays that can grow or shrink in size as per the need.

The following program shows the case of using the for-each construct with a `Collection`:

```
import java.util.ArrayList;
public class ForLoopTest {
    public static void main(String[] args)
        //args the command line arguments
    {
        double[] array = {2.5, 5.2, 7.9, 4.3, 2.0, 4.1, 7.3, 0.1, 2.6};
        //leave details of the loop such as indices out of the picture
        for(double d: array) { System.out.println(d); }
        System.out.println("-----");
        ArrayList<Integer> list = new ArrayList<Integer>();
        list.add(7);
        list.add(15);
        list.add(-67);
        for(Integer number: list) { System.out.println(number); }
        System.out.println("-----");
        //works identically with autounboxing
        for(int item: list) { System.out.println(item); }
        System.out.println("-----"); }
}
```

Look at the line here as you can specify the type of elements the array list is allowed to contain. This statement creates an array list that holds `Integer` objects:

```
ArrayList<Integer> list = new ArrayList<Integer>()
```

Here's the output of this code:

```
C:\>java ForLoopTest
2.5
5.2
7.9
4.3
2.0
4.1
7.3
0.1
2.6
-----
7
15
-67
-----
7
15
-67
```

Supporting **for-each** in Your Own Class

Imagine that you are building a `Catalog` class. A `Catalog` object collects any number of `Product` objects. You store these objects by using an `ArrayList` instance variable defined in `Catalog`. The program will have to iterate with a `Catalog` object frequently through the entire list of products, to do something with each object in turn. Shown here is an example of the `Catalog` class:

```
import java.util.*;
class Catalog{
    private List<Product> products = new ArrayList<Product>();
    void add(Product product) { products.add(product); }
}
```

The `Product` class includes a method that allows you to discount the price on a product:

```
import java.math.BigDecimal;
class Product {
    private String id;
    private String name;
    private BigDecimal cost;
    Product(String id, String name, BigDecimal cost) {
        this.id = id; this.name = name;
        this.cost = cost; }
    void discount(BigDecimal percent) {
        cost = cost.multiply(new BigDecimal("1.0").subtract(percent)); }
    public String toString() { return id + ": " + name + " @ " + cost; }
}
```

A (Poor) Solution

To allow the client code to work with all products, you could create a method in the `Catalog` that returned the `ArrayList` of products:

```
List<Product> getProducts() { return products; }
```

For choosing an object in the list the client code would have to iterate through the list. However, returning a collection to a client is not feasible. If this is done, the client can modify the list in your collection, as you have given the control over the contents of that collection to the client. And hence, the client code could add or remove elements to the collection without the knowledge of the `Catalog` object. Moreover, the client will have to do more work than necessary.

Significance of **for-each**

The `for-each` loop provides a simple, consistent solution for iterating arrays, collection classes, and even your own collections. It helps eliminate much of the repetitive code that you would otherwise require. They also

eliminate the need for casting as well as other potential problems related to iteration. This is one such addition to Java that was long overdue.

The `for-each` loop can be used with both collections and arrays. The basic function is to simplify the most common form of iteration, where the iterator or index is used solely for iteration, and not for any other kind of operation, such as removing or editing an item in the collection or array. When there is a choice, the `for-each` loop should be preferred over the `for` loop, since it increases legibility. Here is an example on `for-each` loop:

```
import java.util.*;
import java.math.BigDecimal;
public final class For_each_Examples {
    public static void main(String[] aArgs) {
        List<Number> numbers = new ArrayList<Number>();
        numbers.add(new Integer(42));
        numbers.add(new Integer(-30));
        numbers.add(new BigDecimal("654.2"));
        for ( Number number : numbers ) { log(number); }
        String[] names = {"Ethan Hawke", "Julie Delpy"};
        for( String name : names ) { log("Name : " + name); }
        //iterator :
        Collection<String> words = new ArrayList<String>();
        words.add("Il ne lui faut que deux choses:");
        words.add("le");
        words.add("pain");
        words.add("et");
        words.add("le");
        words.add("temps");
        words.add("- Alfred de Vigny");
        for(Iterator<String> iter = words.iterator(); iter.hasNext();) {
            if (iter.next().length() == 4) { iter.remove(); } }
        log("Edited words: " + words.toString());
        Collection<String> stuff = new ArrayList<String> ();
        stuff.add("blah");
        for (Object thing : stuff) {
            String item = (String)thing;
            log("Thing : " + item); }
    }
    private static void log(Object aThing) { System.out.println(aThing); }
}
```

Here's the result of the code:

```
C:\>java For_each_Examples
42
-30
654.2
Name : Ethan Hawke
Name : Julie Delpy
Edited words : [Il ne lui faut que deux choses:, le, et, le, temps, - Alfred de Vigny]
Thing : blah
```

Nested Loops

Java lets you nest loops, one within another. Here's an example showing how this works (in this case, we are finding the average value of the elements in a two-dimensional array by looping over all the elements with two `for` loops):

```
public class App {
    public static void main(String[] args) {
        double array[][] = {{1, 2, 3}, {3, 2, 1}, {1, 2, 3}};
        int sum = 0, total = 0;
        for(int outer_index = 0; outer_index < array.length;
            outer_index++) {
            for(int inner_index = 0; inner_index <
                array[outer_index].length; inner_index++) {
                sum += array[outer_index][inner_index];
                total++; } }
    }
```

```
        System.out.println("Average array value = " + (sum / total)); }
    }
```

Here's the result of this code:

```
C:\>java App
Average array value = 2
```

Using the **break** Statement

Some languages include a `goto` statement that you can use to jump to any statement you want to in your code, but most languages consider `goto` too unstructured—as does Java, which does not include a `goto` statement. However, a `goto` statement lets you jump out of a loop that's no longer useful, which is a valid thing to do. Because Java doesn't have a `goto` statement you can use to do that, it supports the `break` statement for this purpose.

You can use the `break` statement to end a loop, as in this case where we are ending a loop if a sum becomes greater than 12:

```
public class App {
    public static void main(String[] args) {
        double array[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}; int sum = 0;
        for(int loop_index = 0; loop_index < array.length; loop_index++) {
            sum += array[loop_index];
            if (sum > 12) break;
            System.out.println("Looping..");
            System.out.println("The sum exceeded the maximum value");
        }
    }
}
```

Here's the result of this code:

```
C:\>java App
Looping..
Looping..
Looping..
Looping..
The sum exceeded the maximum value
```

What if you have multiple loops that you want to break? In that case, you can label the loops and indicate the one you want to break out of. Here's an example in which we break out of a double-nested loop:

```
public class App {
    public static void main(String[] args) {
        double array[][] = { {1, 2, 3}, {3, 2, 1}, {1, 2, 3} }; int sum = 0;
        outer: for(int outer_index = 0; outer_index < array.length; outer_index++) {
            for(int inner_index = 0; inner_index <
                array[outer_index].length; inner_index++) {
                sum += array[outer_index][inner_index];
                if (sum > 3) break outer;
            }
            System.out.println("I'm not going to print.");
            System.out.println("The loop has finished.");
        }
    }
}
```

Here's the result of this code:

```
C:\>java App
The loop has finished.
```

Note that if you don't use a label with the `break` statement, you'll only break out of the current loop.

Using the **continue** Statement

To skip to the next iteration of a loop, you can use the `continue` statement. Here's an example in which we are printing out reciprocals, and we want to avoid trying to print out the reciprocal of 0. If the current loop index equals 0, we skip the current iteration and move on to the next one. Here's the code:

```
public class App {
    public static void main(String[] args) {
        for(double loop_index = 5; loop_index > -5; loop_index--) {
            if (loop_index == 0) continue;
            System.out.println("The reciprocal of " + loop_index +
```

```
        " = " + (1 / loop_index)); }  
    }  
}
```

Here's the result of this code (note that this output skips over the line where the code would try to calculate the reciprocal of 0):

```
C:\>java App  
The reciprocal of 5.0 = 0.2  
The reciprocal of 4.0 = 0.25  
The reciprocal of 3.0 = 0.3333333333333333  
The reciprocal of 2.0 = 0.5  
The reciprocal of 1.0 = 1.0  
The reciprocal of -1.0 = -1.0  
The reciprocal of -2.0 = -0.5  
The reciprocal of -3.0 = -0.3333333333333333  
The reciprocal of -4.0 = -0.25
```

*Using the **return** Statement*

The `return` statement is usually used in the method that causes the control of execution to return back to the caller of the method. Let's understand the use of `continue` statement with the help of a class named `ReturnExample` as follows:

```
//To display square to a number supplied by a user at run time  
public class ReturnExample  
{  
    public static void main(String args[])  
    {  
        int input=Integer.parseInt(args[0]);  
        int result=square(input);  
        System.out.println(result);  
    }  
    public static int square(int x)  
    {  
        return x*x;  
    }  
}
```

Here a method `square()` accepts an integer value, calculates its square value, and returns that value to the `main()` method, which is its calling method. In the `main()` method, you are calling the `square()` method and passing the input that is supplied by the user at runtime. Note that you are directly calling the `square()` method since it is a static method that can only be referred inside a static context. The output of the `ReturnExample` class will be 4.

You have learned about all the categories of the control flow statements. These control flow statements are useful to a programmer for writing better and complex programs as they are designed to implement any sort of logic. Before closing this chapter, let's have a look on all that you have studied in this chapter.

Summary

In this chapter, we have discussed about the operators, conditionals, jump statements, and loops. You also understood how to work with operators, use the `Math` Class, the `StrictMath` Class, compare strings, use strings in switch statement, and use the conditional statements. You have also learned about the implementation of iterative control structure. Towards the end, you have studied about the nested loops and the jump statements.



4

Class, Object, Packages and Access Specifiers

<i>If you need information on:</i>	<i>See page:</i>
Overview of a Class	96
Working with Objects	97
Working with Methods	98
Working with Constructors	101
Exploring Packages	105
Working with Streams API	109
Working with Time API	115
Summary	115

In Java, every small or large program can only be created using a template called class. A class is used to create and define objects. Objects are the instances of a class. You can also create a class within another class and the class created inside another class is called the inner class. A class also consists of methods in which the actual code to perform any specific task is written. A method can have any logical name that expresses its functionality. For example, the logical name of the method used for adding two numbers can be sum, add, or addition. Java also provides us with constructors that are syntactically comparable to the methods. Constructors are used to initialize objects.

In this chapter, you get an overview of class and objects. The chapter discusses how work with methods. In addition, the chapter discusses about constructors in Java. It also explores package. You will also learn about Stream API, Date and Time API and how to use access specifiers.

Overview of a Class

Class is a template used for developing Java programs, which consists of variables and methods. The variables declared in a class are known as instance variables. The functionality of a class is associated with declaration and definition sections of it. The `class` keyword is used to declare a class that is defined using the various methods in it. The following code snippet shows the basic syntax of a class:

```
class <class_name> // Class declaration
{
    // declaring variables
    Data type <var_name1>;
    Data type <var_name2>;
    // starting class definition
    Method_name (parameters)
    {
        //Method body
    }
    // End of class definition
}
```

In the preceding code snippet, only two variables have been declared; however, more variables can be declared as per the requirement. Similarly, the number of methods that are defined in a class also depends on the requirement. A Java class must contain a `main()` method because the `main()` method is the starting point of the execution of the Java class.

Let's develop a simple class named `sample` to understand the creation of a class. Listing 4.1 shows the code of the sample class:

Listing 4.1: Showing the Code of the `sample.java` File

```
class sample {
    public static void main (String args[])
    {
        // variable declaration
        String fname;
        String sname;
        int age;

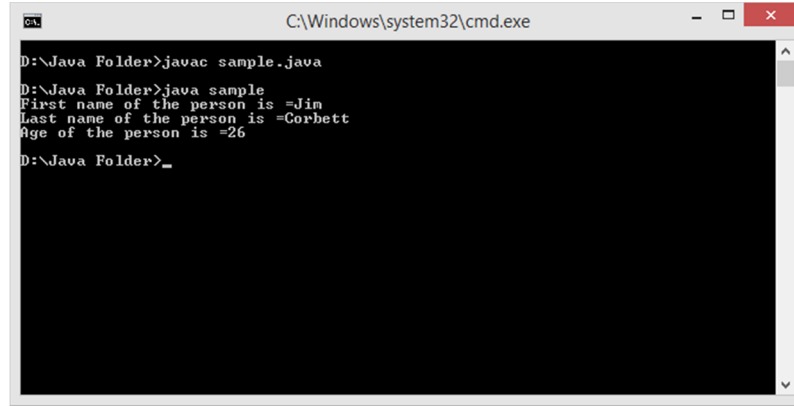
        // Assigning values to the variables declared
        fname = "Jim";
        sname = "Corbett";
        age = 26;

        //Displaying the values assigned
        System.out.println ("First name of the person is " + fname);
        System.out.println ("Last name of the person is " + sname);
        System.out.println ("Age of the person is " + age);
    }
}
```

In Listing 4.1, a class named `sample` is created. This class contains the `main()` method, which is the very first method called during the execution of a Java class, which consists of only one `main()` method that is

responsible for calling other methods used in the class. Further, you have declared the `fname`, `sname`, and `age` variables and then assigned values to them. Finally, the values assigned to the variables are displayed.

Figure 4.1 shows the output of the `sample` class:



```

C:\Windows\system32\cmd.exe
D:\Java Folder>javac sample.java
D:\Java Folder>java sample
First name of the person is =Jim
Last name of the person is =Corbett
Age of the person is =26
D:\Java Folder>_

```

Figure 4.1: Output of listing 4.1

It can be seen in Figure 4.1 that the values assigned to the variables are displayed along with the statements specified within the `System.out.println()` method.

You have learned how to create a class. Now, let's discuss about objects, which are run time representation of a class.

Working with Objects

Instances of a class are known as objects, and the process of creating objects of a class is known as instantiation. When you create a class instance, the class works as a data type of the instance. For example, in the `sample` class created in the previous section, you have declared variables `fname` and `sname` of the `String` class, which act as a data type.

To create an object, you have to declare a variable of the class type to store the object reference. For example, if you have to create an object of the class `sample`, then you have to first declare a variable of the `sample` class type. You can create the variable of the `sample` class using the following code statement:

```
sample info;
```

Once the variable of the `sample` class is created, you can create the object of the class. To create the `sample` class object, a `new` operator is used, which dynamically allocates memory for an object and returns a reference for the object. This reference is then stored in the variable declared using the preceding code statement. The following code statement shows how to create an object of the `sample` class:

```
info = new sample ();
```

You can create a variable and an object in a single statement, as shown in the following code statement:

```
sample info = new sample();
```

Every object of a class has its own copy of the variables declared in the class definition. Let's rework on the `sample` class and create and use its objects. Listing 4.2 shows the code of the `sample` class:

Listing 4.2: Showing the Code of the `sample` Class

```

class sample
{
    public void disp()
    {
        // variable declaration
        String fname;
        String sname;
        int age;
    }
}

```



```

// Assigning values to the variables declared
fname = "Jim";
sname = "Corbett";
age = 26;

//Displaying the values assigned
System.out.println ("First name of the person is =" + fname);
System.out.println ("Last name of the person is =" + sname);
System.out.println ("Age of the person is =" + age);
}
public static void main (String args[])
{
    sample info;           // declaring variable of class type
    info = new sample();    // creating object and storing its reference in variable
    info.disp();            // calling the disp method through object reference
                           // variable
}
}

```

In Listing 4.2, you have created a method called `disp`, which contains the code to display the values of the variables.

In the `main()` method, first you have declared a variable called `info` of the `sample` class type, which is then followed by creating object of the `sample` class.

You have also used the object of the `sample` class to call the `disp()` method. Figure 4.2 shows the output of Listing 4.2:

```

C:\Windows\system32\cmd.exe
D:\Java Folder>javac sample.java
D:\Java Folder>java sample
First name of the person is =Jim
Last name of the person is =Corbett
Age of the person is =26
D:\Java Folder>

```

Figure 4.2: Output of Listing 4.2

Working with Methods

Methods are considered as functions that are used to perform some specific task in an application. A method can have any name, such as `disp`, `vol`, and so on. Depending on the requirement of the class, many methods can be created in a class. The format to create a method is shown in the following syntax:

```

<access_modifier> <return_type> <method_name> [para_list]
{
    // method body;
}

```

where:

- `access_modifier`—Sets the visibility of the methods of a class to the members of the same or different class declared in the same or different packages. An access modifier can be `public`, `private`, or `protected`. Access modifiers will be discussed in detail in the Exploring Packages section of this chapter.

- ❑ `return_type`—Represents the type of values that a method is returning. If the method does not return any value, then its return type is `void`. A method returns the value to the calling routine through the `return` statement.
- ❑ `method_name`—Signifies the name of the method.
- ❑ `para_list`—Represents a list of parameters that are passed to a method. It may happen that there is no need to use parameters in a method. In such case, `para_list` is not used and the brackets remain empty.

In the preceding syntax, the body of the method is written between the beginning and ending curly braces. Let's create a class named `sum` to create and use methods.

Listing 4.3 shows the code of the `sum` class:

Listing 4.3: Showing the Code for Creating and Using Methods in the `sum` Class

```
class sum
{
    public void no_para()
    {
        int j=5;
        int k=6;
        int s= j+k;
        System.out.println("sum of the two numbers in a method without parameters are = "
            + s);
    }
    public void para_y( int a, int b )
    {
        int c, d;
        c=a;
        d=b;
        int sum = c+d;
        System.out.println("Sum of the two numbers passed as parameters are = " + sum);
    }
    public static void main(String args[])
    {
        sum get_sum = new sum();
        sum get_sum1 = new sum();
        get_sum.no_para(); // Invoking the no_para method
        get_sum1.para_y(15, 12); // invoking the para_y method
    }
}
```

In Listing 4.3, you have created a class named `sum` in which two methods are declared. The first method is `no_para()`, which takes no parameter, and the second one is `para_y()` in which two parameters are passed.

You have used both `no_para()` and `para_y()` methods to calculate the sum of two numbers. In the method `no_para()`, you have declared two variables and then initialized the variables with some values. Moreover, the `+` operator is used to calculate their sum, which is then assigned to a third variable. The value of `sum` is displayed using the `System.out.println()` method.

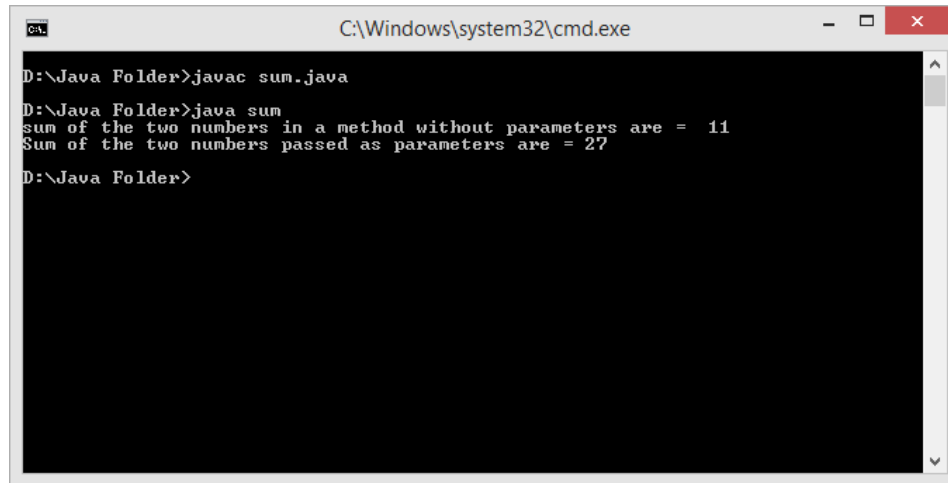
In the `para_y()` method, the same functioning as in the `no_para()` method is provided. The only difference between the methods is that two parameters `a` and `b` of the type `int` are defined in the `para_y()` method, while, two variables `c` and `d` of type `int` are declared in the `para_y()` method. In the variables `c` and `d`, the values retrieved in the `a` and `b` parameters are assigned.

Finally, you have used the `main()` method to use the methods created in the `sum` class. In the `main()` method, you have created two objects of the `sum` class. Then, these objects are used to invoke the `no_para()` and `para_y()` methods. The `no_para()` method is invoked with no parameter values because it is a non-parameterized method, while two parameters are passed in the `para_y()` method because it is a parameterized method.

NOTE

In Listing 4.3, two methods having different names are used. However, same names can also be used for various methods in a class. The process of using the same name with different parameters for various methods in a class is called method overloading.

Figure 4.3 displays the output of the `sum` class:



```

C:\Windows\system32\cmd.exe
D:\Java Folder>javac sum.java
D:\Java Folder>java sum
sum of the two numbers in a method without parameters are = 11
Sum of the two numbers passed as parameters are = 27
D:\Java Folder>

```

Figure 4.3: Output of listing 4.3

Defining Default Methods

A method that is declared by using the `default` keyword is known as the default method. It is accessible by all implementation classes of that interface. These classes can override the implementation of the default method and provide a different implementation. The concept of default method is new, and it is an important feature included in Java SE 8.0. The default method also allows adding new functionality to interfaces present in libraries and makes sure that the new code written for these interfaces should be compatible with their older versions. Listing 4.4 shows an example in which a default method is created with the name `div` and is called using the object of the interface:

Listing 4.4: Showing the Code for default Methods

```

//Functional interface with a default and an abstract methods
interface FuncInterface {
    //abstract method
    void add(int num1, int num2);
    //default method
    default void div(int num1, int num2) {
        int division = num1/num2;
        System.out.println("Result of Division:" +division);
    }
}

class ImplClass implements FuncInterface {
    public void add(int num1, int num2) {
        int sum = num1 + num2;
        System.out.println("Result of Addition:" +sum);
    }
}

class DefaultDemo {
    public static void main(String args[]) {
        FuncInterface fi = new ImplClass();
        fi.add(2,3);
    }
}

```

```

    fi.div(15,3);
}
}

```

Figure 4.4 displays the output of the `DefaultDemo` class:

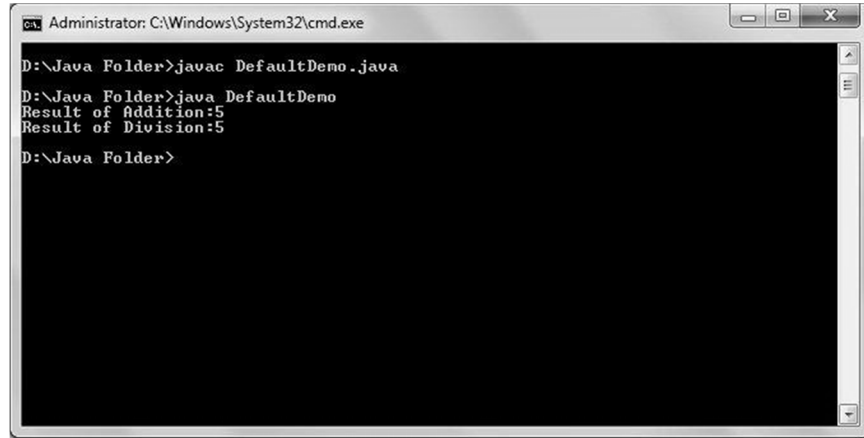


Figure 4.4: Output of listing 4.4

Working with Constructors

Constructors are the methods having the same name as the class name and no return type. For example, if a class has the name `demo`, then the constructor will also have the name `demo`. The syntax of the constructors is similar to the syntax of the methods and is used for initializing objects. A constructor initializes the object as soon as the object is created. Some important points that must be remembered while working with constructors are as follows:

- ❑ The name of the constructor should be same as that of a class.
- ❑ There is no return type of a constructor because implicitly the return type of the constructor is the class itself.
- ❑ The first statement of a constructor always includes either an invocation to other constructor of the same class or a call to the constructor of its superclass. If you do not call other constructor or superclass constructor explicitly using the `super` keyword, then the compiler implicitly calls the default constructor, or the constructor with no parameters, of its superclass.

Let's learn about the types of constructors and how to create and use the constructors under the topics:

- ❑ Using default constructors
- ❑ Using parameterized constructors

Using Default Constructor

A constructor without any parameter is called a default constructor. A default constructor is automatically created by Java. When no constructor is created explicitly, Java first implicitly creates a constructor without parameter and then calls the default parent constructor. The default parent constructor then initializes the instance variables to default values, that is, zero for numeric data types, null for object reference and string type, and false for boolean.

Let's reconsider the `sum` class to understand the default constructor. In the `sum` class, two objects named `get_sum` and `get_sum1` are created; however, you have not created any constructor to initialize the instance variable for these objects. A default constructor is created by the Java compiler and is used to initialize the instance variables of the `sum` objects.

Default constructors are used implicitly by Java when no other constructor is available. If the default or a constructor without a parameter is created explicitly in a class, then Java uses the constructor to initialize the instance variable of the class objects. Let's create a default constructor in a class named `def_cnstr`. Listing 4.5 shows the code of the `def_cnstr` class:

Listing 4.5: Showing the Code of the `def_cnstr` Class

```
class def_cnstr
{
    //declaring instance variables
    int x, y;
    def_cnstr () // default constructor
    {
        x=10;
        y=5;
    }
    public void mul ()
    {
        int m=x*y;
        System.out.println ("Value after multiplication of two numbers is = " + m);
    }
    public static void main (String args[])
    {
        def_cnstr chk = new def_cnstr();
        chk.mul();
    }
}
```

In Listing 4.5, you have created a class named `def_cnstr` to show how to create and use a default constructor in a class. In the `def_cnstr` class, you have declared two instance variables `x` and `y` and a parameterless constructor in which the variables `x` and `y` are initialized. In addition, you have also created a method called `mul` to calculate the product of the instance variable that is initialized in the constructor. An object called `chk` is created in the `def_cnstr` class, which is then used to invoke the `mul` method.

Figure 4.5 shows the output of the `def_cnstr` class:

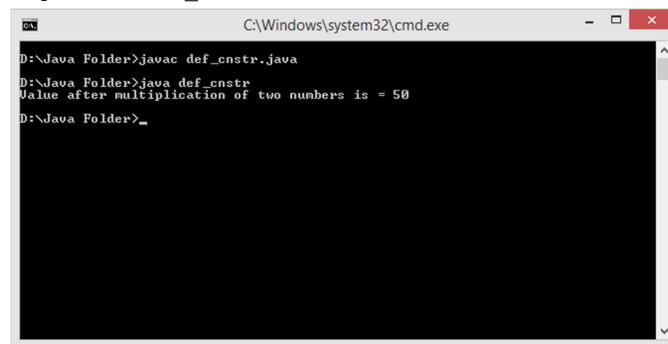


Figure 4.5: Output of Listing 4.5

Using Parameterized Constructors

The constructors having parameters are known as parameterized constructors. In parameterized constructors, the instance variable is initialized automatically at run time according to the values passed to the parameters during object creation. Parameterized constructor allows us to initialize the instance variable with different values, whereas in case of default constructor, the values will remain same for all objects. The default constructor is used in cases where the instance variables are initialized with some fixed value and that value remains the same for all the objects of the class. In case of the parameterized constructor, you do not initialize the instance variables with some fixed values; rather, use parameters to initialize the instance variables. Parameters used in the parameterized constructor receive the values passed during the object creation and assign these values to the

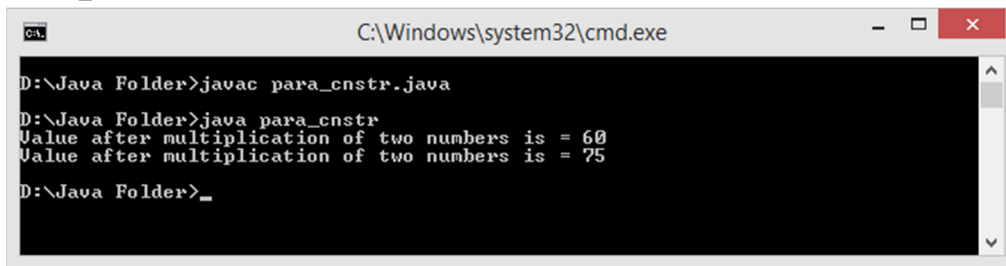
instance variables. Let's create a parameterized constructor in a class named `para_cnstr`. Listing 4.6 shows the code of the `para_cnstr` class:

Listing 4.6: Showing the Code of the `para_cnstr` Class

```
class para_cnstr
{
    // declaring instance variables
    int x, y;
    para_cnstr (int a, int b) // parameterized constructor
    {
        x=a;
        y=b;
    }
    public void mul () //method to calculate the multiple of two numbers
    {
        int m=x*y;
        System.out.println ("Value after multiplication of two numbers is
        = " + m);
    }
    public static void main (String args[]) {
        para_cnstr chk = new para_cnstr(12, 5);
        para_cnstr chk1 = new para_cnstr(15, 5);
        //invoking mul method
        chk.mul();
        chk1.mul();
    }
}
```

In Listing 4.6, you have created a class named `para_cnstr` to show how to create and use a parameterized constructor in a class. In the `para_cnstr` class, first you have declared two instance variables `x` and `y`. Then a parameterized constructor is created in which the variables `x` and `y` are initialized with some dynamic values through the parameters used in the constructor.

In the class `para_cnstr`, a method called `mul()` is created to calculate the product of the instance variables initialized in the constructor. You have also created two objects called `chk` and `chk1`, which are used to invoke the `mul()` method. Then different values are passed during the creation of both the `chk` and `chk1` objects to show that the instance variables will be initialized with separate values at run time. Figure 4.6 shows the output of the `para_cnstr` class:



```
C:\Windows\system32\cmd.exe

D:\Java Folder>javac para_cnstr.java

D:\Java Folder>java para_cnstr
Value after multiplication of two numbers is = 60
Value after multiplication of two numbers is = 75

D:\Java Folder>_
```

Figure 4.6: Output of Listing 4.6

In the `para_cnstr` class, you have created only one constructor, but you can create more than one constructor in a class. The creation of more than one constructor in a class is known as constructor overloading.

You can call the constructors and methods of a class from other constructors and methods of the same class using the `this` keyword. The `this` keyword can also be used to refer to the instance variables of the class in which the `this` keyword is used. Let's create a class named `this_key` to understand the functioning of the `this` keyword. Listing 4.7 shows the code of the `this_key` class:

Listing 4.7: Showing the Code of the `this_key` Class

```
class this_key
{
    // declaring instance variables
```

```

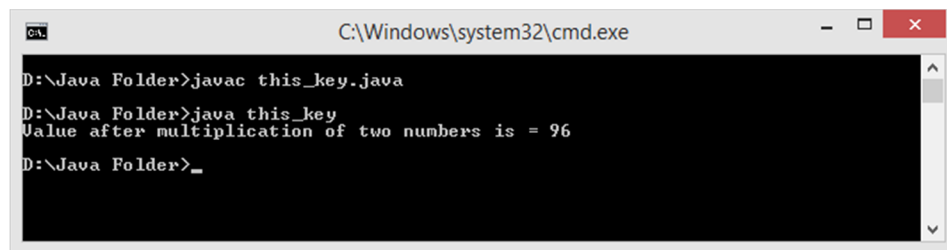
int x, y;
this_key() // default constructor
{
    this(12,8); // calling parameterized constructor
    this.mul(); // calling method
}
this_key (int x, int y) // parameterized constructor
{
    this.x=x; // referring instance variable
    this.y=y; // referring instance variable
}
public void mul () // method
{
    int m=x*y;
    System.out.println ("Value after multiplication of two numbers is
    = " + m);
}
public static void main (String args[]) {
    this_key dem = new this_key(); // creating object
}
}

```

In the `this_key` class, you have created a method and two constructors — default constructor and parameterized constructor. In the default constructor, you have used `this` keyword to call the parameterized constructor and the `mul()` method. On the other hand, in the parameterized constructor, you have used `this` keyword to refer the instance variables `x` and `y`. If you compare parameterized constructor of Listing 4.7 with the parameterized constructor created in Listing 4.7, then you can see that different names are used for the parameters and instance variables. However, in the constructor created in Listing 4.7, you have used the same name for parameters and instance variables.

In Java, it is not allowed to use the same name for local variable or variables enclosed in the same scope. However, using the `this` keyword, you can use the same name for instance variables and parameters, thereby decreasing the burden of remembering the different names. You have also created a method named `mul()` to calculate the product of the values passed as parameters during the call to parameterized constructor from the default constructor.

In the `main()` method of the `this_key` class, the object of the `this_key` class is created. On creation of the `this_key` class object, the control transfers to the default constructor of the class, thereby invoking the parameterized constructor of the class. The instance variables are then initialized with the values passed to the parameterized constructor and then the `mul()` method is called to calculate the product of the instance variables. Figure 4.7 shows the output of the `this_key` class:



```

C:\Windows\system32\cmd.exe
D:\Java Folder>javac this_key.java
D:\Java Folder>java this_key
Value after multiplication of two numbers is = 96
D:\Java Folder>_

```

Figure 4.7: Output of this_key class

In Java, it is possible to create a class inside another class. The class that is created inside the other class is known as inner class, and the class in which the inner class resides is called outer class. An inner class has full access to the member variables of its outer class.

The benefit of using inner class is that inner class is visible to its outer class only. Therefore, no other class can use the code written inside the inner class. The object of an inner class can only be created in the context of its outer class. You can use inner class to implement security features when you are writing some important code, such as password checking in an application, and do not want to grant access of this code to other classes except

its outer class. Let's create an inner class named `in_cls` within an outer class named `out_cls`. Listing 4.8 shows the code of the outer class `out_cls`:

Listing 4.8: Showing the Code of the `out_cls` Class

```
class out_cls // outer class
{
    int[] a={2,4,5,7,10}; // Array of numbers
    class in_cls // creating inner class
    {
        public void odd_even() // inner class method to segregate odd and even values
        {
            for (int i=0; i<a.length; i++)
            {
                if (a[i]%2 == 0)
                {
                    System.out.println("The value at the position " + a[i] + " is even");
                }
                else {
                    System.out.println("The value at the position " + a[i] + " is odd");
                }
            }
        }
    } // ending inner class
    public void cal_inr() // outer class method to call inner class members
    {
        in_cls val = new in_cls(); // creating object of inner class
        val.odd_even(); // calling method of inner class
    }
    public static void main(String args[])
    {
        out_cls show = new out_cls(); // creating object of outer class
        show.cal_inr(); // calling method of outer class
    }
}
```

In Listing 4.8, an outer class named `out_cls` is created, wherein another class called `in_cls` is created, which is the inner class. In the `in_cls` class, you have created a method called `odd_even()` to segregate odd and even values from the values stored in the array `a`.

A method called `cal_inr()` is created in which you have created the object of the inner class. The method `odd_even()` of the inner class are called from the outer class method `cal_inr()`.

In the `main()` method, an object of the outer class is created, which is then followed by calling the outer class method. As soon as the outer class method is called, the control transfers to the `cal_inr` method, which in turn performs its functioning. Figure 4.8 shows the output of the `out_cls` class:

```
C:\Windows\system32\cmd.exe

D:\Java Folder>javac out_cls.java
D:\Java Folder>java out_cls
The value at the position 2 is even
The value at the position 4 is even
The value at the position 5 is odd
The value at the position 7 is odd
The value at the position 10 is even
D:\Java Folder>
```

Figure 4.8: Output of listing 4.8

Exploring Packages

A package can be considered as a directory or folder that allows you to store various classes related to each other in an application. Packages resolve the problem of class name collision. Till now, you have created many classes

and you should note that every class has a unique name because in Java two classes with the same name inside a package cannot be created. It means that for every new class, you have to assign a different name. Assigning a different name to every new class is a tedious task; therefore, to resolve this problem, you can use package provided in the Java programming language.

To understand how packages resolve the class name collision, let's consider an example. Suppose you have created some classes with logical names, such as `pass`, `detail`, and so on, in an application to justify the purpose of classes. You want to use the same names for some other classes as well. To use the same name for the classes, you have to create another application in which you can use the same name of the classes as in the previous application. This issue can be solved by using packages.

To resolve the class name collision, put all the related classes in a package. If you have created a class named `pass` in a package and you also want to use the same name for any other class outside the package, then you are allowed to do so.

To understand packages, you need to know the following concepts:

- ❑ Types of packages
- ❑ How to import or use packages
- ❑ Access specifiers

Let's learn about these concepts in detail.

Studying the Types of Packages

Packages in Java are the containers of the related classes that can be either predefined in the Java programming language or created by the programmers. In Java, packages are divided into the following two broad categories:

- ❑ Built-in packages
- ❑ User-defined packages

Built-In Packages

The built-in packages are predefined in the Java programming language and help in performing important programming tasks. For example, there is a package called `java.lang` that provides classes required to develop the basic Java programs. To take advantage of the built-in packages, you need to import them. Table 4.1 explains some important built-in packages:

Table 4.1: Important Built-in Packages	
Package Name	Description
<code>java.lang</code>	Possesses the primary classes that are essential for developing basic Java programs. In the <code>java.lang</code> package, <code>lang</code> stands for language. The most important class it contains is <code>Object</code> class, which is the root of class hierarchy and represents the class instances at run time. The <code>java.lang</code> package contains the <code>Wrapper</code> classes that are used for converting primitive data types into objects. This package also contains <code>Runtime</code> and <code>System</code> classes that have methods to execute an application and find the total available memory in Java Virtual Machine (JVM).
<code>java.util</code>	Contains collection classes, such as <code>Arrays</code> , <code>Stack</code> , and <code>Linked List</code> . In the <code>java.util</code> package, <code>util</code> stands for utility. It also contains classes to handle date and time operations and classes to implement internationalization in your application.
<code>java.io</code>	Contains streams that are useful in storing the data in the form of files and performing input/output operations. In the <code>java.io</code> package, <code>io</code> stands for input and output.
<code>java.awt</code>	Contains classes to use the graphics and images in your application. In the <code>java.awt</code> package, <code>awt</code> stands for abstract window toolkit. This package contains classes to develop Graphical User Interface (GUI) application.
<code>javax.swing</code>	Allows creating the GUI application as the <code>java.awt</code> package, and <code>x</code> in this package signifies that the package possesses some additional features to develop GUI applications.
<code>java.net</code>	Performs networking operations, such as creating sockets, to establish communication between the client and server. In this package, <code>net</code> stands for network.

Table 4.1: Important Built-in Packages	
Package Name	Description
java.applet	Allows creating applets. An applet is a Java program that you can include in a Hypertext Markup Language (HTML) page.
java.text	Provides classes to format date, time, and numbers.
java.sql	Establishes connection between your application and database, such as Oracle and MS-Access. In this package, <code>sql</code> stands for structured query language.

In Table 4.1, some important and most commonly used built-in packages that have predefined classes to perform various tasks are discussed. In Java, packages are not only limited to the built-in packages; you can also create your own package called user-defined package to perform the required tasks.

User-Defined Packages

The package created by the user to perform some specific tasks is known as a user-defined package. The user-defined package can also be imported and used in the same way as the built-in package. To create a package, the package statement is used. The following code statement shows how to create a package:

```
package <pack_name> // statement to create a package
```

The `pack_name` in the preceding statement is any logical name that can be assigned to your package.

A package statement must be the first statement in the Java source file. Listing 4.9 shows the code of the `bub_sort1.java` file where you create a package:

Listing 4.9: Showing the Code of the `bub_sort1.java` File

```
package sort; // package creation
public class bub_sort1 // class in a package
{
    int x[];
    public bub_sort1(int a[]) // constructor having array as parameter
    {
        for (int i=0; i<a.length; i++)
        {
            x = a; // Assigning array elements to another array
        }
    }
    public void arr () // method for bubble sort
    {
        int temp=0, j, k;
        for (j=0; j<x.length; j++)
        {
            for(k=0; k<x.length-1; k++)
            {
                if (x[k]>x[k+1])
                {
                    temp=x[k+1];
                    x[k+1]= x[k];
                    x[k]=temp;
                }
            }
        }
        System.out.println("Elements after sorting are:");
        for (j=0; j<x.length; j++)
        {
            System.out.println(x[j]); // Printing sorted array elements on command prompt
        }
    }
    public static void main (String args[])
    {
        int m[]={5,3,2}; // Declaring and initializing array
```

```

        bub_sort1 val = new bub_sort1(m); // creating object of class and passing array
        val.arr(); // calling arr method
    }
}

```

In Listing 4.9, you have created a package named `sort`. In this package, a class named `bub_sort1` is created to perform sorting operation on some numbers through bubble sort algorithm. In the `main()` method of the `bub_sort1` class, you have created an array `m`. During the creation of the `bub_sort1` object, the array `m` is passed as parameter because the values on which the sorting has to be performed are stored in the array `m`.

Further, a method `arr` is created in which the code to perform sorting on the array elements has been written. In the `main()` method, you declared and initialized an array `m`, which is then followed by the creation of a `bub_sort1` object called `val`. You have passed the reference of the array `m` during the creation of the `val` object.

Now, let's learn how to compile and execute the class contained in a package. The process to compile a class in a package is same as a basic Java class, but it includes one more option, that is, `-d`.

To compile the classes in a package, you need to tell the compiler to create a package directory in the current directory. You can use the following code statement to create a package directory and place the `.class` file of the compiled class in it:

```
classpath> javac -d . <class name>.java
```

The `<class name>` in the preceding code statement is the name of the class to be compiled. After compiling the class, you need to execute the class. To run the program, you just have to refer the class name along with the package name. You can refer the class name by using the dot operator, as shown in the following code statement:

```
classpath> java <package name>.<class name>
```

The preceding code statements are used to compile and execute the `bub_sort1` class from the `sort` package. Figure 4.9 shows the output of the `bub_sort1` class:

```

C:\Windows\system32\cmd.exe

D:\Java Folder>javac -d . bub_sort1.java
D:\Java Folder>java sort.bub_sort1
Elements after sorting are:
2
3
5
D:\Java Folder>_

```

Figure 4.9: Output of listing 4.9

In Listing 4.9, you have created only a single program of sorting. You can also create more programs of sorting using different algorithms and put them in the same package. In this way, you can create separate package to assemble related Java classes.

Importing Packages

Importing packages in an application allows you to take the advantages of classes stored in these packages. You can import as many packages as you need to use in an application. You can import both the user-defined and built-in packages in your class. You can take the benefit of any package by using the `import` statement, which consists of the `import` keyword with the package name, in a Java program. Import statement comes after the package specification and before the class definition. Listing 4.10 shows how to import packages using the `import` statement in the `imp_pack` class:

Listing 4.10: Showing the Code of the `imp_pack` Class

```

import sort.bub_sort1; // importing package
class imp_pack
{
    public static void main(String args[]) {
        int m[]={8,5,7,6,9};
        bub_sort1 obj = new bub_sort1(m);
    }
}

```

```

        obj.arr();
    }
}

```

In Listing 4.10, you have imported the package created in Listing 4.9. Then, a class named `imp_pack` is created in which an array has been declared and initialized. You have also created an object of the `bub_sort1` class provided in the `sort` package and then called its method `arr()` to perform sorting on the array elements.

It can be seen in Listing 4.10 that you have passed an array during the creation of the `bub_sort1` object to sort the array elements. The code to perform sorting has not been written in Listing 4.10 because the code is already written in the package `sort` that you have imported. In brief, by importing the packages, you can simplify your work by using the code provided in the classes stored in the packages.

Figure 4.10 shows the output of the `imp_pack` class:

```

C:\Windows\system32\cmd.exe
D:\Java Folder>javac imp_pack.java
D:\Java Folder>java imp_pack
Elements after sorting are:
5
6
7
8
9
D:\Java Folder>_

```

Figure 4.10: Output of Listing 4.10

It can be seen in Figure 4.10 that the elements stored in the array are successfully sorted.

Using Access Specifiers

In Java, you can restrict or allow any class or method to access the code written in other class or method in a program. In the same way, you can also set the visibility of the classes enclosed in the packages. Similarly, Java has provided the following access specifiers that help in controlling the access to code, class, and packages:

- ❑ `public`—Represents an access specifier that is used when you want the classes, methods, and fields to be visible to other classes and methods. A `public` specifier makes the methods, classes, and fields accessible to all the other classes, methods, or packages.
- ❑ `protected`—Signifies an access specifier that is used when the variable or method is to be accessed only by the members of the same class, subclasses related to that class, or within the classes of the same package.
- ❑ `default`—Refers to an access specifier that is applied for a variable or method in a class when no specifier is used explicitly for it. The fields, methods, and classes with the `default` access specifier are available to other classes and methods provided in the same package.
- ❑ `private`—Defines an access specifier that is just opposite of the `public` access specifier. If the `private` specifier is used for the methods and fields, then they can only be accessed within the same class in which they are declared.

Working with Streams API

In Java SE 8, new classes are added and the existing classes are improved to take advantage of the streams. The new and enhanced classes are available in the following packages:

- ❑ `java.util`—Integrates the Java Collections Framework with streams and provides general utility functionality used by streams
- ❑ `java.util.function`—Includes general purpose functional interfaces that provide target types for lambda expressions and references for method

- ❑ `java.util.stream`: Includes the majority of interfaces and classes that provide functionality to streams and aggregate operations

Besides the additions made to the existing classes, some other additions include methods that accept instances of functional interfaces, which can be invoked with lambda expressions or method references. The `java.util.function` and `java.util.stream` packages are two new packages included in Java SE 8. Let's now discuss the `java.util.stream` package.

Stream API Overview

The stream class is the main abstraction provided in the `java.util.stream` package. Classes such as `Stream`, `IntStream`, `LongStream`, and `DoubleStream` are streams over objects and the primitive `int`, `long`, and `double` types. Streams can be obtained in the following ways:

- ❑ They can be obtained from a `Collection` using the `stream()` and `parallelStream()` methods.
- ❑ They can be obtained from an array using the `Arrays.stream(Object[])` method.
- ❑ They can be obtained from static factory methods on the stream classes, such as `Stream.of(Object[])`, `IntStream.range(int, int)`, or `Stream.iterate(Object, UnaryOperator)`.
- ❑ The lines of a file can be obtained from the `BufferedReader.lines()` method.
- ❑ Streams of file paths can be obtained from the methods in `Files`.
- ❑ Streams of random numbers can be obtained from the `Random.ints()` method.
- ❑ Various other stream-bearing methods in the JDK are `BitSet.stream()`, `Pattern.splitAsStream(java.lang.CharSequence)`, and `JarFile.stream()`.

The various interfaces available in the `java.util.stream` package and their description are listed in Table 4.2:

Table 4.2: Interfaces and their Description	
Interface	Description
<code>BaseStream<T,S extends BaseStream<T,S>></code>	Refers to the base interface for streams, which are sequences of elements that support sequential and parallel aggregate operations
<code>Collector<T,A,R></code>	Refers to a mutable reduction operation that accumulates input elements into a mutable result container, optionally transforming the accumulated result into a final representation after the processing of all the input elements
<code>DoubleStream</code>	Refers to a sequence of primitive double-valued elements which support sequential and parallel aggregate operations
<code>DoubleStream.Builder</code>	Refers to a mutable builder for a <code>DoubleStream</code>
<code>IntStream</code>	Refers to a sequence of primitive int-valued elements which support sequential and parallel aggregate operations
<code>IntStream.Builder</code>	Refers to a mutable builder for <code>IntStream</code>
<code>LongStream</code>	Refers to a sequence of primitive long-valued elements which support sequential and parallel aggregate operations
<code>LongStream.Builder</code>	Refers to a mutable builder for <code>LongStream</code>
<code>Stream<T></code>	Refers to a sequence of elements which support sequential and parallel aggregate operations
<code>Stream.Builder<T></code>	Refers to a mutable builder for a <code>Stream</code>

Table 4.3 presents the classes in the `java.util.stream` package:

Table 4.3: Classes in the java.util.stream Package and their Description	
Class	Description
<code>Collectors</code>	Contains implementations of <code>Collector</code> that implements several useful reduction operations, such as gathering elements

Table 4.3: Classes in the `java.util.stream` Package and their Description

Class	Description
	into collections, summarizing elements on the basis of different criteria, etc.
<code>StreamSupport</code>	Provides low-level utility methods to create and manipulate streams

Collection and Stream

The main differences between stream and collections are as follows:

- ❑ **No storage**—A stream does not store elements as it is not a data structure; whereas, a collection does.
- ❑ **Functional nature**—Whenever an operation is performed on a stream, it produces a result and therefore does not alter its source. For example, when the filtering operation is performed on a stream, which is retrieved from a collection, it generates a new `Stream`. This new stream does not contain filtered elements and is created instead of deleting elements present in the source collection. On the other hand, in a collection, the elements are deleted from the existing collection.
- ❑ **Laziness-seeking**—Several operations on stream can be lazily implemented due to which optimization can take place. These operations may include filtering, mapping, or duplicate removal. Generally, stream operations are broken down into intermediate (or `Stream`-producing) operations and terminal (or value- or side-effect-producing) operations. Intermediate operations are always considered lazy. On the other hand, optimization cannot take place in a collection.
- ❑ **Possibly unbounded**—Streams do not have a finite size unlike collections.
- ❑ **Consumable**—The elements of a stream are only visited once during the life of a stream. Like an iterator, a new stream must be generated to revisit the same elements of the source. However, in a collection, the elements can be navigated again and again.

Commonly Used Functional Interfaces in Stream

The `java.util.function` package includes interfaces that can be used by the JDK and with the code developed by a user. The package also includes enough functionality that can meet the common requirements of lambda expressions. This package also provides functional interfaces such as `FileFilter` that fulfills specific purposes.

The interfaces in the `java.util.function` package are annotated with `FunctionalInterface`. However, putting this annotation is not compulsory for making the compiler identify an interface as a functional interface. The annotations are merely used for capturing design intent and providing help to the compiler to determine the accidental violations of design intent.

Functional interfaces generally focus on abstract concepts such as functions, actions, or predicates. While using functional interfaces or referring the variables typed as functional interfaces, you need to refer to the abstract concepts; for example, you can use "this function" in place of "the function represented by this object." The different types of functional interfaces and their description are presented in Table 4.4:

Table 4.4: Types of Interfaces and their Description

Interface	Description
<code>BiConsumer<T, U></code>	Signifies an operation that takes two input arguments and gives no result.
<code>BiFunction<T, U, R></code>	Signifies a function that takes two arguments and generates a result.
<code>BinaryOperator<T></code>	Signifies an operation on two operands of the same type, providing a result of the same type as the operands.
<code>BiPredicate<T, U></code>	Signifies a predicate (boolean-valued function) of two arguments.
<code>BooleanSupplier</code>	Signifies a supplier of boolean-valued results.
<code>Consumer<T></code>	Signifies an operation that takes a single input argument and gives no result.

Table 4.4: Types of Interfaces and their Description

Interface	Description
DoubleBinaryOperator	Signifies an operation on two double-valued operands and generating a double-valued result.
DoubleConsumer	Signifies an operation that gets a single double-valued argument and gives no result.
DoubleFunction<R>	Signifies a function that takes a double-valued argument and generates a result.
DoublePredicate	Signifies a predicate (boolean-valued function) of one double-valued argument.
DoubleSupplier	Signifies a supplier of double-valued results.
DoubleToIntFunction	Signifies a function that takes a double-valued argument and generates an int-valued result.
DoubleToLongFunction	Signifies a function that takes a double-valued argument and generates a long-valued result.
DoubleUnaryOperator	Signifies an operation on a single double-valued operand that generates a double-valued result.
Function<T, R>	Signifies a function that takes one argument and generates a result.
IntBinaryOperator	Signifies an operation on two int-valued operands and generating an int-valued result.
IntConsumer	Signifies an operation that takes a single int-valued argument and gives no result
IntFunction<R>	Signifies a function that takes an int-valued argument and generates a result.
IntPredicate	Signifies a predicate (boolean-valued function) of one int-valued argument.
IntSupplier	Signifies a supplier of int-valued results.
IntToDoubleFunction	Signifies a function that takes an int-valued argument and generates a double-valued result
IntToLongFunction	Signifies a function that takes an int-valued argument and generates a long-valued result
IntUnaryOperator	Signifies an operation on a single int-valued operand that generates an int-valued result.
LongBinaryOperator	Signifies an operation on two long-valued operands and generates a long-valued result.
LongConsumer	Signifies an operation that takes a single long-valued argument and gives no result.
LongFunction<R>	Signifies a function that takes a long-valued argument and generates a result.
LongPredicate	Signifies a predicate (boolean-valued function) of one long-valued argument.
LongSupplier	Signifies a supplier of long-valued results.
LongToDoubleFunction	Signifies a function that takes a long-valued argument and generates a double-valued result.
LongToIntFunction	Signifies a function that takes a long-valued argument and generates an int-valued result.
LongUnaryOperator	Signifies an operation on a single long-valued operand that generates a long-valued result.
ObjDoubleConsumer<T>	Signifies an operation that takes an object-valued argument and a double-valued argument and gives no result.
ObjIntConsumer<T>	Signifies an operation that takes an object-valued argument and an int-valued argument and gives no result
ObjLongConsumer<T>	Signifies an operation that takes an object-valued argument and a long-valued argument and gives no result.
Predicate<T>	Signifies a predicate (boolean-valued function) of one argument.
Supplier<T>	Signifies a supplier of results.
ToDoubleBiFunction<T, U>	Signifies a function that takes two arguments and generates a double-valued result.
ToDoubleFunction<T>	Signifies a function that generates a double-valued result.

Table 4.4: Types of Interfaces and their Description

Interface	Description
<code>ToIntBiFunction<T, U></code>	Signifies a function that takes two arguments and generates an int-valued result.
<code>ToIntFunction<T></code>	Signifies a function that generates an int-valued result
<code>ToLongBiFunction<T, U></code>	Signifies a function that takes two arguments and generates a long-valued result.
<code>ToLongFunction<T></code>	Signifies a function that generates a long-valued result.
<code>UnaryOperator<T></code>	Signifies an operation on a single operand that produces a result of the same type as its operand.

java.util.Optional

`Optional` is a container object that may or may not contain a non-null value. In case the value is present, the `isPresent()` method will return the true value and the `get()` method will return the value. The additional methods depend on the availability or non-availability of a contained value. For example, the `orElse()` method will return a default value in case the value is not present and the `ifPresent()` method will run a block of code provided the value is present. Some important methods of the `java.util.Optional` package are given in Table 4.5:

Table 4.5: Noteworthy Methods of the `java.util.Optional` Package

<code>empty()</code>	Returns an empty <code>Optional</code> instance.
<code>equals(Object obj)</code>	Checks whether or not the other object is equal to the object passed as an argument.
<code>filter(Predicate<? super T> predicate)</code>	Returns the values matching with the given predicate in case the value is present, else returns an empty <code>Optional</code> object.
<code>flatMap(Function<? super T, Optional<U> mapper)</code>	Applies the specified <code>Optional</code> bearing mapping function and returns the result in case the value is present, else returns an empty <code>Optional</code> object.
<code>get()</code>	Returns a value in case the value is present in the <code>Optional</code> object, else throws the <code>NoSuchElementException</code> exception.
<code>hashCode()</code>	Returns the hash code value if a value is present in the <code>Optional</code> object, else returns 0.
<code>ifPresent(Consumer<? super T> consumer)</code>	Invokes the provided consumer with the value in case a value is present else performs nothing.
<code>isPresent()</code>	Returns true if the value is present else false.

Aggregate Operations

An operation that is performed on a data structure rather than on an individual element is known as an aggregate operation. Aggregate operations, like `forEach`, give an impression that these are like iterators. However, they are different from iterators in the following ways:

- ❑ **They use internal iteration**—Aggregate operations perform internal iteration as they do not contain a method like `next`, which enables them to iterate to the next element of the collection. With the help of internal delegation, your application identifies the collection over which it iterates. But the JDK identifies the iteration over the collection. On the other hand, with the help of external iteration, your application can determine about the collection over which it iterates and the way it iterates. However, there is a limitation with external iterators as they can only iterate sequentially over the elements of a collection. This limitation does not exist with internal iterators. Moreover, the internal iteration takes benefits of parallel computing, which involves breaking a problem into subproblems and then solving them simultaneously.
- ❑ **They process elements from a stream**—Aggregate operations do not process elements directly from a collection, but directly from a stream. Therefore, they are also called stream operations.
- ❑ **They support behavior as parameters**—The lambda expressions can also be specified as parameters for most of the aggregate operations which allow customization of the behavior of a specific aggregate operation.

To better understand the concept of aggregate operations, consider a scenario in which you are developing a social networking site. You want to provide the facility that allows an administrator to perform different actions, such as sending a message to the members of the social networking application, which satisfy certain criteria. Suppose the members of this social networking application are represented by the following `SNMember` class:

```
public class SNMember {
    public enum Gender {
        M, FM
    }
    String sname;
    LocalDate mem_birthday;
    Gender sex;
    String email;
    // ...
    public int getMemage() {
        // ...
    }
    public String getMemname() {
        // ...
    }
}
```

Consider the following code snippet which prints the name of all members contained in the collection `memlist` using the `for-each` loop:

```
for (SNMember sn : memlist) {
    System.out.println(sn.getMemname());
}
```

Now, consider the following code snippet which prints all the members present in the collection `memlist`, but with the use of aggregate operation `forEach`:

```
memlist
    .stream()
    .forEach(e -> System.out.println(e.getMemname()));
```

In the previous code snippet, the aggregate operation `for-each` loop is used.

Pipelines and Streams

A pipeline refers to a sequence of aggregate operations. Consider the following code snippet that prints the male members available in the collection `memlist` with a pipeline that comprises the aggregate operations such as `filter` and `forEach`:

```
memlist
    .stream()
    .filter(e -> e.getsex() == SNMember.Gender.M)
    .forEach(e -> System.out.println(e.getMemname()));
```

A pipeline consists of the following components:

- ❑ **Source**—It might be a collection, an array, a generator function, or an I/O channel. In the previous code snippet, the example of a source is the collection `memlist`.
- ❑ **Zero or more intermediate operations**—An intermediate operation, such as `filter`, produces a new stream.
- ❑ **Stream**—A stream does not store elements; instead, it carries values from a source through a pipeline. In the previous code snippet, a stream is created from the collection `memlist` by invoking the method `stream`.
- ❑ **Predicate**—A new stream is returned by the filter operation. This stream comprises those elements that match its predicate. In the preceding code snippet, the predicate is the lambda expression `e -> e.getsex() == SNMember.Gender.M`, which gives the boolean value (true or false). It returns true if the sex field of object `e` has the value `SNMember.Gender.M`. Subsequently, the filter operation returns a stream that comprises all male members in the collection `memlist`.
- ❑ **Terminal operation**—A pipeline also comprises a terminal operation, such as `forEach`, which generates a non-stream result, such as a primitive value, a collection, or no value at all. In the preceding code snippet, the parameter of the `forEach` operation is the lambda expression `e -> System.out.println(e.getMemname())`, which invokes the method `getMemname` on the object `e`.

Working with Time API

It has always been difficult to work with `Date`, `Time`, and `TimeZones` in Java. There was no standard approach or defined API for handling date and time in Java. In Java SE 8, the `java.time` package has been introduced which facilitates the working with the time data in Java. Working with Time API is very simple as it provides some sub packages such as `java.time.format` and `java.time.zone`. The `java.time.format` package gives classes to print and parse dates and times. The `java.time.zone` package provides classes for `TimeZones`.

The `Clock` class is used to represent the current time. The `systemUTC()` static method will return the current time based on the system clock and set to UTC. The code for this is as follows:

```
import javax.time.Clock;
Clock clock = Clock.systemUTC();
```

The `systemDefaultZone()` method is used to represent the default time zone on the system. The code for this is as follows:

```
Clock clock = Clock.systemDefaultZone();
```

In order to access the current time in milliseconds from January 1st, 1970, the `millis()` method can be used. Thus the `Clock` class and all subclasses are wrapped around `System.currentTimeMillis()`. The code for this is as follows:

```
Clock clock = Clock.systemDefaultZone();
long time = clock.millis();
```

A time zone is a set of rules corresponding to a region in which the standard time is the same. It is required to import the `ZoneId` class to work with time zones. The class provides a method to get the default system time zone. The code for this is as follows:

```
import javax.time.ZoneId;
import javax.time.Clock;
ZoneId zone = ZoneId.systemDefault();
Clock clock = Clock.system(zone);
```

As seen above, the `ZoneId` can then be used to get an instance of a `Clock` with that time zone. Other time zones can be accessed by their name as follows:

```
ZoneId zone = ZoneId.of("Europe/Berlin");
Clock clock = Clock.system(zone);
```

Summary

In this chapter, you have learnt about of class and objects. You have also learnt about how work with methods. In addition, you have learnt about constructors in Java. You have also explored about package. Finally, you have learnt about Stream API, Date and Time API and how to use access specifiers.



5

Implementing Object-Oriented Programming in Java

<i>If you need information on:</i>	<i>See page:</i>
Understanding Encapsulation	118
Understanding Abstraction	119
Understanding Inheritance	120
Understanding the final Keyword	123
Implementing Interfaces	124
Working with Lambda Expressions	126
Implementing Abstract Classes and Methods	129
Difference between Abstract Classes and Interfaces	130
Implementing Polymorphism	130
Summary	133

Object-Oriented Programming (OOP) is a programming technique where you specify various objects and define the interaction among them to implement program logic. Classes and objects are the two main elements of OOP. Objects in OOP are based on real-life objects, such as a car. Similar to the real-life objects that have specific properties and behavior, objects in OOP encompass specific properties and behaviors, which make them unique. For example, the color of a car represents a property and the speed of the car represents its behavior. The properties and behaviors of objects in OOP are represented through variables and methods, which are defined in a class. You have learned in the earlier chapters that a class is a template for creating objects and an instance of a class is called an object. For example, car is an instance of the automobile class.

In Java, OOP is implemented through encapsulation, which protects data of a class from being accessed by members of another class; abstraction, which hides data of a class from other classes; inheritance, which uses the code written in a class within other classes; and polymorphism, which uses various methods with the same name.

In this chapter, you explore the principles of OOP, such as encapsulation, abstraction, inheritance, and polymorphism along with their types. The chapter discusses how to use `final` keyword for preventing a class from being inherited by another class, declaring a constant variable, and preventing a method from being overridden. In addition, the chapter discusses about interfaces and abstract classes in Java. It also explores the differences between abstract classes and interfaces. You will also learn about the lambda expressions, a new feature introduced in Java SE 8.

Understanding Encapsulation

Encapsulation is a protective mechanism by which members of a class (methods and variables) are prevented from being accessed by members of other classes. You can think of a class as an example of encapsulation because a class binds its variables and methods and hides their complexity from other classes.

Variables in a class are generally marked `private` to prevent other classes in the Java program from accessing them. Now, the concern is if variables cannot be accessed by other classes in a program, then how can their data be used? The answer lies in creating `public` methods that other classes can access. By accessing these `public` methods, you can access the variables used in the methods. This is the only way of interacting with the variables declared `private` in a class. This means that other classes are unaware of the variables used in a particular class as well as the code written in the methods to access the variables. In this way, other classes are able to access the methods declared in a class.

Now that you have a brief idea about encapsulation, let's learn how to create a class that uses the concept of encapsulation, as shown in Listing 5.1:

Listing 5.1: Implementing the Concept of Encapsulation

```
class Student
{
    private String name;
    private String subject;
    public void desc(String n,String sub)
    {
        name=n;
        subject=sub;
        System.out.println(name+" doing Post-Graduation in :"+subject);
    }
}
public class StudentInstance
{
    public static void main(String args[])
    {
        Student s=new Student();
        s.desc("JOHN SMITH","Information Technology");
    }
}
```

In Listing 5.1, the `Student` class has declared two private variables, `name` and `subject`, and a method, `desc()`. In addition, another class called `StudentInstance` is created, which is used to access the private

variables of the `Student` class. In Java, one class cannot directly access the `private` members of another class. Therefore, the `desc()` method of the `Student` class is marked as `public` so that the method can be accessed by the `StudentInstance` class. The `s` instance of the `Student` class is created in the `StudentInstance` class and used to invoke the `desc()` method, which accesses the `private` variables of the `Student` class. In this way, the `private` variables of a class can be accessed with the help of a `public` method. In Listing 5.1, the `desc()` method of the `Student` class can only be accessed in the `StudentInstance` class, but the method's definition cannot be modified. Figure 5.1 shows the output of Listing 5.1:

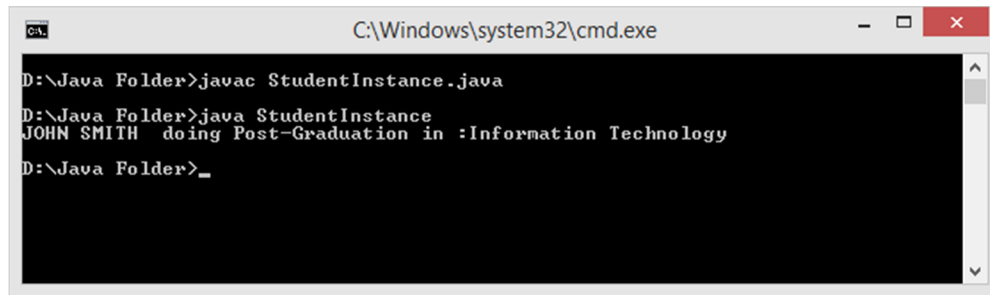


Figure 5.1: Output of Listing 5.1

Figure 5.1 displays the message specified in the `desc()` method. In this way, the concept of encapsulation is implemented in Java. Now, let's explore another important OOP's principle called abstraction.

Understanding Abstraction

Abstraction is the mechanism by which you hide data that is not required by a user. The advantage of abstraction is that the user can work only with the needed data and is not required to view the unwanted data.

Let's take the example of the car to understand the concept of abstraction. The owner of the car must know how to drive it. This obviously implies that the owner must know about the various components of the car and how to use them. For example, the owner must know that the accelerator pedal increases the speed of the car and pressing the brake pedal stops it. To perform these simple actions, the owner just needs to know how to use these components, but not how they function. Similarly, in OOP, you need to know about the specific classes or methods that are called to implement specific program logic, without knowing how these classes or methods function. This concept is known as abstraction. Listing 5.2 explains the concept of abstraction in Java:

Listing 5.2: Implementing the Concept of Abstraction

```
class Abstraction
{
    private int x ;
    private String str;
    private double bal;
    private double sal;
    private double credit;
    public void disp(int anum, String nam, double bal )
    {
        x=anum;
        str=nam;
        bal=bal;
        System.out.println("Account Number="+x);
        System.out.println("Customer Name="+str);
        System.out.println("Total Balance=$"+bal);
    }
}
public class AbstractionInstance
{
    public static void main(String args[])
    {
```

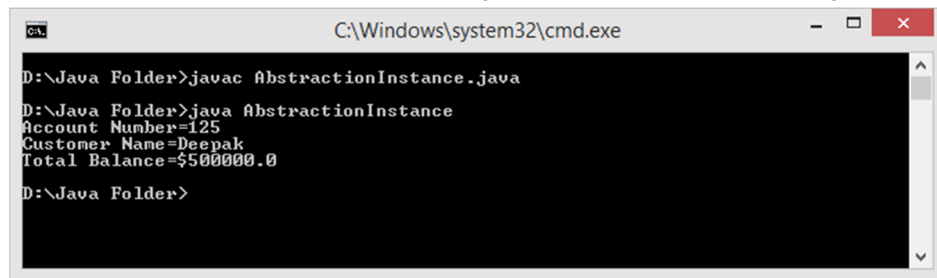
```

        Abstraction b=new Abstraction();
        b.disp(125,"Deepak", 500000);
    }
}

```

In Listing 5.2, you have assumed that an accountant working in a finance company is liable to view a customer's information, such as his account number, name, and balance amount. However, the accountant is not permitted to view confidential information related to the company, such as staff salaries, the profit and loss of the company, or the total amount of loan that the company has given to its customers. Therefore, the data that an accountant is not authorized to view can be abstracted from his view, whereas the same data can be viewed by a manager of the company.

In this way, data can be abstracted according to the user's point of view. In Listing 5.2, a class named `Abstraction` is created in which five private variables, `x`, `str`, `bal`, `sal`, and `credit`, are declared. In the `Abstraction` class, the `disp()` method is created, which can accept three parameters: `anum`, `nam`, and `bal`. The values received by the `anum`, `nam`, and `bal` parameters are assigned to the variables `x`, `str`, and `bal`, respectively. The `disp()` method accesses only customer-related information such as account number of a customer, his/her name, and the total balance in his/her account; however, this method cannot access the confidential information of the company, such as the employee's salary or the total amount of loan the company has given to its customers. This means the `sal` and `credit` variables cannot be accessed by the `disp()` method because these variables are hidden from the method. Figure 5.2 shows the output of Listing 5.2:



```

C:\Windows\system32\cmd.exe
D:\Java Folder>javac AbstractionInstance.java
D:\Java Folder>java AbstractionInstance
Account Number=125
Customer Name=Deepak
Total Balance=$500000.0
D:\Java Folder>

```

Figure 5.2: Output of Listing 5.2

You can implement abstraction in Java in this way. Let's learn about inheritance in Java in the next section.

Understanding Inheritance

The feature by which one class acquires the characteristics of an existing class is known as inheritance. In Java, a class that is inherited is called a superclass and the class that inherits the superclass is called a subclass. For example, the `BallPointPen` class is a subclass of the `Pen` class. In terms of OOP, this means that `BallPointPen` is a subclass derived from its superclass `Pen`. The principle behind inheritance is that each derived class acquires all the characteristics of its superclass, thereby ensuring reusability of the code.

NOTE

Multiple inheritance is not supported in Java, which means a subclass can extend only one superclass in Java.

In Java, the `extends` keyword is used in the subclass definition to inherit a superclass. Let's understand inheritance with the help of an example. Listing 5.3 declares two classes, `Person` and `Sportsperson`, to help you understand the concept of inheritance:

Listing 5.3: Implementing the Concept of Inheritance

```

class Person
{
    private String name;
    public void setName(String n)
    {
        name = n;
    }
    public void getName()

```

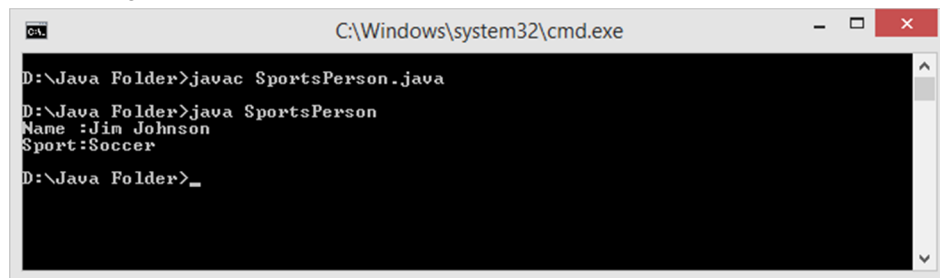
```

    {
        System.out.println("Name :" + name);
    }
}
public class SportsPerson extends Person
{
    private String sport;
    public void setSport(String sp)
    {
        sport = sp;
    }
    public void getSport()
    {
        System.out.println("Sport:" + sport);
    }
    public static void main(String[] args)
    {
        SportsPerson sp = new SportsPerson();
        sp.setName("Jim Johnson");
        sp.setSport("Soccer");
        sp.getName();
        sp.getSport();
    }
}

```

In Listing 5.3, the `SportsPerson` class has inherited the `Person` class, which means that the `Person` class is a superclass of the `SportsPerson` class. In the `Person` class, two methods named `setName()` and `getName()` are defined. The `setName()` method sets the name of a player and the `getName()` method displays the name of the player. Next, two methods named `setSport()` and `getSport()` are defined in the `SportsPerson` class. The `setSport()` method sets the value for the sport variable and the `getSport()` method displays the value of the sport variable.

You have also created an object `sp` of the `SportsPerson` class. Since the `SportsPerson` class extends the `Person` class, it has accessed to the members of both the `SportsPerson` and `Person` classes. Figure 5.3 shows the output of Listing 5.3:



```

C:\Windows\system32\cmd.exe
D:\Java Folder>javac SportsPerson.java
D:\Java Folder>java SportsPerson
Name :Jim Johnson
Sport:Soccer
D:\Java Folder>_

```

Figure 5.3: Output of Listing 5.3

In Listing 5.3, note that you have used different names for the methods defined in both the `Person` and `SportsPerson` classes. However, you do not always need to use different method names in a subclass to inherit the functionality of its superclass. In other words, you can specify the methods in a subclass with the same name and type signature as that in its superclass to override the functionality of the superclass methods.

When a subclass includes a method definition that has the same name, number, and types of parameters as a method already defined in its superclass, the definition of the method created in the subclass replaces the definition of the method created in the superclass. This is called method overriding. In other words, you can create a new definition for an already existing method through method overriding.

Let's take an example to explain method overriding. Suppose you have the `calculate()` method in the `Vehicle` superclass, which is used to calculate the speed of a vehicle, but cannot display the speed of the vehicle. You want to create a method that can both calculate and display the speed of the vehicle. Now, rather than creating a new method, you can extend the `Vehicle` superclass and override its `calculate()` method

by appending the code in it to display the speed in the superclass. In this way, method overriding saves your time and helps in creating a new application quickly. Listing 5.4 declares the `Vehicle` superclass and `Car` subclass to help you understand the concept of method overriding:

Listing 5.4: Implementing the Concept of Method Overriding

```
class Vehicle
{
    public double calculate()
    {
        double speed, distance, time;
        distance=150;
        time=2.5;
        speed=distance/time;
        return speed;
    }
}
class Car extends Vehicle
{
    public double calculate()
    {
        double x;
        x=super.calculate();
        System.out.println(" Speed of the vehicle is  : " + x + "
        Kilometer per hour");
        return x;
    }
    public static void main(String args[])
    {
        Car c=new Car();
        c.calculate();
    }
}
```

In Listing 5.4, the `Car` class extends the `Vehicle` class. In the `Vehicle` class, a method named `calculate()` is defined to calculate the speed of a vehicle. Similarly, in the `Car` class, a method named `calculate()` is defined, which overrides the `calculate()` method of its superclass `Vehicle`, and is used to calculate and display the speed of the vehicle.

In the `calculate()` method of the `Car` class, the `calculate()` method of the `Vehicle` class is called and the speed of the vehicle is displayed. The advantage of using method overriding is that you do not have to rewrite the code to calculate the speed of the vehicle. Instead, you have to call the `calculate()` method of the superclass and append the code to display the speed of the vehicle. In other words, you can reuse the code written in the methods of a superclass through method overriding. Figure 5.4 displays the output of Listing 5.4:

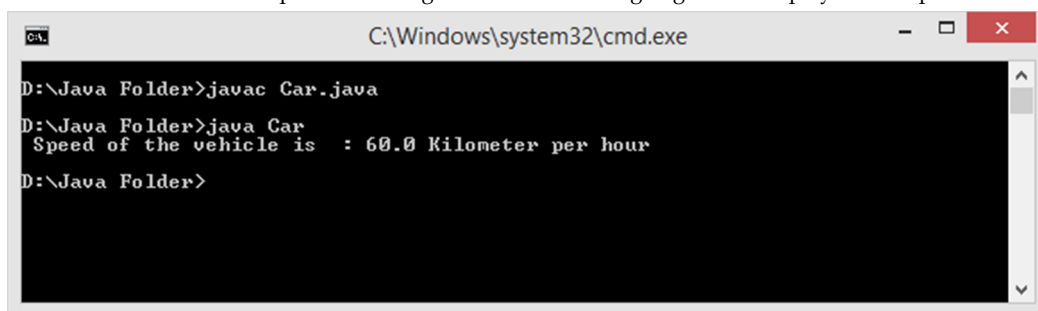


Figure 5.4: Output of Listing 5.3

In Listing 5.4, note the use of the `super` keyword inside the `calculate()` method of the `Car` class. The `super` keyword is used to access the members of the superclass in a subclass. This allows the code to be reused and reduces the time that would have been required in creating a new method for calculating and displaying the

speed of the vehicle. The `super` keyword can be used in the following ways to access the members of a superclass:

- ❑ To access superclass variables in a subclass, use the `super` keyword with the variable name shown in the following code snippet:

```
super.variable;
```

- ❑ To access superclass methods in a subclass, use the `super` keyword with the method name shown in the following code snippet:

```
super.method();
```

This is how Java helps in implementing code reusability. Sometimes, it may happen that you have written some methods in a class and want no other class to inherit and override them. You can do this by using the `final` keyword.

Let's discuss more about the `final` keyword in the following sections.

Understanding the `final` Keyword

Once a Java variable is declared `final`, then that variable can be initialized only for a single time. All the Java classes that are declared `final` cannot be further extended. The `final` keyword is a modifier, which is used for the following purposes in Java:

- ❑ Preventing a class from being inherited by another class
- ❑ Declaring a constant variable
- ❑ Preventing a method from being overridden

Let's discuss about each purpose in detail in the following sections.

Preventing Inheritance

Preventing inheritance simply means that a superclass cannot be extended by a subclass. For example, suppose you have created a class `Bank` in which a method `balance()` is defined to display the balance amount of a user. The `Bank` class can be inherited by other Java classes that can alter the definition of the `balance()` method, which leads to incorrect display of the balance amount. To prevent such problems, the `Bank` class should not be extended by any other class. To prevent other classes from extending the `Bank` class, the `final` keyword is used before the `class` keyword during class declaration. By declaring a class `final`, all of its methods automatically become `final`.

NOTE

*An abstract class should not be declared `final` because abstract classes are created to be subclassed or extended. To learn more about abstract classes, see the section *Implementing Abstract Classes and Methods* in this chapter.*

The following code snippet is used to declare a final class:

```
public final class ABC
{
    // . . . . .
}
```

In the preceding code snippet, the `ABC` class is declared `final` to prevent it from being extended by other classes.

Let's learn how to declare constant variables.

Declaring Constant Variables

Variables can be declared `final`, which means that a variable becomes constant when it is declared `final`. Value once assigned to a `final` variable cannot be changed. For example, in an application, you might want to display some static information, such as a company's registration number. In such case, you have to declare the variable as `final`. When a variable is declared `final` in a program, the methods defined in the program can access the variable, but cannot modify its value. The variables that are declared `final` must be initialized at the time of declaration. The following code snippet shows how to declare a variable `final`:

```
public class ABC
{
    public final int PI=3.14;
    // Methods
}
```

In the preceding code snippet, the variable `PI` is declared `final`. Moreover, the variable `PI` can be accessed by the methods declared in the `ABC` class, but its value cannot be modified.

In the next section, let's learn how to prevent method overriding in a subclass.

Preventing Method Overriding

A method is declared `final` in a superclass to prevent it from getting overridden or its implementation from being changed in a subclass. The prevention of method overriding ensures the method's safety and security because it is not possible to change a `final` method of a superclass in the subclass. The following code snippet shows how to declare a `final` method:

```
public class ABC{
    public final void show()
    {
        // code
    }
}
```

In the preceding code snippet, the `show()` method whose definition cannot be modified is declared `final`.

You know that Java does not support multiple inheritance. However, you may sometimes need to implement the functionalities of more than one class in a subclass. In such a case, Java provides an alternative tool known as interface to support multiple inheritance. Let's explore the concept of interface in the following section.

Implementing Interfaces

In Java, an interface is a collection of various methods without definition. It is similar to a Java class, except that unlike an interface, a class may contain method definition. An interface can be implemented by one or more classes using the `implements` keyword in the class definition. A class that implements an interface is known as the implementation class.

An advantage of using interfaces is that they support multiple inheritances, as a class can implement more than one interface. The only requirement is that the implementation class must define all the methods declared in the interface which the class implements.

Let's consider a situation where you have to write a Java program to connect to a database. For this purpose, you write the `ABC` class that provides connectivity to the Oracle database and another class `XYZ` that provides connectivity to the SQL Server database, as shown in the following code snippet:

```
// Class to connect to the Oracle
Class ABC
{
    void connect()
    {
        // code to connect to the Oracle database
    }
}
// Class to connect to the SQL Server
Class XYZ
{
    void connect()
    {
        // code to connect to the SQL Server database
    }
}
```

In the preceding code snippet, both the classes `ABC` and `XYZ` have limitations. The `ABC` class can only connect to the Oracle database and the `XYZ` class can only connect to the SQL Server database. In other words, you cannot use these classes to connect to any other database. To do that, you have to either change the definition of these classes or create other classes.

Creating several different classes, each with code to connect to different databases, takes a lot of time and effort. However, let's suppose you manage to do this. Now, suppose that by the time you finish creating the classes, the versions of the databases you want to connect to the classes have changed. What would you do then? You are left with no choice but to rewrite the classes and connect them to the new versions of the databases.

To solve this problem, you can create an interface and declare methods in it. The definition of a method declared in an interface is provided within the class that implements the interface. The following code snippet shows how to create an interface:

```
interface MyInterface
{
    //All the methods of an interface are by default public
    void connect();
}
```

The preceding code snippet creates the `MyInterface` interface, which declares the `connect()` method. However, you cannot instantiate an interface in Java because an interface does not contain method definition. Therefore, to use an interface, you have to implement it in a class. The following code snippet shows how to implement an interface:

```
class OracleImp implements MyInterface
{
    public void connect()
    {
        // code to connect to the Oracle server database
    }
}
```

In the preceding code snippet, the `OracleImp` class implements the `MyInterface` interface, which is created to connect to the Oracle database. Similarly, if you want to connect to the SQL Server database, you can use another implementation class `SQLImp`, which contains the code to connect to the SQL Server database. The following code snippet shows how to create a class that connects to the SQL Server database:

```
class SQLImp implements MyInterface
{
    public void connect()
    {
        // code to connect to the SQL server database
    }
}
```

In both the `OracleImp` and `SQLImp` classes, the `connect()` method can be defined according to the statements required to connect to the respective databases. To connect to a database, database vendors provide drivers. These drivers vary from database to database; therefore, the types of drivers used to connect to the Oracle database are quite different from the drivers specified for SQL Server database. The method definition also varies according to the driver's implementation. The `connect()` method of the `OracleImp` class provides implementation to connect to Oracle database according to the database drivers of Oracle. Similarly, the `connect()` method of the `SQLImp` class provides implementation to connect to the SQL Server database according to the database drivers of SQL Server. Now, both the classes have the same method but different implementation. In this way, you can create other classes and connect them to other databases. All you have to do is to define the `connect()` method in the class according to the specifications of the driver.

Now, to connect to a database, you need to create an instance of the implementation class whose `connect()` method you want to use. Listing 5.5 shows how to create and use the `MyInterface` interface:

Listing 5.5: Showing the Code to Create and Use the `MyInterface` interface

```
interface MyInterface
{
    void connect();
}
class OracleImp implements MyInterface
{
    // ...
}
```

```

    public void connect()
    {
        System.out.println("Successfully connected to Oracle database");
    }
}
class SQLImp implements MyInterface
{
    public void connect()
    {
        System.out.println("Successfully connected to SQL Server database");
    }
}
public class DemoInterface
{
    public static void main(String args[]) throws Exception
    {
        Class c=Class.forName(args[0]);
        MyInterface obj=(MyInterface)c.newInstance();
        obj.connect();
    }
}

```

Listing 5.5 accepts the name of a class at runtime to connect to a database. The name is then accepted by the `forName()` method that belongs to the `Class` class, which is present in the `java.lang` package. The `forName()` method returns the name of the `Class` object, which is passed at runtime. The `Class` object is then used to access the `connect()` method of the class.

In Listing 5.5, the `connect()` method is declared in the `MyInterface` interface. The `OracleImp` and `SQLImp` classes implement the `MyInterface` interface and provide the definition of the `connect()` method in their own terms. In short, an interface provides empty methods, which are defined in the implementation classes according to the user's requirements. Figure 5.5 shows the output of Listing 5.5:

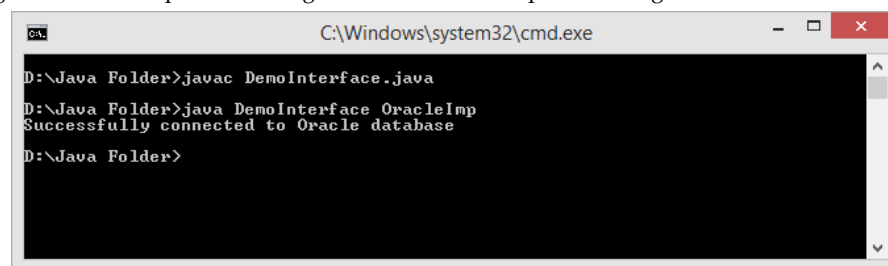


Figure 5.5: Output of Listing 5.5

It can be seen in Figure 5.5 that the `OracleImp` class is passed at runtime and accordingly its `connect()` method is invoked and the statement specified in the method is displayed. Similarly, if you pass the `SQLImp` class at runtime, the `connect()` method of the class is invoked. This is how interfaces are used in Java. Let's introduce a new concept of Java SE 8, that is, lambda expressions.

Working with Lambda Expressions

A lambda expression refers to a method that has no name; no access specifier `private`, `public`, or `protected`; and no value return declaration. This type of method is also known as anonymous method or closure. Like anonymous class, a lambda expression can be used for performing a task without a name. It is a new and significant feature that is included in Java SE 8. Through lambda expressions, functionality can be passed as method arguments and code as data.

NOTE

An anonymous class does not have a name, but an object of this class can be created.

Consider the following code snippet for understanding the concept of lambda expression, in which a simple method is created for showing a message. Let's first declare a simple message as:

```
public void showMsg()
{
    System.out.println("Dreamtech Press");
}
```

Now, convert the preceding method into a lambda expression by removing the `public` access specifier, return type declaration, such as `void`, and the name of method '`showMsg`'. The lambda expression is shown as follows:

```
() -> {System.out.println("Dreamtech Press"); }
```

The basic structure of a lambda expression comprises:

- ❑ Formal parameters separated by commas and enclosed in parentheses.
- ❑ Arrow token, `->`
- ❑ A single expression or a statement block

If a single expression is specified, the Java runtime environment automatically evaluates the expression and then returns its value. You can also use a `return` statement, but it should be enclosed in braces as it is not an expression.

You know that everything is treated as object in Java; therefore, objects are in large numbers therein. Performing operations on such large number of objects is very difficult. To handle operation on large number of objects, Java provides `java.util` package in which various objects are stored as a single object, known as collection object. You can perform operations on various objects by using the loops such as `for-each` loop. However, this loop will allow you to handle the object separately and not collectively. Therefore, you need to improve the functional aspect, which can be easily done by using lambda expressions as these are treated as objects by the JVM.

Method References

As discussed earlier, you can use lambda expressions for creating anonymous methods. Sometimes, a lambda expression is used only for calling an existing method by using its name but not for performing operations. Lambda expressions use method references that are compact and easy-to-read for calling existing methods by their names. There are mainly four different types of method references, as shown in Table 5.1:

Table 5.1: Different Types of Method References	
Types of References	Example
Referencing a static method	<code>ContainingClass::staticMethodName</code>
Referencing an instance method of a particular object	<code>containingObject::instanceMethodName</code>
Referencing an instance method of an arbitrary object of a particular type	<code>ContainingType::methodName</code>
Referencing to a constructor	<code>ClassName::new</code>

Functional Interface

A functional interface is an interface in which there is exactly one abstract method. Some examples of functional interface are `java.util.comparator`, `java.lang.Runnable`, `java.awt.event.ActionListener`, `java.util.concurrent.Callable`, `java.security.PrivilegedAction`, etc. are some examples of functional interface. In Java 8, various functional interfaces are introduced that can be extensively used in lambdas expressions. Table 5.2 gives the list of functional interfaces in `java.util.function` packages:

Table 5.2: List of Functional Interface in java.util.Function:

Functional Interface	Description
<code>BiConsumer<T,U></code>	It represents an operation which accepts two input arguments, but returns no result
<code>BiFunction<T,U,R></code>	It represents a function which accepts two arguments and produces a result
<code>BinaryOperator<T></code>	It represents an operation upon two operands of the same type, producing a result of the same type as the operands
<code>BiPredicate<T,U></code>	It represents a predicate (i.e., Boolean-valued function) of two arguments
<code>BooleanSupplier</code>	It represents a supplier of Boolean-valued results
<code>Consumer <T></code>	It represents an operation which accepts a single input argument but returns no result
<code>DoubleBinaryOperator</code>	It represents an operation upon two double-valued operands and producing a double-valued result
<code>DoubleConsumer</code>	It represents an operation that accepts a single double-valued argument but returns no result
<code>DoubleFunction<R></code>	It represents a function that accepts a double-valued argument and produces a result
<code>Function<T,R></code>	It represents a function that accepts one argument and produces a result
<code>IntFunction<R></code>	It represents a function that accepts an int-valued argument and produces a result
<code>LongFunction<R></code>	It represents a function that accepts a long-valued argument and produces a result

Now that you have the concepts behind inheritance, interfaces, inner classes, and lambda expressions, it is time to turn to the section on using lambda expressions.

Using Lambda Expressions

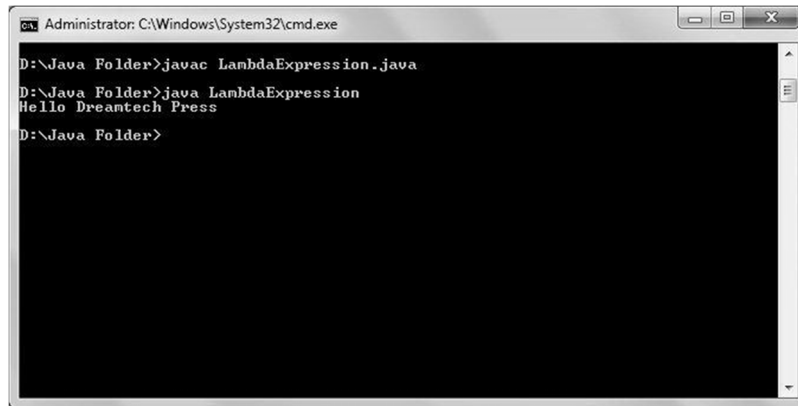
After writing a lambda expression, you can call and execute it like a method. You can call a lambda expression by creating a functional interface, which is an interface comprising only one abstract method. However, it may have multiple static or default methods. The advantage of using functional interface is that it can be used for assigning a lambda expression or as a method reference. Listing 5.6 shows an example in which a lambda expression is created and then it is assigned to an object of a functional interface:

Listing 5.6: Showing the code to create lambda expression and assigned to an object of a functional interface

```

class LambdaExpression
{
    //functional interface
    interface FuncInterface
    {
        void showMsg();
    }
    public static void main(String args[])
    {
        FuncInterface fi = () -> { System.out.println("Hello Dreamtech Press"); };
        fi.showMsg();
    }
}

```

A screenshot of a Windows command prompt window titled "Administrator: C:\Windows\System32\cmd.exe". The window shows the following commands and output:

```
D:\Java Folder>javac LambdaExpression.java
D:\Java Folder>java LambdaExpression
Hello Dreantech Press
D:\Java Folder>
```

Figure 5.6: Output of Listing 5.6

Let's now explore abstract classes and methods in the next section.

Implementing Abstract Classes and Methods

A method without a definition is known as an abstract method, while an abstract class is a class that has at least one abstract method. A method is declared abstract when it needs to be overridden in its subclasses. For example, a programmer can define an abstract method `colour()` in the `Car` class as different cars have different colors. Therefore, each subclass of the `Car` class can override and implement the abstract `colour()` method in its own way.

Java allows us to define abstract methods and classes by using the `abstract` keyword. An abstract method consists of signature declaration followed by a semicolon. An abstract class cannot be instantiated, that is, you cannot create objects of an abstract class.

To use the method declared in an abstract class, the abstract class should be extended by the Java class that uses the abstract class. The Java class that extends (uses) the abstract class should provide the implementation (that is, a method body) of all the methods declared in the abstract class. Listing 5.7 shows the creation of the `Car` abstract class and abstract `colour()` method to help you understand the concepts of abstract classes and methods:

Listing 5.7: Implementing the Concepts of Abstract Classes and Methods

```
abstract class Car
{
    abstract void colour(String a);
}
class SubClass extends Car
{
    public void colour (String a)
    {
        System.out.println("My favourite car colour is:"+ a);
    }
}
class SubClass1 extends Car
{
    public void colour (String a)
    {
        System.out.println("My favourite car colour is:"+ a);
    }
}
public class MainClass2
{
    public static void main(String args[])
    {
        SubClass obj=new SubClass();
```



```

        obj.colour("Blue");
        SubClass1 obj1=new SubClass1();
        obj1.colour("Green");
    }
}

```

In Listing 5.7, you have created an abstract class named `Car` in which the `colour()` abstract method is declared. In addition, you have created two more classes, `SubClass` and `SubClass1`, which extend the `Car` class. These classes override the `colour()` method of the `Car` class to provide their own definition. Apart from this, a class named `MainClass2` is created in which objects of the `SubClass` and `SubClass1` classes are created. These objects are used to access the `colour()` method defined in the `SubClass` and `SubClass1` classes. Figure 5.7 shows the output of Listing 5.7:

Figure 5.7: Output of Listing 5.7

Now that you know about abstract classes and how they are used in Java, let's discuss the differences between an abstract class and an interface.

Difference between Abstract Classes and Interfaces

Both the abstract classes and interfaces contain methods without definition. This poses a problem for users because they may not be sure where to use an abstract class and where to use an interface. To remove confusion, let's discuss the differences between the abstract class and interface. Table 5.3 discusses the basic differences between an abstract class and an interface:

Table 5.3: Differences between an Abstract Class and an Interface	
Abstract Class	Interface
Represents an IS-A relationship between classes. For example, BMW is a car, which means that BMW is a subclass of its superclass Car. This kind of relationship is called an IS-A relationship.	Signifies no IS-A relationship between the classes.
Contains both declared and defined methods.	Contains only declared methods.
Possesses the <code>abstract</code> keyword before the class keyword.	Consists of the <code>interface</code> keyword.
Uses the <code>extends</code> keyword to extend an abstract class in its subclass. A subclass can extend only one abstract class.	Uses the <code>implements</code> keyword to apply an interface in a class. A class can implement as many interfaces as required. Therefore, interfaces are considered an alternative of multiple inheritances, which is not supported by Java.

By keeping these differences in mind, you can easily decide where to use an abstract class and where to use an interface.

Let's discuss another important OOP concept known as polymorphism.

Implementing Polymorphism

The word polymorphism is derived from two Greek words *poly*, which means many, and *morphs*, which means forms. Therefore, the meaning of the word polymorphism is many forms. In OOP, polymorphism allows you to

perform various operations by using methods with the same names. In Java, polymorphism is performed by changing the implementation of the method. Polymorphism can be divided into the following two types:

- ❑ Static
- ❑ Dynamic

The type of polymorphism depends on how the method is invoked in a class. In dynamic polymorphism, the behavior of a method is decided during runtime, whereas in static polymorphism, it is decided during compilation.

Static polymorphism is also known as early binding because binding is performed during compilation. Dynamic polymorphism is also known as late binding because binding occurs during runtime, depending on the type of object.

Let's discuss the two types of polymorphism in detail.

Understanding the Static Polymorphism

It is aforementioned that polymorphism that occurs at the time of compilation is called static polymorphism. In static polymorphism, the Java compiler binds the method calls with method code at compile time. This type of polymorphism is also known as compile-time polymorphism.

In Java, static polymorphism is achieved through method overloading, which is the mechanism in which you can define various methods with the same name. However, there is one condition: these methods should have different signatures. Method signatures represent the method name, the number of parameters the method can accept, and the type of method parameters. The Java compiler differentiates the various methods with the same names by their method signatures. The difference in method signatures is due to the following reasons:

- ❑ Difference in the number of parameters passed with methods having the same name, as shown in the following code snippet:

```
void add(int x, int y)
void add(int x, int y, int z)
```

According to the preceding code snippet, the first `add()` method has two parameters and the second `add()` method has three parameters. During compilation, the Java compiler binds the definition of the `add(int x, int y)` method with the method call `add(10,12)` that has two `int` type arguments. However, the `add(1,2,3)` method call with three `int` type arguments is bound to the `add(int x, int y, int z)` method.

- ❑ Difference in the sequence or order of parameters, as shown in the following code snippet:

```
void add(int a, float b)
void add(float a, int b)
```

According to the preceding code snippet, the Java compiler binds the `add()` method definition depending on the sequence of parameters.

- ❑ Difference in datatypes of parameters passed in the methods, as shown in the following code snippet:

```
void add(int x, int y)
void add(float x, float y)
```

In this case, the first `add()` method has two parameters of datatype `int` and the second `add()` method also has two parameters, but of datatype `float`. During compilation, the Java compiler binds the definition of the `add(int x, int y)` method with the method call `add(10,20)`. However, the method call `add(10.5,12.5)` with the `float` type arguments is bound to the `add(float x, float y)` method.

The Java compiler matches the values passed to a method during compilation and binds the method call with the appropriate method definition. Listing 5.8 shows how various methods with the name `product` are created to help you understand the way static polymorphism is implemented:

Listing 5.8: Implementing the Static Polymorphism

```
class StaticPoly
{
    void product(int x, int y)
```

```

    {
        System.out.println("Product of two numbers:"+(x*y));
    }
    void product(int x, int y, int z)
    {
        System.out.println("Product of three numbers:"+(x*y*z));
    }
    public static void main(String args[])
    {
        StaticPoly obj=new StaticPoly();
        obj.product(20,10);
        obj.product(5,6,7);
    }
}

```

In Listing 5.8, you have created a class named `StaticPoly`, wherein you have created two methods with the same name `product`. Both the methods have different signatures. The signatures vary in the number of parameters associated with each method. The first `product()` method accepts two parameters, while the second `product()` method accepts three parameters. Further, an object `obj` of the `StaticPoly` class is created, which is used to invoke both overloaded `product()` methods. In the first and second `product()` method calls, you have passed two and three values, respectively. On the basis of the number of values passed in the method calls, the Java compiler invokes the appropriate method. Figure 5.8 shows the output of Listing 5.8:

```

C:\Windows\system32\cmd.exe
D:\Java Folder>javac StaticPoly.java
D:\Java Folder>java StaticPoly
Product of two numbers:200
Product of three numbers:210
D:\Java Folder>

```

Figure 5.8: Output of Listing 5.8

It can be seen in Figure 5.8 that the product of two numbers is displayed by invoking the `product()` method, which has two arguments. The product of three numbers is also displayed in the output which is evaluated by invoking the `product()` method with three arguments. This is possible because of the difference in the signature of the `product()` methods or static polymorphism.

Let's learn about dynamic polymorphism next.

Understanding the Dynamic Polymorphism

Polymorphism exhibited at runtime is called dynamic polymorphism. This means that when a method is called, the method call is bound to the method body during runtime or execution of the program. The Java compiler is not aware of the method to be invoked during compilation; therefore, Java Virtual Machine (JVM) invokes the relevant method during runtime. Methods are invoked at runtime because they are called using the class objects and objects are created at runtime. In Listing 5.9, a method named `calc` is created in classes A and B to help you understand implementation of the dynamic polymorphism:

Listing 5.9: Implementing the Dynamic Polymorphism

```

class A
{
    static void calc(double x)
    {
        System.out.println("Square of the given value: "+(x*x));
    }
}
class B extends A

```

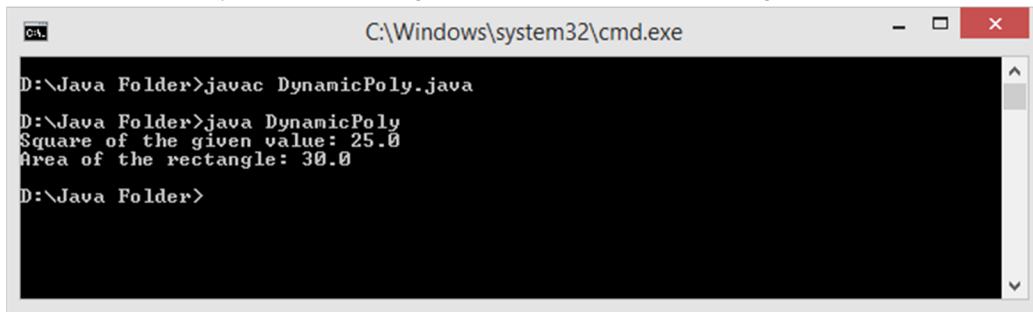
```

{
    static void calc(double x)
    {
        double y=5;
        System.out.println("Area of the rectangle: "+ (x*y));
    }
}
public class DynamicPoly
{
    public static void main(String args[])
    {
        A a=new B();
        a.calc(5);
        B b = new B();
        b.calc(6);
    }
}

```

In Listing 5.9, you have created two classes named A and B, respectively, and a method named `calc()` in each of these classes. The `calc()` method in class A is used to calculate the square of a given value, whereas the `calc()` method of class B is used to calculate the area of a rectangle. Further, you have created a class named `DynamicPoly`, and in the `main()` method of the `DynamicPoly` class, you have created an object `a` of class A, which has reference to class B. Object `a` is used to invoke the `calc()` method of class A. Next, an object `b` of class B is created, which is used to invoke the `calc()` method of class B.

Note that the `calc()` method in the A and B classes has the same name and signature. Therefore, the Java compiler cannot bind the methods to their definitions and JVM decides the suitable method to execute during runtime, based on the objects of the class. Figure 5.9 shows the output of Listing 5.9:



```

C:\Windows\system32\cmd.exe
D:\Java Folder>javac DynamicPoly.java
D:\Java Folder>java DynamicPoly
Square of the given value: 25.0
Area of the rectangle: 30.0
D:\Java Folder>

```

Figure 5.9: Output of Listing 5.9

With this, you have come to the end of this chapter. Before closing this chapter, let's go through the summary to recap the main topics covered in this chapter.

Summary

In this chapter, you have learnt the principles of OOP, such as encapsulation, abstraction, inheritance, and polymorphism along with their types. You have also learnt how to use `final` keyword for preventing a class from being inherited by another class, declaring a constant variable, and preventing a method from being overridden. Further you have learnt about interfaces and abstract classes in Java. You have also known the differences between abstract classes and interfaces. Finally, you have learnt about the lambda expressions, a new feature introduced in Java SE 8.



6

Working with Streams, Files, and I/O Handling

<i>If you need information on:</i>	<i>See page:</i>
Streams, Readers, and Writers	136
NIO	136
Essentials in NIO	136
Enhancements in NIO with Java 8	140
Prospects of NIO	149
Working with Streams	149
Working with the Reader Class	159
Working with the Writer Class	160
Accepting Input from the Keyboard with the InputStreamReader Class	160
Working with the OutputStreamWriter Class	162
Working with Files	162
Working with the RandomAccessFile Class	168
Working with Character Arrays	170
Working with Buffers	173
Working with the PushbackReader Class	175
Working with the PrintWriter Class	176
Working with the StreamTokenizer Class	178
Implementing the Serializable Interface	180
Working with the Console Class	184
Working with the Clipboard	186
Working with the Printer	186
Printing with the Formatter Class	187
Scanning Input with the Scanner class	190
Summary	190

In this chapter, we'll take a look at stream, I/O, and file handling in Java. Using streams is the way you handle I/O in languages such as Java. A stream is composed of bytes or characters (which are used to be considered much the same in Java until Unicode and its 2-byte characters arrived) that you can read from or write to, and you can connect them to various data sources and stores—even byte and character arrays.

Streams, Readers, and Writers

The concept of streams was introduced to make programming more uniform for handling disparate data sources or stores. When using streams, you use the `java.io` package, and you can handle input and output in much the same way—whether you are working with a disk file, a memory buffer, or a network.

In Java, stream I/O is based on the `InputStream`, `OutputStream`, `Reader`, and `Writer` classes. `InputStream` and `OutputStream` are for byte-oriented streams (such as when you work with binary files), and the `Reader` and `Writer` classes are for character-oriented streams (as when you work with text files). You don't use these classes directly; instead, you derive classes from them. In fact, Java has already done a lot of that for you so much, so that there are dozens of classes to work with.

In fact, the I/O stream classes have become somewhat of a thicket in Java—there seems to be at least one available for every possible use and some of you probably haven't thought of. Some streams provide data buffering, some do not. Some are designed to work with data in memory buffers, some with files. We'll cover the important ones here.

Typically, you use a stream class's constructor to create a stream; the stream classes are usually divided into input and output streams, so you select the one that's appropriate. After you have a stream object, you can use its methods—typically `read()` and `write()`—to transfer the data. When you are done with the stream object, you typically use its `close()` method to close it and end your work with the data source or store.

NIO

Initially, New Input Output, also known as NIO, was introduced with the J2SE 1.4 by Sun Microsystems. It is a collection of Java programming language Application Programming Interface (API) that provides features for performing I/O operations.

The APIs of NIO were created for providing accessibility to the low-level I/O operations. The objective of creating such high-level APIs is to enable an implementation that can easily utilize the most effective operations of the existing platform.

The `java.nio` package and its sub-packages contain the Java NIO APIs. Some of their features are as follows:

- ❑ Buffers for the primitive type data
- ❑ Character set encoders and decoders
- ❑ Pattern matching on the basis of Perl-style regular expressions (in package `java.util.regex`). A new primitive I/O abstraction provisionA file interface inclusion, which supports locking and memory mapping of files up to `Integer.MAX_VALUE` bytes (2 GiB)
- ❑ A multiplexed, non-blocking I/O facility for writing scalable servers

Essentials in NIO

NIO is built on two fundamental items—buffers and channels. These are used in data handling—a buffer holds data, while a channel represents an open connection to an I/O device, such as a file or a socket. Generally, to use the NIO system, you obtain a channel to an I/O device and a buffer to hold that data. Operating can be done on the buffer by providing input or output data.

Buffers

Buffers are the integral part of the `java.nio` package. All buffers are subclasses of the `Buffer` class. The `Buffer` class defines the core functionality—current position, limit, and capacity. The current position is the index in the buffer where the next read or write operation takes place. The current position is advanced by most

read and write operations. The limit in the index is the end of the buffer. The capacity is the number of elements that the buffer can hold.

The `Buffer` class also supports the `mark()` and `reset()` methods. It defines several methods, which are shown in Table 6.1, and Table 6.2 shows the specific buffer sub classes derived from the `Buffer` class:

Table 6.1: The methods of the `Buffer` class

Method	Description
<code>abstract Object array()</code>	It returns an array backing this buffer.
<code>abstract int arrayOffset()</code>	It returns the offset of the first element of the buffer in backing array.
<code>int capacity()</code>	It returns the number of elements that the invoking buffer is capable of holding.
<code>Buffer clear()</code>	It clears the invoking buffer and returns a reference to the buffer.
<code>Buffer flip()</code>	It sets the invoking buffer's limit to the current positions and resets the current positions to 0. It returns a reference to the buffer.
<code>abstract boolean hasArray()</code>	It returns true if the buffer is backed by some accessible array.
<code>boolean hasRemaining()</code>	It returns true if there are elements remaining in the invoking buffer; otherwise, it returns false.
<code>abstract boolean isDirect()</code>	It returns true if the buffer is direct.
<code>abstract boolean isReadOnly()</code>	It returns true if the invoking buffer is read only; otherwise, it returns false.
<code>int limit()</code>	It returns the invoking buffer's limit.
<code>Buffer limit(int n)</code>	It sets the invoking buffer limit to n. It returns a reference to the buffer.
<code>Buffer mark()</code>	It sets the mark and returns a reference to the invoking buffer.
<code>int position()</code>	It returns the current positions.
<code>Buffer position(int n)</code>	It sets the invoking buffer's current position to n. It returns a reference to the buffer
<code>int remaining()</code>	It returns the number of elements between current position and the limit.
<code>Buffer reset()</code>	It resets the current position of the invoking buffer to the previously set mark. It returns a reference to the buffer.
<code>Buffer rewind()</code>	It sets the positions of the invoking buffer to 0. It returns a reference to the buffer.

Table 6.2: The specific buffer subclasses derived from the `Buffer` class

<code>ByteBuffer</code>	<code>CharBuffer</code>
<code>DoubleBuffer</code>	<code>FloatBuffer</code>
<code>IntBuffer</code>	<code>LongBuffer</code>
<code>MappedByteBuffer</code> (a subclass of <code>ByteBuffer</code> ; used to map a file to a buffer)	<code>ShortBuffer</code>

All buffers support methods that perform different operations. For example, the `allocate()` method is used for allocating a buffer manually. On the other hand, the `wrap()` method is used for wrapping an array inside a buffer. Moreover, a subsequence of a buffer can be created by using the `slice()` method. The `Buffer` class also supports various `get()` and `put()` methods, which allow you to get data from a buffer or put data into a buffer. The `get()` and `put()` methods are described in Table 6.3:

Table 6.3: The get() and put() methods

Method	Description
<code>abstract byte get()</code>	It returns the byte at the current position.
<code>ByteBuffer get(byte vals[])</code>	It copies the invoking buffer into the array referred to by vals. It returns a reference to the buffer.
<code>ByteBuffer get(byte vals[], int start, int num)</code>	It copies num elements from the invoking buffer into the array referred to by vals, beginning from the index specified by start. It returns a reference to the buffer. If there are not num elements remaining in the buffer, a <code>BufferUnderflowException</code> exception is thrown.
<code>abstract byte get(int idx)</code>	It returns byte at the index specified by idx within the invoking buffer.
<code>abstract ByteBuffer put(byte b)</code>	It copies b into the invoking buffer at the current position. It returns a reference to the buffer.
<code>final ByteBuffer put(byte vals[])</code>	It copies all elements of vals into the invoking buffer, beginning from the current position. It returns a reference to the buffer.
<code>ByteBuffer put(byte vals[], int start, int num)</code>	It copies num elements from vals, beginning from start, into the invoking buffer. It returns a reference to the buffer if the buffer cannot hold all the elements a <code>BufferOverflowException</code> is thrown.
<code>ByteBuffer put(ByteBuffer bb)</code>	It copies the elements in bb to the invoking buffer, beginning from the current position. If the buffer cannot hold all of the elements, a <code>BufferOverflowException</code> is thrown. It returns a reference to the buffer
<code>abstract ByteBuffer put(int idx, byte b)</code>	It copies b into the invoking buffer at the location specified by idx. It returns a reference to the buffer.

Channels

A channel represents an open connection to an I/O source or destination and is defined in the `java.nio.channels` package.

A channel can be retrieved by using the `getChannel()` method on an object supporting channels. Table 6.4 shows its I/O classes:

Table 6.4: The I/O classes supported by the getChannel() method

<code>DatagramSocket</code>	<code>FileInputStream</code>
<code>FileOutputStream</code>	<code>RandomAccessFile</code>
<code>ServerSocket</code>	<code>Socket</code>

To obtain a channel, you first obtain an object of one of these classes and then call `getChannel()` on that object. The type of channel returned totally depends upon the type of object on which the `getChannel()` method is called.

For example, when the `getChannel()` method is called using a `FileInputStream`, `FileOutputStream`, or `RandomAccessFile` object, a channel of type `FileChannel` is returned. When called on a `Socket`, `getChannel()` method returns a `SocketChannel`.

Channels such as `FileChannel` and `SocketChannel` support various `read()` and `write()` methods that allow you to perform I/O operations through the channel. For example, a few of the `read()` and `write()` methods defined for `FileChannel` are listed in Table 6.5 in which all can throw an `IOException`:

Table 6.5: The read() and write() methods of the FileChannel class

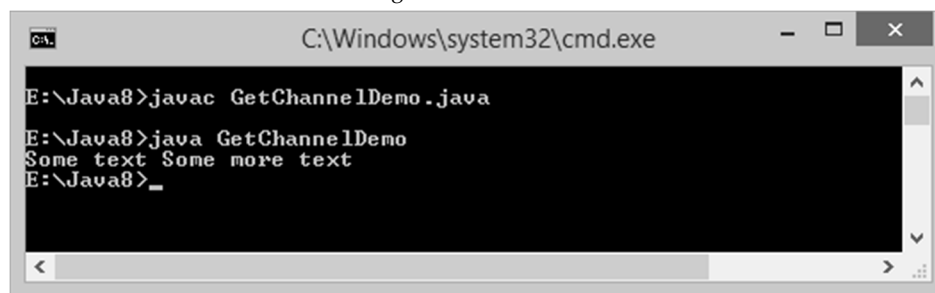
Method	Description
<code>abstract int read(ByteBuffer bb)</code>	It reads bytes from the invoking channel into bb until the buffer is full or there is no more input. It returns the number of bytes actually read.
<code>abstract int read(ByteBuffer bb, long start)</code>	Beginning at the file location specified by start, read bytes from the invoking channels into bb until the buffer is full or there is no more input. The current position is unchanged. It returns the number of bytes actually read or -1 if start is beyond the end of the file.
<code>abstract int write(ByteBuffer bb)</code>	It writes the contents of bb to the invoking channel, starting at the current position. It returns the number of bytes written.
<code>abstract int write(ByteBuffer bb, long start)</code>	Beginning at the file location specified by start, writes the contents of bb to the invoking channel. The current position is unchanged. It returns the number of bytes written.

The methods are supported by all channels that let access to and control over them. For example, the `map()` method is supported by `FileChannel` which allows you to map a file to a buffer. Here's a simple example that uses all three types of stream to produce channels that are writeable, readable/writeable, and readable:

```
import java.io.*;
import java.nio.*;
import java.nio.channels.*;

public class GetChannelDemo {
    private static final int BSIZE = 1024;
    public static void main(String[] args) throws Exception
    {
        FileChannel filech1 = new FileOutputStream("file6.txt").getChannel();
        filech1.write(ByteBuffer.wrap("Some text ".getBytes()));
        filech1.close();
        filech1 = new RandomAccessFile("file6.txt", "rw").getChannel();
        filech1.position(filech1.size());
        filech1.write(ByteBuffer.wrap("Some more text".getBytes()));
        filech1.close();
        filech1 = new FileInputStream("file6.txt").getChannel();
        ByteBuffer buff = ByteBuffer.allocate(BSIZE);
        filech1.read(buff);
        buff.flip();
        while(buff.hasRemaining())
            System.out.print((char)buff.get());
    }
}
```

Create a file named `file6.txt` before executing this code file in the same location where you are saving Java and the class files. The result is as shown in Figure 6.1:

**Figure 6.1: The output of the GetChannelDemo class**

Charsets and Selectors

NIO supports two more entities—charsets and selectors. A charset allows mapping of bytes to characters. An encoder encodes a sequence of characters into bytes, while a decoder decodes a sequence of bytes into characters. The classes defined in the `java.nio.charset` package support charsets, encoders, and decoders.

Selectors allow you to execute I/O operations through multiple channels. The classes defined in the `java.nio.channels` package support selectors. There are three sets of keys maintained by a selector: key set, selected-key set, and cancelled-key set.

Each time a channel is registered with a selector, a selection key is created. A selection key encapsulates two operation sets—Interest Set and Ready Set.

Enhancements in NIO with Java 8

Several improvements have been made to the standard `java.nio.charset.Charset` and extended charset implementations. These improvements are as follows:

- ❑ Inclusion of a new `SelectorProvider` implementation for Solaris operating system on the basis of the Solaris event port technique. This implementation is expected to enhance performance or scalability for some server workloads. The `/dev/poll` `SelectorProvider` continues to be the default. You can use the Solaris event port technique by running with the system property `java.nio.channels.spi.Selector`, whose value is set to `sun.nio.ch.EventPortSelectorProvider`. Reduction in the size of `<JDK_HOME>/jre/lib/charsets.jar` file
- ❑ Performance improvement for the `java.lang.String(byte[], *)` constructor and the `java.lang.String.getBytes()` method.

The Path Interface

The `Path` acts as an object that may be used to locate a file in the file system. Usually, it represents a system-dependent file path. The `Path` also represents a path that is categorized and composed of an arrangement of directory and file name elements separated by a special separator or delimiter. The root element that identifies a file system hierarchy should be present. The name element that is farthest from the `root` element of the hierarchy is the name of the file or directory and the other name elements are directory names. The `Path` represents a root and an arrangement of names, or simply one or more name elements. The `Path` is considered to be an empty path if its only name element is empty. Accessing a file using an empty path is similar to accessing the default directory of the file system. The `Path` interface defines the `getFileName()`, `getParent()`, `getRoot()`, and `subpath()` methods to access the path elements or a subsequence of its name elements.

Paths are utilized with the `Files` class for performing operations on files, directories, and other kinds of files. For example, suppose we want a `BufferedReader` object to read text from a file "file.txt" and the file is located in a directory "info" relative to the current working directory and is UTF-8 encoded. To implement this, the following code snippet should be used:

```
Path path = FileSystems.getDefault().getPath("info", "file.txt");
BufferedReader reader = Files.newBufferedReader(path, StandardCharsets.UTF_8);
```

The methods of the `Path` interface class are provided in Table 6.6:

Table 6.6: Methods of the Path interface	
Method	Does this
<code>int compareTo(Path other)</code>	It compares two abstract paths lexicographically.
<code>boolean endsWith(Path other)</code>	It checks whether or not the resulting path ends with the given path.
<code>boolean endsWith(String other)</code>	It checks if this path ends with a <code>Path</code> constructed by converting the given path string in exactly the same manner specified by the <code>endsWith(Path)</code> method.

Table 6.6: Methods of the Path interface

Method	Does this
<code>boolean equals(Object other)</code>	It checks whether or not the given object is a path.
<code>Path getFileName()</code>	It returns the name of the file or directory denoted by this path as a <code>Path</code> object.
<code>FileSystem getFileSystem()</code>	It returns the file system that created this object.
<code>Path getName(int index)</code>	It returns a name element of this path as a <code>Path</code> object.
<code>int getNameCount()</code>	It returns the number of name elements in the path.
<code>Path getParent()</code>	It returns the parent path or null if this path does not have a parent.
<code>Path getRoot()</code>	It returns the root component of this path as a <code>Path</code> object or null if this path does not have a root component.
<code>int hashCode()</code>	It computes a hash code for this path.
<code>boolean isAbsolute()</code>	It states whether or not this path is absolute.
<code>Iterator<Path> iterator()</code>	It returns an iterator over the name elements of this path.
<code>Path normalize()</code>	It returns a normalized path in which redundant name of elements are eliminated.
<code>WatchKey register(WatchService watcher, WatchEvent.Kind<?>... events)</code>	It registers the file located by this path with a watch service.
<code>WatchKey register(WatchService watcher, WatchEvent.Kind<?>[] events, WatchEvent.Modifier... modifiers)</code>	It registers the file located by this path with a watch service. The parameters of this method defines modifications that qualify how the directory is registered.
<code>Path relativize(Path other)</code>	It creates a relative path between this path and a given path.
<code>Path resolve(Path other)</code>	It resolves the given path against this path.
<code>Path resolve(String other)</code>	It translates a given path string to a <code>Path</code> and resolves it against this <code>Path</code> in exactly the same manner specified by the <code>resolve</code> method.
<code>Path resolveSibling(Path other)</code>	It resolves the given path against resulting path's parent path.
<code>Path resolveSibling(String other)</code>	It translates a given path string to a <code>Path</code> and resolves it against this path's parent path in exactly the same manner specified by the <code>resolveSibling</code> method.
<code>boolean startsWith(Path other)</code>	It checks if this path starts with the given path.
<code>boolean startsWith(String other)</code>	It checks if this path starts with a <code>Path</code> constructed by converting the given path string in exactly the same manner specified by the <code>startsWith(Path)</code> method.
<code>Path subpath(int beginIndex, int endIndex)</code>	It returns a relative <code>Path</code> that is a subsequence of the name elements of this path.
<code>Path toAbsolutePath()</code>	It returns a <code>Path</code> object representing the absolute path of this path.
<code>File toFile()</code>	It returns a <code>File</code> object representing this path.
<code>Path toRealPath(LinkOption... options)</code>	It returns the real path of an existing file.
<code>String toString()</code>	It returns the string representation of this path.
<code>URI toUri()</code>	It returns a <code>URI</code> to represent this path.

The **Files** Class

The `Files` class comprises static methods, exclusively, which perform operations on files, directories, or other kinds of files. The inheritance diagram for the `Files` class is as follows:

```

java.lang.Object
|___java.nio.file.Files

```

The `Files` class exposes the methods described in Table 6.7:

Table 6.7: Methods and their description

Table 6.7: Methods of the Files class	
Method	Does this
<code>static long copy(InputStream in, Path target, CopyOption... options)</code>	It copies all bytes from an input stream to a file.
<code>static long copy(Path source, OutputStream out)</code>	It copies all bytes from a file to an output stream.
<code>static Path copy(Path source, Path target, CopyOption... options)</code>	It copies a file to a target file.
<code>static Path createDirectories(Path dir, FileAttribute<?>... attrs)</code>	It generates a directory by creating all nonexistent parent directories first.
<code>static Path createDirectory(Path dir, FileAttribute<?>... attrs)</code>	It generates a new directory.
<code>static Path createFile(Path path, FileAttribute<?>... attrs)</code>	It generates a new and empty file; if the file already exists, then it does not create any file.
<code>static Path createLink(Path link, Path existing)</code>	It generates a new link (directory entry) for an existing file (optional operation).
<code>static Path createSymbolicLink(Path link, Path target, FileAttribute<?>... attrs)</code>	It generates a symbolic link to a target (optional operation).
<code>static Path createTempDirectory(Path dir, String prefix, FileAttribute<?>... attrs)</code>	It generates a new directory in the specified directory using the given prefix to generate its name.
<code>static Path createTempDirectory(String prefix, FileAttribute<?>... attrs)</code>	It generates a new directory in the default temporary-file directory and uses the given prefix to generate its name.
<code>static Path createTempFile(Path dir, String prefix, String suffix, FileAttribute<?>... attrs)</code>	It generates a new empty file in the specified directory and uses the given prefix and suffix strings to generate its name.
<code>static Path createTempFile(String prefix, String suffix, FileAttribute<?>... attrs)</code>	It generates an empty file in the default temporary-file directory using the given prefix and suffix to generate its name.
<code>static void delete(Path path)</code>	It deletes a file.
<code>static boolean deleteIfExists(Path path)</code>	It deletes a file if it exists.
<code>static boolean exists(Path path, LinkOption... options)</code>	It checks whether or not a file exists.
<code>static Object getAttribute(Path path, String attribute, LinkOption... options)</code>	It reads the file attribute value.
<code>static <V extends FileAttributeView> V getFileAttributeView(Path path, Class<V> type, LinkOption... options)</code>	It returns a file attribute view of a given type.

Table 6.7: Methods of the Files class

Method	Does this
<code>static FileStore getFileStore(Path path)</code>	It returns the <code>FileStore</code> object specifying the path where the file is stored.
<code>static FileTime getLastModifiedTime(Path path, LinkOption... options)</code>	It returns a file's last modified time.
<code>static UserPrincipal getOwner(Path path, LinkOption... options)</code>	It returns the owner of a file.
<code>static Set<PosixFilePermission> getPosixFilePermissions(Path path, LinkOption... options)</code>	It returns a file's POSIX file permissions.
<code>static boolean isDirectory(Path path, LinkOption... options)</code>	It checks whether or not a file is a directory.
<code>static boolean isExecutable(Path path)</code>	It checks whether or not a file is executable.
<code>static boolean isHidden(Path path)</code>	It states whether or not a file is considered hidden.
<code>static boolean isReadable(Path path)</code>	It checks whether or not a file is readable.
<code>static boolean isRegularFile(Path path, LinkOption... options)</code>	It checks whether or not a file is a regular file with opaque content.
<code>static boolean isSameFile(Path path, Path path2)</code>	It checks if two paths locate the same file.
<code>static boolean isWritable(Path path)</code>	It checks whether or not a file is writable.
<code>static Path move(Path source, Path target, CopyOption... options)</code>	It moves or renames a file to a target file.
<code>static BufferedReader newBufferedReader(Path path, Charset cs)</code>	It opens a file to read text from it and returns a new <code>BufferedReader</code> object with default buffer size. This <code>BufferedReader</code> object is used to read the content of the file.
<code>static BufferedWriter newBufferedWriter(Path path, Charset cs, OpenOption... options)</code>	It opens or creates a file to write text and returns a new <code>BufferedWriter</code> object with default buffer size. This <code>BufferedWriter</code> object is used to write the text in the file.
<code>static SeekableByteChannel newByteChannel(Path path, OpenOption... options)</code>	It opens or creates a file to access it and returns a seekable byte channel, which is used to access the file.
<code>static SeekableByteChannel newByteChannel(Path path, Set<? extends OpenOption> options, FileAttribute<?>... attrs)</code>	It opens or generates a file, returning a seekable byte channel to access the file by accepting the path and attributes of the file as arguments.
<code>static DirectoryStream<Path> newDirectoryStream(Path dir)</code>	It opens a directory and returns a <code>DirectoryStream</code> to iterate over all entries in the directory.
<code>static DirectoryStream<Path> newDirectoryStream(Path dir, DirectoryStream.Filter<? super Path> filter)</code>	It opens a directory, returning a <code>DirectoryStream</code> to iterate over the entries in the directory. In this method entries are filtered by given filter.
<code>static DirectoryStream<Path> newDirectoryStream(Path dir, String glob)</code>	It opens a directory, returning a <code>DirectoryStream</code> to iterate over the entries in the directory. In this method entries are filtered by matching the String representation.
<code>static InputStream newInputStream(Path path, OpenOption... options)</code>	It opens a file, returning an input stream to read from the file.

Table 6.7: Methods of the Files class

Method	Does this
<code>static OutputStream newOutputStream(Path path, OpenOption... options)</code>	It opens or generates a file, returning an output stream that may be used to write bytes to the file.
<code>static boolean notExists(Path path, LinkOption... options)</code>	It checks whether or not the file located by this path does not exist.
<code>static String probeContentType(Path path)</code>	It probes the content type of a file.
<code>static byte[] readAllBytes(Path path)</code>	It reads all the bytes from a file.
<code>static List<String> readAllLines(Path path, Charset cs)</code>	It reads all lines from a file.
<code>static <A extends BasicFileAttributes> A readAttributes(Path path, Class<A> type, LinkOption... options)</code>	It reads a file's attributes as a bulk operation.
<code>static Map<String, Object> readAttributes(Path path, String attributes, LinkOption... options)</code>	It reads a set of file attributes as a bulk operation.
<code>static Path readSymbolicLink(Path link)</code>	It reads the target of a symbolic link (optional operation).
<code>static Path setAttribute(Path path, String attribute, Object value, LinkOption... options)</code>	It sets the value of a file attribute.
<code>static Path setLastModifiedTime(Path path, FileTime time)</code>	It updates a file's last modified time attribute.
<code>static Path setOwner(Path path, UserPrincipal owner)</code>	It updates the file owner.
<code>static Path setPosixFilePermissions(Path path, Set<PosixFilePermission> perms)</code>	It defines a file's POSIX permissions.
<code>static long size(Path path)</code>	It returns the size of a file (in bytes).
<code>static Path walkFileTree(Path start, FileVisitor<? super Path> visitor)</code>	It walks a file tree rooted at a given starting file. It does not follow symbolic links automatically but when the option parameter contains the FOLLOW_LINKS option then symbolic links are followed by this method.
<code>static Path walkFileTree(Path start, Set<FileVisitOption> options, int maxDepth, FileVisitor<? super Path> visitor)</code>	It walks a file tree rooted at a given starting file. It does not allow to follow symbolic links, and visit all levels of the file tree.
<code>static Path write(Path path, byte[] bytes, OpenOption... options)</code>	It writes bytes to a file.
<code>static Path write(Path path, Iterable<? extends CharSequence> lines, Charset cs, OpenOption... options)</code>	It writes lines of text to a file.

The **Paths** Class

The `Paths` class includes entirely static methods that return a `Path` by changing a path string or URI. This class is introduced in the Java SE 7 release. It is one of the primary features of the `java.nio.file` package. The inheritance diagram for the `Paths` class is as follows:

```
java.lang.Object
|___java.nio.file.Paths
```

The methods of the `Paths` class are listed in Table 6.8:

Table 6.8: Methods of the Paths class	
Method	Does this
<code>static Path get(String first, String... more)</code>	It converts the strings passed as arguments to a Path by joining all the specified strings.
<code>static Path get(URI uri)</code>	It changes the given URI to a Path.

The File Attribute Interfaces

An attribute view provides a read-only view of the transparent values, or metadata, associated with objects in a file system. The `FileAttributeView` interface is extended by several other interfaces that view to other specific sets of file attributes. The `FileAttributeView` interface is selected by requesting the `Files.getFileAttributeView(java.nio.file.Path, java.lang.Class<V>, java.nio.file.LinkOption...)` method with a type-token to identify the required view. You can identify views by name as well.

The `FileStoreAttributeView` interface provides access to file store attributes. The `FileStoreAttributeView` interface of a given type is obtained by invoking the `FileStore.getFileStoreAttributeView(java.lang.Class<V>)` method.

Various interfaces of the file attributes are provided in Table 6.9:

Table 6.9: Interface Summary	
Interface	Does this
<code>AclFileAttributeView</code>	A file attribute view that supports reading or updating a file's ACL or file owner attributes.
<code>AttributeView</code>	An object that provides a read-only or updatable view of non-opaque values associated with an object in a file system.
<code>BasicFileAttributes</code>	Basic attributes associated with a file in a file system.
<code>BasicFileAttributeView</code>	A file attribute view that provides a view of a basic set of file attributes common to many file systems.
<code>DosFileAttributes</code>	File attributes associated with a file in a file system that supports legacy "DOS" attributes.
<code>DosFileAttributeView</code>	A file attribute view that provides a view of the legacy "DOS" file attributes.
<code>FileAttribute<T></code>	An object that encapsulates the value of a file attribute that can be set atomically when creating a new file or directory by invoking the <code>createFile()</code> or <code>createDirectory()</code> methods.
<code>FileAttributeView</code>	An attribute view that is a read-only or updatable view of non-opaque values associated with a file in a filesystem.
<code>FileOwnerAttributeView</code>	A file attribute view that supports reading or updating the owner of a file.
<code>FileStoreAttributeView</code>	An attribute view that is a read-only or updatable view of the attributes of a <code>FileStore</code> .
<code>GroupPrincipal</code>	A <code>UserPrincipal</code> representing a group identity used to determine access rights to objects in a file system.
<code>PosixFileAttributes</code>	File attributes associated with files on file systems used by operating systems that implement the Portable Operating System Interface (POSIX) family of standards.

Table 6.9: Interface Summary

Interface	Does this
<code>PosixFileAttributeView</code>	A file attribute view that provides a view of the file attributes commonly associated with files on file systems used by operating systems that implement the POSIX family of standards.
<code>UserDefinedFileAttributeView</code>	A file attribute view that provides a view of a file's user-defined attributes, sometimes known as extended attributes.
<code>UserPrincipal</code>	A Principal representing an identity used to determine access rights to objects in a file system.

The *FileSystem* Class

The `FileSystem` class is the collection of objects providing an interface, which is used to access files and other objects present in the file system. It is the default file system used with the `FileSystems.getDefault()` method invoked. This method allows accessibility to the file system that can be easily retrieved by the JVM.

The `FileSystem` class defines methods for creating file systems that provide access to other types of file systems.

The `FileSystem` class provides the following types of objects:

- ❑ The `getPath()` method converts the path of a system, which is in `String` format, into a `Path` object. This object is used to locate a file and retrieve its content.
- ❑ The `getPathMatcher()` method creates a `PathMatcher` object, which is used to perform matching operations on paths.
- ❑ The `getFileStores()` method returns an object for performing iteration on the existing file stores.
- ❑ The `getUserPrincipalLookupService()` method returns the `userPrincipalLookupService`, which is used to lookup users or groups by name.
- ❑ The `newWatchService()` method constructs a `WatchService` used to look the objects for changes and events.
- ❑ The `getRootDirectory()` method returns an object for performing iteration on the root directories present in the file system.

The file system sometimes have single hierarchy of files with one top-level root directory or several distinct hierarchy of files, each with its own top-level root directory.

Generally, a file system contains one or more essential file stores that are used to store the files on the system. The file attributes or metadata and features of one file store will differ from the other.

When you create an object of the `FileSystem` class, the file system is opened to access files and can be closed by calling its `close()` method. If the file system is closed, then any attempt to access objects from the files will result into the `ClosedFileSystemException` exception. You cannot close the File systems that are created by the default provider. Whenever you want to close the file system, the `close()` method can be invoked but the way a file system will be closed is provider specific. In other words, the type of provider used with the file system will determine during the runtime how the system will be closed.

A file system provides read-only or read-write operations to retrieve the data from the file system. When the file system is created, you can check whether it provides read-only access by using the `isReadOnly()` method of the file system. When you try to write or change in any file store through an object related to its read only file system, the object throws the `ReadOnlyFileSystemException` exception. File systems are secure when they are used by several simultaneous threads. The inheritance diagram for this class is as follows:

```

java.lang.Object
|
____java.nio.file.FileSystem

```

The constructors of the `FileSystem` class are provided in Table 6.10 and its methods in Table 6.11:

Table 6.10: The constructor of the <code>FileSystem</code> class	
Constructor	Does this
<code>Protected FileSystem()</code>	It is used to initialize a new instance of the <code>FileSystem</code> class.

Table 6.11: Methods of the <code>FileSystem</code> class	
Method	Does this
<code>abstract void close()</code>	It closes the file system that is currently in use.
<code>abstract Iterable<FileStore> getFileStores()</code>	It returns an object to iterate over the underlying file stores.
<code>abstract Path getPath(String first, String... more)</code>	It changes a path string, or a sequence of strings to a <code>Path</code> . The sequence of strings are joined together to form a <code>Path</code> string.
<code>abstract PathMatcher getPathMatcher(String syntaxAndPattern)</code>	It returns a <code>PathMatcher</code> to perform the math operation on the <code>String</code> representation of <code>Path</code> object.
<code>abstract Iterable<Path> getRootDirectories()</code>	It returns an object to iterate over the paths of the root directories.
<code>abstract String getSeparator()</code>	It returns the name separator, represented as a string.
<code>abstract UserPrincipalLookupService getUserPrincipalLookupService()</code>	It returns the <code>UserPrincipalLookupService</code> for this file system (optional operation).
<code>abstract boolean isOpen()</code>	It returns a boolean value specifying whether or not this file system is open.
<code>abstract boolean isReadOnly()</code>	It returns a boolean value specifying whether or not this file system allows only read-only access to its file stores.
<code>abstract WatchService newWatchService()</code>	It creates a new <code>WatchService</code> (optional operation).
<code>abstract FileSystemProvider provider()</code>	It returns the provider that created this file system.
<code>abstract Set<String> supportedFileAttributeViews()</code>	It returns the set of the names of the file attribute views supported by this <code>FileSystem</code> .

The *FileSystems* Class

The `FileSystems` class gives collection of methods for file systems. The `getDefault()` method provided by this class returns the default file system and the collection of methods to create other types of file systems.

Whenever the default provider is loaded, the method of this class will definitely be invoked for the first time. The default provider is denoted by the URI scheme "file", which creates the file system that allows you to retrieve the data. But if the process of loading or instantiating the default provider is not successful, an undetermined error occurs.

The `installedProviders()` method is invoked for the first time by calling any of the `newFileSystem()` methods defined by the `FileSystems` class. This method identifies and loads all installed file system providers. With the help of the service-provider loading capability, the installed providers can be loaded, which are defined by service loader class. If the system class loader is not found, then the extension class loader is used. But if there is no extension class loader, then the bootstrap class loader is used for loading the file system.

Generally, you can install the providers by storing them in a JAR file, which is configured on the application classpath. You can also store the providers to be installed in the extension directory. The JAR file includes a provider-configuration, that is, `java.nio.file.spi.FileSystemProvider`, in the source directory `META-INF/services`. The ordering in which the installed providers are available depends upon the implementation of the provider. If during the instantiation of a provider, the `getScheme()` method returns the same URI pattern

as that of the earlier instantiated provider, then the file system provider is rejected because of duplicity. You should note that the URI schemes are case-insensitive. During instantiation, a provider can retrieve the data related to the default provider from files in the secured manner, but the circular loading of other installed providers should be avoided. (This is because the circular loading of installed providers can cause an unknown error).

The `FileSystems` class also defines some methods that allow you to specify the `ClassLoader` to be used while locating a provider. If a thread starts loading of installed file system providers and another thread requests a method that also wants to load the providers, then the method will block until the loading completes. The inheritance diagram for this class is as follows:

```
java.lang.Object
|___java.nio.file.FileSystems
```

The methods of the `FileSystems` class are listed in Table 6.12:

Table 6.12: Methods of the <code>FileSystems</code> class	
Method	Does this
<code>static FileSystem getDefault()</code>	It returns the default <code>FileSystem</code> .
<code>static FileSystem getFileSystem(URI uri)</code>	It returns a reference to an existing <code>FileSystem</code> .
<code>static FileSystem newFileSystem(Path path, ClassLoader loader)</code>	It creates a new <code>FileSystem</code> to access the contents of a file as a file system.
<code>static FileSystem newFileSystem(URI uri, Map<String,?> env)</code>	It creates a new file system that is identified by a URI.
<code>static FileSystem newFileSystem(URI uri, Map<String,?> env, ClassLoader loader)</code>	It creates a new file system that is identified by a URI.

The *FileStore* Class

(The `FileStore` class defines the storage for files). A file store is a volume or partition in which files are stored. You can obtain the location of the storage of files by calling the `getFileStore()` method. Another methods of the `FileStore` class, `getFileStores()` method, is used to count the total number of file stores. A file store can support more than one `FileStoreAttributeView` class which provides the read-only or updatable view of the `FileStore` attributes. The inheritance diagram for this class is as follows:

```
java.lang.Object
|___java.nio.file.FileStore
```

The constructor of the `FileStore` class listed in Table 6.13 and its methods in Table 6.14:

Table 6.13: The constructor of the <code>FileStore</code> class	
Constructor	Does this
<code>protected FileStore()</code>	It initializes new instance of this class.

Table 6.14: Methods of the <code>FileStore</code> class	
Method	Does this
<code>abstract Object getAttribute(String attribute)</code>	It reads the value of a file store attribute.
<code>abstract <V extends FileStoreAttributeView>V getFileStoreAttributeView(Class<V> type)</code>	It returns a <code>FileStoreAttributeView</code> of the given type.
<code>abstract long getTotalSpace()</code>	It returns the size of the file store in bytes.
<code>abstract long getUnallocatedSpace()</code>	It returns the number of unallocated bytes in the file store.

Table 6.14: Methods of the FileStore class

Method	Does this
<code>abstract long getUsableSpace()</code>	It returns the number of bytes available to this Java virtual machine on the file store.
<code>abstract boolean isReadOnly()</code>	It states whether this file store is read-only.
<code>abstract String name()</code>	It returns the name of this file store.
<code>abstract boolean supportsFileAttributeView(Class<? extends FileAttributeView> type)</code>	It states whether or not this file store supports the file attributes identified by the given file attribute view.
<code>abstract boolean supportsFileAttributeView(String name)</code>	It states whether or not this file store supports the file attributes identified by the given file attribute view.
<code>abstract String type()</code>	It returns the type of this file store.

Prospects of NIO

Only time and usage can tell whether the new approach will someday replace the traditional approach. However, it is certain that the NIO APIs offer an exciting new way to think about and handle various types of operations.

So what more does NIO give us? A lot, but not everything—native code is still required. Some things that could be done only with native code are now provided in NIO.

There's a lot coming up next: I/O is a huge topic in Java. It's time to turn to the "Immediate Solutions" section.

Working with Streams

Generally, programs use the data from external sources, which can be from the remote machines or from the local files on the same system. Java uses the concept of streams for reading data from the system and writing data to it.

Stream sources include files, buffers, sockets, threads, and other streams. Stream destinations include the same objects as stream sources and others as well, such as printers. There are generally two types of streams: input stream and output stream. In case of input stream, a stream of data items flows from a source, whereas, in case of output stream, the data items flow to a destination.

The *InputStream* Class

The Novice Programmer appears and says, "I'm ready to start working with files. Where do I start?" "You start," you say, "with the `InputStream` and `OutputStream` classes."

The `InputStream` and `OutputStream` classes are the base classes for byte-oriented I/O in Java, so it's worth taking a look at the methods that these classes provide to all other byte-oriented stream classes. Here's the inheritance diagram for the `InputStream` class, on which you base input streams:

```

java.lang.Object
|____java.io.InputStream

```

The `InputStream` is an abstract class that defines Java's model of streaming byte input. All the methods of this class will throw an `IOException`.

You'll find the constructor of the `InputStream` class in Table 6.15 and its methods in Table 6.16:

Table 6.15: The constructor of the `InputStream` class

Constructor	Does this
<code>InputStream()</code>	It constructs an <code>InputStream</code> object

Table 6.16: Methods of the InputStream class

Method	Does this
<code>int available()</code>	It gets the number of bytes that can be read from this input stream
<code>void close()</code>	It closes this input stream
<code>void mark(int readlimit)</code>	It marks the current position in this input stream
<code>boolean markSupported()</code>	It returns True if this input stream supports the <code>mark()</code> and <code>reset()</code> methods
<code>abstract int read()</code>	It reads the next byte of data from the input stream
<code>int read(byte[] b)</code>	It reads some number of bytes from the input stream and stores them into the buffer array <code>b</code>
<code>int read(byte[] b, int off, int len)</code>	It reads up to <code>len</code> bytes of data from the input stream into an array of bytes
<code>void reset()</code>	It repositions this stream to the position where the <code>mark()</code> method was last called on this input stream
<code>long skip(long n)</code>	It skips over <code>n</code> bytes of data from this input stream

We'll take a look at the `OutputStream` class in the next solution.

The **OutputStream** Class

The counterpart of the `InputStream` class, introduced in the previous section, is the `OutputStream` class, on which you base output streams. The inheritance diagram for the `OutputStream` class is as follows:

```

java.lang.Object
|
+--- java.io.OutputStream

```

The `OutputStream` class refers to an abstract class defining streaming byte output. All the methods of this class return `void` and throw `IOException` in case of errors. While `InputStream` implements the `Closeable` interface, `OutputStream` implements the `Closeable` and `Flushable` interfaces both.

You'll find the constructor of the `OutputStream` class in Table 6.17 and its methods in Table 6.18:

Table 6.17: The constructor of the OutputStream class

Constructor	Does this
<code>OutputStream()</code>	It constructs an <code>OutputStream</code> object.

Table 6.18: Methods of the OutputStream class

Method	Does this
<code>void close()</code>	It closes this output stream.
<code>void flush()</code>	It flushes this output stream and writes any waiting buffered output bytes.
<code>void write(byte[] b)</code>	It writes <code>b.length</code> bytes from the given byte array to this output stream.
<code>void write(byte[] b, int off, int len)</code>	It writes <code>len</code> bytes from the given byte array, starting at offset <code>off</code> , to this output stream.
<code>abstract void write(int b)</code>	It writes the given byte to this output stream.

Now that you have seen `InputStream` and `OutputStream`, it's time to put these classes to work, starting in the next solution.

The **ByteArrayInputStream** Class

You can use the `ByteArrayInputStream` class to open an input stream from an array of bytes in memory. Here's the inheritance diagram for this class:

```

java.lang.Object
|___java.io.InputStream
|___java.io.ByteArrayInputStream

```

You'll find the fields of the `ByteArrayInputStream` class in Table 6.19, its constructors in Table 6.20, and its methods in Table 6.21:

Table 6.19: Fields of the `ByteArrayInputStream` class

Field	Does this
protected byte[] buf	It is an array of bytes that was supplied by the creator of the stream.
protected int count	It is the number of characters in the input stream buffer.
protected int mark	It is the currently marked position in the stream.
protected int pos	It is the index of the next character to read from the input stream.

Table 6.20: Constructors of the `ByteArrayInputStream` class

Constructor	Does this
<code>ByteArrayInputStream(byte[] buf)</code>	It constructs a <code>ByteArrayInputStream</code> object.
<code>ByteArrayInputStream(byte[] buf, int offset, int length)</code>	It constructs a <code>ByteArrayInputStream</code> object that uses <code>buf</code> as its buffer array. In this case, we are just creating a memory buffer with the text.

Table 6.21: Methods of the `ByteArrayInputStream` class

Method	Does this
<code>int available()</code>	It gets the number of bytes that can be read from this input.
<code>void close()</code>	It closes this input stream.
<code>void mark(int readAheadLimit)</code>	It sets the current marked position in the stream.
<code>boolean markSupported()</code>	It returns <code>True</code> if this <code>ByteArrayInputStream</code> object supports <code>mark()/reset()</code> .
<code>int read()</code>	It reads the next byte of data.
<code>int read(byte[] b, int off, int len)</code>	It reads up to <code>len</code> bytes of data into an array of bytes from this input stream.
<code>void reset()</code>	It resets the buffer to the marked position.
<code>long skip(long n)</code>	It skips <code>n</code> bytes of input from this input stream.

Here's an example. In this case, we are just creating a memory buffer with the text "Dreamtech Press" in it and then opening a `ByteArrayInputStream` object to that buffer, reading and displaying the text byte by byte:

```

import java.io.*;
class ByteArrayInputStreamDemo
{
    public static void main(String args[]) throws IOException
    {
        byte data[] = "Dreamtech Press".getBytes();
        ByteArrayInputStream in = new ByteArrayInputStream(data);
        int character;
        while((character = in.read()) != -1) {

```

```

        System.out.print((char) character); }
    }
}

```

The output from this code is shown in Figure 6.2, showing a string stored in an array of bytes:

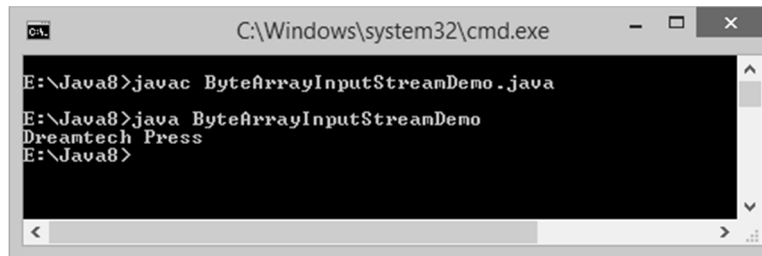


Figure 6.2: An array storing a number of bytes

Take the case of another example, where the `reset()` method is used to read the same input twice, i.e. the string “Dreantech” is read and printed once in lowercase and then again in uppercase:

```

import java.io.*;
class ResetDemo
{
    public static void main(String args[]) throws IOException
    {
        String p = "Dreantech";
        byte B[] = p.getBytes();
        ByteArrayInputStream in = new ByteArrayInputStream(B);
        for (int j=0; j<2; j++)
        {
            int N;
            while ((N = in.read()) != -1)
            {
                if (j == 0) { System.out.print((char) N); }
                else {
                    System.out.print(Character.toUpperCase((char) N)); }
            }
            System.out.println();
            in.reset(); }
        }
    }
}

```

The output is shown in Figure 6.3, showing the same input printed twice:

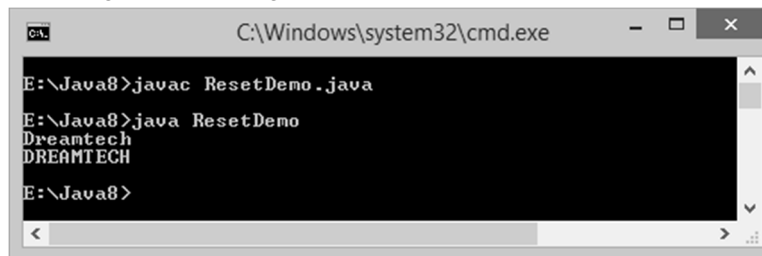


Figure 6.3: Input printed twice

The **ByteArrayOutputStream** Class

The `ByteArrayOutputStream` class lets you write streams of bytes to buffers. Here’s the inheritance diagram for this class:

```

java.lang.Object
|___ java.io.OutputStream
      |___ java.io.ByteArrayOutputStream

```

You'll find the fields of the `ByteArrayOutputStream` class in Table 6.22, its constructors in Table 6.23, and its methods in Table 6.24:

Table 6.22: Fields of the `ByteArrayOutputStream` class

Field	Does this
<code>protected byte[] buf</code>	The buffer where data is stored
<code>protected int count</code>	The number of bytes in the buffer

Table 6.23: Constructors of the `ByteArrayOutputStream` class

Constructor	Does this
<code>ByteArrayOutputStream()</code>	It constructs a new byte array output stream
<code>ByteArrayOutputStream(int size)</code>	It constructs a new byte array output stream with a buffer of the given size

Table 6.24: Methods of the `ByteArrayOutputStream` class

Method	Does this
<code>void close()</code>	It closes this output stream
<code>void reset()</code>	It resets the count field of this byte array output stream to zero
<code>int size()</code>	It gets the current size of the buffer
<code>byte[] toByteArray()</code>	It constructs a newly allocated byte array
<code>String toString()</code>	It converts the buffer's contents into a string
<code>String toString(int hibyte)</code>	It is deprecated and it uses the <code>toString()</code> method
<code>String toString(String enc)</code>	It converts the buffer's contents into a string
<code>void write(byte[] b, int off, int len)</code>	It writes <code>len</code> bytes from the given byte array starting at offset <code>off</code>
<code>void write(int b)</code>	It writes the given byte to this byte array output stream
<code>void writeTo(OutputStream out)</code>	It writes the complete contents of this byte array output stream to the given output stream argument

Here's an example. In this case, we are writing bytes to a `ByteArrayOutputStream` object, writing bytes to a memory buffer, and sending bytes to a file output stream:

```
import java.io.*;
class ByteArrayOutputStreamDemo
{
    public static void main(String args[]) throws IOException
    {
        ByteArrayOutputStream bytearrayoutputstream = new
        ByteArrayOutputStream();
        byte data[] = " welcome to Dreamtech Press ".getBytes();
        bytearrayoutputstream.write(data);
        System.out.println(bytearrayoutputstream.toString());
        byte buffer[] = bytearrayoutputstream.toByteArray();
        for (int loop_index = 0; loop_index < data.length; loop_index++)
        {
            System.out.print((char) buffer[loop_index]); }
        OutputStream fileoutputstream = new FileOutputStream("file.txt");
        bytearrayoutputstream.writeTo(fileoutputstream);
        fileoutputstream.close(); }
}
```


The result can be seen in Figure 6.4, showing the bytes to be stored in the buffer:

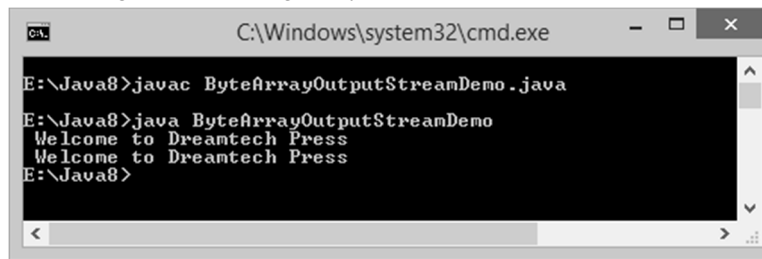


Figure 6.4: Bytes stored in the buffer

The *BufferedInputStream* Class

Using the *BufferedInputStream* class, you can use the *mark()* and *reset()* methods to actually move backward in a buffered input stream. The inheritance diagram for this class is as follows:

```

java.lang.Object
|
+-- java.io.InputStream
|   |
|   +-- java.io.FilterInputStream
|       |
|       +-- java.io.BufferedInputStream

```

You'll find the fields of the *BufferedInputStream* class in Table 6.25, its constructors in Table 6.26, and its methods in Table 6.27:

Table 6.25: Fields of the *BufferedInputStream* class

Field	Does this
protected byte[] buf	The internal buffer where the data is stored.
protected int count	The count of bytes in the buffer.
protected int marklimit	The maximum read-ahead allowed after a call to the <i>mark()</i> method.
protected int markpos	The value of the <i>pos</i> field at the time the last <i>mark()</i> method was called.
protected int pos	The position in the buffer.

Table 6.26: Constructors of the *BufferedInputStream* class

Constructor	Does this
<i>BufferedInputStream</i> (<i>InputStream in</i>)	It constructs a <i>BufferedInputStream</i> object and saves its argument, the input stream <i>in</i> , for later use.
<i>BufferedInputStream</i> (<i>InputStream in</i> , <i>int size</i>)	It constructs a <i>BufferedInputStream</i> object with the given buffer size.

Table 6.27: Methods of the *BufferedInputStream* class

Method	Does this
<i>int available()</i>	It gets the number of bytes that can be read from this input.
<i>void close()</i>	It closes this input stream.
<i>void mark(int readAheadLimit)</i>	It sets the current marked position in the stream.
<i>boolean markSupported()</i>	It tests whether this input stream supports the <i>mark()</i> and <i>reset()</i> methods.
<i>int read()</i>	It reads the next byte of data.
<i>int read(byte[] b, int off, int len)</i>	It reads bytes from this byte input stream into the given byte array, starting at the given offset.

Table 6.27: Methods of the `BufferedInputStream` class

Method	Does this
<code>void reset()</code>	It resets the buffer to the marked position.
<code>long skip(long n)</code>	It skips <code>n</code> bytes of input from this input stream.

Here's an example. In this case, we'll remove comments from a `ByteArrayInputStream` object that are delimited with the `#` character. That means when we encounter a `#` character, we'll just stop displaying bytes until we encounter a closing `#`.

For the purposes of this example, we'll only use comments that don't include spaces. Note that sometimes the `#` character is a valid character in text, so if we encounter a `#` and stop displaying text and then encounter a space before seeing a closing `#`, the text was not a comment. Therefore, we have to move back to the first `#` and redisplay the text up to the space.

We'll write this example using the `mark()` and `reset()` methods; we mark a location, then, if needed, use the `reset()` method to go back to that location. Here's what the code looks like (note that you pass a length, in bytes, to the `mark()` method that indicates the maximum number of bytes you'll read before possibly using `reset()`):

```
import java.io.*;
class BufferedInputStreamDemo {
    public static void main(String args[]) throws IOException {
        byte data[] = "welcome to Dreamtech Press.#comment_goes_here#".getBytes();
        ByteArrayInputStream in = new ByteArrayInputStream(data);
        BufferedInputStream bufferedinputstream = new
            BufferedInputStream(in);
        int character;
        boolean silentflag = false;
        while ((character = bufferedinputstream.read()) != -1) {
            switch(character) { case '#':
                if (silentflag) { silentflag = false; }
                else {
                    silentflag = true;
                    bufferedinputstream.mark(100); }
                break;
            case ' ':
                if (silentflag) {
                    silentflag = false;
                    System.out.print("#");
                    bufferedinputstream.reset(); }
                else
                    System.out.print((char) character);
                break;
            default:
                if (!silentflag) System.out.print((char) character);
            }
        }
    }
}
```

Here's the output of this code: `E:\Java8\chapter18\>java BufferedInputStreamDemo`
 welcome to Dreamtech Press.

The ***BufferedOutputStream*** Class

The `BufferedOutputStream` class is the output counterpart of the `BufferedInputStream` class (see the previous solution). The inheritance diagram for the `BufferedOutputStream` class is as follows:

```
java.lang.Object
|____java.io.OutputStream
|____java.io.FilterOutputStream
|____java.io.BufferedOutputStream
```

You'll find the fields of the `BufferedOutputStream` class in Table 6.28, its constructors in Table 6.29, and its methods in Table 6.30:

Table 6.28: Fields of the <code>BufferedOutputStream</code> class	
Field	Does this
<code>protected byte[] buf</code>	The internal buffer where data is stored.
<code>protected int count</code>	The number of bytes in the buffer.

Table 6.29: Constructors of the <code>BufferedOutputStream</code> class	
Constructor	Does this
<code>BufferedOutputStream(OutputStream out)</code>	It constructs a new-buffered output stream.
<code>BufferedOutputStream(OutputStream out, int size)</code>	It constructs a new buffered output stream to write data with the given buffer size.

Table 6.30: Methods of the <code>BufferedOutputStream</code> class	
Method	Does this
<code>void flush()</code>	It flushes this buffered output stream.
<code>void write(byte[] b, int off, int len)</code>	It writes <code>len</code> bytes from the given byte array, starting at offset <code>off</code> , to this buffered output stream.
<code>void write(int b)</code>	It writes the given byte to this buffered output stream.

The idea behind `BufferedOutputStream` is that you don't have to write data to disk every time a byte is written, because output is buffered. To force a write operation, you can use the `flush()` method.

The ***FileInputStream*** Class

The `FileInputStream` class is specially designed to work with byte-oriented input from files and is derived from `InputStream`. The inheritance diagram for the `FileInputStream` class is as follows:

```

java.lang.Object
|
+--- java.io.InputStream
|
+--- java.io.FileInputStream

```

You'll find the constructors of the `FileInputStream` class in Table 6.31 and its methods in Table 6.32:

Table 6.31: Constructors of the <code>FileInputStream</code> class	
Constructor	Does this
<code>FileInputStream(File file)</code>	It constructs a <code>FileInputStream</code> object by opening a connection to an actual file
<code>FileInputStream(FileDescriptor fdObj)</code>	It constructs a <code>FileInputStream</code> object by using the file descriptor <code>fdObj</code>
<code>FileInputStream(String name)</code>	It constructs a <code>FileInputStream</code> object by opening a connection to an actual file given by the file named by <code>name</code>

Table 6.32: Methods of the <code>FileInputStream</code> class	
Method	Does this
<code>int available()</code>	It gets the number of bytes that can be read from this file input stream
<code>void close()</code>	It closes this file input stream
<code>protected void finalize()</code>	It makes sure that the <code>close()</code> method of this file input stream is called when there are no more references to it
<code>FileDescriptor getFD()</code>	It gets the <code>FileDescriptor</code> object
<code>int read()</code>	It reads a byte of data from this input stream

Table 6.32: Methods of the FileInputStream class

Method	Does this
<code>int read(byte[] b)</code>	It reads up to <code>b.length</code> bytes of data from this input stream called when there are no more references to it into an array of bytes
<code>int read(byte[] b, int off, int len)</code>	It reads up to <code>len</code> bytes of data from this input stream into an array of bytes
<code>long skip(long n)</code>	It skips over and discards <code>n</code> bytes

All the constructors can throw a `FileNotFoundException`. To create a file input stream, you use the `FileInputStream` class's constructor. The reading methods you use with this class are as follows:

- ❑ `int read()`—It reads a byte of data from this input stream and returns it.
- ❑ `int read(byte[] b)`—It reads up to `b.length` bytes of data from this input stream into an array of bytes and returns the number of bytes read.
- ❑ `int read(byte[] b, int off, int len)`—It reads up to `len` bytes of data from this input stream into an array of bytes and returns the number of bytes read.

Let's consider an example: We open the source code file of the application, read and display 50 bytes of code, skip 50 bytes with the `skip()` method, and then read and display 50 more bytes (note that we can also determine how many bytes are waiting there to be read using the `available()` method, and we can close the stream at the end of the code):

```
import java.io.*;
class FileInputStreamDemo {
    public static void main(String args[]) throws Exception {
        int size;
        FileInputStream fileinputstream = new
            FileInputStream("FileInputStreamDemo.java");
        System.out.println("Available bytes: " + (size
            =fileinputstream.available()));
        System.out.println("Reading 20 bytes....");
        byte bytearray[] = new byte[20];
        if (fileinputstream.read(bytearray) != 20) {
            System.out.println("Could not get 20 bytes"); }
        System.out.println(new String(bytearray, 0, 20));
        System.out.println("Skipping 20 bytes...");
        fileinputstream.skip(20);
        System.out.println("Reading 20 bytes....");
        if (fileinputstream.read(bytearray) != 20) {
            System.out.println("Could not get 20 bytes"); }
        System.out.println(new String(bytearray, 0, 20));
        fileinputstream.close(); }
    }
```

The output from this code is shown in Figure 6.5, showing the input to a file:

```
C:\Windows\system32\cmd.exe
E:\Java8>javac FileInputStreamDemo.java
E:\Java8>java FileInputStreamDemo
Available bytes: 840
Reading 20 bytes....
import java.io.*;
c
Skipping 20 bytes...
Reading 20 bytes....
Demo <
public stat
E:\Java8>
```

Figure 6.5: The FileInputStreamDemo Class

TIP

An untrusted applet is not allowed to read and write files at all. So, unless you make specific security arrangements, you should not try to read or write files from applets. Therefore, for the sake of convenience, all the programs in this chapter are applications.

The *FileOutputStream* Class

You can use the `FileOutputStream` class to write data, byte by byte, to a file. They can throw a `FileNotFoundException` class or a `SecurityException` class. Remember that the creation of a `FileOutputStream` class is not dependent on the file that already exists. In fact, the `FileOutputStream` class will create the file before opening it for output. An `IOException` is thrown in case you try to open a read-only file. All `FileOutputStream` methods have a void return type.

The inheritance diagram for that class, which is derived from the `OutputStream` class, is as follows:

```

java.lang.Object
|____java.io.OutputStream
|____java.io.FileOutputStream

```

The constructors of the `FileOutputStream` class are provided in Table 6.33 and its methods in Table 6.34:

Table 6.33: Constructors of the `FileOutputStream` class

Constructor	Does this
<code>FileOutputStream(File file)</code>	It constructs a file output stream to write to the given <code>File</code> object.
<code>FileOutputStream(File file, boolean append)</code>	It constructs a file output stream to write to the file represented by the specified <code>File</code> object.
<code>FileOutputStream(FileDescriptor fdObj)</code>	It constructs an output file stream to write to the given file descriptor.
<code>FileOutputStream(String name)</code>	It constructs an output file stream to write to the file with the given name.
<code>FileOutputStream(String name, boolean append)</code>	It constructs an output file stream to append to the file with the given name.

Table 6.34: Methods of the `FileOutputStream` class

Method	Does this
<code>void close()</code>	It closes this file output stream.
<code>protected void finalize()</code>	It makes sure that the <code>close()</code> method of this file output stream is called when there are no more references to this stream.
<code>FileChannel getChannel()</code>	It gets the unique <code>FileChannel</code> object associated with this file output stream.
<code>FileDescriptor getFD()</code>	It gets the file descriptor associated with this stream.
<code>void write(byte[] b)</code>	It writes <code>b.length</code> bytes from the given byte array to this file output stream.
<code>void write(byte[] b, int off, int len)</code>	It writes <code>len</code> bytes from the given byte array, starting at offset <code>off</code> , to this file output stream.
<code>void write(int b)</code>	It writes the given byte to this file output stream.

The writing methods you use with this class are as follows:

- ❑ `void write(byte[] b)`—Writes `b.length` bytes from the given byte array to this file output stream.
- ❑ `void write(byte[] b, int off, int len)`—Writes `len` bytes from the given byte array, starting at offset, to the file output stream.
- ❑ `void write(int b)`—Writes the given byte to the file output stream.

Here's an example in which we write the string of text "This is a string of text." to a file in three ways—byte by byte, all at once, and in part. Here's what the code looks like:

```
import java.io.*;
class FileOutputStreamDemo {
    public static void main(String args[]) throws Exception {
        byte data[] = "This is a string of text.".getBytes();
        FileOutputStream fileOutputStream1 = new
            FileOutputStream("file1.txt");
        for (int loop_index = 0; loop_index < data.length; loop_index++) {
            fileOutputStream1.write(data[loop_index]); }
        FileOutputStream fileOutputStream2 = new
            FileOutputStream("file2.txt");
        fileOutputStream2.write(data);
        FileOutputStream fileOutputStream3 = new
            FileOutputStream("file3.txt");
        fileOutputStream3.write(data, 5, 10);
        fileOutputStream1.close();
        fileOutputStream2.close();
        fileOutputStream3.close(); }
}
```

Compile and execute this code and find three files—file1.txt, file2.txt, and file3.txt—that are created.

Working with the Reader Class

You use the Reader and Writer classes to read and write character-streams. As with InputStream and OutputStream, you base classes on Reader and Writer instead of using them directly. The Reader implements the Closeable and Readable interfaces. All the methods of this class throw an IOException on error conditions. Here's the inheritance diagram for Reader:

```
java.lang.Object
|___java.io.Reader
```

You'll find the field of the Reader class in Table 6.35, its constructors in Table 6.36, and its methods in Table 6.37:

Table 6.35: The field of the Reader class

Field	Does this
protected Object lock	The lock used to synchronize operations.

Table 6.36: Constructors of the Reader class

Constructor	Does this
protected Reader()	It constructs a new character-stream.
protected Reader(Object lock)	It constructs a new character stream reader whose critical sections will synchronize on the given lock.

Table 6.37: Methods of the Reader class

Method	Does this
abstract void close()	It closes the stream.
void mark(int readAheadLimit)	It marks the present position in the stream.
boolean markSupported()	It tells whether this stream supports the mark() operation.
int read()	It reads a single character.
int read(char[] cbuf)	It reads characters into an array.
abstract int read(char[] cbuf, int off, int len)	It reads characters into a portion of an array.
boolean ready()	It tells whether this stream is ready to be read.

Table 6.37: Methods of the Reader class

Method	Does this
<code>void reset()</code>	It resets the stream.
<code>long skip(long n)</code>	It skips characters.

Working with the **Writer** Class

The `Writer` class is the character-based counterpart of the `Reader` class (see the previous solution). Here's the inheritance diagram for `Writer`:

```

java.lang.Object
|
+-- java.io.Writer

```

The `Writer` implements the `Closeable`, `Flushable`, and `Appendable` interfaces. A `void` value is returned and an `IOException` is thrown by all the methods in this class in the event of errors. You'll find the field of the `Writer` class in Table 6.38, its constructors in Table 6.39, and its methods in Table 6.40:

Table 6.38: The field of the Writer class

Field	Does this
<code>protected Object lock</code>	The lock used to synchronize operations on this stream.

Table 6.39: Constructors of the Writer class

Constructor	Does this
<code>protected Writer()</code>	It creates a new character-stream writer.
<code>protected Writer(Object lock)</code>	It creates a new character-stream writer whose critical sections will synchronize on the given lock.

Table 6.40: Methods of the Writer class

Method	Does this
<code>Writer append(char char)</code>	It appends the character to the end of the output stream.
<code>Writer append(CharSequence chars)</code>	It appends the string to the end of the output stream.
<code>Writer append(CharSequence csq, int start, int end)</code>	It appends a subsequence of the specified character sequence to this writer.
<code>abstract void close()</code>	It closes the stream, flushing it first.
<code>abstract void flush()</code>	It flushes the stream.
<code>void write(char[] cbuf)</code>	It writes an array of characters.
<code>abstract void write(char[] cbuf, int off, int len)</code>	It writes a portion of an array of characters.
<code>void write(int c)</code>	It writes a single character.
<code>void write(String str)</code>	It writes a string.
<code>void write(String str, int off, int len)</code>	It writes a portion of a string.

Accepting Input from the Keyboard with the **InputStreamReader** Class

You usually construct an `InputStreamReader` stream from `System.in` for reading keys from the keyboard. After that, you can use the `read()` method of the `InputStreamReader` class for reading the text typed by the user. Here's the inheritance diagram for `InputStreamReader`:

```

java.lang.Object
|___java.io.Reader
|   |___java.io.InputStreamReader

```

You'll find the constructors of the `InputStreamReader` class in Table 6.41 and its methods in Table 6.42:

Table 6.41: Constructors of the <code>InputStreamReader</code> class	
Constructor	Does this
<code>InputStreamReader(InputStream in)</code>	It creates an <code>InputStreamReader</code> object.
<code>InputStreamReader(InputStream in, Charset cs)</code>	It creates an <code>InputStreamReader</code> that uses the given charset.
<code>InputStreamReader(InputStream in, CharsetDecoder dec)</code>	It creates an <code>InputStreamReader</code> that uses the given charset decoder.
<code>InputStreamReader(InputStream in, String enc)</code>	It creates an <code>InputStreamReader</code> object that uses the named character encoding.

Table 6.42: Methods of the <code>InputStreamReader</code> class	
Method	Does this
<code>void close()</code>	It closes the stream.
<code>String getEncoding()</code>	It gets the name of the character encoding.
<code>int read()</code>	It reads a single character.
<code>int read(char[] cbuf, int off, int len)</code>	It reads characters into a portion of an array.
<code>boolean ready()</code>	It returns <code>True</code> if this stream is ready to be read.

Here's an example: We'll just read keys from the keyboard and echo them. Note that the input stream does not become available until the user presses the Enter key. Here's what the code looks like:

```

import java.io.*;
class InputStreamReaderDemo
{
    public static void main(String args[])
    {
        try {
            String character;
            InputStreamReader inputstreamreader = new InputStreamReader(System.in);
            BufferedReader br = new BufferedReader(inputstreamreader);
            character = br.readLine();
            System.out.print(character);
        }
        catch (IOException e) { }
    }
}

```

The output of this code is shown in Figure 6.6, printing the string taken from the keyboard as an input:

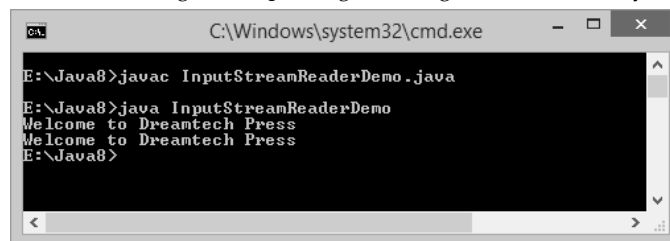


Figure 6.6: Taking input from the user

Note that the echoed line appears after the user enters text and presses the Enter key.

Working with the **OutputStreamWriter** Class

The `OutputStreamWriter` class works as a bridge from character streams to byte streams. This class performs the encoding of characters into bytes with the help of a specified character set. Here's the inheritance diagram for `OutputStreamWriter`:

```

java.lang.Object
|___java.io.Writer
    |___java.io.OutputStreamWriter
  
```

You'll find the constructors and methods of the `OutputStreamWriter` class in Table 6.43 and Table 6.44, respectively:

Table 6.43: Constructors of the `OutputStreamWriter` class

Constructor	Does this
<code>OutputStreamWriter(OutputStream out)</code>	It constructs an <code>OutputStreamWriter</code> object.
<code>OutputStreamWriter(OutputStream out, Charset cs)</code>	It creates an <code>OutputStreamWriter</code> that uses the given charset.
<code>OutputStreamWriter(OutputStream out, CharsetEncoder enc)</code>	It creates an <code>OutputStreamWriter</code> that uses the given charset encoder.
<code>OutputStreamWriter(OutputStream out, String enc)</code>	It constructs an <code>OutputStreamWriter</code> object that uses the named character encoding.

Table 6.44: Methods of the `OutputStreamWriter` class

Method	Does this
<code>void close()</code>	It closes the stream.
<code>void flush()</code>	It flushes the stream.
<code>String getEncoding()</code>	It gets the name of the character encoding being used by this stream.
<code>void write(char[] cbuf, int off, int len)</code>	It writes a section of an array of characters.
<code>void write(int c)</code>	It writes a single character.
<code>void write(String str, int off, int len)</code>	It writes a portion of a string.

Working with Files

After learning about the streams which provide a simple model for reading and writing data that supported all the operations common with a disk file, you will learn how to work with a file using the non-stream file I/O.

The `File` class handles files that are machine-dependent in an independent manner, that is, you can create platform-independent code easily that keeps monitoring and manipulating files using the `File` class. This class is available in the `java.io` package.

Using the **File** Class

Except for the `File` class, most of the classes that are defined by `java.io` operate on streams, i.e., they deal directly with files and the file system. The `File` class does not state how information is accessed from or stored in files. Instead, it describes the properties of a file itself.

You can use the `File` class to determine a lot about files, such as their sizes; whether you can write to them; whether an item is a file, directory, or named pipe; and a lot more for manipulating the information associated with a disk file, such as permissions, time, date, and for navigating subdirectory hierarchies. Here's the inheritance diagram for the `File` class:

```
java.lang.Object
|___java.io.File
```

A Java directory is considered to be an additional property, in which an array of filenames can be verified by using the `list()` method. You'll find the fields for the `File` class in Table 6.45, its constructors in Table 6.46, and its methods in Table 6.47:

Table 6.45: Fields of the File class

Field	Does this
<code>static String pathSeparator</code>	It is a system-dependent path-separator string.
<code>static char pathSeparatorChar</code>	It is a system-dependent path-separator character.
<code>static String separator</code>	It is a system-dependent default name-separator character, represented as a string for convenience.
<code>static char separatorChar</code>	It is a system-dependent default name-separator character.

Table 6.46: Constructors of the File class

Constructor	Does this
<code>File(File parent, String child)</code>	It constructs a new <code>File</code> instance from a parent pathname and a child pathname string.
<code>File(String pathname)</code>	It constructs a new <code>File</code> instance by converting the given pathname string into an abstract pathname.
<code>File(String parent, String child)</code>	It constructs a new <code>File</code> instance from a parent pathname string and a child pathname string.
<code>File(URI uri)</code>	It constructs a new <code>File</code> instance by converting the given file:URI into an abstract pathname.

Table 6.47: Methods of the File class

Method	Does this
<code>boolean canExecute()</code>	It returns <code>True</code> if the application can execute the file denoted by this abstract pathname.
<code>boolean canRead()</code>	It returns <code>True</code> if the application can read the file given by this abstract pathname.
<code>boolean canWrite()</code>	It returns <code>True</code> if the application can modify the file given by this abstract pathname.
<code>int compareTo(File pathname)</code>	It compares two abstract pathnames.
<code>int compareTo(Object o)</code>	It compares this abstract pathname to another object.
<code>boolean createNewFile()</code>	It constructs a new, empty file.
<code>static File createTempFile(String prefix, String suffix)</code>	It constructs an empty file in the default temporary file directory.
<code>static File createTempFile(String prefix, String suffix, File directory)</code>	It constructs a new, empty file in the given directory.
<code>boolean delete()</code>	It deletes the file or directory.
<code>void deleteOnExit()</code>	It requests that the file or directory given by this pathname be deleted when the virtual machine exits.
<code>boolean equals(Object obj)</code>	It compares this pathname for equality with the given object.
<code>boolean exists()</code>	It compares whether the file given by this abstract pathname exists.
<code>File getAbsolutePath()</code>	It gets the absolute form of this abstract pathname.

Table 6.47: Methods of the File class

Method	Does this
<code>String getAbsolutePath()</code>	It gets the absolute pathname string of this abstract pathname.
<code>File getCanonicalFile()</code>	It gets the canonical form of this abstract pathname.
<code>String getCanonicalPath()</code>	It gets the canonical pathname string of this abstract pathname.
<code>long getFreeSpace()</code>	It gets the number of unallocated bytes in the partition named by this abstract pathname.
<code>String getName()</code>	It gets the name of the file or directory given by this abstract pathname.
<code>String getParent()</code>	It gets the pathname string of this abstract pathname's parent or gets null if this pathname does not name a parent directory.
<code>File getParentFile()</code>	It gets the abstract pathname of this abstract pathname's parent or gets null if this pathname does not name a parent directory.
<code>String getPath()</code>	It converts this abstract pathname into a pathname string.
<code>long getTotalSpace()</code>	It gets the size of the partition named.
<code>long getUsableSpace()</code>	It gets the number of bytes available on the partition named.
<code>int hashCode()</code>	It computes a hash code for this abstract pathname.
<code>boolean isAbsolute()</code>	It returns True if this abstract pathname is absolute.
<code>boolean isDirectory()</code>	It returns True if the file given by this abstract pathname is a directory.
<code>boolean isFile()</code>	It returns True if the file given by this abstract pathname is a normal file.
<code>boolean isHidden()</code>	It returns True if the file named by this abstract pathname is a hidden file.
<code>long lastModified()</code>	It returns the time that the file given by this abstract pathname was last modified.
<code>long length()</code>	It returns the length of the file given by this abstract pathname.
<code>String[] list()</code>	It returns an array of strings naming the files and directories in the directory given by this abstract pathname.
<code>String[] list(FilenameFilter filter)</code>	It returns an array of strings naming the files and directories in the directory given by this pathname that satisfies the given filter.
<code>File[] listFiles()</code>	It returns an array of abstract pathnames indicating the files in the directory.
<code>File[] listFiles(FileFilter filter)</code>	It returns an array of abstract pathnames indicating the files and directories in the directory that fulfills the given filter.
<code>File[] listFiles(FilenameFilter filter)</code>	It accepts the <code>FilenameFilter</code> object and returns an array of abstract pathnames indicating the files and directories in the directory that fulfills the given filter.
<code>static File[] listRoots()</code>	It lists the available file system roots.
<code>boolean mkdir()</code>	It creates the directory named by this abstract pathname.
<code>boolean mkdirs()</code>	It creates the directory named by this pathname, including any necessary but nonexistent parent directories.
<code>boolean renameTo(File dest)</code>	It renames the file given by this pathname.
<code>boolean setExecutable(boolean executable)</code>	It sets owner's execute permission for this abstract pathname.
<code>boolean setExecutable(boolean executable, boolean ownerOnly)</code>	It sets owner's or everybody's execute permission for this abstract pathname.
<code>boolean setLastModified(long</code>	It sets the last modified time of the file or directory.

Table 6.47: Methods of the File class

Method	Does this
<code>time()</code>	
<code>boolean setReadable(boolean readable)</code>	It sets owner's read permission for this abstract pathname.
<code>boolean setReadable(boolean readable, boolean ownerOnly)</code>	It sets read permission for the owner of the file or other users for the given abstract pathname.
<code>boolean setReadOnly()</code>	It marks the file or directory named by this pathname so that it can only be read.
<code>boolean setWritable(boolean writable)</code>	It sets owner's write permission for this abstract pathname.
<code>boolean setWritable(boolean writable, boolean ownerOnly)</code>	It sets owner's or everybody's write permission for this abstract pathname.
<code>String toString()</code>	It gets the pathname string of this abstract pathname.
<code>URI toURI()</code>	It constructs a file URI that represents this abstract pathname.
<code>URL toURL()</code>	It converts this abstract pathname into a URL. It is recommended that abstract pathname should be converted to URI first using the <code>toURI()</code> method and then use <code>URI.toURL()</code> as it helps in escaping illegal characters.

Note that you can use `isFile()` to determine whether an item is a true file or something else, such as a directory (which you can check with `isDirectory()`) or a named pipe. In other words, the `isFile()` method returns true if called on a file (and behaves like one) and false if called on a directory. Use `toURL()` to convert a file's path into a URL, rename files, and do more with the `File` class. The `isAbsolute()` method returns true if the file has an absolute path and false if it is relative.

Let's look at an example. Here, we put several of the methods of the `File` class to work, getting some information about a file named `file.txt`. Here's what the code looks like:

```
import java.io.File;
class FileDemo {
    public static void main(String args[]) {
        File file1 = new File("file.txt");
        System.out.println("File: " + file1.getName() + (file1.isFile() ?
            " is a file" : " is a named pipe or directory"));
        System.out.println("Size: " + file1.length());
        System.out.println("Path: " + file1.getPath());
        System.out.println("Parent: " + file1.getParent());
        System.out.println("Absolute Path: " + file1.getAbsolutePath());
        System.out.println("File was last modified: " +
            file1.lastModified());
        System.out.println(file1.exists() ? "File exists" :
            "File does not exist");
        System.out.println(file1.canRead() ? "File can be read from" :
            "File cannot be read from");
        System.out.println(file1.canWrite() ? "File can be written to" :
            "File cannot be written to");
        System.out.println(file1.isDirectory() ? "File is a directory" :
            "File is not a directory");
        System.out.println(file1.isFile() ? "is a true file" :
            "is a special file");
        System.out.println(file1.isAbsolute() ? "is absolute" :
            "is not absolute"); }
}
```

Create a file named `file.txt` before executing this code file in the same location where you are saving Java and class files. The result of this code is shown in Figure 6.7 (note that the time the file was last modified is returned

in the usual fashion—in milliseconds since 1/1/70—which you can convert into a more readable date with the `Date` class):

```

C:\Windows\system32\cmd.exe
E:\Java8>javac FileDemo.java
E:\Java8>java FileDemo
File: file.txt is a file
Size: 28
Path: file.txt
Parent: null
Absolute Path: E:\Java8\file.txt
File was last modified: 1467265537461
File exists
File can be read from
File can be written to
File is not a directory
is a true file
is not absolute
E:\Java8>

```

Figure 6.7: Creating a file

In the preceding case, `getName()` returns the name of the file, `getParent()` returns the name of the parent directory, and `exists()` returns true.

The `File` class is not symmetrical, which signifies that there are many methods that allow you to monitor the properties of a simple file, though no simultaneous function exists to change those attributes. This class also includes two useful utility methods:

- ❑ `boolean renameTo(File newname)` —Here `newname` refers to the new filename. This will return true upon success and false if the directory cannot be renamed.
- ❑ `boolean delete()` —This utility method deletes a directory if the directory is empty. This will return true if it deletes the file and false if the file cannot be deleted.

As you can see, using the `File` class, you can determine a great deal about files and directories.

Using the *FileReader* Class

You use the `FileReader` class to create a character-based stream that reads from a file. Here's the inheritance diagram for this class:

```

java.lang.Object
|___java.io.Reader
      |___java.io.InputStreamReader
            |___java.io.FileReader

```

This class only defines constructors in addition to the functionality it inherits from `InputStreamReader`, and you'll find them in Table 6.41. All the constructors can throw a `FileNotFoundException`.

Here's an example. In this case, we'll open a file, `file.txt`, having the contents "Welcome to Dreamtech Press", read that file's data into a buffer, and then display the data:

```

import java.io.*;
class FileReaderDemo
{
    public static void main(String args[]) throws Exception
    {
        FileReader filereader = new FileReader("file.txt");
        char data[] = new char[1024];
        int charsread = filereader.read(data);
        System.out.println(new String(data, 0, charsread));
        filereader.close();
    }
}

```

The output of this program is shown in Figure 6.8, reading the file:



Figure 6.8: Reading the text from the file

As you can see, working with `FileReader` is relatively a simple process in Table 6.48:

Table 6.48: Constructors of the `FileReader` class

Constructor	Does this
<code>FileReader(File file)</code>	It constructs a <code>FileReader</code> object
<code>FileReader(FileDescriptor fd)</code>	It constructs a <code>FileReader</code> object from a file descriptor
<code>FileReader(String fileName)</code>	It constructs a <code>FileReader</code> object from a file name

Using the *FileWriter* Class

The `FileWriter` class is the character-based writer counterpart of the `FileReader` class (see the previous solution for more details). Here's the inheritance diagram for the `FileWriter` class:

```

java.lang.Object
|___ java.io.Writer
      |___ java.io.OutputStreamWriter
            |___ java.io.FileWriter

```

This class only defines constructors in addition to the functionality it inherits from `OutputStreamWriter`, and you'll find them in Table 6.49:

Table 6.49: Constructors of the `FileWriter` class

Constructor	Does this
<code>FileWriter(File file)</code>	It constructs a <code>FileWriter</code> object from a <code>File</code> object
<code>FileWriter(File file, boolean append)</code>	It constructs a <code>FileWriter</code> object given a <code>File</code> object
<code>FileWriter(FileDescriptor fd)</code>	It constructs a <code>FileWriter</code> object from a file descriptor
<code>FileWriter(String fileName)</code>	It constructs a <code>FileWriter</code> object from a file name
<code>FileWriter(String fileName, boolean append)</code>	It constructs a <code>FileWriter</code> object for appending

The constructors can throw an `IOException`.

Creation of a `FileWriter` is not dependent on the file that already exists. It creates the file before opening it for output when an object is already created. An `IOException` is thrown in case you attempt to open a read-only file. Here's an example: We'll write data from a character buffer to a file, character by character, all at once, and in part, using `FileWriter`:

```

import java.io.*;
class FileWriterDemo {
    public static void main(String args[]) throws Exception {
        char data[] = {'T','h','i','s',' ','i','s',' ','a',
            ' ','s','t','r','i','n','g',' ','o','f',

```

```

        ' ','t','e','x','t','.' } };
        FileWriter filewriter1 = new FileWriter("f1.txt");
        for (int loop_index = 0; loop_index < data.length; loop_index++) {
            filewriter1.write(data[loop_index]); }
        FileWriter filewriter2 = new FileWriter("f2.txt");
        filewriter2.write(data);
        FileWriter filewriter3 = new FileWriter("f3.txt");
        filewriter3.write(data, 5, 10);
        filewriter3.append("Dreamtech Press");
        filewriter1.close();
        filewriter2.close();
        filewriter3.close(); }
    }
}

```

After executing the code, three files named f1.txt, f2.txt and f3.txt will be created in the same location where you have saved the java file.

Working with the **RandomAccessFile** Class

Using the `RandomAccessFile` class, you can move around in a file using the `seek()` method. That's what random access is all about—being able to move around in a file at random. The inheritance diagram for the `RandomAccessFile` class is as follows:

```

java.lang.Object
|___java.io.RandomAccessFile

```

`RandomAccessFile` is not the child class of `InputStream` or `OutputStream`; instead, it encapsulates a random-access file. It also implements the `Closeable` interface. `RandomAccessFile` supports positioning requests, i.e., you can position the file pointer within the file.

With random access comes the type of access:

- ☐ `r`—File is read and not written.
- ☐ `rw`—File is opened in the read-write mode.
- ☐ `rws`—File is opened for read-write operation and every change to the content/metadata needs to be written synchronously to the underlying storage device.
- ☐ `rwd`—File is opened for read-write operation and every change to the content needs to be written synchronously to the underlying storage device.

The `seek()` method is used to set the current position of the file pointer within the file. You'll find the constructors of the `RandomAccessFile` class in Table 6.50 and its methods in Table 6.51:

Table 6.50: Constructors of the `RandomAccessFile` class

Constructor	Does this
<code>RandomAccessFile(File file, String mode)</code>	It constructs a random-access file stream to read from and optionally to write to the file given by the <code>File</code> argument.
<code>RandomAccessFile(String name, String mode)</code>	It constructs a random-access file stream to read from and optionally to write to a file with the given name.

Table 6.51: Methods of the `RandomAccessFile` class

Method	Does this
<code>void close()</code>	It closes this random-access file stream.
<code>FileChannel getChannel()</code>	It gets the unique <code>File Channel</code> object associated with this file.
<code>FileDescriptor getFD()</code>	It gets the file descriptor object associated with this stream.
<code>long getFilePointer()</code>	It gets the current offset in this file.
<code>long length()</code>	It gets the length of this file.
<code>int read()</code>	It gets a byte of data from this file.

Table 6.51: Methods of the RandomAccessFile class

Method	Does this
<code>int read(byte[] b)</code>	It gets up to <code>b.length</code> bytes of data from this file into an array of bytes.
<code>int read(byte[] b, int off, int len)</code>	It gets up to <code>len</code> bytes of data from this file into an array of bytes.
<code>boolean readBoolean()</code>	It gets a <code>boolean</code> from this file.
<code>byte readByte()</code>	It gets a signed 8-bit value from this file.
<code>char readChar()</code>	It gets a Unicode character from this file.
<code>double readDouble()</code>	It gets a <code>double</code> from this file.
<code>float readFloat()</code>	It gets a <code>float</code> from this file.
<code>void readFully(byte[] b)</code>	It gets <code>b.length</code> bytes from this file into the byte array, starting at the current file pointer.
<code>void readFully(byte[] b, int off, int len)</code>	It gets exactly <code>len</code> bytes from this file into the byte array, starting at the current file pointer.
<code>int readInt()</code>	It gets a signed 32-bit integer from this file.
<code>String readLine()</code>	It gets the next line of text from this file.
<code>long readLong()</code>	It gets a signed 64-bit integer from this file.
<code>short readShort()</code>	It gets a signed 16-bit number from this file.
<code>int readUnsignedByte()</code>	It gets an unsigned 8-bit number from this file.
<code>int readUnsignedShort()</code>	It gets an unsigned 16-bit number from this file.
<code>String readUTF()</code>	It gets a string from this file.
<code>void seek(long pos)</code>	It sets the file-pointer offset at which the next read or write happens.
<code>void setLength(long newLength)</code>	It sets the length of this file.
<code>int skipBytes(int n)</code>	It attempts to skip over <code>n</code> bytes of input.
<code>void write(byte[] b)</code>	It writes <code>b.length</code> bytes from the given byte array to this file, starting at the file pointer.
<code>void write(byte[] b, int off, int len)</code>	It writes <code>len</code> bytes from the given byte array, starting at offset <code>off</code> , to this file.
<code>void write(int b)</code>	It writes the given byte to this file.
<code>void writeBoolean(boolean v)</code>	It writes a <code>boolean</code> to the file as a 1-byte value.
<code>void writeByte(int v)</code>	It writes a byte to the file as a 1-byte value.
<code>void writeBytes(String s)</code>	It writes the string to the file as a sequence of bytes.
<code>void writeChar(int v)</code>	It writes a char to the file as a 2-byte value, high byte first.
<code>void writeChars(String s)</code>	It writes a String to the file as a sequence of characters.
<code>void writeDouble(double v)</code>	It converts the <code>double</code> argument to a long and then writes that long value to the file.
<code>void writeFloat(float v)</code>	It converts the <code>float</code> argument to an <code>int</code> and then writes that <code>int</code> value to the file.
<code>void writeInt(int v)</code>	It writes an <code>int</code> to the file as 4 bytes, high byte first.
<code>void writeLong(long v)</code>	It writes a long to the file as 8 bytes, high byte first.
<code>void writeShort(int v)</code>	It writes a short to the file as 2 bytes, high byte first.
<code>void writeUTF(String str)</code>	It writes a string to the file using UTF-8 encoding.

Note that one of the attractions of the `RandomAccessFile` class is that you can both read and write using this class, without having to break things up into input and output objects.

Working with Character Arrays

A string is a contiguous array of characters, so different operations can be performed on a string by using the array of characters. The operations performed on strings include dividing them up into parts called substrings, combining them together to create new strings, and replacing parts of them with other strings.

Using the *CharArrayReader* Class

You use `CharArrayReader` to read from a character array. Here's the inheritance diagram for this class:

```

java.lang.Object
|___java.io.Reader
|___java.io.CharArrayReader

```

You'll find the fields of the `CharArrayReader` class in Table 6.52, its constructors in Table 6.53, and its methods in Table 6.54:

Table 6.52: Fields of the CharArrayReader class

Field	Does this
protected char[] buf	The character buffer.
protected int count	The number of characters in the buffer.
protected int markedPos	The position of the current mark in the buffer.
protected int pos	The current buffer position.

Table 6.53: Constructors of the CharArrayReader class

Constructor	Does this
<code>CharArrayReader(char[] buf)</code>	It constructs a <code>CharArrayReader</code> object from the given array of characters.
<code>CharArrayReader(char[] buf, int offset, int length)</code>	It constructs a <code>CharArrayReader</code> object from the given array of characters.

Table 6.54: Methods of the CharArrayReader class

Method	Does this
<code>void close()</code>	It closes the stream.
<code>void mark(int readAheadLimit)</code>	It marks the present position in the stream.
<code>boolean markSupported()</code>	It returns <code>True</code> if this stream supports the mark operation.
<code>int read()</code>	It reads a single character.
<code>int read(char[] b, int off, int len)</code>	It reads characters into a portion of an array.
<code>boolean ready()</code>	It tells whether this stream is ready to be read
<code>void reset()</code>	It resets the stream to the most recent mark
<code>long skip(long n)</code>	It skips characters

Here's an example. In this case, we just read characters from an array and print them out:

```

import java.io.*;
public class CharArrayReaderDemo {
    public static void main(String args[]) throws IOException {
        char data[] = {'T','h','i','s',' ','i','s',' ','a',
                       ' ','s','t','r','i','n','g',' ','o','f',
                       ' ','t','e','x','t','.'};
    }
}

```

```

CharArrayReader chararrayreader = new CharArrayReader(data);
int character;
while((character = chararrayreader.read()) != -1) {
    System.out.print((char)character); }
}

```

Here's the output of this code:

```

E:\Java8\chapter18>java CharArrayReaderDemo
This is a string of text.

```

Take the case of this example which constructs one object using the entire sentence, while the other object has only the first six letters:

```

import java.io.*;
public class CharArrayReaderDemo1 {
    public static void main(String args[]) throws IOException {
        String t = "The quick brown fox ran over the mount";
        int l = t.length();
        char w[] = new char[l];
        t.getChars(0, l, w, 0);
        CharArrayReader input1 = new CharArrayReader(w);
        CharArrayReader input2 = new CharArrayReader(w, 0, 6);
        int j;
        System.out.println("All the characters:");
        while((j = input1.read()) != -1) { System.out.print((char)j); }
        System.out.println();
        System.out.println("First 6 characters:");
        while((j = input2.read()) != -1) { System.out.print((char)j); }
        System.out.println(); }
}

```

The result of the code is shown in Figure 6.9:

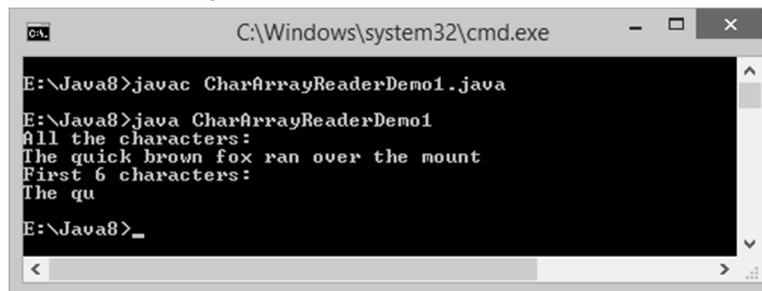


Figure 6.9: Writing the characters from the array

Using the **CharArrayWriter** Class

The `CharArrayWriter` class is the writer counterpart of the `CharArrayReader` class (see the previous solution). You use this class to write to character arrays. Here's the inheritance diagram for this class:

```

java.lang.Object
|___ java.io.Writer
    |___ java.io.CharArrayWriter

```

You'll find the fields of the `CharArrayWriter` class in Table 6.55, its constructors in Table 6.56, and its methods in Table 6.57:

Table 6.55: Fields of the <code>CharArrayWriter</code> class	
Field	Does this
<code>protected char[] buf</code>	The buffer where data is stored.
<code>protected int count</code>	The number of characters in the buffer.

Table 6.56: Constructors of the CharArrayWriter class

Constructor	Does this
<code>CharArrayWriter()</code>	It constructs a new <code>CharArrayWriter</code> object.
<code>CharArrayWriter(int initialSize)</code>	It constructs a new <code>CharArrayWriter</code> object with the given initial size.

Table 6.57: Methods of the CharArrayWriter class

Method	Does this
<code>CharArrayWriter append(char c)</code>	It appends the specified character to this writer.
<code>CharArrayWriter append(CharSequence csq)</code>	It appends the specified character sequence to this writer.
<code>CharArrayWriter append(CharSequence csq, int start, int end)</code>	It appends a subsequence of the specified character sequence to this writer.
<code>void close()</code>	It closes the stream.
<code>void flush()</code>	It flushes the stream.
<code>void reset()</code>	It resets the buffer.
<code>int size()</code>	It gets the current size of the buffer.
<code>char[] toCharArray()</code>	It gets a copy of the input data.
<code>String toString()</code>	It converts input data to a string.
<code>void write(char[] c, int off, int len)</code>	It writes characters to the buffer.
<code>void write(int c)</code>	It writes a character to the buffer.
<code>void write(String str, int off, int len)</code>	It writes a portion of a string to the buffer.
<code>void writeTo(Writer out)</code>	Writes contents of buffer to another character-stream.

Let's concentrate on two constructors. In the first form, a buffer with a default size is created. In the second, a buffer is created with a size equal to that specified by `initialSize`. The `buf` field of the `CharArrayWriter` class stores the buffer, which will be increased automatically. The number of characters stored in the buffer is contained in the `count` field of the `CharArrayWriter` class. Both `buf` and `count` are protected fields.

Look at this example which demonstrates this concept of `CharArrayWriter`:

```
import java.io.*;
class CharArrayWriterDemo {
    public static void main(String args[]) throws IOException {
        CharArrayWriter f = new CharArrayWriter();
        String w = "I/O stream classes have become a thicket in Java";
        char buf[] = new char[w.length()];
        w.getChars(0, w.length(), buf, 0);
        f.write(buf);
        System.out.println("\nBuffer of default size:");
        System.out.println(f.toString());
        System.out.println("\n\nwhat goes into the array-->");
        char A[] = f.toCharArray();
        for (int j=0; j<A.length; j++) { System.out.print(A[j]); }
        System.out.println("\n\nFinally going to FileWriter-->");
        FileWriter f2 = new FileWriter("cofbean.txt");
        f.writeTo(f2); f2.close();
        System.out.println("\n..... after a quick reset");
        f.reset();
    }
}
```

```

        for (int j=0; j<3; j++)
            f.write('0'); System.out.println(f.toString());
    }
}

```

This code will create a file `cofbean.txt` in the current directory to store the string 'I/O stream classes have become a thicket in Java'. But after the call to `reset()`, the three 0's end up in the beginning. See the output of the code in Figure 6.10:

```

C:\Windows\system32\cmd.exe
E:\Java8>javac CharArrayWriterDemo.java
E:\Java8>java CharArrayWriterDemo
Buffer of default size:
I/O stream classes have become a thicket in Java

What goes into the array-->
I/O stream classes have become a thicket in Java

Finally going to FileWriter<-->
..... after a quick reset
000
E:\Java8>_

```

Figure 6.10: Using the `CharArrayWriter` class

Working with Buffers

In the `java.io` package, a file or stream is a sequence of bytes or byte array(s). The `DataInput` and `DataOutput` interfaces can help out, but they work on streams and don't allow byte order to be specified. Here, the NIO Buffer class and its subclasses come in use. A commonly used buffer class is `ByteBuffer`.

It allows a byte array (or block of memory/file) to be managed as a buffer with methods to deal with the current read/write position, maximum position, and array of bytes.

Using the *BufferedReader* Class

The `BufferedReader` class provides a buffered character reader class and the `readLine()` method. Here's the inheritance diagram for `BufferedReader`:

```

java.lang.Object
|___java.io.Reader
|___java.io.BufferedReader

```

The `BufferedReader` class consists of two constructors. In the first one, a buffered character-stream uses a default buffer size. In the second, the size of the buffer is passed in `sz`. You'll find the constructors of the `BufferedReader` class in Table 6.58 and its methods in Table 6.59:

Table 6.58: Constructors of the `BufferedReader` class

Constructor	Does this
<code>BufferedReader(Reader in)</code>	It constructs a buffering character input stream.
<code>BufferedReader(Reader in, int sz)</code>	It constructs a buffering character input stream that uses an input buffer of the given size.

Table 6.59: Methods of the `BufferedReader` class

Method	Does this
<code>void close()</code>	It closes the stream.
<code>void mark(int readAheadLimit)</code>	It marks the present position in the stream.
<code>boolean markSupported()</code>	It returns <code>True</code> if this stream supports the <code>mark</code> operation.

Table 6.59: Methods of the BufferedReader class

Method	Does this
<code>int read()</code>	It reads a single character.
<code>int read(char[] cbuf, int off, int len)</code>	It reads characters of a portion of an array.
<code>String readLine()</code>	It reads a line of text.
<code>boolean ready()</code>	It tells whether this stream is ready to be read.
<code>void reset()</code>	It resets the stream to the most recent mark.
<code>long skip (long n)</code>	It skips characters.

Here's another example using the `BufferedReader` class. In this case, we read the file `file4.txt` line by line with the `readLine()` method. Here's the content of that file:

```
This is a wonderful program where you can find the capability of File() class.
Some streams provide data buffering.
And some do not.
Some are designed to work with data in memory buffers.
Some with files.
```

Here's the code:

```
import java.io.*;
class BufferedReaderDemo
{
    public static void main(String args[]) throws Exception
    {
        FileReader filereader = new FileReader("file4.txt");
        BufferedReader bufferedreader = new BufferedReader(filereader);
        String instring;
        while((instring = bufferedreader.readLine()) != null) {
            System.out.println(instring); } filereader.close(); }
}
```

Here's the output of the code when it's run:

```
E:\Java8\>java BufferedReaderDemo
This is a wonderful program where you can find the capability of File() class.
Some streams provide data buffering.
And some do not.
Some are designed to work with data in memory buffers.
Some with files.
```

Using the **BufferedWriter** Class

The `BufferedWriter` class is the writer counterpart of the `BufferedReader` class (see the previous solution). Here's the inheritance diagram for the `BufferedWriter` class:

```
java.lang.Object
|___java.io.Writer
|___java.io.BufferedWriter
```

You'll find the constructors of the `BufferedWriter` class in Table 6.60 and its methods in Table 6.61:

Table 6.60: Constructors of the BufferedWriter class

Constructor	Does this
<code>BufferedWriter(Writer out)</code>	It constructs a buffered character-output stream.
<code>BufferedWriter(Writer out, int sz)</code>	It constructs a new-buffered character-output stream that uses an output buffer of the given size.

Table 6.61: Methods of the BufferedWriter class

Method	Does this
<code>void close()</code>	It closes the stream.
<code>void flush()</code>	It flushes the stream.
<code>void newLine()</code>	It writes a line separator.
<code>void write(char[] cbuf, int off, int len)</code>	It writes a section of an array of characters.
<code>void write(int c)</code>	It writes a single character.
<code>void write(String s, int off, int len)</code>	It writes a portion of a string.

In Table 6.60, the first form of constructor creates a buffered stream with a default size and in the second constructor, the size of the buffer is passed in `sz`.

Working with the **PushbackReader** Class

The `PushbackReader` class enables characters to be pushed back into the stream. When you use this class's `unread()` method, you can return one or more characters to the stream. Here's the inheritance diagram for the `PushbackReader` class:

```

java.lang.Object
|
+--- java.io.Reader
|      |
|      +--- java.io.FilterReader
|             |
|             +--- java.io.PushbackReader

```

You'll find the constructors of the `PushbackReader` class in Table 6.62 and its methods in Table 6.63:

Table 6.62: Constructors of the PushbackReader class

Constructor	Does this
<code>PushbackReader(Reader in)</code>	It creates a new pushback reader.
<code>PushbackReader(Reader in, int size)</code>	It creates a new pushback reader with a pushback buffer of the given size.

Table 6.63: Methods of the PushbackReader class

Method	Does this
<code>void close()</code>	It closes the stream.
<code>void mark(int readAheadLimit)</code>	It marks the present position in the stream.
<code>boolean markSupported()</code>	It returns <code>True</code> if this stream supports the mark operation.
<code>int read()</code>	It reads a single character.
<code>int read(char[] cbuf, int off, int len)</code>	It reads characters into a portion of an array.
<code>boolean ready()</code>	It tells whether this stream is ready to be read.
<code>void reset()</code>	It resets the stream.
<code>long skip(long n)</code>	It keeps character while reading.
<code>void unread(char[] cbuf)</code>	It pushes back an array of characters.
<code>void unread(char[] cbuf, int off, int len)</code>	It pushes back a section of an array of characters.
<code>void unread(int c)</code>	It pushes back a single character.

In Table 6.62, the first form of constructor creates a buffered stream that allows one character to be pushed back at a time and in the second one, the size of the pushback buffer is passed in size.

Working with the **PrintWriter** Class

The methods of this class are used to print formatted representation of objects to a text output stream. Methods of this class never throw I/O exception. If automatic flushing is enabled, that will take place only when one of the `println()`, `printf()`, or `format()` method is invoked. We can check for some error using the `checkError()` method.

Here is the inheritance diagram for the `PrintWriter` class:

```

java.lang.Object
|___java.io.Writer
|___java.io.PrintWriter

```

You'll find the constructors of the `PrintWriter` class in Table 6.64 and its methods in Table 6.65:

Table 6.64: Constructors of the <code>PrintWriter</code> class	
Constructor	Does this
<code>PrintWriter(File file)</code>	It creates a new <code>PrintWriter</code> with a specified file without automatic line flushing.
<code>PrintWriter(File file, String cs)</code>	It creates a new <code>PrintWriter</code> with a specified file and charset, without automatic line flushing.
<code>PrintWriter(OutputStream out)</code>	It creates a new <code>PrintWriter</code> with existing <code>OutputStream</code> without automatic line flushing.
<code>PrintWriter (OutputStream out, boolean autoFlush)</code>	It creates a new <code>PrintWriter</code> with existing <code>OutputStream</code> .
<code>PrintWriter(String fileName)</code>	It creates a new <code>PrintWriter</code> without automatic line flushing with a given file name.
<code>PrintWriter(String fileName, String cs)</code>	It creates a new <code>PrintWriter</code> without automatic line flushing with a given file name and charset.
<code>PrintWriter(Writer out)</code>	It creates a <code>PrintWriter</code> without automatic line flushing.
<code>PrintWriter(Writer out, boolean autoFlush)</code>	It creates a <code>PrintWriter</code> .

The flushing of output buffer, whenever the `println()`, `printf()`, or `format()` method is called, is controlled by the `autoFlush` parameter. If this parameter is true, the flushing automatically takes place.

Table 6.65: Methods of the <code>PrintWriter</code> class	
Method	Does this
<code>PrintWriter append(char c)</code>	It appends the given character to the writer.
<code>PrintWriter append(CharSequence csq)</code>	It appends the given character sequence to the writer.
<code>PrintWriter append(CharSequence csq, int start, int end)</code>	It appends subsequence of given character sequence to the writer.
<code>boolean checkError()</code>	It flushes the stream if not closed and checks its error states.
<code>void clearError()</code>	It clears error state of the stream.
<code>void close()</code>	It closes stream, returns system resources associated with stream.
<code>void flush()</code>	It flushes the stream.

Table 6.65: Methods of the `PrintWriter` class

Method	Does this
<code>PrintWriter format(Locale l, String format, Object... args)</code>	It writes args according to the formatted string and let you specify locale.
<code>PrintWriter format(String format, Object... args)</code>	It writes args according to the formatted string with default locale.
<code>void print(boolean b)</code>	It prints a boolean value.
<code>void print(char c)</code>	It prints a character.
<code>void print(char[] c)</code>	It prints an array of characters.
<code>void print(double d)</code>	It prints a double-precision floating-point number.
<code>void print(float f)</code>	It prints a floating-point number.
<code>void print(int i)</code>	It prints an integer.
<code>void print(long l)</code>	It prints a long integer.
<code>void print(Object ob)</code>	It prints an object
<code>void print(String s)</code>	It prints a string
<code>PrintWriter printf(Locale l, String fmt, Object... args)</code>	It writes arguments to this writer according to formatted string and let you specify Locale.
<code>PrintWriter printf(String fmt, Object... args)</code>	It writes arguments to this writer according to formatted string.
<code>void println()</code>	It terminates the current line by writing the line separator string.
<code>void println(boolean x)</code>	It prints a boolean value and then terminates the line.
<code>void println(char x)</code>	It prints a character and then terminates the line.
<code>void println(char[] x)</code>	It prints an array of characters and then terminates the line.
<code>void println(double x)</code>	It prints a double-precision floating-point number and then terminates the line.
<code>void println(float x)</code>	It prints a floating-point number and then terminates the line.
<code>void println(int x)</code>	It prints an integer and then terminates the line.
<code>void println(long x)</code>	It prints a long integer and then terminates the line.
<code>void println(Object x)</code>	It prints an Object and then terminates the line.
<code>void println(String x)</code>	It prints a String and then terminates the line.
<code>protected void setError()</code>	It indicates that an error has occurred.
<code>void write(char[] buf)</code>	It writes an array of characters.
<code>void write(char[] buf, int off, int len)</code>	It writes a portion of an array of characters.
<code>void write(int c)</code>	It writes a single character.

Table 6.65: Methods of the PrintWriter class

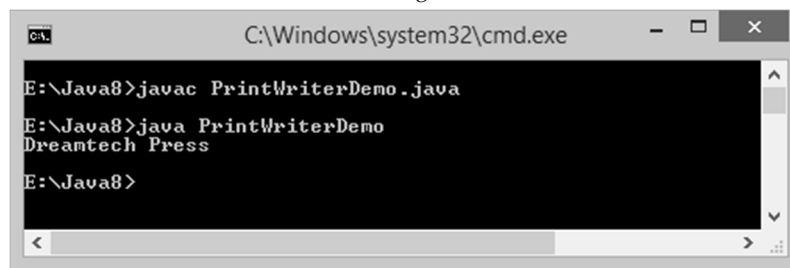
Method	Does this
<code>void write(String s)</code>	It writes a string.
<code>void write(String s, int off, int len)</code>	It writes a portion of a string.

Similarly, we have `set`, `println()` methods and `write()` methods of this class. The `format()` method is also supported, which works exactly like the `printf()` method.

Here is an example:

```
import java.io.PrintWriter;
public class PrintWriterDemo
{
    public static void main(String[] args)
    {
        PrintWriter out = new PrintWriter(System.out, true);
        out.println("Dreamtech Press");
    }
}
```

Compile and run the code, and the result is shown in Figure 6.11:

**Figure 6.11: Working with the PrintWriter class**

Working with the **StreamTokenizer** Class

You can use a `StreamTokenizer` class to break an input stream into tokens, such as words. Here's the inheritance diagram for the `StreamTokenizer` class:

```
java.lang.Object
|___java.io.StreamTokenizer
```

You'll find the fields of the `StreamTokenizer` class in Table 6.66, its constructors in Table 6.67, and its methods in Table 6.68:

Table 6.66: Fields of the StreamTokenizer class

Field	Does this
<code>double nval</code>	The value of a number token.
<code>String sval</code>	The string giving the characters of a word token.
<code>static int TT_EOF</code>	A constant indicating that end of the stream is read.
<code>static int TT_EOL</code>	A constant indicating that the end of the line has been read.
<code>static int TT_NUMBER</code>	A constant indicating that a number token has been read.

Table 6.66: Fields of the StringTokenizer class

Field	Does this
<code>static int TT_WORD</code>	A constant indicating that a word token has been read.
<code>int ttype</code>	It contains the type of the token just read.

Table 6.67: Constructors of the StringTokenizer class

Constructor	Does this
<code>StringTokenizer (InputStream is)</code>	It is deprecated as in JDK 1.1, the preferred way to tokenize an input stream that converts it into a character stream. For example, <pre>Reader rd= new BufferedReader(new InputStreamReader(is)); StringTokenizer strt= new StringTokenizer(rd);</pre>
<code>StringTokenizer(Reader r)</code>	It constructs a tokenizer that parses the given character-stream.

Table 6.68: Methods of the StringTokenizer class

Method	Does this
<code>void commentChar(int ch)</code>	The character argument starts a single-line comment.
<code>void eolIsSignificant (boolean flag)</code>	It returns True if the ends of lines are treated as tokens.
<code>int lineno()</code>	It gets the current line number.
<code>void lowerCaseMode (boolean fl)</code>	It determines whether word tokens are automatically lowercased.
<code>int nextToken()</code>	It parses the next token from the input stream of this tokenizer.
<code>void ordinaryChar(int ch)</code>	It specifies that the character argument is ordinary in this tokenizer.
<code>void ordinaryChars (int low, int hi)</code>	It specifies that all characters in the range low to high are considered ordinary in this tokenizer.
<code>void parseNumbers()</code>	It specifies that numbers should be parsed by this tokenizer.
<code>void pushBack()</code>	It pushes back a token.
<code>void quoteChar(int ch)</code>	It indicates that matching pairs of this character delimit string constants.
<code>void resetSyntax()</code>	It resets this tokenizer's syntax table so that all characters are "ordinary."
<code>void slashSlashComments (boolean flag)</code>	It sets whether the tokenizer recognizes C++-style comments.
<code>void slashStarComments (boolean flag)</code>	It sets whether the tokenizer recognizes C-style comments.
<code>String toString()</code>	It gets the string representation of the current stream token.
<code>void whitespaceChars (int low, int hi)</code>	It makes all characters in the range low to high white space characters.

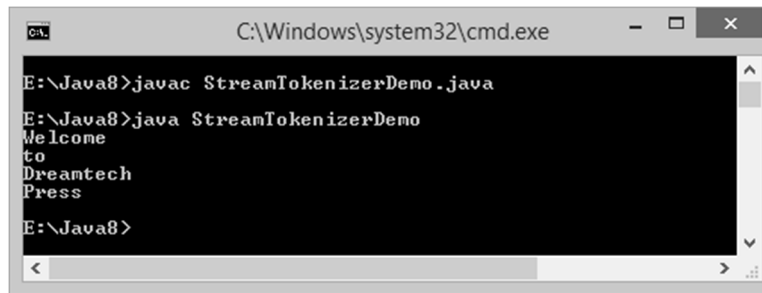
Table 6.68: Methods of the StringTokenizer class

Method	Does this
<code>void wordChars (int low, int hi)</code>	It specifies that all characters in the range low to high are word characters.

Here's an example in which we use `FileReader` to read from a file, `file.txt`, whose content is the string "Welcome to Dreamtech Press". We divide that text into words, verify words with the `TT_WORD` field, and retrieve the current value of token as a string using the `sval` field, which is as follows:

```
import java.io.*;
class StringTokenizerDemo
{
    public static void main(String args[]) throws Exception
    {
        FileReader filereader = new FileReader("file.txt");
        StringTokenizer streamtokenizer = new StringTokenizer(filereader);
        String instring;
        while(streamtokenizer.nextToken() != StringTokenizer.TT_EOF) {
            if(streamtokenizer.ttype == StringTokenizer.TT_WORD)
                System.out.println(streamtokenizer.sval); }
        filereader.close(); }
}
```

The output of this example is shown in Figure 6.12 using the `StringTokenizer` class:

**Figure 6.12: Using the StringTokenizer class**

Implementing the **Serializable** Interface

Serialization is the process of writing objects to a stream and reading them from the stream. This is particularly useful when you want to save the state of your program to a persistent storage area, such as a file. Later on, you may restore these objects by the process of deserialization. To be serialized, your objects must implement the `Serializable` interface.

This interface has no fields, constructors, or methods—it just shows that an object is serializable. To work with serializable objects, you use the `ObjectInputStream` and `ObjectOutputStream` classes, which are derived from `InputStream` and `OutputStream` respectively. Here's the inheritance diagram for `ObjectInputStream`:

```
java.lang.Object
|___java.io.InputStream
|___java.io.ObjectInputStream
```

You'll find the constructors of the `ObjectInputStream` class in Table 6.69 and its methods in Table 6.70:

Table 6.69: Constructors of the ObjectInputStream class

Constructor	Does this
<code>protected ObjectInputStream()</code>	It constructs an <code>ObjectInputStream</code> object.

Table 6.69: Constructors of the `ObjectInputStream` class

Constructor	Does this
<code>ObjectInputStream(InputStream in)</code>	It creates an <code>ObjectInputStream</code> object that reads from the given <code>InputStream</code> .

Table 6.70: Methods of the `ObjectInputStream` class

Method	Does this
<code>int available()</code>	It gets the number of bytes that can be read.
<code>void close()</code>	It closes the input stream.
<code>void defaultReadObject()</code>	It reads the nonstatic and nontransient fields of the current class from this stream.
<code>protected boolean enableResolveObject(boolean enable)</code>	It enables the stream to allow objects read from the stream to be replaced.
<code>int read()</code>	It reads a byte of data.
<code>int read(byte[] b, int off, int len)</code>	It reads into an array of bytes.
<code>boolean readBoolean()</code>	It reads in a <code>Boolean</code> .
<code>byte readByte()</code>	It reads an 8-bit byte.
<code>char readChar()</code>	It reads a 16-bit <code>char</code> .
<code>protected ObjectStreamClass readClassDescriptor()</code>	It reads a class descriptor from the serialization stream.
<code>double readDouble()</code>	It reads a 64-bit double.
<code>ObjectInputStream.GetField readFields()</code>	It reads the persistent fields from the stream and makes them available by name.
<code>Float readFloat()</code>	It reads a 32-bit float.
<code>void readFully(byte[] data)</code>	It reads bytes, blocking until all bytes are read.
<code>void readFully(byte[] data, int offset, int size)</code>	It reads bytes, blocking until all bytes are read.
<code>int readInt()</code>	It reads a 32-bit int.
<code>String readLine()</code>	It is deprecated and it uses a buffered stream instead.
<code>long readLong()</code>	It reads a 64-bit long.
<code>Object readObject()</code>	It reads an object from the <code>ObjectInputStream</code> .
<code>protected Object readObjectOverride()</code>	It is called by trusted subclasses of <code>ObjectOutputStream</code> .
<code>Short readShort()</code>	It reads a 16-bit short.
<code>Object readUnshared()</code>	It reads an unshared object from the <code>ObjectInputStream</code> .
<code>protected void readStreamHeader()</code>	It allows subclasses to read and verify their own stream headers.
<code>int readUnsignedByte()</code>	It reads an unsigned 8-bit byte.
<code>int readUnsignedShort()</code>	It reads an unsigned 16-bit short.
<code>String readUTF()</code>	It reads a UTF format string.

Table 6.70: Methods of the ObjectOutputStream class

Method	Does this
void registerValidation(ObjectInputValidation obj, int prio)	It registers an object to be validated before the graph is returned.
protected Class resolveClass(ObjectStreamClass v)	It loads the local class equivalent of the given stream class description.
protected Object resolveObject(Object obj)	It allows trusted subclasses of ObjectOutputStream to substitute one object for another.
protected Class resolveProxyClass (String [] interfaces)	It returns a proxy class that implements the interfaces named in a proxy class descriptor.
int skipBytes(int len)	It skips bytes.

Here's the inheritance diagram for ObjectOutputStream:

```

java.lang.Object
|___java.io.OutputStream
|___java.io.ObjectOutputStream

```

You'll find the constructors of the ObjectOutputStream class in Table 6.71 and its methods in Table 6.72.

Table 6.71: Constructors of the ObjectOutputStream class

Constructor	Does this
protected ObjectOutputStream()	It creates an ObjectOutputStream object.
ObjectOutputStream(OutputStream out)	It constructs an ObjectOutputStream object that writes to the given OutputStream.

Table 6.72: Methods of the ObjectOutputStream class

Method	Does this
protected void annotateClass(Class c1)	It implements this method to allow class data to be stored in the stream.
protected void annotateProxyClass (Class c1)	It implements this method to store custom data in the stream along with descriptors for dynamic proxy classes.
void close()	It closes the stream.
void defaultWriteObject()	It writes the nonstatic and nontransient fields of the current class to this stream.
protected void drain()	It clears any buffered data in ObjectOutputStream.
protected boolean enableReplaceObject(boolean enable)	It enables the stream to replace objects in the stream.
void flush()	It flushes the stream.

Table 6.72: Methods of the ObjectOutputStream class

Method	Does this
<code>ObjectOutputStream.PutField putFields()</code>	It retrieves the object used to buffer persistent fields.
<code>protected Object replaceObject(Object obj)</code>	It allows trusted subclasses of <code>ObjectOutputStream</code> to substitute one object for another.
<code>void reset()</code>	It disregards the state of any objects already written to the stream.
<code>void useProtocolVersion(int version)</code>	It specifies the stream protocol version to use when writing the stream.
<code>void write(byte[] b)</code>	It writes an array of bytes.
<code>void write(byte[] b, int off, int len)</code>	It writes a sub array of bytes.
<code>void write(int data)</code>	It writes a byte.
<code>void writeBoolean(boolean data)</code>	It writes a <code>Boolean</code> .
<code>void writeByte(int data)</code>	It writes an 8-bit byte.
<code>void writeBytes(String data)</code>	It writes a string as a sequence of bytes.
<code>void writeChar(int data)</code>	It writes a 16-bit char.
<code>void writeChars(String data)</code>	It writes a string as a sequence of chars.
<code>void writeDouble(double data)</code>	It writes a 64-bit double.
<code>void writeFields()</code>	It writes the buffered fields to the stream.
<code>void writeFloat(float data)</code>	It writes a 32-bit float.
<code>void writeInt(int data)</code>	It writes a 32-bit int.
<code>void writeLong(long data)</code>	It writes a 64-bit long.
<code>void writeObject(Object obj)</code>	It writes the given object to the <code>ObjectOutputStream</code> .
<code>protected void writeObjectOverride(Object obj)</code>	It is called by trusted subclasses to override the default <code>writeObject()</code> method.
<code>void writeShort(int data)</code>	It writes a 16-bit short.
<code>protected void writeStreamHeader()</code>	It is provided so subclasses can append or prepend their own headers to the stream.
<code>void writeUnshared (Object obj)</code>	It writes an unshared object to the <code>ObjectOutputStream</code> .
<code>void writeUTF(String data)</code>	It is primitive data-write operation of the given string.

Now we are back to serialization. Serialization is also required to implement Remote Method Invocation (RMI). The RMI works in this way: RMI on one machine invokes a method of a Java object on a remote machine; an object may be passed as an argument to that remote method; the sending machine serializes the object and transfers it; and finally, the receiving machine deserializes it.

Here's an example showing how serialization works. In this case, we create a new serializable class named `NewString` (file named as `NewString.java`). This class has a constructor that takes a string and one method `toString()`, that returns that string (note that we implement the `Serializable` interface here):

```
import java.io.*;
class NewString implements Serializable {
    String data;
    public NewString(String instring) { data = instring; }
    public String toString() { return data; }
}
```

After creating a new object of the `NewString` class containing the text "Dreamtech Press!", we use a `FileOutputStream` stream as encapsulated by an `ObjectOutputStream` stream to write that object out to a file—`serialized.dat`. Next, we use a `FileInputStream` stream encapsulated in an `ObjectInputStream` stream to read that object back from the file and display its text. The code is as follows:

```
import java.io.*;
public class SerializationDemo {
    public static void main(String args[]) {
        NewString inobject, outobject;
        inobject = new NewString("");
        outobject = new NewString("Dreamtech Press!");
        try {
            FileOutputStream fileoutputstring = new
                FileOutputStream("serialized.dat");
            ObjectOutputStream objectoutputstream = new
                ObjectOutputStream(fileoutputstring);
            objectoutputstream.writeObject(outobject);
            objectoutputstream.flush();
            objectoutputstream.close();
            FileInputStream fileinputstring = new
                FileInputStream("serialized.dat");
            ObjectInputStream objectinputstream = new
                ObjectInputStream(fileinputstring);
            inobject = (NewString)objectinputstream.readObject();
            objectinputstream.close(); }
        catch(Exception e){ }
        System.out.println(inobject); }
}
```

Compile `NewString.java` and `SerializationDemo.java`. The output of this code is shown in Figure 6.13:

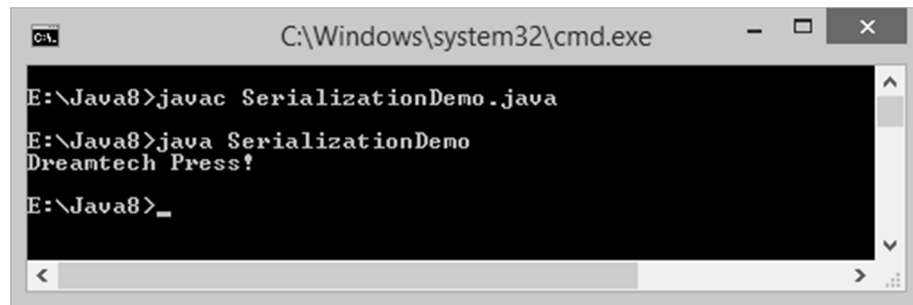


Figure 6.13: Output of the `SerializationDemo` class

Working with the **Console** Class

The `java.io.Console` class has different methods for accessing character-based console device related to the existing Java Virtual Machine (JVM). The existence of associated console depends upon the manner in which the virtual machine is invoked. If it is started by using the command line, then its console exists, and if it is started by some background job scheduler, then it does not have any console. A connected console of a JVM is

represented by the `java.io.Console` class, which can be obtained by using the `System.console()` method. All read and write operations are synchronized. This guarantees the atomic completion of critical operations.

The inheritance diagram of the `Console` class is as follows:

```

java.lang.Object
|
+--- java.io.Console

```

You'll find the methods of the `Console` class in Table 6.73:

Table 6.73: Methods of the Console class	
Method	Does this
<code>void flush()</code>	It flushes console and all buffered output is written immediately.
<code>Console format(String fmt, Object... args)</code>	It writes a formatted string to this console's output stream according to the given format string and arguments.
<code>Console printf(String fmt, Object... args)</code>	It writes a formatted string to this console's output stream using the specified format string and arguments.
<code>Reader reader()</code>	It returns unique <code>Reader</code> object associated with this console.
<code>String readLine()</code>	It reads a single line of text from the console.
<code>String readLine(String fmt, Object... args)</code>	It provides a formatted prompt and reads a single line of text from the console.
<code>char[] readPassword()</code>	It reads password from the console with echoing disabled.
<code>char[] readPassword(String fmt, Object... args)</code>	It provides formatted prompt and reads password from the console with echoing disabled.
<code>PrintWriter writer()</code>	It returns unique <code>PrintWriter</code> object associated with this console.

With the end of console, the input stream read methods of this class return null. All methods throw `NullPointerException` on passing a null argument to them.

Using this new console API, the formatted output can be written and read easily as it is integrated with Formatter API. The `readLine()` methods are key methods for user input here. `readLine(String fmt, Object... args)` is a combination of formatted output with user input. A first time support to read password from console has been introduced through the `readPassword()` methods.

Here is an example using the `java.io.Console` object:

```

import java.io.*;

public class ConsoleDemo{
    public static void main(String args[]) {
        int MIN_PASSWORD_LENGTH = 8;
        char[] password;
        Console console=System.console();
        do {
            password = console.readPassword( "Enter password (minimum
                                           of %d characters): ", MIN_PASSWORD_LENGTH);
            console.printf("Password Length is %d\n", password.length);
        } while (password.length < MIN_PASSWORD_LENGTH);
    }
}

```


You can see the result in Figure 6.14, given how the `Console` object is obtained and how its methods are being used:

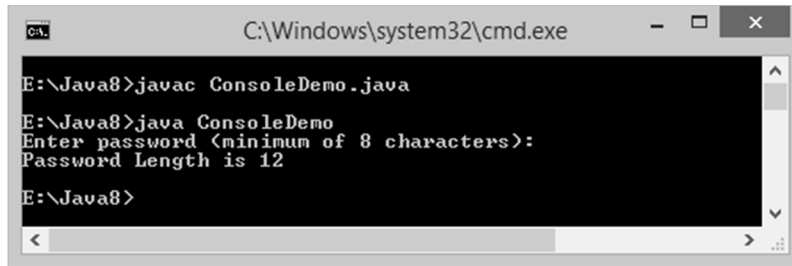


Figure 6.14: Output for the `ConsoleDemo` class

Execute this code and enter a string of length 8 or more to exit.

Working with the Clipboard

You can use the Java toolkit's `getSystemClipboard()` method and `ClipboardOwner` interface to place data into the Clipboard and retrieve it from the Clipboard.

Here's an example. This code places the text "Dreamtech" into the Clipboard (note that we have to create an object that has a toolkit—a `Frame` object here—to use `getSystemClipboard()`):

```
import java.awt.*;
import java.awt.datatransfer.*;

public class ClipBoardDemo
{
    public static void main(String[] argv)
    {
        useClipboard useClipboard = new useClipboard();
    }
}
class useClipboard extends Frame implements ClipboardOwner
{
    useClipboard()
    {
        Clipboard clipboard = getToolkit().getSystemClipboard();
        StringSelection contents = new StringSelection("Dreamtech");
        clipboard.setContents(contents, this);
        System.exit(0);
    }
    public void lostOwnership(Clipboard clipboard,
        Transferable contents)
    {
        System.out.println("Clipboard contents replaced");
    }
}
```

Running of this code will copy string into the Clipboard. You can paste the copied string "Dreamtech" from the Clipboard to any other editor.

Working with the Printer

You can use the Java toolkit's `getPrintJob()` method to create a new print job for the printer, and you can use the print job's `getGraphics()` method to get a `Graphics` object that you can use to draw on the printed page.

Here's an example that draws a small graphics figure on a page and then prints it (note that we have to create an object that has a toolkit—a `Frame` object here—to use `getPrintJob()`):

```

import java.awt.*;
import java.awt.event.*;

public class PrinterDemo
{
    public static void main(String[] argv)
    {
        usePrinter useprinter = new usePrinter();
    }
}
class usePrinter extends Frame {
    usePrinter() {
        PrintJob printjob = getToolkit().getPrintJob(this,
            "Print graphics", null);
        Graphics g = printjob.getGraphics();
        g.drawRect(2, 2, 100, 100);
        g.drawLine(2, 2, 100, 100);
        g.dispose();
        printjob.end();
        System.exit(0); }
}

```

Being able to use the printer this way is a very powerful aspect of Java I/O. The result is shown in Figure 6.15:

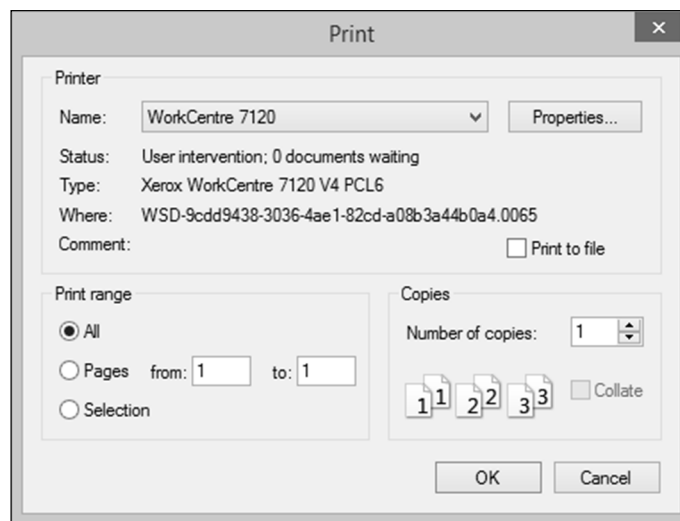


Figure 6.15: Working with the printer

Printing with the **Formatter** Class

Similar to the `printf()` function in C, the class `java.util.Formatter` allows you to perform string formatting. This class was introduced in J2SE 5 and it depends heavily on the `varargs` feature.

Using the ***System.out.printf()*** Method

The most important thing about `Formatter` is that you should avoid direct interaction with it. Instead of this, you can use the new `PrintStream` method `printf()`. `PrintStream` is not new for you because you are already familiar with `System.out.println()` calls. The following example illustrates the `printf()` method:

```

public class PrinterDemo1
{

```

```

public static void main(String[] argv)
{
    String inls = "LKJ";
    String cnt = "just because";
    System.out.printf("reason: %s (noted by %s)", cnt, inls);
}

```

The output of the preceding code is as follows:

```
reason: just because (noted by LKJ)
```

In the previous `printf()` example, a format string with embedded format specifiers is passed to `printf()`. After this, pass a list of arguments corresponding to each format specifier. The previous example has two format specifiers: `%s` and `%s`, and these correspond to the `cnt` argument and `inls` argument respectively.

The format specifier `%s` has some significance. The letter `'s'` signifies a `String` conversion and `%` signifies the start of a format specifier. Some characters that trigger the end of format are called *conversions*. A string conversion signifies that Java calls `toString()` on the corresponding argument and substitutes the result for the format specifier.

Using the ***String.format()*** Method

Sometimes you need only formatted string and print is not required. In such a case, you can employ the `static()` method `format` on the `String` class. The following example shows some numeric conversions:

```

public class PrinterDemo2
{
    public static void main(String[] argv)
    {
        int i = 91;
        String str = String.format("char: %c integral: %d octal: %o hex: %x %n", i, i, i, i);
        System.out.println(str);
    }
}

```

In the preceding example, platform-specific line separator is denoted by `%n`. After print, the output of `String` would be:

```
char: [ integral: 91 octal: 133 hex: 5b
```

Flagging is supported by numeric conversions that support flags for padding, grouping, justification, and sign.

NOTE

There are four arguments, each using same reference, i.e., you can supply a single argument if all are same and that will be used more than once.

Following example shows how one can use the same argument more than once against a format string:

```
String str = String.format("char: %c integral: %<d octal: %<o hex: %<x %n", i);
```

In the preceding example, reference `<` can be used only once. The less-than sign (`<`) is used very frequently and bears some special significance. It signifies that the same argument must be used as the last format specifier.

Formatting Dates Using the ***String.format()*** Method

Many date-related conversions are provided by `Formatter`. For example, consider the following code:

```

import java.util.*;

public class PrinterDemo3
{
    public static void main(String[] argv)
    {
        int i = 65;
        String str =String.format("%1$td %1$tb %1$ty", new Date());
    }
}

```

```

        System.out.println(str);
    }
}

```

The preceding code produces a string with the value, as shown in Figure 6.16:

```

C:\Windows\system32\cmd.exe
E:\Java8>javac PrinterDemo3.java
E:\Java8>java PrinterDemo3
30 Jun 16
E:\Java8>

```

Figure 6.16: Displaying the current date

In the preceding example, all three format specifiers target the first argument. To designate which argument to use, 1\$ is used in the format specifier.

Date conversion is done by using a two-character sequence starting with a t. The date part to converted, is represented by the second character of the sequence. Consider a sequence td. This sequence in the first specifier converts the day of the month.

Using the *java.util.Formatter* Class

Nowadays, the `Formatter` class is equipped with many new formatting capabilities. This class supports internationalization by allowing you to pass a `Locale` to the constructor.

The `Appendable` object is the latest interface that defines the `append()` methods. `Formatter` allows to pass an `Appendable` object. Since the `Appendable` objects encapsulate the `append()` methods, `formatter` can store its results in a text collector, such as a `stream` object. To implement this interface, all relevant Java classes (e.g., `StringBuffer` and `PrintStream`) are modified by Oracle. The following example shows how to implement it:

```

import java.util.*;
public class PrinterDemo4
{
    public static void main(String[] argv)
    {
        double avg      = 6.0e23;
        StringBuffer buf = new StringBuffer();
        Formatter frmter = new Formatter(buf, Locale.US);
        frmter.format("avg's number: %e %n", avg);
        frmter.format("base of the natural log: %e %n", Math.E);
        System.out.println(buf.toString());
    }
}

```

The corresponding output is shown in Figure 6.17 doing the calculations:

```

C:\Windows\system32\cmd.exe
E:\Java8>javac PrinterDemo4.java
E:\Java8>java PrinterDemo4
avg's number: 6.000000e+23
base of the natural log: 2.718282e+00
E:\Java8>_

```

Figure 6.17: Performing calculation

The `Formatter` class encapsulates many conversions and some additional features. See Java API documentation pages for `java.util.Formatter` for more details.

Scanning Input with the **Scanner** class

You can read input from either the keyboard or a text file by using the `Scanner` class. This class provides simple and Sun-standard mechanism for reading input. When you are using `tjXXXXx.jar` file, you can access the `Scanner` class by importing `edu.fcps.Scanner`, and after J2SE 5, you can access the complete `Scanner` class by importing `java.util.Scanner`. A simple text `Scanner` is used for parsing primitive types and strings through regular expressions.

Whenever a `Scanner` receives an input, it breaks it into tokens. For breaking, `Scanner` uses delimiter pattern, which by default matches whitespace. These tokens are then converted into values of different types using the methods discussed here. The following code allows a user to read a number from `System.in`:

```
Scanner scan = new Scanner(System.in);
int i = scan.nextInt();
```

Consider another example in which code allows long types to be assigned from entries in a file `myNumbers`:

```
Scanner scan = new Scanner(new File("myNumbers"));
while (scan.hasNextLong()) { long aLong = scan.nextLong(); }
```

We know that the default value of delimiters is whitespace, but `Scanner` also allows the use of delimiter other than whitespace. The following example reads several items from a string:

```
import java.util.*;
public class ScannerDemo{
    public static void main(String[] argv) {
        String input = "10 fish 20 fish green fish white fish";
        Scanner s = new Scanner(input).useDelimiter("\\s*f\\s*");
        System.out.println(s.nextInt());
        System.out.println(s.nextInt());
        System.out.println(s.next());
        System.out.println(s.next());
        s.close(); }
}
```

The output of the code is shown in Figure 6.18, showing the text in different lines:

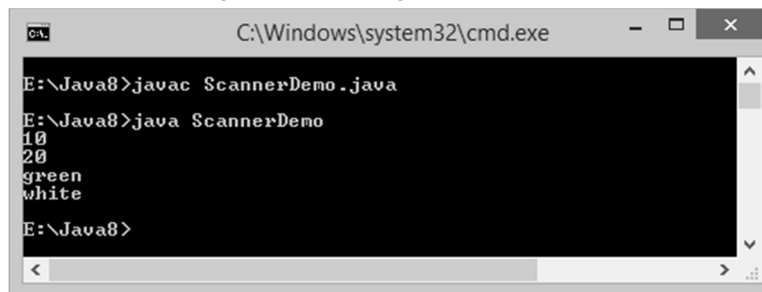


Figure 6.18: Scanning input with the `ScannerDemo` class

The output of the preceding code is shown in Figure 6.18, taking input from the `Scanner` class.

Summary

In this chapter, you have learned the use of file class and to work with streams. Then, we have explored the `RandomAccessFile`, `Reader` and `Writer` classes. Following that, we have discussed to input the value from the keyboard with the `InputStreamReader` class and work with the `OutputStreamWriter` class. You have next learned to work with files, character arrays, and buffers. Moving further, we have discussed how to work with the `StreamTokenizer` class, `Serializable` interface, and `Console` class. You have also learned to work with printer and clipboard. Lastly, you have learned to print with the `Formatter` class and scan input with the `Scanner` class.



7

Implementing Exception Handling

<i>If you need information on:</i>	<i>See page:</i>
Overview of Exceptions	192
Exception Handling Techniques	193
Rethrowing Caught Exception with Improved Type Checking	201
Built-in Exceptions	203
User-Defined Exceptions	204
Summary	205

Exceptions are abnormal conditions that occur at runtime and can cause abrupt termination of a program. They can also be considered as errors that occur at runtime. These kinds of errors can be categorized into memory errors (memory allocation is not correct), file system errors (disk is either full or removed), network errors (network is either down or cannot search for the specified URL), and calculation errors, such as division of a number by 0. A program should be written in such way that it is able to handle exceptions efficiently and can therefore overcome the problem of sudden program termination resulting from the exception. Let's suppose you have created a Java program to perform division operations. However, you have used 0 by mistake to divide a number, as a result of which arithmetic exception (a built-in Java exception) is generated. Now, your program should include code to handle the arithmetic exception in order to prevent abrupt termination of program.

To handle any exception in a program, you need to follow an exception handling mechanism. Exception handling is a mechanism that is used to handle exceptions or errors in a program that occurs at the time of program execution. However, Java provides various techniques to handle exceptions.

The chapter provides an overview of exceptions. Then it explores various exception handling techniques, wherein you learn about try and catch blocks, multiple catch blocks, try-with-resources statement, nested try blocks, throw clause, and throws clause. Next, you learn about rethrowing caught exception with improved type checking. Toward the end, the chapter explains about built-in exceptions and user-defined exceptions.

Overview of Exceptions

Exceptions are events or errors that occur during program execution and interrupt the flow of the program. On occurrence of an exception, an exception object is created, which consists of the information regarding the exception, such as the type of exception and the state of the program when the exception has occurred. After the object is created, it is thrown in the method that caused the exception. When the exception is thrown in the method, the exception handling techniques in the method are searched by the Java runtime system. If no exception handling technique is found, the Java runtime system searches the class for exception handling techniques in which the method is written. In case no technique that handles exceptions is found, the Java runtime system terminates and the program is interrupted. In Java, exceptions are handled by using the try, catch, throw, and throws clauses, which are discussed in detail in the section Exception Handling Techniques.

The exceptions are broadly classified into two categories, namely, checked exceptions and unchecked exceptions. Exceptions that are checked at compilation time are called checked exceptions. They are the exceptions that a well-written program can catch and notify the user by providing the reason why the exception has occurred. For example, the `SQLException` exception will notify the user that the exception has been generated due to an error while establishing connection to the database or inability to find database.

All the exceptions in the `Throwable` class, except for those that come under the `Error` and `RuntimeException` classes and their subclasses, are considered as checked exceptions. The `RuntimeException` class is the subclass of the `Exception` class.

NOTE

Throwable is a built-in class in which all exception classes reside. It has two subclasses: Exception and Error.

Unchecked exceptions are the exceptions that come under the `Error` and `RuntimeException` classes and their subclasses. They can occur due to a logical mistake in a Java program or lack of memory in the computer. For example, you have written a code to read a file in your Java program, but you are unable to read the file due to some problem, such as a hard disk crash or the specified file is not present in your system. In such a case, the exception that will be generated is known as an unchecked exception. Let's create a class, named `except`, to explain how to create an exception. Listing 7.1 shows the code of the `except` class:

Listing 7.1: Generating an Exception

```
class except
{
    public static void main (String args[])
    {
        int x=5;
        int y=0;
```

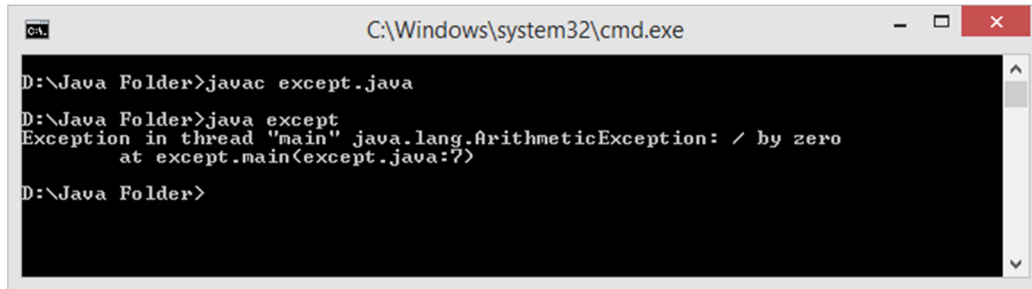
```

        int z=x/y;

        System.out.println("Result of the program is " + z);
    }
}

```

In Listing 7.1, you have created a class named `except` in which a number is divided by zero. Figure 7.1 displays the output of the `except` class:



```

C:\Windows\system32\cmd.exe

D:\Java Folder>javac except.java
D:\Java Folder>java except
Exception in thread "main" java.lang.ArithmeticException: / by zero
    at except.main(except.java:?)
D:\Java Folder>

```

Figure 7.1: Output of Listing 7.1

It can be seen in Figure 7.1 that the program is compiled successfully. However, the program terminates abruptly and displays an exception at runtime. The solution for the exception is not given because you have not used any exception handling techniques. Had you used the exception handling technique, the program would have terminated along with a proper message to resolve the exception.

Let's study the various exception handling techniques and learn how they are implemented in Java programs.

Exception Handling Techniques

In Java, there are various techniques that are used to handle exceptions. These techniques use various clauses, such as `try`, `catch`, and `throw`. You can use these techniques to create well-designed programs in which the exceptions are properly handled. The following techniques are used to handle exceptions:

- ☐ The `try` and `catch` blocks
- ☐ Multiple `catch` blocks
- ☐ The `try-with-resources` statement
- ☐ Nested `try` blocks
- ☐ The `throw` clause
- ☐ The `throws` clause

Let's discuss these techniques in detail one by one.

Using the try and catch Blocks

When you write a program, you may observe certain statements in it, which may raise exceptions. You must enclose such statements inside a `try` block. All the statements that may cause exception are enclosed inside the `try` block. The `try` block specifies the scope of the exception handlers associated with the block. A `catch` block, on the other hand, is used as an exception handler. The statements that are to be monitored are placed inside a `try` block, which is then followed by one or more `catch` blocks to handle exceptions. Whenever an exception arises inside the `try` block, a Java Virtual Machine (JVM) stores the exception details in an exception stack and then executes the `catch` block.

The `catch` block has a parameter that catches the exceptions, which a user wants to handle. It also contains the messages showing the user the reason why the exceptions have occurred. The `catch` block makes the exception handling efficient as it is easy to resolve an exception after its cause is known. The following code snippet shows how to use the `try` and `catch` blocks:


```

try
{
    Program_code;
}
catch(<exception> <exp_var>)
{
    System.out.println(" Any message" + <exp_var>);
}

```

In the preceding code snippet, `Program_code` can be any logical group of statements used to perform any specific operation and `<exception>` in the catch block can be any exception class that you want to handle. To handle the exceptions, you can either use `Exception`, the superclass of all exceptions in Java, or specify a subclass, such as `ArithmeticException`, in the catch block. Moreover, `<exp_var>` can be any variable name that refers to the exception object where the exception details are stored. Let's create a class named `except_tc` that uses the try and catch blocks to handle exceptions. Listing 7.2 shows the code of the `except_tc` class:

Listing 7.2: Showing the Code of the `except_tc` Class

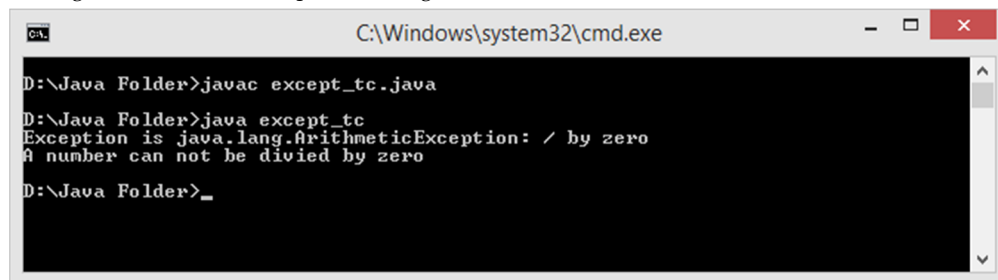
```

class except_tc
{
    public static void main (String args[])
    {
        try // try block
        {
            int x=5;
            int y=0;
            int z=x/y;
            System.out.println("Result of the program is " + z);
        }
        catch(Exception e) // catching exceptions
        {
            System.out.println("Exception is " + e);
            System.out.println("A number can not be divided by zero " );
        }
    }
}

```

In Listing 7.2, you have created the same program as in Listing 7.1; however, this time, you have modified the class name to `except_tc` and enclosed the program code in the try and catch block. Then the `Exception` class in the catch block is used to handle the exception. In Listing 7.2, you know that you can also use any exception subclass if you know the type of exception that may be raised in the program. For example, if the arithmetic exception is expected to generate in the try block, then you can use the `ArithmeticException` class in place of the `Exception` class in the catch block.

You have used the “`System.out.println`” statements inside the catch block in the `except_tc` class to display the message along with the exception so that a user knows the reason for the exception as well as its solution. Figure 7.2 shows the output of Listing 7.2:



```

C:\Windows\system32\cmd.exe

D:\Java Folder>javac except_tc.java

D:\Java Folder>java except_tc
Exception is java.lang.ArithmeticException: / by zero
A number can not be divided by zero

D:\Java Folder>_

```

Figure 7.2: Output of Listing 7.2

If you compare the output of Listing 7.1 with the output of Listing 7.2, you will notice that the exception without any message describing why the exception has occurred is displayed in Figure 7.1. However, in Figure 7.2, a

message showing the reason for the exception, 'A number cannot be divided by zero', is displayed along with the exception. This message will help the user to resolve the problem. Figure 7.2 also illustrates the utility of the try and catch blocks in exception handling.

Next, let's discuss about the try-with-resources statement.

Using the try-with-resources Statement

In Java SE, you are not required to manage the resources explicitly in the code during program execution. A resource is an object that implements the `java.lang.AutoCloseable` and `java.io.Closeable` interfaces in a Java program. A resource must be closed after it has been used in a Java program. Examples of resources are `BufferedReader` and `Connection` objects.

To implement the resource management, Java SE allows the declaration of the resource object in the try statement. The try-with-resources statement checks whether all the declared resource objects have been closed before the closing of the try statement. Let's create a class named `ResourcesDemo` to explain the use of the try-with-resources statement. Listing 7.3 shows the code of the `ResourcesDemo` class:

Listing 7.3: Showing the Code of the `ResourcesDemo` Class

```
import java.io.*;
public class ResourcesDemo
{
    public static void main(String as[])
    {
        ResourcesDemo rd=new ResourcesDemo();
        rd.demo();
    }

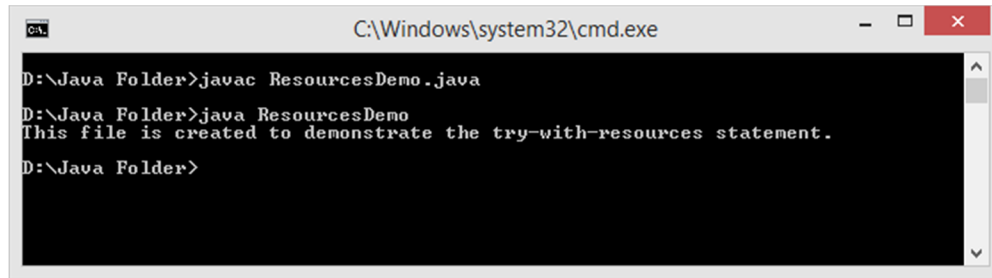
    public void demo()
    {
        try (BufferedReader brRef = new BufferedReader(new FileReader("D:/Java Folder/abc.txt")))
        {
            String str=brRef.readLine();
            System.out.println(str);
        }
        catch(Exception e)
        {
            e.printStackTrace();
        }
    }
}
```

In the `ResourcesDemo` class, two resources, `FileReader` and `BufferedReader` objects, are declared in the try statement. The content of the file `abc.txt` is read by using the `readLine()` method of the `BufferedReader` object. Note that the `close()` method of the `BufferedReader` object has not been called because the try-with-resources statement closes the `BufferedReader` object prior to the closing of the statement. The try-with-resources statement leads to the efficient utilization of the hardware used by the `BufferedReader` object and also simplifies the code.

NOTE

Before Java SE 7, the `BufferedReader` object is closed using its `close()` method.

On execution of the `ResourcesDemo` class, contents of the file `abc.txt` are displayed on the Command Prompt, as shown in Figure 7.3:



```

C:\Windows\system32\cmd.exe

D:\Java Folder>javac ResourcesDemo.java
D:\Java Folder>java ResourcesDemo
This file is created to demonstrate the try-with-resources statement.
D:\Java Folder>

```

Figure 7.3: Output of Listing 7.3

Let's learn about the multiple catch blocks in the next subsection.

Using Multiple catch Blocks

In Listing 7.2, you have used only one catch block to handle exceptions. However, if you have some idea of the kind of exceptions that may cause the program to be interrupted, you can provide specific exceptions in multiple catch blocks to handle these exceptions separately.

It is likely that you may forget to include some exceptions in the catch clauses that may lead to program interruption or termination. To prevent this from happening, use the Exception superclass in the last catch block so that all the other remaining exceptions can be handled. The Exception class must be used in the last catch block; otherwise, an error will occur during compilation because a superclass cannot be used before a subclass. Let's create a class, named `except_mc`, which uses multiple catch blocks to handle exceptions. Listing 7.4 shows the code of the `except_mc` class:

Listing 7.4: Showing the Code of the `except_mc` Class

```

class except_mc
{
    public static void main (String args[])
    {
        try
        {
            int a[] = { };
            System.out.println(a.length);
            for (int i=0; i<=a.length; i++)
            {
                System.out.println("value at position " + i + " is "
                                   + a[i]);
            }
        }
        catch (ArrayIndexOutOfBoundsException ao)
        {
            System.out.println("Array is out of bound; please check the
                                upper bound of array");
        }
        catch (Exception e)
        {
            System.out.println("Exception is" + e);
        }
    }
}

```

In Listing 7.4, the `except_mc` class has used multiple catch blocks to handle exceptions. In the `except_mc` class, you have declared an array '`a[]`', but have not initialized it. Then the `for` loop is used to traverse the array up to its length. Since you have not yet initialized the array, the `ArrayIndexOutOfBoundsException` exception can be generated and therefore a catch block is used to handle the exception. You can also use the `Exception`

class in another catch block for other exceptions that may occur. Figure 7.4 shows the output of the `except_mc` class:

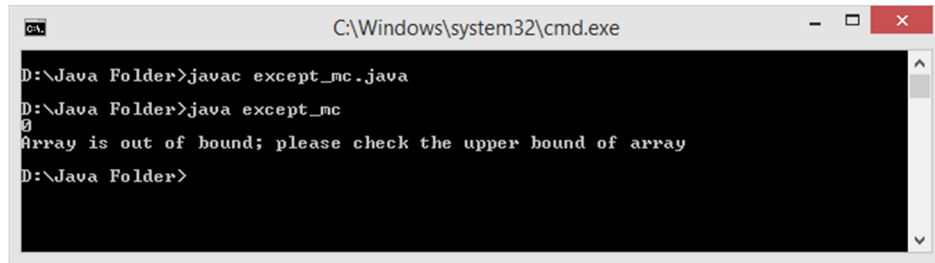


Figure 7.4: Output of Listing 7.4

Since you have not initialized the array but traversed its length, which is 0, the statement in the catch block tells the user that the array is out of bound.

Let's learn about the nested try blocks in the next subsection.

Using Nested try Blocks

In Listing 7.4, you have used only one try block and multiple catch blocks to handle the exceptions. However, you can also use multiple try blocks by nesting one try block inside the other in a program. Every try block should follow a catch block. This method offers better exception handling since it encapsulates the statements that might generate an exception in a try block. Listing 7.5 shows how multiple try blocks are used in a Java program:

Listing 7.5: Showing the Nested try Blocks

```
class except_mt
{
    public static void main (String args[])
    {
        try // first try block
        {
            int j, x=10, y;
            int k[]= new int[10];

            for (int i=0; i<args.length; i++)
            {
                System.out.println("values retrieved through
                command line are " + args[i]);
            }
            try // second try block inside first try block
            {
                for (j=0; j<args.length; j++)
                {
                    k[j]= Integer.parseInt(args[j]);
                    x=x/k[j];
                }
                System.out.println( x );
            }
            catch (ArithmeticException ae) // catch block of second try
            {
                System.out.println("Exception is" + ae);
                System.out.println("value is divided by zero, which
                is not allowed");
            }
        }
        catch (Exception e) // catch block of first try
        {
            System.out.println("Exception is" + e);
        }
    }
}
```

```

    }
}
}

```

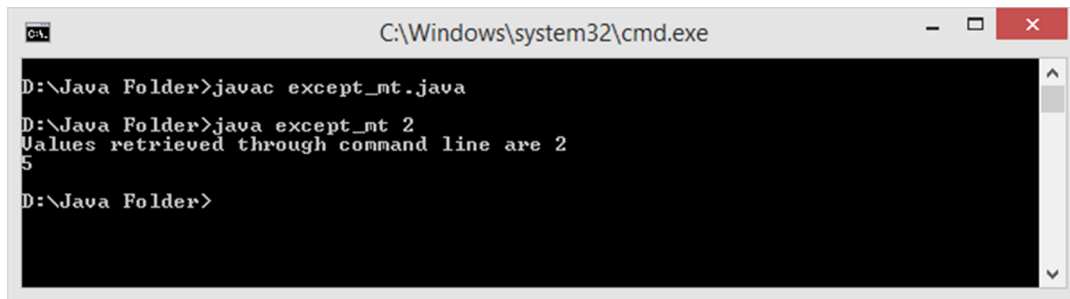
In Listing 7.5, you have created a class named `except_mt`, which receives some values through command line arguments. Then these values are displayed. The code to receive and display the values using the command line arguments has been enclosed in a try block in the `except_mt` class.

Next, you have written the code for dividing a variable `x`, initialized with a value 10, by the value retrieved through the command line. This code is enclosed in another try block, which is nested inside the first try block. Then, in the catch block of the second try block, the `ArithmeticException` class is used to catch the exception that may be raised if a user enters '0' as a command line argument. Further, you have written the catch block of the first try block in which the `Exception` superclass is used to handle all types of exceptions.

NOTE

The command line arguments are used to pass values to the main method. The main method has `args[]` string parameter to store the values. Command line arguments are passed at runtime.

Figure 7.5 displays the output of the `except_mt` class:



```

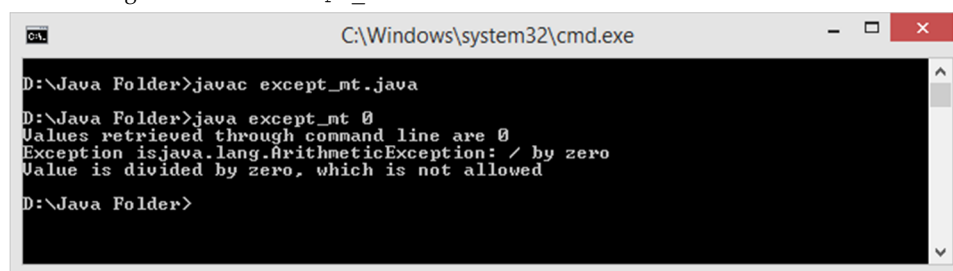
C:\Windows\system32\cmd.exe

D:\Java Folder>javac except_mt.java
D:\Java Folder>java except_mt 2
Values retrieved through command line are 2
5
D:\Java Folder>

```

Figure 7.5: Output of Listing 7.5

In Figure 7.5, the value '2' beside `D:\Java Folder> java except_mt` is a command line argument. You can pass multiple arguments by separating them with a space. In addition, note that the program is compiled and executed successfully because the value passed as a command line argument is not 0. However, if you pass 0 as a command line argument, an exception will be raised. Figure 7.6 shows the exceptions generated if 0 is passed as a command line argument to the `except_mt` class:



```

C:\Windows\system32\cmd.exe

D:\Java Folder>javac except_mt.java
D:\Java Folder>java except_mt 0
Values retrieved through command line are 0
Exception is java.lang.ArithmeticException: / by zero
Value is divided by zero, which is not allowed
D:\Java Folder>

```

Figure 7.6: Exceptions generated

It can be seen in Figure 7.6 that the `ArithmeticException` exception is raised because you cannot divide a number by 0. This exception is caught by the catch block of the second try block because the statements enclosed in the second try block are responsible for this exception. However, in case any other type of exception other than the `ArithmeticException` exception is generated, the catch block of the first try block will be responsible for handling those exceptions.

Let's learn about the `throw` clause.

Using the throw Clause

Till now, you have learned about catching exceptions thrown by the Java runtime system. In this section, you learn how to throw exceptions explicitly. To throw an exception explicitly, you have to create an object of that exception type. You can throw an object of any exception type in Java by using the new operator with throw clause. Listing 7.6 shows how to create the object of an exception and throw the exception:

Listing 7.6: Throwing an Exception

```
class except_throw
{
    public static void chk_throw()
    {
        try
        {
            // Creatig object of NullPointerException
            throw new NullPointerException("Explicit
            NullPointerException");
        }
        catch ( NullPointerException ne)
        {
            System.out.println("Exception is " + ne);
        }
    }

    public static void main (String args[])
    {
        except_throw.chk_throw();
    }
}
```

In Listing 7.6, you have created a class named `except_throw` and a method named `chk_throw` in this class. In the `chk_throw` method, you have created and thrown the object of the `NullPointerException` class and enclosed the code in a try block. In the catch block, you have used the `NullPointerException` class to handle the `NullPointerException` exception that may be raised.

In the `main()` method of the `except_throw` class, the static `chk_throw()` method is called without creating the object of the `except_throw` class. This is done to generate the `NullPointerException` exception so that you can see whether the exception is generated explicitly or not. Figure 7.7 shows the output of the `except_throw` class:

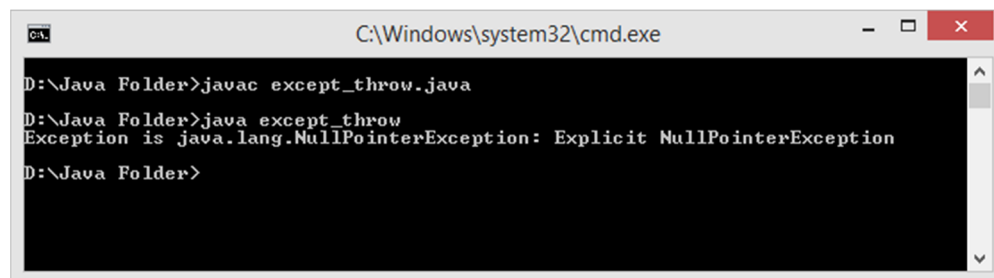


Figure 7.7: Output of Listing 7.6

It can be seen in Figure 7.7 that the statement 'Exception is java.lang.NullPointerException: Explicit NullPointerException' is displayed. The statement is shown because the `NullPointerException` object is created and thrown from the try block, which is displayed in the catch block.

Let's learn about the throws clause next.

Using the throws Clause

The throws clause is used when you know that a method may cause exceptions, but the method does not handle those exceptions. In such a case, a user has to throw those exceptions to the caller of the method by using the

throws clause. A throws clause is used when a method is declared. The following code snippet shows the use of the throws clause:

```
return_type method_name (para) throws excep1, excep2, excep_n
{
    // method body
}
```

The terms used in the preceding code snippet are explained as follows:

- ❑ `return_type` specifies any logical return type according to the values that a method returns
- ❑ `method_name` is any logical method name
- ❑ `para` denotes a list of parameters that needs to be passed in the method
- ❑ `throws` represent the clause used to throw exceptions
- ❑ `excep1, excep2, and excep_n` are the exceptions that you want to throw

Multiple exceptions can be thrown from a method by separating the exceptions with commas. Let's create a class named `except_throws`, which makes use of the throws clause. Listing 7.7 shows the use of the throws clause:

Listing 7.7: Using the throws Clause

```
class except_throws
{
    static void chk_throw()                //throws IllegalAccessException
    {
        throw new IllegalAccessException("Exception");
    }
    public static void main (String args[])
    {
        try
        {
            chk_throw();
        }
        catch (IllegalAccessException ie)
        {
            System.out.println("Testing" + ie);
        }
    }
}
```

In Listing 7.7, you have created a class name `except_throws` and a method named `chk_throw` in this class. You have also created the object of the `IllegalAccessException` exception in the `chk_throw()` method. The static `chk_throw()` method is called in the `main()` method of the `except_throws` class without creating the object of the `except_throw` class. The `chk_throw()` method is called to generate the `IllegalAccessException` exception and show the use of the throws clause. The statement to call the `chk_throw()` method is enclosed in the try block. In the catch block, you handle the `IllegalAccessException` exception. Note that the code to throw the `IllegalAccessException` exception using the throws clause in the `chk_throw()` method declaration has been commented out. Figure 7.8 shows the output without the use of the throws clause:

```

C:\Windows\system32\cmd.exe

D:\Java Folder>javac except_throws.java
except_throws.java:5: error: unreported exception IllegalAccessException; must be
caught or declared to be thrown
    throw new IllegalAccessException("Exception");
           ^
except_throws.java:13: error: exception IllegalAccessException is never thrown i
n body of corresponding try statement
    catch (IllegalAccessException ie)
           ^
2 errors
D:\Java Folder>_

```

Figure 7.8: Output of Listing 7.7

It can be seen in Figure 7.8 that the program is not compiled because you have not thrown the `IllegalAccessException` exception that occurred in the `chk_throw()` method, which is a checked exception. Figure 7.9 shows the output of the `except_throws` class after the comment “throws `IllegalAccessException`” beside the `chk_throw()` method is removed:

```

C:\Windows\system32\cmd.exe

D:\Java Folder>javac except_throws.java
D:\Java Folder>java except_throws
Testing java.lang.IllegalAccessException: Exception
D:\Java Folder>_

```

Figure 7.9: Output of the `except_throws` class

Now, note that the program is compiled successfully and the exceptions are caught.

After learning about the techniques used for exception handling, let’s learn about the improved type checking for the exceptions rethrown from the catch block.

Rethrowing Caught Exception with Improved Type Checking

Java SE 7 has introduced enhanced feature of improved type checking for the exceptions that are rethrown from the catch clause in a Java program. Using Java SE 7, you can throw an exception in a catch block, which is a supertype of the exception declared in the throws clause of the method throwing the exception. Let’s create a Java program to enhance your concept of improved type checking for the exceptions rethrown from the catch clause, as shown in Listing 7.8:

Listing 7.8: Showing the Improved Type Checking for the Exceptions Rethrown from the Catch Clause

```

import java.io.*;
class aException extends Exception
{
}

```



```
.class bException extends Exception
{
}

public class RethrowingDemo
{
    public void rethrowExceptionMethod(String eName)throws aException, bException
    {
        try
        {
            if (eName.equals("aException"))
            {
                System.out.println("a Exception is thrown");
                throw new aException();
            }
            else
            {
                System.out.println("b Exception is thrown");
                throw new bException();
            }
        }
        catch (Exception e)
        {
            throw e;
        }
    }

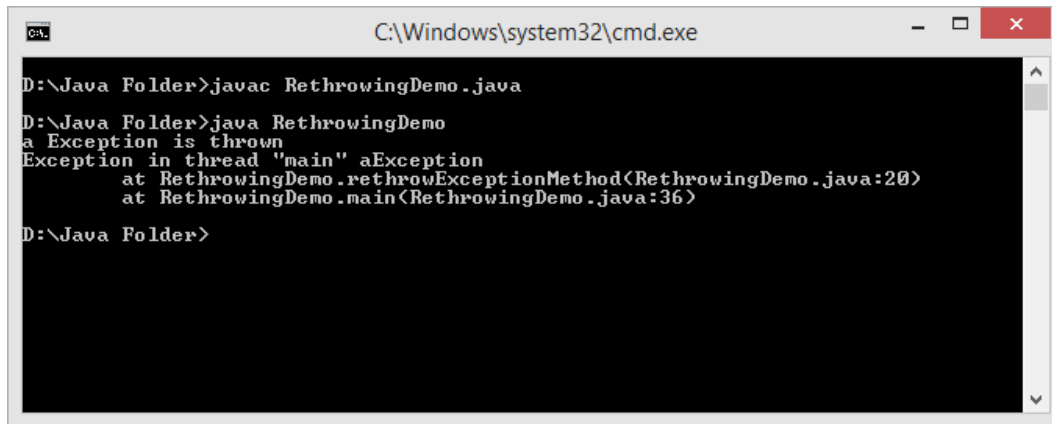
    public static void main(String args[])throws aException, bException
    {
        new RethrowingDemo().rethrowExceptionMethod("aException");
    }
}
```

In Listing 7.8, you have created two user-defined exception classes, namely, `aException` and `bException`. You have also created `RethrowingDemo` class in which you have developed the method `rethrowExceptionMethod()` that accepts a string `eName`. In the try block, you have verified the `eName` string value and accordingly thrown the `aException` or `bException` exception. The thrown exception is handled in the catch block that rethrows the exception. In the `main()` method of the `RethrowingDemo` class, the `rethrowExceptionMethod()` method is called and the exception thrown by the `rethrowExceptionMethod()` method is not handled in the `main()` method.

If the code provided in Listing 7.8 is executed through the versions prior to Java SE 7, the following error is generated:

```
unreported exception Exception; must be caught or declared to be thrown
```

The error is generated because the type of the catch parameter `e` in the `rethrowExceptionMethod()` method is `Exception`. `Exception` is a supertype and not a subtype of the `aException` and `bException` classes that are declared in the throws clause of the `rethrowExceptionMethod()` method. However, the versions prior to Java SE 7 do not produce error in Listing 7.8 if the throws clause of the `rethrowExceptionMethod()` method declares `Exception`, which is the type of catch parameter `e`. Figure 7.10 shows the output of Listing 7.8:



```

C:\Windows\system32\cmd.exe

D:\Java Folder>javac RethrowingDemo.java
D:\Java Folder>java RethrowingDemo
a Exception is thrown
Exception in thread "main" aException
    at RethrowingDemo.rethrowExceptionMethod(RethrowingDemo.java:20)
    at RethrowingDemo.main(RethrowingDemo.java:36)
D:\Java Folder>

```

Figure 7.10: Output of Listing 7.8

It can be seen in Figure 7.10 that the `aException` exception has been thrown and the message ‘a Exception is thrown’ is displayed.

In this chapter, many built-in exceptions such as “`ArithmeticException`” and “`NullPointerException`” have been used. These along with many other built-in exceptions are used in Java during exception handling.

Let’s learn about built-in exceptions in the next section.

Built-in Exceptions

Built-in exceptions are exceptions that are already available in Java. These exceptions are thrown by the Java runtime system to explain the exception condition. Table 7.1 displays some most commonly used built-in exceptions:

Table 7.1: Commonly Used Built-in Exceptions in Java	
Exception	Description
<code>ArithmeticException</code>	Specifies an exception condition that is thrown when a problem in an arithmetic operation is noticed by the Java runtime system. For example, this exception is thrown when a number is divided by 0.
<code>ArrayIndexOutOfBoundsException</code>	Represents an exception condition that is thrown when you try to access an array with an illegal index. Let’s suppose you have declared an array to store a maximum of 5 elements, but while traversing the loop, you try to access the 6 th element. In such a situation, this exception will be thrown.
<code>ClassNotFoundException</code>	Signifies an exception condition that is thrown when you try to access a class, which is not defined in your Java program.
<code>FileNotFoundException</code>	Defines an exception condition that is thrown when you try to access a non-existing file or one that fails to open.
<code>IOException</code>	Represents an exception condition that occurs when the input-output operation has failed or interrupted.
<code>InterruptedException</code>	Specifies a runtime error that occurs when a thread is interrupted during processing, waiting, or sleeping.
<code>IllegalAccessException</code>	Signifies a runtime error that occurs when access to a class is denied.
<code>NoSuchFieldException</code>	Represents an exception condition that occurs when you try to use any field or variable in a class that does not exist.
<code>NoSuchMethodException</code>	Denotes an exception condition that occurs when you try to access a non-existing method.
<code>NullPointerException</code>	Specifies an exceptional condition that occurs when you refer to the members of a null object.

Table 7.1: Commonly Used Built-in Exceptions in Java	
Exception	Description
<code>NumberFormatException</code>	Denotes an exceptional condition when a method is unable to convert a string into a numeric format.
<code>StringIndexOutOfBoundsException</code>	Signifies an exceptional condition that occurs when you try to access a string array with an illegal index.

The exceptions discussed in Table 7.1 are some of the most commonly used built-in exceptions. You can use these exceptions during exception handling to make your program more robust.

Next, let's discuss about the user-defined exceptions.

User-Defined Exceptions

You know that Java provides various built-in exceptions. However, you can also create your own exception types to describe the exceptions related to your applications in your own way. To create your own exception types, you have to create a class that is a subclass of the `Exception` class. To create a subclass of the `Exception` class, you have to inherit the `Exception` class. Let's create the user-defined exceptions in a class named `except_user`. Listing 7.9 shows the code of the `except_user` class:

Listing 7.9: Showing the `except_user` Class

```
class except_user extends Exception
{
    except_user(String str) // constructor of the class
    {
        super(str);        // used super to refer the superclass constructor
    }

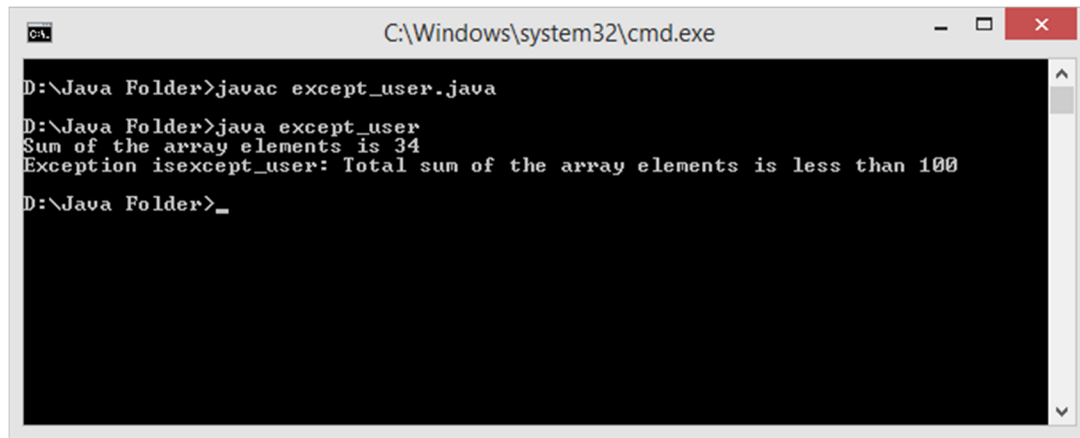
    public static void main(String args[])
    {
        try
        {
            int a[] = {4,6,8,7,9};
            int sum=0;
            for( int i=0; i<a.length; i++)
            {
                sum = sum + a[i];
            }
            System.out.println ("Sum of the array elements is " + sum);
            if (sum<100)
            {
                except_user own = new except_user("Total sum of the
                array elements is less than 100");
                throw own;
            }
        }
        catch(except_user own)
        {
            System.out.println("Exception is" + own);
        }
    }
}
```

In Listing 7.9, you have created the `except_user` class, which inherits the `Exception` superclass. Then you have also created a constructor of the `except_user` class that takes a string parameter, which is passed to the `Exception` superclass constructor using the `super` keyword.

NOTE

The `super` keyword is used to refer to superclass variables, superclass methods, and superclass constructors. These variables and methods are written in the form `super.variable_name` and `super.method_name`, respectively. A constructor can be written in the form `super (values)`.

In Listing 7.9, you have first declared and initialized an array in the `main()` method. Next, you have calculated the sum of the array elements. Then, using the `IF` statement, you have checked whether the sum of the array elements is less than 100 or not. If the sum is less than 100, then an object of the `except_user` class is created and thrown using the `throw` clause. In the `catch` block, the `except_user` exception is handled. Figure 7.11 shows the output of the `except_user` class:



```
C:\Windows\system32\cmd.exe

D:\Java Folder>javac except_user.java

D:\Java Folder>java except_user
Sum of the array elements is 34
Exception isexcept_user: Total sum of the array elements is less than 100

D:\Java Folder>_
```

Figure 7.11: Output of the `except_user` class

With this, you have come to the end of the chapter. By now, you should have some idea of exceptions and the techniques to handle exceptions in Java. You can use the knowledge you have gained in this chapter to implement exception handling in your Java programs. Before moving on to the next chapter, let's quickly recap the main topics you have learned in this chapter by going through a short summary.

Summary

In this chapter you have learnt about an overview of exceptions. You have also learnt about the various exception handling techniques, wherein you have learnt about `try` and `catch` blocks, multiple `catch` blocks, `try-with-resources` statement, nested `try` blocks, `throw` clause, and `throws` clause. Next, you have learnt about rethrowing caught exception with improved type checking. Finally, you have learnt about built-in exceptions and user-defined exceptions.



8

Working with Multiple Threads

<i>If you need information on:</i>	<i>See page:</i>
Using Threads in Java	209
Life Cycle of a Thread	210
Synchronization of Threads	211
Multithreaded Custom Class Loader	212
Getting the Main Thread	212
Naming a Thread	213
Pausing a Thread	213
Creating a Thread with the Runnable Interface	215
Creating a Thread with the Thread Class	217
Creating Multiple Threads	220
Joining Threads	222
Checking if a Thread Is Alive	223
Setting Thread Priority and Stopping Threads	224
Synchronizing	225
Communicating between Threads	236
Suspending and Resuming Threads	237
Creating Graphics Animation with Threads	240
Eliminating Flicker in Graphics Animation Created Using Threads	242
Suspending and Resuming Graphics Animation	243
Using Double Buffering	245
Simplifying Producer-Consumer with the Queue Interface	247
Implementing Concurrent Programming	249
Simplifying Servers Using the Concurrency Utilities	250
Knowing Various Concurrency Utilities	250

<i>If you need information on:</i>	<i>See page:</i>
Learning about the java.util.concurrent Package	251
Learning about the java.util.concurrent.locks Package	253
Learning about the java.util.concurrent.atomic Package	254
Summary	255

In this chapter, we are going to take a look at multithreaded programming in Java and how it lets you support graphics animation. A *thread* is a lightweight entity within a process. Multiple threads can occur within the same process and share the memory. In fact, Java is one of the few programming languages that supports multithreading. We can also say that multithreading is a specialized form of multitasking.

There are two types of multitasking—*Process-based* and *Thread-based*. In process-based multitasking, the computer is able to run two or more programs concurrently. For example, you can listen to music and use the notepad at the same time. On the other hand, a thread-based multitasking involves a single program to perform two or more tasks simultaneously. For example, a text editor can format text at the same time it is being printed.

With the use of multithreading, maximum usage of resources occurs in minimum time. By keeping the idle time to minimum, efficient programs can be created that makes maximum utilization of the CPU.

NOTE

Many components in Java, most Swing component methods notably, are not thread-safe.

Using Threads in Java

When a Java program starts, it has one thread—the main thread. You can interact with the main thread in various ways (as you'll see in this chapter) such as getting or setting its name, pausing it, and more. However, you can also start other threads. You call the `start()` method of the object to start the thread and place the code that you want to execute in the `run()` method. There are two ways to create new threads. One is to declare a class that is a subclass of the `Thread` class. This subclass should override the `run` method of the `Thread` class, and you can then allocate and start an instance of that class. For example, here's how you can define a new thread class that starts when you create an instance of it:

```
class CustomThread extends Thread {
    CustomThread() { start(); }
    public void run() { } //Do the work this thread was created for
}
```

You can then create and start a thread of this type, like this:

```
CustomThread thread1 = new CustomThread();
```

Another way to create a thread is to create a class that implements the `Runnable` interface.

Secondly, this class then implements the `run()` method. An instance of the class that is passed at the time of creating a thread object can then be allocated. Here's how this looks in code:

```
class CustomThread implements Runnable {
    Thread thread;
    CustomThread() {
        thread = new Thread(this, "second");
        thread.start();
    }
    public void run() { } //Do the work this thread was created for
}
```

This code would then create a thread of this type and start it running:

```
CustomThread thread1 = new CustomThread();
```

It's also worth noting that every thread has a name for identification purposes (in fact, more than one thread may have the same name). Every thread also has a priority, and threads with higher priorities are executed in preference to threads with lower priorities.

Let us take one more example to understand this better.

```
class CreateThreadDemo {
    public static void main(String args[]) {
        new NewThread(); // create a new thread
    }
}
```



```

        try {
            for(int k = 10; k > 6; k--) {
                System.out.println("Parent Thread: " + k);
                Thread.sleep(1000); }
        }
        catch (InterruptedException e) {
            System.out.println("Parent Interrupted."); }
        System.out.println("Exiting Parent."); }
    }
    class NewThread implements Runnable{
        Thread T;
        NewThread() { //a new, second thread
            T = new Thread(this, "Creating Thread");
            System.out.println("Child Thread : " + T);
            T.start(); } // Start the thread
        // Point of entry for the second thread.
        public void run() {
            try {
                for(int j = 10; j > 6; j--) {
                    System.out.println("Child Thread: " + j);
                    Thread.sleep(500); } }
                catch (InterruptedException e) {
                    System.out.println("Child interrupted."); }
                System.out.println("Exiting Child."); }
        }
    }

```

Here's the output of the code:

```

Child Thread: Thread[Creating Thread,5,main]
Parent Thread: 10
Child Thread:10
Child Thread: 9
Child Thread:8
Parent Thread: 9
Child Thread:7
Exiting Child.
Parent Thread: 8
Parent Thread: 7
Exiting Parent.

```

Life Cycle of a Thread

A thread is the smallest unit of processing. It is lightweight and therefore easy to access. From the creation of the thread to its termination, it follows a cycle known as the life cycle of a thread. According to Oracle, a thread has six states. They are as follows:

- ☐ New
- ☐ Runnable
- ☐ Blocked
- ☐ Waiting
- ☐ Timed_Waiting
- ☐ Terminated

The diagrammatical representation of the life cycle of a thread is shown in Figure 8.1:

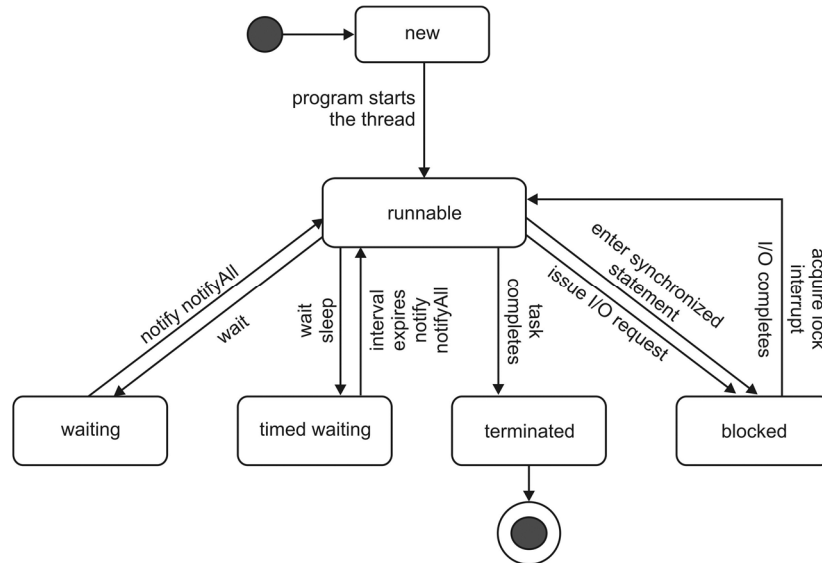


Figure 8.1: Thread Life Cycle UML State Diagram

The various states in the life cycle of a thread, as the figure shows, are as follows:

1. **New**—Before the `start()` method is invoked and an instance of a thread is created, the thread comes to new state.
2. **Runnable**—The life of a thread begins from this state. After the `start()` method is invoked, the thread first enters the runnable state. It can also enter this state after running, waiting, sleeping, or coming back from the blocked state. Here, the thread is waiting for its request to be addressed by the processor.
3. **Blocked**—When the thread is in the waiting or suspended or blocked state, it is not allowed to enter the runnable state or running state; then it appears to be in the blocked state.
4. **Waiting**—The thread is said to be in the waiting state when it waits indefinitely for another thread to complete an action.
5. **Timed_Waiting**—A thread that is waiting for another thread to perform an action for up to a specified waiting time is in this state. When the thread is waiting for a specified time interval and transitions back to the runnable state on expiration of the interval or occurrence of what it was waiting for, it is said to be in the `Timed_Waiting` state.
6. **Terminated**—When the thread has completed the execution of the `run()` method, its life cycle comes to an end. The thread is killed by invoking the `stop()` method, coming to the dead state.

Synchronization of Threads

Thread synchronization, also known as serialization, is a mechanism to ensure that no two concurrently executing threads possess the same parts of a program at the same time. If a thread holds some part of a process to access, the other thread must wait till the execution of the first thread finishes.

Basically, the object accessed by a thread is locked by it so that no other thread can access it. Synchronization is used to control access to the systems, such as multiprocessor systems, systems supporting multithreading, and distributed systems, consisting of bulky data, such as database for colleges or hospitals and many more.

There are two synchronization techniques:

- ❑ To control the order in which the code is executed to perform different tasks
- ❑ To prevent the obstacles that can arise when two threads share the same resource at the same time

Multithreaded Custom Class Loader

The function of the `java.lang.ClassLoader` class is to locate the bytecode for some specific class and then transform that bytecode into a usable class during runtime. The runtime system provides class loaders that can locate bootstrap classes, extension classes, and user classes.

If custom class loaders restrict to an acyclic class loader delegation model, they will never face the condition of deadlocks. The `ClassLoader` recognizes the acyclic delegation model. In this type of model, a class loader always consists of a parent (delegate). When a request for a class is made, the primary task of the class loader is to first check whether the class requested is already loaded. If the class is not already found, the class loader asks its parent to find the location of the class; and if the parent is not able to locate the class either, the class loader itself tries to find the location of the class.

In Java 8, the process of creation of new custom loaders is similar to those in the previous versions of Java, that is, create a subclass of `ClassLoader` and then override the `findClass()` and `loadClass()` methods.

With the release of Java SE, you can avoid deadlocks by following these rules:

- ❑ Ensure that the custom class loader for the concurrent class loading is safe.
 - a. Decide for the internal locking scheme. Example: `java.lang.ClassLoader` uses the locking scheme based on the name of class that is requested.
 - b. Remove all synchronization on the lock of the class loader object.
 - c. Ensure that critical sections are protected in case of multiple threads loading different classes.
- ❑ Invoke `java.lang.ClassLoader`'s static method `registerAsParallelCapable()`. This signifies that all the instances of the custom class loader are safe.
- ❑ Ensure that all the classes that the custom class loader extends also invoke the `registerAsParallelCapable()` method in their class initializers. Ensure that they are multithread safe for concurrent class loading.

Getting the Main Thread

The `currentThread()` method of the `Thread` class gets the current thread; if you are executing in the main thread, this method gets the main thread. Here's an example in which we are getting the main thread of an application and printing out its name by using the `getName()` method (note that you can set a thread's name with `setName()` —see the next solution):

```
class MainThread
{
    public static void main(String args[])
    {
        Thread thread = Thread.currentThread();
        System.out.println("The name of main thread is \"" + thread.getName()+"\"");
    }
}
```

The output of this code is shown in Figure 8.2, displaying the name of the main thread:

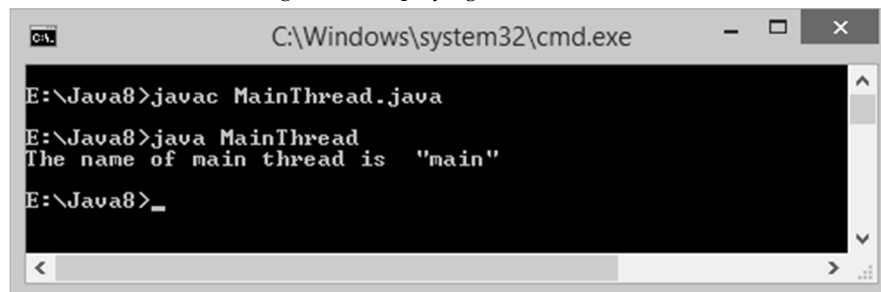


Figure 8.2: Displaying the Name of the Main Thread

Naming a Thread

One way to distinguish threads is by name, and you can set the name of a thread with the `setName()` method. Here's an example in which we rename the main thread in an application and display that name:

```
class SetThreadName
{
    public static void main(String args[])
    {
        Thread thread = Thread.currentThread();
        System.out.println("Main thread's original name is " +
            thread.getName());
        thread.setName("Dreamtech Press");
        System.out.println("Main thread's name is now " + thread.getName());
    }
}
```

The output of this code is shown in Figure 8.3, changing the name of the thread:

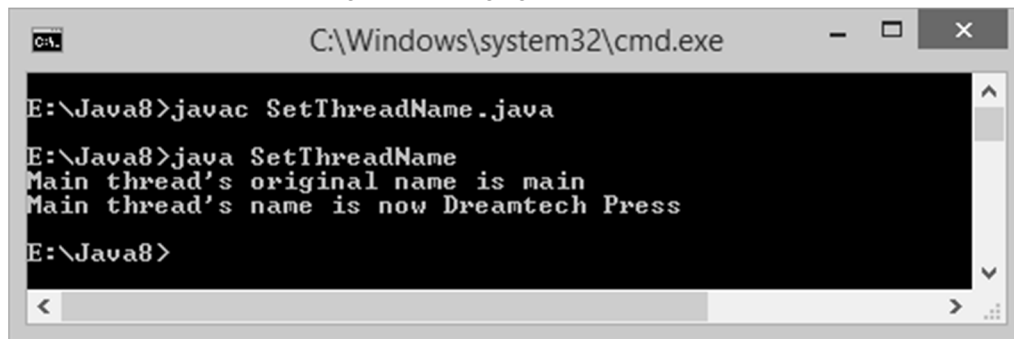


Figure 8.3: Giving Name to the Main Thread

After you have given a thread a name, you can use the `getName()` method to check the name and thus; determine which thread you are working with in a section of code that many threads execute (such as the code in a `Thread` class's `run()` method).

Pausing a Thread

Sometimes, you want to pause a thread's execution, and you can do that with the `sleep()` method. You pass this method the amount of time to pause, in milliseconds (a millisecond is *one-thousandth* of a second), and the thread will wait that amount of time before continuing. Here's an example in which we print out a message, pausing for a second between words:

```
class SleepDemo
{
    public static void main(String args[])
    {
        try
        {
            System.out.println("welcome");
            Thread.sleep(1000);
            System.out.println("to");
            Thread.sleep(1000);
            System.out.println("Dreamtech Press");
        }
        catch (InterruptedException e) { }
    }
}
```

The result is shown in Figure 8.4:

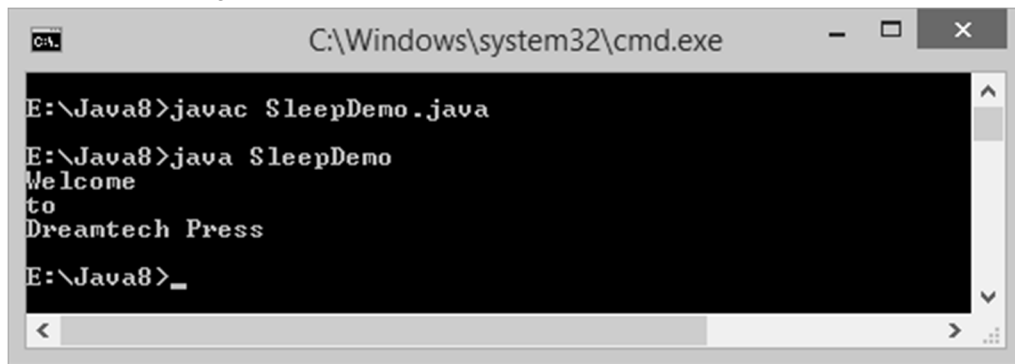


Figure 8.4: Pausing a Thread

When you run this code, you'll see that the words appear with a one-second pause between them.

Let's look at another example on sleeping in Figure 8.5:

```
class NewThread implements Runnable {
    String Tn;
    Thread t;
    NewThread(String threadname) {
        Tn = threadname;
        t = new Thread(this, Tn);
        System.out.println("New thread: " + t);
        t.start();
    }
    public void run() {
        try {
            for(int j = 75; j > 50; j--) {
                System.out.println(Tn + ": " + j);
                Thread.sleep(100);
            }
        } catch (InterruptedException e) {
            System.out.println(Tn + " breaking up...");
        }
        System.out.println(Tn + " finishing...");
    }
}

class Sleep {
    public static void main(String args[]) {
        NewThread ob1 = new NewThread("1");
        NewThread ob2 = new NewThread("2");
        try {
            Thread.sleep(1500);
            ob1.t.suspend();
            System.out.println("Thread 1's sleeping in progress");
            Thread.sleep(1500);
            ob1.t.resume();
            System.out.println("waking up thread one");
            ob2.t.suspend();
            System.out.println("Thread 2 starting to sleep...");
            Thread.sleep(1500);
            ob2.t.resume();
            System.out.println("Thread Two, waking up...");
        } catch (InterruptedException e) {
```

```

        System.out.println("Main thread broken"); }
    try {
        System.out.println("waiting for threads to finish.");
        ob1.t.join();
        ob2.t.join(); }
    catch (InterruptedException e) {
        System.out.println("Main thread broken"); }
    System.out.println("Finishing Main thread"); }
}

```

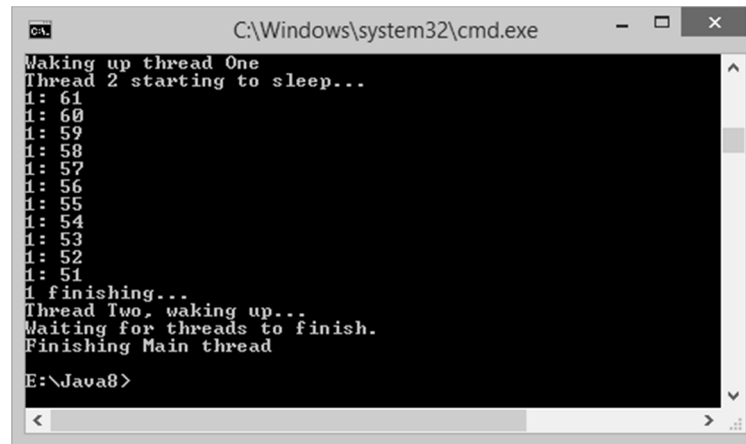


Figure 8.5: Sleeping of a Thread

Creating a Thread with the **Runnable** Interface

Implementing the `Runnable` interface is one of the two ways to create your own threads in Java. This interface has only one method: `run()`. You place the code you want to have the new thread execute in that method. When you create a new `Runnable` object, you can pass that object to the constructor of the `Thread` class to create the new thread.

Here's an example in which we create a new class that implements the `Runnable` interface. It passes itself to a `Thread` class constructor and starts itself when an object of this class is created. When it runs, it prints out its name once a second for 10 seconds and then quits. Here's the code for this new `Runnable` class, which we call `SecondThread` (note that the actual work is done in the `run()` method and that the code is executed with the `Thread` class's `start()` method):

```

class SecondThread implements Runnable {
    Thread thread;
    SecondThread() {
        thread = new Thread(this, "second");
        System.out.println("Starting Child Thread.");
        thread.start(); }
    public void run() {
        try {
            (Thread.currentThread()).setName("Child");
            for(int loop_index = 0; loop_index < 10; loop_index++) {
                System.out.println((Thread.currentThread()).getName()+
                    " thread here...\n");
                Thread.sleep(1000); }
        }
        catch (InterruptedException e) { }
        System.out.println("Second thread Ending."); }
}

```

Here's the application that will create a new thread of the `SecondThread` class and run it; note that we are also having the main thread print out its name so that you can see the two threads alternatively:

```
class RunnableDemo {
    public static void main(String args[]) {
        System.out.println("Starting Parent Thread.");
        SecondThread secondthread = new SecondThread();
        try {
            (Thread.currentThread()).setName("Parent");
            for(int loop_index = 0; loop_index < 10; loop_index++) {
                System.out.println((Thread.currentThread()).getName()+
                    " thread here...");
                Thread.sleep(1000); }
        }
        catch (InterruptedException e) { }
        System.out.println("Parent Thread Ending."); }
}
```

The result of this code is shown in Figure 8.6:

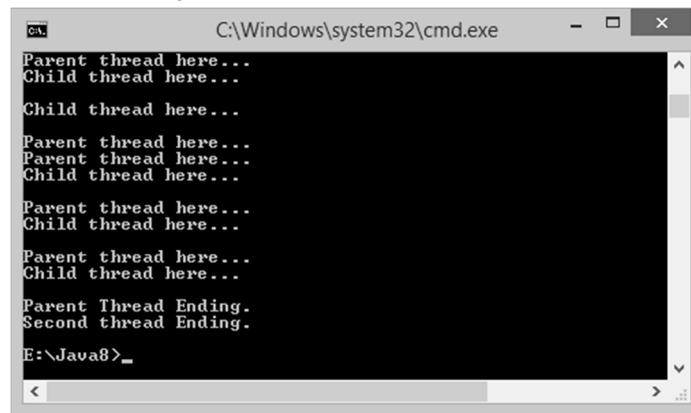


Figure 8.6: Alternate Printing of Threads

Note that the two threads alternately print out their names (also note that you can't guarantee that they'll be displayed alternately—*that's up to the operating system*).

Let us take another example of creating a thread. The easiest way to do that is to write a class that extends the `Thread` class. All that needed to start a thread is to create an instance of the `Thread` class and call its `start()` method.

Look at this class named `CountDownClock` that extends the `Thread` class:

```
public class CountDownClock extends Thread
{
    public void run()
    {
        for (int t = 20; t >= 0; t--)
        {
            System.out.println("T minus " + t);
            try { Thread.sleep(1000); }
            catch (InterruptedException e) { }
        }
    }
}
```

Now, let us create a class that extends the `Thread` class. But, there is a twist—the `Thread` class can have one superclass. In this case, you must create a class that implements the `Runnable` interface instead of extending the `Thread` class.

As you already know that the `Runnable` interface marks an object that can be run as a thread. So, a code for the creation of the `Runnable` object named `LaunchTask` is as follows:

```

public class LaunchTask implements Runnable
{
    private int start;
    private String message;
    public LaunchTask(int start, String message)
    {
        this.start = start;
        this.message = message;
    }
    public void run()
    {
        try { Thread.sleep(20000 - (start * 1000)); }
        catch (InterruptedException e){ }
        System.out.println(message);
    }
}

```

Now, we'll write the code for the main() method that will use LaunchTask and CountDownLatch to produce the result:

```

public class Countdown
{
    public static void main(String[] args)
    {
        Thread clock = new CountDownLatch(1);
        Runnable flood, ignition, liftoff;
        flood = new LaunchTask(16, "READY->Flood the pad!");
        ignition = new LaunchTask(6, "STEADY-->Start engines!");
        liftoff = new LaunchTask(0, "GO-->Liftoff!");
        clock.start();
        new Thread(flood).start();
        new Thread(ignition).start();
        new Thread(liftoff).start();
    }
}

```

Here's the result of the code:

```

E:\Java8>java Countdown
T minus 20
T minus 19
T minus 18
T minus 17
T minus 16
READY->Flood the pad!
T minus 15
T minus 14
T minus 13
T minus 12
T minus 11
T minus 10
T minus 9
T minus 8
T minus 7
T minus 6
STEADY->Start engine!
T minus 5
T minus 4
T minus 3
T minus 2
T minus 1
GO->Liftoff!
T minus 0

```

Creating a Thread with the **Thread** Class

The Thread class supports new threads. It also implements the Runnable interface, so you don't have to. Here's the inheritance diagram for Thread:


```
java.lang.Object
|____java.lang.Thread
```

You'll find the fields for the `Thread` class in Table 8.1, its constructors in Table 8.2, and its methods in Table 8.3:

Table 8.1: Fields of the Thread class

Field	Does this
<code>static int MAX_PRIORITY</code>	The maximum priority a thread can have
<code>static int MIN_PRIORITY</code>	The minimum priority a thread can have
<code>static int NORM_PRIORITY</code>	The default priority assigned to a thread

Table 8.2: Constructors of the Thread class

Constructor	Does this
<code>Thread()</code>	It constructs a new <code>Thread</code> object.
<code>Thread(Runnable target)</code>	It constructs a new <code>Thread</code> object using a <code>Runnable</code> object.
<code>Thread(Runnable target, String name)</code>	It constructs a new <code>Thread</code> object with a <code>Runnable</code> object and a name.
<code>Thread(String name)</code>	It constructs a new <code>Thread</code> object with a name.
<code>Thread(ThreadGroup group, Runnable target)</code>	It constructs a new <code>Thread</code> object as part of a group.
<code>Thread(ThreadGroup group, Runnable target, String name)</code>	It constructs a new <code>Thread</code> object with a <code>Runnable</code> object. This <code>Thread</code> object has the specified name as its name and belongs to the thread group referred to by <code>group</code> .
<code>Thread(ThreadGroup group, Runnable target, String name, long stackSize)</code>	It constructs a new <code>Thread</code> object so that it has <code>target</code> as its run object, has the specified name as its name, belongs to the thread group referred to by <code>group</code> , and has the specified stack size.
<code>Thread(ThreadGroup group, String name)</code>	It constructs a new <code>Thread</code> object that's part of a group and has a name.

Table 8.3: Methods of the Thread class

Method	Does this
<code>static int activeCount()</code>	It gets the current number of active threads in this thread's thread group.
<code>void checkAccess()</code>	It determines whether the currently running thread has the permission to modify this thread.
<code>int countStackFrames()</code>	It is deprecated. The definition of this call depends on <code>suspend</code> , which is deprecated.
<code>static Thread currentThread()</code>	It gets a reference to the currently executing <code>Thread</code> object.
<code>static void dumpStack()</code>	It prints a stack trace of the current thread.
<code>static int enumerate(Thread[] tarray)</code>	It copies to the indicated array every active thread in this thread's thread group.
<code>ClassLoader getContextClassLoader()</code>	It gets the context <code>ClassLoader</code> for this thread.
<code>String getName()</code>	It gets this thread's name.
<code>int getPriority()</code>	It gets this thread's priority.
<code>ThreadGroup getThreadGroup()</code>	It gets the thread group to which this thread belongs.
<code>void interrupt()</code>	It interrupts this thread.
<code>static boolean interrupted()</code>	It returns true if the current thread has been interrupted.
<code>boolean isAlive()</code>	It returns true if this thread is alive.

Table 8.3: Methods of the Thread class

Method	Does this
<code>boolean isDaemon()</code>	It returns true if this thread is a daemon thread.
<code>boolean isInterrupted()</code>	It returns true if this thread has been interrupted.
<code>void join()</code>	It waits for this thread to die.
<code>void join(long millis)</code>	It waits at most <code>millis</code> milliseconds for this thread to die.
<code>void join(long millis, int nanos)</code>	It waits at most <code>millis</code> milliseconds plus <code>nanos</code> nanoseconds for this thread to die.
<code>void resume()</code>	It is deprecated. This method exists solely for use with <code>suspend</code> , which is deprecated.
<code>void run()</code>	If this thread was constructed using a separate <code>Runnable</code> interface <code>run</code> object, that <code>Runnable</code> object's <code>run</code> method is called.
<code>void setContextClassLoader(ClassLoader cl)</code>	It sets the context <code>ClassLoader</code> for this thread.
<code>void setDaemon(boolean on)</code>	It marks this thread as either a daemon thread or a user thread.
<code>void setName(String name)</code>	It changes the name of this thread to be equal to the argument name.
<code>void setPriority(int newPriority)</code>	It changes the priority of this thread.
<code>static void sleep(long millis)</code>	It causes the currently executing thread to sleep for the indicated number of milliseconds.
<code>static void sleep(long millis, int nanos)</code>	It causes the currently executing thread to sleep for the indicated number of milliseconds plus the indicated number of nanoseconds.
<code>void start()</code>	It causes this thread to begin execution, which means that the Java Virtual Machine will call the <code>run</code> method of this thread.
<code>void stop()</code>	It is deprecated. This method is considered inherently unsafe.
<code>void stop(Throwables obj)</code>	It is deprecated. This method is considered inherently unsafe.
<code>void suspend()</code>	It is deprecated. This method has been deprecated because it's considered inherently deadlock prone.
<code>String toString()</code>	It gets a string representation of this thread.
<code>static void yield()</code>	It indicates that the current thread is willing to yield its current use of a processor.

Here's an example in which we create a thread and have it print out its name 10 times, once a second. Note that in this case, we are calling the thread's constructor and passing it a name for this thread, and then we are calling the thread's `start()` method to start execution of the code in the `run` method. The action takes place in the `run()` method, that's where we have the thread print out its name every second. Here's the code:

```
class ChildThread extends Thread { ChildThread() { super("Child"); start(); }
    public void run() { try {
        System.out.println("Child Thread Started.");
        for(int loop_index = 0; loop_index < 10; loop_index++) {
            System.out.println((Thread.currentThread()).getName()+ "
            thread here...");
            Thread.sleep(1000);
        }
    }
    catch (InterruptedException e) { }
    System.out.println("Child Thread Ended."); }
}
```

Here's an application in which we use the `ChildThread` class and also print out the name of the main thread every second:

```
class ThreadDemo {
    public static void main(String args[]) {
```

```
ChildThread childthread = new ChildThread();
try {
    for(int loop_index = 0; loop_index < 10; loop_index++) {
        System.out.println((Thread.currentThread()).getName()+
            " thread here...");
        Thread.sleep(1000); }
    }
catch (InterruptedException e) { }
}
```

The result of this code is shown below. Note that both threads are running and displaying their names, once a second:

[illegible]

Creating Multiple Threads

You can create and run multiple threads in a program—just give each thread a new object. Here’s an example in which we create four new threads and have each one print out its name once a second:

```

class CustomThread extends Thread {
    CustomThread(String name) {
        super(name);
        start(); }
    public void run() {
        try {
            for(int loop_index = 0; loop_index < 4; loop_index++) {
                System.out.println((Thread.
                    currentThread()).getName()+ " thread here...");
                Thread.sleep(1000); }
        }
        catch (InterruptedException e) { }
        System.out.println((Thread.currentThread()).getName() +" ending."); }
}

class MultiThreadDemo {
    public static void main(String args[]) {
        CustomThread thread1 = new CustomThread("First");
        CustomThread thread2 = new CustomThread("Second");
        CustomThread thread3 = new CustomThread("Third");
        CustomThread thread4 = new CustomThread("Fourth");
        try {
            for(int loop_index = 0; loop_index < 10; loop_index++)
                System.out.println((Thread.currentThread()).getName()+
                    " thread here...");
        }
    }
}

```

```

        Thread.sleep(1000); }
    }
    catch (InterruptedException e) { }
}

```

The result of this code is shown below:

```

E:\Java8>java MultiThreadDemo
First thread here...
Third thread here...
Second thread here...
main thread here...
Fourth thread here...
Second thread here...
First thread here...
Third thread here...
main thread here...
Fourth thread here...
Second thread here...
Third thread here...
First thread here...
main thread here...
Fourth thread here...
Third thread here...
First thread here...
Second thread here...
main thread here...
Fourth thread here...
First ending.
Second ending.
Third ending.
main thread here...
Fourth ending.
main thread here...
main thread here...
main thread here...
main thread here...
main thread here...

```

Let us take another example to understand this concept better:

```

class NewThread implements Runnable {
    String name;
    Thread t;
    NewThread(String threadname) {
        name = threadname;
        t = new Thread(this, name);
        System.out.println("New thread: " + t);
        t.start(); }
    public void run() {
        try {
            for(int i = 100; i > 95; i--) {
                System.out.println(name + ": " + i);
                Thread.sleep(1000); }
        }
        catch (InterruptedException e) {
            System.out.println(name + "Interrupted"); }
        System.out.println(name + " exiting."); }
}
class StartThread {
    public static void main(String args[]) {
        new NewThread("1:");
        new NewThread("2:");
        new NewThread("3:");
        try { Thread.sleep(10000); }
        catch (InterruptedException e) {
            System.out.println("Parent thread Interrupted"); }
    }
}

```

```
        System.out.println("Parent thread exiting."); }
    }
```

The result of the code is shown below:

```
New thread: Thread[1:,5,main]
New thread: Thread[2:,5,main]
New thread: Thread[3:,5,main]
1:: 100
3:: 100
2:: 100
2:: 99
3:: 99
1:: 99
2:: 98
1:: 98
3:: 98
2:: 97
3:: 97
1:: 97
2:: 96
1:: 96
3:: 96
2: exiting.
3: exiting.
1: exiting.
Parent thread exiting.
```

Joining Threads

The `Thread` class's `join()` method waits until a thread is finished executing (also called *waiting for a thread to die*) before returning. Here's an example in which we create four new threads and wait for each one to finish executing before ending the main application:

```
class CustomThread extends Thread
{
    CustomThread(String name)
    {
        super(name);
        start();
    }
    public void run()
    {
        try {
            for(int loop_index = 0; loop_index < 4; loop_index++) {
                System.out.println((Thread.currentThread().getName()+ "
                thread here...");
                Thread.sleep(1000); } }
            catch (InterruptedException e) { }
            System.out.println((Thread.currentThread().getName()+" ending."));}
    }
class JoinDemo {
    public static void main(String args[]) {
        CustomThread thread1 = new CustomThread("First");
        CustomThread thread2 = new CustomThread("Second");
        CustomThread thread3 = new CustomThread("Third");
        CustomThread thread4 = new CustomThread("Fourth");
        try {
            thread1.join();
            thread2.join();
            thread3.join();
            thread3.join(); }
            catch (InterruptedException e) { }
        }
    }
```

The output of this code is shown below:

```
E:\Java8>java JoinDemo
Second thread here...
First thread here...
Third thread here...
Fourth thread here...
Fourth thread here...
Third thread here...
Second thread here...
First thread here...
Third thread here...
First thread here...
Second thread here...
Fourth thread here...
Second thread here...
First thread here...
Fourth thread here...
Third thread here...
First ending.
Third ending.
Fourth ending.
Second ending.
```

Checking if a Thread Is Alive

Here's an example in which we create and run four new threads. As the threads are executing, we use the `isAlive()` method on the first new thread, and when all threads are done (which we verify with the `join()` method), we check that thread again with `isAlive()`. Here's the code:

```
class CustomThread extends Thread
{
    CustomThread(String name)
    {
        super(name);
        start();
    }
    public void run() {
        try {
            for(int loop_index = 0; loop_index < 4; loop_index++) {
                System.out.println((Thread.
                    currentThread()).getName()+ " thread here...");
                Thread.sleep(1000);
            }
            catch (InterruptedException e) { }
            System.out.println((Thread.currentThread()).getName() + "
                ending.");
        }
    }
}
class isAliveDemo
{
    public static void main(String args[])
    {
        CustomThread thread1 = new CustomThread("first");
        CustomThread thread2 = new CustomThread("second");
        CustomThread thread3 = new CustomThread("third");
        CustomThread thread4 = new CustomThread("fourth");
        System.out.println("Thread1 is alive- "+thread1.isAlive());
        try {
            thread1.join();
            thread2.join();
            thread3.join();
            thread4.join();
        }
        catch (InterruptedException e) { }
        System.out.println(thread1.isAlive());
    }
}
```

The output of this code is shown below:

```
E:\Java8>java isAliveDemo
first thread here...
third thread here...
Thread1 is alive- true
second thread here...
fourth thread here...
first thread here...
third thread here...
second thread here...
fourth thread here...
first thread here...
third thread here...
fourth thread here...
second thread here...
third thread here...
first thread here...
second thread here...
fourth thread here...
first ending.
third ending.
second ending.
fourth ending.
false
```

In the preceding output, note the reported value of `isAlive()`, which is `true` when `thread1` is running and `False` after that thread has quit.

Setting Thread Priority and Stopping Threads

Each thread in Java has a priority, which is between `Thread.MAX_PRIORITY` and `Thread.MIN_PRIORITY`. Here are the priorities that are currently defined:

- ❑ `MAX_PRIORITY`—The maximum priority that a thread can have
- ❑ `MIN_PRIORITY`—The minimum priority that a thread can have
- ❑ `NORM_PRIORITY`—The default priority assigned to a thread

The value of `Thread.MAX_PRIORITY` is 10, `Thread.MIN_PRIORITY` is 1, and `NORM_PRIORITY` is 5.

Setting priorities is a very tricky business because it's operating system dependent, and the actual results depend very much on what else is running at the same time. Nonetheless, you can set the relative priorities of threads. Here's an example in which we give four threads different priorities and just have them count for 5 seconds. Then, we end them and display their results to get an indication of their relative execution speeds. Here's the code (note that to stop the threads, we are using a flag that's checked in the `run()` method and is defined as `volatile`, which means its value can be changed by another thread):

```
class Counter implements Runnable {
    Thread thread;
    int counter = 0;
    volatile boolean goflag;
    public Counter(int p) {
        thread = new Thread(this);
        thread.setPriority(p); }
    public void start() {
        goflag = true;
        thread.start(); }
    public void run() {
        while (goflag)
            counter++; }
    public void end() { goflag = false; }
}
class PriorityDemo {
    public static void main(String args[]) {
```

```

Counter thread1 = new Counter(Thread.NORM_PRIORITY + 2);
Counter thread2 = new Counter(Thread.NORM_PRIORITY + 1);
Counter thread3 = new Counter(Thread.NORM_PRIORITY - 1);
Counter thread4 = new Counter(Thread.NORM_PRIORITY - 2);
thread1.start();
thread2.start();
thread3.start();
thread4.start();
try { Thread.sleep(5000); }
catch (InterruptedException e) { }
thread1.end();
thread2.end();
thread3.end();
thread4.end();
System.out.println("Thread 1 is counted: " + thread1.counter);
System.out.println("Thread 2 is counted: " + thread2.counter);
System.out.println("Thread 3 is counted: " + thread3.counter);
System.out.println("Thread 4 is counted: " + thread4.counter); }
}

```

To stop the threads, we call the custom `end()` method, which sets a flag named `goFlag` to `False`, which in turn stops the code in the `run()` method. In the early days of Java, you could stop a thread with a built-in `stop()` method, but that method is now deprecated because it's unsafe—it can leave operating system access monitors in an unstable state. Instead, to stop a thread, you now use the preceding method—that is, checking some flag that you can set from outside the thread.

In the output, note that the different number of counts from the threads with different priorities (your results may vary, of course, depending on your operating system and machine speed). The output of the code is shown in Figure 8.7, showing the threads in accordance with their priorities:

```

C:\Windows\system32\cmd.exe
E:\Java8>javac PriorityDemo.java
E:\Java8>java PriorityDemo
Thread 1 is counted: -1724214269
Thread 2 is counted: -1731777872
Thread 3 is counted: -1915446626
Thread 4 is counted: -1938787916
E:\Java8>_

```

Figure 8.7: Counting the Threads with Different Priorities

Synchronizing

Excluding other threads from an operation until that operation has completed is called *synchronization*. Here's an example that shows why synchronization is useful. In this case, we have four new threads, all working with one shared data object. This object prints out a start message and an end message, and it completes a half-second task between the messages. Ideally, the data object should start, perform its task, and end before the next thread starts a new task but that's not the way it works here—each thread grabs the data object for itself. Here's the code:

```

class SynchronizeDemo
{
    public static void main(String args[])
    {
        Shared shared = new Shared();
        CustomThread thread1 = new CustomThread(shared, "one");
        CustomThread thread2 = new CustomThread(shared, "two");
    }
}

```



```

        CustomThread thread3 = new CustomThread(shared, "three");
        CustomThread thread4 = new CustomThread(shared, "four");
        try {
            thread1.join();
            thread2.join();
            thread3.join();
            thread4.join(); }
        catch (InterruptedException e) { }
    }
}
class CustomThread extends Thread {
    Shared shared;
    public CustomThread(Shared shared, String string) {
        super(string);
        this.shared = shared;
        start(); }
    public void run() { shared.dowork(Thread.currentThread().getName()); }
}
class Shared {
    void dowork(String string) {
        System.out.println("Starting " + string);
        try { Thread.sleep((long) (Math.random() * 500)); }
        catch (InterruptedException e) {}
        System.out.println("Ending " + string); }
}

```

The output of this code is shown in Figure 8.8, showing the synchronization between the threads at various intervals:

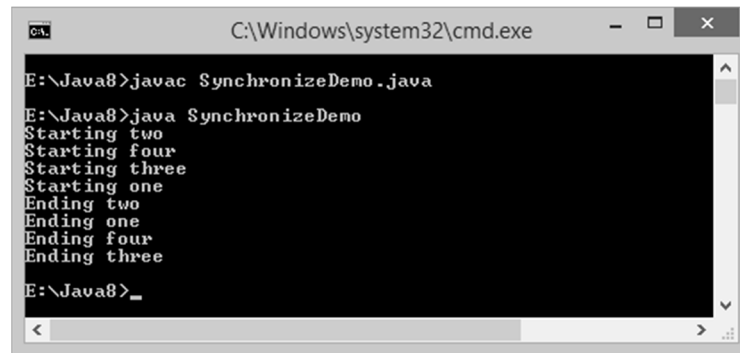


Figure 8.8: Synchronization between Threads

Note that each task overlaps the other.

This is a problem if you are working with actual data—what if you read some data from the data object, worked on it, and were about to rewrite it, but another thread has already done the same thing? When you rewrite the data, the changes from the other thread will be lost. As you can see, it's important to be able to block access to other threads to critical resources at times. Take a look at the next two solutions to see how this works.

Synchronizing Code Blocks

The two ways to synchronize thread code execution are synchronizing blocks of code and synchronizing methods. We'll take a look at synchronizing blocks of code in this solution and synchronizing methods in the next. To synchronize a block of code, you use the `synchronized()` method, indicating the object you want to restrict access to. Extending the example from the previous solution, the process looks like this:

```

class SynchronizeDemo1 {
    public static void main(String args[]) {
        Shared shared = new Shared();
        CustomThread thread1 = new CustomThread(shared, "two");
        CustomThread thread2 = new CustomThread(shared, "one");
        CustomThread thread3 = new CustomThread(shared, "five");
    }
}

```

```

        CustomThread thread4 = new CustomThread(shared, "four");
        try {
            thread1.join();
            thread2.join();
            thread3.join();
            thread4.join(); }
        catch (InterruptedException e) { }
    }
}
class CustomThread extends Thread {
    Shared shared;
    public CustomThread(Shared shared, String string) {
        super(string);
        this.shared = shared;
        start(); }
    public void run() {
        synchronized(shared) {
            shared.dowork(Thread.currentThread().getName()); }
    }
}
class Shared {
    void dowork(String string) {
        System.out.println("Stop " + string);
        try { Thread.sleep((long) (Math.random() * 500)); }
        catch (InterruptedException e) { }
    }
}

```

The output of this code is shown in Figure 8.9, showing the synchronization between code blocks:

```

C:\Windows\system32\cmd.exe
E:\Java8>javac SynchronizeDemo1.java
E:\Java8>java SynchronizeDemo1
Stop two
Stop four
Stop five
Stop one
E:\Java8>_

```

Figure 8.9: Synchronization between Code Blocks

Note that each task starts and ends before any other thread starts a new task.

Synchronizing Methods

Besides synchronizing code blocks, you can also block access to other threads by synchronizing methods. In this case, you use the `synchronized` keyword when defining a method. Here's how we modify the `dowork()` method in the `Shared` class and synchronize the threads:

```

class SynchronizeDemo2 {
    public static void main(String args[]) {
        Shared shared = new Shared();
        CustomThread thread1 = new CustomThread(shared, "one");
        CustomThread thread2 = new CustomThread(shared, "two");
        CustomThread thread3 = new CustomThread(shared, "three");
        CustomThread thread4 = new CustomThread(shared, "four");
        try {
            thread1.join();
            thread2.join();
            thread3.join();
            thread4.join(); }
        catch (InterruptedException e) { }
    }
}

```

```

class CustomThread extends Thread {
    Shared shared;
    public CustomThread(Shared shared, String string) {
        super(string);
        this.shared = shared;
        start(); }
    public void run() { shared.dowork(Thread.currentThread().getName()); }
}
class Shared {
    synchronized void dowork(String string) {
        System.out.println("Starting " + string);
        try { Thread.sleep((long) (Math.random() * 500)); }
        catch (InterruptedException e) { }
        System.out.println("Ending " + string); }
}

```

Now, only one thread at a time can enter the `dowork()` method. The output of this code is shown in Figure 8.10:

```

C:\Windows\system32\cmd.exe

E:\Java8>javac SynchronizeDemo2.java
E:\Java8>java SynchronizeDemo2
Starting one
Ending one
Starting four
Ending four
Starting three
Ending three
Starting two
Ending two
E:\Java8>_

```

Figure 8.10: Synchronization between Methods

Let us look into another program that uses `synchronized()` method. In this program, in order to enter an object's monitor, just call a method that has been modified with the `synchronized` keyword. While a thread is inside a `synchronized()` method, all other threads that try to call it on the same instance have to wait. To relinquish control to the next waiting thread, the object returns from the `synchronized()` method:

```

class Callme {
    void call(String msg) {
        System.out.print("***" + msg);
        try { Thread.sleep(1000); }
        catch (InterruptedException e) { System.out.println("Stop..."); }
        System.out.println("*****"); }
}
class Caller implements Runnable {
    String ms;
    Callme target;
    Thread h;
    public Caller(Callme targ, String s) {
        target = targ;
        ms = s;
        h = new Thread(this);
        h.start(); }
    public void run() { synchronized(target) { target.call(ms); } }
}
class Sync {
    public static void main(String args[]) {
        Callme target = new Callme();
        Caller c1 = new Caller(target, "Your");
        Caller c2 = new Caller(target, "JAVA");
        Caller c3 = new Caller(target, "Synchronized");
    }
}

```

```

        try {
            c1.h.join();
            c2.h.join();
            c3.h.join(); }
        catch (InterruptedException e) { System.out.println("Stop..."); }
    }
}

```

The result of the code is shown below:

```

E:\Java8>java Sync
**Your****
**JAVA****
**Synchronized****

```

Synchronizing Classes

Synchronization gives you the choice of adding the `synchronized` keyword to a method or wrapping a code section in a synchronized block. Several mechanisms for coordinating between different threads in an application have been added. Another useful category of classes in `java.util.concurrent` is the synchronizers. This set of classes coordinates and controls the flow of execution for one or more threads.

The `Semaphore`, `CyclicBarrier`, `CountdownLatch`, and `Exchanger` classes are all examples of synchronizers. These classes have methods that are called by threads, which may or may not block on the basis of the state and rules of the specific synchronizer being used.

The Semaphore Class

Prior to acquiring an item, a thread must obtain a permit from the semaphore, ensuring that an item will be available for use. Once the thread finishes with the item, it goes back to the pool, and a permit is given back to the semaphore, enabling another thread to acquire that item. The semaphore encapsulates the synchronization needed to restrict permit to the pool, separately from any synchronization required for maintaining the consistency of the pool itself.

Consider an example of a queue of people who came for their Aadhaar card registration. Each of them will be allotted a token number. The next day, they will be called according to their token number. A person who got a token number has occupied a certain position that is locked by him/her, and no other person shall be granted the same token number.

So, we can say that a semaphore restricts multiple numbers of users accessing the same token.

Let us discuss some of the terms related to synchronization. They are lock, permit, and block.

A lock is created so as to control the access of multiple threads over the same resource. It provides exclusive access to a shared resource, which means it allows only one thread to access an object at a time.

A permit is like a token to a lock that allows the thread to access a resource. A thread must have a permit from the semaphore before obtaining a resource. When the thread stops, it returns the permit to the semaphore, allowing another thread to access next.

A block indicates that a thread is waiting for the lock or permit to access the resource.

With those definitions in mind, let's look at the new `Semaphore` class, the class in Java that offers support for semaphores. You create the object of class with one of the following two constructors presented in Table 8.4:

Table 8.4: Constructors of the Semaphore class	
Constructor	Does this
<code>Semaphore(int permits)</code>	It creates a semaphore with a given number of permits and non-fair fairness settings
<code>Semaphore(int permits, boolean fair)</code>	It creates a semaphore with a given number of permits and the given fairness settings

The methods of the Semaphore class and their functions are listed in Table 8.5:

Table 8.5: Methods of the Semaphore class	
Method	Does this
<code>void acquire()</code>	It acquires a permit from this semaphore, blocking until one is available, or the thread is interrupted
<code>void acquire(int permits)</code>	It acquires the given number of permits from this semaphore, blocking until all are available, or the thread is interrupted
<code>void acquireUninterruptibly()</code>	It acquires a permit from this semaphore, blocking until one is available
<code>void acquireUninterruptibly(int permits)</code>	It acquires the given number of permits from this semaphore, blocking until all are available
<code>int availablePermits()</code>	It returns the current number of permits available in this semaphore
<code>int drainPermits()</code>	It acquires and returns all permits that are immediately available
<code>protectedCollection<thread> getQueuedThreads()</code>	It returns a collection containing threads that may be waiting to acquire
<code>int getQueueLength()</code>	It returns an estimate of the number of threads waiting to acquire
<code>boolean hasQueuedThreads()</code>	It queries whether any threads are waiting to acquire
<code>boolean isFair()</code>	It returns true if this semaphore has fairness set true
<code>protectedvoid reducePermits(int reduction)</code>	It shrinks the number of available permits by the indicated reduction
<code>void release()</code>	It releases a permit, returning it to the semaphore
<code>void release(int permits)</code>	It releases the given number of permits, returning them to the semaphore
<code>string toString()</code>	It returns a string identifying this semaphore as well as its state
<code>boolean tryAcquire()</code>	It acquires a permit from this semaphore, only if one is available at the time of invocation
<code>boolean tryAcquire(int permits)</code>	It acquires the given number of permits from this semaphore, only if all are available at the time of invocation
<code>boolean tryAcquire(int permits, long timeout, TimeUnit unit)</code>	It acquires the given number of permits from this semaphore, if all become available within the given waiting time, and the current thread has not been interrupted
<code>boolean tryAcquire(long timeout, TimeUnit unit)</code>	It acquires a permit from this semaphore, if one becomes available within the given waiting time, and the current thread has not been interrupted

The created semaphore has a “non-fair” setting. This means that First In First Out (FIFO) behavior is not guaranteed. To prevent starvation (a thread never getting a resource), it is better to create a fair semaphore. However, there are throughput advantages if you ignore fairness. To acquire a permit from a semaphore, the following two methods are invoked:

```
acquire()
acquire(int permits)
```

The `acquire()` method gets a single permit.

The `acquire(int permits)` method gets the requested number of permits. The methods block until a permit is available, or the waiting thread is interrupted. If the thread is interrupted, an `InterruptedException` is thrown.

The next two methods we are going to discuss, too, acquire permits, but their threads are dormant and uninterruptible until the required numbers of permits are available. If a thread is interrupted while waiting, it is regained after the resource becomes available:

```
acquireUninterruptibly()
acquireUninterruptibly(int permits)
```

The `tryAcquire()` method does not block. If the required number of permits is available, the methods return `true`. If the permits are not available, `false` is returned. All permits must be true to get available. The fairness setting is ignored with these untimed `tryAcquire()` methods:

```
tryAcquire()
tryAcquire(int permits)
```

The timed versions work similar to the untimed version, but wait for the specified `TimeUnit` before returning. If interrupted while waiting, the methods throw an `InterruptedException`:

```
tryAcquire(long timeout, TimeUnit unit)
tryAcquire(int permits, long timeout, TimeUnit unit)
```

The following line shows how to wait 40 seconds for one permit:

```
boolean acq = tryAcquire(40, TimeUnit.SECONDS)
```

Now, let's look at an example that uses the `Semaphore` class. The following program, `TestingSemaphore`, matches with an auction house service system. The service allows customers to see multiple items available for purchase and displays an average of the prices of these items. However, due to licensing limitations, only two threads can access the service at a time. To simulate the price sampling and averaging, the `TestingSemaphore` program waits 30 milliseconds and then returns a random number from 0 to 100. If the service is concurrently accessed after the limit of two threads is reached, a second pricing scheme is used. This latter scheme doesn't have the licensing restriction but returns a less-reliable price (in this example, \$20). Here is the `TestingSemaphore` program:

```
import java.util.concurrent.*;
import java.util.*;
public class TestingSemaphore {
    private static final int COUNT_LOOP = 100;
    private static final int MAX_AVAILABLE = 2;
    private final static Semaphore sphore = new
        Semaphore(MAX_AVAILABLE, true);
    private static class Price {
        private static final Random rnd = new Random();
        public static int getPrice() {
            int prc = rnd.nextInt(100);
            try { Thread.sleep(30); }
            catch (InterruptedException ex) { ex.printStackTrace(); }
            return prc; }
        public static int getunfavPrice() { return 20; }
    }
    public static void main(String args[]) {
        for (int i=0; i< COUNT_LOOP; i++) {
            final int count = i;
            new Thread() {
                public void run() {
                    int prc;
                    if (sphore.tryAcquire()) {
                        try { prc = Price.getPrice(); }
                        finally { sphore.release(); }
                    } else { prc = Price.getunfavPrice(); }
                    System.out.println(count + ": " + prc); }
            }.start();
        }
    }
}
```

Depending on the speed of your machine, you might need to modify the 50 millisecond delay for getting reasonable results. "Reasonable results" means that you get some random values generated and some \$20 values. One sample runs as shown in Figure 8.11:

```

C:\Windows\system32\cmd.exe
E:\Java8>javac TestingSemaphore.java
E:\Java8>java TestingSemaphore
2: 20
16: 20
11: 20
13: 20
14: 20
8: 20
12: 20
7: 20
6: 20
5: 20
4: 20
3: 20
9: 20
10: 20
15: 20
39: 20
41: 20
38: 20
37: 20
36: 20

```

Figure 8.11 :Working of Semaphore

Your results might differ.

Notice how the results for pass 0 and 1 don't appear until after many other results. You then get more 20s before seeing more "good" results. Each call to the average pricing service takes time, therefore the interspersed results.

With the use of a semaphore, the program controls access to the limited resources. MUTEX-Mutual Exclusion semaphore or mutex is a special case of semaphore. Semaphores that allow an arbitrary resource count are called **counting semaphores**, while semaphores which are limited to the values 0 and 1 (or locked/unlocked, unavailable/available) are called **binary semaphores** (same functionality that mutexes have).

There are many features that are common in both mutexes and locks, except that mutexes have one additional feature that locks fail to have. It is the ability of the mutex to be released by a thread that is different from the one holding the permit.

The CyclicBarrier Class

The next synchronization pillar is the barrier. A barrier allows a common point (called a barrier point) for a set of threads to wait for each other before continuing their execution. The class that facilitates this functionality is called `CyclicBarrier`. This class allows one set of threads to pass through the barrier point till the time the next set of threads arrives without the requirement to create a new barrier.

An example used to define barriers is the splitting of a task into subtasks or segments such that each subtask or segment can be executed separately. For example, imagine a two-dimensional array or matrix, and you want the sum of all the values in the matrix. If you have a multiprocessor system, the splitting of the matrix will be faster. By dividing it into two halves, each processor sums up the values in its half. Then, when both sums are done, you add the sums together and return the combined total. That's exactly the use of a barrier. The barrier restricts the final addition till the addition of both halves is calculated.

The `CyclicBarrier` class allows you to create a barrier in one of the two ways. You can make a barrier with a number of threads, or with a number of threads and a barrier action. A barrier action is a `Runnable` task that executes when all the threads have joined together but before the threads are released to run on their own.

```

CyclicBarrier(int parties)
CyclicBarrier(int parties, Runnable barrierAction)

```

After each of the parties finishes its individual task, the party joins the barrier by calling its `await()` method. This blocks that party until all parties are present. If a barrier action is provided, it runs before releasing all the parties.

To demonstrate, the following code, `App.java`, performs a parallel summation of the elements of a triangular matrix:

```

import java.util.concurrent.*;
public class App {
    private static int matrices[][] = {
        {1},

```

```

        {2, 2},
        {3, 3, 3},
        {4, 4, 4, 4},
        {5, 5, 5, 5, 5}
    };
    private static int res[];
    private static class Summer extends Thread {
        int row;
        CyclicBarrier barrier;
        Summer(CyclicBarrier barrier, int row) {
            this.barrier = barrier;
            this.row = row; }
        public void run() {
            int cols = matrices[row].length;
            int sum = 0;
            for (int i=0; i<cols; i++) { sum += matrices[row][i]; }
            res[row] = sum;
            System.out.println("Results for row " + row + " are : " +
                sum);
            try { barrier.await(); }
            catch (InterruptedException ex) { ex.printStackTrace(); }
            catch (BrokenBarrierException ex) { ex.printStackTrace(); }
        }
    }
    public static void main(String args[]) {
        final int rows = matrices.length;
        res = new int[rows];
        Runnable merger = new Runnable() {
            public void run() {
                int sum = 0;
                for (int i=0; i<rows; i++) { sum += res[i]; }
                System.out.println("Results are: " + sum); }
        };
        CyclicBarrier barrier = new CyclicBarrier(rows, merger);
        for (int i=0; i<rows; i++) { new Summer(barrier, i).start(); }
        System.out.println("waiting..."); }
    }
}

```

Compile and run the program. The result is shown in Figure 8.12:

```

C:\Windows\system32\cmd.exe
E:\Java8>javac App.java
E:\Java8>java App
Waiting...
Results for row 3 are : 16
Results for row 2 are : 9
Results for row 4 are : 25
Results for row 0 are : 1
Results for row 1 are : 4
Results are: 55
E:\Java8>

```

Figure 8.12: Printing Rows and Columns by Using the CyclicBarrier Class

Creating a new thread and scheduling it doesn't necessarily cause the system to pause the existing thread. Multiple threads might be created, all with the same priority. The system simply picks one thread to schedule at that priority level. If you wish to reuse a cyclic barrier, you can call the `reset` method of `CyclicBarrier` between usages.

The CountdownLatch Class

The next synchronization aid is called a latch. Latching variables specify conditions that never change after being set. This provides a way to start several threads and have them wait until a signal is received from a coordinating thread. Latching works well with tasks where you want no process to run until everything needed

is initialized. Latching also works well for multiplayer games, where you don't want any player to start until all players have joined.

The `CountDownLatch` class encapsulates a latch that is initialized with a given count. The constructor takes the count as its one argument. The count works in a way similar to the `threads` argument to the `CyclicBarrier` constructor. It indicates how many times a `countDown()` method must be called, one for each party involved. After the full count is reached, any threads waiting, because of an `await()` method, are released.

The following program, `TestingLatch`, demonstrates the use of a `CountDownLatch`. The program creates a set of threads but doesn't let any thread start until all the threads are created:

```
import java.util.concurrent.*;
public class TestingLatch {
    private static final int COUNT = 5;
    private static class work implements Runnable {
        CountDownLatch Latch_start;
        CountDownLatch Latch_stop;
        String name;
        work(CountDownLatch Latch_start, CountDownLatch Latch_stop,
            String name) {
            this.Latch_start = Latch_start;
            this.Latch_stop = Latch_stop;
            this.name = name; }
        public void run() {
            try { Latch_start.await(); }
            catch (InterruptedException ex) { ex.printStackTrace(); }
            System.out.println("Running: " + name);
            Latch_stop.countDown(); }
    }
    public static void main(String args[]) {
        CountDownLatch Signal_start = new CountDownLatch(1);
        CountDownLatch Signal_stop = new CountDownLatch(COUNT);
        for (int i=0; i<COUNT; i++) {
            new Thread(new
                work(Signal_start, Signal_stop, Integer.toString(i))).start(); }
        System.out.println("Go");
        Signal_start.countDown();
        try { Signal_stop.await(); }
        catch (InterruptedException ex)
        { ex.printStackTrace(); }
        System.out.println("Done"); }
}
```

The main thread waits on a second latch to make sure that other threads are finished. This example could have simply used a `join` operation on the created threads; however, imagine if you needed to synchronize threads at different phases through their lifetime, instead of just at the end. The result is shown in Figure 8.13:

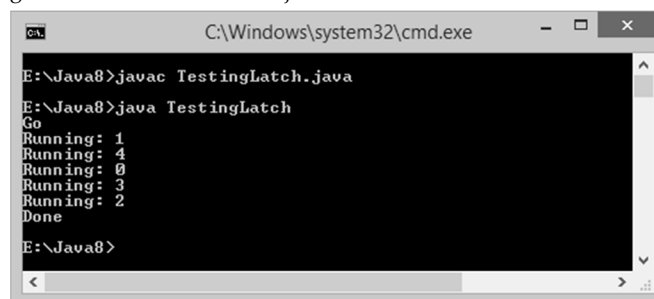


Figure 8.13: Using the `CountDownLatch` Class

The Exchanger Utility

The Exchanger is the last of the synchronization utilities covered in this chapter. It offers a simple way of communicating between threads, that is, by passing a specific object between two threads. That's why there's <V> after the class name. Instead of using the Piped streams for stream-based, inter-thread communication (where one side writes and the other reads), the Exchanger code relies on a single `exchange()` method for the transfer of one-off data between threads. The Exchanger constructor takes no arguments here, other than identifying the type of object to exchange:

```
Exchanger<List<Integer>> exchanger = new Exchanger<List<Integer>>();
```

To exchange objects, you call the `exchange()` method. The method transfers objects in both directions, not just one:

```
V exchange(V x)
```

When one of the threads is consuming a resource and the other one is producing, an exchanger is used. When the buffer used by the producer is full, the producer waits for the consumer. When the buffer consumed by the consumer is empty, it waits for the producer. After both waits occur, the two threads swap buffers.

Here's an example that uses the Exchanger. The `FillLoop` class is the producer type. The `EmptyLoop` class is the consumer. When the producer's data structure is full, it tries to exchange with the consumer. When the consumer's data structure is empty, it tries to exchange data structures with the producer. When both the producer and consumer are waiting, the exchange happens.

```
import java.util.*;
import java.util.concurrent.*;
public class ExchangerDemo {
    private static final int FULL = 10;
    private static final int COUNT = FULL * 20;
    private static final Random rnd = new Random();
    private static volatile int sum = 0;
    private static Exchanger<List<Integer>> exchanger =
        new Exchanger<List<Integer>>();
    private static List<Integer> EmptyBuffer;
    private static List<Integer> FullBuffer;
    private static CountDownLatch Latch_stop = new CountDownLatch(2);
    private static class FillLoop implements Runnable {
        public void run() { List<Integer> currentBuf = EmptyBuffer;
            try {
                for (int i = 0; i < COUNT; i++) {
                    if (currentBuf == null)
                        break;
                    Integer item = rnd.nextInt(100);
                    System.out.println("Added: " + item);
                    currentBuf.add(item);
                    if (currentBuf.size() == FULL)
                        currentBuf = exchanger.exchange(currentBuf);
                }
            } catch (InterruptedException ex) {
                System.out.println("Bad exchange on filling side");
                Latch_stop.countDown();
            }
        }
    }
    private static class EmptyLoop implements Runnable {
        public void run() {
            List<Integer> currentBuf = FullBuffer; try {
                for (int i = 0; i < COUNT; i++) {
                    if (currentBuf == null) break;
                    Integer item = currentBuf.remove(0);
                    System.out.println("Got: " + item);
                    sum += item.intValue();
                    if (currentBuf.isEmpty()) {
                        currentBuf =
                            exchanger.exchange(currentBuf);
                    }
                }
            }
        }
    }
}
```

```

        catch (InterruptedException ex) {
            System.out.println("Bad exchange on emptying side");
        }
        Latch_stop.countDown(); }
    }

    public static void main(String args[]) {
        EmptyBuffer = new ArrayList<Integer>();
        FullBuffer = new ArrayList<Integer>(FULL);
        for (int i=0; i<FULL; i++) { FullBuffer.add(rnd.nextInt(100)); }
        new Thread(new FillLoop()).start();
        new Thread(new EmptyLoop()).start();
        try { Latch_stop.await(); }
        catch (InterruptedException ex) { ex.printStackTrace(); }
        System.out.println("Sum of all items is.... " + sum); }
    }
}

```

The filling and emptying loops run a fixed number of times before stopping. They use a `CountDownLatch` to tell the main thread when they are done. The output includes sections of “Added” and “Got” values that should match, though the actual numbers might differ in your runs. The result is shown in Figure 8.14:

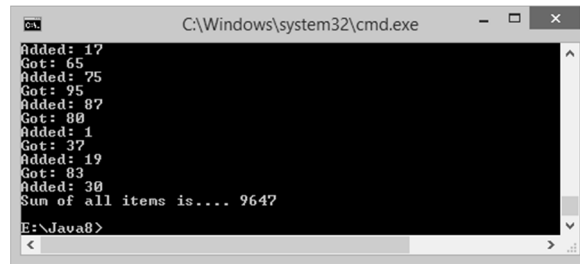


Figure 8.14: Using the Exchanger Class

Communicating between Threads

Often, threads will need to coordinate between themselves, especially when the output of one thread is used by another thread. One way of coordinating threads is to use the `wait()`, `notify()`, and `notifyAll()` methods:

- ❑ `wait()` —It makes a thread sleep until `notify()` or `notifyAll()` method is called.
- ❑ `notify()` —It starts the first thread that called `wait()` on the same object.
- ❑ `notifyAll()` —It starts all the threads that called `wait()` on the same object.

The usual process is for the reader thread to call `wait()` method and the writer thread to call `notify()` method when the data the reader wants to read is ready. Here’s an example. In this case, a writer thread will call an object’s `doWork()` method, which is a time-consuming method, to write some data, and a reader thread will call the same object’s `getResult()` method to read the results. Clearly, we want the reader thread to have to wait until `doWork()` method is done. Therefore, all we have to do is call `wait()` in `getResult()` method to make the reader thread wait and call `notify()` method in `doWork()` when the writer thread is done, and the data is ready to be read. Here’s what the code looks like:

```

class Shared {
    int data = 0;
    synchronized void doWork() {
        try { Thread.sleep(1000); }
        catch(InterruptedException e) { }
        data = 1;
        notify(); }
    synchronized int getResult() {
        try { wait(); }
        catch(InterruptedException e) { }
        return data; }
}

class CustomThread1 extends Thread {
    Shared shared;
}

```

```

    public CustomThread1 (Shared shared, String string) {
        super(string);
        this.shared = shared;
        start(); }
    public void run() {
        System.out.println("The result is " + shared.getResult()); }
}
class CustomThread2 extends Thread {
    Shared shared;
    public CustomThread2 (Shared shared, String string) {
        super(string);
        this.shared = shared;
        start(); }
    public void run() { shared.dowork(); }
}
class Waiting {
    public static void main(String args[]) {
        Shared shared = new Shared();
        CustomThread1 thread1 = new CustomThread1(shared, "one");
        CustomThread2 thread2 = new CustomThread2(shared, "two"); }
}

```

Here's the output of this code. Note that the reader thread did wait until the writer thread was through:

```

E:\Java8>java Waiting
The result is 1

```

Suspending and Resuming Threads

In the early days of Java, threads supported both a `suspend()` method and a `resume()` method, which you could use to temporarily halt and start a thread again. Unfortunately, like the `stop()` method, both `suspend()` and `resume()` methods have been deprecated. In this case, these methods are deprecated because they are deadlock-prone and can block access to other threads. However, you can create your own `suspend()` and `resume` methods, and we'll show how to do that here.

Note that when sub-classing the `Thread` class, you can't call your new "suspend" and "resume" methods `suspend()` and `resume()`, because Java will complain that you are trying to override deprecated methods. Therefore, in this example, we'll call these methods `newSuspend()` and `newResume()` (this isn't a problem when working with an object that implements the `Runnable` interface instead). As is the case when you implement a new `stop()` method (see the solution "Setting Thread Priority and Stopping Threads" earlier in this chapter), these methods work by setting a flag that's checked in the `run()` method. When you set that flag to true in the `newSuspend()` method, you use the `wait()` method in the `run` method to suspend the thread, and you wake the thread up again with the `notify()` method in the `newResume()` method. Here's how this looks in code:

```

class CustomThread extends Thread {
    volatile boolean goFlag = true;
    CustomThread(String name) {
        super(name);
        start(); }
    public void run() {
        try {
            for(int loop_index = 0; loop_index <=3; loop_index++) {
                System.out.println(Thread.currentThread().getName()
                    + "here...");
                Thread.sleep(500);
                synchronized(this) {
                    while(!goFlag) { wait(); }
                }
            }
        }
        catch (InterruptedException e) { }
    }
}

```

```

    }
    public void newSuspend() { goFlag = false; }
    synchronized public void newResume() {
        goFlag = true;
        notify(); } }
class SuspendDemo {
    public static void main(String args[]) {
        CustomThread thread1 = new CustomThread("one");
        CustomThread thread2 = new CustomThread("two");
        try {
            Thread.sleep(1000);
            System.out.println("Suspending thread one...");
            thread1.newSuspend();
            Thread.sleep(1000);
            System.out.println("Resuming thread one...");
            thread1.newResume(); }
        catch (InterruptedException e) { }
        try {
            thread1.join();
            thread2.join(); }
        catch (InterruptedException e) { }
    }
}

```

The output of this code is shown in Figure 8.15:

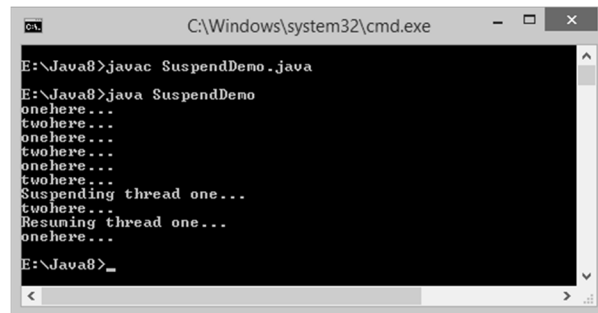


Figure 8.15: Suspending Threads

Note that threads one and two run until the code suspends thread one; then, when the code starts thread one again, they alternate again.

Even though the methods `suspend()` and `resume()` seem to be having a better approach, they should not be used. The function `suspend()` was deprecated by Java 2 because it could cause serious system failures. The `resume()` method was deprecated by Java 2 too. This was because it cannot be used without the `suspend()` method. The best way would be that we design a `run()` method that would periodically check whether the thread should suspend or resume. This can be accomplished using a `counter` variable. As long as this counter is set to running, the `run()` method must continue to execute. If this variable is set to suspend, the thread must pause. If it is set to stop, the thread must terminate:

```

class NewThread implements Runnable
{
    String SR;
    Thread h;
    boolean counter;
    NewThread(String threadname) {
        SR = threadname;
        h = new Thread(this, SR);
        System.out.println("Begin thread: " + h);
        counter = false;
        h.start(); }
}

```

```

public void run() {
    try {
        for(int i = 30; i > 15; i--) {
            System.out.println(SR + ": " + i);
            Thread.sleep(200);
            synchronized(this) {
                while(counter) { wait(); }
            }
        }
    }
    catch (InterruptedException e)
    {
        System.out.println(SR + " Suspended..."); }
        System.out.println(SR + " Ending..."); }
        void mysuspend() { counter = true; }
        synchronized void myresume()
        {
            counter = false;
            notify();
        }
    }
}

class SuspendResume {
    public static void main(String args[]) {
        NewThread ob1 = new NewThread("One");
        NewThread ob2 = new NewThread("Two");
        try {
            Thread.sleep(1000);
            ob1.mysuspend();
            System.out.println("Suspending One...");
            Thread.sleep(1000);
            ob1.myresume();
            System.out.println("Resuming One...");
            ob2.mysuspend();
            System.out.println("Suspending Two...");
            Thread.sleep(1000);
            ob2.myresume();
            System.out.println("Resuming Two..."); }
        catch (InterruptedException e) {
            System.out.println("Main thread Suspended..."); }
        try {
            System.out.println("Waiting for threads to finish.");
            ob1.h.join();
            ob2.h.join(); }
        catch (InterruptedException e) {
            System.out.println("Main thread Suspended"); }
        System.out.println("Main thread exiting..."); }
    }
}

```

The result is shown in Figure 8.16:

```

C:\Windows\system32\cmd.exe
E:\Java8>javac SuspendResume.java
E:\Java8>java SuspendResume
Begin thread: Thread[One,5,main]
Begin thread: Thread[Two,5,main]
One: 30
Two: 30
One: 29
Two: 29
Two: 28
One: 28
Two: 27
One: 27
Two: 26
One: 26
Suspending One...

```

Figure 8.16: Resuming Threads

Creating Graphics Animation with Threads

Here's an example showing how graphics animation by using threads can work. In this applet, we load four images into an image array and then we cycle through them in the `paint()` method, making Java call that method repeatedly by calling `repaint()` in a new thread.

There's one interesting point here—although the `Thread` class's `stop()` method has been deprecated, Web browsers and the Oracle appletviewer still call this method when the page containing the applet is no longer the current page. To stop the applet's animation thread so that it doesn't keep taking up system resources until the browser decides to unload it, you can place code in the `stop()` method yourself—but that means you can't base your code on the `Thread` class, because `stop()` has been deprecated there. Therefore, we'll write an example by using the `Runnable` interface, which doesn't have a deprecated `stop()` method. Here's the code:

```
import java.awt.*;
import java.applet.Applet;
/*<APPLET
  CODE = whirl.class
  WIDTH = 300
  HEIGHT = 300>
</APPLET>*/
public class Whirl extends Applet implements Runnable {
    Image whirlImages[] = new Image[4];
    Image nowImage;
    int whirlIndex = 0;
    Thread whirlThread;
    boolean animateFlag = true;
    public void init() {
        whirlImages[0] = getImage(getCodeBase(), "whirl1.gif");
        whirlImages[1] = getImage(getCodeBase(), "whirl2.gif");
        whirlImages[2] = getImage(getCodeBase(), "whirl3.gif");
        whirlImages[3] = getImage(getCodeBase(), "whirl4.gif");
    }
    public void start() {
        whirlThread = new Thread(this);
        whirlThread.start();
    }
    public void stop() { animateFlag = false; }
    public void run() {
        while(animateFlag) {
            nowImage = whirlImages[whirlIndex++];
            if(whirlIndex > 3) whirlIndex = 0;
            repaint();
            try {Thread.sleep(200);}
            catch (InterruptedException e) { }
        }
    }
    public void paint (Graphics g) {
        if(nowImage != null)
            g.drawImage(nowImage, 10, 10, this);
    }
}
```

You can see the result of this code in Figure 8.17:

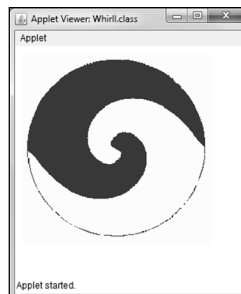


Figure 8.17: Displaying Graphics Animation

This applet makes the image you see in that figure. It appears to whirl around.

We'll create another example that would create a ball that travels across a component—starting from the left side of the component and moving toward the right:

```
import java.applet.*;
import java.awt.*;
import javax.swing.*;
import java.awt.geom.*;
/*<APPLET
  CODE = MoveBall.class
  WIDTH = 300
  HEIGHT = 300>
</APPLET>*/
public class MoveBall extends JApplet {
    private final int WIDTH = 350;
    private final int HEIGHT = 300;
    private PaintSurface canvas;
    public void init() {
        this.setSize(WIDTH, HEIGHT);
        canvas = new PaintSurface();
        this.add(canvas, BorderLayout.CENTER);
        Thread t = new AnimationThread(this);
        t.start(); }
}
class AnimationThread extends Thread {
    JApplet c;
    public AnimationThread(JApplet c) { this.c = c; }
    public void run() {
        while (true) {
            c.repaint();
            try {Thread.sleep(30);}
            catch (InterruptedException ex) { }
        }
    }
}
class PaintSurface extends JComponent {
    int x_pos = 20;
    int y_pos = 50;
    int d = 20;
    public void paint(Graphics g) {
        Graphics2D g2 = (Graphics2D)g;
        g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
            RenderingHints.VALUE_ANTIALIAS_ON);
        x_pos += 4;
        Shape ball = new Ellipse2D.Float(
            x_pos, y_pos, d, d);
        g2.setColor(Color.YELLOW);
        g2.fill(ball); }
}
```


You can see the result of this code in Figure 8.18, where the applet moves the ball across—from left to right:

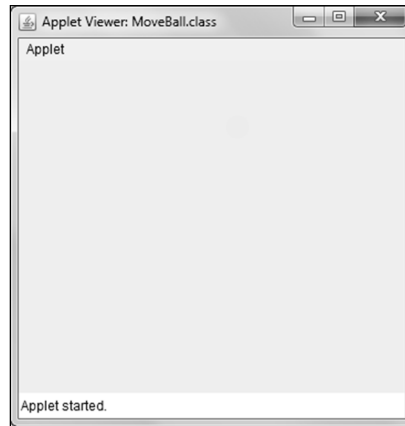


Figure 8.18: Ball Moving across the Applet

However, there's one problem in both these programs. Here, we have implemented these examples as the `JApplet` class, and each time the image is redrawn, the background behind the image is also redrawn, so the animation flickers a lot. Moreover, everything is drawn straight to the screen. To fix this problem, see the next solution.

Eliminating Flicker in Graphics Animation Created Using Threads

In AWT programming, when you redraw an image, its background is redrawn first, which can make for a lot of flickering in graphics animation. The way to fix this is to override the `update()` method, which does the redrawing. When you override `update()`, the image's background is not redrawn. In fact, you can do more—you can make the `paint()` method only redraw the area covered by the image you want to redraw by using the `Graphics` class's `clipRect()` method, like this:

```
import java.awt.*;
import java.applet.Applet;
/*<APPLET
  CODE = whirl2.class
  WIDTH = 300
  HEIGHT = 300 >
</APPLET>*/
public class Whirl2 extends Applet implements Runnable {
    Image whirlImages[] = new Image[4];
    Image nowImage;
    int whirlIndex = 0;
    Thread whirlThread;
    boolean animateFlag = true;
    public void init() {
        whirlImages[0] = getImage(getCodeBase(), "whirl1.gif");
        whirlImages[1] = getImage(getCodeBase(), "whirl2.gif");
        whirlImages[2] = getImage(getCodeBase(), "whirl3.gif");
        whirlImages[3] = getImage(getCodeBase(), "whirl4.gif");
    }
    public void start()
    {
        whirlThread = new Thread(this);
        whirlThread.start();
    }
    public void stop() { animateFlag = false; }
    public void run()
    {
        while(animateFlag)
        {
```

```

        nowImage = whirlImages[whirlIndex++];
        if(whirlIndex > 3)whirlIndex = 0;
        repaint();
        try { Thread.sleep(200); }
        catch(InterruptedException e) { }
    }
}
public void paint (Graphics g)
{
    if(nowImage != null) g.drawImage(nowImage, 10, 10, this);
}
public void update(Graphics g) {
    g.clearRect(100, 100, 280, 280);
    paint(g); }
}

```

Now, the image shown in Figure 8.19 will rotate practically flicker-free:

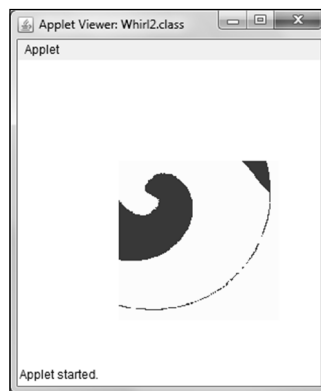


Figure 8.19: Rotation of the Image

Suspending and Resuming Graphics Animation

Here's an example that adds a button to suspend and resume the animation developed over the previous two solutions. This puts to work the thread suspend and resume techniques (that is, using a boolean flag and the `wait()` and `notify()` methods) discussed in the solution "Suspending and Resuming Threads," earlier in this chapter:

```

import java.awt.*;
import java.awt.event.*;
import java.applet.Applet;
/*<APPLET
    CODE = whirl3.class
    WIDTH = 300
    HEIGHT = 300 >
</APPLET>*/
public class Whirl3 extends Applet implements ActionListener, Runnable {
    Image whirlImages[] = new Image[4];
    Image nowImage;
    int whirlIndex = 0;
    Thread whirlThread;
    Button suspendButton, resumeButton;
    boolean animateFlag = true;
    boolean goFlag = true;
    public void init()
    {
        whirlImages[0] = getImage(getCodeBase(), "whirl1.gif");
        whirlImages[1] = getImage(getCodeBase(), "whirl2.gif");
        whirlImages[2] = getImage(getCodeBase(), "whirl3.gif");
        whirlImages[3] = getImage(getCodeBase(), "whirl4.gif");
    }
}

```

```

suspendButton = new Button("Suspend");
add(suspendButton);
suspendButton.addActionListener(this);
resumeButton = new Button("Resume");
add(resumeButton);
resumeButton.addActionListener(this);
}
public synchronized void actionPerformed(ActionEvent e)
{
    if(e.getSource() == suspendButton){goFlag = false;}
    if(e.getSource() == resumeButton) {
        goFlag = true;
        notify();
    }
}
public void start()
{
    whirlThread = new Thread(this);
    whirlThread.start(); }
public void stop() { animateFlag = false; }
public void run() {
    while(animateFlag) {
        nowImage = whirlImages[whirlIndex++];
        if(whirlIndex > 3)whirlIndex = 0;
        repaint();
        try {
            Thread.sleep(200);
            synchronized(this) {
                while(!goFlag)
                    wait(); }
        }
        catch(InterruptedException e) { }
    }
}
public void paint (Graphics g)
{
    if(nowImage != null) g.drawImage(nowImage, 10, 10, this);
}
public void update(Graphics g)
{
    g.clearRect(150,100,200,280);
    paint(g);
}
}

```

The result of this code is shown in Figure 8.20:

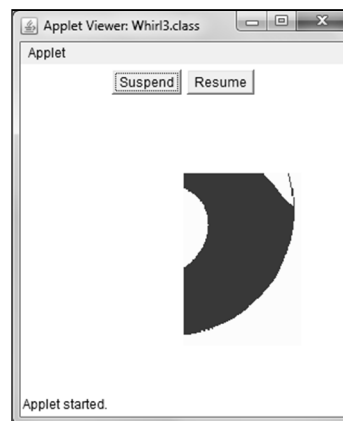


Figure 8.20: Displaying Graphics Animation with the Suspend and Resume Buttons

Using Double Buffering

Double buffering is the process of creating images off-screen and flashing them on screen as required, which means you don't have to create the image while the user watches. You can create a graphics buffer by creating an `Image` object and then using that object's `getGraphics()` method to get a `Graphics` object for the image that you can use to draw in. Here's an example showing how double buffering works. This example draws a succession of boxes off-screen and then displays them, thus creating an animation:

```
import java.awt.*;
import java.applet.Applet;
/*<APPLET
    CODE=DBuffer.class
    WIDTH=200
    HEIGHT=200>
</APPLET>*/
public class DBuffer extends Applet implements Runnable
{
    Image image1;
    Thread thread1;
    Graphics graphics;
    int loop_index = 0;
    boolean goFlag = true;
    public void init()
    {
        image1 = createImage(100, 100);
        graphics = image1.getGraphics();
    }
    public void start() {
        thread1 = new Thread(this);
        thread1.start();
    }
    public void stop() { goFlag = false; }
    public void run()
    {
        while(goFlag)
        {
            repaint();
            try { Thread.sleep(100); }
            catch(InterruptedException e) { }
        }
    }
    public void paint (Graphics g) {
        loop_index += 5;
        if(loop_index >= 100) loop_index = 5;
        graphics.setColor(new Color(255, 255, 255));
        graphics.fillRect(0, 0, 50, 50);
        graphics.setColor(new Color(0, 255, 255));
        graphics.drawRect(0, 0, loop_index, loop_index);
        g.drawImage(image1, 10, 10, this); }
}
```

The result of the code is shown in Figure 8.21:

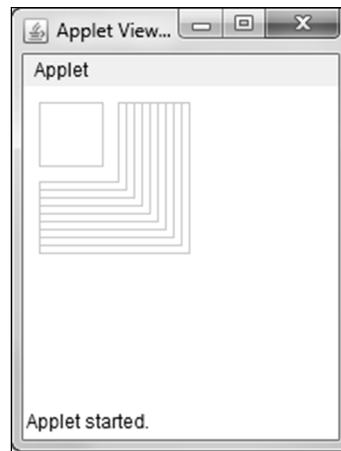


Figure 8.21: An Applet Showing a Succession of Boxes

Let us take a look at another code that will use a technique called double buffering, which will produce smooth, flicker-free animation. This technique first draws an off-screen image, and when everything is done, the off-screen image is drawn on the screen, thus reducing nasty flickering:

```
import java.applet.*;
import java.awt.event.*;
import java.awt.*;
/* <APPLET
   CODE = DoubleBuffering.class
   WIDTH = 300
   HEIGHT = 300>
</APPLET>*/
public class DoubleBuffering extends Applet implements MouseMotionListener {
    Graphics bufferGraphics;
    Image offscreen;
    Dimension dim;
    int curX, curY;
    public void init() {
        dim = getSize();
        addMouseMotionListener(this);
        setBackground(Color.white);
        offscreen = createImage(dim.width,dim.height);
        bufferGraphics = offscreen.getGraphics(); }
    public void paint(Graphics g) {
        bufferGraphics.clearRect(0,0,dim.width,dim.height);
        bufferGraphics.setColor(Color.gray);
        bufferGraphics.drawString("Double-buffered for you",2,10);
        bufferGraphics.fillRect(curX,curY,20,20);
        g.drawImage(offscreen,0,0,this); }
    public void update(Graphics g) { paint(g); }
    public void mouseMoved(MouseEvent evt) {
        curX = evt.getX();
        curY = evt.getY();
        repaint(); }
    public void mouseDragged(MouseEvent evt) { }
```

Update is called automatically when `repaint()` is called. The `update()` method is always required for good double buffering. By this, the applet immediately starts to repaint rather than wiping the previous drawings off first. Flickering is caused by the wiping-off action. The result of the code is shown in Figure 8.22:

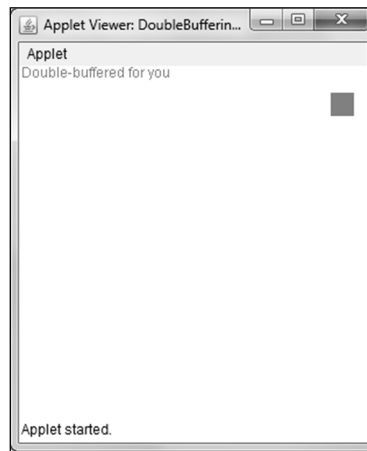


Figure 8.22: Applet Showing Flicker-Free Drawing

Simplifying Producer-Consumer with the **Queue** Interface

Until Java 5, we were rolling our own Queues. Now the `util` package comes with a new interface, `java.util.Queue`, with various implementations (including a retrofitted `LinkedList`). There are various queues, including Priority Queues, Blocking Queues, Delay Queues, and Synchronous Queues. We are here dealing with simplification of producer-consumer with the `java.util.Queue` interface.

One of the fundamental data structures in Computer Science is the queue. You'll recall that a queue is a data structure in which elements are removed in the same order in which they were added. Unfortunately, First in First Out (FIFO) data structure was ignored in the earlier versions of Java. After Java 1.5, queue support has become innate for the first time.

Queues are typically restricted to a given size.

For enqueueing and dequeuing via the `offer` or `add` methods and the `remove` or `poll` methods respectively, if the queue fails to provide enqueueing or dequeuing operation, you need not wait for the execution of the program. The `java.util.concurrent.BlockingQueue` interface adds the `put()` and `take()` methods to the collection. `BlockingQueue` executions are designed to be used for producer-consumer queues. The relationship of producer/consumer can be understood as the producer writes to a queue (more specifically a `BlockingQueue`), just to suppose. There are a number of consumers reading from that queue, and you want all that to happen in a sequential manner. It implies that each consumer needs to wait for the previous consumer before it is allowed to grab an item from the queue. To create such a setup using code, you must spawn one producer thread that would simply write to a queue and also spawn several consumer threads that read from the same thread. It is to be taken into consideration that the threads will block one another until the current thread will grab the item from the queue.

The code here shows the class `Producer` writing to a `BlockingQueue`. Notice that objects in the `run()` method (which you are obligated to implement, since you extended `Thread`) are put onto the `BlockingQueue` after waiting for a random amount of time (ranging from 100 – 500 milliseconds). The object put onto the queue is simply a `String` object consisting of the timestamp when the message was produced.

The actual work of enqueueing the object is done by the statement:

```
blockingQueue.put("Enqueued at: " + time)
```

The `put()` method throws an `InterruptedException`. Accordingly, the `put()` operation catches the operation thrown, which is surrounded by a `try` and a `catch` block. The messages that are thrown from the producer belong to the `Consumer` object, which extends from the `Thread` object and, therefore, must implement the `run()` method.

The `Consumer` class is very similar in design to the `Producer` class. The `Consumer` class makes use of the `take()` method to dequeue messages from the queue rather than putting messages on the `BlockingQueue`. As already stated, this occurs because the thread waits until something is actually on the queue. If, onto the queue, queuing of objects is stopped by the `Producer` thread, the `Consumer` thread will have to wait till there is an item available in the queue. The following code illustrates the `TestBlockingQueue` class that spawns four consumer threads, which try and grab objects from the `BlockingQueue`:

```
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.LinkedBlockingQueue;
import java.util.Date;
import java.util.Random;
import java.io.PrintStream;
class Producer extends Thread
{
    private BlockingQueue<String> blockQ;
    private PrintStream prnStr;
    public Producer(BlockingQueue<String> blockQ, PrintStream prnStr)
    {
        this.prnStr = prnStr;
        this.blockQ = blockQ;
    }
    public void run()
    {
        try {
            while(true)
            {
                String time = String.format("%tc",new Date());
                prnStr.println("Putting item on BlockingQueue
at time: " + time);
                blockQ.put("Enqueued at: " + time);
                Random generate = new Random();
                int randomSlp = generate.nextInt(5) + 1;
                Thread.sleep(randomSlp * 100);
            }
        } catch (InterruptedException e) { e.printStackTrace(); }
    }
}
class Consumer extends Thread
{
    private BlockingQueue blockQ;
    private String consumName;
    private PrintStream prnStr;
    public Consumer(String consumName, BlockingQueue blockQ,
    PrintStream prnStr)
    {
        consumName = this.consumName;
        blockQ = this.blockQ; prnStr = this.prnStr; }
    public void run()
    {
        while (true) {
            try {
                prnStr.println(this.consumName + blockQ.take());
            }
            catch (InterruptedException ie) { ie.printStackTrace(); }
        }
    }
}
public class TestBlockingQueue
{
    public static void main(String args[])
    {
        BlockingQueue<String> blkQue = new LinkedBlockingQueue<String>();
        Producer prod = new Producer(blkQue, System.out);
        Consumer consP = new Consumer("ConsumerP", blkQue, System.out);
```

```

        Consumer consQ = new Consumer("ConsumerQ", blkQue, System.out);
        Consumer consR = new Consumer("ConsumerR", blkQue, System.out);
        Consumer consS = new Consumer("Consumers", blkQue, System.out);
        prod.start();
        consP.start();
        consQ.start();
        consR.start();
        consS.start();
    }
}

```

The following line creates the `BlockingQueue`:

```
BlockingQueue<String> blkQue = new LinkedBlockingQueue<String>();
```

And it shows how it deploys the `LinkedBlockingQueue` implementation of the `BlockingQueue`. As the `BlockingQueue` is an abstract class, it is not possible to directly instantiate it. Another option could be to use the queue type, `ArrayBlockingQueue`. The advantage of using it is that it uses an array to store data, whereas the `LinkedBlockingQueue` class uses a linked list. An `ArrayBlockingQueue` has fixed size. However, in case of a `LinkedBlockingQueue`, you can specify the maximum size, which is by default unbounded.

During the execution of the class, the dequeuing of objects from the queue happens in a sequential order, as shown in Figure 8.23:

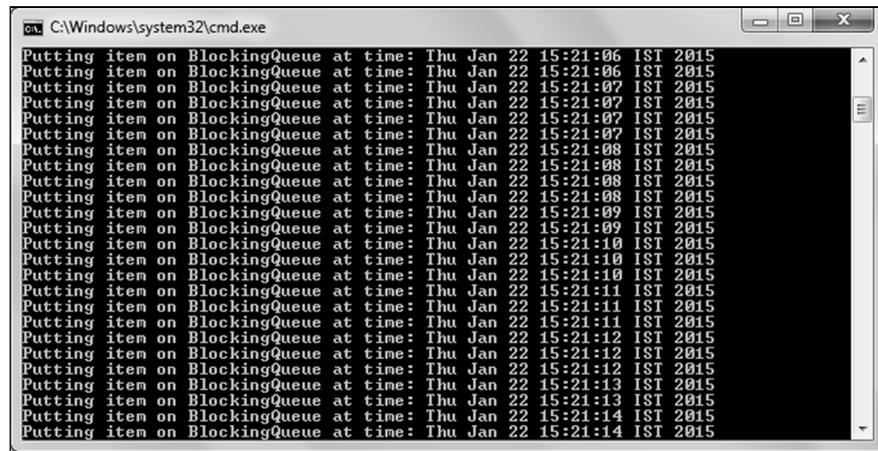


Figure 8.23: Consumer Threads

Further, other consumer threads are blocked by one consumer thread from accessing the `BlockingQueue` till the time it is able to dequeue an object from the queue. Messages are dequeued from the `BlockingQueue` by these threads in the same order as you spawned them.

Implementing Concurrent Programming

There is a built-in language support for threads in Java, and they are used to handle programs in an easy manner. Some of the built-in primitives, such as `synchronized` blocks, `Object.wait()`, and `Object.notify()`, are not enough to perform many programming tasks.

With J2SE 5, a new path to multithreading was provided in the Java programming language. The real processes for coordinating threads with `wait()` and `notify()` are now elaborated with new and sophisticated mechanisms for working with threads. Here, we will focus on concurrency mechanisms that are included in Java SE 7.0. The new packages are now a part of the `java.util.concurrent` package.

Limitations of Synchronization Previous to J2SE 5

While creating multithreaded applications, one of the most difficult problems faced was related to data synchronization. Such errors make design harder and are usually difficult to detect. Built-in synchronization has the following limitations:

- ❑ No option to withdraw an operation of trying to acquire a lock that is already held or to give up after waiting for a specific period of time or to cancel an attempt to lock after an interrupt
- ❑ No option provided to alter the semantics of a lock, such as, with regard to re-entrancy, read as opposed to write protection, or fairness
- ❑ No access control for synchronization. Any method can perform synchronized (*res*) for any accessible resource
- ❑ Synchronization is possible only inside methods and blocks. It implies that a lock cannot be acquired in one method and released in another

These problems can be overcome with the help of the utility classes that control locking, *mutex* for instance, which is another term used for a lock. However, a *mutex* never nests the way synchronization methods or blocks do, as shown here:

```
public class Mutex
{
    public void acq() throws InterruptedException { }
    public void rel() { }
    public boolean attempt(long msec) throws InterruptedException { }
}
```

and it can be used as:

```
try { mutex.acq();
    try {
        // ...
    }
    finally { mutex.rel();
    }
} catch(InterruptedException ie) { }
```

Developers can use the standardized classes that provide building blocks for creating various applications.

Simplifying Servers Using the Concurrency Utilities

Similar to Java SE 7, Java SE 8 too comes with support for multiprocessing in the Java programming language. Originally, threads were coordinated with the help of methods, such as *wait()* and *notify()*. However, they are now enhanced by using advanced mechanisms to work with threads. The *java.util.concurrent* package offers advanced mechanisms that provide several concurrency utilities. These utilities enable the process of developing multithreaded applications and servers easier.

In addition, such standards will improve the quality of such applications. The Java Community Process has defined the package as JSR 166: Concurrent Utilities. The primary goal of the JSR 166 specification is to standardize a framework that brings together commonly used utilities for concurrent programming into a small package, which developers can learn easily.

Its other goals are to facilitate high-quality implementations containing classes and interfaces for atomic variables, locks, barriers, semaphores and condition variables, queues and associated collections created for multithreaded use, and thread pools and a custom execution framework.

Knowing Various Concurrency Utilities

As you have just read, the *java.util.concurrent* package contains classes and interfaces that are used to simplify the process of developing concurrent classes and applications. This package facilitates high-quality implementations of common building blocks that can be used to create concurrent applications. The package contains classes, enhanced to facilitate concurrent access, including:

- ❑ **Task scheduling framework**—The *Executor* framework aims at standardizing tasks such as invocation, scheduling, execution, and control of asynchronous tasks as per execution policies. The processes are permitted to run in a newly created thread or in a pool of threads or in a single background thread. Further, the built-in implementations enable policies that can be configured such as queue length limits and saturation policy. These policies will help in improving the accuracy of applications by avoiding runaway source consumption.

- ❑ **Concurrent collections**—Several collection classes have been introduced, which include `Queue` and `BlockingQueue` interfaces, and high-quality concurrent implementations of `Map`, `List`, and `Queue` as well.
- ❑ **Atomic variables**—The `java.util.concurrent.atomic` package facilitates developers to automatically calculate values that can be of primitive types or references. The implementation of this package offers high quality performance that can be available through synchronization. This package helps in using high-performance concurrent algorithms, counters, and sequence number generators.
- ❑ **Synchronizers**—Synchronization classes that facilitate coordination between threads have been added including semaphores, mutexes, barriers, latches, and exchangers.
- ❑ **Locks**—You can use the `synchronized` keyword for locking purposes. However, as already stated, there are restrictions with this built-in monitor. The `java.util.concurrent.locks` facilitates a high-quality lock implementation and also enables developers to provide a timeout when trying to acquire a lock, multiple condition variables per lock, non-nested scoped locks.
- ❑ **Nanosecond-granularity timing**—The introduction of the new `java.lang.System.nanoTime()` method facilitates high-precision timing facilities. This method provides access to nanosecond-granularity time source to make relative time measurements and methods, and accept timeouts such as `BlockingQueue.offer()`, `BlockingQueue.poll()`, `Lock.tryLock()`, `Condition.await()`, and `Thread.sleep()`. It should be noted that the precision of `nanotime()` is platform-dependent.

The following are the benefits of using the new `java.util.concurrent` package:

- ❑ **Reduced programming effort**—Developers need not have to write new code as they can easily use a standard class.
- ❑ **Increased performance**—The implementations are much faster and more scalable as compared to typical implementations because of the fact that they have been developed and tested by concurrency and performance experts.
- ❑ **Increased reliability**—While using low-level concurrency primitives, such as `synchronized wait()` and `notify()`, it becomes challenging to correctly use these primitives. Further, errors are also difficult to detect or debug. On the contrary, the concurrency utilities are standardized and comprehensively tested for deadlock, starvation, or race conditions.
- ❑ **Improved maintainability**—As applications are based on standardized packages, they become easier to maintain.
- ❑ **Increased productivity**—Developers can easily understand the standard library classes and, hence, need not have to learn new APIs.

Learning about the `java.util.concurrent` Package

The `java.util.concurrent` package has already been discussed in the preceding topic. In this section, we discuss the interfaces classes in the package. The interfaces and their description are mentioned in Table 8.6 and the classes in Table 8.7:

Table 8.6: Interfaces of the concurrent package	
Interface	Does this
<code>BlockingDeque<E></code>	It is deque that additionally supports blocking operations that wait for the deque to become non-empty when retrieving an element and wait for space to become available in the deque when storing an element
<code>BlockingQueue<E></code>	It is queue that additionally supports operations that wait for the queue to become non-empty when retrieving an element, and wait for space to become available in the queue when storing an element
<code>Callable<V></code>	It is a task that returns a result and may throw an exception
<code>CompletionService<V></code>	It is the service that decouples the production of new asynchronous tasks from the consumption of the results of completed tasks
<code>ConcurrentMap<K, V></code>	It creates a <code>Map</code> providing additional atomic <code>putIfAbsent()</code> ,

Table 8.6: Interfaces of the concurrent package

Interface	Does this
	<code>remove()</code> , and <code>replace()</code> methods
<code>ConcurrentNavigableMap<K, V></code>	A <code>ConcurrentMap</code> supporting <code>NavigableMap</code> operations
<code>Delayed</code>	The mix-in style interface for marking objects that should be acted upon after a given delay
<code>Executor</code>	An object that executes submitted <code>Runnable</code> tasks
<code>ExecutorService</code>	It is the executor that provides methods to manage termination and methods that can produce a <code>Future</code> for tracking progress of one or more asynchronous tasks
<code>Future<V></code>	The <code>Future</code> represents the result of an asynchronous computation
<code>RejectedExecutionHandler</code>	The handler for tasks that cannot be executed by a <code>ThreadPoolExecutor</code>
<code>RunnableFuture<V></code>	A <code>Future</code> that is <code>Runnable</code>
<code>RunnableScheduledFuture<V></code>	A <code>ScheduledFuture</code> that is <code>Runnable</code>
<code>ScheduledExecutorService</code>	An <code>ExecutorService</code> that can schedule commands to run after a given delay, or to execute periodically
<code>ScheduledFuture<V></code>	A delayed result-bearing action that can be cancelled
<code>ThreadFactory</code>	An object that creates new threads on demand

Table 8.7: Classes of the concurrent package

Class	Does this
<code>AbstractExecutorService</code>	It provides default implementation of <code>ExecutorService</code> execution methods
<code>ArrayBlockingQueue<E></code>	A bounded blocking queue backed by an array
<code>ConcurrentHashMap<K, V></code>	A hash table supporting full concurrency of retrievals and adjustable expected concurrency for updates
<code>ConcurrentLinkedQueue<E></code>	An unbounded thread-safe queue based on linked nodes
<code>ConcurrentSkipListMap<K, V></code>	A <code>ConcurrentNavigableMap</code> implementation which is scalable
<code>ConcurrentSkipListSet<E></code>	A concurrent <code>NavigableSet</code> implementation based on <code>ConcurrentSkipListMap</code> which is scalable
<code>CopyOnWriteArrayList<E></code>	A thread-safe variant of <code>ArrayList</code> in which all mutative operations (add, set, and so on) are implemented by making a fresh copy of the underlying array.
<code>CopyOnWriteArraySet<E></code>	A set that uses <code>CopyOnWriteArrayList</code> for all of its operations
<code>CountDownLatch</code>	A synchronization aid that allows one or more threads to wait until a set of operations being performed in other threads completes
<code>CyclicBarrier</code>	A synchronization aid that allows a set of threads to all wait for each other to reach a common barrier point
<code>DelayQueue<E extends Delayed></code>	An unbounded blocking queue of <code>Delayed</code> elements, in which an element can only be taken when its delay has expired
<code>Exchanger<V></code>	A synchronization point at which two threads can exchange objects
<code>ExecutorCompletionService<V></code>	A <code>CompletionService</code> that uses a supplied <code>Executor</code> to execute tasks
<code>Executors</code>	Factory and utility methods for <code>Executor</code> , <code>ExecutorService</code> , <code>ScheduledExecutorService</code> , <code>ThreadFactory</code> , and <code>Callable</code> classes defined in this package
<code>FutureTask<V></code>	A cancellable asynchronous computation

Table 8.7: Classes of the concurrent package

Class	Does this
<code>LinkedBlockingDeque<E></code>	A blocking deque based on linked nodes which is optionally bounded
<code>LinkedBlockingQueue<E></code>	An optionally-bounded blocking queue based on linked nodes
<code>PriorityBlockingQueue<E></code>	An unbounded blocking queue that uses the same ordering rules as class <code>PriorityQueue</code> and supplies blocking retrieval operations
<code>ScheduledThreadPoolExecutor</code>	A <code>ThreadPoolExecutor</code> that can additionally schedule commands to run after a given delay, or to execute periodically
<code>Semaphore</code>	A counting semaphore
<code>SynchronousQueue<E></code>	A blocking queue in which each put must wait for a take, and vice versa
<code>ThreadPoolExecutor</code>	An <code>ExecutorService</code> that executes each submitted task using one of possibly several pooled threads, normally configured using <code>Executors</code> factory methods
<code>ThreadPoolExecutor.AbortPolicy</code>	A handler for rejected tasks that throws a <code>RejectedExecutionException</code>
<code>ThreadPoolExecutor.CallerRunsPolicy</code>	A handler for rejected tasks that runs the rejected task directly in the calling thread of the <code>execute</code> method, unless the executor has been shut down, in which case the task is discarded
<code>ThreadPoolExecutor.DiscardOldestPolicy</code>	A handler for rejected tasks that discards the oldest unhandled request and then retries <code>execute</code> , unless the executor is shut down, in which case the task is discarded
<code>ThreadPoolExecutor.DiscardPolicy</code>	A handler for rejected tasks that silently discards the rejected task

Learning about the `java.util.concurrent.locks` Package

The `java.util.concurrent.locks` provides classes for locking and waiting for conditions that are distinct from built-in synchronization and monitors. Even if this framework has awkward syntax, it brings greater flexibility when using locks and conditions. The interfaces and classes of this package are mentioned in the tables that follow. This package provides reader/writer locks.

Reader/Writer Locks

When using a thread to read data from an object, you do not need to prevent another thread from reading data at the same time. As long as threads are just reading the data and not making any modifications to it, there is absolutely no reason to lock threads. The `java.util.concurrent.locks` package provides classes that implement this type of locking.

The `ReadWriteLock` interface provides locks for read-only and for writing. Multiple reader threads can use `readLock()` simultaneously till the time there are no writers. The `writeLock()` is exclusive. Using reader/writer locks increases concurrency and results in better performance, rather than using a mutual exclusion lock. However, such an improvement in the performance can be benefitted only on a multi-processor.

The interfaces and their description are mentioned in Table 8.8 and the classes in Table 8.9:

Table 8.8: Interfaces of the `java.util.concurrent.locks` package

Interface	Does this
<code>Condition</code>	Condition factors out the <code>Object</code> monitor methods (<code>wait</code> , <code>notify</code> and <code>notifyAll</code>) into distinct objects to give the effect of having multiple wait-sets per object, by combining them with the use of arbitrary <code>Lock</code> implementations.
<code>Lock</code>	<code>Lock</code> implementations provide more extensive locking operations than can be obtained using <code>synchronized</code> methods and statements.

Table 8.8: Interfaces of the `java.util.concurrent.locks` package

Interface	Does this
<code>ReadWriteLock</code>	A <code>ReadWriteLock</code> maintains a pair of associated locks, one for read-only operations and one for writing.

Table 8.9: Classes of `java.util.concurrent.locks` package

Class	Does this
<code>AbstractOwnableSynchronizer</code>	A synchronizer which may be exclusively owned by a thread
<code>AbstractQueuedLongSynchronizer</code>	An <code>AbstractQueuedSynchronizer</code> maintaining synchronization state as long.
<code>AbstractQueuedSynchronizer</code>	It provides a framework for implementing blocking locks and related synchronizers (semaphores, events, etc.) that rely on FIFO wait queues.
<code>LockSupport</code>	It is a basic thread blocking primitives for creating locks and other synchronization classes.
<code>ReentrantLock</code>	A re-entrant mutual exclusion Lock with the same basic behavior and semantics as the implicit monitor lock accessed using <code>synchronized</code> methods and statements, but with extended capabilities
<code>ReentrantReadWriteLock</code>	An implementation of <code>ReadWriteLock</code> supporting similar semantics to <code>ReentrantLock</code>
<code>ReentrantReadWriteLock.ReadLock</code>	The lock returned by method <code>ReentrantReadWriteLock.readLock()</code>
<code>ReentrantReadWriteLock.WriteLock</code>	The lock returned by method <code>ReentrantReadWriteLock.writeLock()</code>

Learning about the `java.util.concurrent.atomic` Package

The `java.util.concurrent.atomic` package provides classes that support lock-free thread-safe programming on single variables. The classes here extend the notion of volatile values, fields, and array elements. The classes of this package are listed in Table 8.10:

Table 8.10: Classes of the `java.util.concurrent.atomic` package

Class	Does this
<code>AtomicBoolean</code>	The boolean value that may be updated atomically
<code>AtomicInteger</code>	An int value that may be updated atomically
<code>AtomicIntegerArray</code>	An int array in which elements may be updated atomically
<code>AtomicIntegerFieldUpdater<T></code>	The reflection-based utility that enables atomic updates to designated volatile int fields of designated classes
<code>AtomicLong</code>	A long value that may be updated atomically
<code>AtomicLongArray</code>	A long array in which elements may be updated atomically
<code>AtomicLongFieldUpdater<T></code>	A reflection-based utility that enables atomic updates to designated volatile long fields of designated classes
<code>AtomicMarkableReference<V></code>	An <code>AtomicMarkableReference</code> maintains an object reference along with a mark bit, that can be updated atomically
<code>AtomicReference<V></code>	An object reference that may be updated atomically
<code>AtomicReferenceArray<E></code>	An array of object references in which elements may be updated atomically
<code>AtomicReferenceFieldUpdater<T,V></code>	A reflection-based utility that enables atomic updates to designated volatile reference fields of designated classes
<code>AtomicStampedReference<V></code>	An <code>AtomicStampedReference</code> maintains an object reference along with an integer “stamp”, that can be updated atomically

Multiple operations are treated as atomic if using the classes that belong to this package. For instance, you cannot use a volatile integer with the ++ operator as this operator has several instructions.

The `AtomicInteger` class contains a method using which the integer it holds can be incremented atomically without synchronization. For complicated tasks, atomic classes can be used. These tasks include writing code that needs no synchronization. The following code is a sample sequence number generator that uses the `AtomicLong` class:

```
import java.util.concurrent.atomic.*;
public class SampleSequenceGenerator
{
    private AtomicLong number = new AtomicLong(0);
    public long next()
    {
        return number.getAndIncrement();
    }
}
```

Objects belonging to the classes `AtomicBoolean`, `AtomicInteger`, `AtomicLong`, and `AtomicReference` facilitate access and updates to a single variable of the corresponding type. Furthermore, all these classes have their own utility methods for that type, e.g., atomic increment methods.

These classes also have the `getAndSet()` method that assigns a new value to the variable and returns the previous value without using any synchronization locks. Methods such as `compareAndSet()` and `weakCompareAndSet()` are conditional modifier methods that take two arguments. The first argument takes the original value when the method starts, and the second argument takes a new value to assign the data to.

The `java.util.concurrent` package facilitates concurrency utilities. There are no new features added to the classes that belong to this package; however, these classes are more efficient to use and provide more quality implementations for data synchronization techniques. This package has provided all the functionalities for concurrent programming what the collection framework has provided for data structures, thereby refraining the developer from making any new inventions that might possibly lead to incorrect implementations.

The `java.util.concurrent` package also enables you to create applications that are shorter, cleaner, more reliable, easy to read and write, and easy to maintain.

If developers are using this package, it does not mean that their task has reduced. They will still have to create applications that should be able to handle any synchronization issues.

There can be overlap between the classes. For instance, a semaphore having a single permit can be utilized to hold a lock. Therefore, prior to using a new package in an efficient and protected manner, there are several details that must be explored and clearly understood. It is always a good practice to review the packages and their classes before actually using them.

Summary

In this chapter, you have read about the `Threads`, including creating a thread and naming a thread. You have learned how to use the `Runnable` interface. Further on, there was a comprehensive discussion on `Double Buffering`, `Synchronization`, and the `java.util.concurrent` Package.



9

Working with Collections Framework

<i>If you need information on:</i>	<i>See page:</i>
The Collection Interfaces	258
The Collection Classes	258
The Map Interfaces	259
The Map Classes	259
Collections Framework Enhancements in Java SE 8	259
Using the Collection Interface	265
The Queue Interface	266
The List Interface	266
The Set Interface	267
The SortedSet Interface	268
Using the Collection Classes	268
Using the Comparator Interface	281
Using the Iterator Interface	282
Using the ListIterator Interface	282
Using the AbstractMap Class	284
Using the HashMap Class	285
Using the TreeMap Class	288
Using the Arrays Class	290
Learning the Fundamentals of Enumerations	293
The Legacy Classes and Interfaces	298
Using the Aggregate Operations	306
Using the java.util.function Package	307
Summary	310

In this chapter, we'll take a look at the collection classes in Java, which are supported by `java.util`. As you can infer from the name, the collection classes let you group elements in various ways. They collection classes also define several methods that provide easier way of working with items. These classes are important, not just for their utility but because many other Java methods use or return objects of these classes, such as the `ArrayList` and `HashMap` classes.

The original Java release did not introduce Collections within it. So Java used ad hoc classes, such as `Dictionary`, `Vector`, `Stack`, and `Properties`. The drawback was that these classes lacked a unifying scheme and were not designed to be easily extended or adapted. Collections were finally added by J2SE 1.2. Java SE 8 has significantly increased the power and streamlined the use of collections framework.

The Collection Interfaces

Here are six collection interfaces:

- ❑ **Collection**—It is the top of the collection hierarchy. It supports basic grouping of elements.
- ❑ **Deque**—It extends `Queue` to handle a double-ended queue. (Added in Java SE 6).
- ❑ **List**—It extends `Collection` to implement lists of objects.
- ❑ **Set**—It extends `Collection` to implement sets, in which all elements must be unique.
- ❑ **Map**—It does not have duplicate keys and each key should map to at least one value. This interface generally maps values with keys.
- ❑ **SortedSet**—It extends `Set` to implement a sorted set.
- ❑ **SortedMap**—It extends `Map` to implement a sorted map.
- ❑ **Queue**—It extends `Collection` to handle special type of lists.

The implementations of the interfaces are provided in Table 9.1:

Interface	Hash Table	Resizable Array	Balanced Tree	Linked List
Collection	HashSet	ArrayList	TreeSet	LinkedList
Set	HashSet		TreeSet	
SortedSet			TreeSet	
List		ArrayList		LinkedList
Map	HashMap		TreeMap	
SortedMap			TreeMap	
Queue				LinkedList
Deque				LinkedList

You'll see the details in this chapter; in particular, there's a standard set of classes that implement the collection interfaces—the collection classes.

The Collection Classes

Here are the standard collection classes that implement the collection interfaces:

- ❑ **AbstractCollection**—It implements the `Collection` interface.
- ❑ **AbstractList**—It extends `AbstractCollection` and implements the `List` interface.
- ❑ **AbstractQueue**—It extends `AbstractCollection` and implements parts of the `Queue` interface.
- ❑ **AbstractSequentialList**—It extends `AbstractList` into a sequential (not random access) list.
- ❑ **LinkedList**—It extends `AbstractSequentialList` into a linked list, where each element knows where the next element is.
- ❑ **ArrayList**—It implements a dynamic (resizable) array.

- ❑ `ArrayDeque`—It implements a dynamic double-ended queue by extending `AbstractCollection` and implementing the `Deque` interface (added by Java SE 6).
- ❑ `AbstractSet`—It extends `AbstractCollection` and implements the `Set` interface.
- ❑ `HashSet`—It extends `AbstractSet` to be used with a hash.
- ❑ `TreeSet`—It extends `AbstractSet` to implement a set stored in tree form.

Note the `HashSet` class, which implements a set using a hash internally. In Java, *hash* stores information using hashing; this converts a key to a *hashcode* that's then used to access the corresponding value. You don't deal with the hashcode yourself—it's generated internally in Java.

The Map Interfaces

You can also use maps in Java. A *map* stores data in key/value pairs, much like an array, where the indexes are, themselves, objects. Typically, keys are strings, and you can look up an object in a map by using that object. Here are the map interfaces:

- ❑ `Map`—It implements a map.
- ❑ `Map.Entry`—The inner class of `Map` that describes a key/value pair in a map.
- ❑ `NavigableMap`—It extends `SortedMap` to handle the retrieval of entries based on closest match searches (added by Java SE 6).
- ❑ `SortedMap`—It extends `Map` to keep the keys in ascending order.

Java also derives some standard classes from these interfaces—the map classes.

The Map Classes

Here are the standard map classes defined by Java:

- ❑ `AbstractMap`—It implements the `Map` interface.
- ❑ `HashMap`—It extends `AbstractMap` using a hash.
- ❑ `TreeMap`—It extends `AbstractMap` using a tree.
- ❑ `WeakHashMap`—It extends `AbstractMap` using a hash with weak keys, which means that elements whose keys are no longer in use will be discarded.

You'll also see other classes here that are not technically members of the collection framework—`Arrays`, `Vector`, `Stack`, `Hashtable`, and others. These older collections have been rebuilt on collection framework functionality. In fact, most standard computing collections are implemented in Java in various ways; for example, the `TreeSet` class implements a set using a tree internally, which means access time is quick. `HashSet` implements a set using a hash internally, which means that adding and removing elements from such a set should always take the same amount of time, no matter how large the set grows.

Collections Framework Enhancements in Java SE 8

The Java platform provides support for the collection framework. It is known that a collection is an object representing the group of objects. The collection framework is an architecture representing and manipulating collections.

The collections framework enhancements in Java SE 8 are as follows:

- ❑ Support for lambda expressions, streams, and aggregate operations
- ❑ Performance improvement for `HashMap`s with key collisions
- ❑ Improved type inference

Let's learn more about these enhancements in detail in the following subsections.

Support for Lambda Expressions, Streams, and Aggregate Operations

In Java SE8, the Java Collections Framework supports lambda expressions, streams, and aggregate operations. Moreover, the Stream Application Program Interface (API) is integrated into the Collections API, and as a result,

bulk operations on collections, such as sequential or parallel map-reduce transformations, can be performed. Let's now first learn about the enhancements in lambda expression and streams in Java SE 8.

Lambda Expressions and Streams in Java SE 8

In Java SE 8, new classes are added and improvement in the existing classes has been made for taking advantage of lambda expressions and streams. The new and enhanced classes are available in the following packages:

- ❑ `java.util`—Integrates the Java Collections Framework with streams and provides general utility functionality used by streams
- ❑ `java.util.function`—Includes general purpose functional interfaces that provide target types for lambda expressions and references for method
- ❑ `java.util.stream`—Includes the majority of interfaces and classes that provide functionality to streams and aggregate operations

Besides the additions made to the existing classes, some other additions include methods that accept instances of functional interfaces, which can be invoked with lambda expressions or method references. The `java.util.function` and `java.util.stream` packages are two new packages included in Java SE8. Let's discuss about each in detail.

The `java.util.function` Package

The `java.util.function` package includes interfaces that can be used by the JDK and with the code developed by a user. The package also includes enough functionality that can meet the common requirements of lambda expressions. This package also provides functional interfaces such as `FileFilter` that fulfill specific purposes.

The interfaces in the `java.util.function` package are annotated with `FunctionalInterface`. However, putting this annotation is not compulsory for making the compiler to identify an interface as a functional interface. The annotations are merely used for capturing design intent and providing help to the compiler to determine the accidental violations of design intent.

Functional interfaces generally focus on abstract concepts such as functions, actions, or predicates. While using functional interfaces or referring the variables typed as functional interfaces, you need to refer to the abstract concepts; for example, you can use "this function" in place of "the function represented by this object." The different types of functional interfaces and their description are presented in Table 9.2:

Interface	Description
<code>BiConsumer<T, U></code>	Signifies an operation that takes two input arguments and gives no result
<code>BiFunction<T, U, R></code>	Signifies a function that takes two arguments and generates a result
<code>BinaryOperator<T></code>	Signifies an operation on two operands of the same type, providing a result of the same type as the operands
<code>BiPredicate<T, U></code>	Signifies a predicate (boolean-valued function) of two arguments
<code>BooleanSupplier</code>	Signifies a supplier of boolean-valued results
<code>Consumer<T></code>	Signifies an operation that takes a single input argument and gives no result
<code>DoubleBinaryOperator</code>	Signifies an operation on two double-valued operands and generating a double-valued result
<code>DoubleConsumer</code>	Signifies an operation that gets a single double-valued argument and gives no result
<code>DoubleFunction<R></code>	Signifies a function that takes a double-valued argument and generates a result
<code>DoublePredicate</code>	Signifies a predicate (boolean-valued function) of one double-valued argument
<code>DoubleSupplier</code>	Signifies a supplier of double-valued results
<code>DoubleToIntFunction</code>	Signifies a function that takes a double-valued argument and generates an int-valued result

Table 9.2: Types of Interfaces and their Description

Interface	Description
DoubleToLongFunction	Signifies a function that takes a double-valued argument and generates a long-valued result
DoubleUnaryOperator	Signifies an operation on a single double-valued operand that generates a double-valued result
Function<T,R>	Signifies a function that takes one argument and generates a result
IntBinaryOperator	Signifies an operation on two int-valued operands and generating an int-valued result
IntConsumer	Signifies an operation that takes a single int-valued argument and gives no result
IntFunction<R>	Signifies a function that takes an int-valued argument and generates a result
IntPredicate	Signifies a predicate (boolean-valued function) of one int-valued argument
IntSupplier	Signifies a supplier of int-valued results
IntToDoubleFunction	Signifies a function that takes an int-valued argument and generates a double-valued result
IntToLongFunction	Signifies a function that takes an int-valued argument and generates a long-valued result
IntUnaryOperator	Signifies an operation on a single int-valued operand that generates an int-valued result
LongBinaryOperator	Signifies an operation on two long-valued operands and generates a long-valued result
LongConsumer	Signifies an operation that takes a single long-valued argument and gives no result
LongFunction<R>	Signifies a function that takes a long-valued argument and generates a result
LongPredicate	Signifies a predicate (boolean-valued function) of one long-valued argument
LongSupplier	Signifies a supplier of long-valued results
LongToDoubleFunction	Signifies a function that takes a long-valued argument and generates a double-valued result
LongToIntFunction	Signifies a function that takes a long-valued argument and generates an int-valued result
LongUnaryOperator	Signifies an operation on a single long-valued operand that generates a long-valued result
ObjDoubleConsumer<T>	Signifies an operation that takes an object-valued argument and a double-valued argument and gives no result
ObjIntConsumer<T>	Signifies an operation that takes an object-valued argument and an int-valued argument and gives no result
ObjLongConsumer<T>	Signifies an operation that takes an object-valued argument and a long-valued argument and gives no result
Predicate<T>	Signifies a predicate (boolean-valued function) of one argument
Supplier<T>	Signifies a supplier of results
ToDoubleBiFunction<T,U>	Signifies a function that takes two arguments and generates a double-valued result
ToDoubleFunction<T>	Signifies a function that generates a double-valued result
ToIntBiFunction<T,U>	Signifies a function that takes two arguments and generates an int-valued result
ToIntFunction<T>	Signifies a function that generates an int-valued result
ToLongBiFunction<T,U>	Signifies a function that takes two arguments and generates a long-valued result
ToLongFunction<T>	Signifies a function that generates a long-valued result
UnaryOperator<T>	Signifies an operation on a single operand that produces a result of the same type as its operand

The java.util.stream Package

The stream class is the main abstraction provided in this package. Classes such as `Stream`, `IntStream`, `LongStream`, and `DoubleStream` are streams over objects and the primitive `int`, `long`, and `double` types. Streams can be obtained by several ways, which are as follows:

- ❑ They can be obtained from a `Collection` using the `stream()` and `parallelStream()` methods.
- ❑ They can be obtained from an array using the `Arrays.stream (Object[])` method.
- ❑ They can be obtained from static factory methods on the stream classes, such as `Stream.of (Object[])`, `IntStream.range (int, int)`, or `Stream.iterate (Object, UnaryOperator)`.
- ❑ The lines of a file can be obtained from the `BufferedReader.lines ()` method.
- ❑ Streams of file paths can be obtained from the methods in `Files`.
- ❑ Streams of random numbers can be obtained from the `Random.ints ()` method.
- ❑ Various other stream-bearing methods in the JDK are `BitSet.stream ()`, `Pattern.splitAsStream (java.lang.CharSequence)`, and `JarFile.stream ()`.

The various interfaces available in the `java.util.stream` package and their description are listed in Table 9.3:

Table 9.3: Interfaces and their Description	
Interface	Description
<code>BaseStream<T,S extends BaseStream<T,S>></code>	Refers to the base interface for streams, which are sequences of elements that support sequential and parallel aggregate operations
<code>Collector<T,A,R></code>	Refers to a mutable reduction operation that accumulates input elements into a mutable result container, optionally transforming the accumulated result into a final representation after the processing of all the input elements
<code>DoubleStream</code>	Refers to a sequence of primitive double-valued elements which support sequential and parallel aggregate operations
<code>DoubleStream.Builder</code>	Refers to a mutable builder for a <code>DoubleStream</code>
<code>IntStream</code>	Refers to a sequence of primitive int-valued elements which support sequential and parallel aggregate operations
<code>IntStream.Builder</code>	Refers to a mutable builder for an <code>IntStream</code>
<code>LongStream</code>	Refers to a sequence of primitive long-valued elements which support sequential and parallel aggregate operations
<code>LongStream.Builder</code>	Refers to a mutable builder for a <code>LongStream</code>
<code>Stream<T></code>	Refers to a sequence of elements which support sequential and parallel aggregate operations
<code>Stream.Builder<T></code>	Refers to a mutable builder for a <code>Stream</code>

Table 9.4 presents the classes in the `java.util.stream` package:

Table 9.4: Classes in the java.util.stream Package and their Description	
Class	Description
<code>Collectors</code>	Contains implementations of <code>Collector</code> that implements several useful reduction operations, such as gathering elements into collections, summarizing elements on the basis of different criteria, etc.
<code>StreamSupport</code>	Provides low-level utility methods to create and manipulate streams

The main differences between stream and collections are as follows:

- ❑ **No storage** – A stream does not store elements as it is not a data structure, whereas a collection does.

- ❑ **Functional nature**—Whenever an operation is performed on a stream, it produces a result and therefore does not alter its source. For example, when the filtering operation is performed on a stream, which is retrieved from a collection, it generates a new `Stream`. This new stream does not contain filtered elements and is created instead of deleting elements present in the source collection. On the other hand, in a collection, the elements are deleted from the existing collection.
- ❑ **Laziness-seeking**—Several operations on stream can be lazily implemented due to which optimization can take place. These operations may include filtering, mapping, or duplicate removal. Generally, stream operations are broken down into intermediate (or `Stream`-producing) operations and terminal (or value- or side-effect-producing) operations. Intermediate operations are always considered lazy. On the other hand, optimization cannot take place in a collection.
- ❑ **Possibly unbounded**—Streams do not have a finite size unlike collections.
- ❑ **Consumable**—The elements of a stream are only visited once during the life of a stream. Like an `Iterator`, a new stream must be generated to revisit the same elements of the source. However, in a collection, the elements can be navigated again and again.

Aggregate Operations

An operation that is performed on a data structure rather than on an individual element is known as an aggregate operation. Aggregate operations, like `forEach`, give an impression that these are like iterators. However, they are different from iterators in the following ways:

- ❑ **They use internal iteration**—Aggregate operations perform internal iteration as they do not contain a method like `next`, which enables them to iterate to the next element of the collection. With the help of internal delegation, your application identifies the collection over which it iterates. But the JDK identifies the iteration over the collection. On the other hand, with the help of external iteration, your application can determine about the collection over which it iterates and the way it iterates. However, there is a limitation with external iterators as they can only iterate sequentially over the elements of a collection. This limitation does not exist with internal iterators. Moreover, the internal iteration takes benefits of parallel computing, which involves breaking a problem into subproblems and then solving them simultaneously.
- ❑ **They process elements from a stream**—Aggregate operations do not process elements directly from a collection, but directly from a stream. Therefore, they are also called stream operations.
- ❑ **They support behavior as parameters**—The lambda expressions can also be specified as parameters for most of the aggregate operations which allow customization of the behavior of a specific aggregate operation.

To better understand the concept of aggregate operations, consider a scenario in which you are developing a social networking site. You want to provide the facility that allows an administrator to perform different actions, such as sending a message to the members of the social networking application, which satisfy certain criteria. Suppose the members of this social networking application are represented by the following `SNMember` class:

```
public class SNMember {
    public enum Gender {
        M, FM
    }
    String sname;
    LocalDate mem_birthday;
    Gender sex;
    String email;
    // ...
    public int getMemage() {
        // ...
    }
    public String getMemname() {
        // ...
    }
}
```

Consider the following code snippet which prints the name of all members contained in the collection `memlist` using the `for-each` loop:

```
for (SNMember sn : memlist) {
    System.out.println(sn.getMemname());
}
```

Now, consider the following code snippet which prints all the members present in the collection `memlist`, but with the use of aggregate operation `forEach`:

```
memlist
    .stream()
    .forEach(e -> System.out.println(e.getMemname()));
```

In the previous code snippet, the aggregate operation for-each loop is used.

Pipelines and Streams

A pipeline refers to a sequence of aggregate operations. Consider the following code snippet that prints the male members available in the collection `memlist` with a pipeline that comprises the aggregate operations such as `filter` and `forEach`:

```
memlist
    .stream()
    .filter(e -> e.getsex() == SNMember.Gender.M)
    .forEach(e -> System.out.println(e.getMemname()));
```

A pipeline consists of the following components:

- ❑ **Source**—It might be a collection, an array, a generator function, or an I/O channel. In the previous code snippet, the example of a source is the collection `memlist`.
- ❑ **Zero or more intermediate operations**— An intermediate operation, such as `filter`, produces a new stream.
- ❑ **Stream**—A stream does not store elements; instead, it carries values from a source through a pipeline. In the previous code snippet, a stream is created from the collection `memlist` by invoking the method `stream`.
- ❑ **Predicate**—A new stream is returned by the filter operation. This stream comprises those elements that match its predicate. In the preceding code snippet, the predicate is the lambda expression `e -> e.getsex() == SNMember.Gender.M`, which gives the boolean value (true or false). It returns true if the sex field of object `e` has the value `SNMember.Gender.M`. Subsequently, the filter operation returns a stream that comprises all male members in the collection `memlist`.
- ❑ **Terminal operation**—A pipeline also comprises a terminal operation, such as `forEach`, which generates a non-stream result, such as a primitive value, a collection, or no value at all. In the preceding code snippet, the parameter of the `forEach` operation is the lambda expression `e -> System.out.println(e.getMemname())`, which invokes the method `getMemname` on the object `e`.

Performance Improvement for HashMaps with Key Collisions

The performance has been improved for `HashMap` objects where lots of collisions occur in the keys. A String hash function included in Java 7 update 6 is now removed from JDK 8 along with the `jdk.map.althashing.threshold` system property. Hash bins, having very large number of colliding keys, enhance performance by saving their entries in a balanced tree and not in the linked list. This modification in JDK 8 implies only to `HashMap`, `LinkedHashMap`, and `ConcurrentHashMap`.

This modification might bring a change to the order of iteration in `HashMap` and `HashSet` in very rare situations. Any specific order of iteration is not mentioned for `HashMap` objects and any code that relies on iteration order should be fixed.

There is no change in the `java.util.Hashtable` class other than obsoleting the feature introduced in 7u6. The features added in Java 7 update 6 only apply to `WeakHashMap` and `Hashtable`, but these have been removed in JDK 8.

Improved Type Inference

The Java compiler takes the benefit of target typing for inferring the type parameters belonging to a generic method invocation. The *target type* of an expression refers to the data type that is expected by the Java compiler depending on the location of appearance of expression. For example, earlier, that is, in Java SE7, an assignment statement's target type is used for type inference. However, in case of Java SE 8, the target type for type inference

is used. Consider the following code snippet in which a method invocation's target types are used for inferring the data types of its arguments:

```
List<String> strLst = new ArrayList<>();
strLst.add("B");
strLst.addAll(Arrays.asList());
```

In the preceding code snippet, the method `addAll` is expecting a `Collection` object as its parameter and the method `Arrays.asList` returns a `List` object. This will work as `List` is a subtype of `Collection`.

Now, consider the case of generics; the target type of `addAll` is `Collection<? extends String>`. On the other hand, the `Arrays.asList()` method returns a `List<T>` instance. In Java SE 8, the compiler infers that the value of the type variable `T` is `String`. The compiler can infer this by using the target type `Collection<? extends String>`.

Using the **Collection** Interface

The `Collection` interface revolves around the objects to provide maximum generality. For example, all general-purpose collection executions have a constructor that takes `Collection` parameter. This constructor, known as a conversion constructor, initializes the new collection to hold all of the elements in the intended location, whatever be the collection's subinterface or implementation type. You can say that it permits you to convert the collection's type.

Consider an example that you have a `Collection<String> co`, which can be a `List`, a `Set`, or another kind of `Collection`. It creates a new `ArrayList` (an implementation of the `List` interface), which consists of all the elements in `co`. The code using the `Collection<String> co` is as follows:

```
List<String> list = new ArrayList<String>(co);
```

The foundation of the collection framework is the `Collection` interface, and because the collection classes implement this interface, we'll take a look at its methods here for reference. You'll find the methods for the `Collection` interface in Table 9.5:

Table 9.5: Methods of the Collection interface	
Method	Does this
<code>boolean add(E e)</code>	It adds the given element.
<code>boolean addAll(Collection c)</code>	It adds all the elements in the given collection.
<code>void clear()</code>	It removes all the elements from this collection.
<code>boolean contains(E e)</code>	It returns <code>True</code> if this collection contains the given element.
<code>boolean containsAll(Collection c)</code>	It returns <code>True</code> if this collection contains all the elements in the given collection.
<code>boolean equals(E e)</code>	It compares the given object with this collection for equality.
<code>int hashCode()</code>	It gets the hashcode value for this collection.
<code>boolean isEmpty()</code>	It returns <code>True</code> if this collection has no elements.
<code>Iterator iterator()</code>	It gets an iterator over the elements in this collection.
<code>boolean remove(E e)</code>	It removes a single instance of the given element.
<code>boolean removeAll(Collection c)</code>	It removes all of this collection's elements that are also contained in the given collection (Optional Operation).
<code>default boolean removeIf(Predicate<? super E> filter)</code>	It removes all the elements of this collection which satisfy the given predicate.
<code>boolean retainAll(Collection c)</code>	It keeps only the elements in this collection which are contained in the given collection (Optional Operation).
<code>int size()</code>	It gets the number of elements in this collection.
<code>default Spliterator<E> HYPERLINK "http://docs.oracle.com/javase/8/docs/api/java/util/Collection.html" spliterator()</code>	It creates a <code>Spliterator</code> over the elements in this collection.

Table 9.5: Methods of the Collection interface

Method	Does this
<code>default Stream<E> stream()</code>	It returns a sequential Stream with this collection as its source.
<code>Object[] toArray()</code>	It gets an array containing all the elements in this collection.
<code><T> T[] toArray(T[] a)</code>	It gets an array containing all the elements in this collection whose type is that of the given array.

The Queue Interface

A Queue is a collection of elements to hold them before processing starts. Besides basic Collection operations, queues provide additional insertion, deletion, and inspection operations. The Queue interface follows:

```
public interface Queue<E> extends Collection<E>
{
    E element();
    boolean offer(E e);
    E peek();
    E poll();
    E remove();
}
```

Each Queue method exists in the following two forms:

1. It throws an exception if the operation fails.
2. It returns a special value if the operation fails (either null or false, depending on the operation).

The methods of the Queue interface are listed in Table 9.6:

Table 9.6: Methods of the Queue interface

Method	Does this
<code>boolean add(E e)</code>	It inserts specified elements into this queue and returns true. If no available space, then it throws an <code>IllegalStateException</code> .
<code>E element()</code>	It retrieves and returns the element at the head of this queue.
<code>boolean offer(E e)</code>	It inserts element and returns true if the element is inserted into the queue.
<code>E peek()</code>	It returns the item at the head of this queue without removing it from the queue. It throws an exception if the queue is empty and returns null.
<code>E poll()</code>	It retrieves and removes the head of this queue. If the queue is empty, then it returns null.
<code>E remove()</code>	It retrieves and removes the head of this queue.

The List Interface

The List interface is the foundation of classes such as `LinkedList` and `ArrayList`. It is basically an extension of the Collection interface that stores the behavior of a collection. So, we'll take a look at the methods of this interface for reference. The methods of the List interface are provided in Table 9.7:

Table 9.7: Methods of the List interface

Method	Does this
<code>void add(int index, E element)</code>	It inserts the given element at the given position in this list.
<code>boolean add(E e)</code>	It adds the given element to the end of this list.
<code>boolean addAll(Collection<? extends E> c)</code>	It adds all the elements in the given collection to the end of this list.
<code>boolean addAll(int index, Collection<? extends E> c)</code>	It inserts all the elements in the given collection into this list.

Table 9.7: Methods of the List interface

Method	Does this
<code>void clear()</code>	It removes all the elements from this list.
<code>boolean contains(Object o)</code>	It returns True if this list contains the given element.
<code>boolean containsAll(Collection<?> c)</code>	It returns True if this list contains all the elements of the given collection.
<code>boolean equals(Object o)</code>	It compares the given object with this list for equality.
<code>Object get(int index)</code>	It gets the element at the given position in this list.
<code>int hashCode()</code>	It gets the hashCode value for this list.
<code>int indexOf(Object o)</code>	It gets the index of the first occurrence of the given element in this list or -1 if this list does not contain the element.
<code>boolean isEmpty()</code>	It returns True if this list contains no elements.
<code>Iterator iterator()</code>	It gets an iterator over the elements in this list in proper sequence.
<code>int lastIndexOf(Object o)</code>	It gets the index of the last occurrence of the given element in this list or -1 if this list does not contain the element.
<code>ListIterator listIterator()</code>	It gets a list iterator over the elements in this list (in proper sequence).
<code>ListIterator listIterator(int index)</code>	It gets a list iterator of the elements in this list, starting at the given position in this list.
<code>Object remove(int index)</code>	It removes the element at the given position in this list.
<code>boolean remove(Object o)</code>	It removes the first occurrence in this list of the given element.
<code>boolean removeAll(Collection<?> c)</code>	It removes from this list all of its elements that are contained in the given collection (optional operation).
<code>default replaceAll(UnaryOperator<E> operator)</code>	It replaces each element of this list with the result of applying the operator to that element.
<code>boolean retainAll(Collection<?> c)</code>	It keeps only the elements in this list that are contained in the given collection.
<code>E set(int index, E element)</code>	It replaces the element at the given position in the list with the new element.
<code>int size()</code>	It gets the number of elements in this list.
<code>default void sort(Comparator<? super E> c)</code>	It sorts this list according to the order induced by the specified Comparator.
<code>default Spliterator<E> spliterator()</code>	It creates a Spliterator over the elements in this list.
<code>List<E> subList(int fromIndex, int toIndex)</code>	It returns a view of the portion of this list between the specified fromIndex, inclusive, and toIndex, exclusive.
<code>List subList(int fromIndex, int toIndex)</code>	It gets a view of the section between the given fromIndex (inclusive) and toIndex.
<code>Object[] toArray()</code>	It gets an array containing all the elements in this list in proper sequence (from first to last element).
<code><T> T[] toArray(T[] a)</code>	It gets an array containing all the elements in this list in proper sequence (from first to last element); the runtime type of the returned array is that of the specified array.

The Set Interface

The Set interface is the foundation of classes such as `HashSet` and `TreeSet`, and it is used by the collection framework to implement sets. Therefore, we'll take a look at this interface for reference. The methods of the Set interface are provided in Table 9.8:

Table 9.8: Methods of the Set Interface

Method	Does this
<code>boolean add(E e)</code>	It adds the given element to this set.
<code>boolean addAll(Collection<? extends E> c)</code>	It adds all the elements in the given collection.
<code>void clear()</code>	It removes all the elements from this set.
<code>boolean contains(Object o)</code>	It returns True if this set contains the given element.
<code>boolean containsAll(Collection<?> c)</code>	It returns True if this set contains all the elements of the given collection.
<code>boolean equals(Object o)</code>	It compares the given object with this set for equality.
<code>int hashCode()</code>	It gets the hashcode value for this set.
<code>boolean isEmpty()</code>	It returns True if this set contains no elements.
<code>Iterator iterator()</code>	It gets an iterator over the elements in this set.
<code>boolean remove(Object o)</code>	It removes the given element from this set if it is present.
<code>boolean removeAll(Collection<?> c)</code>	It removes all elements contained in the given collection.
<code>boolean retainAll(Collection<?> c)</code>	It keeps only the elements in this set that are contained in the given collection.
<code>int size()</code>	It gets the number of elements in this set (its cardinality).
<code>Object[] toArray()</code>	It gets an array containing all the elements in this set.
<code><T> T[] toArray(T[] a)</code>	It gets an array containing all the elements in this set whose runtime type is that of the given array.

The SortedSet Interface

The SortedSet interface maintains a sorted set, and you'll find the methods of this interface in Table 9.9:

Table 9.9: Methods of the SortedSet Interface

Method	Does this
<code>Comparator<? Super E> comparator()</code>	It returns the comparator used to order the elements in this set or null if this set uses the natural ordering.
<code>E first()</code>	It gets the first (lowest) element currently in this set.
<code>SortedSet headSet(Object toElement)</code>	It gets a view of the section of this set whose elements are strictly less than <code>toElement</code> .
<code>E last()</code>	It gets the last (highest) element currently in this set.
<code>default Spliterator<E> spliterator()</code>	It creates a Spliterator over the elements in this sorted set.
<code>SortedSet<E> subSet(E fromElement, E toElement)</code>	It gets a view of the portion of this set whose elements range from <code>fromElement</code> to <code>toElement</code> .
<code>SortedSet<E> tailSet(E fromElement)</code>	It gets a view of the portion of this set whose elements are greater than or equal to <code>fromElement</code> .

Using the Collection Classes

We have discussed earlier that a collection is a series of items of the same type unlike arrays. The main drawback of arrays was that you could not know or predict the number of items of the list. The solution to this was to create your own list. The `Collection` class contains exclusive static methods that operate on or return collections. It contains polymorphic (more than one feature) algorithms that operate on collections, "wrappers", which return a new collection.

The **AbstractCollection** Class

The `AbstractCollection` class is the implementation of the `Collection` interface upon which many Java collection classes are built, so we'll take a look at `AbstractCollection` for reference. Here's the inheritance diagram for this class:

```

java.lang.Object
|____java.util.AbstractCollection

```

You'll find the constructor for the `AbstractCollection` class in Table 9.10 and its methods in Table 9.11:

Table 9.10: The constructor of the `AbstractCollection` class

Constructor	Does this
<code>protected AbstractCollection()</code>	It constructs an <code>AbstractCollection</code> object.

Table 9.11: Methods of the `AbstractCollection` class

Method	Does this
<code>boolean add(E e)</code>	It ensures that this collection contains the given element.
<code>boolean addAll(Collection<? extends E> c)</code>	It adds all the elements in the given collection to this collection.
<code>void clear()</code>	It removes all the elements from this collection.
<code>boolean contains(Object o)</code>	It returns <code>True</code> if this collection contains the given element.
<code>boolean containsAll(Collection<?> c)</code>	It returns <code>True</code> if this collection contains all the elements in the given collection.
<code>boolean isEmpty()</code>	It returns <code>True</code> if this collection contains no elements.
<code>abstract Iterator iterator()</code>	It gets an iterator over the elements contained in this collection.
<code>boolean remove(Object o)</code>	It removes the given element from this collection.
<code>boolean removeAll(Collection<?> c)</code>	It removes all of this collection's elements that are also contained in the given collection.
<code>boolean retainAll(Collection<?> c)</code>	It keeps only the elements contained in the given collection.
<code>abstract int size()</code>	It gets the number of elements in this collection.
<code>Object[] toArray()</code>	It gets an array containing all the elements in this collection.
<code><T> T[] toArray(T[] a)</code>	It gets an array that contains all the elements in this collection.
<code>String toString()</code>	It gets a string representation of this collection.

The **AbstractList** Class

The `AbstractList` class extends the `AbstractCollection` class. It is the foundation of other classes such as `ArrayList`, which supports dynamic arrays. Therefore, we'll refer `AbstractList` here for reference. The inheritance diagram for this class is as follows:

```

java.lang.Object
|____java.util.AbstractCollection
|____java.util.AbstractList

```

You'll find the field of the `AbstractList` class in Table 9.12, its constructor in Table 9.13, and its methods in Table 9.14:

Table 9.12: The field of the `AbstractList` class

Field	Does this
<code>protected int modCount</code>	It indicates the number of times this list has been modified.

Table 9.13: The constructor of the `AbstractList` class

Constructor	Does this
<code>protected AbstractList()</code>	It constructs an <code>AbstractList</code> object.

Table 9.14: Methods of the `AbstractList` class

Method	Does this
<code>void add(int index, E element)</code>	It inserts the given element into this list.
<code>boolean add(E e)</code>	It adds the given element to the end of this list.
<code>boolean addAll(int index, Collection<? extends E> c)</code>	It inserts the elements in the given collection into this list.
<code>void clear()</code>	It removes all the elements from this list.
<code>boolean equals(Object o)</code>	It compares the given object with this list for equality.
<code>abstract Object get(int index)</code>	It gets the element at the given position in this list.
<code>int hashCode()</code>	It gets the hashcode value for this list.
<code>int indexOf(Object o)</code>	It gets the index of the first occurrence of the given element in this list, or -1 if this list does not contain the element.
<code>Iterator iterator()</code>	It gets an iterator over the elements in this list in proper sequence.
<code>int lastIndexOf(Object o)</code>	It gets the index of the last occurrence of the given element in this list, or -1 if this list does not contain the element.
<code>ListIterator listIterator()</code>	It gets a list iterator over the elements in this list (in proper sequence).
<code>ListIterator listIterator(int index)</code>	It gets a list iterator of the elements in this list (in proper sequence), starting at the given position in this list.
<code>E remove(int index)</code>	It removes the element at the given position in this list.
<code>protected void removeRange(int fromIndex, int toIndex)</code>	It removes from this list all the elements whose index is between <code>fromIndex</code> and <code>toIndex</code> .
<code>E set(int index, E element)</code>	It replaces the element at the given position in this list with the new element.
<code>List subList(int fromIndex, int toIndex)</code>	It gets a view of the portion of this list between the specified <code>fromIndex</code> (inclusive) and <code>toIndex</code> (exclusive).

The ***AbstractSequentialList*** Class

The `AbstractSequentialList` class is derived from the `AbstractList` class. It is the foundation of classes such as `LinkedList`. Let's look at the inheritance diagram for the `AbstractSequentialList` class:

```

java.lang.Object
|___java.util.AbstractCollection
|   |___java.util.AbstractList
|       |___java.util.AbstractSequentialList

```

You'll find the constructor of the `AbstractSequentialList` class in Table 9.15 and its methods in Table 9.16:

Table 9.15: The constructor of the `AbstractSequentialList` class

Constructor	Does this
<code>protected AbstractSequentialList()</code>	It constructs an <code>AbstractSequentialList</code> object.

Table 9.16: Methods of the `AbstractSequentialList` class

Method	Does this
<code>void add(int index, E element)</code>	It inserts the given element at the given position in this list (optional operation).
<code>boolean addAll(int index, Collection<? extends E> c)</code>	It inserts all the elements in the given collection into this list at the specified position (optional operation).

Table 9.16: Methods of the AbstractSequentialList class

Method	Does this
Object get(int index)	It gets the element at the given position in this list.
Iterator<E> iterator()	It gets an iterator over the elements in this list.
abstract ListIterator<E> listIterator(int index)	It gets a list iterator over the elements in this list.
E remove(int index)	It removes the element at the given position in this list (optional operation).
E set (int index, E element)	It replaces the element at the given position in this list with the specified element (optional operation).

The ArrayList Class

The ArrayList class is an array class that can grow or shrink at runtime. Note that arrays of this class must hold objects, not just simple data types. Here's the inheritance diagram for this class:

```

java.lang.Object
|___java.util.AbstractCollection
|   |___java.util.AbstractList
|       |___java.util.ArrayList

```

You'll find the constructors of the ArrayList class in Table 9.17 and its methods in Table 9.18:

Table 9.17: Constructors of the ArrayList class

Constructor	Does this
ArrayList()	It constructs an ArrayList object.
ArrayList(Collection<? extends E> c)	It constructs a list containing the elements of the given collection.
ArrayList(int initialCapacity)	It constructs an empty list with the given initial capacity.

Table 9.18: Methods of the ArrayList class

Method	Does this
void add(int index, E element)	It inserts the given element at the given position in this list.
boolean add(E e)	It adds the given element to the end of this list.
boolean addAll (Collection<? extends E> c)	It adds all the elements in the given collection to the end of this list in the order that they are returned by the specified collection's Iterator.
boolean addAll (int index, Collection<? extends E> c)	It inserts all the elements in the given collection into this list, starting at the given position.
void clear()	It removes all the elements from this list.
Object clone()	It gets a copy of this ArrayList instance.
boolean contains (Object elem)	It returns True if this list contains the given element.
void ensureCapacity (int minCapacity)	It increases the capacity of this ArrayList instance.
void forEach(Consumer<? super E> action)	It performs the given action for each element of the Iterable until all elements have been processed or the action throws an exception.
Object get(int index)	It gets the element at the given position in this list.
int indexOf(Object elem)	It returns the index of the first occurrence of the given element in this list, or -1 if this list does not contain the element.
boolean isEmpty()	It returns true if this list has no elements.
int lastIndexOf (Object elem)	It gets the index of the last occurrence of the given element in this list, or -1 if this list has no element.
E remove(int index)	It removes the element at the given position in this list.

Table 9.18: Methods of the ArrayList class

Method	Does this
<code>boolean remove(Object elem)</code>	It removes the first occurrence of the specified element from this list, if it is present.
<code>protected void removeRange(int fromIndex, int toIndex)</code>	It removes from this list all the elements whose index is between <code>fromIndex</code> and <code>toIndex</code> .
<code>E set(int index, E element)</code>	It replaces the element at the given position in this list with the given element.
<code>int size()</code>	It gets the number of elements in this list.
<code>void sort(Comparator<? super E> c)</code>	It sorts this list according to the order induced by the specified <code>Comparator</code> .
<code>Splitterator<E> splitterator()</code>	It creates a late-binding and fail-fast <code>Splitterator</code> over the elements in this list.
<code>List<E> subList(int fromIndex, int toIndex)</code>	It returns a view of the portion of this list between the specified <code>fromIndex</code> , inclusive, and <code>toIndex</code> , exclusive.
<code>Object[] toArray()</code>	It gets an array containing all the elements in this list in proper sequence (from first to last element).
<code><T> T[] toArray(T[] a)</code>	It gets an array containing all the elements in this list in proper sequence (from first to last element); the runtime type of the returned array is that of the specified array.
<code>void trimToSize()</code>	It trims the capacity to be the list's current size.

You can add elements to an `ArrayList` object with the `add()` method, get an element at a certain index with the `get()` method, and remove elements with the `remove()` method.

Let's consider an example in which we add elements to an array at runtime. Remember we are using two forms of the `add()` method: general form and the form that allows to add an element at a specific location (index). We are also using the `remove()` method to remove one element. The `Iterator` interface is defined in the `java.util` package. This `Iterator` interface has three methods—`hasNext()`, `next()`, and `remove()`. These methods are all you need to access each element of a collection, as you see here:

```
import java.util.*;
class ArrayList
{
    public static void main(String args[])
    {
        ArrayList<String> arraylist = new ArrayList<String>();
        arraylist.add("Item 0");
        arraylist.add("Item 2");
        arraylist.add("Item 3");
        arraylist.add("Item 4");
        arraylist.add("Item 5");
        arraylist.add("Item 6");
        arraylist.add(1, "Item 1");

        System.out.println("\nusing the add method");
        System.out.println(arraylist);

        arraylist.remove("Item 5");

        System.out.println(arraylist);
        System.out.println("\nusing the Iterator interface");
        String s;
        Iterator e = arraylist.iterator();
        while (e.hasNext())
        {
            s = (String)e.next();
            System.out.println(s);
        }
    }
}
```

```
    }
}
}
```

Here's the output of this example:

```
C:\>java ArrayList
Using the add method
[Item 0, Item 1, Item 2, Item 3, Item 4, Item 5, Item 6]
[Item 0, Item 1, Item 2, Item 3, Item 4, Item 6]

Using the Iterator interface
Item 0
Item 1
Item 2
Item 3
Item 4
Item 6
```

NOTE

Generics is on the leading edge of Object-Oriented Programming. You can get by without knowing anything about generics, but it is better to just have an idea about them so as to learn how `ArrayList` and `LinkedList` classes use the new generics feature.

The `ArrayDeque` Class

The `ArrayDeque` class was added in Java SE 6. It extends `AbstractCollection` and implements the `Deque` interface. It does not add methods on its own. The `ArrayDeque` class forms a dynamic array which has no limitations on capacity. The `Deque` interface supports implementation that limits capacity, but does not require such restrictions.

The `ArrayDeque` class is a generic class and its inheritance diagram is as follows:

```
java.lang.Object
|___java.util.AbstractCollection
|___java.util.ArrayDeque
```

You'll find the constructors of the `ArrayDeque` class in Table 9.19 and its methods in Table 9.20:

Table 9.19: Constructors of the `ArrayDeque` class

Constructor	Does this
<code>ArrayDeque()</code>	It constructs an <code>ArrayDeque</code> object.
<code>ArrayDeque(Collection<? extends E> c)</code>	It constructs a deque containing the elements of the given collection.
<code>ArrayDeque(int numElements)</code>	It constructs an empty deque with the given initial capacity.

Table 9.20: Methods of the `ArrayDeque` class

Method	Does this
<code>boolean add(E e)</code>	It adds the given element to the end of this deque.
<code>void addFirst(E e)</code>	It adds the given element to the front of this deque.
<code>void addLast(E e)</code>	It inserts the given element at the end of this deque.
<code>void clear()</code>	It removes all the elements from this deque.
<code>ArrayDeque<E> clone()</code>	It gets a copy of this <code>ArrayDeque</code> instance.
<code>boolean contains(Object elem)</code>	It returns <code>True</code> if this deque contains the given element.
<code>Iterator<E> descendingIterator()</code>	It returns an iterator over the elements in this deque in reverse sequential order.
<code>E element()</code>	It retrieves, but does not remove, the head of the queue represented by this deque.
<code>E getFirst()</code>	It retrieves, but does not remove, the first elements of this deque.
<code>E getLast()</code>	It retrieves, but does not remove, the last elements of this deque.

Table 9.20: Methods of the ArrayDeque class

Method	Does this
<code>boolean isEmpty()</code>	It returns true if this deque has no elements.
<code>Iterator<E> iterator()</code>	It returns an iterator over the elements in this deque.
<code>boolean offer(E e)</code>	It inserts the specified element at the end of this deque.
<code>boolean offerFirst(E e)</code>	It inserts the specified element at the front of this deque.
<code>boolean offerLast(E e)</code>	It inserts the specified element at the end of this deque.
<code>E peek()</code>	It retrieves, but does not remove, the head of the queue represented by this deque, or return null if this deque is empty.
<code>E peekFirst()</code>	It retrieves, but does not remove, the first element of this deque, or return null if this deque is empty.
<code>E peekLast()</code>	It retrieves, but does not remove, the last element of this deque, or return null if this deque is empty.
<code>E poll()</code>	It retrieves and removes, the head of the queue represented by this deque, or return null if this deque is empty.
<code>E pollFirst()</code>	It retrieves and removes the first element of this deque or return null if this deque is empty.
<code>E pollLast()</code>	It retrieves and removes the last element of this deque or return null if this deque is empty.
<code>E pop()</code>	It pops an element from the stack represented by this deque.
<code>void push(E e)</code>	It pushes an element from the stack represented by this deque.
<code>E remove()</code>	It retrieves and removes the head of the queue represented by this deque.
<code>E removeFirst()</code>	It retrieves and removes the first element of this deque.
<code>E removeLast()</code>	It retrieves and removes the last element of this deque.
<code>boolean removeFirstOccurrence(Object o)</code>	It removes the last occurrence of the specified element in this deque.
<code>int size()</code>	It returns the number of elements in this deque.
<code>Spliterator<E> spliterator()</code>	It creates a late-binding and fail-fast Spliterator over the elements in this deque.
<code>Object toArray()</code>	It returns an array containing all of the elements in this deque in proper sequence(from first to last element).
<code><T> T[] toArray(T[] a)</code>	It returns an array containing all of the elements in this deque in proper sequence(from first to last element); the runtime type of the returned array is that of the specified array.

To build a linked list, you can use the `add()`, `addFirst()`, and `addLast()` methods; to get an element at a certain index, you can use the `get()`, `getFirst()`, and `getLast()` methods; to set an element's value, you can use the `set` method; and to remove an element, you can use the `remove()`, `removeFirst()`, and `removeLast()` methods.

Here's an example which demonstrates `ArrayDeque` by using it to create a stack:

```
import java.util.*;
class ArrayDequeDemo { public static void main(String args[])
{
    ArrayDeque<String> arrdeque= new ArrayDeque<String>();
    arrdeque.push("Item 0"); arrdeque.push("Item 2");
    arrdeque.push("Item 3");
    arrdeque.push("Item 4");
    arrdeque.push("Item 5");
    arrdeque.push("Item 6");
    System.out.println("Popping the stack: ");
    while (arrdeque.peek() != null)
```

```

        {
            System.out.println(arrdeque.pop() + " ");
        }
        System.out.println();
    }
}

```

Here's the output of this example:

```

C:\>java ArrayDequeDemo
Popping the stack:
Item 6
Item 5
Item 4
Item 3
Item 2
Item 0

```

The **LinkedList** Class

The `LinkedList` class supports linked lists in which a record of items is maintained in these lists. With the use of such a list, you can move one element to the next using an iterator, which you'll learn in this chapter. The inheritance diagram for the `LinkedList` class is as follows:

```

java.lang.Object
|___java.util.AbstractCollection
|   |___java.util.AbstractList
|       |___java.util.AbstractSequentialList
|           |___java.util.LinkedList

```

You'll find the constructors of the `LinkedList` class in Table 9.21 and its methods in Table 9.22:

Table 9.21: Constructors of the `LinkedList` class

Constructor	Does this
<code>LinkedList()</code>	It constructs a <code>LinkedList</code> object.
<code>LinkedList(Collection<? extends E> c)</code>	It constructs a list containing the elements of the given collection.

Table 9.22: Methods of the `LinkedList` class

Method	Does this
<code>void add(int index, E element)</code>	It inserts the given element at the given position.
<code>boolean add(E e)</code>	It adds the given element to the end of this list.
<code>boolean addAll(Collection<? extends E> c)</code>	It adds all the elements in the given collection to the end of this list.
<code>boolean addAll(int index, Collection<? extends E> c)</code>	It inserts all the elements in the given collection into this list.
<code>void addFirst(E e)</code>	It inserts the specified element at the beginning of this list.
<code>void addLast(E e)</code>	It adds the specified element to the end of this list.
<code>void clear()</code>	It removes all the elements from this list.
<code>Object clone()</code>	It gets a copy of this <code>LinkedList</code> object.
<code>boolean contains(Object o)</code>	It returns <code>True</code> if this list contains the given element.
<code>Iterator<E> descendingIterator()</code>	It gets an iterator over the elements in this deque in reverse sequential order.
<code>E element()</code>	It gets, but does not remove, the head (first element) of this list.
<code>E get(int index)</code>	It gets the element at the given position in this list.
<code>E getFirst()</code>	It gets the first element in this list.
<code>E getLast()</code>	It gets the last element in this list.
<code>int indexOf(Object o)</code>	It gets the index of the first occurrence of the specified element in this list, or <code>-1</code> if this list does not contain the element.

Table 9.22: Methods of the LinkedList class

Method	Does this
<code>int lastIndexOf(Object o)</code>	It gets the index of the last occurrence of the specified element in this list or -1, if this list does not contain the element.
<code>ListIterator<E> listIterator(int index)</code>	It gets a list iterator of the elements in this list, starting at the given position in the list.
<code>boolean offer(E e)</code>	It appends the specified elements as the tail (last element) of this list.
<code>boolean offerFirst(E e)</code>	It inserts the specified element at the front of this list.
<code>boolean offerLast(E e)</code>	It inserts the specified element at the end of this list.
<code>E peak()</code>	It gets the element, but does not remove, the head (first element) of this list.
<code>E peakFirst()</code>	It gets the element, but does not remove, the first element of this list, or returns null if this list is empty.
<code>E peakLast()</code>	It gets the element, but does not remove, the last element of this list, or returns null if this list is empty.
<code>E poll()</code>	It gets and removes the head (first element) of this list.
<code>E pollFirst()</code>	It gets and removes the first element of this list, or returns null if this list is empty.
<code>E pollLast()</code>	It gets and remove the last element of this list or returns null if this list is empty.
<code>E pop()</code>	It pops an element from the stack represented by this list.
<code>void push(E e)</code>	It pushes an element onto the stack represented by this list.
<code>E remove(int index)</code>	It removes the element at the given position in this list.
<code>boolean remove(Object o)</code>	It removes the first occurrence of the given element in this list.
<code>E removeFirst()</code>	It removes and returns the first element from this list.
<code>boolean removeFirstOccurrence(Object e)</code>	It removes the first occurrence of the specified element in this list.
<code>E removeLast()</code>	It removes and returns the last element from this list.
<code>boolean removeLastOccurrence(Object e)</code>	It removes the last occurrence of the specified element in this list.
<code>E set(int index, E element)</code>	It replaces the element at the given position in this list with the given element.
<code>int size()</code>	It gets the number of elements in this list.
<code>Spliterator<E> HYPERLINK "http://docs.oracle.com/javase/8/docs/api/java/util/LinkedList.html" \1 "spliterator--" spliterator()</code>	It creates a late-binding and fail-fast Spliterator over the elements in this list.
<code>Object[] toArray()</code>	It gets an array containing all the elements in this list in the correct order.
<code><T> T[] toArray(T[] a)</code>	It gets an array containing all the elements in this list in the correct order.

To build a linked list, you can use the `add()`, `addFirst()`, and `addLast()` methods; to get an element at a certain index, you can use the `get()`, `getFirst()`, and `getLast()` methods; to set an element's value, you can use the `set()` method; and to remove an element, you can use the `remove()`, `removeFirst()`, and `removeLast()` methods.

Here's an example: We are simply building a linked list, adding some elements, and then removing some elements:

```
import java.util.*;

class LinkedList
{
    public static void main(String args[])
    {
```

```

        LinkedList<String> linkedlist1 = new LinkedList<String>();
        linkedlist1.add("Item 2");
        linkedlist1.add("Item 3");
        linkedlist1.add("Item 4");
        linkedlist1.add("Item 5");
        linkedlist1.addFirst("Item 0");
        linkedlist1.addLast("Item 6");
        linkedlist1.add(1, "Item 1");
        System.out.println(linkedlist1);
        linkedlist1.remove("Item 6");
        System.out.println(linkedlist1);
        linkedlist1.removeLast();

        System.out.println(linkedlist1);
        System.out.println("\nUpdating linked list items");

        linkedlist1.set(0, "Red");
        linkedlist1.set(1, "Blue");
        linkedlist1.set(2, "Green");
        linkedlist1.set(3, "Yellow");
        linkedlist1.set(4, "Purple");
        System.out.println(linkedlist1);
    }
}

```

Here's the output of this example:

```

[Item 0, Item 1, Item 2, Item 3, Item 4, Item 5, Item 6]
[Item 0, Item 1, Item 2, Item 3, Item 4, Item 5]
[Item 0, Item 1, Item 2, Item 3, Item 4]

Updating linked list items
[Red, Blue, Green, Yellow, Purple]

```

NOTE

Generics is on the leading edge of Object-Oriented Programming. You can move further without knowing much about generics, but it is preferable to have an idea about them so as to get utter knowledge on how the `ArrayList` and `LinkedList` classes use the new generic features.

The **Generic Class**

In case of the `ArrayList` and `LinkedList` classes, the compiler does not allow us to add the wrong type of data because it took the benefit of a new feature of Java 1.5 called Generics.

This feature will be used in the `LinkedList` class to implement a particular type of collection stacks. In this case, the items are always added to the beginning of the list and also retrieved from the beginning of the list.

Let's begin by designing a `GenStack` class that uses a `LinkedList` to execute a stack:

```

import java.util.*;

public class GenStack<E>
{
    private LinkedList<E> list = new LinkedList<E>();
    public void push(E item) { list.addFirst(item); }
    public E pop() { return list.poll(); }
    public E peek() { return list.peek(); }
    public boolean hasItems() { return !list.isEmpty(); }
    public int size() { return list.size(); }
}

```

The formal type parameter `<E>` specifies the type of elements that are stored in the list. Compile the code now. Now we'll make an example that works with the `GenStack` class:

```

import java.util.*;

public class StackTest
{
    public static void main(String[] args)
    {
    }
}

```

```

{
    GenStack<String> gs = new GenStack<String>();
    System.out.println("Pushing Tomato, Potato , Onion and Capsicum
into the stack...");
    gs.push("Tomato"); gs.push("Potato "); gs.push("Onion ");
    gs.push("Capsicum "); System.out.println("...Done...\n");
    System.out.println("The stack contains" + gs.size() + " items in the
stack and the top item is " + gs.peek() + ".\n");
    System.out.println("When the " + gs.size() + " items in the stack
are Popped, they are displayed as: ");
    while (gs.hasItems()) { System.out.println(gs.pop()); }
    System.out.println("\nNow there are " + gs.size() + " item(s) in
the stack and the top item is " + gs.peek() + ".\n"); }
}

```

When the items are popped off the stack, they come out in a reverse manner—that's a normal behavior of stack based on LIFO (Last-in First-out) principle. The result of the example is as follows:

```

Pushing Tomato, Potato, Onion and Capsicum into the stack...
...Done...

```

```

The stack contains 4 items in the stack and the top item is Capsicum.

```

```

when the 4 items in the stack are Popped, they are displayed as:

```

```

Capsicum
Onion
Potato
Tomato

```

```

Now there are 0 item(s) in the stack and the top item is null.

```

The *HashSet* Class

A set is defined as a group of unique elements, and the `HashSet` class supports sets that use a hash internally for storage. Because this class uses a hash internally, methods such as `contains()`, `remove()`, `add()`, and `size()` always take the same amount of time to execute. The inheritance diagram for this class is as follows:

```

java.lang.Object
|___java.util.AbstractCollection
|   |___java.util.AbstractSet
|       |___java.util.HashSet

```

You'll find the constructors of the `HashSet` class in Table 9.23 and its methods in Table 9.24:

Table 9.23: Constructors of the `HashSet` class

Constructor	Does this
<code>HashSet()</code>	It constructs a new, empty set.
<code>HashSet(Collection<? extends E> c)</code>	It constructs a new set containing the elements in the given collection.
<code>HashSet(int initialCapacity)</code>	It constructs a new, empty set with the given initial capacity and default load factor (0.75).
<code>HashSet (int initialCapacity, float loadFactor)</code>	It constructs a new, empty set with the given initial capacity and the given load factor.

Table 9.24: Methods of the `HashSet` class

Method	Does this
<code>boolean add(E e)</code>	It adds the given element to this set if it's not already present.
<code>void clear()</code>	It removes all the elements from this set.
<code>Object clone()</code>	It gets a copy of this <code>HashSet</code> instance.
<code>boolean contains(Object o)</code>	It returns <code>True</code> if this set contains the given element.
<code>boolean isEmpty()</code>	It returns <code>True</code> if this set contains no elements.
<code>Iterator<E> iterator()</code>	It gets an iterator over the elements in this set.

Table 9.24: Methods of the HashSet class

Method	Does this
<code>boolean remove(Object o)</code>	It removes the given element from this set if it's present.
<code>int size()</code>	It gets the number of elements in this set.
<code>Spliterator<E> spliterator()</code>	It creates a late-binding and fail-fast Spliterator over the elements in this set.

Here's an example in which we add elements to a set and then print out that set:

```
import java.util.*;

class HashSet
{
    public static void main(String args[])
    {
        HashSet<String> hashset1 = new HashSet<String>();
        hashset1.add("Item 0");
        hashset1.add("Item 1");
        hashset1.add("Item 2");
        hashset1.add("Item 3");
        hashset1.add("Item 4");
        hashset1.add("Item 5");
        hashset1.add("Item 6");
        System.out.println(hashset1);
    }
}
```

One thing to note about hashes is that you can't guarantee the order in which elements are stored internally. In fact, when you print out this set, you can see that all elements are stored in exactly reverse order:

```
C:\>java HashSet
[Item 0, Item 4, Item 3, Item 2, Item 1, Item 6, Item 5]
```

The TreeSet Class

The `TreeSet` class uses a tree construct for internal storage to implement a set, which means that access times are fast. The elements of the `TreeSet` class are stored in a sorted order and its inheritance diagram is as follows:

```
java.lang.Object
|___java.util.AbstractCollection
|   |___java.util.AbstractSet
|       |___java.util.TreeSet
```

You'll find the constructors for the `TreeSet` class in Table 9.25 and its methods in Table 9.26:

Table 9.25: Constructors of the TreeSet class

Constructor	Does this
<code>TreeSet()</code>	It constructs a new, empty tree set, sorting according to the natural ordering of its elements.
<code>TreeSet(Collection<? extends E> c)</code>	It constructs a new tree set containing the elements in the given collection, sorting according to the natural ordering of its elements.
<code>TreeSet(Comparator<? super E> comparator)</code>	It constructs a new, empty tree set, sorted according to the given comparator.
<code>TreeSet(SortedSet<E> s)</code>	It constructs a new tree set containing the same elements and using the same ordering as the given sorted set.

Table 9.26: Methods of the TreeSet class

Method	Does this
<code>boolean add(E e)</code>	It adds the given element to this set.
<code>boolean addAll(Collection<? extends E> c)</code>	It adds all the elements in the given collection.
<code>E ceiling(E e)</code>	It returns the least element in this set greater than or equal to the given

Table 9.26: Methods of the TreeSet class

Method	Does this
	element, or null if there is no such element.
<code>void clear()</code>	It removes all the elements from this set.
<code>Object clone()</code>	It gets a shallow copy of this <code>TreeSet</code> instance.
<code>Comparator<? super E> comparator()</code>	It gets the comparator used to order the element in this set or null if this set uses the natural ordering.
<code>boolean contains(Object o)</code>	It returns <code>True</code> if this set contains the given element.
<code>Iterator<E> descendingIterator()</code>	It returns an iterator over the elements in this set in descending order.
<code>NavigableSet<E> descendingSet()</code>	It returns a reverse order view of the elements contained in this set.
<code>E first()</code>	It gets the first (lowest) element currently in this set.
<code>E floor(E e)</code>	It gets the greatest element in this set less than or equal to the given element, or null if there is no such element.
<code>SortedSet<E> headSet(E e)</code>	Equivalent to <code>navigableHeadSet (Object e)</code> <code>SortedSet</code> interface.
<code>NavigableSet<E> headSet(E toElement, boolean inclusive)</code>	It returns a view of the portion of this set whose elements are less than (or equal to, if inclusive is true) <code>toElement</code> .
<code>E higher(E e)</code>	It returns the least element in this set, which is greater than the given element, or null if there is no such element.
<code>boolean isEmpty()</code>	It returns <code>True</code> if this set contains no elements.
<code>Iterator<E> iterator()</code>	It gets an iterator over the elements in this set in ascending order.
<code>E last()</code>	It gets the last (highest) element currently in this set.
<code>E lower(E e)</code>	It returns the greatest element in this set strictly less than the given element, or null if there is no such element.
<code>NavigableSet<E> subSet(E fromElement, boolean fromInclusive, E toElement, boolean toInclusive)</code>	It returns a view of the portion of this set whose elements range from <code>fromElement</code> to <code>toElement</code> .
<code>SortedSet<E> subSet(E fromElement, E toElement)</code>	It returns a view of the portion of this set whose elements range from <code>fromElement</code> , inclusive, to <code>toElement</code> , exclusive.
<code>SortedSet<E> tailSet(E fromElement)</code>	It returns a view of the portion of this set whose elements are greater than or equal to <code>fromElement</code> .
<code>NavigableSet<E> tailSet(E fromElement, boolean inclusive)</code>	It returns a view of the portion of this set whose elements are greater than (or equal to, if inclusive is true) <code>fromElement</code> .
<code>E pollFirst()</code>	It gets and removes the first element, or returns null if this set is empty.
<code>E pollLast()</code>	It gets and removes the last element, or returns null if this set is empty.
<code>boolean remove(E e)</code>	It removes the given element from this set if it's present.
<code>int size()</code>	It gets the number of elements in this set.

The `TreeSet` class just implements a set using a tree for internal storage. A difference from the `HashSet` class is that elements in a `TreeSet` object are stored in sorted order. Here's the same example as the previous one, where we stored elements in a `HashSet` object; however, this time we are using a `TreeSet` object:

```
import java.util.*;

class TreeSet
{
    public static void main(String args[])
    {
        TreeSet<String> treeset1 = new TreeSet<String>();
        treeset1.add("Item 0");
        treeset1.add("Item 1");
        treeset1.add("Item 2");
        treeset1.add("Item 3");
    }
}
```

```

        treeset1.add("Item 4");
        treeset1.add("Item 6");
        treeset1.add("Item 5");
        System.out.println(treeset1);
    }
}

```

Here's the output of this example (note that, unlike the `HashSet` example in the previous solution, the `TreeSet` elements are sorted in ascending order):

```

C:\>java TreeSet
[Item 0, Item 1, Item 2, Item 3, Item 4, Item 5, Item 6]

```

In fact, you can specify the sorting order for the `TreeSet` objects—see the next solution for the details.

Using the **Comparator** Interface

You can use a comparator to find the sorting order used in the `TreeSet` objects; for example, with the implementation of the `Comparator` interface. The methods of this interface are provided in Table 9.27:

Table 9.27: Methods of the `Comparator` interface

Method	Does this
<code>int compare(Object obj1, Object obj2)</code>	It compares its two arguments.
<code>boolean equals(Object obj)</code>	It specifies whether another object is equal to this comparator.

The `compare()` method compares two objects, `obj1` and `obj2`, and returns `-1` if `obj1` is less than `obj2`; `0` if they are equal; and `1` if `obj1` is greater than `obj2`. By overriding this method, you can support a custom sorting order.

Here's an example in which we sort a `TreeSet` object in ascending order—except for one element, `Item 3`, which we always want to appear at the beginning of the set. Here's what this looks like in code:

```

import java.util.*;

class ComparatorDemo
{
    @SuppressWarnings("unchecked")
    public static void main(String args[])
    {
        TreeSet<String> treeset = new TreeSet<String>(new NewComparator());
        treeset.add("Item 0");
        treeset.add("Item 1");
        treeset.add("Item 2");
        treeset.add("Item 3");
        treeset.add("Item 4");
        treeset.add("Item 5");
        treeset.add("Item 6");
        Iterator iterator = treeset.iterator();
        while(iterator.hasNext()) { System.out.println(iterator.next()); }
    }
}

class NewComparator implements Comparator {
    public int compare(Object obj1, Object obj2) {
        if (((String) obj1).equals("Item 3")) return -1;
        return ((String) obj1).compareTo((String) obj2); }
}

```

Here's the output of this example. Note that `Item 3` comes first:

```

C:\>java ComparatorDemo
Item 3
Item 0
Item 1
Item 2

```



```

Item 4
Item 5
Item 6

```

Using the **Iterator** Interface

The methods of the `Iterator` interface can be used to move through an accumulation using the `next()` method, allowing you to cycle through the elements of the collection. The methods of the `Iterator` interface are shown in Table 9.28:

Table 9.28: Methods of the <code>Iterator</code> interface	
Method	Does this
<code>boolean hasNext()</code>	It returns <code>True</code> if the iteration has more elements.
<code>Object next()</code>	It gets the next element in the iteration.
<code>void remove()</code>	It removes the last element returned.

Here's an example in which we print out each element in a linked list from first to last using an iterator and the methods `next()` and `hasNext()` (which returns `True` if the collection has a next element):

```

import java.util.*;
class IteratorDemo {
public static void main(String args[]) {
LinkedList<String> linkedlist = new LinkedList<String>();
linkedlist.add("Item 0");
linkedlist.add("Item 1");
linkedlist.add("Item 2");
linkedlist.add("Item 3");
linkedlist.add("Item 4");
linkedlist.add("Item 5");
linkedlist.add("Item 6");
Iterator<String> iterator = linkedlist.iterator();
while(iterator.hasNext()) { System.out.println(iterator.next()); }
}
}

```

Here's the output of this example:

```

C:\>java IteratorDemo
Item 0
Item 1
Item 2
Item 3
Item 4
Item 5
Item 6

```

Besides iterators, you can also use list iterators. See the next solution.

Using the **ListIterator** Interface

Whereas iterators can only work forward through a list using the `next()` method, list iterators also support the `previous()` method, so they can work backwards through lists. This is particularly useful for linked lists, where each element often must know about the next and previous elements (called a *bidirectional linked list*). This is handy, for example, when you are implementing a buffer that can grow or shrink. You'll find the methods of the `ListIterator` interface in Table 9.29:

Table 9.29: Methods of the <code>ListIterator</code> interface	
Method	Does this
<code>void add(E e)</code>	It inserts the given element into the list.
<code>boolean hasNext()</code>	It returns <code>True</code> if this list iterator has more elements in the forward direction.
<code>boolean hasPrevious()</code>	It returns <code>True</code> if this list iterator has more elements in the reverse direction.

Table 9.29: Methods of the ListIterator interface

Method	Does this
<code>E next()</code>	It gets the next element in the list.
<code>int nextIndex()</code>	It gets the index of the element that would be returned by a subsequent call to <code>next()</code> .
<code>E previous()</code>	It gets the previous element in the list.
<code>int previousIndex()</code>	It gets the index of the element that would be returned by a subsequent call to <code>previous()</code> .
<code>void remove()</code>	It removes from the list the last element that was returned by <code>next</code> or <code>previous</code> .
<code>void set(E e)</code>	It replaces the last element returned by <code>next</code> or <code>previous</code> with the given element.

Here's an example in which we print a linked list backward:

```
import java.util.*;

class ListIterator
{
    public static void main(String args[])
    {
        LinkedList<String> linkedlist = new LinkedList<String>();
        linkedlist.add("Item 0");
        linkedlist.add("Item 1");
        linkedlist.add("Item 2");
        linkedlist.add("Item 3");
        linkedlist.add("Item 4");
        linkedlist.add("Item 5");
        linkedlist.add("Item 6");
        ListIterator<String> listiterator = linkedlist.listIterator();
        while(listiterator.hasNext())
        {
            listiterator.set("This is " + listiterator.next());
            while(listiterator.hasPrevious()) {
                System.out.println(listiterator.previous());
            }
        }
    }
}
```

Here's the output of this example:

```
C:\>java ListIterator
This is Item 6
This is Item 5
This is Item 4
This is Item 3
This is Item 2
This is Item 1
This is Item 0
```

Here's another example in which we modify and print an arraylist backwards:

```
import java.util.*;

class ListArray {
    public static void main(String args[]) {
        ArrayList<String> arr = new ArrayList<String>();
        arr.add("Item 1");
        arr.add("Item 2");
        arr.add("Item 3");
        arr.add("Item 4");
        arr.add("Item 5");
        arr.add("Item 6");
        System.out.print("The contents of arr:\n");
        Iterator<String> iter = arr.iterator();
        while(iter.hasNext()) {
```

```

        String itrTxt = iter.next();
        System.out.print(itrTxt + " "); }
    System.out.println();
    ListIterator<String> litarr = arr.listIterator();
    while(litarr.hasNext()) {
        String itrTxt = litarr.next();
        litarr.set(itrTxt + " |"); }
    System.out.print("\nContents of arr modified:\n");
    iter = arr.iterator();
    while(iter.hasNext()) {
        String itrTxt = iter.next();
        System.out.print(itrTxt + " "); }
    System.out.println();
    System.out.print("\nContents listed backwards:\n");
    while(litarr.hasPrevious()) {
        String itrTxt = litarr.previous();
        System.out.print(itrTxt + " "); }
    System.out.println(); }
}

```

Here's the result of this example:

```

The contents of arr:
Item 1 Item 2 Item 3 Item 4 Item 5 Item 6

Contents of arr modified:
Item 1 | Item 2 | Item 3 | Item 4 | Item 5 | Item 6 |

Contents listed backwards:
Item 6 | Item 5 | Item 4 | Item 3 | Item 2 | Item 1 |

```

Using the **AbstractMap** Class

The `AbstractMap` class is the base of the map classes in Java, so we'll give a quick glance to refer it. The inheritance diagram for the `AbstractMap` class is as follows:

```

java.lang.Object
|
+--- java.util.AbstractMap

```

You'll find the constructor of the `AbstractMap` class in Table 9.30 and its methods in Table 9.31:

Table 9.30: The constructor of the `AbstractMap` class

Constructor	Does this
<code>protected AbstractMap()</code>	It constructs an <code>AbstractMap</code> object.

Table 9.31: Methods of the `AbstractMap` class

Method	Does this
<code>void clear()</code>	It removes all mappings.
<code>boolean containsKey(Object key)</code>	It returns <code>True</code> if this map contains a mapping for the given key.
<code>boolean containsValue(Object value)</code>	It returns <code>True</code> if this map maps one or more keys to the specified.
<code>abstract Set<Map.Entry<K,V>> entrySet()</code>	It returns a <code>Set</code> .
<code>boolean equals(Object o)</code>	It compares the given object with this map for equality.
<code>V get(Object key)</code>	It gets the value to which the specified key is mapped or <code>null</code> if this map contains no mapping for the given key.
<code>int hashCode()</code>	It gets the <code>hashCode</code> value for this map.
<code>boolean isEmpty()</code>	It returns <code>True</code> if this map contains no key/value mappings.
<code>Set<K> keySet()</code>	It returns a <code>Set</code> .

Table 9.31: Methods of the AbstractMap class

Method	Does this
V put(K key, V value)	It associates the given value with the given key in this map.
void putAll(Map<? extends K, ? extends V> m)	It copies all the mappings from the given map to this map.
Object remove(Object key)	It removes the mapping for a key from this map, if it is present.
int size()	It gets the number of key/value mappings in this map.
String toString()	It gets a string representation of this map.
Collection<V> values()	It returns a Collection.

Using the **HashMap** Class

Maps make it possible to store data as key/value pairs, where both the key and the values are the objects. For example, creation of a map indexed with strings and not numbers. The inheritance diagram for the `HashMap` class is as follows:

```

java.lang.Object
|___java.util.AbstractMap
|   |___java.util.HashMap

```

You'll find the constructors for the `HashMap` class in Table 9.32 and its methods in Table 9.33:

Table 9.32: Constructors of the HashMap class

Constructor	Does this
HashMap()	It constructs a new, empty map.
HashMap(int initialCapacity)	It constructs a new, empty map with the given initial capacity.
HashMap(int initialCapacity, float loadFactor)	It constructs a new, empty map with the given initial capacity and the given load factor.
HashMap(Map<? extends K, ? extends V> m)	It constructs a new map with the same mappings as the given map.

Table 9.33: Methods of the HashMap class

Method	Does this
void clear()	It removes all of the mappings from this map.
Object clone()	It gets a copy of this <code>HashMap</code> instance.
boolean containsKey(Object key)	It returns <code>True</code> if this map contains a mapping for the given key.
boolean containsValue(Object value)	It returns <code>True</code> if this map maps one or more keys to the given value.
Set<Map.Entry<K, V>> entrySet()	It returns a <code>Set</code> .
void forEach(BiConsumer<? super K, ? super V> action)	It performs the given action for each entry in this map until all entries have been processed or the action throws an exception.
V get(Object key)	It gets the value to which the given key is mapped, or <code>null</code> if this map contains no mapping for the key.
V getOrDefault(Object key, V defaultValue)	It returns the value to which the specified key is mapped or <code>defaultValue</code> if this map contains no mapping for the key.
boolean isEmpty()	It returns <code>True</code> if this map contains no key/value mappings.
Set<K> keySet()	It returns a <code>Set</code> .
V put(K key, V value)	It associates the given value with the given key in this map.
void putAll(Map<? extends K, ? extends V> m)	It copies all the mappings from the given map to this map.
V remove(Object key)	It removes the mapping for the specified key from this map if present.

Table 9.33: Methods of the HashMap class

Method	Does this
<code>boolean remove(Object key, Object value)</code>	It removes the entry for the specified key only if it is currently mapped to the specified value.
<code>boolean replace(K key, V oldValue, V newValue)</code>	It replaces the entry for the specified key only if currently mapped to the specified value.
<code>void replaceAll(BiFunction<? super K,? super V,? extends V> function)</code>	It replaces each entry's value with the result of invoking the given function on that entry until all entries have been processed or the function throws an exception.
<code>int size()</code>	It gets the number of key/value mappings in this map.
<code>Collection<V> values()</code>	It gets a Collection.

The `put()` method is used to add a key/value pair to a map, and the `get()` method is used to retrieve a value, given a key. For example, the string `drink` will be left holding the text “root beer” after this example executes:

```
hashmap.put("drink", "root beer");
String drink = hashmap.get("drink");
```

You can also get a set corresponding to the map with the `entrySet()` method, and you can iterate over that set using the `Map.Entry` interface. Here's an example in which we create a map and then print out all the key/value pairs in the map:

```
import java.util.*;

class Hashmap
{
    public static void main(String args[])
    {
        HashMap<String, String> hashmap1 = new HashMap<String, String>();
        hashmap1.put("Item 0", "value 0");
        hashmap1.put("Item 1", "value 1");
        hashmap1.put("Item 2", "value 2");
        hashmap1.put("Item 3", "value 3");
        hashmap1.put("Item 4", "value 4");
        hashmap1.put("Item 5", "value 5");
        hashmap1.put("Item 6", "value 6");
        Set set = hashmap1.entrySet();
        Iterator iterator = set.iterator();
        while(iterator.hasNext())
        {
            Map.Entry mapentry = (Map.Entry) iterator.next();
            System.out.println(mapentry.getKey() + "/" +
                               mapentry.getValue());
        }
    }
}
```

Here's the output of this example:

```
C:\>java Hashmap
Item 0/Value 0
Item 4/Value 4
Item 3/Value 3
Item 2/Value 2
Item 1/Value 1
Item 6/Value 6
Item 5/Value 5
```

The following code iterates through the `System` property map [available through `System.getProperties()`], and displays each key-value pair:

```
import java.util.Map;
import java.util.Properties;
import java.util.Set;
```

```

public class SetEntry
{
    public static void main(String args[])
    {
        Properties props = System.getProperties();
        Set<Map.Entry<Object, Object>> entrySet = props.entrySet();
        for (Map.Entry entry: entrySet)
        {
            System.out.println(entry.getKey() + ": " +
                               entry.getValue());
        }
    }
}

```

Notice the following line:

```
Set<Map.Entry<Object, Object>> entrySet = props.entrySet();
```

A set of Map.Entry objects is returned by the `entrySet()` method of the Properties object that doesn't require any casting operations for the for-each construct.

```
for (Map.Entry entry: entrySet) { . . . }
```

The result is as shown here:

```

C:\> java SetEntry
java.runtime.name: Java(TM) SE Runtime Environment
sun.boot.library.path: C:\Program Files\Java\jre1.8.0_31\bin
java.vm.version: 25.31-b07
java.vm.vendor: Oracle Corporation
java.vendor.url: http://java.oracle.com/
path.separator: ;
java.vm.name: Java HotSpot(TM) 64-Bit Server VM
file.encoding.pkg: sun.io
user.country: IN
user.script:
sun.java.launcher: SUN_STANDARD
sun.os.patch.level:
java.vm.specification.name: Java Virtual Machine Specification
user.dir: E:\Java8
java.runtime.version: 1.8.0_31-b13
java.awt.graphicsenv: sun.awt.win32GraphicsEnvironment
java.endorsed.dirs: C:\Program Files\Java\jre1.8.0_31\lib\endorsed
os.arch: amd64
java.io.tmpdir: C:\Users\DEEPAK~1\AppData\Local\Temp\
line.separator:
java.vm.specification.vendor: Oracle Corporation
user.variant:
os.name: Windows 8.1
sun.jnu.encoding: Cp1252
java.library.path: C:\ProgramData\Oracle\Java\javapath;C:\Windows\Sun\Java\bin;C:\Windows\system32;C:\Windows;C:\ProgramData\Oracle\Java\javapath;C:\Program Files\Java\jdk1.8.0_25\bin;;
java.specification.name: Java Platform API Specification
java.class.version: 52.0
sun.management.compiler: HotSpot 64-Bit Tiered Compilers
os.version: 6.3
user.home: C:\Users\Deepak Sharma
user.timezone:
java.awt.printerjob: sun.awt.windows.WPrinterJob
file.encoding: Cp1252
java.specification.version: 1.8
java.class.path: .
user.name: Deepak Sharma
java.vm.specification.version: 1.8
sun.java.command: SetEntry
java.home: C:\Program Files\Java\jre1.8.0_31
sun.arch.data.model: 64

```

```

user.language: en
java.specification.vendor: Oracle Corporation
awt.toolkit: sun.awt.windows.WToolkit
java.vm.info: mixed mode
java.version: 1.8.0_31
java.ext.dirs: C:\Program Files\Java\jre1.8.0_31\lib\ext;C:\Windows\Sun\Java\lib\ext
sun.boot.class.path: C:\Program Files\Java\jre1.8.0_31\lib\resources.jar;C:\Program Files\Java\jre1.8.0_31\lib\rt.jar;C:\Program Files\Java\jre1.8.0_31\lib\sunr
sasign.jar;C:\Program Files\Java\jre1.8.0_31\lib\jsse.jar;C:\Program Files\Java\jre1.8.0_31\lib\jce.jar;C:\Program Files\Java\jre1.8.0_31\lib\charsets.jar;C:\Pr
ogram Files\Java\jre1.8.0_31\lib\jfr.jar;C:\Program Files\Java\jre1.8.0_31\class
es
java.vendor: Oracle Corporation
sun.stderr.encoding: cp437
file.separator: \
java.vendor.url.bug: http://bugreport.sun.com/bugreport/
sun.io.unicode.encoding: UnicodeLittle
sun.cpu.endian: little
sun.stdout.encoding: cp437
sun.desktop: windows
sun.cpu.isalist: amd64

```

Using the **TreeMap** Class

The `TreeMap` class executes a map with the use of a tree internally to store data. The inheritance diagram for this class is as follows:

```

java.lang.Object
|___java.util.AbstractMap
|   ___java.util.TreeMap

```

You'll find the constructors for the `TreeMap` class in Table 9.34 and its methods in Table 9.35:

Table 9.34: Constructors of the `TreeMap` class

Constructor	Does this
<code>TreeMap()</code>	It constructs a new, empty tree map, using natural ordering of its keys.
<code>TreeMap(Comparator<? super K> comparator)</code>	It constructs a new, empty tree map, ordered according to the given comparator.
<code>TreeMap(Map<? extends K, ? extends V> m)</code>	It constructs a new tree map containing the same mappings as the given map, ordered according to the natural ordering of its keys.
<code>TreeMap(SortedMap<K, ? extends V> m)</code>	It constructs a new tree map containing the same mappings and using the same ordering as the given sorted map.

Table 9.35: Methods of the `TreeMap` class

Method	Does this
<code>Map.Entry(K,V) ceilingEntry(K key)</code>	It returns a key-value mapping associated with the least key greater than or equal to the given key, or null if there is no such key.
<code>K ceilingKey(K key)</code>	It returns the least key greater than or equal to the given key, or null if there is no such key.
<code>void clear()</code>	It removes all of the mappings from this map.
<code>Object clone()</code>	It gets a copy of this <code>TreeMap</code> instance.
<code>Comparator<? super K> comparator()</code>	It gets the comparator used to order the keys in this map, or null if this map uses the natural ordering.
<code>boolean containsKey(Object key)</code>	It returns True if this map contains a mapping for the specified key.
<code>boolean containsValue(Object value)</code>	It returns True if this map maps one or more keys to the specified value.

Table 9.35: Methods of the TreeMap class

Method	Does this
<code>NavigableSet<K> descendingKeySet()</code>	It returns a reverse order <code>NavigableSet</code> view of the keys contained in this map.
<code>NavigableMap<K,V> descendingMap()</code>	It returns a reverse order view of the mappings contained in this map.
<code>Set<Map.Entry<K,V>> entrySet()</code>	It returns a <code>Set</code> .
<code>Map.Entry<K,V> firstEntry()</code>	It returns a key-value mapping associated with the least key in this map, or null if the map is empty.
<code>K firstKey()</code>	It returns the first (lowest) key currently in this map.
<code>Map.Entry<K,V> floorEntry(K key)</code>	It returns a key-value mapping associated with the greatest key less than or equal to the given key, or null if there is no such key.
<code>K floorKey(Object key)</code>	It returns the greatest key less than or equal to the given key, or null if there is no such key.
<code>void forEach(BiConsumer<? super K, ? super V> action)</code>	It performs the given action for each entry in this map until all entries have been processed or the action throws an exception.
<code>V get(Object key)</code>	It gets the value to which the specified key is mapped, or null if this map contains no mapping for the key.
<code>SortedMap<K,V> headMap(K toKey)</code>	Equivalent to <code>navigableHeadMap(Object toKey)</code> <code>SortedMap</code> interface.
<code>Map.Entry<K,V> higherEntry(K key)</code>	It returns a key-value mapping associated with the least key strictly greater than the given key or null if there is no such key.
<code>K higherKey(K key)</code>	It returns the least key strictly greater than the given key or null if there is no such key.
<code>Set<K> keySet()</code>	It returns a <code>Set</code> .
<code>Map.Entry<K,V> lastEntry()</code>	It returns a key-value mapping associated with the greatest key in this map, or null if the map is empty.
<code>K lastKey()</code>	It gets the last key currently in this map.
<code>Map.Entry< K key, V value > lowerEntry(K key)</code>	It returns a key-value mapping associated with the greatest key strictly less than the given key, or null if the map is empty.
<code>K lowerKey(K key)</code>	It returns the greatest key strictly less than the given key, or null if there is no such key.
<code>SortedMap<K,V> headMap(K toKey)</code>	It returns a view of the portion of this map whose keys are strictly less than <code>toKey</code> .
<code>SortedMap<K,V> subMap(K fromKey, K toKey)</code>	It returns a view of the portion of this map whose keys range from <code>fromKey</code> (inclusive) to <code>toKey</code> (exclusive).
<code>SortedMap<K,V> tailMap(K fromKey)</code>	It returns a view of the portion of this map whose keys are greater than or equal to <code>fromKey</code> .
<code>Map.Entry<K key, V value> pollFirstEntry()</code>	It removes and returns a key-value mapping associated with the least key in this map, or null if the map is empty.
<code>Map.Entry<K,V> pollLastEntry()</code>	It removes and returns a key-value mapping associated with the greatest key in this map, or null if the map is empty.
<code>V put(K key, Object V)</code>	It associates the specified value with the specified key in this map.
<code>void putAll (Map<? extends K, ? extends V> map)</code>	It copies all the mappings from the specified map to this map.
<code>V remove(Object key)</code>	It removes the mapping for this key.
<code>int size()</code>	It gets the number of key/value mappings.
<code>SortedMap<K,V> subMap(K fromKey, K toKey)</code>	Equivalent to <code>navigableSubMap (Object fromKey, Object toKey)</code> <code>SortedMap</code> interface.

Table 9.35: Methods of the TreeMap class

Method	Does this
<code>SortedMap<K,V> tailMap(K fromKey)</code>	Equivalent to <code>navigableTailMap (Object fromKey)</code> <code>SortedMap</code> interface.
<code>Collection<V> values()</code>	It returns a <code>Collection</code> .

Using the **Arrays** Class

The `Arrays` class can be used to work with arrays, including sorting and searching arrays. The inheritance diagram for this class is as follows:

```

java.lang.Object
|
+---java.util.Arrays

```

You'll find the methods for the `Arrays` class in Table 9.36:

Table 9.36: Methods of the Arrays class

Method	Does this
<code>static <T> List<T> asList(T... a)</code>	It gets a fixed-size list.
<code>static int binarySearch (byte[] a, byte key)</code>	It searches the given array of bytes for the given value.
<code>static int binarySearch (char[] a, char key)</code>	It searches the given array of characters for the given value.
<code>static int binarySearch (double[] a, double key)</code>	It searches the given array of doubles for the given value.
<code>static int binarySearch (float[] a, float key)</code>	It searches the given array of floats for the given value.
<code>static int binarySearch (int[] a, int key)</code>	It searches the given array of ints for the given value.
<code>static int binarySearch (long[] a, long key)</code>	It searches the given array of longs for the given value.
<code>static int binarySearch (Object[] a, Object key)</code>	It searches the given array for the given object.
<code>static <T> int binarySearch(T[] a, T key, Comparator<? super T> c)</code>	It searches the given array for the given object using a comparator.
<code>static int binarySearch (short[] a, short key)</code>	It searches the given array of shorts for the given value.
<code>static boolean[] copyOf (boolean[] original, int newLength)</code>	It copies the specified array, truncating or padding with false (if necessary) so the copy has the specified length.
<code>static byte[] copyOf(byte[] original, int newLength)</code>	It copies the specified array, truncating or padding with zeros (if necessary) so the copy has the specified length.
<code>static char[] copyOf(char[] original, int newLength)</code>	It copies the specified array, truncating or padding with null characters (if necessary) so the copy has the specified length.
<code>static double[] copyOf (double[] original, int newLength)</code>	It copies the specified array, truncating or padding with zeros (if necessary) so the copy has the specified length.
<code>static float[] copyOf (float[] original, int newLength)</code>	It copies the specified array, truncating or padding with zeros (if necessary) so the copy has the specified length.
<code>static int[] copyOf(int[] original, int newLength)</code>	It copies the specified array, truncating or padding with zeros (if necessary) so the copy has the specified length.
<code>static long[] copyOf(long[] original, int newLength)</code>	It copies the specified array, truncating or padding with zeros (if necessary) so the copy has the specified length.

Table 9.36: Methods of the Arrays class

Method	Does this
static short[] copyOf (short[] original, int newLength)	It copies the specified array, truncating or padding with zeros (if necessary) so the copy has the specified length.
static boolean equals (boolean[] a, boolean[] a2)	It returns True if the two given arrays of booleans are equal.
static boolean equals (byte[] a, byte[] a2)	It returns True if the two given arrays of bytes are equal to one another.
static boolean equals (char[] a, char[] a2)	It returns True if the two given arrays of characters are equal to one another.
static boolean equals (double[] a, double[] a2)	It returns True if the two given arrays of doubles are equal.
static boolean equals (float[] a, float[] a2)	It returns True if the two given arrays of floats are equal.
static boolean equals (int[] a, int[] a2)	It returns True if the two given arrays of ints are equal.
static boolean equals (long[] a, long[] a2)	It returns True if the two given arrays of longs are equal.
static boolean equals (Object[] a, Object[] a2)	It returns True if the two given arrays of objects are equal.
static boolean equals (short[] a, short[] a2)	It returns True if the two given arrays of shorts are equal to one another.
static void fill (boolean[] a, boolean val)	It assigns the given boolean value to each element of the given array of Booleans.
static void fill(boolean[] a, int fromIndex, int toIndex, boolean val)	It assigns the given boolean value to each element of the given range of the given array of Booleans.
static void fill(byte[] a, byte val)	It assigns the given byte value to each element of the given array of bytes.
static void fill(byte[] a, int fromIndex, int toIndex, byte val)	It assigns the given byte value to each element of the given range of the given array of bytes.
static void fill(char[] a, char val)	It assigns the given character value to each element of the given array of characters.
static void fill(char[] a, int fromIndex, int toIndex, char val)	It assigns the given character value to each element of the given range of the given array of characters.
static void fill(double[] a, double val)	It assigns the given double value to each element of the given array of doubles.
static void fill(double[] a, int fromIndex, int toIndex, double val)	It assigns the given double value to each element of the given range of the given array of doubles.
static void fill(float[] a, float val)	It assigns the given float value to each element of the given array of floats.
static void fill(float[] a, int fromIndex, int toIndex, float val)	It assigns the given float value to each element of the given range of the given array of floats
static void fill(int[] a, int val)	It assigns the given int value to each element of the given array of ints.
static void fill(int[] a, int fromIndex, int toIndex, int	It assigns the given int value to each element of the given range of the

Table 9.36: Methods of the Arrays class

Method	Does this
<code>val)</code>	given array of ints.
<code>static void fill(long[] a, int fromIndex, int toIndex, long val)</code>	It assigns the given long value to each element of the given range of the given array of longs.
<code>static void fill(long[] a, long val)</code>	It assigns the given long value to each element of the given array of longs
<code>static void fill(Object[] a, int fromIndex, int toIndex, Object val)</code>	It assigns the given object reference to each element of the given range of the given array of objects.
<code>static void fill(Object[] a, Object val)</code>	It assigns the given object reference to each element of the given array of objects.
<code>static void fill(short[] a, int fromIndex, int toIndex, short val)</code>	It assigns the given short value to each element of the given range of the given array of shorts.
<code>static void fill(short[] a, short val)</code>	It assigns the given short value to each element of the given array of shorts.
<code>static void sort(byte[] a)</code>	It sorts the given array of bytes into ascending numerical order.
<code>static void sort(byte[] a, int fromIndex, int toIndex)</code>	It sorts the given range of the given array of bytes into ascending numerical order.
<code>static void sort(char[] a)</code>	It sorts the given array of characters into ascending numerical order.
<code>static void sort(char[] a, int fromIndex, int toIndex)</code>	It sorts the given range of the given array of characters into ascending numerical order.
<code>static void sort(double[] a)</code>	It sorts the given array of doubles into ascending numerical order.
<code>static void sort(double[] a, int fromIndex, int toIndex)</code>	It sorts the given range of the given array of doubles into ascending numerical order.
<code>static void sort(float[] a)</code>	It sorts the given array of floats into ascending numerical order.
<code>static void sort(float[] a, int fromIndex, int toIndex)</code>	It sorts the given range of the given array of floats into ascending numerical order.
<code>static void sort(int[] a)</code>	It sorts the given array of ints into ascending numerical order.
<code>static void sort(int[] a, int fromIndex, int toIndex)</code>	It sorts the given range of the given array of ints into ascending numerical order.
<code>static void sort(long[] a)</code>	It sorts the given array of longs into ascending numerical order.
<code>static void sort(long[] a, int fromIndex, int toIndex)</code>	It sorts the given range of the given array of longs into ascending numerical order.
<code>static void sort(Object[] a)</code>	It sorts the given array of objects into ascending order, according to the natural ordering of its elements.
<code>static void sort(Object[] a, Comparator c)</code>	It sorts the given array of objects according to the order induced by the given comparator.
<code>static void sort(Object[] a, int fromIndex, int toIndex)</code>	It sorts the given range of the given array of objects into ascending order, according to the natural ordering of its elements.
<code>static void sort(Object[] a, int fromIndex, int toIndex, Comparator c)</code>	It sorts the given range of the given array of objects according to the order induced by the given comparator.
<code>static void sort(short[] a)</code>	It sorts the given array of shorts into ascending numerical order.

Table 9.36: Methods of the Arrays class

Method	Does this
<code>static void sort(short[] a, int fromIndex, int toIndex)</code>	It sorts the given range of the given array of shorts into ascending numerical order.

Here's an example in which we create an array, print it out, sort it, and then search for a particular element:

```
import java.util.*;
class ArrayDemo
{
    public static void main(String args[])
    {
        int array[] = new int[10];
        for(int loop_index = 9; loop_index > 0; loop_index--)
            array[loop_index] = -loop_index;
        for(int loop_index = 0; loop_index < array.length; loop_index++)
            System.out.print(array[loop_index] + " ");
        System.out.println();
        Arrays.sort(array);
        for(int loop_index = 0; loop_index < array.length; loop_index++)
            System.out.print(array[loop_index] + " ");
        System.out.println();
        System.out.print("Found -5 at position " +
            Arrays.binarySearch(array, -5));
    }
}
```

Note that these are static methods and can be used without instantiating an object of this class.

Here's the output of this example:

```
C:\>java ArrayDemo
0 -1 -2 -3 -4 -5 -6 -7 -8 -9
-9 -8 -7 -6 -5 -4 -3 -2 -1 0
Found -5 at position 4
```

Learning the Fundamentals of Enumerations

Let's talk about the basics of enumerations in detail. In the original version of Java, one feature is absent and many programmers felt that this feature is one of the vital features in a code.

Many programmers felt that this was needed very much. This is known as Enumeration. In its simplest form, you can define Enumeration as a list of named constants.

An Enumeration type can be considered to be a special form of class while defining an enumeration type at the time of writing code. The specified enumeration constant gets created as objects of the class that contains the enum class. This class is defined in the `java.lang` package as a superclass. The object corresponding to each enumeration constants stores the constant's name in a field. The `toString()` method is inherited by the enumeration class type from the `enum` class.

Enumeration in Java is different in existence from other languages, though it seems to be similar. In C++, enumeration is a list of named integer constants, while in Java, enumeration defines a class type. This concept is elaborated by making enumeration into classes.

Using the new `enum` keywords, an Enumeration is created. For example, given here is a simple enumeration that lists various Wheat varieties:

```
enum Wheat { Durum, Spelt, Commonwheat, Emmer }
```

Enumeration constants are the identifiers such as `Durum`, `Spelt`, and so on each of which is implicitly declared as a `public` and `static` member of `Wheat`. In addition to this, their type is the type of enumeration in which they are declared, which is `Wheat` in this case. These constants in Java are known as self-typed which self corresponds to the enclosed enumeration.

You can create a variable of enumeration type once you have defined enumeration. However, it is found that, even though enumeration defines a class type, you do not instantiate an `enum` by using a `new`. Instead, you

declare and use an enumeration variable in much the same way as you have done in the case of the primitive types. For example, the code given here declares `a1` as a variable of enumeration type `Wheat`:

```
wheat a1 ;
```

The only values that `a1` can assign are those defined by the enumeration, because `a1` is of type `Wheat`. For example, the value `Emmer` is assigned to `a1` by this:

```
a1 = wheat.Emmer;
```

The point to note here is that the symbol `Emmer` is preceded by `Wheat`. The `==` relational operator can be used for comparing the two enumeration constants for equality.

For example, this statement compares the value in `a1` with the `Spelt` constant:

```
if(a1 == wheat.Spelt) //...
```

An enumeration value can also be used for controlling a switch statement. Actually, it is very necessary that all of the `case` statements must use constants from the same `enum` as that used by the `switch` expression. For example, this `switch` is perfectly valid:

```
switch (a1) {
    case Spelt:
        case CommonWheat :
```

In the `case` statements, you can note that the names of the enumeration constants can be used without being qualified by the name of the enumeration type. For example, `CommonWheat` is used, but not `Wheat.CommonWheat`. The reason behind this is that the type of the enumeration in the `switch` expression has already implicitly declared the `enum` type of the `case` constants. So, for `case` statement, there is no requirement for the constants to be passed with their `enum` type name. In fact, a compilation error will occur if you will do so. When an enumeration constant is shown, such as in a `println()` statement, the result will be its name. For example, in the given statement:

```
System.out.println(Wheat.Durum);
```

The name `Durum` is displayed as output. The code explained here will put together all the pieces and examine the `Wheat` enumeration:

```
enum wheat { Emmer,Durum, Commonwheat, Spelt }
class EnumDemo
{
    public static void main (String args[])
    {
        wheat a1;
        a1 = wheat.Emmer;
        System.out.println("Value of a1 is : " + a1);
        System.out.println();
        a1 = wheat.Emmer;
        if (a1 == wheat.Spelt)
        {
            System.out.println ("a1 contains Spelt.\n");
        }
        switch(a1) {
        case Emmer:
            System.out.println("It is ancient time wheat."); break;
        case Durum:
            System.out.println("It is second most widely cultivated wheat.");
            break;
        case Commonwheat:
            System.out.println("It is most wildy cultivated wheat."); break;
        case Spelt:
            System.out.println("It is found in limited quantities."); break;
        }
    }
}
```

Here's the output is as follows:

```
Value of a1 is: Emmer
It is ancient time wheat.
```

The **values()** and **valueOf()** Methods

Two predefined methods, namely, `values()` and `valueOf()`, are present automatically in all enumeration. Given are their general forms. One form is:

```
public static enum-type[] values()
```

And the other is:

```
public static enum-type valueOf(String str)
```

The `values()` method returns an array containing a set of the enumeration constants. On the other hand, the enumeration constant whose value corresponds to the string passed in `str` is returned by the `valueOf()` method. In both cases it is found that `enum-type` is the type of the enumeration. For example, in the case of the `Wheat` enumeration shown earlier, the return type of `Wheat.valueOf("Durum")` is `Durum`. The `values()` and `valueOf()` methods are demonstrated by the following code:

```
enum Wheat { Emmer, Durum, Commonwheat, Spelt }
class WheatC
{
    public static void main (String args[])
    {
        Wheat a1;
        System.out.println("Here are all wheat constant:");
        Wheat allwheat[] = Wheat.values();
        for(Wheat a : allwheat)
            System.out.println(a);
        System.out.println();
        a1 = Wheat.valueOf("Durum");
        System.out.println("a1 contains " + a1);
    }
}
```

Here's the output of the code as follows:

```
Here are all wheat constants:
Emmer
Durum
Commonwheat
Spelt

a1 contains Durum
```

The point to be noted here is that this code uses a `for-each` style for loop to cycle through the array of constants which can be obtained by calling `values()`. For better understanding, the variable `allwheat` was formed and thus assigned a reference to the enumeration array. However, this step is not necessary to explain. Because, by another way, the `for` could have been written as shown here by eliminating the need for the `allwheat` variable:

```
for (Wheat a : Wheat.values()) { System.out.println(a); }
```

Now, you have to notice that, by calling `valueOf()`, the value corresponding to the name `Durum` is obtained:

```
a1 = Wheat.valueOf ("Durum");
```

We have discussed in the preceding topic that `valueOf()` returns the enumeration value related to the name of the constant which is denoted as a string.

Java Enumeration as a Class Type

As already explained, the enumeration of Java is a class type. Although you do not instantiate an `enum` using `new`, it has much more the same capabilities as other classes have. Here, the important fact is that `enum` defines a class that gives power to the Java enumeration. However, as far as other codes are concerned, that enumeration simply does not have this priority. For example, you can give them constructors, add instance variables and methods, and even implement interfaces.

You must keep in mind that each enumeration constant is an instance of its own enumeration type. Thus, at the time when you define a constructor for an `enum`, the constructor is called only after each enumeration constant is

created. Moreover, each enumeration constant contains its own copy of an instance variable, which is defined by the enumeration. For example, see the following version of the `Mango`:

```
enum Mango {
    Brooks(10), Manilla(9), Alphanso(12), Kesar(15), Maya(8);
    private int price;
    Mango(int p) { price = p;}
    int getPrice() { return price;}
}
class Apple5
{
    public static void main(String args[])
    {
        Mango ap;
        System.out.println("Kesar costs "+Mango.Kesar.getPrice()+ " cents. \n");
        System.out.println("All mango prices:");
        for(Mango a : Mango.values())
            System.out.println(a + " costs " + a.getPrice() + " cents.");
    }
}
```

The output of the code is as follows:

```
Kesar costs 15 cents.

All mango prices:
Brooks costs 10 cents.
Manilla costs 9 cents.
Alphanso costs 12 cents.
Kesar costs 15 cents.
Maya costs 8 cents.
```

Here, the version of `Mango` adds three things to it. The first one which is used to hold the price of each variety of `Mango` is the instance variable `price`. The second one is the `Mango` constructor, which is to pass the price of a `Mango`. And finally, the third one which returns the value of `price` is the method `getPrice()`.

When the variable `ap` is declared in `main()`, then the constructor for `Mango` is called once for each of the constant which is specified. You have to notice how the argument to the constructor is specified, by putting them inside parentheses after each constant, as shown here:

```
Brooks(10), Manilla(9), Alphanso(12), Kesar(15), Maya(8);
```

Here, these values are passed to the `p` parameter of `Mango()`, and after that, it assigns this value to `price`. Again, for each of the constant, the constructor is called only once. The reason being that each enumeration constant has its own copy of `price`. In order to obtain the price of a specified type of `Mango`, you can have it by calling `getPrice()`. For example, in `main()`, the price of a `Kesar` is obtained by the following call:

```
Mango.Kesar.getPrice()
```

In order to obtain the prices of all varieties, you have to go by cycling through the enumeration using a `for` loop. The reason is, there is a copy of `price` for each enumeration constant, and also the value associated with one constant is separate and distinct from the value associated with another constant. As you have already noticed, this is a powerful concept. This concept is only available when enumeration is implemented as classes, as Java does.

In the preceding example, there is only one constructor, but an `enum` can provide multiple overloaded forms, like it provides in any other class. For example, this version of `Mango` provides a default constructor that initializes the price to `-1`, just to indicate that there is no price data available here:

```
enum Mango
{
    Brooks(10), Manilla(9), Alphanso(), Kesar(15), Maya(8);
    private int price;
    Mango(int p) { price = p; }
    Mango() {price = -1; }
    int getPrice() { return price; }
}
```

However, please note that in this version, `Alphanso` is not given as an argument. This means that the default constructor is called and the value `-1` has been given to `Alphanso`'s `price` variable.

Let's mention about two different restrictions that apply to the enumeration. The first restriction to remember is that an enumeration can't inherit another class. The second restriction is, an `enum` cannot be a superclass. To make it simpler, an `enum` can't be extended.

Enumeration Inheriting Enum

We know that at declaration time for `enum`, you can't inherit a superclass. Therefore, all enumerations automatically inherit one: `java.lang.Enum`. This class defines various methods that are meant to be used by all enumerations. The constructor and methods are shown in Table 9.37 and Table 9.38, respectively:

Table 9.37: Showing the constructor of the Enum

Constructor	Does this
<code>protected Enum(String name, int ordinal)</code>	The only constructor

Table 9.38: Showing the methods of the Enum

Methods	Does this
<code>protected Object clone()</code>	It throws <code>CloneNotSupportedException</code> .
<code>int compareTo(E o)</code>	It compares this <code>enum</code> with the specified object for order.
<code>boolean equals(Object other)</code>	It returns true if the specified object is equal to this <code>enum</code> constant.
<code>protected void finalize()</code>	<code>Enum</code> classes cannot have finalized methods.
<code>Class<E> getDeclaringClass()</code>	It returns the <code>Class</code> object corresponding to this <code>enum</code> constant's <code>enum</code> type.
<code>int hashCode()</code>	It returns a hash code for this <code>enum</code> constant.
<code>String name()</code>	It returns the name of this <code>enum</code> constant, exactly as declared in its <code>enum</code> declaration.
<code>int ordinal()</code>	It returns the ordinal of this enumeration constant (its position in its <code>enum</code> declaration, where the initial constant is assigned an ordinal of zero).
<code>String toString()</code>	It returns the name of this <code>enum</code> constant, as contained in the declaration.
<code>static valueOf(Class<T> enumType, String name)</code>	It returns the <code>enum</code> constant of the specified <code>enum</code> type with the specified name.

Questions that arise over here are what do you mean by ordinal value and how can you achieve it? You can obtain a value that symbolizes an enumeration constant's position in the list of constants. This is obtained by calling the `ordinal()` method as follows:

```
final int ordinal()
```

The preceding statement returns the ordinal value of the invoking constant. Note that ordinal values begin at Zero. Thus, `Brooks` has an ordinal value of Zero, `Manilla` has an ordinal value of 1, `Alphanso` has an ordinal value of 2, and so on and so forth in the `Mango` enumeration.

You can perform comparison between the ordinal values of two constants of the similar enumeration by using the `compareTo()` method. It has this general form:

```
final int compareTo(enum-type e)
```

where `enum-type` is the type of enumeration and `e` is the constant that is compared to the constant being requested. Also keep in mind that both the requested constant and `e` must be of the same enumeration. If the invoking constant has an ordinal value less than `e`'s, then `compareTo()` returns a negative value. Moreover, if the two ordinal values are similar, then zero is returned, and if the value of the invoking constant is greater than `e`'s, then `compareTo()` returns a positive value.

To obtain the equality, you can compare an enumeration constant with any other object by using `equals()`, which overrides the `equal()` method defined by `Object`. Although `equals()` can compare an enumeration

constant to any other object, those two objects will remain equal if both of them refer to the same constant, that too within the same enumeration. For better understanding, make a note that having ordinal values in common will not cause `equals()` to return true if the two constants are from different enumerations.

As discussed earlier, you can perform the comparison between two enumeration references for equality by using the `==` operator. You will get it by the following code which examines the `ordinal()`, `compareTo()`, and `equals()` methods:

```
enum Mango { Brooks, Manilla, Alphanso, Kesar, Maya }
class Apple6
{
    public static void main (String args[])
    {
        Mango ap, ap2, ap3;
        System.out.println("Here are all mango constants" + " and their
        ordinal values: ");
        for (Mango a : Mango.values())
            System.out.println(a + " " + a.ordinal());
        ap = Mango.Alphanso;
        ap2 = Mango.Manilla;
        ap3 = Mango.Alphanso;
        System.out.println();
        if(ap.compareTo(ap2) < 0)
            System.out.println(ap + " comes before " + ap2);
        if(ap.compareTo(ap2) > 0)
            System.out.println(ap2 + " comes before " + ap);
        if(ap.compareTo(ap2) == 0)
            System.out.println(ap + " equals " + ap3);
        System.out.println();
        if(ap.equals(ap2)) { System.out.println("Error"); }
        if(ap.equals(ap3)) { System.out.println("Error"); }
        if(ap == ap3) { System.out.println(ap + " = " + ap3); }
    }
}
```

Here's the output of the code:

```
Here are all mango constants and their ordinal values:
Brooks 0
Manilla 1
Alphanso 2
Kesar 3
Maya 4

Manilla comes before Alphanso
Error
Alphanso = = Alphanso
```

The **Enumeration** Interface

You use the `Enumeration` interface's methods to loop over elements in classes such as `Hashtable`. This interface is a useful one. For now, we'll list its methods for reference. You'll find the methods for the `Enumeration` interface in Table 9.39:

Table 9.39: Methods of the Enumeration interface	
Method	Does this
<code>boolean hasMoreElements()</code>	It returns True if this enumeration contains more elements.
<code>Object nextElement()</code>	It gets the next element of this enumeration.

The **Legacy** Classes and Interfaces

Talking about earlier versions of Java, legacy classes and interfaces formed the collections framework and now they are reconstructed in coming versions. With the Java SE 7 release, for legacy file I/O code, the `java.io.File` class was the process used for file I/O; however, it has the following demerits:

- ❑ Many methods didn't throw exceptions when they failed, so it was impossible to retrieve a useful error message.
- ❑ The rename method didn't work properly across platforms.
- ❑ There was no real support for symbolic links.
- ❑ Needed more support for metadata, such as file permissions, file owner, and other security attributes.
- ❑ Executing file metadata was inefficient.
- ❑ It was impossible to write a reliable code that could iteratively walk a file tree and result back accurately if there were circular symbolic links.

So, you have the legacy code that uses `java.io.File` and would prefer taking benefit of the `java.nio.file.Path` functionality with minimal impact on your code.

The various legacy classes defined by `java.util` package are:

- ❑ Dictionary
- ❑ Hashtable
- ❑ Stack
- ❑ Vector
- ❑ Properties

The *Java Native Interface* is developed for using Java in order to encapsulate native legacy code on small embedded platforms. We must know why existing technologies for encapsulating legacy code, Java Native Interface (JNI), is not enough for an important range of small embedded platforms, and we depict how the JNI provides previously missing functionality.

The **Vector** Class

The `Vector` class predates the collection framework, and it implements a dynamic array. Because of the appearance of the collection framework, `Vector` has been rewritten to be compatible with it. Here's the inheritance diagram for `Vector`:

```

java.lang.Object
|
+--- java.util.AbstractCollection
|      |
|      +--- java.util.AbstractList
|             |
|             +--- java.util.Vector

```

You'll find the fields of the `Vector` class in Table 9.40, its constructors in Table 9.41, and its methods in Table 9.42:

Table 9.40: Fields of the Vector class	
Field	Does this
<code>protected int capacityIncrement</code>	This is the amount by which the capacity of the vector is increased when its size exceeds its capacity.
<code>protected int elementCount</code>	Total number of components in this <code>Vector</code> object.
<code>protected Object[] elementData</code>	The array where the components of the vector are stored.

Table 9.41: Constructors of the Vector class	
Constructor	Does this
<code>Vector()</code>	It constructs an empty vector.
<code>Vector(Collection c)</code>	It constructs a vector containing the elements of the given collection.
<code>Vector(int initialCapacity)</code>	It constructs an empty vector with the given initial capacity.
<code>Vector(int initialCapacity, int capacityIncrement)</code>	It constructs an empty vector with the given initial capacity and capacity increment.

Table 9.42: Methods of the Vector class

Method	Does this
<code>void add(int index, Object element)</code>	It inserts the given element.
<code>boolean add(Object o)</code>	It adds the given element to the end of this vector.
<code>boolean addAll(Collection c)</code>	It adds all the elements in the given collection to the end of this vector in the order that they are returned by the given collection's iterator.
<code>boolean addAll(int index, Collection c)</code>	It inserts all the elements in the given collection into this vector at the given position.
<code>void addElement(Object obj)</code>	It adds the given component to the end of this vector, increasing its size by one.
<code>int capacity()</code>	It gets the current capacity of this vector.
<code>void clear()</code>	It removes all the elements from this vector.
<code>Object clone()</code>	It gets a clone of this vector.
<code>boolean contains (Object elem)</code>	It returns True if this vector contains the specified element.
<code>boolean containsAll (Collection c)</code>	It returns True if this vector contains all the elements in the given collection.
<code>void copyInto(Object[] anArray)</code>	It copies the components of this vector into the given array.
<code>Object elementAt(int index)</code>	It gets the component at the given index.
<code>Enumeration elements()</code>	It gets an enumeration of the components of this vector.
<code>void ensureCapacity (int minCapacity)</code>	It increases the capacity of this vector, if necessary.
<code>boolean equals(Object o)</code>	It compares the given object with this vector for equality.
<code>Object firstElement()</code>	It gets the first component of this vector.
<code>Object get(int index)</code>	It gets the element at the given position in this vector.
<code>int hashCode()</code>	It gets the hashcode value for this vector.
<code>int indexOf(Object elem)</code>	It returns the index of the first occurrence of the given element. In this vector, or -1 if this vector does not contain element.
<code>int indexOf (Object elem, int index)</code>	It searches for the first occurrence of the given argument, beginning the search at index.
<code>void insertElementAt (Object obj, int index)</code>	It inserts the given object as a component in this vector at the given index.
<code>boolean isEmpty()</code>	It tests whether this vector has no components.
<code>Object lastElement()</code>	It gets the last component of the vector.
<code>int lastIndexOf (Object elem)</code>	It gets the index of the last occurrence of the specified element in this vector or -1 if this vector does not contain the element.
<code>int lastIndexOf (Object elem)</code>	It returns the index of the last occurrence of the specified element in this vector, searching backwards from index, or returns -1 if the element is not found.
<code>int lastIndexOf (Object elem, int index)</code>	It searches backwards for the given object and returns an index to it.
<code>Object remove(int index)</code>	It removes the element at the given position in this vector.
<code>boolean remove(Object o)</code>	It removes the first occurrence of the given element.
<code>boolean removeAll (Collection c)</code>	It removes from this vector all its elements that are contained in the given collection.
<code>void removeAllElements()</code>	It removes all components from this vector and sets its size to zero.

Table 9.42: Methods of the Vector class

Method	Does this
<code>boolean removeElement(Object obj)</code>	It removes the first occurrence of the argument from this vector.
<code>void removeElementAt(int index)</code>	It deletes the component at the given index.
<code>protected void removeRange(int fromIndex, int toIndex)</code>	It removes from this list all the elements whose indexes are between <code>fromIndex</code> and <code>toIndex</code> .
<code>boolean retainAll(Collection c)</code>	It keeps only the elements in this vector that are contained in the given collection.
<code>Object set(int index, Object element)</code>	It replaces the element at the given position in this vector with the given element.
<code>void setElementAt(Object obj, int index)</code>	It sets the component at the given index of this vector to be the given object.
<code>void setSize(int newSize)</code>	It sets the size of this vector.
<code>int size()</code>	It gets the number of components in this vector.
<code>List subList(int fromIndex, int toIndex)</code>	It gets a view of the portion of this list between <code>fromIndex</code> and <code>toIndex</code> .
<code>Object[] toArray()</code>	It gets an array containing all the elements in this vector.
<code>Object[] toArray(Object[] a)</code>	It gets an array containing all the elements in this vector.
<code>String toString()</code>	It gets a string representation of this vector.
<code>void trimToSize()</code>	It trims the capacity of this vector to the current size.

Each vector has a *capacity*, which is the maximum size it can grow to. When you exceed that size, the capacity is automatically increased. You can use the `add()` and `addElement()` methods to add elements to a vector, the `remove()` and `removeElement()` methods to remove elements, and the `contains()` method to do a search. Here's an example using the `Vector` class in which we create a vector, check its size and capacity, and search for an element:

```
import java.util.*;

class VectorDemo
{
    public static void main(String args[])
    {
        Vector<Number> vector = new Vector<Number>(5);
        System.out.println("Capacity: " + vector.capacity());
        vector.addElement(new Integer(0));
        vector.addElement(new Integer(1));
        vector.addElement(new Integer(2));
        vector.addElement(new Integer(3));
        vector.addElement(new Integer(4));
        vector.addElement(new Integer(5));
        vector.addElement(new Integer(6));
        vector.addElement(new Double(3.14159));
        vector.addElement(new Float(3.14159));
        System.out.println("Capacity: " + vector.capacity());
        System.out.println("Size: " + vector.size());
        System.out.println("First item: " + (Integer)
            vector.firstElement());
        System.out.println("Last item: " + (Float) vector.lastElement());
        if(vector.contains(new Integer(3)))
            System.out.println("Found a 3.");
    }
}
```

Here's the output of this example:

```
C:\>java VectorDemo
Capacity: 5
Capacity: 10
Size: 9
First item: 0
Last item: 3.14159
Found a 3.
```

The **Stack** Class

The `Stack` class is built on the `Vector` class, and it implements a stack construct. Here's the inheritance diagram for the `Stack` class:

```
java.lang.Object
|___java.util.AbstractCollection
|   |___java.util.AbstractList
|       |___java.util.Vector
|           |___java.util.Stack
```

You'll find the constructor for the `Stack` class in Table 9.43 and its methods in Table 9.44:

Table 9.43: The constructor of the `Stack` class

Constructor	Does this
<code>Stack()</code>	It creates an empty stack.

Table 9.44: Methods of the `Stack` class

Method	Does this
<code>boolean empty()</code>	It returns <code>True</code> if this stack is empty.
<code>Object peek()</code>	It gets the object at the top of this stack without removing it.
<code>Object pop()</code>	It removes the object at the top of this stack and returns that object.
<code>Object push(Object item)</code>	It adds an item onto the top of this stack.
<code>int search(Object o)</code>	It gets the position of an object on this stack.

You use the `push()` method to add an element to a stack and the `pop()` method to retrieve it; note that elements come off a stack in the reverse order in which they were added to it. Here's an example in which we push and then pop elements on a stack:

```
import java.util.*;

class StackDemo
{
    public static void main(String args[])
    {
        Stack<Integer> stack1 = new Stack<Integer>();
        try {
            stack1.push(new Integer(0));
            stack1.push(new Integer(1));
            stack1.push(new Integer(2));
            stack1.push(new Integer(3));
            stack1.push(new Integer(4));
            stack1.push(new Integer(5));
            stack1.push(new Integer(6));

            System.out.println((Integer) stack1.pop());
            System.out.println((Integer) stack1.pop());
            System.out.println((Integer) stack1.pop());
            System.out.println((Integer) stack1.pop());
            System.out.println((Integer) stack1.pop());
            System.out.println((Integer) stack1.pop());
            System.out.println((Integer) stack1.pop());
        }
    }
}
```

```

        catch (EmptyStackException e) { }
    }
}

```

Here's the output of this example:

```

C:\>java StackDemo
6
5
4
3
2
1
0

```

The Dictionary Class

The `Dictionary` class is a class used by classes such as `Hashtable`, and it stores elements much like a map does, although it was introduced before the collection framework appeared. Because it forms the foundation of the `Hashtable` class, we'll include `Dictionary` here for reference. Here's the inheritance diagram for `Dictionary`:

```

java.lang.Object
|
+-- java.util.Dictionary

```

You'll find the constructor for `Dictionary` in Table 9.45 and its methods in Table 9.46:

Table 9.45: The constructor of the Dictionary class

Constructor	Does this
<code>Dictionary()</code>	It creates a dictionary.

Table 9.46: Methods of the Dictionary class

Method	Does this
<code>abstract Enumeration elements()</code>	It gets an enumeration of the values in this dictionary.
<code>abstract Object get (Object key)</code>	It gets the value for this key.
<code>abstract boolean isEmpty()</code>	It returns <code>True</code> if this dictionary has no keys.
<code>abstract Enumeration keys()</code>	It gets an enumeration of the keys in this dictionary.
<code>abstract Object put (Object key, Object value)</code>	It maps the given key to the given value in this dictionary.
<code>abstract Object remove (Object key)</code>	It removes the key and its value from this dictionary.
<code>abstract int size()</code>	It gets the number of entries in this dictionary.

The Hashtable Class

The `Hashtable` class implements the `Map` interface, and it was how you used to implement maps before the collection framework came. Here's the inheritance diagram for `Hashtable`:

```

java.lang.Object
|
+-- java.util.Dictionary
|
+-- java.util.Hashtable

```

You'll find the constructors for the `Hashtable` class in Table 9.47 and its methods in Table 9.48:

Table 9.47: Constructors of the Hashtable class

Constructor	Does this
<code>Hashtable()</code>	It constructs a new, empty hashtable.
<code>Hashtable (int initialCapacity)</code>	It constructs a new, empty hashtable with the given initial capacity.
<code>Hashtable(int initialCapacity,</code>	It constructs a new, empty hashtable with the given initial capacity and the

Table 9.47: Constructors of the Hashtable class

Constructor	Does this
<code>float loadFactor()</code>	given load factor.
<code>Hashtable(Map t)</code>	It constructs a new hashtable with the same mappings as the given map.

Table 9.48: Methods of the Hashtable class

Method	Does this
<code>void clear()</code>	It clears this hashtable so that it contains no keys.
<code>Object clone()</code>	It creates a copy of this hashtable.
<code>boolean contains(Object value)</code>	It returns True if some key maps into the given value in this hashtable.
<code>boolean containsKey(Object key)</code>	It returns True if the given object is a key in this hashtable.
<code>boolean containsValue(Object value)</code>	It returns True if this hashtable maps one or more keys to this value.
<code>Enumeration elements()</code>	It gets an enumeration of the values in this hashtable.
<code>Set entrySet()</code>	It gets a set view of the entries contained in this hashtable.
<code>boolean equals(Object o)</code>	It compares the given object with this map for equality.
<code>Object get(Object key)</code>	It gets the value to which the given key is mapped in this hashtable.
<code>int hashCode()</code>	It gets the hashCode value for this map, as per the definition in the Map interface.
<code>boolean isEmpty()</code>	It tests whether this hashtable maps no keys to values.
<code>Enumeration keys()</code>	It gets an enumeration of the keys in this hashtable.
<code>Set keySet()</code>	It gets a set view of the keys contained in this hashtable.
<code>Object put(Object key, Object value)</code>	It maps the given key to the given value in this hashtable.
<code>void putAll(Map t)</code>	It copies all the mappings from the given map to this hashtable.
<code>protected void rehash()</code>	It increases the capacity of and internally reorganizes this hashtable.
<code>Object remove(Object key)</code>	It removes the key (and its corresponding value) from this hashtable.
<code>int size()</code>	It gets the number of keys in this hashtable.
<code>String toString()</code>	It gets a string representation of this Hashtable object in the form of a set of entries.
<code>Collection values()</code>	It gets a collection view of the values contained in this hashtable.

You store and retrieve key/value pairs in a hashtable using the `put()` and `get()` methods. You can get an `Enumeration` object for the keys in a hashtable using the `keys()` method, and you can use that object to loop over the elements of the hash. Here's an example in which we create a hashtable and then print out its key/value pairs:

```
import java.util.*;

class Hashtable
{
    public static void main(String args[])
    {
        Hashtable<String, String> hashtable1 = new Hashtable<String, String>();
        hashtable1.put("Item 0", "value 0");
        hashtable1.put("Item 1", "value 1");
        hashtable1.put("Item 2", "value 2");
        hashtable1.put("Item 3", "value 3");
        hashtable1.put("Item 4", "value 4");
        hashtable1.put("Item 5", "value 5");
    }
}
```

```

        hashtable1.put("Item 6", "Value 6");
        Enumeration keys = hashtable1.keys();
        while(keys.hasMoreElements())
        {
            String key = (String) keys.nextElement();
            System.out.println(key + "/" + hashtable1.get(key));
        }
    }
}

```

Here's the output of this example. Note that the hash determines the order in which elements are stored in a hash, and you can't count on any specific order:

```

C:\>java HashTable
Item 0/Value 0
Item 6/Value 6
Item 5/Value 5
Item 4/Value 4
Item 3/Value 3
Item 2/Value 2
Item 1/Value 1

```

The **Properties** Class

The `Properties` class is derived from the `Hashtable` class (see the previous solution) and uses string keys and values. This class maintains the properties of various other classes, and an object of the `Properties` class is returned by those classes's `getProperty()` method. Here's the inheritance diagram for the `Properties` class:

```

java.lang.Object
|____java.util.Dictionary
|      |____java.util.Hashtable
|      |      |____java.util.Properties

```

You'll find the single field of the `Properties` class in Table 9.49, its constructors in Table 9.50, and its methods in Table 9.51:

Table 9.49: The field of the `Properties` class

Field	Does this
protected <code>Properties defaults</code>	A property list that contains default values for keys

Table 9.50: Constructors of the `Properties` class

Constructor	Does this
<code>Properties()</code>	It constructs a property list.
<code>Properties(Properties defaults)</code>	It constructs a property list with the given defaults.

Table 9.51: Methods of the `Properties` class

Method	Does this
<code>String getProperty(String key)</code>	It searches for the property with the given key.
<code>String getProperty(String key, String defaultValue)</code>	It searches for the property with the given key and returns the default value if the key is not found.
<code>void list(PrintStream out)</code>	It prints this property list to the given output stream.
<code>void list(PrintWriter out)</code>	It prints this property list to the given writer stream.
<code>void load(InputStream inStream)</code>	It reads a property list from the input stream.
<code>Enumeration propertyNames()</code>	It gets an enumeration of all the keys in this property list.
<code>void save(OutputStream out, String header)</code>	It is deprecated and it uses the <code>store</code> method instead.

Table 9.51: Methods of the Properties class

Method	Does this
Object setProperty (String key, String value)	It calls the hashtable method put.
void store (OutputStream out, String header)	It writes this property list in this Properties table to the output stream in a format acceptable for use with the load method.
Set <String> stringPropertyNames()	It returns a set of keys in this property list where the key and its corresponding value are strings, including distinct keys in the default property list if a key of the same name has not already been found from the main properties list.

You can put property values into a Properties object with the `setProperty()` method, and you can get property values with the `getProperty()` method. Here's an example in which we create a Properties object, add property values to it, and then retrieve a property value. Note, in particular, that you can supply a default value to `getProperty()` that it will return if there's no property value matching the property you have requested:

```
import java.util.*;

class PropertiesDemo
{
    public static void main(String args[])
    {
        Properties properties1 = new Properties();
        Set states; String outString;
        properties1.setProperty("Property 0", "value 0");
        properties1.setProperty("Property 1", "value 1");
        properties1.setProperty("Property 2", "value 2");
        properties1.setProperty("Property 3", "value 3");
        properties1.setProperty("Property 4", "value 4");
        properties1.setProperty("Property 5", "value 5");
        properties1.setProperty("Property 6", "value 6");
        outString = properties1.getProperty("Property 3", "Missing");
        System.out.println(outString);
        outString = properties1.getProperty("Property 7", "Missing");
        System.out.println(outString);
    }
}
```

Here's the output of this example:

```
C:\>java PropertiesDemo
value 3
Missing
```

Using the Aggregate Operations

It has been discussed earlier that aggregate operations are performed on a data structure rather than on an individual element. In order to better understand the concept of aggregate operations, consider the following example in which we are displaying the capital of countries with their country code:

```
public class CountriesCapital {
    Integer CountryCode;
    String CountryName;
    String CapitalName;
    Integer States;
    CountriesCapital() {}
    CountriesCapital(int cCode, String cName, String capName, int states) {
        this.CountryCode = cCode;
        this.CountryName = cName;
```

```

        this.CapitalName = capName;
        this.States = states;
    }
    public String toString() {
        return "" + CountryCode + ", " + CountryName + ", " + CapitalName + ", " + States;
    }
}

```

Let's create a class with the name CountryCapitalDemo which uses the earlier created CountriesCapital class in it:

```

import java.util.*;
import java.util.stream.*;
public class CountryCapitalDemo {
    private static List<CountriesCapital> countriesCapital = Arrays.asList(
        new CountriesCapital(91, "India", "Delhi", 36),
        new CountriesCapital(1, "USA", "Washington", 50),
        new CountriesCapital(43, "Austria", "Vienna", 40),
        new CountriesCapital(86, "China", "Beijing", 55),
        new CountriesCapital(81, "Japan", "Tokyo", 50),
        new CountriesCapital(61, "Australia", "Canberra", 8),
        new CountriesCapital(33, "France", "Paris", 27),
        new CountriesCapital(64, "New Zealand", "Wellington", 53),
        new CountriesCapital(94, "Sri Lanka", "Colombo", 22)
    );
    public static void main(String[] args) {
        System.out.println("Output with Lambda Expressions:");
        withLEExpression();
    }
    private static void withLEExpression() {
        List<CountriesCapital> cc = countriesCapital.stream().filter(c -> c.CountryCode >
40).collect(Collectors.toList());
        printList(cc);
    }
    private static void printList(List<CountriesCapital> cc) {
        cc.forEach(c -> System.out.println("\t" + c));
        System.out.println();
    }
}

```

Here's the output of this example:

```

Output with Lambda Expressions:
91, India, Delhi, 36
43, Austria, Vienna, 40
86, China, Beijing, 55
81, Japan, Tokyo, 50
61, Australia, Canberra, 8
64, New Zealand, Wellington, 53
94, Sri Lanka, Colombo, 22

```

Using the **java.util.function** Package

It is aforementioned that the java.util.function package includes interfaces that can be used by the JDK and with the code developed by a user. This package also includes enough functionality that can fulfill the common requirements of the lambda expressions. Consider the following example in which the java.util.function package is used:

```

import java.util.List;
import java.util.ArrayList;
import java.time.chrono.IsoChronology;
import java.time.LocalDate;

public class Employee {

```

```
public enum Sex {
    M, F
}

String empName;
LocalDate empBirthday;
Sex empGender;
String empEmailAddress;

Employee(String name, LocalDate bday, Sex gen, String email) {
    empName = name;
    empBirthday = bday;
    empGender = gen;
    empEmailAddress = email;
}

public int getAge() {
    return empBirthday
        .until(IsoChronology.INSTANCE.dateNow())
        .getYears();
}

public void printPerson() {
    System.out.println(empName + ", " + this.getAge());
}

public Sex getGender() {
    return empGender;
}

public String getName() {
    return empName;
}

public LocalDate getBirthday() {
    return empBirthday;
}

public static int compareByAge(Employee a, Employee b) {
    return a.empBirthday.compareTo(b.empBirthday);
}

public static List<Employee> createRoster() {
    List<Employee> roster = new ArrayList<>();
    roster.add(
        new Employee(
            "Deepak",
            IsoChronology.INSTANCE.date(1987, 11, 20),
            Employee.Sex.M,
            "deepak.sharma@dreamtechpress.com"));
    roster.add(
        new Employee(
            "Puneet",
            IsoChronology.INSTANCE.date(1986, 7, 15),
            Employee.Sex.M, "puneet@dreamtechpress.com"));
    roster.add(
        new Employee(
            "Nikhil",
```

```

        IsoChronology.INSTANCE.date(1991, 8, 13),
        Employee.Sex.M, "nikhil@dreamtechpress.com"));
    roster.add(
        new Employee(
            "Hari Narayan",
            IsoChronology.INSTANCE.date(1991, 8, 18),
            Employee.Sex.M, "hari.narayan@dreamtechpress.com"));

    return roster;
}
}

```

Let's now create a class with the name ReferenceMethodsDemo which uses the Employee class created earlier:

```

import java.util.*;
import java.util.function.Supplier;

public class ReferenceMethodsDemo {
    public static <T, SOURCE extends Collection<T>, DEST extends Collection<T>>
        DEST transElmnts(SOURCE sCollection, Supplier<DEST> cFactory) {

        DEST res = cFactory.get();
        for (T tt : sCollection) {
            res.add(tt);
        }
        return res;
    }

    public static void main(String... args) {

        List<Employee> roster = Employee.createRoster();

        for (Employee e : roster) {
            e.printPerson();
        }

        Employee[] rstrArray =
            roster.toArray(new Employee[roster.size()]);

        class EmployeeAgeComparator
            implements Comparator<Employee> {
            public int compare(Employee e, Employee ee) {
                return e.getBirthDay().compareTo(ee.getBirthDay());
            }
        }

        Arrays.sort(rstrArray, new EmployeeAgeComparator());

        Arrays.sort(rstrArray,
            (Employee e, Employee ee) -> {
                return e.getBirthDay().compareTo(ee.getBirthDay());
            }
        );

        Arrays.sort(rstrArray, Employee::compareByAge);
        class CmprsnProvider {
            public int cmprByEmpName(Employee e, Employee ee) {
                return e.getName().compareTo(ee.getName());
            }

            public int compareByAge(Employee e, Employee ee) {

```

```
        return e.getBirthday().compareTo(ee.getBirthday());
    }
}
CmprsnProvider myComparisonProvider = new CmprsnProvider();
Arrays.sort(rstrArray, myComparisonProvider::cmprByEmpName);

String[] stringArray = { "Deepak", "Puneet", "Nikhil", "Amit", "Hari", "Chirag" };
Arrays.sort(stringArray, String::compareToIgnoreCase);

Set<Employee> rosterSet = transElmnts(roster, HashSet::new);
System.out.println("New way of Display:");
rosterSet.stream().forEach(q -> q.printPerson());
    }
}
```

Here's the output of the preceding example:

```
Deepak, 27
Puneet, 28
Nikhil, 23
Hari Narayan, 23
New way of Display:
Deepak, 27
Hari Narayan, 23
Puneet, 28
Nikhil, 23
```

Summary

In this chapter, you have read about the collections framework which included the `Collection` interface and collection classes. You have also come across generics in Java.



10

Creating Packages, Interfaces, JAR Files, and Annotations

<i>If you need information on:</i>	<i>See page:</i>
Packages and Interfaces	312
JAR Files	312
The Core Java API Package	312
The java.lang Package	314
Basics of Annotation	315
Other Built-In Annotations	316
Creating a Package	319
Creating Packages that have Subpackages	320
Creating an Interface	320
Implementing an Interface	321
Extending an Interface	322
Using Interfaces for Callbacks	323
Performing Operations on a JAR File	324
Marker Annotations	329
Single Member Annotations	330
Summary	331

In this chapter, we are going to take a look at several important Java topics—creating packages, interfaces, JAR files, and annotations. All these topics are popular and good to have under your belt.

Packages and Interfaces

We have been using packages and interfaces throughout the book, and in this chapter, we'll look at how to create them. Being able to create your own packages is a useful skill—one that allows you to divide class libraries into smaller units, much as Oracle has done with the Java packages. You'll see how to create packages in code and how to arrange class files on the disk to match package structure. We'll also take a look at how to create interfaces, which is Java's answer to multiple inheritance.

JAR Files

To make it easier to download class files from the Web, Java supports Java Archive (JAR) files. The archives pack files using the ZIP format, and you'll see how to create JAR files and use them with the JAR tool that comes with Java. In addition, JAR files are the foundation of other parts of Java, such as Java beans.

The Core Java API Package

All of the core API packages defined by Java and their functioning are given in Table 10.1:

Table 10.1: The core Java API packages	
Package	Primary function
<code>java.applet</code>	It provides supports for applets constructions
<code>java.awt</code>	It provides Graphical User Interfaces capabilities
<code>java.awt.color</code>	It provides supports for color spaces and profiles
<code>java.awt.datatransfer</code>	Data transfers to and from the system clipboard
<code>java.awt.dnd</code>	It provides supports for drag-and-drop operations
<code>java.awt.event</code>	It handles events
<code>java.awt.font</code>	It represents various types of fonts
<code>java.awt.geom</code>	It provides the facility to work with geometric shapes
<code>java.awt.im</code>	It allows input of Chinese, Japanese, and Korean characters to text editing components
<code>java.awt.im.spi</code>	It supports alternative input devices
<code>java.awt.image</code>	It processes images
<code>java.awt.image.renderable</code>	It supports rendering- independent images
<code>java.awt.print</code>	It provides general print capabilities
<code>java.beans</code>	It allows you to build software components
<code>java.beans.beancontext</code>	It provides an execution environment for Beans
<code>java.io</code>	It inputs and outputs data
<code>java.lang</code>	It provides core functionality
<code>java.lang.annotation</code>	It provides supports for annotations
<code>java.lang.instrument</code>	It provides supports for program instrumentation
<code>java.lang.invoke</code>	It provides dynamic language support provided directly by the Java core class libraries and virtual machine
<code>java.lang.management</code>	It provides supports for management of the execution environment
<code>java.lang.ref</code>	It enables some interaction with the garbage collector

Table 10.1: The core Java API packages

Package	Primary function
<code>java.lang.reflect</code>	It analyzes code at runtime
<code>java.math</code>	It handles large integers and decimal numbers
<code>java.net</code>	It supports networking
<code>java.nio</code>	Top-level package for the NIO classes and encapsulates buffers
<code>java.nio.channels</code>	It encapsulates channels, used by the NIO system
<code>java.nio.channels.spi</code>	It supports service providers for channels
<code>java.nio.charset</code>	It encapsulates character sets
<code>java.nio.charset.spi</code>	It supports service providers for character sets
<code>java.nio.file</code>	It defines interfaces and classes for the JVM to access files, file attributes, and file systems.
<code>java.nio.file.attribute</code>	Interfaces and classes providing access to file and file system attributes
<code>java.nio.file.spi</code>	It provides service-provider classes for the <code>java.nio.file</code> package
<code>java.rmi</code>	It provides remote method invocation
<code>java.rmi.activation</code>	It activates persistent objects
<code>java.rmi.dgc</code>	It manages distributed garbage collection
<code>java.rmi.registry</code>	It maps names to remote object references
<code>java.rmi.server</code>	It supports remote method invocation
<code>java.security</code>	It handles certificates, keys, digests, signatures, and other security functions
<code>java.security.acl</code>	It manages access control lists
<code>java.security.cert</code>	It parses and manages certificates
<code>java.security.interfaces</code>	It defines interfaces for Digital Signature Algorithm (DSA) keys
<code>java.security.spec</code>	It specifies keys and algorithm parameters
<code>java.sql</code>	It communicates with a Structured Query Language (SQL) database
<code>java.text</code>	It provides the facilities for formatting, searching, and manipulating texts
<code>java.util</code>	It contains common utilities
<code>java.util.concurrent</code>	It supports the concurrent utilities
<code>java.util.jar</code>	It creates and reads JAR files
<code>java.util.logging</code>	It supports logging of information relating to user preference
<code>java.util.prefs</code>	It encapsulates information relating to user and system preferences
<code>java.util.regex</code>	It supports regular expression processing
<code>java.util.spi</code>	It supports service providers for the utility classes in <code>java.util</code> (Added by Java SE 6)
<code>java.util.stream</code>	It provides classes to support functional-style operations on streams of elements, such as map-reduce transformations on collections

Table 10.1: The core Java API packages

Package	Primary function
java.util.zip	It reads and writes compressed and uncompressed ZIP files

The java.lang Package

The `java.lang` package offers classes that are primarily required for the designing of Java programming language. Java defines the following subpackages:

- ❑ `java.lang.annotation`
- ❑ `java.lang.instrument`
- ❑ `java.lang.management`
- ❑ `java.lang.reflect`
- ❑ `java.lang.invoke`
- ❑ `java.lang.ref`

The *java.lang.annotation* Subpackage

Java's new annotation is supported by the `java.lang.annotation` subpackage. It defines the `Annotation` interface and the `ElementType` and `RetentionPolicy` enumerations.

The *java.lang.instrument* Subpackage

The `java.lang.instrument` subpackage is used for defining the characteristics that you can use for adding instrumentation to several aspects of execution of a program. It defines the `Instrumentation` and `ClassFileTransformer` interfaces, and the `ClassDefinition` class.

The *java.lang.management* Subpackage

The `java.lang.management` subpackage offers execution environment and management supports for the JVM. You can verify and manage various aspects of program execution by using the features available in the `java.lang.management` package.

The *java.lang.reflect* Subpackage

The `java.lang.reflect` subpackage offers interfaces and classes for obtaining reflective information about them. Reflection is the ability of software to examine itself. It is an important feature, especially when using components called Java Beans. It allows you to monitor a software component and describe its capabilities dynamically at runtime rather than at compile time. There are 10 classes in this package. They are listed in Table 10.2:

Table 10.2: Classes of the java.lang.reflect subpackage

Class	Does this
<code>AccessibleObject</code>	It is the base class for <code>Field</code> , <code>Method</code> and <code>Constructor</code> objects
<code>Array</code>	It provides static methods to dynamically create and access Java arrays
<code>Constructor<T></code>	It provides information about a constructor
<code>Executable</code>	It is used as a superclass for the common functionality of <code>Method</code> and <code>Constructor</code> .
<code>Field</code>	It provides information about, and dynamic access to, a single field of a class or an interface
<code>Method</code>	It provides information about a method
<code>Modifier</code>	It provides details about class and member access modifiers
<code>Parameter</code>	It provides information about method parameters

Table 10.2: Classes of the `java.lang.reflect` subpackage

Class	Does this
<code>Proxy</code>	It supports dynamic proxy classes
<code>ReflectPermission</code>	It allows reflection of private or protected members of class

The *`java.lang.invoke`* Subpackage

The `java.lang.invoke` subpackage includes support for dynamic language offered directly by the Java core class libraries and virtual machine.

As we have explained earlier in the Java Virtual Machine Specification, there are several types included in this package that have special relations to dynamic language support in the virtual machine:

- ❑ The class `MethodHandle` includes signature polymorphic methods. You can connect these methods irrespective of their descriptor type. Mainly, the linkage of method requires exact matching of type descriptors.
- ❑ Immediate constants of the classes `MethodHandle` and `MethodType` are supported by the JVM bytecode format.

The *`java.lang.ref`* Subpackage

The `java.lang.ref` subpackage supports reference-object classes that have interaction with the garbage collector up to a certain limit. The programs may use a reference object to establish references with various other objects in a manner that the latter object may still be retrieved by the collector. The programs may also organize to be notified later when the collector has analyzed that the reachability of a given object has altered.

Basics of Annotation

The Java platform provides an annotation (also known as Metadata) facility that permits defining and use of your own annotation types. It has specific syntaxes for declaring annotation types, for annotating declarations, APIs for reading annotations, a class file representation for annotations, and an annotation-processing tool.

Although annotations do not affect the program semantics directly, they do affect the way used by tools and libraries for treating programs, and that may affect the semantics of the running program. Annotations may be reflected at runtime or may be read from the source files and/or class files. These also complement the use of javadoc tags. A javadoc tag or an annotation is generally used for markup intended to produce or affect documentation.

Annotation type declarations and normal interface declarations are quite similar. As in the interface keyword, an at sign (@) precedes the annotation type declarations, which is further followed by a parenthesized list of element-value pairs. Each method defined by the declaration defines an element of the annotation type. They must not have any parameters or throws clause. Return types for this declaration are limited to `String`, primitives, enums, `Class`, annotations, and arrays of the preceding types. Methods can have default values. Shown here is an example of annotation type declaration:

```
public @interface RequestForEnhancement
{
    int identification();
    String abstract();
    String operator() default "[unassigned]";
    String date() default "[unimplemented]";
}
```

After defining an annotation type, the annotate declarations can be made. You can use annotation anywhere like other modifiers, such as `public`, `static`, or `final`, as it is a special kind of modifier. Conventionally, other modifiers are preceded by annotations. These values must be compile-time constants.

Even though annotations cannot include the `extends` clause, they automatically extend the `Annotation` interface. Thus, `Annotation` can be termed as the super-interface and is declared in the `java.lang.annotation` package. Just look at this code:

```
@classic(txt = "Know me", uni = 224)
public static void Classical() { . . . }
```

In the example, you'll notice that within an annotation (`@classic`), its members are given values. This annotation is linked with the method `Classical()`.

Other Built-In Annotations

Java defines 11 built-in annotations. From them, six were imported from `java.lang.annotation` and five more were included from `java.lang`:

- ❑ `@Retention`
- ❑ `@Documented`
- ❑ `@Target`
- ❑ `@Native`
- ❑ `@Repeatable`
- ❑ `@Retention`
- ❑ `@Inherited`
- ❑ `@Override`
- ❑ `@Deprecated`
- ❑ `@SuppressWarnings`
- ❑ `@FunctionalInterface`
- ❑ `@SafeVarargs`

@Retention Annotation

The `@Retention` annotation can only be used as an annotation to another annotation. Moreover, it provides the retention policy for Java. A retention policy can determine the point with the help of which an annotation can be discarded. The `java.lang.annotation.RetentionPolicy` class includes three such policies:

- ❑ **Retention policy of SOURCE** — An annotation of this policy is not included during compilation and retained only in the source file.
- ❑ **Retention policy of CLASS** — During compilation, an annotation of this policy gets stored in the class file and does not have availability during runtime through the JVM.
- ❑ **Retention policy of RUNTIME** — In the class file, an annotation of this policy gets stored and is also available at runtime through the JVM.

```
@Retention (RetentionPolicy.RUNTIME)
@interface classica { }
```

In short, it can be said that the new Metadata facility lets you embed annotation into a program. Various programming tools, in turn, process the annotations. For example, a tool might generate Java source code as requested by an annotation. In the Metadata facility, a programmer describes an action, but relies on a tool in actually providing the code. This type of approach reduces the amount of repetitious code that a programmer may be required to enter manually resulting in considerable time saving.

@Documented Annotation

The `@Documented` annotation can be defined as a marker interface that informs a tool about the documentation of an annotation. It can only be used as an annotation to an annotation declaration.

@Target Annotation

The `@Target` annotation defines the types of declaration to which an annotation can be applied. It is used as annotation for other annotations as well. The `@Target` annotation takes on constant argument from the `ElementType` enumeration. This argument specifies the types of declaration to which the annotation can be applied. Table 10.3 shows the constants along with the type of declaration to which they correspond:

Table 10.3: The Target constants and their type of declaration

Target constant	Target annotation
ANNOTATION_TYPE	Another annotation
CONSTRUCTOR	Constructor
TYPE	Class, interface, or enumeration
FIELD	Field
LOCAL_VARIABLE	Local variable
PARAMETER	Parameter
PACKAGE	Package
METHOD	Method

In a `@Target` annotation, you can mention one or more of these values. You must specify multiple values inside a braces-delimited list. For example, to specify that a `@Target` annotation applies only to fields and local variables:

```
@Target ({ ElementType.FIELD, ElementType.LOCAL_VARIABLE })
```

@Native Annotation

The `@Native` annotation specifies a field that defines a constant value which can be referenced from native code. This annotation can be used as a hint by tools that are used for generating native header files. These files are used for determining whether a header file is needed or not, and if needed, then what declarations it should have.

@Repeatable Annotation

The `@Repeatable` annotation is used for indicating that an annotation type whose declaration it (meta-) annotates is repeatable. The value of the `@Repeatable` annotation specifies the stored annotation type for the repeatable annotation type.

@Retention Annotation

The `@Retention` annotation specifies the time of retention of annotations having annotated type are to be retained. The default value of the retention policy is `RetentionPolicy.CLASS` in case no `Retention` annotation is available on an annotation type declaration.

The `@Retention` meta-annotation is affected only if you use the meta-annotated type directly for annotation.

@Inherited Annotation

The `@Inherited` annotation can be referred to as a marker annotation that can only be used on declaration of another annotation. In addition to this, it only affects that annotation which is used on class declaration. The `@Inherited` annotation allows the superclass annotation to be inherited by a subclass. Therefore, whenever a request is made for a particular annotation to the subclass, the superclass is checked if the annotation for the subclass is not available. Suppose that annotation is available in the superclass which is annotated with the `@inherited` keyword, then an annotation is returned.

@Override Annotation

The `@Override` annotation is used only on methods and can be referred to as a marker annotation. A method annotated with the `@Override` keyword needs to override a method from a superclass. If it does not override a method, a compile-time error appears. This annotation can be used for ensuring overriding of a superclass method.

@Deprecated Annotation

The `@Deprecated` annotation specifies that a declaration is removed and replaced by a newer form.

@SuppressWarnings Annotation

The `@SuppressWarnings` the annotation specifies suppressing of one or more notifications that might be provided by the compiler. The suppressed notifications are required to be mentioned by names in the form of string. You can apply this annotation to any type of declaration.

@FunctionalInterface

An informative annotation type is used for specifying an interface type declaration that is intended to be a functional interface. Generally, a functional interface contains exactly one abstract method. Since Default methods cannot be treated as abstract as they have an implementation.

You can create instances of functional interfaces by using lambda expressions, method references, or constructor references.

Compilers must generate an error message, if a type is annotated with this annotation type, unless the following conditions are satisfied:

- ❑ The type is an interface type and not an annotation type, enum, or class.
- ❑ The annotated type fulfills the need of a functional interface.

The compiler treats any interface satisfying the definition of a functional interface as a functional interface irrespective of whether or not a `@FunctionalInterface` annotation is available on the interface declaration.

@ SafeVarargs

Application of the `@SafeVarargs` annotation to a method or constructor suppresses unchecked notifications about a non-reifiable variable arity (vararg) type and creation of parameterized array creation at call sites.

In addition to the imposing of usage restrictions/constraints imposed applied by its [HYPERLINK "http://docs.oracle.com/javase/8/docs/api/java/lang/annotation/Target.html"](http://docs.oracle.com/javase/8/docs/api/java/lang/annotation/Target.html) \o "annotation in java.lang.annotation" `@Target` meta-annotation, compilers are need to implement more usage constraints on this annotation type. It is a compile-time error in case of declaration of a method or a constructor annotated with a `@SafeVarargs` annotation, and either:

- ❑ the declaration is a fixed arity method or constructor, or
- ❑ the declaration is a variable arity method neither static nor final.

Compilers issue notifications when this annotation type is used for to a method or constructor declaration, where the following conditions hold:

- ❑ The element type of the variable arity parameter is reifiable which includes primitive types: Object, and String.
- ❑ Unsafe operations are performed by the body of the method or constructor. Array Sometimes, unchecked warnings are not generated by some unsafe operations. For example, consider the following code snippet:

```
@SafeVarargs // Not actually safe!
static void m(List<String>... stringLists) {
    Object[] array = stringLists;
    List<Integer> tmpLlist = Arrays.asList(42);
    array[0] = tmpLlist; // Semantically invalid, but compiles without warnings
    String str = stringLists[0].get(0); // Oh no, ClassCastException at runtime!
}
```

The aliasing in the preceding code snippet leads to the `ClassCastException` exception at runtime. The statement `arr[0]=tmpLst` is semantically invalid, but will execute without warnings. However, the statement `String str=stringLists[0].get(0)` gets executed and will give the `ClassCastException` exception.

Type Annotations and Pluggable Type Systems

Before the release of Java SE 8, you could only apply annotations to declarations. But the release of Java SE 8 has let you apply annotations to any type. Some instances where types are used are implements clauses, object creation expressions (`new`), and casts and throws clauses. This kind of annotation is known as type annotation.

Type annotations allow better analysis of Java programs and ensure stronger type checking. A type checking framework is not provided in Java SE 8, but it enables you to write (or download) a type checking framework. This framework can be implemented as single or multiple pluggable modules that can be used in conjunction with the Java compiler.

Repeating Annotations

In Java SE8, you can use repeating annotations in situations where you want to apply the same annotation to a declaration or type use. Suppose you want to use a timer service which allows you to execute a method at a specified time like the UNIX `cron` service. For creating this timer service, you need to write a code. Using the code, you can set a timer for executing a method, `doPeriodicCleanup`, on every month's last day and on every Saturday at 10:00 p.m. Now, in order to set the timer for execution, you need to write an `@Schedule` annotation and use it twice for the `doPeriodicCleanup` method: first for specifying the last day of the month and the second for specifying Saturday at 10 p.m., as shown in the following statements:

```
@Schedule(dayOfMonth="last")
@Schedule(dayOfWeek="Sat", hour="23")
public void doPeriodicCleanup() { ... }
```

In the preceding statements, an annotation to a method is applied. Now, you can repeat an annotation anywhere so that you will be able to use a standard annotation. The repeating annotations are stored in a *container annotation* that is generated automatically by the Java compiler for compatibility reasons. For making the compiler to do this, the following two declarations are needed in your code:

- ❑ The annotation type must be specified with the `@Repeatable` meta-annotation
- ❑ The containing annotation type must contain a value element with an array type

Retrieving Annotations

Reflection API provides various methods that can be used for retrieving annotations. The behavior of these methods remains unchanged, which return a single annotation, such as `AnnotatedElement.getAnnotationByType(Class<T>)`, if there is availability of one annotation of the requested type. In case there is availability of more than one annotation of the requested type, you can retrieve them by getting their container annotation first. In this way, legacy code functions continuously. Other methods were also introduced in Java SE 8 for scanning through the container annotation for returning multiple annotations together, such as `AnnotatedElement.getAnnotations(Class<T>)`.

Creating a Package

When you have a number of class files, it's often a good idea to arrange them on disk by using a directory hierarchy. In fact, Java packages were originally designed to reflect such a file organization. You can arrange class files into a directory hierarchy and let Java know what's going on with the packages. For example, if you have a package named `package1` with one class in it, `app.class`, the class file would go into a directory named `package1`, like this:

```
package1
|___app
```

Multiple class files in the same package go into the same directory:

```
package1
|___app1
|___app2
|___app3
```

As long as the `package1` directory is in a location that Java will search, Java will look for the class files you list as part of that package in that directory.

You need to indicate what package a class file is part of by using the `package` statement in your code. Here's an example where we are creating `app.class` and indicating that it's part of `package1` by using the `package` statement:

```
package package1;
public class app {
    public static void main(String[] args) {
```

```
    System.out.println("Hello from Java!"); }
}
```

After we have compiled `app.class` and stored it in the `package1` directory, we can import the `app` class into my code with statements such as the following, just as we would with any other Java package:

```
import package1.*;
or
import package1.app;
```

The Java runtime tool also knows about packages; since `app.class` is itself an application, we can run this class with a command line that specifies the `app` class's package by using a dot (`.`) for the directory separator:

```
E:\Java8>java package1.app
Hello from Java!
```

In fact, the dot package separator comes in handy when you create subpackages of packages – take a look at the next solution for the details.

Creating Packages that have Subpackages

When you have a lot of class files, you may want to organize them into quite a complex package structure, and you do that by creating the corresponding directory structure on disk, including giving and subdirectories. For example, if you want the `app` class to be in `package2`, which itself is a subpackage of `package1`, this is the directory structure you'd use:

```
package1
|___package2
|___app
```

To create this package structure in code, you just use the dot package separator. Here's an example:

```
package package1.package2;

public class app
{
    public static void main(String[] args)
    {
        System.out.println("Hello from Java!");
    }
}
```

Now you can import the `app` class into your code with statements such as these:

```
import package1.package2.*;
or
import package1.package2.app;
```

Since here `app` is already an application class, you can also run it like this, specifying the package structure:

```
E:\Java8 >java package1.package2.app
Hello from Java!
```

Creating an Interface

We have used interfaces throughout the book, and we'll take a look at how to create one here. When you create an interface, you specify the methods for the interface, and when you implement the interface, you provide code for those methods. Interfaces have two advantages over inheritance:

- ❑ A class can extend to only one other class, but as many interfaces as you need.
- ❑ They are easier to work with than interfaces because you don't have to provide any implementation details in the interface.

NOTE

When an interface is declared as public, it can be used by other classes as well.

Here's an example in which we create an interface named `Printem` that has one method: `printText()`. We implement this interface in a class named `class1`. To create an interface, we simply use the `interface` statement and list the prototypes (declarations without bodies) of the methods we want in the interface, like this:

```
interface Printem { void printText(); }
```

Now we can implement this interface in `class1`:

```
interface Printem { void printText(); }

class interfaces
{
    public static void main(String[] args)
    {
        class1 object1 = new class1();
        object1.printText();
    }
}

class class1 implements Printem
{
    public void printText() { System.out.println("Dreamtech Press"); }
}
```

The output of the code is shown in Figure 10.1:

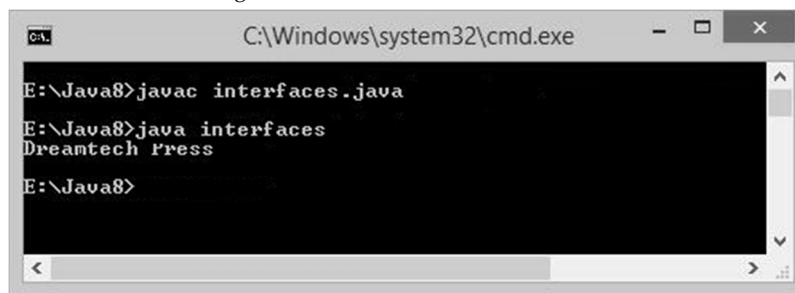


Figure 10.1: Printing text using interface

Implementing an Interface

You can create classes that partly implement an interface, but these classes must be declared abstract because they are only partial implementations and therefore cannot be instantiated into objects. Here's an example in which we create an interface with two methods: `printText1()` and `printText2()`:

```
interface Printem { void printText1(); void printText2(); }
```

Next, we create a new class, `Class1`, that implements the interface but only defines one method, `printText1()` (note that we must declare this class abstract as it is not a full implementation of the interface):

```
interface Printem { void printText1(); void printText2(); }
abstract class Class1 implements Printem
{
    public void printText1() { System.out.println("Hello from Java!"); }
}
```

Finally, we can create a new class, `Class2`, that extends `Class1` and defines the `printText2()` method, which means we can instantiate objects of `Class2` as shown here:

```
interface Printem { void printText1(); void printText2(); }

abstract class Class1 implements Printem
{
    public void printText1() { System.out.println("Hello from Dreamtech Press"); }
}

class Class2 extends Class1
{
    public void printText2()
    {
        System.out.println("Hello from Java interfaces!");
    }
}
```



```

public class Interfaces2
{
    public static void main(String[] args)
    {
        Class2 object2 = new Class2();
        object2.printText1();
        object2.printText2();
    }
}

```

The output of this code is shown in Figure 10.2:



Figure 10.2: Implementing an interface

Note that partial implementations can be useful, such as when you want to specify how some of an interface's methods, but not all, should work in a series of subclasses.

In fact, an interface is a kind of type, just like class. As a result, you can use an interface as the type for a variable, parameter, or method return value. For example, consider this interface:

```

public interface Play { void play(); }
. . .
Play gme = getGame();
gme.play();
. . .

```

Here the method `getGame()` returns an object that implements the `Play` interface and this object is assigned to a variable of type `Play`. Then the object's `play()` method is called.

Extending an Interface

Interfaces are extended with the use of `extends` keyword. An interface that is getting extended is known as the *superinterface*; on the other hand, the interface which is extending an existing interface is known as the *subinterface*.

```

public interface color
{
    void blue();
    void red();
}
public interface shirt
{
    void sz1();
    void sz2();
}
public interface combination extends color, shirt { void combin1(); }

```

When the `extends` keyword is used with interfaces, all the fields of the superinterface are copied into the subinterface. In other words, three interfaces are declared—the first defines two methods `blue()` and `red()`; the second defines two methods `sz1()` and `sz2()`; and the third extends `color` and `shirt`, and adds a method of its own called `combin1()`.

Using Interfaces for Callbacks

A callback is a technique wherein an object lets another object know that the second object should call one of the first object's methods whenever a certain event happens. The first object used in this technique is known as an event listener as it waits till it is called by the other object. The second object in this technique is known as the event source as it is the source of events that provides calls to the listener.

Such callbacks are handled in Java through Interfaces. The callbacks are generally used in graphical Swing applications, where event listeners are built for handling the events of user interface, such as mouse clicks.

Let's look at this example where we have implemented callbacks. In this example, a timer is implemented in the class Ticking. This timer generates events after regular intervals and sets up a listener object for handling these events:

```
import java.awt.event.*;
import javax.swing.*;

public class Ticking
{
    public static void main(String[] args)
    {
        Timer tm = new Timer(1000, new Ticker());
        tm.start();
        JOptionPane.showMessageDialog(null, "You wish to exit...cool...Click OK"); }

    class Ticker implements ActionListener
    {
        private boolean tick = true;

        public void actionPerformed(ActionEvent event)
        {
            if (tick) { System.out.println("Tick..."); }
            else { System.out.println("Tock..."); }
            tick = !tick;
        }
    }
}
```

Here we have used the `ActionListener` interface that consists of a single method named `actionPerformed()`. A parameter of type `ActionEvent` is received by the `actionPerformed()` method. The result displays the messages `Tick...` and `Tock...` on the console, as shown in Figure 10.3:

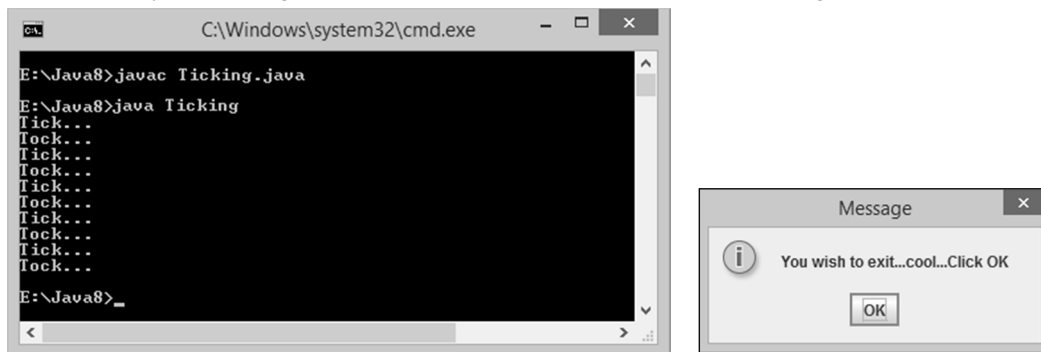


Figure 10.3: Displaying Tick and Tock

Performing Operations on a JAR File

A Java Archive (JAR) file comprises various directories and isolated files that are saved in a condensed format in zip pattern. The `.jar` is the extension for JAR files. A JAR file is unzipped using software and then all its contents are extracted.

Also, many processors, such as the Java Virtual Machine, or browsers, can use the files in a JAR directly without extracting the files.

Creating a JAR File

A JAR file allows you to wrap various files into one. In fact, JAR files compress their contents by using the ZIP format, so if your applet needs a lot of large files, it makes sense to create and download a JAR file. Web browsers only need one connection rather than new connections for each sub file, which can improve download times. The digital signature is used by the files in a JAR file for proving their origin. You create JAR files with the `jar` tool, and we'll take a look at that tool in this and the next few solutions. This tool comes with Java, and here's how you use it:

```
jar [options] [manifest] destination input-file [additional input files]
```

Here, *options* refer to the options you can use with the `jar` tool, and they are much like the options you can use with the UNIX `tar` tool. The optional *manifest* argument consists of the name of a manifest file, which supports digital signing and lists the contents of the JAR file. When you create a Java bean, you use the manifest file to indicate which classes are beans, as we'll do later in this chapter. Here are the possible options you can use with the `jar` tool:

- ❑ `c`—It creates a new or empty archive on the standard output.
- ❑ `t`—It lists the table of contents from standard output.
- ❑ `x file`—It extracts all files or just the named files. In case a `file` is not present, then all the files are accessed; else, only the specific files are accessed.
- ❑ `f`—The argument following this option specifies a JAR file to work with.
- ❑ `v`—It generates “verbose” output on `stderr`.
- ❑ `m`—It includes manifest information from a specified manifest file.
- ❑ `0`—It indicates “store only,” without using ZIP compression.
- ❑ `M`—It specifies that a manifest file should not be created for the entries.
- ❑ `u`—It updates an existing JAR file by adding files or changing the manifest.
- ❑ `-C`—It changes directories during execution of the `jar` command. For example, `jar uf jarfile.jar -C classes` would add all files within the `classes` directory, but not the `classes` directory itself, to `jarfile.jar`.

TIP

Use an argument beginning with the character “@” to specify a file containing additional arguments, one argument per line. The arguments are inserted into the command line at the position of `@filename` argument.

Here's a typical usage of the `jar` tool:

```
E:\Java8 >jar cf jarfile.jar *.class
```

Here, all the class files present in the current directory are stored in the file with the name `jarfile.jar`. A default manifest file gets generated automatically by the `jar` tool and is always the first entry in the jar file. If you have a manifest file that you want the `jar` tool to use for a new JAR archive, you can use the `-m` option and specify it like this (we'll create manifest files later in this chapter):

```
E:\Java8 >jar cmf manifest.mft jarfile.jar *.class
```

You must note that if you mention the options `cfm` in place of `cmf`, then you must first mention the name of the JAR archive, which will be further followed by the manifest file's name:

```
E:\Java8 >jar cfm jarfile.jar manifest.mft *.class
```

Note also that JAR files are not just for class files; they can store any type of files. Here's how to pack all the files in a directory into a JAR file:

```
E:\Java8 >jar cfm jarfile.jar manifest.mft *.*
```

If the file names you want to pack include any directories, those directories are searched recursively and the files in them are stored. When the JAR file is unpacked, the directory structure will be re-created.

Getting the Contents of a JAR File

You can determine what's in a JAR file with the `tf` options as in this case, where the JAR file contains a default manifest in the `META-INF` internal directory and a number of class files:

```
E:\Java8 >jar tf SampleJar.jar
```

The output of the command given above will be as follows:

```
META-INF/
META-INF/MANIFEST.MF
applet.class
jpanel.class
newButton.class
textPanel.class
```

Extracting Files from a JAR File

You can extract files from a JAR file by using the `xf` options. For example, here's how to extract all the files in the file `jarfile.jar`, introduced in the previous solution:

```
E:\Java8>jar xf jarfile.jar
```

Unpacking all the files in a JAR file this way will also re-create their directory structures. In this case, the `jar` tool unpacks the class files, and also makes a directory named `META-INF` and places the default manifest file `MANIFEST.MF` in it. You can also extract files by specifying their names. Here's an example in which we are extracting `applet.class`:

```
E:\Java8>jar xf jarfile.jar applet.class
```

Updating JAR Files

Here's how to update a JAR file with a new version of one of the files stored in it, `class1.class`, using the `-uf` options:

```
E:\Java8>jar -uf jarfile.jar class1.class
```

Reading from JAR Files in Code

The `JarFile` class can be used for reading the data stored in a JAR file from any file that can be opened using the `java.io.RandomAccessFile` package. The `java.util.zip.ZipFile` class is extended by the `JarFile` class with support to read an optional manifest entry. Here's the inheritance diagram for the `ZipFile` class:

```
java.lang.Object
|
|___java.util.zip.ZipFile
```

You'll find the constructors for the `ZipFile` class in Table 10.4 and its methods in Table 10.5:

Table 10.4: Constructors of the ZipFile class	
Constructor	Does this
<code>ZipFile(File file)</code>	It opens a ZIP file for reading from the indicated <code>File</code> object
<code>ZipFile(File file, int mode)</code>	It opens a ZIP file for reading from the specified <code>File</code> object in the specified mode
<code>ZipFile(String name)</code>	It opens a ZIP file for reading

Table 10.5: Methods of the ZipFile class	
Method	Does this
<code>void close()</code>	It closes the ZIP file

Table 10.5: Methods of the ZipFile class

Method	Does this
Enumeration entries()	It gets an enumeration of the ZIP file entries
protected void finalize()	It ensures that the <code>close()</code> method of this ZIP file is called when there are no more references to it
ZipEntry getEntry (String name)	It gets the ZIP file entry for the indicated name (or null if not found)
InputStream getInputStream (ZipEntry entry)	It gets an input stream for reading the contents of the indicated ZIP file entry
String getName()	It gets the pathname of the ZIP file
int size()	It gets the number of entries in the ZIP file

The `JarFile` class is built on the `ZipFile` class. Here's the inheritance diagram for the `JarFile` class:

```

java.lang.Object
|___java.util.zip.ZipFile
|___java.util.jar.JarFile

```

You'll find the field of the `JarFile` class in Table 10.6, its constructors in Table 10.7, and its methods in Table 10.8:

Table 10.6: The field of the JarFile class

Field	Does this
static String MANIFEST_NAME	It holds the name of the manifest

Table 10.7: Constructors of the JarFile class

Constructor	Does this
JarFile(File file)	Constructs new JAR file to read from indicated File object
JarFile(File file, boolean verify)	It constructs a new JAR file to read from the indicated File object and allows for verification
JarFile(File file, boolean verify, int mode)	It constructs a new JAR file to read from the indicated File object in the specified mode
JarFile(String name)	Constructs a new JAR file to read from indicated file name
JarFile(String name, boolean verify)	It constructs a new JAR file to read from the indicated file name and allows for verification

Table 10.8: Methods of the JarFile class

Method	Does this
Enumeration<JarEntry> entries()	It gets an enumeration of the ZIP file entries
ZipEntry getEntry (String name)	It gets the ZIP file entry for the indicated name (or null if not found)
InputStream getInputStream (ZipEntry ze)	It gets an input stream for reading the contents of the indicated ZIP file entry
JarEntry getJarEntry (String name)	It gets the JarEntry object for the given entry name (or null if not found)
Manifest getManifest()	It gets the JAR file manifest (or null if none)

You actually handle the items in a JAR file as `JarEntry` objects, and the `JarEntry` class is derived from the `ZipEntry` class. Here's the inheritance diagram for `ZipEntry`:

```

java.lang.Object
|___java.util.zip.ZipEntry

```

You'll find the fields of the `ZipEntry` class in Table 10.9, its constructors in Table 10.10, and its methods in Table 10.11:

Table 10.9: Fields of the <code>ZipEntry</code> class	
Field	Does this
<code>static int DEFLATED</code>	Specifies the compression method for compressed entries
<code>static int STORED</code>	Specifies compression method for uncompressed entries

Table 10.10: Constructors of the <code>ZipEntry</code> class	
Constructor	Does this
<code>ZipEntry(String name)</code>	It constructs a new ZIP entry with the indicated name
<code>ZipEntry(ZipEntry e)</code>	It constructs a new ZIP entry with fields taken from the indicated ZIP entry

Table 10.11: Methods of the <code>ZipEntry</code> class	
Method	Does this
<code>Object clone()</code>	It gets a copy of this entry
<code>String getComment()</code>	It gets the comment string for the entry (or null if none)
<code>long getCompressedSize()</code>	It gets the size of the compressed entry data (or -1 if not known)
<code>long getCrc()</code>	It gets the CRC-32 checksum of the uncompressed entry data (or -1 if not known)
<code>byte[] getExtra()</code>	It gets the extra field data for the entry (or null if none)
<code>int getMethod()</code>	It gets the compression method of the entry (or -1 if not indicated)
<code>String getName()</code>	It gets the name of the entry
<code>long getSize()</code>	It gets the uncompressed size of the entry data (or -1 if not known)
<code>long getTime()</code>	It gets the modification time of the entry (or -1 if not indicated)
<code>int hashCode()</code>	It gets the hashcode value for this entry
<code>boolean isDirectory()</code>	It returns True if this is a directory entry
<code>void setComment(String comment)</code>	It sets the optional comment string for the entry
<code>void setCompressedSize(long csize)</code>	It sets the size of the compressed entry data
<code>void setCrc(long crc)</code>	It sets the CRC-32 checksum of the uncompressed entry data
<code>void setExtra(byte[] extra)</code>	It sets the optional extra field data for the entry
<code>void setMethod(int method)</code>	It sets the compression method for the entry
<code>void setSize(long size)</code>	It sets the uncompressed size of the entry data
<code>void setTime(long time)</code>	It sets the modification time of the entry
<code>String toString()</code>	It gets a string representation of the ZIP entry

Here's the inheritance diagram for the `JarEntry` class:

```

java.lang.Object
|___java.util.zip.ZipEntry
    |___java.util.jar.JarEntry
  
```

You'll find the constructors for the `JarEntry` class in Table 10.12 and its methods in Table 10.13:

Table 10.12: Constructors of the <code>JarEntry</code> class	
Constructor	Does this
<code>JarEntry(JarEntry je)</code>	Constructs new <code>JarEntry</code> object from given <code>JarEntry</code> object
<code>JarEntry(String name)</code>	It constructs a new <code>JarEntry</code> object for the given JAR file entry name
<code>JarEntry(ZipEntry ze)</code>	It constructs a new <code>JarEntry</code> object from the given <code>ZipEntry</code> object

Table 10.13: Methods of the JarEntry class

Method	Does this
Attributes getAttributes()	It gets the manifest attributes for this entry (or null if none)
Certificate[] getCertificates()	It gets the certificate objects for this entry (or null if none)
CodeSigner getCodeSigners()	It gets the CodeSigner objects for this entry (or null if none)

Using *javac* to Get Classes from a JAR File

For convenience, you can use the Java compiler, *javac*, to unpack class files from JAR files as needed. Here's an example in which we have a class named *display* with one method, *displayText()*, which displays a message:

```
public class display
{
    public void displayText() { System.out.println("Hello from Java!"); }
}
```

Here's how we make a JAR file from this class:

```
E:\Java8>jar cf display.jar display.class
```

Next, we delete the *display.class* file, so all that's left is the JAR file, *display.jar*. Now we can make use of the *display* class in an application, like this:

```
public class useDisplay
{
    public static void main(String[] args)
    {
        display display1 = new display();
        display1.displayText();
    }
}
```

To compile this application, include *display.jar* in your CLASSPATH so that *javac* will search the JAR file for *display.class* and unpack that class. Because *display.class* is now unpacked, it's available when we run the application that uses it:

```
E:\Java8>javac -cp E:/Java8/display.jar useDisplay.java
E:\Java8>java useDisplay
Hello from Java!
```

Letting Applets Get Classes from JAR Files

You can tell Web browsers and the Oracle appletviewer that you are storing files in a JAR file with the *archive* parameter in the `<APPLET>` tag. Here's an example. In this case, we are creating an applet that will be stored in a JAR file named *applett.jar*. Here's the code for this applet, including the `<APPLET>` tag:

```
import java.awt.*;
import java.applet.Applet;
/*<APPLET
    CODE=applett.class
    WIDTH=200
    HEIGHT=200><PARAM NAME=archive VALUE="applett.jar"></APPLET> */
public class applett extends Applet
{
    public void paint(Graphics g) { g.drawString("Dreamtech Press", 30, 80); }
}
```

After compiling this applet to *applett.class*, we create the *applett.jar* file:

```
E:\Java8>jar cf applett.jar applett.class
```

Next, we delete *applett.class* and open *applett.java* by using the Oracle appletviewer:

```
E:\Java8>appletviewer applett.jar
```

The result appears as shown in Figure 10.4:



Figure 10.4: Opening an applet from a JAR file

As you can see, the appletviewer has opened the applet from the JAR file.

Marker Annotations

A marker annotation is a special kind of annotation that is used for marking a declaration, and it does not contain any members. The `AnnotatedElement` interface defines the `isAnnotationPresent()` method, which denotes whether the marker annotation is present. Since a marker interface contains no members, simply determining whether it is present or not is sufficient. The code to implement the marker annotations is as follows:

```
import java.lang.annotation.*;
import java.lang.reflect.*;

@Retention(RetentionPolicy.RUNTIME)
@interface Mark {}
class Marker
{
    @Mark
    public static void Meth ()
    {
        Marker obj = new Marker();
        try {
            Method mt = obj.getClass () .getMethod ("Meth");
            if (mt.isAnnotationPresent (Mark.class))
                System.out.println ("Here is the marker of that application");
            System.out.println ("Welcome to DreamTech Press");
        }
        catch (NoSuchMethodException exc) {
            System.out.println ("Method Not Found.");
        }
    }
    public static void main (String args[])
    {
        Meth();
    }
}
```


The output shown in Figure 10.5 confirms that `@Mark` is present:

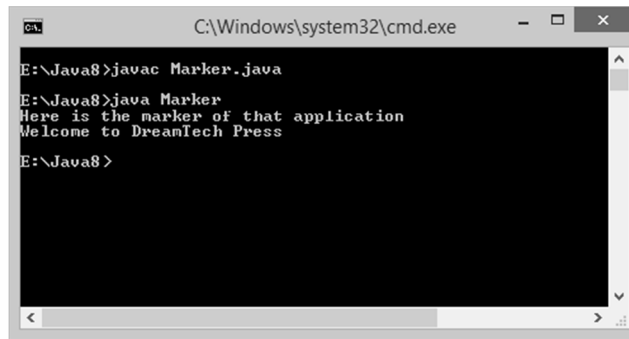


Figure 10.5: Use of marker

Notice that you do not need to follow `@Mark` with parentheses when it is applied. `@Mark` is applied simply by using its name, like this `@Mark`.

NOTE

An empty set of parenthesis can also be supplied.

Single Member Annotations

A single member annotation comprises only one member and functions similar to a normal annotation. It enables a shorthand form for mentioning the member's value. In case only a single member is present, then you do not require to mention the member's name. In fact, you can simply specify the value for that member when the annotation is applied.

See how we use a single-member annotation in the example:

```
import java.lang.annotation.*;
import java.lang.reflect.*;

@Retention(RetentionPolicy.RUNTIME)
@interface Sngle { int value(); }

class Single
{
    @Sngle(100)
    public static void Meth()
    {
        Single obj = new Single();
        try { Method mt = obj.getClass().getMethod("Meth");
            Sngle sng = mt.getAnnotation(Sngle.class);
            System.out.println(sng.value()); }
        catch (NoSuchMethodException exc)
        {
            System.out.println ("Method Not Found.");
        }
    }
    public static void main(String args[]) { Meth(); }
}
```

As expected, the result of this program gets displayed as 100. In the program, `@Sngle` is used to annotate `Meth()` as `@Sngle(100)`. Notice the way a value is inserted. There is no need for 'value = '.

NOTE

In single-member annotation, the name of the member must be value.

Summary

In this chapter, You have learned about the concept of packages, interfaces, and JAR files. Then, you have learned core Java API package and the `java.lang` package. We explored how to create a package, a subpackage; create and implement an interface; and use interfaces for callbacks. Lastly, we have discussed how to perform operations on a JAR file and use annotations.

The next chapter discusses working with Java Beans.



11

Working with Java Beans

<i>If you need information on:</i>	<i>See page:</i>
What is Java Bean?	335
Advantages of Java Bean	336
Introspection	336
Persistence	338
Customizers	338
Understanding Java Beans	338
Designing Programs Using Java Beans	338
Creating Applets that Use Java Beans	341
Creating a Java Bean	341
Creating a Bean Manifest File	342
Creating a Bean JAR File	343
Creating a New Bean	343
Adding Controls to Beans	343
Giving a Bean Properties	344
Design Patterns for Properties	349
Using Simple Properties	349
Designing Patterns for Events	352
Learning Methods and Design Patterns	352
Creating Bound Properties	353
Giving a Bean Methods	353
Giving a Bean an Icon	354
Creating a BeanInfo Class	355
Setting Bound and Constrained Properties	356
Implementing Persistence	357
Using the Java Beans API	358
Learning the Basics of an Event	359
Using the Java Beans Conventions	360

<i>If you need information on:</i>	<i>See page:</i>
Using the Remote Notification and Distributed Notification	360
Using Beans with JSP	360
Summary	360

Sun provides the JavaBeans Bridge for ActiveX components. Beans are reusable components written in Java that you can use in a variety of programming environments, including third-party programming environments. In fact, they work much like ActiveX controls, exposing properties, methods, and events to other code components. Using beans, you can create your own new Java “controls,” such as buttons that turn colors when you click them or stock tickers that download information from the Internet.

Java Beans is designed to be used in application builder tools, which are programming environments that let you configure beans. One such tool, the *BeanBox*, comes with the Sun Bean Development Kit (BDK), which you can download and install from <http://www.ecst.csuchico.edu/~amk/foo/csci611/notes/beans/BDK/>. We'll use the BDK in this chapter to construct and use beans. As reusable code components that you can create applications from, beans represent a powerful new way of looking at programming—you can create your own applications on the fly from pre-written components by adding just what you need—and nothing more.

Java Beans has a very important part for the Java users. This part is very important. The reason for their importance is they allow you to build complex systems from software components. Usually, there are different vendors who supply these components, and sometimes, these components are also made available by you. Java Beans defines the structural design that shows how the different building units work together.

While using a component-based approach, the software components can be easily reused and interoperability between the components can also be made possible. This approach allows the programs to be collected from software building blocks provided by different vendors to use the benefits. It is necessary for a designer to select a component, understand it, and also check for its availability. Then, it becomes easy to implement its operation into the existing code. Such kind of architecture is provided by Java Beans.

Oracle supports Java Beans in the `java.beans` package. We will discuss about the interfaces and classes under this package in this chapter.

In this chapter, we'll create a number of Java Beans—from simple ones to ones that support properties and methods and let users embed other controls inside them.

TIP

The way you create beans varies a little among operating systems because the BDK differs a little among operating systems. We use the BDK for Windows in this chapter.

What is Java Bean?

A Java Bean refers to a software component that can be reused in different environments. It can perform different types of function, i.e., simple as well as complex. Simple function examples include obtaining an inventory value, and complex function examples include forecasting the performance of a stock portfolio. A Bean may be visible to an end user. An example of this is using a radio within a car, which is also used at homes; thus, it serves the purpose at various places.

A Bean may also be visible to a user. An example of this type of building block is software that encodes a stream of data in real time. Finally, a Bean can work separately on a user's computer or in association with a group of distributed components. An example of a Bean that runs locally is the software that creates a bar chart from a set of data points. However, it is necessary that a Bean which provides real-time price information from a stock or commodities exchange would need to work in corporation with other distributed software to obtain its data.

The builder tools use a process for discovering the different features, such as methods, events, and properties, of a Bean. This process is called introspection. The following are the features of Beans:

- ❑ It provides support by adhering to specific rules, known as design patterns, at the time when naming Bean features. The `Introspector` class monitors Beans for such design patterns to explore Bean features. This class relies on the core reflection API. The Reflection API is an excellent place to learn about reflection.
- ❑ It provides support by explicitly providing property, method, and event information with a related `Bean` information class. This Bean information class executes the `BeanInfo` interface. Also, the Bean information class explicitly records the Bean features that are to be used for application builder tools.
- ❑ Properties are the Bean's look and its behavior, which can be modified at design time. Builder tools look into a Bean to discover its properties and expose those properties for calculation.

- ❑ The properties of Beans are customized at design time. Here, customization is achieved through two methods, that is, by using property editors and by using more technical Bean customizers.
- ❑ Beans use events in order to communicate with other Beans. Beans are of two types, namely, listener Bean and source Bean. Listener Bean is a Bean that shows its interest in receiving events and registers its interest with the source Bean. The source Bean is the Bean that fires the event. A Bean is monitored by Builder tools to ascertain which events are sent and which events can be handled by the bean. *Persistence* enables Beans to save and restore their state. Once we modify the properties of Beans, we can save the Bean's state and resume it later. Remember that property changes remain intact. Java Beans use Java Object Serialization to support persistence. A Bean's method can be invoked from other Beans or a scripting environment. The methods are similar to Java's methods. Beans are created to be understood by builder tools and all key APIs, including properties, events, and persistence, meant to be easily understandable by programmers. The component technology is brought to the Java platform by the Java Beans. You can create reusable, platform-independent components with the Java Beans API. All the components can be combined with the use of Java Beans-compliant application builder tools into applets, applications, or composite components. Beans are known as Java Bean components.

Advantages of Java Bean

The Java Bean technology plays a very important role. Some of the benefits that Java Bean technology provides for a component developer are:

- ❑ All the benefits of Java's "write-once, run-anywhere" paradigm are present in the Bean.
- ❑ The properties, events, and methods of a Bean are viewed by another application, such as a builder tool, which can be monitored.
- ❑ To configure a Bean, virtual software can be used. This software is only required when the design-time arguments for that component are being set. In the run-time environment, it does not need to be added.
- ❑ In the persistent storage, the configuration settings of a Bean can be stored and resumed at a later time.
- ❑ You can also use Bean to register the receive events from other objects and generate events that are sent to other objects.

Introspection

Introspection is present at the core of Java Beans. It is defined as looking up or examining a Bean to determine its functionalities. It allows another application, such as a design tool, to obtain information about a component; therefore, it is an essential feature of the Java Bean API. The Java Bean technology cannot operate without introspection.

There are two methods through which the developer can specify the properties, methods, and events needed to be explored by an application builder tool. The first method uses the naming conventions that allow ways to inspect the information about the Bean. In the second method, an additional class that extends the `BeanInfo` interface is provided, which explicitly provides this information.

Design Patterns for Properties

Design patterns are used as solutions to specific problems and so they are not the final designs. They provide developers a common language understandable at both ends. For example, if one developer understands about `builder`, then the other developer knows exactly what it is. A design pattern can be reused in multiple areas, and that is the main benefit of using it. It solves the problems that occur in various situations. A new code can be created by using a design pattern. Object-oriented design patterns represent various relationships and interactions taking place between classes or objects without showing the final application classes or objects that are added.

Each design pattern consists of the following parts:

1. **Problem/Requirement**—To create a design pattern, the first step is to determine the requirements of the problem to solve and the mini analysis of design followed by coding to test. This is usually a common problem that will arise in more than one application.

2. **Forces**—Forces indicate the technical boundaries that direct toward the solution of a problem.
3. **Solution**—This section tells how to write the code to solve a problem. This is the design part of the design pattern. It may contain class diagrams, sequence diagrams, and others to indicate how to write the code for the solution.

A property represents a subgroup of a Bean's state. The properties of a component are assigned some values that represent its behavior and appearance. Properties can be of three types: simple, Boolean, and indexed.

Simple Properties

A simple property contains a single value. The following design patterns are used to identify it:

```
public K getName();
public void setName(K arg);
```

where Name is the name of the property and K is its type.

A read/write property contains both the `get()` and `set()` methods to access the values. A read-only property contains a `get()` method and a write-only property contains a `set()` method.

Boolean Properties

A Boolean property consists of only two values, either true or false. It can be identified by the following design patterns:

```
public boolean isN();
public boolean getN();
public void setN(boolean value);
```

where N is the name of the property.

Indexed Properties

An indexed property can have multiple values. The following design patterns are used to identify an indexed property:

```
public K getName(int index);
public void setName(int index, K value);
public void setName(K values[]);
```

where Name is the name of the property and K is its type.

We will discuss these properties later in this chapter.

Design Patterns for Events

The delegation event model has been discussed earlier in this book and it is used by the Beans. The Beans can perform various operations, such as they can fire events and transfer them to other objects. These can be identified by the following design patterns, where T is the type of the event:

```
public void addTListener(TListener eventListener);
public void removeTListener(TListener eventListener);
```

To insert or delete a listener for the specified event, these methods are used. For example, assuming an event interface type called `TemperatureListener`, a Bean that examines temperature might supply the following mentioned methods:

```
public void addTemperatureListener(TemperatureListener t1) { . . . }
public void removeTemperatureListener(TemperatureListener t1) { . . . }
```

Methods and Design Patterns

If a problem occurs repeatedly, a solution needs to be implemented. That solution is called a pattern. The design patterns provide language-independent strategies to solve problems related to the object-oriented design. While making a design, the names of some common solutions must be known beforehand. Design patterns make the communication effective. Further, they are not used to name non-property methods. The introspection method finds all of the public methods of a Bean protected, while the private methods are not viewed.

Using the BeanInfo Interface

We have discussed earlier that design patterns implicitly determine the information available to the user. The role of the `BeanInfo` interface is to enable you to explicitly control what information is available. The `BeanInfo` interface defines several methods. To discuss it further, let's introduce the following methods:

```
PropertyDescriptor[] getPropertyDescriptors()
EventSetDescriptor[] getEventSetDescriptors()
MethodDescriptor[] getMethodDescriptors()
```

The role of the return array of objects is to provide information about the properties, events, and methods of a Bean. With respect to this, the classes `getPropertyDescriptors()`, `getEventSetDescriptors()`, and `getMethodDescriptors()` methods are well defined within the `java.beans` package, and these methods describe the indicated elements. By implementing these methods, a developer can designate exactly what is shown to a user, by passing introspection based on design patterns.

We will discuss these methods later in detail.

Persistence

Persistence means saving the current position, properties, and instance variables of a Bean. Basically, it means restoring the current position of a Bean with the values of its properties and instance variables to nonvolatile storage. One way to serialize a Bean is to implement the `java.io.Serializable` interface, which acts like a marker interface. Implementing the `java.io.Serializable` interface makes serialization automatic.

We will discuss it later in detail.

Customizers

The role of the customizer, which is provided by a Bean developer, is to help another developer configure the Bean.

A customizer provides step-by-step information about the process to be followed to use a component in a specific context.

To develop a customizer, a Bean developer has greater flexibility to differentiate his or her product in the marketplace.

Understanding Java Beans

A Java Bean is any Java class that has limited conventions related to property and event interface definitions. It can be calculated in a visual builder tool and formed into applications.

Introspection is the method by which a builder tool examines how a Bean operates and also discriminates Beans from the Java classes. Beans have their predefined patterns for their method parameters and class definitions; tools that identify these patterns can view inside a Bean and specify its properties and behavior.

Moreover, introspection calculates Bean's state at design time, that is, the time it is gathered as a part from a larger application. The parameters supplied to a method within Beans must follow a specific pattern so as to help the introspection tools in identifying how Beans can be manipulated at runtime and design time.

Designing Programs Using Java Beans

After you have downloaded and installed the Bean Development Kit (BDK), which is currently at <http://www.ecst.csuchico.edu/~amk/foo/csci611/notes/beans/BDK/>, you can work with beans in the BeanBox tool that comes with the BDK. Assuming that you have installed the BDK (the default path varies by system), you can open the BeanBox; for example, in Windows, you use the `run.bat` file in the `beanbox` directory. When you run this batch file, the BeanBox opens, as shown in Figure 11.1.

You can see the available beans in the toolbox on the left. When you add beans to an application, they'll appear in the BeanBox window next to the toolbox. You can also use the Properties window to set the properties that have been designed into a bean, and you can use the Method Tracer window to handle method execution. Here's an example showing how to work with some beans.

Click the Juggler bean in the toolbox, which changes your cursor to a cross. Next, “draw” a Juggler bean in the BeanBox by dragging the mouse (this bean displays the Java mascot image “juggling” coffee beans). Do the same for the bean named `OurButton`. This results in the application under design (Figure 11.2) You can connect the beans in a BeanBox, thus creating a single application from several beans. For example, you can connect the button to the Juggler bean, so when you click the button, the juggler will stop juggling.

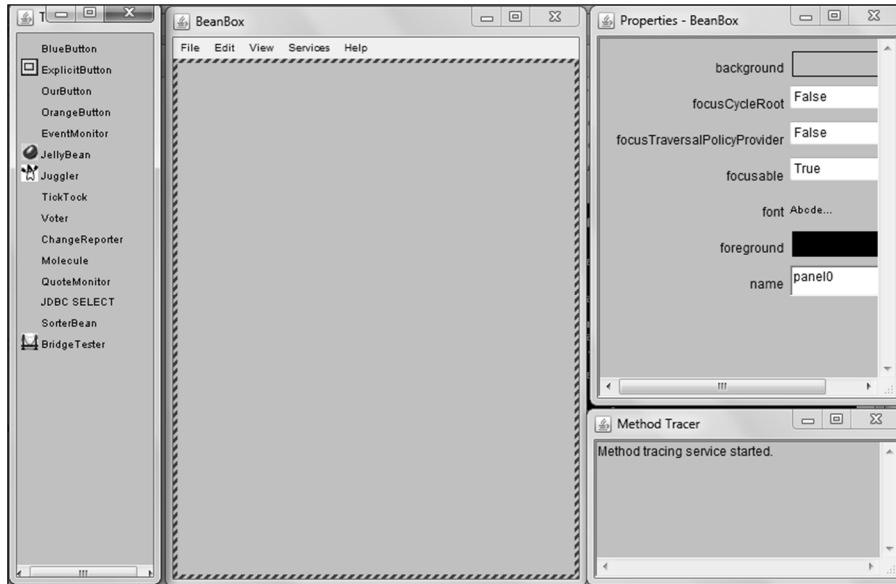


Figure 11.1: Using the BeanBox

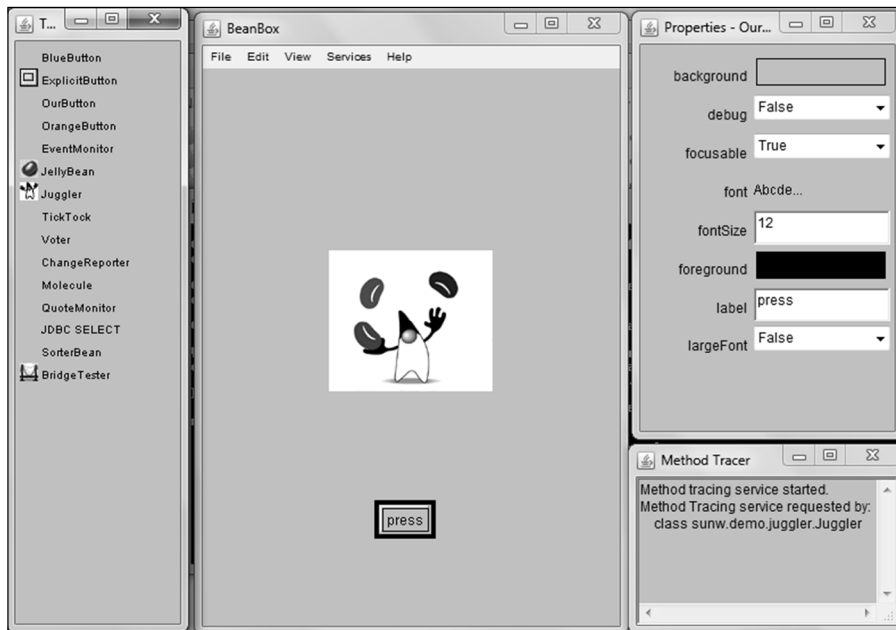


Figure 11.2: Creating an application

First, click the button bean in the BeanBox. The Properties window will display the properties you can set for this bean, including its label—if you want to enter a new label for this button (such as “Click Me”), you can do it in the Properties window (you’ll see how to set the properties of your own beans this way later). To make the button do something when you click it, select the **Edit | Events | action | actionPerformed** menu item now. When you do, a red line appears between the mouse location and the button. Stretch that line to the Juggler bean now and click the Juggler bean, as shown in Figure 11.3. This connects the button and the Juggler bean.

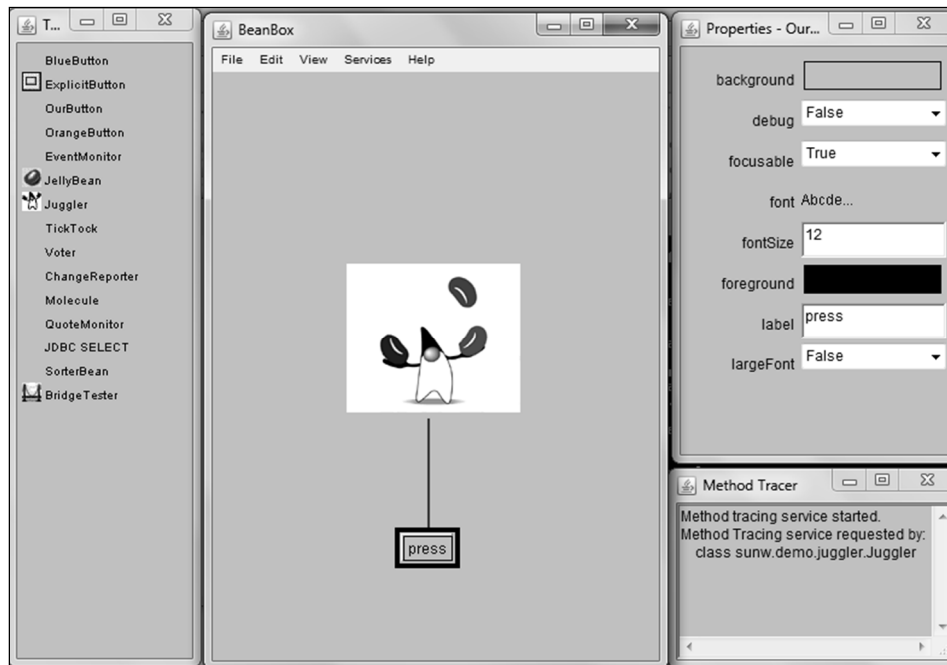


Figure 11.3: Connecting one bean to another

When you click the Juggler bean, the **EventTargetDialog** box appears, as shown in Figure 11.4, displaying the available methods you can call in the Juggler bean when the button is clicked:

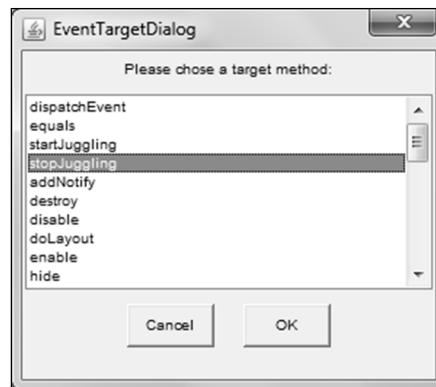


Figure 11.4: The EventTargetDialog box

For this example, choose the `stopJuggling()` method to make the juggler stop juggling; then, click the **OK** button to close the **EventTargetDialog** box.

Now, when you click the button in the BeanBox, the juggler will stop juggling. Congratulations, you have connected two beans, thus creating a new program.

How do you actually run this program outside the BeanBox? Take a look at the next solution.

Creating Applets that Use Java Beans

After you have designed an applet using the BeanBox, you can use the `MakeApplet` item in the File menu to create a JAR file containing your new applet and all the beans it uses. When you select this menu item, it opens the Make an Applet dialog box, you see in Figure 11.5:

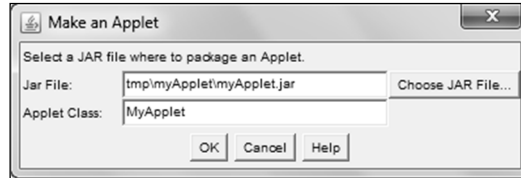


Figure 11.5: The Make an Applet dialog box

In this example, we'll use the applet we developed in the previous solution. We'll stick to the default JAR file name, `myApplet.jar`, and applet class, `MyApplet`. When we click OK, the `myApplet.jar` file is created.

NOTE

If you want to use any of the beans that come with the JDK in an applet JAR file, you'll have to compile them first and make sure the java compiler can find their class files. You'll find those beans in the `demo\sunw\demo` directory. In addition, you'll need to compile the `AppletSupport.java` file in the `beanbox\sunw\beanbox` directory and make sure the java compiler can find the resulting class files.

The `myApplet.jar` file can be used with Web browsers and the Oracle appletviewer. Here's the HTML page that we'll use with the `myApplet.jar` file:

```
<HTML>
<HEAD>
  <TITLE>Beans applet example</TITLE>
</HEAD>
<BODY>
  <APPLET>
    CODE=MyApplet.class
    WIDTH=200
    HEIGHT=200
    <PARAM NAME=archive VALUE="myApplet.jar">
  </APPLET>
</BODY>
</HTML>
```

After running the code, you can see the applet in the appletviewer.

Creating a Java Bean

We'll create a simple Java Bean in this solution to show how beans work, and we'll elaborate on this bean in the rest of the chapter. This bean will just draw itself in red, and when you click it, it'll display a count of the number of times it's been clicked.

We'll place this bean in the JDK's demo directory, so we create a directory named `bean`—`demo\sunw\demo\bean`—and store the class files for this bean in that directory. We'll start the code, `bean.java`, by indicating that this bean is part of a package named `sunw.demo.bean`:

```
package sunw.demo.bean;
import java.awt.*;
import java.awt.event.*;
...
```

As far as the actual bean class goes, we use the `Canvas` class to draw the bean itself (you can also use other classes, such as the `Panel` class, which we'll cover in this chapter). The rest of the code is pretty ordinary; we

just add a mouse listener to the canvas to record mouse clicks and set the size of the canvas (thus setting the size of the bean):

```
package sunw.demo.bean;
import java.awt.*;
import java.awt.event.*;
public class bean extends Canvas
{
    int count;
    public bean()
    {
        addMouseListener(new MouseAdapter()
        {
            public void mousePressed(MouseEvent me) { clicked(); }
        });
        count = 0; setSize(200, 100);
    }
    . . .
}
```

Finally, we implement the method that handles mouse clicks, and we draw the bean, including the click count, in the `paint()` method:

```
package sunw.demo.bean;

import java.awt.*;
import java.awt.event.*;

public class bean extends Canvas
{
    int count;
    public bean()
    {
        addMouseListener(new MouseAdapter() {
            public void mousePressed(MouseEvent me) { clicked(); }
        });
        count = 0; setSize(200, 100);
        public void clicked() { count++; repaint(); }
        public void paint(Graphics g)
        {
            Dimension dimension = getSize();
            int height = dimension.height;
            int width = dimension.width;
            g.setColor(new Color(255, 0, 0));
            g.fillRect(0, 0, --width, --height);
            g.setColor(new Color(0, 0, 0));
            g.drawString("Click count = " + count, 50, 50);
        }
    }
}
```

As you can see, the code for a new Java Bean is pretty simple—essentially, you just draw a component. However, it's not a bean yet—we still have to put it into a JAR file and label it in that JAR file as a bean. To learn more about how this works, see the next solution.

Creating a Bean Manifest File

To make a bean into a bean, you have to store the class file(s) to use in a JAR file, and you use a manifest to indicate which classes are beans. To show you how this works, we'll use the bean we started developing in the previous solution and create a manifest file, `bean.mft`, for it here. We'll place this manifest file in the *demo* directory to make creating the JAR file easier (see the next solution).

To indicate that a class in a JAR file is a Java Bean, you have to set its `Java-Bean` attribute to `True`. Here's how that works with the bean introduced in the previous solution—`bean.class`. This class file is in the `sunw.demo.bean` package, which means it'll be stored in the JAR file as `sunw/demo/bean/bean.class` (like Unix, JAR files use forward slashes as directory separators). To indicate that this class file is a bean, here's what we put in the manifest file, `bean.mft`:

```
Name: sunw/demo/bean/bean.class
Java-Bean: True
```

We'll put this manifest to work in the next solution when we create the JAR file for this bean.

Creating a Bean JAR File

In this solution, we'll create the JAR file for the bean we have been developing over the previous two solutions. First, compile `bean.java`, which will create two class files: `bean.class` and `bean$1.class`. Copy these files to the `demo\sunw\demo\bean` directory we have created. Next, place the manifest file developed in the previous solution, `bean.mft`, into the `demo` directory. Now we are ready to go. Make sure you are in the `demo` directory and use the `jar` tool, like this:

```
C:\...\demo>jar cfm ../jars/bean.jar bean.mft .
```

This creates the new JAR file for this bean, `bean.jar`, and stores it in the `demo\jars` directory, which is where the `BeanBox` will look for it. That's how you install a bean—by placing its JAR file in that directory. The new bean is ready to go, and it's put to work in the next solution.

Creating a New Bean

We have developed a new Java Bean over the previous few solutions and installed it in the `demo\jars` directory. When you open the `BeanBox`, you'll see this bean (which we have just called "bean") listed in the toolbox. You can draw a bean of this new type in the `BeanBox`.

This new bean is already active in the `BeanBox`, as you can see by clicking it, because it shows the number of times it's been clicked. This new Java Bean is a good start, but what about doing some more, such as adding other Java controls to it? Take a look at the next solution.

Adding Controls to Beans

You can add Java controls such as buttons to your beans—you just have to make sure you base your bean on a class that's a container, such as the `Panel` class. Here's an example in which we add a button to a bean and have the bean display the number of times it has been clicked. We start by basing this bean, which we'll call "button," on the `Panel` class and adding it to the `sunw.demo.button` package (which means we'll store its class files in the `demo\sunw\button` directory). Here's how we create the panel, size it, and add a button to it:

```
package sunw.demo.button;
import java.awt.*;
import java.awt.event.*;

public class button extends Panel implements ActionListener
{
    int count;
    Button button1;
    public button()
    {
        count = 0;
        setSize(200, 100);
        button1 = new Button("Click me");
        button1.addActionListener(this);
        add(button1);
    }
}
```

All that's left now is to make the button active by incrementing the click count the button, repainting the bean to display the click count, and creating the `paint()` method:

```
package sunw.demo.button;

import java.awt.*;
import java.awt.event.*;

public class button extends Panel implements ActionListener
{
    int count; Button button1;
```

```

public button()
{
    count = 0;
    setSize(200, 100);
    button1 = new Button("Click me");
    button1.addActionListener(this);
    add(button1);
}
public void actionPerformed(ActionEvent e)
{
    count++;
    repaint();
}
public void paint(Graphics g)
{
    Dimension dimension = getSize();
    int h = dimension.height;
    int w = dimension.width;
    g.setColor(new Color(255, 0, 0));
    g.fillRect(0, 0, w-1, h-1);
    g.setColor(new Color(0, 0, 0));
    g.drawString("Click count = " + count, 50, 50);
}
}

```

As you can see, it's easy to add Java controls to a bean.

Giving a Bean Properties

The properties of a bean let you configure it; setting its caption, size, color, and any other aspect of the bean for which properties are defined. Although any public data member of a bean class can be treated as a property, there's a formal procedure you should follow to inform the Java framework about the properties of your beans: implementing the `BeanInfo` interface. The fields of the `BeanInfo` interface appear in Table 11.1 and its methods in Table 11.2:

Table 11.1: Fields of the BeanInfo interface

Field	Does this
static int ICON_COLOR_16x16	It indicates a 16x16 color icon.
static int ICON_COLOR_32x32	It indicates a 32x32 color icon.
static int ICON_MONO_16x16	It indicates a 16x16 monochrome icon.
static int ICON_MONO_32x32	It indicates a 32x32 monochrome icon.

Table 11.2: Methods of the BeanInfo interface

Method	Does this
BeanInfo[] getAdditionalBeanInfo()	It allows a <code>BeanInfo</code> object to return an arbitrary collection of other <code>BeanInfo</code> objects.
BeanDescriptor getBeanDescriptor()	It gets the bean's bean descriptor.
int getDefaultEventIndex()	It gets the default event. A bean may have a default event (the event that will mostly commonly be used).
int getDefaultPropertyIndex()	It gets the default property index. A bean may have a default property (the property that will mostly commonly be initially chosen for update).
EventSetDescriptor[] getEventSetDescriptors()	It gets the bean's event set descriptors.
Image getIcon (int iconKind)	It gets an image object that can be used to represent the bean in toolboxes, toolbars, and so on.

Table 11.2: Methods of the BeanInfo interface

Method	Does this
MethodDescriptor[] getMethodDescriptors()	It gets the bean's method descriptors.
PropertyDescriptor[] getPropertyDescriptors()	It gets the bean's property descriptors.

In fact, most beans don't implement the `BeanInfo` interface directly. Instead, they extend the `SimpleBeanInfo` class, which implements `BeanInfo`. Here's the inheritance diagram for `SimpleBeanInfo`:

```

java.lang.Object
|____java.beans.SimpleBeanInfo

```

You'll find the constructor of the `SimpleBeanInfo` class in Table 11.3 and its methods in Table 11.4:

Table 11.3: The constructor of the SimpleBeanInfo class

Constructor	Does this
<code>SimpleBeanInfo()</code>	It constructs a <code>BeanInfo</code> object.

Table 11.4: Methods of the SimpleBeanInfo class

Method	Does this
<code>BeanInfo[]</code> <code>getAdditionalBeanInfo()</code>	It is implemented to indicate that there are no other relevant <code>BeanInfo</code> objects.
<code>BeanDescriptor</code> <code>getBeanDescriptor()</code>	It is implemented to indicate the deny of knowledge about the class and the customizer bean.
<code>int</code> <code>getDefaultEventIndex()</code>	It is implemented to deny knowledge of a default event.
<code>int</code> <code>getDefaultPropertyIndex()</code>	It is implemented to deny knowledge of a default property.
<code>EventSetDescriptor[]</code> <code>getEventSetDescriptors()</code>	It is implemented to deny knowledge of event sets.
<code>Image</code> <code>getIcon</code> <code>(int iconKind)</code>	It is implemented to deny there are no icons available.
<code>MethodDescriptor[]</code> <code>getMethodDescriptors()</code>	It is implemented to deny knowledge of methods.
<code>PropertyDescriptor[]</code> <code>getPropertyDescriptors()</code>	It is implemented to deny knowledge of properties.
<code>Image</code> <code>loadImage</code> <code>(String resourceName)</code>	It is used to help in loading icon images.

To actually describe a property, you use the `PropertyDescriptor` class, which in turn is derived from the `FeatureDescriptor` class. Here's the inheritance diagram for the `FeatureDescriptor` class:

```

java.lang.Object
|____java.beans.FeatureDescriptor

```

You'll find the constructor of the `FeatureDescriptor` class in Table 11.5 and its methods in Table 11.6:

Table 11.5: The constructor of the FeatureDescriptor class

Constructor	Does this
<code>FeatureDescriptor()</code>	It constructs a feature descriptor

Table 11.6: Methods of the FeatureDescriptor class

Method	Does this
<code>Enumeration<string></code> <code>attributeNames()</code>	It gets an enumeration of the locale-independent names of this feature.
<code>String</code> <code>getDisplayName()</code>	It gets the localized display name of this feature.

Table 11.6: Methods of the FeatureDescriptor class

Method	Does this
<code>String getName()</code>	It gets the programmatic name of this feature.
<code>String getShortDescription()</code>	It gets the short description of this feature.
<code>Object getValue (String attributeName)</code>	It retrieves a named attribute with this feature.
<code>boolean isExpert()</code>	It returns True for features that are intended for expert users.
<code>boolean isHidden()</code>	It returns True for features that are intended only for tool use.
<code>boolean isPreferred()</code>	It returns True for features that are particularly important for presenting to people.
<code>void setDisplayName (String displayName)</code>	It sets the localized display name of this feature.
<code>void setExpert (boolean expert)</code>	It sets the expert flag for features that are intended for expert users.
<code>void setHidden (boolean hidden)</code>	It sets the hidden flag for features intended only for tool use.
<code>void setName(String name)</code>	It sets the name of this feature.
<code>void setPreferred (boolean preferred)</code>	It sets the preferred flag, used to identify features that are particularly important for presenting to people.
<code>void setShortDescription (String text)</code>	You can associate a short descriptive string with this feature.
<code>void setValue (String attributeName, Object value)</code>	You can associate a named attribute with this feature.

Here's the inheritance diagram for the PropertyDescriptor class:

```

java.lang.Object
|___java.beans.FeatureDescriptor
|___java.beans.PropertyDescriptor

```

You'll find the constructors of the PropertyDescriptor class in Table 11.7 and its methods in Table 11.8:

Table 11.7: Constructors of the PropertyDescriptor class

Constructor	Does this
<code>PropertyDescriptor(String propertyName, Class<?> beanClass)</code>	It constructs a PropertyDescriptor for a property that follows the standard Java convention by having <code>getFoo()</code> and <code>setFoo()</code> accessor methods.
<code>PropertyDescriptor(String propertyName, Class<?> beanClass, String readMethodName, String writeMethodName)</code>	It takes the name of a simple property as well as method names for reading and writing the property.
<code>PropertyDescriptor(String propertyName, Method readMethod, Method writeMethod)</code>	It takes the name of a simple property as well as Method objects for reading and writing the property.

Table 11.8: Methods of the PropertyDescriptor class

Method	Does this
<code>PropertyEditor createPropertyEditor(Object bean)</code>	It creates an instance of a property editor using the current property editor class.
<code>boolean equals(Object obj)</code>	It compares this PropertyDescriptor against the specified object.
<code>Class<?> getPropertyEditorClass()</code>	It gets any explicit PropertyEditor class that has been registered for

Table 11.8: Methods of the PropertyDescriptor class

Method	Does this
	this property.
<code>Class<?> getPropertyType()</code>	It gets the <code>Class</code> object for the property.
<code>Method getReadMethod()</code>	It gets the method that should be used to read the property value.
<code>Method getWriteMethod()</code>	It gets the method that should be used to write the property value.
<code>int hashCode()</code>	It gets a hash code value for the object.
<code>boolean isBound()</code>	It updates to bound properties and that will cause a <code>PropertyChange</code> event to be fired when the property is changed.
<code>boolean isConstrained()</code>	It attempts updates to constrained properties and that will cause a <code>VetoableChange</code> event to be fired when the property is changed.
<code>void setBound (boolean bound)</code>	It updates to bound properties and that will cause a <code>PropertyChange</code> event to be fired when the property is changed.
<code>void setConstrained (boolean constrained)</code>	It attempts updates to constrained properties and that will cause a <code>VetoableChange</code> event to be fired when the property is changed.
<code>void setPropertyEditorClass(Class<?> propertyEditorClass)</code>	Normally, property editors will be found using the property editor manager.
<code>void setReadMethod (Method getter)</code>	It sets the method that should be used to read the property value.
<code>void setWriteMethod (Method setter)</code>	It sets the method that should be used to write the property value.

Let's look at an example that implements a property in a Java Bean. In this case, we'll add a property named `filled` to the click-counting operation developed in this chapter. This property is a Boolean property that, when `True`, makes sure the bean will be filled in with color. We'll call this new bean `Bean2`.

To keep track of the new `filled` property, we'll add a private Boolean variable of that name to the `Bean2` class:

```
package sunw.demo.bean2;
import java.awt.*;
import java.awt.event.*;
public class Bean2 extends Canvas {
    private boolean filled;
    . . .
```

We initialize this property to `False` when the bean is created:

```
package sunw.demo.bean2;

import java.awt.*;
import java.awt.event.*;

public class Bean2 extends Canvas
{
    private boolean filled;
    int count;
    public Bean2()
    {
        addMouseListener(new MouseAdapter()
        {
            public void mousePressed(MouseEvent me) { clicked(); }
        });
        count = 0; filled = false;
        setSize(200, 100);
    }
    . . .
}
```

When you implement a property, Java will look for two methods: `getPropertyName()` and `setPropertyName()`, where `PropertyName` is the name of the property. The `get()` method returns the

current value of the property, which can be any supported type, and the `set()` method takes an argument of that type, which you are supposed to set the property to. Here's how we implement the `getfilled()` and `setfilled()` methods:

```
package sunw.demo.bean2;

import java.awt.*;
import java.awt.event.*;

public class Bean2 extends Canvas
{
    private boolean filled;
    int count;
    public Bean2()
    {
        addMouseListener(new MouseAdapter() {
            public void mousePressed(MouseEvent me) { clicked(); }
        });
        count = 0;
        filled = false;
        setSize(200, 100);
    }
    public void clicked()
    {
        count++;
        repaint();
    }
    public boolean getfilled() { return filled; }
    public void setfilled(boolean flag)
    {
        this.filled = flag;
        repaint();
    }
    . . .
}
```

All that's left is to put the `filled` property to use. If it's `True`, the bean should be filled in with color, so here's the code we add to the `paint()` method:

```
public void paint(Graphics g)
{
    Dimension dimension = getSize();
    int height = dimension.height;
    int width = dimension.width;
    if(filled)
    {
        g.setColor(new Color(255, 0, 0));
        g.fillRect(0, 0, --width, --height);
    }
    g.setColor(new Color(0, 0, 0));
    g.drawString("Click count = " + count, 50, 50);
}
```

Now we have to create a new class, `Bean2BeanInfo`, which will return information about this new bean property. This class will go into the same package as the bean itself, but it's based on the `SimpleBeanInfo` class:

```
package sunw.demo.bean2;
import java.beans.*;
public class Bean2BeanInfo extends SimpleBeanInfo { . . . }
```

To let Java know about properties, you implement the `getPropertyDescriptors()` method, which returns an array of `PropertyDescriptor` objects. Each `PropertyDescriptor` object holds the name of a property and points to the class that supports that property. Here's how we create a `PropertyDescriptor` object for the `filled` property:

```
package sunw.demo.bean2;
import java.beans.*;
```

```

public class Bean2BeanInfo extends SimpleBeanInfo {
    public PropertyDescriptor[] getPropertyDescriptors() {
        try {
            PropertyDescriptor filled = new
            PropertyDescriptor("filled", Bean2BeanInfo.class);
            . . . }
        catch(Exception e) { }
        return null; }
    }

```

This is the only property in Bean2, so we add the new `PropertyDescriptor` object to an array and return it, like this:

```

package sunw.demo.bean2;
import java.beans.*;
public class Bean2BeanInfo extends SimpleBeanInfo
{
    public PropertyDescriptor[] getPropertyDescriptors()
    {
        try {
            PropertyDescriptor filled = new
            PropertyDescriptor("filled", Bean2BeanInfo.class);
            PropertyDescriptor propertydescriptor[] = {filled};
            return propertydescriptor;
        }
        catch(Exception e) {}
        return null;
    }
}

```

After compiling this new class (`Bean2BeanInfo.java`), we place `Bean2BeanInfo.class` in the directory `demo\sunw\demo\bean2`, along with the classes created when we compiled `Bean2.java`. We'll need a new manifest file that includes the `Bean2BeanInfo` class. Here's what `bean2.mft` looks like:

```

Name: sunw/demo/bean2/Bean2.class
Java-Bean: True
Name: sunw/demo/bean2/Bean2BeanInfo.class
Java-Bean: True

```

We place this new manifest file in the `demo` directory. Finally, here's how we create the new `bean2.jar` file and install it:

```

C:\...\demo>jar cfm ../jars\bean2.jar bean2.mft .

```

Now when we run the `BeanBox` and add a new `bean2` bean to the `BeanBox`, the new `filled` property will appear in the `Properties` window. Setting `filled` to `True` causes the bean to be filled with color, which is also shown in the figure.

Design Patterns for Properties

The subset of a Bean's state is a property. The behavior and appearance of that component is determined by the values assigned to the properties. The `setter()` method sets a property and the `getter()` method retrieves that property. There are three types of properties, namely, simple, Boolean, and indexed.

Using Simple Properties

The characteristic of the simple property is that it has a single value. It is specified by the following design patterns, where `N` is the name of the property and `T` is its type:

```

public T getN();
public void setN(T arg);

```

To access its values, a read/write property has both of these methods. A read-only property has only a `get()` method and a write-only property has only a `set()` method. The following are three read/write simple properties along with their `getter()` and `setter()` methods:

```
private double depth, height, width;
public double getDepth() { return depth; }
public void setDepth(double dbl) { depth = dbl; }
public double getHeight() { return height; }
public void setHeight(double hdbl) { height = hdbl; }
public double getWidth() { return width; }
public void setWidth(double wdbl) { width = wdbl; }
```

Adding a Color Property to SimpleBean

To add a Color property, you have to make the following changes to `SimpleBeanapp.java`:

```
Create and initialize a private instance variable.
private Color color = Color.green;
Write a public getter method.
public Color getColor() { return color; }
Write a public setter method.
public void setColor(Color newClr) {
    color = newClr;
    repaint(); }
Override the paint() method inherited from Canvas.
public void paint(Graphics g) {
    g.setColor(color);
    g.fillRect(20, 5, 20, 30);
}
```

Compile, load in the Toolbox, and create an instance of the Bean. The following results are discussed here:

- ❑ `SimpleBean` is shown with a green-centered rectangle.
- ❑ Consequently, the Properties window will include a new color property. Simultaneously, the introspection process will also look for a color property editor. A color property editor is one of the default editors supplied with the `BeanBox` class, and is defined as `SimpleBean`'s color property editor. To execute this editor, click on the color property in the Properties window.
- ❑ In simple language, Figure 11.6 shows the new `SimpleBean` instance within the `BeanBox`; `SimpleBean`'s new color property within the Properties window and the color property editor loaded with the `BeanBox`.

By menu or by RGB value, you can change the color property. Therefore, you can try to change colors. The following is the complete `SimpleBeanapp` source code that has been revised to add a color property:

```
import java.awt.*;
import java.io.Serializable;
public class SimpleBeanapp extends Canvas implements Serializable {
    private Color clr = Color.green;
    public Color getColor() { return clr; }
    public void setColor(Color newClr) {
        clr = newClr;
        repaint(); }
    public void paint(Graphics g) {
        g.setColor(clr);
        g.fillRect(20, 5, 20, 30); }
    public SimpleBeanapp() {
        setSize(60,40);
        setBackground(Color.red); }
}
```

The result is as shown in Figure 11.6:

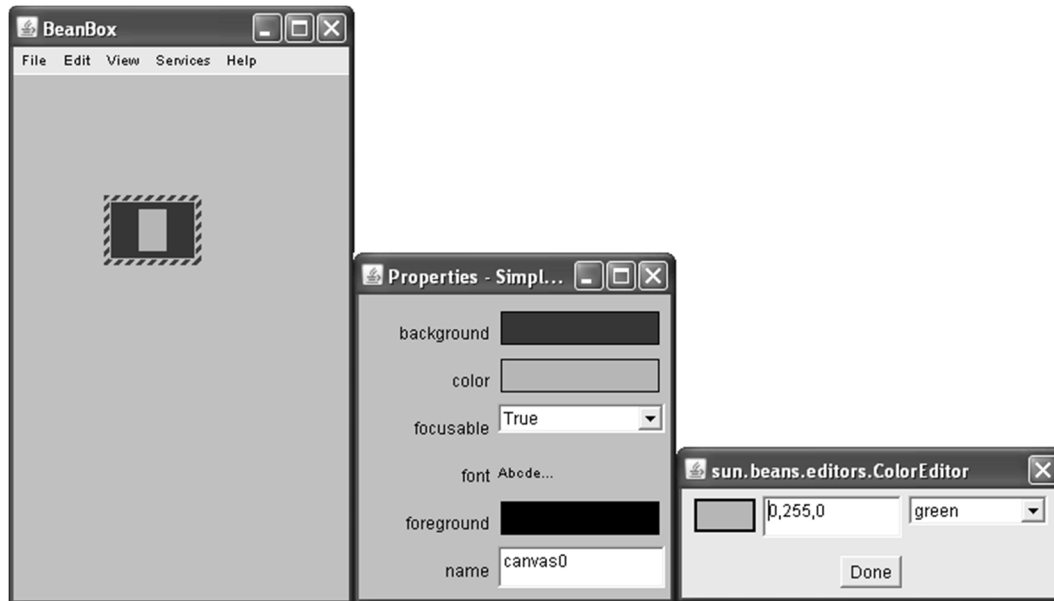


Figure 11.6: Showing SimpleBean Instance within the BeanBox

Using Indexed Properties

Indexed properties consist of multiple values. By the following design pattern, it can be identified. In this design pattern, N is the name of the property and T is its type:

```
public T getN(int index);
public void setN(int index, T value);
public T[] getN();
public void setN(T values[]);
```

Here is an indexed property called data along with the getter () and setter () methods:

```
private double data[];
public double getData (int ind) { return data[ind]; }
public void setData(int ind, double dblvalue) { data[ind] =  dblvalue; }
public double [] getData() { return data; }
public void setData(double[] dblvalue) {
    data = new double[dblvalue.length];
    System.arraycopy(dblvalue, 0, data, 0, 0, dblvalue.length);
}
```

Using Boolean Properties

A Boolean property consists of only two values, either true or false. It can be identified by the following design patterns:

```
public boolean isN();
public boolean getN();
public void setN(boolean value);
```

where N is the name of the property.

To retrieve the values of a Boolean property, either the first or second pattern is used; if a class consists of both of its methods, then it uses the first pattern. The following example shows a class that has one Boolean property:

```
public class polygon
{
```

```

private boolean square = false;
public boolean issquare( )
{
    return square;
}
public void setSquare(boolean square)
{
    this.square = square;
}
}

```

Designing Patterns for Events

The delegation event model has been discussed earlier in this book and it is used by the Beans. The Beans operate differently, such as they can generate events and provide to other objects. These can be identified by the following design patterns, where *T* is the type of the event:

```

public void addTListener(TListener eventListener);
public void removeTListener(TListener eventListener);

```

The preceding methods help to add new events or deleting the already existing ones. For instance, consider an event interface type known as *TemperatureListener*; a Bean that monitors temperature can supply the following methods:

```

public void addTemperatureListener(TemperatureListener t1) { . . . }
public void removeTemperatureListener(TemperatureListener t1) { . . . }

```

Learning Methods and Design Patterns

For naming non-property methods, design patterns are not used. Introspection searches all of the public methods of a Bean protected, while the private methods are not available.

Using the *BeanInfo* Interface

As we discussed in the *In Depth* section, the *BeanInfo* interface defines several methods. To discuss it further, let's introduce the following methods:

```

PropertyDescriptor[] getPropertyDescriptors()
EventSetDescriptor[] getEventSetDescriptors()
MethodDescriptor[] getMethodDescriptors()

```

Here, the role of the return array of objects is to provide information about the properties, events, and methods of a Bean. In this regard, the classes *PropertyDescriptor*, *EventSetDescriptor*, and *MethodDescriptor* are well defined within the *java.beans* package, and these methods describe the marked elements. With the implementation of these methods, a developer can design the same what is presented to the user, bypassing introspection based on design patterns.

At the time of creating a class that implements *BeanInfo* interface, you have to remember that you must call that class *bnameBeanInfo*, where *bname* is the name of the Bean. For instance, assume that the name of the Bean is *ExampleBean*. In this case, the name of the information class will be called *ExampleBeanBeanInfo*.

Now, the question is how to simplify the use of *BeanInfo*? In this package, to simplify the *BeanInfo*, Java Beans supplies the *SimpleBeanInfo* class. It provides implementations of the *BeanInfo* interface by default, taking the three methods into consideration. However, you are free to extend this class and override one or more of the methods to explicitly control what aspects of a Bean are exposed. If you fail to override a method, then the design-pattern introspection will be used to find Bean's properties. For example, if you don't override *setPropertyDescriptors()* method, then design patterns are used to find Bean's properties.

Using Feature Descriptors

As discussed earlier, *BeanInfo* classes contain *descriptors* that accurately explain the target Bean's features. The BDK implements the following descriptor classes:

- ❑ The first type of descriptor is `FeatureDescriptor`. It is the base class for the other descriptor classes. It explains the features that are common to all descriptor types.
- ❑ The second type of descriptor is `BeanDescriptor`. It explains the target Bean's class type and name, and describes the target Bean's customizer class, if it exists.
- ❑ The third type is `PropertyDescriptor`, and it provides details on the target Bean's properties.
- ❑ The fourth descriptor is `IndexedPropertyDescriptor`. It is a subclass of `PropertyDescriptor` and discusses the target Bean's indexed properties.
- ❑ The fifth type of descriptor is `EventSetDescriptor`. The role of this type is to describe the events the target Bean fires.
- ❑ The sixth type is `MethodDescriptor`. It describes the target Bean's methods.
- ❑ The seventh type of feature descriptor is `ParameterDescriptor`. As the name indicates, it describes the method parameters.
- ❑ The `BeanInfo` interface declares methods that return arrays of the previous descriptors.

Creating Bound Properties

You can also create *bound* properties in Java Beans. Bound properties generate an event when their values change. This event is of type `PropertyChangeEvent` and is sent to all registered event listeners of this type. To make a property a bound property, use the `setBound()` method like this (note that we are adapting code from the previous solution):

```
package sunw.demo.bean2;
import java.beans.*;
public class Bean2BeanInfo extends SimpleBeanInfo
{
    public PropertyDescriptor[] getPropertyDescriptors()
    {
        try {
            PropertyDescriptor filled = new
                PropertyDescriptor("filled", Bean2.class);
            filled.setBound(true);

            PropertyDescriptor propertydescriptor[] = {filled};
            return propertydescriptor;
        }
        catch(Exception e) {}
        return null;
    }
}
```

Giving a Bean Methods

You can formally describe methods of Java Beans to the Java framework using the `MethodDescriptor` class. Here's the inheritance diagram of this class:

```
java.lang.Object
|____java.beans.FeatureDescriptor
|____java.beans.MethodDescriptor
```

You'll find the constructors of the `MethodDescriptor` class in Table 11.9 and its methods in Table 11.10:

Table 11.9: Constructors of the <code>MethodDescriptor</code> class	
Constructor	Does this
<code>MethodDescriptor</code> (<code>Method method</code>)	It constructs a method descriptor.
<code>MethodDescriptor</code> (<code>Method method</code> , <code>ParameterDescriptor[]</code> <code>parameterDescriptors</code>)	It constructs a method descriptor from a method, providing descriptive information for each of the method's parameters.

Table 11.10: Methods of the MethodDescriptor class

Methods	Does this
Method <code>getMethod()</code>	It gets the method that this method descriptor encapsulates.
ParameterDescriptor[] <code>getParameterDescriptors()</code>	It gets the parameter descriptor for each of the parameters of this method descriptor's methods.

Many beans, however, don't bother to use `MethodDescriptor` objects, because any public bean method is accessible from other beans. Here's an example in which we create a new bean based on the ones we have developed in this chapter. This bean, `Bean3`, will count the number of times it's been clicked and will also support a method named `increment()` that, when invoked, will increment the click count.

Here's the code for `Bean3`, including the public `increment()` method:

```
package sunw.demo.bean3;
import java.awt.*;
import java.awt.event.*;
public class Bean3 extends Canvas
{
    int count;
    public Bean3()
    {
        addMouseListener(new MouseAdapter()
        {
            public void mousePressed(MouseEvent me) { clicked(); }
        });
        count = 0;
        setSize(200, 100); }
    public void clicked()
    {
        count++;
        repaint(); }
    public void increment()
    {
        count++;
        repaint(); }
    public void paint(Graphics g)
    {
        Dimension dimension = getSize();
        int height = dimension.height;
        int width = dimension.width;
        g.setColor(new Color(255, 0, 0));
        g.fillRect(0, 0, --width, --height);
        g.setColor(new Color(0, 0, 0));
        g.drawString("Click count = " + count, 50, 50); }
}
```

After creating `Bean3` and adding it to the `BeanBox`, we can connect other beans to the `increment()` method. We have connected a button to that method, and each time the button is clicked, the click count in the `Bean3` bean is incremented and displayed.

Giving a Bean an Icon

You may have noticed that some beans have icons in the `BeanBox`. You can add your own icons to your beans — all you have to do is add a `getIcon()` method to the `BeanInfo` class. Here's how to implement this method and handle all possibilities — monochrome or color icons of either 16×16 or 32×32 pixels:

```
public java.awt.Image getIcon(int iconKind) {
    if (iconKind == BeanInfo.ICON_MONO_16x16 ||
        iconKind == BeanInfo.ICON_COLOR_16x16 ) {
        java.awt.Image image = loadImage("Icon16.gif");
```

```

return image; }
if (iconKind == BeanInfo.ICON_MONO_32x32 ||
    iconKind == BeanInfo.ICON_COLOR_32x32 ) {
    java.awt.Image image = loadImage("Icon32.gif");
    return image; }
return null;
}

```

Creating a **BeanInfo** Class

This part of the chapter uses the `ExplicitButtonBeanInfo` demo class to explain the creation of the `BeanInfo` class. There are different steps for creating a `BeanInfo` class:

First name your `BeanInfo` class. As a naming convention, you must add “`BeanInfo`” to the target class name. For example, if the target class name is `CreateButton`, then its bean information class name should be `CreateButtonBeanInfo`. Then make a `BeanInfo` class by extending the `SimpleBeanInfo` class as follows:

```
public class ExplicitButtonBeanInfo extends SimpleBeanInfo {
```

By extending the `SimpleBeanInfo` class, you only have to override those methods you need, not all the methods of the `SimpleBeanInfo` class. So, override those methods which return the properties, methods, or events that you need. In the following code, the `ExplicitButtonBeanInfo` demo class overrides the `getPropertyDescriptors()` method to return four properties:

```

public PropertyDescriptor[] getPropertyDescriptors()
{
    try {
        PropertyDescriptor background =
            new PropertyDescriptor("background", beanClass);
        PropertyDescriptor foreground =
            new PropertyDescriptor("foreground", beanClass);
        PropertyDescriptor font =
            new PropertyDescriptor("font", beanClass);
        PropertyDescriptor label =
            new PropertyDescriptor("label", beanClass);
        background.setBound(true);
        foreground.setBound(true);
        font.setBound(true);
        label.setBound(true);
        PropertyDescriptor rv[] =
            {background, foreground, font, label};
        return rv; }
    catch (IntrospectionException e) { throw new Error(e.toString()); }
}

```

If overriding is not done on the `SimpleBeanInfo` default method, which returns null, low-level reflection is used for that characteristic. After overriding the appropriate method, associate an icon with the target Bean as follows:

```

public java.awt.Image getIcon(int icontype)
{
    if (icontype == BeanInfo.ICON_MONO_16x16 ||
        icontype == BeanInfo.ICON_COLOR_16x16 ) {
        java.awt.Image img = loadImage("tom.gif");
        return img; }
    if (icontype == BeanInfo.ICON_MONO_32x32 ||
        icontype == BeanInfo.ICON_COLOR_32x32 ) {
        java.awt.Image img = loadImage("ExplicitButtonIcon32.gif");
        return img; }
    return null;
}

```

The `BeanBox` then presents this icon next to the Bean name in the `ToolBox`. You can expect the builder tools to do the same. Specify the target Bean class, and if the Bean has a customizer, specify it too as follows:

```

    public BeanDescriptor getBeanDescriptor() {
        return new BeanDescriptor(beanClass, customizerClass);
    }
    . . .
    resulting, private final static Class beanClass = ExplicitButton.class;
    private final static Class customizerClass = OurButtonCustomizer.class;

```

In the previous steps, locate the `BeanInfo` class in the same directory as its target class. Firstly, the `BeanBox` searches for a target Bean's `BeanInfo` class in the target Bean's package path. If no `BeanInfo` is found, then secondly the Bean information package searches path (maintained by the `Introspector`). The default Bean information search path is `sun.beans.infos`. Again, if no `BeanInfo` class is found, then low-level reflection is used to discover a Bean's features.

Setting Bound and Constrained Properties

A Bean that contains a bound property generates an event when the property is changed. Here, the event is of type `PropertyChangeEvent`. And, it is sent to objects that previously registered an interest in receiving such notifications. A class that handles this event must implement the `PropertyChangeListener` interface.

The Java Beans model supports two variations on standard properties—the first one is called bound properties and the second one is called constrained properties.

The task of the bound properties is to provide the registration and notification of parties whenever there is modification in the values. The constrained properties take this notification model to a step ahead by allowing the notified party to perform a veto in order to prevent the property change. Different from event handling, Java Beans handle the bound and constrained properties.

To whom are the Bound properties useful? Bound properties are useful for Beans that want to allow instances of the same Bean class or some other Bean class to monitor a property value and subsequently change their values accordingly (i.e. to match the “trend setting” Bean). Let's cite an example. Consider a GUI Bean that wants to allow other GUI Beans to keep a check on change in background color to update their backgrounds accordingly.

However, note that implementation of a bound property is quite easy because of the user-friendly underlying framework provided by the Java Beans architecture. Bean class supports a bound property. In order to provide a bound property, the Bean class must instantiate an object in the Java Beans framework, which provides a lot of this functionality, and implements registration and unregistration methods that simply invoke the appropriate methods in the Java Beans framework:

```

    private PropertyChangeSupport changes = new PropertyChangeSupport(this);
    public void addPropertyChangeListener(PropertyChangeListener p) {
        changes.addPropertyChangeListener(p);
    }
    public void removePropertyChangeListener(PropertyChangeListener p) {
        changes.removePropertyChangeListener(p);
    }

```

Each bound property must then invoke the `firePropertyChange()` method from its `set()` method:

```

    public void setMood(int mood) {
        int old = this.mood;
        this.mood = mood;
        repaint();
        changes.firePropertyChange("mood",
            new Integer(old), new Integer(mood));
    }

```

At this level, the `PropertyChangeSupport` object handles the notification of all registered targets. You have to keep in mind that the `PropertyChangeSupport` object provides a general-purpose functionality following a prescribed protocol. Particularly, the method invocation for `firePropertyChange()` must provide the property name as well as the old and new values, which are passed along to notified targets.

Moreover, the listener (i.e., target object) must provide a `propertyChange()` method to receive the property-related notifications:

```
public void propertyChange(PropertyChangeEvent e) { } // . . .
```

Here, constrained properties add the functionality, which might be the notified listener that can object to the property change and execute a veto. Additionally, the Bean class must instantiate the Java Beans object that provides this service, namely, `VetoableChangeSupport`, and executes the corresponding registration-related methods:

```
private VetoableChangeSupport veto = new VetoableChangeSupport(this);
public void addVetoableChangeListener(VetoableChangeListener vetolisten) {
    veto.addVetoableChangeListener(vetolisten);
}
public void removeVetoableChangeListener(VetoableChangeListener vetolisten) {
    veto.removeVetoableChangeListener(vetolisten);
}
```

Keep in mind that a Bean could provide one or more bound and one or more constrained properties. In this case, it must instantiate both `PropertyChangeSupport` and `VetoableChangeSupport` and provide both sets of registration-related methods. Also, constrained properties tend to be bound, but that is not an absolute requirement. The `set()` method for bound-constrained properties is slightly more complicated:

```
public void setMood(int md)
    throws PropertyVetoException {
    vetoes.fireVetoableChange("mood",
        new Integer(this.md), new Integer(md));
    int old = this.md;
    this.md = md;
    repaint();
    changes.firePropertyChange("mood",
        new Integer(old), new Integer(md));
}
```

In particular, the `set()` method must accommodate the exception `PropertyVetoException`. In this regard, the sequence of operations is as follows:

- ❑ Fire the vetoable change notification.
- ❑ Update the appropriate state variables.
- ❑ Fire the standard property change notification, if bound.

A veto-interested target object must implement the `vetoableChange()` method:

```
public void vetoableChange(PropertyChangeEvent e)
    throws PropertyVetoException { . . . }
```

It exercises a veto by the following exceptions:

- ❑ Including a throws clause for `PropertyVetoException` and
- ❑ Raising the exception (`throw new PropertyVetoException()`), as appropriate.

Here, the Java Beans framework receives the propagated exception. In return, the Java Beans framework transfers it to `setMood()` through the `fireVetoableChange()` invocation. Note that `setMood()` is organized such that in the event of a veto: (1) it does not handle the exception (instead, returning and transferring the exception) and (2) it does not complete the update to state variables or the property notification for bound-property listeners.

Now, it is very much clear that the Java Beans framework supports a lot of property-related functionality and requires minimal work from the Bean implementation.

Implementing Persistence

As we have discussed in the *In Depth* section persistence is the process of restoring the current state of a Bean and its properties with its instance variables also being restored at later stage. What helps to provide persistence to Beans? The object serialization capabilities provided by the Java class libraries provide persistence to Beans.

Among the different methods available, the easiest way to serialize a Bean is to have it implement the `java.io.Serializable` interface, which is nothing but a marker interface. Implementing `java.io.Serializable` makes serialization automatic. Your Bean need not take any other action in this regard. Moreover, remember that automatic serialization can also be inherited. So, if any superclass of a Bean executes `java.io.Serializable`, then automatic serialization is achieved. But, the only restriction is that any class that implements `java.io.Serializable` must supply a parameter-less constructor.

When the user is using automatic serialization, they can optionally avoid a field from being saved by the use of the `transient` keyword. As a result, data members of a Bean specified as `transient` will not be serialized.

If a Bean does not implement `java.io.Serializable`, then we must provide serialization ourselves. This can be done by implementing `java.io.Externalizable`. Else, the configuration of your component cannot be saved by containers.

Using the Java Beans API

Table 11.11 lists the interfaces in `java.beans` and provides description of their functionality, whereas Table 11.12 lists the classes in `java.beans`:

Table 11.11: The interfaces of java.beans	
Interface	Description
<code>AppletInitializer</code>	This interface is used to initialize the Applet with Java Bean.
<code>BeanInfo</code>	Any class that implements the <code>BeanInfo</code> interface provides explicit information about the methods, properties, events, etc. of their bean.
<code>Customizer</code>	This interface is used to customize a target Java Bean.
<code>DesignMode</code>	This interface is used to define the notation of design time as a mode in which <code>JavaBean</code> instances should function.
<code>ExceptionListener</code>	This exception is thrown during any problem in listening.
<code>PropertyChangeListener</code>	This interface is used to change the property of listener.
<code>PropertyEditor</code>	This interface provides the methods that is used to edit the property.
<code>VetoableChangeListener</code>	This interface is used to notify the any changes made in constrained property.
<code>Visibility</code>	This interface determines that bean needs GUI or not and whether GUI is available.

Table 11.12: The classes of java.beans	
Class	Description
<code>BeanDescriptor</code>	It provides the complete information about the bean.
<code>Beans</code>	Basic bean methods.
<code>DefaultPersistenceDelegate</code>	It is a concrete implementation of the abstract <code>PersistenceDelegate</code> class and is the delegate used by default for classes about which no information is available.
<code>Encoder</code>	Its methods are used to encode the Java Beans.
<code>EventHandler</code>	The <code>EventHandler</code> class is used by developers to make connections between user interface bean (source) and an application logic bean (target).
<code>EventSetDescriptor</code>	It describe a collection of events that delivered as method calls on an event listener interface.
<code>Expression</code>	It is the subclass of <code>java.beans.Statement</code> and returns the value when expression is evaluated.
<code>FeatureDescriptor</code>	It is the superclass of many descriptor classes like <code>PropertyDescriptor</code> , <code>EventSetDescriptor</code> , and <code>MethodDescriptor</code> .

Table 11.12: The classes of java.beans

Class	Description
IndexedPropertyChangeEvent	It gets delivered whenever a component of Java Beans changes a bound indexed property.
IndexedPropertyDescriptor	It describes a property that acts like an array having index read or write method of the array.
Introspector	It provides a standard way for tools to learn about the properties, events, and methods supported by a target Java Bean.
MethodDescriptor	It describes a particular method that a Java Bean supports for external access from other components
ParameterDescriptor	It allows bean implementers to provide additional information on each of their parameters.
PersistenceDelegate	It provides the information about the state of an instance of a given class in terms of the methods in the class's public API
PropertyChangeEvent	It delivered whenever a bean changes a "bound" or "constrained" property.
PropertyChangeListenerProxy	It extends the <code>EventListenerProxy</code> , specifically for adding a named <code>PropertyChangeListener</code> .
PropertyChangeSupport	It is a utility class that can be used by beans that support bound properties.
PropertyDescriptor	It describes one property that a Java Bean get through a pair of accessor methods.
PropertyEditorManager	It <code>PropertyEditorManager</code> used to get a property editor for any given type name.
PropertyEditorSupport	This is a support class which helps to build property editors
SimpleBeanInfo	It is a support class to make it easier for people to provide the <code>BeanInfo</code> classes.
Statement	It represents a primitive statement in which a single method is applied to a target and a set of arguments - as in " <code>a.setFoo(b)</code> ".
VetoableChangeListenerProxy	A class which extends the <code>EventListenerProxy</code> , specifically for associating a <code>VetoableChangeListener</code> with a "constrained" property
VetoableChangeSupport	It is a utility class that can be used by beans that support constrained properties.
XMLDecoder	It is used to read XML documents created using the <code>XMLEncoder</code> and is used just like the <code>ObjectInputStream</code> .
XMLEncoder	It is an alternative class, like <code>ObjectOutputStream</code> , which is used to generate a textual representation of a Java Bean.

Learning the Basics of an Event

Java event model is used by Java Beans to make communication happen. Here, events provide second option to (synchronous) method invocations for any type of communication between components in which "background notifications" are appropriate. That means, here, components provide one or more computational services which can give feedback and handle other services on an event-driven, or asynchronous, or "logical interrupt" basis.

Let's cite an example: Consider a software component that principally provides telecommunications data analysis. As far as this software component is concerned, you can design this to "listen" for incoming data notifications, by adding new satellite data to its model on an asynchronous basis. In this case, the data processing component is nothing but a target of some other software component that declares the availability of data.

In an event-driven program, the source and target orientation is a matter of concern. A component can be a source for one type of event and, at the same time, can be a target for another. While working with Java Beans, some type of source functionality is required for some specific events such as temperature changes, progress-bar state changes, and so on and so forth. In general, event-driven designs are ideal for the following tasks:

- ❑ Handling user interface events:
 - Mouse actions
 - Keyboard events
- ❑ Managing/reporting inter-client connections:
 - JDBC Bean that connects to database server
 - Accepts database requests and notifies a client when the data is available
- ❑ Other events:
 - Property changes in a Bean
 - Any general-purpose notification from one object to another

The event time occurrence is followed by some type of event notification (as defined by Java Beans Component). The process for event notification delivers data related to event to each registered target object within an event object. `java.util.EventObject` is the base class for event objects. Here, new subclasses of `EventObject` specialize `java.util.EventObject` for Bean-specific event types.

Moreover, you have to note that event objects carry pertinent/timely data to registered targets; this data is typically supplied as read-only. Therefore, the target object uses the event object's access methods to retrieve data.

Using the Java Beans Conventions

Java Beans follow a simple yet very powerful naming convention. Here are the two most important Java Beans naming conventions:

- ❑ Properties are created using the `set*` and `get*` methods.
- ❑ Events are distributed using the `add*Listener` event and the `Interface*Listener` event, which is the standard Java event delegation model.

Using the Remote Notification and Distributed Notification

One of the very important characteristics of Java Bean components is that they run in the same virtual machine as the container or application that contains it. Java Beans can access remote machines/object/beans through:

- ❑ Standard socket-based client/server
- ❑ Java RMI — We expect that the use of Java RMI will be a general way for writing Java Beans within its networked systems.
- ❑ JDBC — It allows Java code, hence Java Beans, access SQL databases

Using Beans with JSP

The role of JavaServer Pages technology is also to provide direct supports using Java Beans components with JSP language elements. Here, you can easily build and initialize beans and get and set the values of their properties.

Summary

In this chapter, you have learned about beans, using beans, adding control to beans, giving a bean properties, simple properties, and design pattern for properties. Then, you have learned designing pattern and methods for events. We also explained how to create bound properties, giving a bean methods, and give a bean an icon. Then, we explored the `BeanInfo` class, set bound & constrained properties, and implement persistence. Toward the end, we taught the basics of an event, Java Beans convention, use of remote notification and distributed notification, and use of beans with JSP.



12

Networking and Security with Java

<i>If you need information on:</i>	<i>See page:</i>
Basics of Networking	362
Sockets in Java	364
Client-Server Networking	365
Proxy Servers	365
Internet Addressing	365
Domain Name Service	366
Inet4Addresses and Inet6Addresses	366
The URL Class	367
The URI Class	367
URI Syntax and Components	368
TCP/IP and Datagram	368
Blackboard Assignment Retrieval Transaction	368
Understanding Networking Interfaces and Classes in the java.net Package	369
Understanding the InetAddresses	371
Caching InetAddress	372
Creating and Using Sockets	375
Creating TCP Clients and Servers	378
Understanding the Whois Example	381
Submitting an HTML Form from a Java Program	384
Handling URL	385
Using the URLConnection Objects	387
Working with Datagrams	392
Datagrams Server and Client	394
Working with BART	395
Learning about the java.security Package	397
Summary	402

Networking is a particularly important topic in Java. In this chapter, we'll take a look at the `URL` class, which is how about Java handles URLs, the `URLConnection` class, which allows you to get information about Web pages (including their content), and the `Socket` class, which you use to create sockets for communication across the Internet and intranets. The `Socket` class is particularly powerful—using it, you can write client/server Internet programs in which the client is on one machine and the server is on another.

In this chapter, you'll also come across with Security concepts in Java. These concepts basically tell you about the various classes and policies required for providing security in Java.

As we all know that Java, is much more appropriate as the networking language than any other language like C++ or FORTRAN for networking, which makes it be called “programming for the Internet” by its creators. In light of the preceding fact, the support provided by the `java.net` package for networking will be the focus of this chapter.

Basics of Networking

The systems connected over a network through the Internet connect to each other with the help of Transmission Control Protocol (TCP) or User Datagram Protocol (UDP). In Java, programming is done on the application layer that uses the concept of classes in the `java.net` package; these classes provide a system-independent network.

Consider an example of communication that occurs between two ends, such as a telephone conversation. TCP ensures that the data sent from one end to other goes in the same order as it was sent. Otherwise, error occurs.

The examples of applications that need a consistent communication network are Hypertext Transfer Protocol (HTTP), File Transfer Protocol (FTP), and Telnet. The success of these applications depends on the sequence in which the data is sent and received over the network. To read the data from the Uniform Resource Locator (URL), the data must be received in the same order it was sent. Otherwise, a corrupt zip file or an HTML file is retrieved in the end.

The UDP is connectionless unlike TCP. It sends datagrams (independent packets of data) from one application to another. Here, there is no significance of the order of delivery and cannot be guaranteed because each message is not related to the other.

Basically, a computer has a single physical connection with a network, and all the other devices are associated with it through this network. The data transmitted over the network is identified by its address that consists of IP address and port number.

Ports are specified by 16-bit numbers and ranged from 0 to 65,535. However, the port numbers that are ranging from 0 to 1023 are reserved to be used by the protocols, such as HTTP and FTP, and other services of system. These ports are known as well-known ports.

In case of datagram-based communication, such as UDP, the port number of destination is contained by the datagram, packet and UDP transfers the packet to the destination accordingly. On the other hand, in case of connection-based communication, such as TCP, a socket with a particular port number encapsulates a server application. This allows the registration of the server with the system to receive all data efficiently.

Java programs use the TCP or UDP protocols for communicating over the Internet by using the classes in `java.net`. The `URL`, `URLConnection`, `Socket`, and `ServerSocket` classes, all use TCP for communicating over the network. The `DatagramPacket`, `DatagramSocket`, and `MulticastSocket` classes use UDP for communicating over the network.

Networking includes two types of programs—client programs and server programs. A server program is a program that provides services to one or more users who run client programs to access those services. These client and server computers interact with each other through well-established protocols, which define the nature of the communications between clients and servers.

Networking Enhancements in Java SE 8

With the evolution of Java SE 8, some additions have been made in the networking feature of Java. The various networking enhancements in Java SE 8 are as follows:

- ❑ Addition of a new class `java.net.URLPermission`. The is used for providing permission for accessing a resource (or collection of resources) defined by a given URL, and for a given set of request methods and request headers which can be set by a user.
- ❑ Addition of the `jdk.net` package which contains socket options that are platform specific and a way to set these options on all of the standard socket types. The definition of socket options is provided in the `jdk.net.ExtendedSocketOptions` package. The first socket option is defined with the name `SO_FLOW_SLA` for creating socket flows for multi-tenant applications. The way of setting and getting these extended options is provided by the class `jdk.net.Sockets`.

NOTE

In a multi-tenancy architecture, a software application's single instance is used for serving multiple customers.

- ❑ Changes in the class `URLConnection`. When a security manager is installed in this class, a method is called, which will try to open a connection. For this, the caller must provide one of the following conditions:
 - a "connect" `SocketPermission` to the combination of host/port of the destination URL
 - a `URLPermission` that allows this request

In case automatic redirection is enabled, this request gets redirected to the other destination. But the caller must have the permission for connecting to the host/URL that is redirected.

JDK 8 Security Enhancements

The various features and enhancements in JDK 8 release are as follows:

- ❑ **Enabled TLS 1.1 and TLS 1.2 protocols**—The TLS 1.1 and TLS 1.2 protocols are enabled by the SunJSSE provider by default on the client.
- ❑ **Limited doPrivileged**—A version of the method `AccessController.doPrivileged` is added, which allows code to emphasize on a subset of its permissions or privilege, without restricting full searching of the stack for checking other permissions.
- ❑ **Addition of stronger algorithms for encryption of password**—Algorithms such as `PBEWithSHA256AndAES_128` and `PBEWithSHA512AndAES_256` are added to the SunJCE provider for better encryption of password.
- ❑ **SSL/TLS Server Name Indication (SNI) extension support in JSSE server**—The feature of SNI extension extends the SSL/TLS protocols. It is done to specify with what server name the client is trying to connect while handshaking. By default, the SNI extension for client applications has been enabled by SunJSSE in JDK 7. However, JDK 8 supports the SNI extension only for the server applications.
- ❑ **Support AEAD algorithms**—The enhancement is done in SunJCE provider for supporting AES/GCM/NoPadding cipher implementation along with GCM algorithm parameters. The SunJSSE provider is also enhanced for supporting AEAD mode-based cipher suites.
- ❑ **Enhancements in KeyStore**—For accepting a password and storing it securely as a secret key, the `importpassword` command option is added to the `keytool` utility. Moreover, a new class with the name `java.security.DomainLoadStoreParameter` is added for supporting DKS keystore type. More number of new classes such as `java.security.PKCS12Attribute` are added and other security APIs gets updated for supporting enhancements in KeyStore.
- ❑ **SHA-224 Message digests**—The enhancement of cryptographic algorithms is done in JDK 8 with the SHA-224 variant of the SHA-2 family of message-digest implementations.
- ❑ **Enhanced support for NSA suite B cryptography**—The enhancement done in NSA suite B cryptography includes the following:
 - OID registration for NSA Suite B cryptography algorithms
 - Support for generation of 2048-bit DSA key pair and additional signature algorithms for 2048-bit DSA keys such as `SHA224withDSA` and `SHA256withDSA`
 - Keysize limitation pncrase from 1024 to 2048 for Diffie-Hellman (DH)

- ❑ **Better support for high entropy random number generation**—The `SecureRandom` class allows cryptographically strong random numbers to be generated for private or public keys, ciphers, signed messages, etc. The `getInstanceStrong()` method is provided in JDK 8 for returning an instance of the strongest `SecureRandom` implementation. Other changes done in the `SecureRandom` class are as follows:
 - For UNIX-like platforms, two new implementations `NativePRNGBlocking` and `NativePRNGNonBlocking` were introduced, which provide blocking and non-blocking behavior, respectively
 - `SHA1PRNG` and `NativePRNG` are fixed to follow the `SecureRandom` seed source properties in the `java.security` file
- ❑ **Addition of a new `PKIXRevocationChecker` class**—The `PKIXRevocationChecker` class uses the PKIX algorithm for verifying the cancellation status of certificates. Moreover, the `PKIXRevocationChecker` class supports end-entity certificate checking, best-effort checking, and mechanism-specific options and parameters.
- ❑ **64-bit PKCS11 for Windows**—In JDK 8, the PKCS 11 provider support has been enhanced for Windows from 32 bit to include 64 bit.
- ❑ **New rcache types in Kerberos 5 replay caching**—Kerberos 5 includes two new rcache types such as `none` and `df1`. The type `none` signifies no rcache at all, whereas the type `df1` signifies the DFL style file-based rcache. Moreover, the acceptor-requested subkey is also supported. The rcache and subkey are configured by using the `sun.security.krb5.rcache` and `sun.security.krb5.acceptor.subkey` system properties.
- ❑ **Protocol transition and constrained delegation for Kerberos 5**—JDK 8 supports Kerberos 5 protocol transition and constrained delegation.
- ❑ **Weak encryption disabled by default**—By default, the DES-related Kerberos 5 encryption types are not supported, but can be enabled by adding the `allow_weak_crypto=true` in the `krb5.conf` file.
- ❑ **Unbound SASL for the GSS-API/Kerberos 5 mechanism**—The `Krb5LoginModule` principal value in a JAAS configuration file is set to asterisk (*) on the acceptor side for denoting an unbound acceptor. This means the server can be accessed by the initiator using any service principal name if the acceptor includes the long-term secret keys to that service. Using the `GSSContext.getTargetName()` method, the name can be accessed by the acceptor after establishing the context.
- ❑ **SASL service for multiple host names**—While building a SASL server, you can set the server name to null for denoting an unbound server; therefore, a service can be requested by a client with the help of any server name. After establishing a context, the server can access the name as a negotiated property by using the key name `SASL.BOUND_SERVER_NAME`.
- ❑ **JNI bridge to native JGSS on Mac OS X**—JNI bridge to native JGSS is now supported on Mac OS X. You can enable it by setting the system property `sun.security.jgss.native` to `true`.
- ❑ **Support stronger ephemeral DH keys in the SunJSSE provider**—During the handshaking of SSL/TLS in the SunJSSE provider, make ephemeral DH key match with the length of the certificate key. A new system property, `jdk.tls.ephemeralDHKeySize`, is defined for customizing the ephemeral DH key sizes. The minimum acceptable DH key size is 1024 bits, except for exportable cipher suites or legacy mode.
- ❑ **Support cipher suites preference customization in JSSE**—The SunJSSE provider allows the preference of client's cipher suite by default. However, you can change the behavior of the server's cipher suite preference by invoking `SSLParameters.setUseCipherSuitesOrder(true)` in the server.

Sockets in Java

An *IP Port* on a specific host machine is called a *Socket*. The emergence of sockets was first apparent in early Unix systems in the 1970s and have now become the ‘standard’ low-level communication primitive.

There exists a close resemblance of a network socket with an electrical socket. The plugs have their own standard way of delivering the payload. It is very easy to “plug in” the socket and communicate by anything that understands the standard protocol.

Let's now understand the two types of sockets—*connection-oriented* sockets, almost always based on TCP, and *connectionless* sockets, usually based on *User Datagram Protocol* (UDP). There are quite a few distinguishing differences between the two of them as TCP-type sockets guarantee data arrival and in the correct order, whereas UDP-type ones do not. The major point to note here is that despite the preceding reason, many sophisticated users sometimes use UDP.

TIP

It may appear obvious that 'server sockets execute on servers and client sockets execute on clients.' However, it may not be true all the time- servers can be socket servers, and socket clients, and clients can judiciously run server sockets.

Client-Server Networking

The relationship between two computer programs in which one program, the client, makes a service request from another program, the server, which fulfills the request is self-explanatory to discriminate the server from the client. A network architecture comprises clients and servers among which communication occurs. A server is a powerful computer that manages disk drives (*file servers*), printers (*print servers*), or network traffic (*network servers*), whereas clients are those computers on which users run their applications. Clients depend on servers for various resources, such as files, devices, and even processing power.

For the programs running on a network, client server model is a better idea to be implemented compared with a single computer. The client server model is preferred for distributed systems where the systems are at several locations.

An example of cash withdrawal from an ATM machine uses the concept of client server model. The customer requests to the ATM machine for cash from his/her account the ATM machine forwards the request from the user to the bank and after verification (through its database) from the bank, the bank sends approval to the ATM. Further, the ATM responds back by providing the cash according to the instructions given in the field of amount. It automatically generates the receipt if the user asks for it and deducts the amount from the customer's account.

Proxy Servers

In simple words, a proxy server can be viewed as a bridge between the client application, such as a Web browser, and a real server. It gets and gathers all requests from the real server to view if it can satisfy the requests itself. If not, it forward the request to the real server. The following two main goals are served by proxy servers:

- ❑ **Improve performance**—To improve the performance of all users, a proxy server is used. A proxy server saves the response for all the requests for a limit of time. Consider an example where both user A and user B access the World Wide Web through a proxy server. User A first requests for a webpage like a website of Java called page 1. After some time, user B also requests for the same page; instead of iterating the request to the server, it invokes page 1 from user A, which was already fetched by user A. It is a time-saving process and proxy servers support hundreds and thousands of users. The major online services that establish an array of proxy servers are Yahoo, America Online, MSN, and many others.
- ❑ **Filter requests**—Proxy servers can also be used to filter requests, which means to allow only those requests that are appropriate for the organization. For example, a college might use a proxy server to prevent its students from accessing a specific set of websites.

So we can conclude that a proxy server is a connection between a LAN and an external network such as the Internet. These days, proxy servers are facing competition from NAT and firewalls. In order to remain in demand, the role of proxy servers has been widened to content caching and authentication besides guard-dog security and connection sharing.

Internet Addressing

A computer accessing the Internet has a specific address which is a number that uniquely finds each computer on the Internet. On the Internet, there is only one type of electronic address. Once we know someone's Internet address, we can send mail, transfer files, have a conversation, and even find out information about that person.

The word address on the Internet is always referred to as an electronic address which consists of the form—a string followed by a @ character (the “at” sign), which is further followed by the unique name of a computer.

The part of the address mentioned after @ character is known as domain. Each part of a domain is called a sub-domain and they are separated by periods. For example, xyz@pccrrn.co.uk.

In the example, there are three sub-domains—pccrrn, co, and uk. You can understand a domain name by looking at the sub-domains, considering them from right to left. The name is created so that each sub-domain notifies the user about the computer. Top-level domain is the rightmost sub-domain. As it is read to the left, the sub-domains become more specific. The top-level domain uk, means the computer is in the United Kingdom. The next sub-domain, co, refers to a company. Next, pccrrn tells the user that is the name of a specific computer.

NOTE

Generally, only lowercase letters are used for Internet addresses. There is no need to mix in uppercase letters. If an address has some of the letters in uppercase, it is always safe to change them to lowercase.

Domain Name Service

The Internet wouldn’t be user friendly if users had to refer their intended addresses as numbers. So, a parallel hierarchy of names to go with all these numbers exists. Such a hierarchy is called Domain Name Service (DNS).

Let’s understand the concept this way. Host names are created using a naming standard called DNS. If DNS wouldn’t have existed, we would be ordering books from 176.32.98.166 instead of `www.amazon.com`; we might sell our CDs at 23.207.146.7 instead of `www.ebay.com` and; we’d search the Web at 173.194.36.84 instead of `www.google.com`.

The DNS server on the Internet resolve hostnames for their corresponding IP addresses. DNS is related to URL, which stands for universal resource locator. URL is a naming scheme that can uniquely identify any resource (file) on the internet.

Inet4Addresses and Inet6Addresses

As we had discussed in the Internet Addressing section that all Internet addresses consist of 32 bit values, organized as four 8-bit values. IP addresses are represented by the dotted-decimal notation. In the dotted-decimal notation, each group of eight bits is called an octet and is specified by its decimal equivalent, such as 192.167.134.12. Besides the IP addresses, a port must be used for accessing data from a computer over the Internet. For example, the port number for the Web protocol HTTP is 80. Port numbers are specified with a colon. For example, the HTTP service on a server whose IP address is 192.167.134.20 is 192.167.143.20:80.

There are three classes provided for IP addressing—`InetAddress`, `Inet4Address`, and `Inet6Address`. `InetAddress` is either a 32- or 128-bit unsigned number used by IP. `Inet4Address` is provided for representing 32-bit IPv4address and 128-bit IPv6addresses. An IPv4address is represented by the form `nnn.nnn.nnn.nnn`, where `n` is an integer. `Inet4Address` and `Inet6Address` are the subclasses of `InetAddress`, as shown in Figure 12.1:

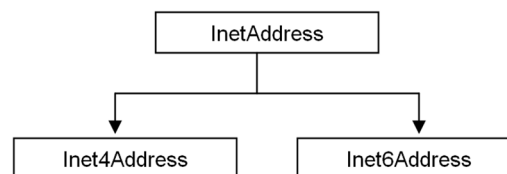


Figure 12.1: The `InetAddress`

There are two classes for socket addressing, namely, `SocketAddress` and `InetSocketAddress`, as shown in Figure 12.2:

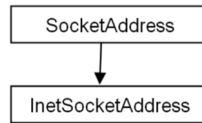


Figure 12.2: The `SocketAddress` and its class

`SocketAddress` refers to an abstract socket address not dependent on a particular protocol. It is used for subclassing for a particular protocol. The `InetSocketAddress` class is a subclass of the `SocketAddress` class; it specifies an IP socket address. It can contain an IP address and a port (e.g., 80), or a hostname (e.g., `oracle.com`) and a port (e.g., 2000), or a port only (e.g., 1110). In the latter case, a wildcard IP address is assumed.

The **URL** Class

No browser can identify any information from the Web without using URL. URLs are nothing but Uniform Resource Locator and identify a resource anywhere on the Internet. The advent of URLs with World Wide Web.

Tim Berners-Lee defined a flexible way to locate all the resources of the network. WWW is a light collection of high-level protocols and file formats; URL transforms this loose collection into heavy collection when the information is accurately defined.

The URL is based on four components—protocol (separated from the rest by a colon); host name or IP address of the host to use; port number; and actual file path (most of the HTTP servers will append a file named `index.html`). Some protocols used are `http`, `ftp`, `gopher`, and `file`. Nowadays, almost everything is being done by HTTP.

For example, `http://docs.oracle.com/javase/8/docs/api/java/net/URL.html` specifies that the protocol used is `http` (HyperText Transfer Protocol) and that the information is available on a host machine named `docs.oracle.com`. The name of the information on that host machine is `/javase/8/docs/api/java/net/URL.html` and its accurate meaning is dependent on both protocol and host.

A URL can optionally indicate a “port,” which is the port number to the TCP connection made on the remote host machine. If the port is not specified, the default port is used. Example: the default port for HTTP is 80.

A relative URL can be specified with sufficient information required to reach the resource relative to another URL. Relative URLs are frequently used within HTML pages. For example, if the content of the URL `http://docs.oracle.com/index.html` contains within it the relative URL, `URL.html`, it would be shorthand for `http://docs.oracle.com/URL.html`.

The **URI** Class

The term URI stands for Uniform Resource Identifier.

This class provides constructors for the URI instances to be created with the help of their components or by parsing (analyzing) their string forms to access the various components of an instance and methods for normalizing and judging URI instances. Instances of this class are immutable, which means that once the object is created, it cannot be modified.

Every URL is a URI, but not every URI is a URL. This is because there is another subdivision of URIs, Uniform Resource Names (URNs), which name the resources but do not specify their location. The news and ISBN URIs are examples of URNs.

A URL class’s instance specifies the syntactic components of a URL along with some of the information needed for accessing the resource that is described by it.

A URL must always define a scheme, that is, it must be absolute. A URL string follows this scheme for parsing. A stream handler is necessary for URL scheme, and without this, it is impossible to create a URL instance. A

URL manages the syntactic operation of resolution as well as the I/O functions of looking up for the host and setting up the connection to the resource.

URI Syntax and Components

At the highest level, a URI reference (called URI) in string has a syntax [scheme:] scheme-specific-part [#fragment], where square brackets [...] denote optional components and the symbols : and # denote for themselves. An opaque URI is referred to as an absolute URI and its scheme-specific portion does not initiate with a slash symbol ('/'). Opaque URIs are not further parsed. The examples for opaque URIs are:

```
mailto:java-net@oracle.com
mailto:java-net@java.sunrise.com
news:comp.lang.java
urn: isbn:087149110x
```

A hierarchical URI is either an absolute URI or a relative URI. In absolute URI, the scheme-specific part of hierarchical URI starts with a slash character, whereas in case of relative URI, the scheme is not specified. An instance of hierarchical URIs is as follows:

```
http://www.oracle.com/technetwork/java/javase/8-compatibility-guide-2156366.html
```

A URI component has nine components, as shown in Table 12.1:

Table 12.1: The component and its type of URI Instance	
Component	Type
Scheme	String
Scheme-specific-part	String
Authority	String
User-info	String
Host	String
Port	int
Path	String
Query	String
Fragment	String

At any given instance, a particular component remains undefined or defined by using a unique value. The former are represented by null, while later are represented by -1. A String component is defined with the empty string as its value.

The type of URI being represented decides whether any particular component would be defined or not. A scheme component is present in an absolute URI. An opaque URI comprises a scheme-specific part and a fragment, but contains no other components. A hierarchical URI always comprises a path that can be empty and a scheme-specific-part that might contain any of the other components.

TCP/IP and Datagram

TCP/IP provides serialized, expected, reliable, and flexible stream of packet data. But it comes with a cost. TCP includes many complications. One of the complications is that once the datagram gets released to its target, there is no guarantee that the datagram will arrive or someone is there for catching it. Similarly, when the datagram is received, it could be damaged as well.

Blackboard Assignment Retrieval Transaction

Bart, short for Blackboard Assignment Retrieval Transaction, is named such because it enables blackboard assignments to be retrieved randomly.